CADERNO EDITORIAL · 28 SET 2026

JavaScript + TypeScript

Do zero ao avançado · Roteiro completo do site · 3ª edição AMOLED

261 atividades ativas em 24 etapas e 5 itens de diagnóstico; 90 observações de auditoria rastreadas. Cada atividade explicita o que o aluno vê e o contrato da plataforma. Esta é a fonte para implementar o site, atividade por atividade.

Revisão editorial SHA-256: 268049b8b3a022c4df17b120c8db89bcdc7514c7850f9b1dc4ab62bf76ead3aa

Estado: contratos editoriais completos. Publicação no site exige arquivos, testes executáveis, verificação no runtime e segunda revisão.

Como usar este caderno

Comece pelo protocolo de implementação e pelo grafo. Para publicar uma atividade, localize seu ID, crie o material inicial, transforme os casos descritos em testes executáveis, execute a solução de referência e revise a transferência. A trilha de aprendizagem usa as pontes conceituais antes das atividades indicadas.

A visão do aluno contém todo comportamento que pode ser cobrado. A visão da plataforma guarda variações reservadas, diagnóstico e ajuda. O teste reservado nunca acrescenta regra não publicada.

Mapa das etapas

D · Diagnóstico inicial
00 · Ambiente e leitura de código
01 · Valores e tipos
02 · Operadores, Math e Number
03 · Decisões e repetição
04 · Funções e decomposição
05 · Strings, arrays, regex e Unicode
06 · Objetos, referências e JSON
07 · Callbacks, coleções e pipelines
08 · Algoritmos e complexidade
09 · Escopo e closures
10 · this, protótipos e classes
11 · DOM, eventos, formulário e Kanban
12 · HTTP, assincronismo e tempo
13 · Teste, debugger e manutenção do Kanban
14 · ESM, Node, npm e Git
15 · TypeScript: contratos e runtime
16 · Unions, narrowing e validação
17 · Generics e APIs reutilizáveis
18 · Utility types e relações entre tipos
19 · Tipos avançados com propósito
20 · Iteradores, streams e bytes
21 · Performance, workers e recursos
22 · Pacotes, declarations e schemas
23 · Arquitetura, manutenção e capstone

Protocolo de implementação do site

Fonte e estados

Cada ficha deste PDF é um contrato editorial versionado. O estado contract_ready indica que enunciado, exemplos, bordas, solução explicada, erro alvo e ajuda estão definidos para iniciar a implementação. Ele não é o estado de publicação READY. Ao implementar um exercício, crie os arquivos iniciais literais, a suíte pública executável, a suíte reservada, a solução de referência executável quando houver código, o pacote de transferência e o registro de compatibilidade. Rode a suíte sobre a solução e sobre ao menos um mutante que reproduza a concepção errada alvo. Um segundo editor compara resultados públicos, ocultos e enunciado. Só então marque READY. Se o oráculo exigir opinião de design, mantenha critérios observáveis e revisão humana; não crie teste oculto que imponha uma técnica não exigida publicamente.

Fluxo editorial: DRAFT -> CONTRACT_READY -> IMPLEMENTED -> REVIEW -> READY -> RETIRED. Migração de ID exige redirecionamento estável, preservação do histórico e changelog. 15.05 aponta para 16.09 e não aparece como atividade ativa. Toda mudança de comportamento incrementa a versão; tentativas antigas permanecem vinculadas à versão original. Para a Etapa 00, stage00-preparation.mjs é a fonte única; npm run build:curriculum gera o JSON derivado em course-src/.generated/ antes de compilar o catálogo.

Anatomia do registro e publicação

Campos obrigatórios: ID, versão, título, etapa, tipo, competência primária/secundárias, pré-requisitos, introduz, nove eixos de dificuldade, tempo apenas informativo, enunciado, arquivos/assets, exemplos e critérios públicos, casos reservados, restrições, rubrica, documentação, runtime/compatibilidade, erro conceitual, falha observável, pergunta diagnóstica, três níveis de dica, solução explicada, tarefa isomórfica, evidência de domínio, telemetria e próxima atividade. O compilador editorial falha em ID duplicado, campo vazio, link quebrado, conceito inexistente, requisito futuro, ciclo, contagem divergente, caractere corrompido, caso sem resultado ou contradição entre público e oculto. A implementação do site acrescenta teste de execução e visualização por dispositivo.

O pacote do site e o PDF são derivados da mesma revisão da fonte. Hash de conteúdo acompanha publicação. Enunciado público informa comportamento de entrada vazia, duplicata, valores inválidos, mutação, ordenação, erro, cancelamento e formato de retorno quando pertinentes. Testes reservados variam dados, ordem e limites já previstos; jamais inauguram regra nova. Para READ, DOCS, COMPARE e projetos abertos, a rubrica avalia evidência e explicação, não coincidência textual. Para TYPE, typecheck, casos positivos/negativos e runtime são camadas separadas. Para UI, testes automatizados cobrem semântica, teclado, foco, mensagens e texto não confiável; uma amostra é revisada manualmente com tecnologia assistiva, pois automação não prova acessibilidade completa. Na ficha, "Bordas e erros publicados" pertence à visão do aluno mesmo quando o mesmo cenário aparece na visão reservada. O caso reservado é um exemplo de teste para a equipe; na implementação ele deve variar a fixture sem acrescentar comportamento. Esse espelhamento deliberado permite verificar que toda reprovação oculta tem regra pública correspondente.

Diagnóstico, domínio e revisão

Antes da etapa 00, D.01–D.05 levam cerca de 12–18 minutos, capturam resposta e confiança antes de Run e produzem mapa inicial de evidência. Não há reprovação. Estados por competência são não visto, reconhece, resolve guiado, resolve independente, transfere e retido. Uma resposta correta imediatamente após explicação não promove a retido. Uma resposta com dica é evidência de resolução guiada; solução aberta exige nova tarefa isomórfica sem solução; transferência em outro domínio e recuperação posterior formam evidência distinta. O aluno pode escolher revisar, continuar ou pausar e vê o motivo da recomendação.

Regra operacional inicial do motor: pré-requisito não visto/contradito encaminha a exemplo e verificação curta; duas respostas independentes corretas em formatos distintos permitem tentativa de transferência; transferência correta permite seguir; revisão D+7 e D+21 comprova retenção quando acertada sem solução. Essas quantidades são parâmetros editoriais, não verdades de aprendizagem: o painel de métricas monitora falso bloqueio, abandono e retenção, e a equipe revisa os parâmetros. Erro repetido na mesma concepção abre recuperação do pré-requisito mais próximo. Alta confiança com erro abre comparação/contraexemplo antes de outra tentativa; baixa confiança com erro oferece exemplo resolvido. Tempo jamais reduz domínio nas etapas iniciais.

Revisão é por competência e intercala habilidades. Uma revisão de async pode exigir função pura, Map, AbortController e tratamento de erro num problema novo. O contexto deve mudar de verdade: de inventário RPG para contagem de erros em logs, por exemplo. A mesma solução com nomes trocados não comprova transferência. Os checkpoints sem tutorial de funções, DOM, async e TS apresentam apenas requisitos, documentação permitida e critérios visíveis; não retiram apoio quando pré-requisitos ainda faltam. Checkpoints concretos: 04.09 (funções), 11.08 (DOM/Kanban), 12.08 (async/dashboard) e 15.08 (migração TS). O motor oferece o modo independente só depois de evidência guiada suficiente; o aluno ainda pode abrir documentação e ajuda, registrando que a

resolução passou a ser guiada. O formato PROJECT não exige um método específico como reduce salvo quando esse método é o objetivo explícito do item.

Ajuda, feedback e orçamento de luta produtiva

Estado da tentativa: draft -> predicted -> running -> feedback -> hint_requested -> resubmitted -> accepted/needs_revision. predicted vale onde há previsão e guarda resposta/confiança antes do primeiro Run. Timeout, falha do executor e teste inválido são estados técnicos separados de erro do aluno. A plataforma registra o primeiro caso divergente, a regra publicada, o comportamento observado e uma próxima ação concreta. Erro de sintaxe pede leitura da mensagem/local; erro semântico pede contraexemplo/traço; erro de design pede revisão de contrato; ambiente pede inspeção de runtime/configuração. Não se penaliza escolha de for versus reduce quando o método não é o objetivo da atividade.

Escada: tentativa própria; pergunta diagnóstica; pista conceitual; decomposição; exemplo análogo; solução comentada. A próxima pista é sugerida após repetição da mesma falha, ausência de progresso ou pedido do aluno, com janela inicial curta para sintaxe e mais espaço para projeto aberto. Não se exige esperar um número fixo de minutos para pedir ajuda. Após abrir solução, a atividade pode registrar compreensão com ajuda, e o motor oferece transferência em outro domínio. A IA usa o mesmo contrato/versão e a mesma escada; não altera oráculo, não inventa requisito, não acusa plágio por semelhança. Ambiguidade de enunciado abre revisão editorial.

Explicações curtas são solicitadas em referência, closure, this, async, narrowing, módulos e arquitetura. Tracing começa linha a linha e depois passa a entrada -> transformação -> estado -> saída. Pergunte o que uma variável representa, de quais valores depende e qual entrada teria chance de refutar a hipótese do aluno. O “Museu de bugs” usa categorias observadas como truthiness, aliasing, mutação, off-by-one, Promise sem await, resposta obsoleta e narrowing indevido. Um toque opcional pergunta o que confundiu: sintaxe, escolha de API, estado ou decomposição. Esses dados mudam ajuda e revisão, não uma nota punitiva.

Telas, execução e dados

Entrada/diagnóstico mostra objetivo, D.01–D.05 e mapa de evidências. Painel mostra próxima atividade justificada, revisão vencida, competências e projetos em andamento. Aula contém problema motivador, exemplos resolvidos progressivos, previsão, execução, explicação, contraste e documentação. Exercício tem enunciado, arquivos, testes públicos, saída/erros, dicas, docs e histórico. Projeto mantém árvore de arquivos, README, testes, changelog e pedidos de mudança. Revisão usa novo domínio e registra evidência. Administrador edita as duas visões, vê diff de versões, roda suites, inspeciona a11y e solicita segunda revisão editorial.

Perfis de execução são console/Node, browser/DOM e TypeScript. Código do aluno roda isolado com limites de CPU, memória, duração, saída e arquivos; sem segredo nem rede livre. Exercícios HTTP usam fixture/servidor determinístico; testes públicos e reservados usam a mesma versão de runtime e dependências. UI roda em origem/iframe isolado. Uma seed reproduzível acompanha tentativa. Métricas por competência: tentativa independente sem dica, classe de erro, tempo até primeiro acerto, confiança, nível de dica, solução aberta, abandono, D+7, D+21 e transferência. Tempo mede fricção, não nota. Código integral e segredo não entram na telemetria por padrão. Progresso pode ser exportado e excluído na implementação.

Aceite por etapa do site

Para cada exercício: (1) arquivo inicial abre no runner correto; (2) exemplo público e critérios visíveis executam ou têm rubrica reproduzível; (3) solução passa casos públicos e reservados; (4) mutante alvo falha pelo motivo esperado; (5) erro técnico não vira erro pedagógico; (6) teclado/foco/texto seguro passam nos itens UI; (7) próxima atividade respeita grafo; (8) registro de versão/hash e compatibilidade existe; (9) revisor aprova enunciado, solução e transferência. Para o produto: diagnóstico não salta fundamento por um acerto; resposta prevista não muda após Run; abrir solução agenda transferência; item não READY não aparece no catálogo; rascunho/projeto persiste; dados antigos migram; nenhuma credencial aparece em log; PDF e pacote têm o mesmo hash editorial.

JavaScript + TypeScript — especificação do site

Versão editorial 3.1, 28/09/2026. Este texto e as 254 fichas individuais formam a fonte do novo PDF. A 2ª edição é referência histórica, não camada editorial sobreposta. O inventário contém 254 IDs ativos: 203 da base, 46 atividades novas e 5 de diagnóstico. O ID 15.05 foi aposentado e redireciona para 16.09. Contrato editorial completo não significa implementação executável já publicada no site.

1. Produto e fonte editorial

- Dificuldade é avaliada por atividade, nos nove eixos, a partir do starter, número de conceitos simultâneos, estado, abstração, arquivos, leitura e ferramentas; o formato PREDICT/MAKE/PROJECT sozinho não determina a nota. Os valores atuais são estimativas editoriais estáticas. Duração também é só uma estimativa inicial para planejar a sessão, nunca proxy de domínio ou critério de aprovação; recalibrar ambas após piloto com tempos, abandono, tentativas e pedidos de ajuda, sem expor velocidade como nota.
- IDs existentes são chaves estáveis para progresso e revisão. A ordem curricular é definida pela sequência publicada e aparece separadamente como “n de total”; não renumerar IDs sem uma migração de progresso versionada. APIs raras ou dependentes de suporte recente são extras opt-in: ficam visíveis no catálogo, mas não bloqueiam conclusão nem navegação pelo caminho obrigatório.
- O site atual oferece diagnóstico, aulas com exemplos resolvidos, exercícios de leitura e escrita, editor de texto, feedback editorial, revisão espaçada e progresso. Ele não executa o código do aluno nem valida automaticamente a implementação. Runner isolado, testes, mutation testing e feedback de execução são requisitos futuros deste produto; a interface não pode sugerir que a reflexão manual equivale a aprovação automática.
- Há duas visões do mesmo item. ALUNO vê enunciado final, arquivos iniciais, exemplos públicos, restrições, critérios visíveis, documentação permitida e seu progresso. PLATAFORMA guarda testes ocultos, solução, regras diagnósticas, grafo de requisitos, telemetria e próxima atividade. Teste oculto varia dados, mas não inventa um comportamento omitido do contrato público.
- A fonte canônica de cada atividade é um registro estruturado versionado; PDF e pacote do site devem ser gerados dele. As 254 fichas deste PDF são a especificação de implementação, não uma afirmação de que runner, arquivos e testes já existem no site. contract_ready significa que o contrato editorial foi escrito; READY só pode ser atribuído depois da execução automatizada suportada e revisão independente. Itens sem esse pacote permanecem explicitamente contract_ready, sem alegação de correção automática.
- Esquema mínimo: ID estável, título, etapa, trilha, formato, competência principal e secundárias, requires, introduces, nove dimensões de dificuldade, tempo estimado, enunciado, arquivos, starter code, assets, exemplos e testes públicos, contrato e fixtures dos ocultos, restrições, rubrica visível, docs permitida, erro conceitual alvo, falha observável, pergunta diagnóstica, pistas conceitual/decomposição/exemplo análogo, solução comentada, transferência, evidência de domínio, runtime, compatibilidade, telemetria, próximas atividades e versão.
- O build falha com ID duplicado, campo vazio, conceito inexistente, requisito futuro, ciclo no grafo, link inválido, teste oculto contraditório, borda sem resposta definida, solução ausente, caracteres corrompidos ou contagem manual divergente. Um projeto aberto usa rubrica observável e revisão humana quando não há oráculo determinístico.

2. Jornada e domínio

- Antes da Etapa 00, diagnóstico de 12–18 minutos: leitura de saída/fluxo, valores/tipos, decisões, funções e localização de erro. Sem nota punitiva. Resposta + confiança 0–100% vêm antes de Run; “não sei” é opção válida. O resultado cria mapa de evidências; um acerto isolado não libera pré-requisito.
- Estados por competência: não visto → reconhece → resolve guiado → resolve independente → transfere → retido. Solução aberta impede avanço para “independente” naquele item e agenda problema isomórfico em outro domínio. Retenção exige recuperação D+7 e D+21, com agenda flexível.
- Para closure, this, event loop, generics, narrowing e conditional types, usar 2–3 exemplos resolvidos progressivos. No formato FADED, retirar uma decisão de cada vez; a retirada depende de evidência de domínio, não de uma posição fixa na etapa.
- Ajuda: tentativa própria → pergunta diagnóstica → pista conceitual → decomposição → exemplo análogo → solução comentada. Após alguns minutos sem progresso ou mesmo erro repetido, oferecer a próxima ajuda. Tempo é sinal de fricção, não nota. Depois da solução, agendar transferência.
- Revisão é por competência, não por etapa concluída. Erro de mutação em projeto async reabre mutação. Revisões misturam habilidades antigas. Projetos terminam com três perguntas: o que falhou, que decisão mudaria, que pré-requisito reapareceu.

- Tracing inicial usa linha/estado; o modo abstrato pede entrada → transformação → estado → saída. Perguntar “o que esta variável representa aqui?”, “de que depende?” e “o que muda se removida?”. Depois da hipótese, pedir uma entrada que possa refutá-la.

3. Formatos e avaliação

- Tipos formais: PREDICT, TRACE, FADED, PARSONS, COMPARE, READ, DOCS, INVESTIGATE, MODIFY, MAKE, DEBUG, TEST, UI, TYPE, PROJECT, TRANSFER. Progressão: prever → executar → investigar → modificar → criar → depurar → transferir. A ordem pode mudar conforme evidência.
- PREDICT salva resposta/confiança antes da execução. READ pergunta finalidade do módulo, origem de regra, fluxo entre arquivos e local para mudança, sem edição. COMPARE contrasta duas soluções válidas: ||/??, sort/toSorted, spread/structuredClone, all/allSettled, satisfies/anotação, Map/objeto e assertion/validação.
- MODIFY entrega código funcional, teste de regressão e change request. Aprovação exige mudança localizada, comportamento antigo preservado e caso novo. DEBUG exige reprodução, hipótese, ferramenta escolhida, correção e regressão. DOCS começa com link e seção; depois só API; por fim busca autônoma. O aluno identifica assinatura, parâmetros, retorno, erros e compatibilidade.
- MAKE aceita estratégias comportamentalmente equivalentes; pedir resultado, não método específico, salvo quando o método é a competência. Erros são classificados em sintaxe, modelo mental, comportamento, design/manutenção e ambiente. Cada classe determina feedback e recuperação.
- UI testa semântica nativa, label, botão, teclado, foco, loading/erro, aria-live quando necessário, cleanup e texto não confiável. Não pontuar quantidade de ARIA. TYPE combina typecheck, type tests e runtime quando aplicável; tipos não substituem validação de dados externos. PROJECT usa rubrica publicada, testes de aceitação e revisão de segurança/operação.
- “Museu de bugs”: truthiness, aliasing, mutação, this, closure, off-by-one, Promise esquecida, resposta obsoleta e narrowing indevido. Após erro importante, um toque registra o que confundiu: sintaxe, escolha de API, estado ou implementação. Isso muda a ajuda, não a nota.

4. Motor adaptativo, IA e métricas

- Evento de tentativa: usuário pseudônimo, ID/versão, competências, data, resultado, classe de erro, confiança, nível de pista, solução aberta, Run, tempo até primeiro acerto, transferência e runtime. Não gravar código integral ou segredo por padrão. Métricas por item: acerto sem dica, abandono, pistas, retenção D+7/D+21 e transferência.
- Acerto independente sobe evidência; com pistas sobe só “guiado”; solução aberta pede problema isomórfico; falha repetida volta ao pré-requisito próximo. Item difícil e habilidade não dominada são variáveis diferentes. Velocidade não mede domínio no núcleo.
- A sugestão seguinte considera requisitos provados, revisão vencida, erro, carga cognitiva e escolha do aluno. O aluno vê por que foi sugerida e pode revisar, continuar ou pausar.
- IA recebe contrato e solução do item. Antes de tentativa, não entrega resposta. Depois segue os níveis da escada. Nunca altera esperado, restrições ou aprovação; ambiguidade vai à revisão editorial. Código muito próximo à solução gera transferência, sem acusação automática.
- Painel editorial mostra erros por competência e versão, abandono anômalo, pistas ineficazes, falsos negativos, acessibilidade e retenção. Alteração de critério cria nova versão; tentativas antigas não são reinterpretadas silenciosamente.

5. Publicação dos exercícios

- Nenhuma das 254 fichas editoriais deve ser publicada automaticamente. As fichas definem dados, resultados, bordas, ajuda e solução explicada; na implementação de cada atividade, autor e revisor materializam arquivos iniciais e testes executáveis, executam-nos no runtime declarado, conferem acessibilidade, requisitos e transferência, então marcam READY.
- O manifesto gerado informa total, IDs por etapa, status e hash da fonte. A contagem oficial desta edição é 254. A capa da 2ª edição é registro histórico, não contagem vigente.
- Contratos abertos não usam testes ocultos para impor uma escolha de design. Bordas como lista vazia, duplicatas, string inválida e cancelamento devem informar publicamente o resultado esperado antes da tentativa.

6. Sequência curricular: fundamentos

- 00: execução, console, leitura sem variáveis obrigatórias, erro de sintaxe e escolha de ferramenta. Reescrever 00.01 com literais e três logs; variável que muda passa para 01. Precedência de 00.07 só é cobrada depois de ponte na 02.

- 01–02: bindings, tipos, coerção, operadores e precedência. Math.floor/ceil/round/trunc/min/max/abs/random, divisão e resto precisam anteceder tarefa que os usa. Math.random não serve para segredo. Microbloco Number: IEEE-754, 0.1+0.2, NaN, Infinity, inteiro seguro, BigInt e arredondamento. Tolerância de float depende da escala e precisão dos dados; Number.EPSILON isolado não é receita universal.
- 03A: decisões, boolean, if/else, switch e guard clauses. 03B: for/while, contadores, acumuladores, break/continue; for...of só depois de iterável concreto. Duas unidades obrigatórias na mesma Etapa 03.
- 04: funções, parâmetros, retorno, decomposição, nomes e explicação curta. Primeiro checkpoint sem tutorial: requisitos e documentação permitida, sem passos.
- 05A: strings/arrays, indexação, slice/splice, mutação e cópia, toSorted/toReversed/toSpliced/with, flat. 05B: parsing/regex, classes, grupos, flags, RegExp.escape e retorno em formulário. Unicode: emoji.length, code units/code points, for...of, codePointAt, caracteres combinantes e normalize. API nova segue conteúdo → prática → ponte → microprojeto; 05.10 vem antes do projeto.
- 06: propriedade própria/herdada, enumerabilidade, Object.keys/values/entries, Object.hasOwn, in, descriptors introdutórios, aliasing, cópia rasa, structuredClone e JSON. Só depois protótipos. Stack/heap são analogias, não promessa da especificação.
- 07: callbacks, métodos de array, Map/Set e flatMap antes do projeto. Comparar loop, map, reduce, Map e objeto pela intenção. 08 é paralela e não bloqueia DOM/async.
- 09–10: this, closures e classes com exemplos resolvidos, leitura, MODIFY e transferência. Exercícios “quando NÃO usar” classe, Proxy, regex ou biblioteca avaliam seleção de ferramenta.

7. Sequência curricular: navegador, HTTP e async

- 11: parser HTML, script clássico/defer/async/type=module e DOMContentLoaded. Ponte ESM de navegador com state.js, view.js e main.js antes do Kanban. Eventos capture → target → bubble; target/currentTarget; preventDefault versus stopPropagation; delegação; once/passive/signal e cleanup. Texto não confiável usa textContent/createTextNode; HTML só se necessário e sanitizado. Trusted Types é extensão condicionada a suporte. Kanban V1 cobre estado, DOM, teclado e persistência; V2 volta após testes. DevTools Application acompanha storage.
- 12: antes de fetch, HTTP: URL, método, request/response, status, headers, Content-Type, body e CORS como fronteira do navegador. Response.ok, text/json/blob/arrayBuffer/formData, bodyUsed e corpo consumível. URL/URLSearchParams; AbortController, AbortSignal.timeout, cancelamento composto, timeout de UI versus rede cancelada e resposta obsoleta. Date, timestamp, expiração, ISO, UTC/local e Intl.DateTimeFormat entram com TTL. Promise.all/allSettled/any/race, AggregateError e Promise.try antes do projeto. Depois async generator, Symbol.asyncIterator, for await...of; só então Array.fromAsync. Comparar materialização e streaming.
- 13: Sources/DevTools e testes com Vitest; vi.fn, vi.spyOn, fake/stub/spy/mock, fake timers, rede simulada, rejeições e estados intermediários. Error.cause e captura/contextualização/relançamento. Testar latência, timeout e resposta fora de ordem. Kanban V2 exige testes escritos pelo aluno e migração storage V1→V2.
- 14: microtrilha Git paralela: status, diff, add/commit, branch, merge/conflito, .gitignore e PR. ESM/CJS, package scripts, semver, lockfile; import(), import.meta, top-level await, ciclos e efeitos de import. Pacote antes de instalar: necessidade, manutenção, dependências, licença, risco e auditoria. Bundler: conceito e experiência mínima. Frameworks front/back, banco e deploy são trilhas opcionais, fora do núcleo.

8. Sequência curricular: TypeScript e prática profissional

- 15: unknown e validação mínima da fronteira; never apenas para retorno impossível. A atividade de exaustividade atual 15.05 migra para 16. Readonly não congela runtime; alias mutável pode alterar objeto. Excess property checks variam entre literal fresco e variável. Compilar não garante segurança de dado externo.
- 16: union discriminada, assertNever(x: never): never, type test que adiciona variante, catch(e: unknown) e narrowing por instanceof Error, inclusive throw que não é Error. Type guards, assertion functions e parsers combináveis.
- 17–18: reutilizar parsers em API/formulário; import type e verbatimModuleSyntax; diferença entre import runtime e tipo apagado. Utility types Awaited, NonNullable, Exclude, Extract, ReturnType e Parameters, cada qual ligado a problema. Enum numérico/string versus objeto as const + union, JS emitido e risco de ambient const enum em biblioteca.
- 19: conditional/distributive, infer, mapped e template literal types em API client, event map, schema, registry e DTO; menos type golf. Branding/opaque para IDs é padrão avançado, validado na borda; NoInfer opcional. Orçamento de complexidade: abstração nomeada ou runtime quando o tipo esconde a regra.
- 20–21: Performance/Memory DevTools; workers e transferência de ArrayBuffer com buffer detached; Streams/ReadableStream e backpressure após async iterators. Using/await using condicionado a suporte, com SuppressedError quando corpo e cleanup falham. Event loop simples de navegador primeiro; Node depois, sem regra universal de nextTick antes de Promise.

- 22: comparar parser manual e Zod/Valibot em erro, inferência, bundle, desempenho e fonte da verdade. IsolatedDeclarations é gate opcional no track de bibliotecas, não em todo projeto. Project References opcionais para projetos grandes/monorepo.
- 23: capstone recebe change request em codebase existente. Rubrica: testes, typecheck, a11y, DOM sinks, entradas/URL/arquivos, segredos, dependências, cleanup, cancelamento e migração. Observabilidade exige logs estruturados sem token, causa/contexto, ID de operação quando útil, duração e warning/error; entregar relatório de falha reproduzida.

9. Projetos e experiência de leitura

- A cadeia de código evolutiva já representada no catálogo é o Kanban: 11.08 cria a V1, 11.13 acrescenta a migração do save existente e 13.14 retoma esse baseline para estender a V2 com prioridade e regressões. Só declare herança de código/testes quando o starter ou contrato identificar explicitamente a versão anterior; os demais microprojetos, inclusive os de RPG, são independentes salvo quando essa cadeia for reutilizada. Projetos independentes verificam transferência.
- Contextos ao longo da trilha: RPG/jogos, checkout simulado, importação de arquivo, relatório, agenda, API, configuração, logs e formulários. Os microprojetos de RPG não formam um produto contínuo por si só. Não exigir pagamento real nem serviço externo instável para avaliar fundamento.
- Em funções, DOM, async e TS, checkpoint sem tutorial. Mais tarde, manutenção entrega mini-repositório de 8–15 arquivos, README, testes e issue; localizar módulo, regra e fluxo de dados antes de mudar código. Renomeação sem mudança de comportamento e revisão de nomes são avaliadas.
- Documentação é atividade: primeiro link e seção; depois só API; mais tarde busca autônoma. Abrir docs não reduz nota. Explicações curtas são exigidas para referência, closure, this, async, narrowing, módulos e arquitetura, não em todo exercício.

11. Fontes técnicas e regra de compatibilidade

- MDN Number e Number.EPSILON: formato IEEE-754, inteiros seguros e tolerância dependente de magnitude/precisão. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/EPSILON
- MDN script e módulos: módulo é adiado por padrão; defer e DOMContentLoaded têm semântica própria. https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script
- MDN Fetch: status HTTP, corpos consumíveis e cancelamento. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN Streams: backpressure e ReadableStream. https://developer.mozilla.org/en-US/docs/Web/API/Streams_API/Concepts
- TypeScript Handbook Object Types: readonly e excess property checks. https://www.typescriptlang.org/docs/handbook/2/objects
- TypeScript TSConfig: verbatimModuleSyntax e isolatedDeclarations. https://www.typescriptlang.org/tsconfig/verbatimModuleSyntax.html e https://www.typescriptlang.org/tsconfig/isolatedDeclarations.html
- Vitest: mocks, fake timers e expectTypeOf. https://vitest.dev/guide/mocking e https://vitest.dev/api/expect-typeof
- Node process/microtasks: confirmar o contexto CommonJS/ESM quando comparar nextTick e microtasks. https://nodejs.org/download/release/v24.x/docs/api/process.html
- Toda API recente tem runtime mínimo ou fallback no cartão. Compatibilidade é revisada no momento da publicação, não herdada cegamente da data deste PDF.

12. Telas, estados e execução do site

- ENTRADA/DIAGNÓSTICO: apresentação do curso, escolha de objetivo e teste inicial curto. Após cada previsão, confiança é registrada antes do Run. No fim, o aluno recebe mapa de competências, incertezas e um primeiro percurso sugerido; pode revisar a sugestão. Não mostrar “aprovado/reprovado” no diagnóstico.
- PAINEL: próxima atividade justificada, revisão vencida, projetos em andamento e mapa por competência. Mostrar os seis estados de domínio, com evidências recentes. “Continuar”, “revisar” e “pausar” são ações distintas. O aluno pode voltar à atividade sem perder código.
- AULA: problema motivador, modelo mental, 2–3 exemplos quando o conceito é difícil, previsão antes da execução, contraste com solução conhecida, erro comum e documentação relevante. Ponte conceitual ocorre antes do microprojeto. A aula registra leitura, mas ler não equivale a dominar.
- EXERCÍCIO: cabeçalho com contrato e runtime; abas Enunciado, Arquivos, Testes públicos, Saída/erros, Dicas, Documentação e Histórico. A ordem visível não esconde critérios de aceite. Salvar rascunho automaticamente; antes de Run em PREDICT salvar resposta/confiança imutáveis para aquela tentativa. O botão “Ver solução” pede confirmação informativa e agenda transferência

depois de aberto.

- PROJETO: árvore de arquivos, README, requisitos, testes, changelog das versões e pedido de mudança. A rubrica separa comportamento, testes, acessibilidade, segurança e manutenção. O aluno vê o que é automático e o que depende de revisão humana.
- REVISÃO: fila por competência, data e razão da revisão. Itens isomórficos mudam domínio e dados, não só nomes. Após resposta, mostrar evidência atualizada, não uma nota arbitrária de velocidade.
- ADMINISTRADOR EDITORIAL: editor de cartão com prévia da visão do aluno e da plataforma; validação do grafo, execução de testes públicos/ocultos, inspeção de a11y, comparação de versões, status DRAFT → REVIEW → READY → RETIRED e aprovação por segundo editor. A publicação registra versão, hash e runtime.
- ESTADOS DA TENTATIVA: draft → predicted (quando aplicável) → running → feedback → hint_requested → resubmitted → accepted ou needs_revision. Um timeout é estado técnico distinto de resposta errada. Após solução aberta, o item pode ficar “compreendido com ajuda”, mas o domínio independente depende da transferência.
- EXECUTOR: perfis separados Node/console, navegador DOM e TypeScript. Código do aluno roda em ambiente isolado, com limites de CPU, memória, tempo, saída e arquivos; sem acesso aos segredos do site nem à rede por padrão. Exercícios HTTP usam servidor/fake determinístico declarado. DOM usa origem/iframe isolado. Testes públicos e ocultos partilham versão de runtime e dependências, mudando apenas fixtures e visibilidade. Toda execução guarda versão do cartão e seed reproduzível.
- CAMADA DE DADOS: ExerciseVersion, Skill, PrerequisiteEdge, Attempt, Evidence, ReviewDue, ProjectSnapshot e ContentEvent são entidades versionadas. Attempt guarda resultado e níveis de ajuda; código-fonte fica em armazenamento próprio com retenção clara. Telemetria agregada não contém código integral, dados pessoais em logs nem token. Exportação/exclusão de progresso deve ser prevista na implementação.
- CRITÉRIOS DE ACEITE DO PRODUTO: (1) diagnóstico recomenda percurso sem saltar fundamento após um acerto; (2) Run não altera previsão já registrada; (3) abrir solução reduz evidência e agenda transferência; (4) teste oculto reprova só por comportamento publicado; (5) UI aceita teclado/foco e anuncia erro/loading; (6) voltar ao projeto preserva arquivos e versão; (7) item DRAFT não aparece no catálogo; (8) erro/timeout do executor não é contado como erro do aluno; (9) conteúdo publicado e PDF carregam o mesmo hash editorial.

Pontes conceituais e exemplos resolvidos

Estas aulas curtas entram no site antes das atividades indicadas. Em cada ponte, o aluno primeiro prevê, depois executa, compara o resultado com a explicação e só então resolve a atividade. A terceira variação retira ajuda quando houver evidência de domínio. O conteúdo abaixo é a redação de referência para a aula, não apenas um título no currículo.

Ponte 00 — Linguagem, ambiente e execução

JavaScript é a linguagem. Navegador e Node.js são ambientes que executam JavaScript e oferecem APIs diferentes. console.log('A'); console.log(2 + 3); console.log('B') produz A, 5, B nesta ordem: cada chamada é síncrona e o programa passa à próxima instrução depois que a anterior termina. O console é saída observável, não o próprio código. Um SyntaxError impede analisar o programa; um erro em execução surge depois que o programa começou; uma saída incorreta pode ocorrer sem exceção. No primeiro contato, leia a mensagem, a linha e a menor expressão que a explica antes de tentar uma alteração. A microponte HTML da etapa 00 apresenta elemento, atributo, hierarquia, id, class, input, button, form e label, somente como pré-requisito explícito para a etapa 11.

Este site usa Node.js em modo ES Module para carregar os arquivos dos exercícios. Nas fichas iniciais, export function nome(...) apenas identifica a função que o executor chama; o aluno prevê e altera o corpo da função, sem configurar imports ou package.json. A organização de vários módulos, imports, exports e configuração de projeto é ensinada formalmente na etapa 14.

Ponte 01 — Valores, bindings e reatribuição

Um binding é um nome associado ao seu valor atual. = inicializa um binding ou atribui um novo valor; const impede reatribuir aquele nome e let permite mudanças previstas. Nesta trilha, prefira const para valores que não mudam e let para os que mudam — uma convenção de estilo, não uma regra que torna let inválido. Antes da declaração, o binding lexical ainda não foi inicializado; descreva esse estado temporal com precisão, sem dizer que o nome não existe.

Exemplo resolvido:

js const nome = "Ana"; let pontos = 0; console.log(nome, pontos); // Ana 0 pontos = 5; console.log(nome, pontos); // Ana 5

nome permanece Ana; pontos passa de 0 para 5. Zero e false também podem ser valores escolhidos intencionalmente. Aspas e crases delimitam strings no código; elas não fazem parte do valor textual exibido. typeof valor informa um rótulo de tipo; typeof null retorna "object" por uma peculiaridade histórica. Um let ou const não pode ser lido antes de sua inicialização: essa região é chamada temporal dead zone e a leitura causa ReferenceError.

Template literals usam crases e ${...} para inserir valores: ` ${nome}: ${pontos} resulta em Ana: 5. Nesta etapa, o vocabulário se limita a nomes de identificadores que o código realmente usa. NaN, Infinity e BigInt aparecem apenas para classificar o resultado de typeof`; operações, precisão e comparações numéricas são tratadas na Etapa 02.

Exemplo resolvido A: const codigo = "Q7"; let tentativas = 0; guarda um código fixo e um contador que pode mudar. Exemplo resolvido B: depois de tentativas = 1, o resumo ` ${codigo}: ${tentativas} produz Q7: 1; mudar o contador não muda codigo. Exemplo com ajuda retirada: um registro de manutenção fornece intervaloDias = 0 e alertaAutomatico = false; o aluno prevê o resumo e marca qual binding pode receber nova atribuição. Esta ponte prepara o cartão 01.08 e a ficha de 01.09`. Symbol e objetos ficam explicitamente para etapas posteriores.

Ponte 02 — Expressões, operadores, Number e Math

Ponte 02A — Expressões, operadores e coerção

Unidade A (02.01–02.09): aritmética e precedência → comparação e lógica → ausência → conversão e detecção de validade → transferência. Em 02.01, resolva e confira o bloco aritmético antes do lógico; a comparação curta entre == e === vem ao final.

+, -, *, /, % (resto) e ** (potência) calculam valores. Parênteses mudam a ordem; 2 + 3 * 4 é 14 e (2 + 3) * 4 é 20. === compara sem converter tipos; >, >= comparam grandezas. ! inverte truthiness. && devolve o primeiro operando falsy, ou o último se nenhum for falsy; || devolve o primeiro truthy, ou o último se nenhum for truthy. Por isso true && "ok" devolve "ok" (string), e não um boolean.

Na matriz 02.02, || substitui os seis valores falsy **desta tabela**; ?? substitui apenas null e undefined, preservando 0, false, "" e NaN. String vazia deve ser mostrada como "". Estes seis valores não são a lista completa de falsy de JavaScript.

Coerção é conversão de tipo: "2" + 3 concatena e produz "23"; Number("2") + 3 calcula 5. Compare depois dos blocos: 0 === "0" é false, 0 == "0" é true porque == aplica regras adicionais de coerção. A trilha prefere ===, salvo intenção explícita. Number("") e Number(" ") dão zero; a flag de vazio da fixture registra ausência. Number("abc") produz NaN; Number("Infinity") produz infinito.

Number.isNaN identifica apenas NaN; Number.isFinite rejeita NaN e ambos os infinitos. A validação de 02.04 é vazio || !Number.isFinite(numero), preservando o texto "0" com flag false.

Exemplo resolvido A: (8 - 2) / 3 é 2. Exemplo resolvido B: com saldo 15, valor 6 e conta ativa, saldo >= valor && ativa === true é true; qualquer requisito false torna a autorização false. Exemplo com ajuda retirada: escolha um fallback para tentativas explicitamente iguais a zero e explique por que zero não significa ausência.

Interface das fixtures: todos os arquivos iniciais declaram entradas literais. O runner substitui apenas essas inicializações entre casos, preservando seus nomes e o código do aluno; nenhuma variável vem de um contexto invisível. Flags de vazio são booleanos declarados e correspondem a campos vazios ou só espaços. Em 02.05, apenas detecte validade: ainda não há if, throw nem tratamento de erro. Em 02.07, preço e taxa já pertencem ao domínio publicado; não existe validação escondida no starter.

Ponte 02B — Number, Math e tolerância

Unidade B (02.10–02.12): arredondamento e utilitários → sorteios e intervalos → decimais → inteiros seguros → tolerância. Releia esta ponte antes de 02.10. Em 02.11, confira o bloco de decimais/valores especiais antes do bloco de inteiros/BigInt.

Number usa ponto flutuante binário para números comuns. Alguns decimais, como 0.5 e 0.25, são exatos; 0.1 + 0.2 é 0.30000000000000004 e sua igualdade com 0.3 é false. Inteiros só são representados sem lacunas dentro do intervalo seguro indicado por Number.isSafeInteger; BigInt serve a inteiros maiores, mas não resolve frações decimais nem deve ser misturado implicitamente com Number. NaN representa resultado numérico inválido e não é igual a si mesmo; Infinity pode surgir de operações como divisão por zero. Mostre Math.floor(-1.8)=-2, Math.ceil(-1.8)=-1, Math.trunc(-1.8)=-1 e Math.round(1.5)=2: arredondar, truncar e escolher teto/piso são decisões de domínio distintas. Math.min(3,5)=3, Math.max(3,5)=5 e Math.abs(-3)=3. Math.random() produz um valor em [0,1); Math.floor(sorteio*6)+1 transforma esse valor em uma face de 1 a 6. Teste os extremos com sorteios literais 0 e 0.9999999999999999, sem exigir uma saída aleatória fixa. Math.random não é fonte criptográfica. Como curiosidade opcional, Math.round(-0.5) é -0.

Exemplo resolvido A: a regra Math.abs(a-b) <= tolerancia compara dois valores finitos quando a tolerância vem da precisão exigida pelo problema. Com a=0.1+0.2, b=0.3 e tolerância=1e-15, o resultado é verdadeiro. Exemplo resolvido B: compare valores perto de 2000 e observe que uma tolerância absoluta escolhida para números perto de 1 pode ser inadequada. A fórmula não deve usar Number.EPSILON como constante universal. Exemplo com ajuda retirada: o aluno recebe a resolução de um sensor e uma política explícita que aceita até metade da resolução, inclusive; precisa declarar seu limiar antes de implementar. Resolução sozinha não determina uma política universal. A ponte antecede 02.11 e 02.12.

Ponte 03 — Decidir antes de repetir

Na unidade 03A, if/else, operadores booleanos, switch e guard clauses escolhem um caminho. Uma condição não percorre uma coleção. Exemplo A: let faixa = 'menor'; if (idade >= 18) faixa = 'adulto'; produz uma única decisão; na etapa 04 a mesma ideia vira guard clause com return dentro de uma função. Exemplo B: switch (estado) escolhe uma entre variantes conhecidas; uma alternativa desconhecida recebe tratamento explícito. A unidade 03B começa só depois dessa prova: for e while repetem ações; contador mede quantas vezes, acumulador mantém um resultado e break encerra o loop. Exemplo A: somar 1,2,3 usa acumulador inicial zero e três atualizações. Exemplo B: encontrar primeiro valor negativo encerra no primeiro match; não precisa percorrer todos. No terceiro exemplo o aluno explica o que a variável representa, do que depende e qual entrada refutaria sua hipótese, em vez de decorar um rótulo de “papel”.

Ponte 04 — Funções pequenas que compõem uma regra

Uma função nomeia uma transformação com entradas e saída. Separe regras independentes em funções pequenas; a função de nível mais alto coordena seus resultados sem duplicar fórmulas.

Exemplo resolvido A: custoPeso(kg) recebe quilogramas e devolve um custo; custoDistancia(km) recebe quilômetros e devolve outro. Exemplo resolvido B: frete(pedido) chama as duas funções e soma os valores, mantendo as unidades claras. Exemplo com ajuda retirada: para um resumo de expediente, o aluno identifica entradas e saídas de minutosTrabalhados e intervalo, depois decide o que o resumo deve chamar. A ponte antecede 04.08, que transfere composição para frete, e 04.09, que combina regras derivadas em uma ficha.

Ponte 05 — Texto, array, Unicode e regex

Strings e arrays entram com indexação, slice, split, join e operações de cópia/mutação. Aqui, callback significa função passada a outra operação para ser chamada por ela; o comparador passado a sort é o primeiro exemplo. sort altera o array; toSorted devolve outro array. Uma cópia rasa separa a coleção externa, mas não clona objetos internos. map, filter e pipelines de callbacks entram depois na etapa 07. Regex só é escolhida quando a regra textual permanece legível; formato estruturado complexo pede parser explícito. RegExp.escape só aparece sob contrato de runtime compatível.

Unicode exige três contagens diferentes: unidades UTF-16 ('□'.length é 2), pontos de código (um para este emoji, percorrível por for...of) e grafemas percebidos pela pessoa (que podem envolver vários pontos de código). codePointAt lê um ponto de código, enquanto normalize() permite comparar sequências canonicamente equivalentes; normalizar não resolve todas as regras de língua nem segmentação visual. Exemplo A: 'é' pré-composto e 'e' seguido de acento combinante podem parecer iguais e ter comprimentos diferentes. Exemplo B: conte pontos de código com [...texto].length e explique por que isso não é contador universal de caracteres visuais. A ponte antecede 05.11.

Ponte 06 — Objeto próprio, herança e referência

const a={x:1}; const b=a; b.x=2 faz a.x valer 2 porque ambos referem o mesmo objeto. const c={...a} cria outro objeto externo, mas objetos aninhados continuam compartilhados. structuredClone pode copiar certos grafos de dados, com limites e tipos suportados; JSON não é clone universal e pode perder tipos. Em uma cadeia de protótipos, 'x' in objeto consulta a cadeia; Object.hasOwn(objeto,'x') pergunta se o objeto possui a propriedade. Object.keys lista propriedades próprias enumeráveis, não tudo que in encontra. Descriptor mostra, por exemplo, se uma propriedade é enumerável ou gravável. Esses conceitos aparecem antes de class e this para que “herança” não seja uma caixa preta. “Stack/heap” aqui é analogia didática, não promessa do layout interno do motor JavaScript.

Antes de 06.06, um JSON com sintaxe inválida lança um erro de parse. try/catch delimita somente essa operação: o catch converte a falha de sintaxe em null; JSON sintaticamente válido ainda precisa de validação de esquema separada. Mais tarde, a etapa 13 amplia esse modelo para classes de erro, causas e diagnóstico, sem tratar catch como validação automática.

Ponte 07 — Escolher coleção pelo significado

Arrays preservam uma sequência de registros; callbacks transformam ou selecionam itens; Map guarda um valor por chave e Set representa membros únicos. A estrutura decorre da pergunta que cada métrica responde.

Exemplo resolvido A: map transforma [3,5] em [6,10] sem alterar a entrada. Exemplo resolvido B: ao percorrer doações, inicialize totais com zero quando totais.has(campanha) for falso e depois some o valor; doadores.add(id) elimina repetição de identidade. Exemplo com ajuda retirada: em uma lista curta de eventos, o aluno marca se precisa preservar ordem, agregar por chave ou contar únicos antes de escolher array, Map ou Set. A ponte prepara o exercício compacto 07.08 e depois o analisador 07.09.

Ponte 08 — Busca em largura antes de medir crescimento

Um grafo representa vértices e suas arestas; em um mapa sem pesos, BFS explora por camadas usando uma fila FIFO. Marque o vértice como visitado ao enfileirá-lo e guarde o predecessor na primeira descoberta. Assim ciclos não repetem trabalho e a cadeia de predecessores reconstrói um caminho mínimo.

Exemplo resolvido A: de A, enfileire B e C, nessa ordem. Exemplo resolvido B: ao remover B, descubra D e grave anterior[D] = B; C já está na fila, então não troque esse predecessor ao encontrar D outra vez. Exemplo com ajuda retirada: dado um mapa de estações com um ciclo, o aluno preenche a fila após cada remoção e reconstrói a rota de Central até Museu. A ponte antecede 08.07, que pratica o traçado, e o planejador completo 08.08.

Ponte 09 — Closure com ajuda decrescente

Exemplo resolvido A: function make(){let n=0;return ()=>++n} devolve uma função que ainda acessa o binding n depois do retorno de make. Duas chamadas a make() criam bindings independentes. Exemplo resolvido B: uma factory retorna inc e read; ambas leem o mesmo n, então read observa o novo valor após inc. A função conserva acesso ao binding, não uma fotografia do valor inicial. Exemplo parcialmente resolvido: forneça a factory e retire a implementação de reset, pedindo previsão antes de editar. Exemplo independente: criar um store com subscribe e unsubscribe. Se o aluno errar captura em loop, compare var compartilhado com let por iteração e agende transferência em domínio diferente, como handlers de linhas de tabela.

Ponte 10 — this, método e arrow

Exemplo resolvido A: obj.metodo() define o receptor pela forma da chamada; const f=obj.metodo; f() perde a base. Em módulo estrito, ler this.x nessa chamada solta pode lançar. Exemplo resolvido B: obj.metodo.call(outro) chama método comum com outro receptor, mas .call não substitui o this lexical de uma arrow. Exemplo parcialmente resolvido: o aluno recebe um callback extraído e escolhe bind ou wrapper ()=>obj.metodo() explicando qual objeto será consultado no momento da execução. Só depois aparecem classes e protótipos. Classe é ferramenta para um contrato de instâncias, não prêmio por avançar no currículo; composição é preferível quando capacidades variam independentemente.

Ponte 11 — Do HTML ao Kanban em módulos

O parser constrói DOM. Script clássico no head pode executar antes do body; defer e type="module" aguardam o parse; async executa quando disponível, sem ordem fixa frente ao parse. DOMContentLoaded tem relação específica com scripts adiados/módulos. Primeiro construa o Kanban V1 funcionando (11.08); depois reorganize-o em três arquivos (11.10): state.js exporta operações sobre tarefas; view.js recebe estado e cria nós; main.js importa os dois e instala eventos. Carregue main.js com type="module" por servidor local. Separar um código que já funciona permite conferir que o comportamento continua igual. O primeiro projeto pode ter testes manuais determinísticos; a etapa 13 acrescenta testes automatizados e evolução de dados.

Exemplo resolvido de segurança: p.textContent='<img src=x onerror=alert(1)>' mostra texto literal e não cria imagem. innerHTML interpreta markup e não recebe texto não confiável por padrão. Em eventos, capture desce, target identifica a origem e bubble sobe. preventDefault() cancela uma ação padrão cancelável; stopPropagation() interrompe propagação e não é sinônimo. once remove após uma chamada, passive promete não cancelar ação padrão e signal liga o listener a um AbortController para cleanup. Em formulário, FormData.get fornece uma entrada e getAll preserva nomes repetidos; números continuam strings até conversão e validação. Labels, foco e feedback dinâmico fazem parte do comportamento, não da decoração. A migração de save V1 para V2 (11.13) é a base da V3 de 13.14, que volta com testes.

Ponte 12 — HTTP, Promise e tempo

HTTP é uma troca entre request e response: método e URL pedem uma operação; status e headers descrevem a resposta; Content-Type ajuda a decidir como interpretar o corpo. fetch resolve a Promise para muitos status HTTP ruins, então cheque response.ok. O corpo é consumível: após json() ou text(), bodyUsed muda e uma segunda leitura falha; faça clone() antes da primeira leitura se duas leituras forem realmente necessárias. Nem toda resposta é JSON: use text, blob, arrayBuffer ou formData conforme o protocolo. Depois de parsear, ainda valide a forma dos dados externos. A aba Network do DevTools mostra método, URL, status, headers, corpo e tempo para uma falha concreta.

Exemplo resolvido de event loop A: código síncrono A, Promise.then P e timer T produzem A,P,T no navegador. Exemplo B: uma microtask que agenda outra entra na fila; ela não volta à pilha síncrona anterior. Exemplo parcialmente resolvido: compare Promise.all (todos necessários), allSettled (resultados parciais), race (primeiro assentamento) e any (primeiro sucesso; todas rejeitam com AggregateError.errors). Promise.try(fn) chama fn imediatamente e encapsula retorno/throw em Promise. Promise.resolve().then(fn) também encapsula, mas chama fn em uma microtask; não é substituto de mesma ordem. Uma alternativa fiel à ordem básica usa try { return Promise.resolve(fn()) } catch (e) { return Promise.reject(e) } quando a API moderna não estiver disponível.

Cancelamento e resposta obsoleta são problemas distintos: AbortController tenta interromper trabalho cancelável; um número de geração impede que a resposta antiga pinte a UI mesmo se chegar depois. TTL compara instantes numéricos (epoch ms), não datas formatadas. Guarde ISO/UTC para persistência e use Intl.DateTimeFormat apenas para apresentar em um fuso declarado. for await...of processa valores gradualmente; Array.fromAsync materializa tudo quando suportado. Em Node, a ordem detalhada de process.nextTick, Promise e fases depende do contexto e do formato CJS/ESM; a aula avançada 14.13 explicita o runtime antes da previsão.

Ponte 13 — Testar, depurar e preservar causas

Um teste útil fixa entrada, saída e borda observável. Um teste de regressão reproduz primeiro a falha antiga, depois protege a correção. vi.fn registra chamadas; vi.spyOn observa ou substitui um método; timers falsos controlam tempo; uma resposta de rede simulada evita instabilidade externa. O aluno escolhe Run para confirmar hipótese curta, debugger para acompanhar estado em código com múltiplos ramos, teste para uma regra repetível e documentação quando a dúvida é contrato de API. O painel Sources entra junto do breakpoint; Application/Storage junto da migração; Network junto do HTTP; Performance e Memory junto de gargalos.

Use tipos de erro de acordo com o contrato: erro de validação de entrada, status HTTP, parse, timeout, cancelamento e falha interna não são a mesma coisa. new Error('mensagem pública',{cause:erroOriginal}) preserva a causa técnica sem exibir token. Em catch de

TypeScript o valor pode não ser Error; trate unknown. AggregateError.errors lista falhas de um grupo; SuppressedError preserva erro durante descarte de recurso quando outro erro já existia. A etapa 13.14 retoma o Kanban como software existente e exige testes antes da migração.

Preparar o ambiente antes de 13.01: numa pasta nova, npm init -y, depois npm install -D vitest. Coloque "type": "module" e "scripts": { "test": "vitest" } no package.json, crie soma.test.js com import { test, expect } from 'vitest' e rode npm test: o Vitest fica em modo watch e reexecuta a cada arquivo salvo; npx vitest run roda uma vez. O site ainda não executa Vitest, então as suítes desta etapa rodam no seu computador.

Ponte 14 — Módulos, contratos e execução

Um módulo é um arquivo com fronteira explícita: export torna um valor disponível e import declara de onde outro arquivo o recebe. Um nome exportado não fica automaticamente global. Caminhos relativos partem do arquivo importador; extensão e modo do projeto fazem parte do contrato do runtime. Separe interface e implementação quando isso melhora a leitura, mas não crie arquivos só para imitar uma arquitetura maior.

Exemplo resolvido A: export function total(itens) { return itens.length; } publica uma função nomeada; import { total } from "./total.js" recebe exatamente esse nome. Exemplo resolvido B: export default function iniciar() {} pode ser importado com um nome escolhido pelo consumidor, enquanto export nomeado exige corresponder ao nome. Exemplo com ajuda retirada: diante de main.js, dados.js e um erro de import, o aluno identifica o caminho relativo e distingue falha de resolução de caminho de erro na função. Antes de avançar, confirme qual arquivo é a entrada e se o runtime executa ES Modules.

Ponte 15 — O compilador como primeiro leitor

TypeScript verifica relações estáticas e apaga os tipos: o JavaScript que roda é o mesmo sem as anotações. Leia um diagnóstico em três partes: código (TS2322), local e frase "Type A is not assignable to type B"; a parte importante costuma ser a última linha do detalhe. A compatibilidade é estrutural: duas interfaces com nomes diferentes e mesma forma são intercambiáveis. Literal fresco é exceção controlada: {nome:'Ana', extra:1} passado direto gera erro de propriedade em excesso, mas a mesma forma guardada antes numa variável passa. any desliga a verificação e se espalha para tudo que toca; unknown exige prova antes do uso. type Usuario = { nome: string } e interface Usuario { nome: string } descrevem a mesma forma de objeto. Tipos com parâmetro aparecem desde já: Array<number> é o mesmo que number[], e Promise<User> é a Promise que entrega um User; escrever os seus próprios fica para a etapa 17.

Exemplo resolvido A: com strict (padrão desde o TypeScript 6), function nome(u:{nome?:string}){ return u.nome.length } falha porque nome pode ser undefined; a correção é decidir o que fazer na ausência, não acrescentar !. Exemplo resolvido B: noUncheckedIndexedAccess faz lista[0] valer T | undefined, e exactOptionalPropertyTypes distingue "chave ausente" de {apelido: undefined}. Exemplo com ajuda retirada: diante de const u = JSON.parse(txt) as User, o aluno mostra que o cast compila e mesmo assim u.name.toUpperCase() falha quando name é número; a verificação do formato pertence à execução. readonly impede escrita pela visão tipada, não congela o objeto em runtime.

Para rodar fora do site: tsc --noEmit só verifica; o Node.js 24 executa .ts com sintaxe apagável (sem enum nem namespace com valor), e erasableSyntaxOnly faz o compilador acusar o que o Node não consegue apagar. O TypeScript 7 rejeita opções que o 6 apenas marcava como obsoletas (baseUrl, moduleResolution: "node", alvo ES5).

Ponte 16 — Estados possíveis, narrowing e exaustividade

Um tipo bom descreve só os estados que podem existir. Dois booleanos carregando e erro permitem quatro combinações, inclusive a contraditória; uma union discriminada {status:'loading'} | {status:'success', data:T} | {status:'error', message:string} permite exatamente três. Narrowing é o compilador acompanhando provas que o código faz: typeof, in, instanceof, comparação com o discriminante e predicates próprios.

Exemplo resolvido A: uma entrada unknown só aceita .name depois de provar que é objeto não nulo e que typeof v.name === 'string'. Exemplo resolvido B: no switch (estado.status), cada case libera apenas os campos daquele ramo; ler data no ramo error é erro de compilação. Exemplo parcialmente resolvido: acrescente uma variante e use assertNever(x:never):never no default; o compilador aponta cada switch que ainda não trata a novidade. catch (e) recebe unknown em modo estrito: teste e instanceof Error (ou Error.isError(e), quando disponível) antes de ler .message.

Um type predicate (v is Pix) é uma promessa do autor: se a função checa menos do que promete, o compilador confia e o erro aparece em runtime. Cast (as) também não verifica nada. as const congela literais no tipo e permite derivar a união a partir de um array runtime, mantendo uma fonte única.

Ponte 17 — Generics preservam relações

Um parâmetro de tipo existe para ligar duas posições: entrada e saída, chave e valor, item e callback. Se a saída é sempre a mesma, uma union ou um tipo fixo basta. identity<T>(x:T):T preserva o tipo; first<T>(xs:readonly T[]):T | undefined preserva o tipo e admite a lista vazia; map<T,U>(xs:T[], f:(x:T)=>U):U[] precisa de dois papéis porque a saída pode mudar de tipo, enquanto filter preserva um.

Exemplo resolvido A: get<T, K extends keyof T>(obj:T, key:K):T[K] faz o retorno acompanhar a chave escolhida. Exemplo resolvido B: um Repository<T extends {id:ID}, ID> mantém o tipo do identificador separado da entidade; uma constraint maior do que o corpo usa (exigir nome só para ler id) bloqueia consumidores legítimos. Exemplo com ajuda retirada: diante de três assinaturas, o aluno escolhe overload, union ou generic conforme a relação que o consumidor precisa preservar e confirma pela inferência no editor.

const em parâmetro de tipo (<const T>) mantém literais sem pedir as const ao chamador. enum gera um objeto em runtime; um objeto as const com união derivada produz o mesmo uso sem sintaxe que o Node.js não consegue apagar, por isso é a escolha padrão em código novo. Mantenha enum quando ele já existe e o projeto compila com tsc.

Ponte 18 — Derive tipos em vez de copiar

Quando dois tipos repetem os mesmos campos, uma mudança futura atualiza um e esquece o outro. Derive de uma fonte: keyof dá as chaves, T[K] dá o valor de uma chave, typeof valor sobe um objeto runtime para o tipo, e Partial, Pick, Omit, Record e Readonly transformam uma forma existente. Mapped types expandem uma chave de cada vez e só valem a pena quando preservam uma relação útil.

Exemplo resolvido A: type Patch<T> = { [K in keyof T]?: T[K] } aceita alteração parcial; um merge raso ainda substitui um objeto aninhado inteiro, e o tipo não muda essa regra de runtime. Exemplo resolvido B: const rotas = {...} satisfies Record<string, Rota> verifica a forma e preserva as chaves literais; a anotação const rotas: Record<string, Rota> alarga as chaves para string e perde o autocompletar. Exemplo com ajuda retirada: num mapa de handlers indexado pela união de eventos, o aluno acrescenta um evento sem handler e explica o erro que aparece.

import type some do JavaScript emitido; um import de valor permanece e executa o módulo importado. Com verbatimModuleSyntax, o código precisa dizer explicitamente qual dos dois quer. Índices seguros dependem de noUncheckedIndexedAccess; a flag muda a análise, não o código gerado.

Ponte 19 — Tipos condicionais com orçamento

Um tipo condicional é um if avaliado pelo compilador. Expandir à mão antes de confiar no resultado é a habilidade central desta etapa. Resolvido A: type Unwrap<T> = T extends Promise<infer U> ? U : T devolve number para Promise<number> e conserva string para string. Resolvido B: type IsString<T> = T extends string ? true : false aplicado a string | number devolve true | false, pois o parâmetro T nu distribui sobre a união; IsString<never> vira never. Exemplo com ajuda retirada: o aluno recebe só type AllStrings<T> = [T] extends [string] ? true : false, prevê por que AllStrings<string | number> é false e testa o efeito de tirar e pôr os colchetes num caso próprio.

Template literal types combinam uniões de strings: ` on${Capitalize<Evento>} gera um nome por evento. Tipos recursivos como JsonValue descrevem estruturas aninhadas, mas mensagens de erro ficam difíceis de ler rapidamente. IDs nominais por convenção (string & {__brand:'UserId'}`) só ajudam se um construtor validar a string antes de aplicar a marca.

Orçamento de complexidade: um tipo avançado precisa de type tests (@ts-expect-error para o caso proibido, igualdade de tipos para o permitido) e de um colega que consiga ler a mensagem de erro. Se ninguém da equipe consegue explicar o tipo, prefira uma API mais simples ou validação em runtime.

Ponte 20 — Iteradores, laziness e bytes

Um iterable entrega valores sob demanda: [Symbol.iterator]() devolve um objeto com next(), e cada chamada retorna {value, done}. O iterável guarda a configuração (início e fim); o iterador guarda o cursor. Por isso cada for...of deve criar um cursor novo. Laziness significa que take(3) sobre uma fonte infinita lê só três valores. Um teste com contador de next() comprova isso melhor do que olhar o resultado. Os iterator helpers (map, filter, take, drop em Iterator.prototype) e Iterator.concat (ECMAScript 2026) fazem o mesmo sem criar arrays intermediários.

Exemplo resolvido A: function* contar(n){ for(let i=0;i<n;i++) yield i }: o corpo só avança quando alguém chama next(). Exemplo resolvido B: interromper um for...of com break chama return() no iterador, e é assim que a fonte libera recursos. Exemplo com ajuda retirada: num pipeline lazy, o aluno prevê quantas leituras take(0) faz (nenhuma) e o que acontece quando o consumidor para antes de a fonte falhar.

Um async iterable pode aguardar entre valores; Array.fromAsync materializa tudo de uma vez e só convém quando o volume é pequeno. ReadableStream torna visível o backpressure quando o consumidor não acompanha o produtor. TextEncoder transforma string em bytes UTF-8: 'é'.length é 1, mas ocupa 2 bytes, e um emoji pode ter length 2 e 4 bytes.

Ponte 21 — Medir, limitar e liberar recursos

Desempenho começa com uma hipótese e uma medida, não com uma otimização. Separe o aquecimento das amostras, repita a medição, reporte mediana e dispersão e descreva o ambiente; uma execução única não prova nada. Confirme que as duas versões produzem o mesmo resultado antes de comparar tempo.

Exemplo resolvido A: um cache Map sem limite cresce com cada chave nova; uma política LRU usa a ordem de inserção do Map (apagar e reinserir no acesso, remover keys().next().value ao passar do limite). Exemplo resolvido B: WeakMap serve para associar dados a um objeto sem impedir que ele seja coletado, mas não aceita string como chave nem permite contar ou listar entradas. getOrInsert e getOrInsertComputed (ECMAScript 2026) substituem o par has/set quando o runtime oferece. Exemplo com ajuda retirada: o aluno desenha uma linha do tempo de eventos e marca onde debounce e throttle (leading/trailing) chamam a função.

Todo recurso criado tem um dono que o libera: listener, timer, worker, conexão. using x = recurso chama x[Symbol.dispose]() ao sair do bloco, e await using aguarda Symbol.asyncDispose; o Node.js 24 suporta os dois, e em navegadores é preciso conferir o suporte. Se o corpo lança A e o descarte lança B, o erro propagado é um SuppressedError com error = B e suppressed = A. Em postMessage(buf, [buf]), o ArrayBuffer é transferido e fica com byteLength 0 no emissor; sem a lista de transferência, o worker recebe uma cópia estruturada.

Ponte 22 — Pacotes, declarações e fronteiras

Uma biblioteca tem dois contratos: o JavaScript que roda e os arquivos .d.ts que descrevem esse JavaScript. Quando eles divergem, o compilador do consumidor confia na declaração e o erro aparece em runtime. Em package.json, "type": "module" define o formato e "exports" lista as entradas públicas; a condição "types" vem antes de "import" ou "default" dentro de cada entrada.

Exemplo resolvido A: export function parse(txt:string): Result<Config> com Result = {ok:true, value:T} | {ok:false, error:string} obriga o consumidor a tratar os dois casos, sem try/catch invisível. Exemplo resolvido B: com isolatedDeclarations, cada export precisa de um tipo de retorno explícito para que ferramentas gerem .d.ts sem rodar o verificador completo. Exemplo com ajuda retirada: o aluno empacota com npm pack, instala o .tgz num projeto vazio e confirma que import, tipos e execução funcionam fora do workspace.

Dados externos continuam unknown até serem validados: um guard manual serve para formas pequenas, e uma biblioteca de schema compensa quando a mesma forma precisa de validação, tipo derivado e mensagem de erro. Augmentation de módulo global muda o tipo para todo o programa; prefira um wrapper. Migração para TypeScript vai de módulo em módulo, com um critério verificável para avançar (typecheck limpo e testes passando), e um código legado em checkJs ou JSDoc continua válido durante a transição.

Ponte 23 — Ler, mudar e operar um sistema

Leia um mini-repositório antes de editar: README, árvore de arquivos, testes, fluxo da entrada até a regra e persistência. Uma arquitetura útil separa domínio (regras puras), portas (interfaces que o domínio precisa) e adapters (storage, HTTP, DOM). A pergunta de cada mudança é qual camada é dona da regra.

Exemplo resolvido A: uma issue "limite permite 11 itens" começa com um teste que reproduz o caso exatamente no limite; só depois a correção entra, na regra do domínio e não na interface. Exemplo resolvido B: numa revisão de PR, cada achado tem local, impacto e evidência ou correção mínima; "não gostei" não é achado. Exemplo com ajuda retirada: o aluno refatora código legado em passos pequenos, cada um protegido por um teste que passava antes e continua passando depois.

O capstone recebe mudança de requisito e auditoria de operação: entradas externas validadas na fronteira, DOM seguro, dados antigos migrados, logs estruturados sem segredo, causas de erro preservadas com cause e dependências justificadas. Construa o sistema (23.08), torne as falhas observáveis (23.09) e só então aplique a mudança de autorização com auditoria (23.07). Escolha ferramentas pelo problema: não há bônus por classe, Proxy, generic, regex, worker ou biblioteca quando uma solução simples cumpre melhor o contrato.

Grafo curricular e dificuldade

O grafo contém 105 conceitos, 261 atividades e 486 arestas verificadas sem ciclo. O ID 15.05 redireciona para 16.09. Conceitos de uma etapa são ensinados na ponte/aula antes dos cartões que os exigem; pré-requisito de etapa futura invalida o build.

A dificuldade registrada em cada ficha tem nove eixos: novidade, sintaxe, composição, estado, depuração, quantidade de arquivos, leitura, independência e ferramentas. A nota visual 1–5 resume o perfil, mas o motor usa os eixos e a evidência do aluno; minutos são apenas previsão de esforço.

Conceitos por etapa

00 Ambiente e leitura de código: execution.order, environment.boundaries, html.tree_reading, modules.esm, execution.contradiction, execution.sequence, execution.fill_console, debug.first_error, execution.make, execution.explain_program

01 Valores e tipos: bindings.values, primitives.types, primitives.typeof, bindings.reassignment, bindings.declaration_order, bindings.const_let_choice, primitives.special_values, bindings.template_literal, bindings.configuration, bindings.textual_profile

02 Operadores, Math e Number: operators.basic, number.model, math.basic, operators.predict, operators.arithmetic, operators.comparison, operators.logical, operators.falsy_nullish, operators.boolean_validation, coercion.number_validation, coercion.explicit, operators.nullish_fallback, operators.discount_expression, operators.preferences, operators.scoreboard, math.rounding, math.random, number.ieee754, number.safe_integers, number.tolerance

03 Decisões e repetição: control.branch, control.loop

04 Funções e decomposição: functions.return, functions.decomposition, errors.throw, errors.types

05 Strings, arrays, regex e Unicode: arrays.basic, strings.basic, regex.basic, unicode.basic, callbacks.basic

06 Objetos, referências e JSON: objects.identity, objects.own, serialization.json, errors.try_catch

07 Callbacks, coleções e pipelines: arrays.pipeline, mapset.basic

08 Algoritmos e complexidade: algorithms.complexity, algorithms.recursion

09 Escopo e closures: scope.lexical, closures.basic

10 this, protótipos e classes: this.binding, prototype.chain, classes.basic

11 DOM, eventos, formulário e Kanban: dom.basic, events.propagation, forms.basic, storage.versioned, modules.browser

12 HTTP, assincronismo e tempo: async.promise, http.basic, fetch.basic, date.time, async.iterables

13 Teste, debugger e manutenção do Kanban: testing.behavior, debugging.tools, errors.cause

14 ESM, Node, npm e Git: modules.project_structure, node.basics, git.basics, npm.safety

15 TypeScript: contratos e runtime: ts.basic, ts.unknown, ts.readonly

16 Unions, narrowing e validação: ts.unions, ts.narrowing, validation.manual

17 Generics e APIs reutilizáveis: ts.generics, ts.functions

18 Utility types e relações entre tipos: ts.keyof_indexed_access, ts.utility, ts.satisfies, ts.import_type

19 Tipos avançados com propósito: ts.advanced, ts.branding

20 Iteradores, streams e bytes: iterators.sync, iterators.async, streams.basic

21 Performance, workers e recursos: performance.measure, workers.basic, resources.cleanup

22 Pacotes, declarations e schemas: ts.packages, validation.schema

23 Arquitetura, manutenção e capstone: architecture.boundaries, operations.observability

D Diagnóstico inicial

5 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: D.01 · D.02 · D.03 · D.04 · D.05

D.01 Leitura de saída

PREDICT · 3 min (estimativa, sem nota por velocidade) · Navegador ou Node.js 24 LTS · dificuldade multidimensional 1.0/5

Competência: diagnostic.execution_order | Requer: nenhum | Introduz: diagnostic.execution_order

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 1 · depuração 1 · arquivos 1 · leitura 1 · autonomia 1 · ferramentas 1

VISÃO DO ALUNO

Sem executar, preveja as saídas e registre confiança de 0–100%; depois confira. O diagnóstico não dá nota.

Objetivo da atividade

Anote a saída prevista para cada chamada, na ordem do arquivo, e sua confiança de 0 a 100%.

Explicação e modelo mental

• Modelo mental: As chamadas síncronas seguem a ordem do arquivo. Cada argumento é avaliado antes de console.log escrever sua linha.
• Exemplo resolvido: console.log("porta"); escreve porta. A expressão 3 + 2 vale 5 antes da chamada seguinte escrever 5; depois, a última chamada escreve aberta.
• Armadilha comum: Ler 3 + 2 como texto literal ou imaginar que as chamadas síncronas podem trocar de ordem.
• Por que importa: Prever a saída antes de executar ajuda a perceber qual linha divergiu quando o resultado não corresponde à expectativa.

Material/arquivo inicial

main.js: console.log("porta");
console.log(3 + 2);
console.log("aberta");

Exemplo e resultado público

código mostrado → porta, 5, aberta, nessa ordem

Bordas e erros publicados

trocar a linha do meio por console.log(0) -> porta, 0, aberta

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao percorrer as três chamadas de main.js sem executar, qual regra permite prever as linhas corretamente?

 (Correta) As chamadas síncronas seguem a ordem textual; cada argumento é avaliado antes de sua chamada imprimir.
• JavaScript espera todas as chamadas de console.log em uma fila assíncrona, então a ordem pode inverter.
• console.log recebe uma expressão aritmética como texto literal se ela não estiver guardada numa variável.

Por que a resposta está certa

Correto. A expressão é calculada antes da chamada escrever a linha, e as instruções síncronas seguem a ordem do arquivo.

Armadilha da alternativa incorreta

Leia uma chamada de cada vez, de cima para baixo. Sem aspas, 3 + 2 é uma expressão avaliada antes de console.log escrever o resultado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

trocar a linha do meio por console.log(0) → porta, 0, aberta

Concepção errada e falha observável

Confundir ordem de execução e ordem imaginada da narrativa. / Previsão registrada antes de Run difere na primeira ou segunda linha.

Pergunta diagnóstica

Qual chamada aparece primeiro no arquivo?

Pistas graduais

1. Leia uma linha por vez. 2. Anote a saída logo após cada chamada. 3. Em logs A, 1+1, B a saída é A, 2, B.

Solução de referência explicada

As chamadas síncronas imprimem porta, 5, aberta, em ordem textual.

Transferência em outro contexto

—

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe básica de ECMAScript 2026 (variáveis, condicionais, funções); roda no navegador ou no Node.js 24 LTS sem APIs recentes. | Próxima sugestão: D.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

D.02 Valores e tipos

PREDICT · 4 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: diagnostic.values_types | Requer: nenhum | Introduz: diagnostic.values_types

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 1 · depuração 1 · arquivos 1 · leitura 1 · autonomia 1 · ferramentas 1

VISÃO DO ALUNO

Classifique os valores apresentados, incluindo null, 0 e string vazia; informe confiança.

Objetivo da atividade

Para cada cartão apresentado, registre o valor, seu tipo conceitual e, quando solicitado, o resultado de typeof; marque sua confiança.

Explicação e modelo mental

• Modelo mental: Tipo, resultado de typeof e truthiness são perguntas diferentes. null é um valor próprio apesar do resultado histórico typeof null === "object"; typeof NaN retorna "number".
• Exemplo resolvido: 0 é number e falsy; "0" é string e truthy. null continua sendo o valor null, embora typeof null retorne "object".
• Armadilha comum: Classificar pela aparência ou confundir ser truthy/falsy com o tipo do valor.
• Por que importa: Separar essas noções evita erros ao interpretar dados e condições.

Material/arquivo inicial

material.md: cartões: 0, '', false, null, '0', undefined

Exemplo e resultado público

0 → number; '' -> string; false -> boolean; null -> null conceitual e typeof object; '0' -> string; undefined -> undefined

Bordas e erros publicados

NaN -> number em typeof, embora não represente número válido para cálculo

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual afirmação classifica corretamente o valor null e o resultado de typeof null?

• null é um objeto comum; typeof null retorna "null".
 (Correta) null é um valor distinto; typeof null retorna "object" por uma exceção histórica.
• null é outra forma de undefined; typeof null retorna "undefined".

Por que a resposta está certa

Correto. null é seu próprio valor; typeof null retornar "object" é uma exceção histórica da linguagem.

Armadilha da alternativa incorreta

Separe o valor da saída de typeof: null continua distinto, embora typeof null retorne a string "object".

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

NaN → number em typeof, embora não represente número válido para cálculo

Concepção errada e falha observável

Tratar aparência textual e truthiness como tipo. / O aluno diz que '0' é number ou que typeof null é null.

Pergunta diagnóstica

Qual é a natureza do valor, separada de ser truthy ou falsy?

Pistas graduais

1. Aspas indicam string. 2. typeof null é exceção histórica, embora null seja valor distinto. 3. typeof NaN retorna number; validade numérica é outra pergunta.

Solução de referência explicada

Classifique cada valor pelo tipo da linguagem; registre separadamente a exceção histórica de null.

Transferência em outro contexto

—

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Tipos primitivos, null, typeof e coerção básica; ECMAScript 2026 no Node.js 24 LTS, sem APIs recentes. | Próxima sugestão: D.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

D.03 Fluxo de decisão

TRACE · 4 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: diagnostic.branch | Requer: nenhum | Introduz: diagnostic.branch

Eixos: novidade 1 · sintaxe 1 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 2 · autonomia 1 · ferramentas 1

VISÃO DO ALUNO

Leia uma regra if/else e preveja o ramo em duas entradas.

Objetivo da atividade

Para cada entrada, anote se a condição pontos >= 10 é verdadeira e qual mensagem o ramo escolhido produz.

Explicação e modelo mental

• Modelo mental: Em um if/else, a condição escolhe um dos ramos. O operador >= inclui o limite comparado.
• Exemplo resolvido: Com pontos igual a 10, pontos >= 10 é verdadeiro e a saída é passou. Com 9, a condição é falsa e a saída é tente.
• Armadilha comum: Ler >= como se fosse > e mandar o próprio valor-limite para o ramo else.
• Por que importa: Testar abaixo, no limite e acima da condição ajuda a confirmar regras de decisão.

Material/arquivo inicial

main.js: if (pontos >= 10) console.log("passou"); else console.log("tente");

Exemplo e resultado público

pontos=10 → passou; pontos=9 -> tente

Bordas e erros publicados

pontos=11 -> passou; pontos=0 -> tente

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que caracteriza o ramo else em uma estrutura if/else?

• Ele substitui a condição inicial por uma segunda comparação independente.
• Ele executa sempre após o bloco if, mesmo quando a condição foi verdadeira.
 (Correta) Ele oferece o caminho alternativo quando a condição do if resulta em false.

Por que a resposta está certa

Correto. O if trata o caminho em que sua condição é verdadeira; o else trata a alternativa.

Armadilha da alternativa incorreta

O else não repete a condição nem executa em todos os casos: ele é o caminho alternativo quando a condição do if é falsa.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

pontos=11 → passou; pontos=0 -> tente

Concepção errada e falha observável

Esquecer que >= inclui o próprio limite. / Resposta de pontos=10 toma o ramo inferior.

Pergunta diagnóstica

O valor 10 satisfaz a comparação >= 10?

Pistas graduais

1. Leia o operador antes dos ramos. 2. Teste abaixo, exatamente no limite e acima. 3. Para x >= 5, x=5 entra no primeiro ramo.

Solução de referência explicada

Com 10 ou mais, imprime passou; com menos de 10, imprime tente.

Transferência em outro contexto

—

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: if/else e comparações estritas; ECMAScript 2026 no Node.js 24 LTS. | Próxima sugestão: D.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

D.04 Função e retorno

READ · 4 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: diagnostic.function_return | Requer: nenhum | Introduz: diagnostic.function_return

Eixos: novidade 2 · sintaxe 1 · composição 2 · estado 2 · depuração 1 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Identifique entrada, retorno e efeito de uma função curta.

Objetivo da atividade

Registre a entrada da função, o valor que return devolve e a saída causada por console.log.

Explicação e modelo mental

• Modelo mental: return devolve um valor à chamada; console.log produz uma saída. Uma função pode retornar sem imprimir.
• Exemplo resolvido: dobro(3) chama a função com entrada 3 e recebe 6. A chamada externa console.log(resultado) é que escreve uma linha com 6.
• Armadilha comum: Afirmar que return imprime o valor ou que console.log devolve o resultado de dobro.
• Por que importa: Distinguir retorno de efeito permite acompanhar o fluxo de dados entre funções e suas chamadas.

Material/arquivo inicial

main.js: function dobro(n) { return n * 2; }
const resultado = dobro(3);
console.log(resultado);

Exemplo e resultado público

entrada 3 → retorno 6 e efeito de console: uma linha 6

Bordas e erros publicados

entrada 0 -> retorno 0 e uma linha 0

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em dobro(3), seguida de console.log(resultado), qual descrição separa corretamente retorno e efeito?

 (Correta) dobro recebe 3 e retorna 6; a chamada externa a console.log imprime 6.
• dobro imprime 6 por causa de return; console.log recebe um valor sem retorno.
• console.log calcula e retorna 6; dobro apenas imprime o argumento.

Por que a resposta está certa

Correto. A função calcula e retorna o valor; a chamada externa causa a saída.

Armadilha da alternativa incorreta

Localize separadamente return dentro da função e console.log fora dela: um devolve um valor, o outro escreve a saída.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada 0 → retorno 0 e uma linha 0

Concepção errada e falha observável

Confundir return com imprimir e contar console como parte da função. / O aluno afirma que dobro imprime ou que console.log retorna o dobro.

Pergunta diagnóstica

Qual linha devolve o valor à chamada e qual linha causa saída?

Pistas graduais

1. Sublinhe return dentro da função. 2. Sublinhe console.log fora dela. 3. Em function f(x){return x+1}, f(2) devolve 3 mesmo sem log.

Solução de referência explicada

dobro(3) retorna 6; a chamada a console.log fora da função imprime esse resultado.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Declaração de função e return; ECMAScript 2026 no Node.js 24 LTS. | Próxima sugestão: D.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

D.05 Localize a falha

INVESTIGATE · 5 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: diagnostic.tool_choice | Requer: nenhum | Introduz: diagnostic.tool_choice

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Escolha leitura, Run ou teste para localizar uma falha curta e explique sua escolha.

Objetivo da atividade

Para a pergunta apresentada, registre sua hipótese, a ferramenta inicial e a evidência que espera obter.

Explicação e modelo mental

• Modelo mental: Escolha a ferramenta de acordo com a incerteza: leia ou execute para conferir uma saída curta, use um teste pequeno para reproduzir uma borda e inspecione o estado quando o fluxo divergir.
• Exemplo resolvido: Para [2, -1, 3], a soma dos positivos é 5 e leitura ou execução pode conferir a previsão. Para investigar uma falha de borda, reproduza-a num teste pequeno; se o estado do loop divergir, observe-o no debugger.
• Armadilha comum: Abrir o debugger sem hipótese ou limitar-se a ler quando há uma divergência de estado que precisa ser localizada.
• Por que importa: Uma hipótese e uma evidência apropriadas tornam a investigação reproduzível e ajudam a distinguir causa de palpite.

Material/arquivo inicial

main.js: function total(xs) { let n=0; for (const x of xs) if(x>0) n+=x; return n; }

Exemplo e resultado público

pergunta 'qual retorno para [2,-1,3]?' → leitura/Run é suficiente, 5; pergunta 'por que falha em caso grande?' -> teste de borda e debugger se fluxo divergir

Bordas e erros publicados

lista vazia -> 0; ferramenta inicial para hipótese de borda: teste pequeno reproduzível

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Um caso grande produz soma errada, e o contador diverge do esperado dentro do loop. Qual abordagem corresponde a essa incerteza?

• Executar repetidamente o caso grande sem registrar hipótese, até que uma saída pareça correta.
 (Correta) Criar um caso pequeno reproduzível e usar um breakpoint para observar o estado na primeira divergência.
• Ler apenas a assinatura da função, pois observar o estado durante a iteração não ajuda a localizar a divergência.

Por que a resposta está certa

Correto. O teste preserva a reprodução, e o debugger ajuda a localizar onde o estado deixa de corresponder à hipótese.

Armadilha da alternativa incorreta

Combine uma reprodução pequena com a ferramenta adequada à divergência observada: inspecione o estado no ponto em que o loop se comporta diferente do esperado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

lista vazia → 0; ferramenta inicial para hipótese de borda: teste pequeno reproduzível

Concepção errada e falha observável

Usar debugger por reflexo sem hipótese ou apenas ler quando há divergência de estado. / Escolha de ferramenta não responde ao tipo de incerteza apresentado.

Pergunta diagnóstica

Você precisa confirmar uma saída simples, observar transição de estado ou reproduzir falha?

Pistas graduais

1. Para código curto, tente prever antes de Run. 2. Para desvio entre previsão e realidade, inspecione estado no ponto da divergência. 3. Teste registra a borda para que a falha não volte.

Solução de referência explicada

Leia e preveja 5; rode para conferir. Para falha reproduzível, escreva teste; use debugger se o estado divergir entre iterações.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Leitura de erro e stack trace no terminal do Node.js 24 LTS ou no console do navegador; sem API específica. | Próxima sugestão: 00.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00 Ambiente e leitura de código

9 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 00.01 · 00.02 · 00.03 · 00.04 · 00.07 · 00.05 · 00.06 · 00.08 · 00.09

00.01 Seu primeiro contato com código e saída

PREDICT · 6 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: execution.order | Requer: nenhum | Introduz: execution.order

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 1 · depuração 1 · arquivos 1 · leitura 1 · autonomia 1 · ferramentas 1

VISÃO DO ALUNO

Leia as três linhas de main.js e use o exemplo resolvido acima. Anote a saída prevista de cada linha e sua confiança. Registre a previsão, execute o arquivo e compare. Não altere o código nesta atividade.

Objetivo da atividade

Para cada linha, leia o valor entre parênteses. Se for uma conta, calcule primeiro; depois console.log mostra esse valor. Só então vem a próxima linha. Chamamos essa sequência de execução síncrona: neste programa, as chamadas acontecem uma após outra.

Explicação e modelo mental

• Modelo mental: Para cada linha, leia o valor entre parênteses. Se for uma conta, calcule primeiro; depois console.log mostra esse valor. Só então vem a próxima linha. Chamamos essa sequência de execução síncrona: neste programa, as chamadas acontecem uma após outra.
• Exemplo resolvido: A linha 1 mostra começo; as aspas pertencem ao código, não à saída. Na linha 2, 4 + 2 vira 6 antes de a mensagem aparecer. A linha 3 mostra pronto depois das anteriores. Há três chamadas e três linhas de resultado.
• Armadilha comum: Não leia console.log(2 + 3) como caracteres para imprimir: sem aspas, 2 + 3 é uma expressão. Também não reordene chamadas síncronas pela narrativa do texto.
• Por que importa: Prever uma saída antes de executar deixa visível seu modelo de execução. A mesma prática ajuda depois a localizar a primeira divergência ao depurar, em vez de mudar linhas às cegas.

Material/arquivo inicial

main.js: console.log("A");
console.log(2 + 3);
console.log("B");

Exemplo e resultado público

código mostrado → A, 5, B, nessa ordem

Bordas e erros publicados

troque literais por "X", 4 + 1, "Y" -> X, 5, Y, nessa ordem

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

MDN console.log: https://developer.mozilla.org/en-US/docs/Web/API/console/log_static; Node.js Console: https://nodejs.org/api/console.html#consolelogdata-args

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que console.log(4 + 1); mostra?

• O texto 4 + 1, sem calcular.
• Duas linhas: uma com 4 e outra com 1.
 (Correta) O número 5, depois de calcular a soma.

Por que a resposta está certa

Sem aspas, a expressão é calculada antes de a chamada mostrar o valor.

Armadilha da alternativa incorreta

Releia a explicação da atividade. Sem aspas, a expressão é calculada antes de a chamada mostrar o valor.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

troque literais por "X", 4 + 1, "Y" → X, 5, Y, nessa ordem

Concepção errada e falha observável

Confundir ordem visual de leitura com ordem das chamadas síncronas. / A primeira linha divergente da previsão revela que o aluno pulou ou reordenou uma chamada.

Pergunta diagnóstica

Qual console.log é executado primeiro, e o que ele imprime?

Pistas graduais

1. Leia as chamadas de cima para baixo. 2. Anote uma saída por chamada, sem reorganizar. 3. Exemplo análogo: logs "I", 1 + 1, "F" produzem I, 2, F.

Solução de referência explicada

A primeira chamada imprime "A"; a segunda calcula 2 + 3 e imprime 5; a terceira imprime "B".

Transferência em outro contexto

Em um arquivo de status com logs "início", 1 + 2 e "fim", preveja as três linhas e indique em qual etapa a soma é calculada.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: console.log e execução de módulos ES no Node.js 24 LTS; sem APIs além da saída padrão. | Próxima sugestão: 00.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.02 Compare uma alteração no programa

TRACE · 7 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: execution.contradiction | Requer: execution.order | Introduz: execution.contradiction

Eixos: novidade 1 · sintaxe 1 · composição 2 · estado 1 · depuração 1 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Compare versao-1.js e versao-2.js. Preveja três saídas para cada arquivo e indique qual chamada mudou. Execute os dois arquivos separadamente pelo botão e explique por que as outras mensagens continuam iguais.

Objetivo da atividade

Nestes programas, cada chamada mostra sua própria mensagem. Substituir uma mensagem na segunda linha não muda as chamadas de abertura e encerramento.

Explicação e modelo mental

• Modelo mental: Nestes programas, cada chamada mostra sua própria mensagem. Substituir uma mensagem na segunda linha não muda as chamadas de abertura e encerramento.
• Exemplo resolvido: // exemplo é um comentário e não aparece na saída. Se você trocar apenas "revisar" por "confirmar", a saída central passa a ser confirmar. As mensagens abrir e fechar permanecem nas mesmas posições.
• Armadilha comum: A previsão inicial não continua válida só porque o arquivo parece familiar. Releia a versão atual e recalcule a linha alterada antes de concluir.
• Por que importa: Em uma sequência de status, uma mudança pequena deve produzir uma consequência localizada que possa ser explicada. Isso ajuda a revisar alterações sem atribuir efeitos às linhas que não mudaram.

Material/arquivo inicial

versao-1.js: console.log("entrada");
console.log("processo");
console.log("saída"); | versao-2.js: console.log("entrada");
console.log("novo");
console.log("saída");

Exemplo e resultado público

versão inicial → entrada, processo, saída; segunda linha trocada por console.log('novo') -> entrada, novo, saída

Bordas e erros publicados

inverta primeira e última linhas -> saída, processo, entrada

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

MDN console.log: https://developer.mozilla.org/en-US/docs/Web/API/console/log_static; Node.js Console: https://nodejs.org/api/console.html#consolelogdata-args

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Você troca somente a mensagem da segunda chamada. O que acontece com as outras duas?

 (Correta) Continuam com suas mensagens na mesma ordem.
• Também mudam de mensagem automaticamente.
• Deixam de aparecer.

Por que a resposta está certa

Nestes trechos as chamadas são independentes. A edição muda somente a mensagem alterada.

Armadilha da alternativa incorreta

Releia a explicação da atividade. Nestes trechos as chamadas são independentes. A edição muda somente a mensagem alterada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

inverta primeira e última linhas → saída, processo, entrada

Concepção errada e falha observável

Repetir a primeira previsão mesmo após uma linha mudar. / A resposta conserva uma saída que já não está no arquivo.

Pergunta diagnóstica

Qual linha mudou? A previsão foi refeita a partir do programa atual?

Pistas graduais

1. Compare as duas versões linha por linha. 2. Recalcule a saída da chamada alterada e confira as demais no código atual. 3. Neste programa, as três chamadas são independentes; por isso, trocar um literal muda só a saída daquela chamada.

Solução de referência explicada

Na segunda versão, a chamada do meio imprime novo; as chamadas da primeira e terceira linhas não mudaram.

Transferência em outro contexto

Em um log de três passos de uma implantação, substitua mentalmente a mensagem final e registre quais linhas da saída permanecem iguais antes de executar.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Dois arquivos .js executados separadamente no Node.js 24 LTS; console.log e chamadas de função. | Próxima sugestão: 00.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.03 Monte um programa com linhas prontas

PARSONS · 7 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: execution.sequence | Requer: execution.order | Introduz: execution.sequence

Eixos: novidade 1 · sintaxe 1 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Monte um programa que mostre exatamente Início, Olá, Ana e Fim, nessa ordem. Opções: A = console.log("Início"); B = console.log("Olá, Ana"); C = console.log("Fim"); D = console.log("Olá");. Escolha três blocos e deixe de fora a mensagem que não foi pedida.

Objetivo da atividade

Primeiro associe cada mensagem pedida a uma linha pronta. Depois copie somente as linhas escolhidas na ordem das mensagens. Os rótulos A=, B= etc. não entram no editor.

Explicação e modelo mental

• Modelo mental: Primeiro associe cada mensagem pedida a uma linha pronta. Depois copie somente as linhas escolhidas na ordem das mensagens. Os rótulos A=, B= etc. não entram no editor.
• Exemplo resolvido: Se as opções são P=encerrar, Q=abrir, R=verificar e S=pausa, a sequência dos rótulos é Q, R, P. Copie as chamadas correspondentes, uma por linha. Não copie os rótulos. O bloco pausa fica de fora porque não faz parte das três mensagens solicitadas.
• Armadilha comum: Usar todos os blocos só porque foram apresentados acrescenta uma saída que o requisito não pediu. Verifique o objetivo antes de incluir um distrator.
• Por que importa: Separar requisito de material disponível é uma habilidade de leitura de tarefas. Em código real, distinguir comportamento solicitado de log temporário evita incluir efeitos que confundem quem usa o programa.

Material/arquivo inicial

main.js:

Exemplo e resultado público

ordem pedida Início/Olá, Ana/Fim → A,B,C; D não usado

Bordas e erros publicados

se a saudação literal vira Olá, Rui -> A,bloco alterado,C; distrator segue fora

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

MDN console.log: https://developer.mozilla.org/en-US/docs/Web/API/console/log_static

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Há quatro linhas disponíveis, mas o pedido exige três mensagens. Como escolher?

• Usar as quatro porque todas foram oferecidas.
 (Correta) Escolher as linhas que correspondem às mensagens pedidas e ordenar essas três.
• Escolher as três linhas mais curtas.

Por que a resposta está certa

O resultado pedido orienta a escolha; uma opção disponível pode ficar de fora.

Armadilha da alternativa incorreta

Releia a explicação da atividade. O resultado pedido orienta a escolha; uma opção disponível pode ficar de fora.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

se a saudação literal vira Olá, Rui → A,bloco alterado,C; distrator segue fora

Concepção errada e falha observável

Tratar bloco distrator como parte obrigatória do programa. / A saída contém a saudação curta do distrator ou aparece fora da ordem pedida.

Pergunta diagnóstica

Quais três mensagens o enunciado pede, em que ordem?

Pistas graduais

1. Compare cada bloco com as três mensagens pedidas. 2. O distrator também parece uma saudação, mas não corresponde ao texto solicitado. 3. A saída pedida "Olá, Ana" não é a mesma mensagem que "Olá".

Solução de referência explicada

Use A, B, C. O bloco D é distrator e não entra na montagem.

Transferência em outro contexto

Ordene os três logs necessários para um recibo — abertura, valor e fecho — e marque o quarto bloco de depuração que não deve aparecer na saída.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: console.log com strings, na ordem do arquivo; Node.js 24 LTS. | Próxima sugestão: 00.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.04 Complete os valores de uma mensagem

FADED · 6 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: execution.fill_console | Requer: execution.order, execution.sequence | Introduz: execution.fill_console

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Substitua somente as três lacunas ___ por "Ana", 0 e "fim", respectivamente. A saída deve ser nome: Ana, pontos: 0 e fim, uma mensagem por linha. Preserve os rótulos, vírgulas, parênteses e a ordem.

Objetivo da atividade

Em console.log("vidas:", 2), a chamada recebe texto e número. Nesta saída eles aparecem juntos, separados por um espaço: vidas: 2. Use aspas para textos e deixe o número sem aspas.

Explicação e modelo mental

• Modelo mental: Em console.log("vidas:", 2), a chamada recebe texto e número. Nesta saída eles aparecem juntos, separados por um espaço: vidas: 2. Use aspas para textos e deixe o número sem aspas.
• Exemplo resolvido: A primeira chamada recebe dois textos e mostra cidade: Recife. A segunda recebe o rótulo e o número zero: a linha aparece normalmente. A última tem um único argumento. Cada chamada gera uma linha.
• Armadilha comum: Não troque o rótulo pelo valor nem apague uma chamada ao preencher uma lacuna. Confira uma linha para cada chamada do arquivo.
• Por que importa: Separar rótulo e valor ajuda a ler saídas de estado e a localizar qual chamada produziu cada linha.

Material/arquivo inicial

main.js: console.log("nome:", ___);
console.log("pontos:", ___);
console.log(___);

Exemplo e resultado público

lacunas "Ana", 0, "fim" → nome: Ana; pontos: 0; fim

Bordas e erros publicados

lacunas "Ana", 0, "fim" -> nome: Ana; pontos: 0; fim

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

MDN console.log: https://developer.mozilla.org/en-US/docs/Web/API/console/log_static

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o resultado de console.log("vidas:", 0);?

• Uma linha vazia porque zero não aparece.
• Duas linhas separadas: vidas: e 0.
 (Correta) vidas: 0

Por que a resposta está certa

A vírgula separa os argumentos da mesma chamada. O rótulo e o zero aparecem juntos.

Armadilha da alternativa incorreta

Releia a explicação da atividade. A vírgula separa os argumentos da mesma chamada. O rótulo e o zero aparecem juntos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

lacunas "Ana", 0, "fim" → nome: Ana; pontos: 0; fim

Concepção errada e falha observável

Confundir o valor mostrado com o rótulo ou apagar uma chamada ao preencher uma lacuna. / A saída fica sem uma das três linhas ou mostra o rótulo no lugar do valor.

Pergunta diagnóstica

Quantas chamadas há no arquivo e qual valor cada uma recebe?

Pistas graduais

1. Preencha as três lacunas, uma em cada chamada. 2. O rótulo e o valor são argumentos separados por vírgula. 3. Exemplo: console.log("pontos:", 0) mostra pontos: 0.

Solução de referência explicada

Substitua as lacunas pelos literais "Ana", 0 e "fim", sem alterar o restante do arquivo.

Transferência em outro contexto

Preencha três chamadas para mostrar itens: 4, saldo: 0 e pronto, cada uma em sua própria linha; registre a saída antes de executar.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Literais de string e número em console.log; Node.js 24 LTS. | Próxima sugestão: 00.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.07 Escreva seu primeiro programa

MAKE · 7 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: execution.make | Requer: execution.fill_console | Introduz: execution.make

Eixos: novidade 1 · sintaxe 1 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

No main.js vazio, escreva três chamadas console.log: texto início, número 0 e texto fim, nessa ordem. Antes de executar, registre a saída prevista. Use uma chamada completa por linha.

Objetivo da atividade

Para produzir três mensagens, escreva três chamadas completas, uma por linha. Reaproveite a estrutura conhecida e escolha o argumento de cada uma. O programa usa a ordem em que você colocou essas linhas.

Explicação e modelo mental

• Modelo mental: Para produzir três mensagens, escreva três chamadas completas, uma por linha. Reaproveite a estrutura conhecida e escolha o argumento de cada uma. O programa usa a ordem em que você colocou essas linhas.
• Exemplo resolvido: A primeira chamada recebe o texto ligado. A segunda recebe o número 7, sem aspas. A terceira recebe o texto pronto. Você pode usar essa estrutura com outros valores.
• Armadilha comum: Não escreva só os valores sem a chamada nem troque o número 0 por "0" quando o enunciado pede um valor numérico. Conte uma chamada por saída.
• Por que importa: Escrever um programa curto do zero transforma leitura de código em produção: o aprendiz pratica a estrutura mínima que permite pedir uma saída ao computador.

Material/arquivo inicial

main.js:

Exemplo e resultado público

arquivo vazio → três chamadas console.log imprimem início, 0 e fim, cada uma na linha pedida

Bordas e erros publicados

troque os textos por "abriu" e "fechou" e mantenha o número 0 sem aspas -> abriu, 0, fechou; ainda são três chamadas completas

Teste público e aprovação visível

Resultado observável: arquivo vazio -> três chamadas console.log imprimem início, 0 e fim, cada uma na linha pedida. Borda a conferir: troque os textos por "abriu" e "fechou" e mantenha o número 0 sem aspas -> abriu, 0, fechou; ainda são três chamadas completas. Entrega esperada: Escreva três linhas: console.log("início");, console.log(0); e console.log("fim");, nessa ordem.

Restrições e documentação

MDN console.log: https://developer.mozilla.org/en-US/docs/Web/API/console/log_static; Node.js Console: https://nodejs.org/api/console.html#consolelogdata-args

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual chamada usa zero como número?

 (Correta) console.log(0);
• console.log("0");
• console.log("zero");

Por que a resposta está certa

Aspas delimitam texto. O número zero é escrito sem aspas.

Armadilha da alternativa incorreta

Releia a explicação da atividade. Aspas delimitam texto. O número zero é escrito sem aspas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

troque os textos por "abriu" e "fechou" e mantenha o número 0 sem aspas → abriu, 0, fechou; ainda são três chamadas completas

Concepção errada e falha observável

Tratar console.log como algo que só serve para exibir código pronto, ou escrever o número zero como texto sem perceber a diferença. / Falta uma chamada, a ordem não corresponde ao pedido ou o valor numérico 0 foi substituído por outro tipo de saída.

Pergunta diagnóstica

Quantas chamadas são necessárias e quais valores cada uma deve mostrar?

Pistas graduais

1. Escreva uma chamada console.log completa para cada saída pedida. 2. Use aspas nos textos; o zero pedido é o número 0, sem aspas. 3. Exemplo paralelo: console.log("ligado");, console.log(7);, console.log("pronto");.

Solução de referência explicada

Escreva três linhas: console.log("início");, console.log(0); e console.log("fim");, nessa ordem.

Transferência em outro contexto

Escreva três chamadas completas para mostrar abriu, o número 5 e fechou, nessa ordem; explique por que a chamada do meio não usa aspas.

Evidência de domínio

Resultado observável: arquivo vazio -> três chamadas console.log imprimem início, 0 e fim, cada uma na linha pedida. Borda a conferir: troque os textos por "abriu" e "fechou" e mantenha o número 0 sem aspas -> abriu, 0, fechou; ainda são três chamadas completas. Entrega esperada: Escreva três linhas: console.log("início");, console.log(0); e console.log("fim");, nessa ordem. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: console.log com texto e número, no Node.js 24 LTS. | Próxima sugestão: 00.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.05 Descubra quem fornece cada recurso

READ · 8 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: environment.boundaries | Requer: environment.boundaries | Introduz: environment.boundaries

Eixos: novidade 2 · sintaxe 1 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Classifique Array, document, process, node:fs, console, fetch, npm e TypeScript na tabela de resposta. Use as explicações acima e consulte as referências para conferir console e fetch. Marque ambas as colunas de ambiente quando aplicável. Não escreva chamadas JavaScript.

Objetivo da atividade

Separe a origem do recurso de sua disponibilidade. Um recurso do padrão JavaScript, como Array, acompanha a linguagem. Outros são oferecidos pelo ambiente. Para conferir disponibilidade, a documentação precisa corresponder ao ambiente e à versão usados.

Explicação e modelo mental

• Modelo mental: Separe a origem do recurso de sua disponibilidade. Um recurso do padrão JavaScript, como Array, acompanha a linguagem. Outros são oferecidos pelo ambiente. Para conferir disponibilidade, a documentação precisa corresponder ao ambiente e à versão usados.
• Exemplo resolvido: Math reúne operações matemáticas padrão; não depende de uma página. O contexto entre parênteses indica qual documentação está sendo usada. Um mesmo nome pode ter outra implementação em outro ambiente. — significa que você não marcou aquela categoria de origem; não é código nem afirmação de que o nome jamais existe ali.
• Armadilha comum: Não force cada nome em uma categoria exclusiva. Também não trate fs como global: a API de arquivos é importada do módulo node:fs.
• Por que importa: Separar padrão da linguagem, APIs de host e ferramentas evita usar um recurso no ambiente errado ou atribuí-lo à camada errada.

Material/arquivo inicial

material.md: Array | document | process | node:fs | console | fetch | npm | TypeScript | resposta.md: Nome | Padrão JavaScript | Navegador | Node.js 24 | Ferramenta/linguagem externa
Array | | | |
document | | | |
process | | | |
node:fs | | | |
console | | | |
fetch | | | |
npm | | | |
TypeScript | | | |

Exemplo e resultado público

Array → ECMAScript padrão; document -> navegador; process e node:fs -> Node.js; console e fetch -> navegador e Node.js 24; npm e TypeScript -> ecossistema/ferramenta externa

Bordas e erros publicados

cenário Browser + Node.js 24 -> fetch e console têm marca nas duas colunas; node:fs fica na coluna Node como módulo, não como global fs

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

MDN standard built-in objects: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects; Node.js globals: https://nodejs.org/api/globals.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

A documentação confirma console e fetch no navegador e no Node.js 24. Como marcar a tabela?

• Marcar somente navegador.
 (Correta) Marcar os dois ambientes para ambos os nomes.
• Marcar somente ferramenta externa.

Por que a resposta está certa

Um mesmo recurso pode existir em dois ambientes. A consulta à versão correta confirma isso.

Armadilha da alternativa incorreta

Releia a explicação da atividade. Um mesmo recurso pode existir em dois ambientes. A consulta à versão correta confirma isso.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

cenário Browser + Node.js 24 → fetch e console têm marca nas duas colunas; node:fs fica na coluna Node como módulo, não como global fs

Concepção errada e falha observável

Tratar a presença de uma API no ambiente como se fosse sintaxe ECMAScript. / O aluno fixa fetch em apenas um host ou classifica npm como linguagem.

Pergunta diagnóstica

Em quais colunas cada nome deve receber uma marca, e quais linhas pertencem a mais de um host?

Pistas graduais

1. Marque primeiro os built-ins do ECMAScript. 2. Depois preencha Browser e Node.js 24 separadamente; não force APIs compartilhadas em uma única categoria. 3. fetch e console podem receber marca nas duas colunas de host.

Solução de referência explicada

Array é padrão ECMAScript; document é Browser; process e node:fs são Node; console e fetch estão nos dois hosts; npm e TypeScript pertencem ao ecossistema/ferramenta externa.

Transferência em outro contexto

Classifique setTimeout em Browser e Node.js 24 usando duas marcas de host; explique por que uma matriz representa melhor sua disponibilidade do que uma categoria exclusiva.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Distinção entre linguagem (Array, console), ambiente (document, fetch, process, node:fs) e ferramentas (npm, TypeScript); conceitos sem API nova, verificados no Node.js 24 LTS e no navegador. | Próxima sugestão: 00.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.06 Investigue um erro de cada vez

DEBUG · 9 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: debug.first_error | Requer: execution.order, execution.fill_console | Introduz: debug.first_error

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 3 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

A saída final deve ser Olá, fim e ok, nessa ordem. Execute primeiro o arquivo intacto. Registre a primeira falha; feche a aspa ausente e execute de novo. Depois investigue o nome consle e, por último, a mensagem diferente do alvo. Faça uma alteração por vez.

Objetivo da atividade

Comece executando o arquivo intacto. Registre o primeiro erro antes de editar. Corrija uma causa e execute novamente. Só investigue a falha seguinte quando a anterior tiver desaparecido. No final compare a saída com o alvo.

Explicação e modelo mental

• Modelo mental: Comece executando o arquivo intacto. Registre o primeiro erro antes de editar. Corrija uma causa e execute novamente. Só investigue a falha seguinte quando a anterior tiver desaparecido. No final compare a saída com o alvo.
• Exemplo resolvido: Se a primeira linha fosse console.log("porta);, a aspa faltante causaria SyntaxError e nenhuma saída apareceria. Depois de fechar a aspa, uma linha com consle.log("pronto"); causaria ReferenceError: o nome correto é console. Se a linha final mostrasse "errado" em vez de "pronto", não haveria exceção, mas o resultado ainda estaria diferente do alvo.
• Armadilha comum: Não comece por uma chamada posterior: enquanto o parse falha, nenhuma linha do arquivo executa. Corrija uma causa, rode novamente e use o próximo erro como nova evidência.
• Por que importa: Separar falha de sintaxe, falha durante execução e resultado incorreto sem exceção ajuda a escolher a próxima ação e evita declarar sucesso só porque o programa terminou.

Material/arquivo inicial

main.js: console.log("Olá);
consle.log("fim");
console.log("erro");

Exemplo e resultado público

arquivo intacto → SyntaxError; após fechar aspas -> "Olá" aparece e a linha consle gera ReferenceError; após corrigir consle -> Olá, fim, erro, sem exceção; trocar "erro" por "ok" -> Olá, fim, ok

Bordas e erros publicados

com as três linhas corrigidas -> Olá, fim, ok; a versão com a palavra erro roda sem exceção, mas mostra saída diferente da pedida

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

MDN JavaScript errors: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O programa terminou sem mensagem de erro, mas mostrou uma palavra diferente da pedida. O que fazer?

• Concluir: terminar sem erro garante a resposta correta.
• Apagar todas as chamadas.
 (Correta) Comparar com o resultado esperado e corrigir a mensagem.

Por que a resposta está certa

Uma execução sem erro pode ter resultado incorreto. Compare também a saída com o pedido.

Armadilha da alternativa incorreta

Releia a explicação da atividade. Uma execução sem erro pode ter resultado incorreto. Compare também a saída com o pedido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

com as três linhas corrigidas → Olá, fim, ok; a versão com a palavra erro roda sem exceção, mas mostra saída diferente da pedida

Concepção errada e falha observável

Tentar corrigir uma linha posterior antes de remover a primeira barreira de sintaxe. / A execução para na primeira falha ou termina sem exceção com uma saída diferente do contrato.

Pergunta diagnóstica

Depois de corrigir a primeira falha, qual é o próximo comportamento observável?

Pistas graduais

1. Comece pelo primeiro erro mostrado pelo runtime. 2. Feche a string antes de investigar a próxima linha. 3. Depois de corrigir consle, compare o texto impresso com o alvo: código válido também pode mostrar a saída errada.

Solução de referência explicada

Feche "Olá", troque consle por console e execute; depois mude o literal "erro" para "ok" e confirme que a saída corresponde ao alvo.

Transferência em outro contexto

Classifique três observações em outro programa: string sem fechamento; nome inexistente chamado durante a execução; execução normal que imprime aguardando quando o alvo é concluído.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Erro de sintaxe com aspas e leitura da mensagem do Node.js 24 LTS; nenhuma API recente. | Próxima sugestão: 00.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.08 Leia seu primeiro formulário HTML

READ · 8 min (estimativa, sem nota por velocidade) · Leitura no navegador, sem execução · dificuldade multidimensional 2.0/5

Competência: html.tree_reading | Requer: html.tree_reading | Introduz: html.tree_reading

Eixos: novidade 2 · sintaxe 1 · composição 2 · estado 1 · depuração 1 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Leia index.html sem alterá-lo. Na resposta, desenhe a árvore pai/filhos, identifique os valores de id e class e diga a qual input o label se conecta pelo atributo for. Use os atributos como evidência.

Objetivo da atividade

Comece pela tag externa e acompanhe até seu fechamento. As tags internas mostram os filhos. Depois leia os atributos: compare for do label com id do input para encontrar a associação entre rótulo e campo.

Explicação e modelo mental

• Modelo mental: Comece pela tag externa e acompanhe até seu fechamento. As tags internas mostram os filhos. Depois leia os atributos: compare for do label com id do input para encontrar a associação entre rótulo e campo.
• Exemplo resolvido: form é o pai direto de label, input e button; os três são irmãos. for="postal" e id="postal" têm o mesmo valor: o rótulo CEP corresponde ao campo. campo-form é a classe do input. O botão tem texto e tag de fechamento; input não precisa de fechamento.
• Armadilha comum: Não associe for à classe nem trate class como identificador único. Leia abertura/fechamento para reconstruir a árvore antes de mapear atributos.
• Por que importa: Ler relações semânticas do HTML permite criar formulários que tecnologias assistivas conseguem associar e depurar seletores sem confundir identidade e agrupamento.

Material/arquivo inicial

index.html: <form id="entrada">
<label for="nome">Nome</label>
<input id="nome" class="campo">
<button>Enviar</button>
</form> | resposta.md: Árvore pai/filhos:

Valores de id:
Valor de for:
Valor de class:
Associação entre rótulo e campo:

Exemplo e resultado público

pai de label/input/button → form; label associado ao input -> for=nome e id=nome; id do form -> entrada; classe do input -> campo

Bordas e erros publicados

adicione segundo input id=idade -> label nome continua associado somente ao input nome

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

MDN label element: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/label; input element: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Um label tem for="email". Qual atributo do input faz a associação?

 (Correta) id="email"
• class="email"
• id="outro"

Por que a resposta está certa

Na associação explícita, o valor de for corresponde ao id do campo.

Armadilha da alternativa incorreta

Releia a explicação da atividade. Na associação explícita, o valor de for corresponde ao id do campo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

adicione segundo input id=idade → label nome continua associado somente ao input nome

Concepção errada e falha observável

Confundir classe compartilhável com ID e ignorar hierarquia pai/filho. / O mapa de árvore coloca label fora de form ou associa for ao elemento errado.

Pergunta diagnóstica

Qual tag abre o elemento pai, e qual id corresponde ao for do label?

Pistas graduais

1. Marque tags de abertura e fechamento. 2. for e id devem ter o mesmo valor. 3. Dois elementos podem ter class campo, mas o id nome deve ser único.

Solução de referência explicada

form contém label, input e button. for=nome aponta ao input id=nome; class=campo é metadado de seleção.

Transferência em outro contexto

Num formulário de reserva, o input de quantidade tem id="quantidade". Desenhe a relação pai/filho e indique qual valor deve ligar o label a esse input pelo atributo for.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: HTML5 padrão e árvore de elementos (label, input, id, class); leitura sem script, em qualquer navegador evergreen. | Próxima sugestão: 00.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

00.09 Integre as habilidades em um programa curto

TRACE · 8 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: execution.explain_program | Requer: execution.order, execution.sequence | Introduz: execution.explain_program

Eixos: novidade 1 · sintaxe 1 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

No main.js vazio, escreva chamadas que mostrem uma abertura, a pessoa Ana, a quantidade 3 e um encerramento. Preveja, execute e compare. Depois aceite a nova exigência de quantidade 0, altere o programa e execute outra vez. Por fim responda aos dois desafios de transferência.

Objetivo da atividade

Planeje a saída pedida, escreva chamadas completas e confira a execução. Quando um requisito mudar, altere o programa e compare de novo. console.log mostra texto; não realiza a ação descrita nele.

Explicação e modelo mental

• Modelo mental: Planeje a saída pedida, escreva chamadas completas e confira a execução. Quando um requisito mudar, altere o programa e compare de novo. console.log mostra texto; não realiza a ação descrita nele.
• Exemplo resolvido: Linha 1 → carregar; linha 2 → 11; linha 3 → pronto. Sem a chamada da linha 2, a saída seria carregar e pronto, em duas linhas. pronto passa a ser a segunda saída, mas sua chamada continua sendo a que era a terceira instrução no arquivo original.
• Armadilha comum: Uma explicação geral como "o arquivo mostra o estado" não basta: conecte cada linha à sua própria saída e não atribua à remoção um efeito em comandos sem dependência.
• Por que importa: Documentar a relação entre linha e efeito é a base de revisão de código e investigação. Explicações verificáveis ajudam outra pessoa a avaliar uma alteração pequena sem adivinhar a intenção.

Material/arquivo inicial

main.js:

Exemplo e resultado público

arquivo completo → abrir, 3, fechar; removida segunda linha -> abrir, fechar sem erro; apenas o log da linha removida desaparece

Bordas e erros publicados

removida primeira linha -> 3, fechar; removida terceira -> abrir, 3; em ambos os casos as chamadas restantes executam sem erro

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

MDN console.log: https://developer.mozilla.org/en-US/docs/Web/API/console/log_static; Node.js Console: https://nodejs.org/api/console.html#consolelogdata-args

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Um programa mostra começar, 8 e terminar. Se remover só a chamada do meio, o que resta?

• 8 e terminar.
 (Correta) começar e terminar, nessa ordem.
• terminar e começar, nessa ordem.

Por que a resposta está certa

A retirada elimina somente a saída da chamada removida. As outras mantêm a ordem.

Armadilha da alternativa incorreta

Releia a explicação da atividade. A retirada elimina somente a saída da chamada removida. As outras mantêm a ordem.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

removida primeira linha → 3, fechar; removida terceira -> abrir, 3; em ambos os casos as chamadas restantes executam sem erro

Concepção errada e falha observável

Descrever intenção geral sem ligar cada saída à linha que a produz. / Explicação não prevê corretamente o efeito de remover uma chamada.

Pergunta diagnóstica

Que saída pertence exclusivamente à linha removida?

Pistas graduais

1. Anote uma saída ao lado de cada linha. 2. Remover uma chamada remove sua saída, não reordena as demais. 3. No programa A/B/C, remover B deixa A/C.

Solução de referência explicada

Explique cada chamada; as saídas são abrir, 3, fechar. Remover qualquer linha elimina apenas sua própria saída.

Transferência em outro contexto

Documente um arquivo com logs de etapas de backup e explique o que muda na saída se a chamada que confirma a cópia for removida.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: console.log com variáveis const e let no Node.js 24 LTS; sem APIs além da saída padrão. | Próxima sugestão: 01.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01 Valores e tipos

9 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 01.01 · 01.02 · 01.03 · 01.04 · 01.05 · 01.06 · 01.07 · 01.08 · 01.09

01.01 Classifique valores

PREDICT · 8 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: primitives.typeof | Requer: execution.order | Introduz: primitives.typeof

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Preveja cinco rótulos antes de executar e explique apenas a diferença entre valor e rótulo.

Objetivo da atividade

Cinco rótulos, um por chamada, antes do Run.

Explicação e modelo mental

• Modelo mental: typeof recebe um valor e devolve uma string com seu rótulo.
• Exemplo resolvido: typeof false produz boolean; typeof "oi" produz string.
• Armadilha comum: Não responda com o valor original; null é uma peculiaridade documentada.
• Por que importa: Classificar valores ajuda a prever o comportamento de APIs.

Material/arquivo inicial

main.js: console.log(typeof 42);
console.log(typeof "42");
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof null);

Exemplo e resultado público

typeof 42, typeof "42", typeof true, typeof undefined, typeof null → number, string, boolean, undefined, object

Bordas e erros publicados

typeof "azul", typeof 7, typeof false -> string, number, boolean; null continua retornando object

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual rótulo typeof null retorna?

• "null"
• "undefined"
 (Correta) "object"

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

typeof "azul", typeof 7, typeof false → string, number, boolean; null continua retornando object

Concepção errada e falha observável

Confundir aspas, valor e rótulo retornado. / Uma previsão não corresponde à string devolvida por typeof.

Pergunta diagnóstica

Qual rótulo cada chamada devolve?

Pistas graduais

1. Separe valor e rótulo. 2. Aspas fazem texto; sem aspas, 42 é número. 3. null é a peculiaridade histórica que retorna object.

Solução de referência explicada

typeof devolve strings: number, string, boolean, undefined e object para as cinco chamadas.

Transferência em outro contexto

Classifique três novas chamadas typeof -> resposta conforme a regra demonstrada

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.02 Rastreie bindings

TRACE · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.reassignment | Requer: bindings.values | Introduz: bindings.reassignment

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Preencha o valor atual de base e total depois de cada instrução.

Objetivo da atividade

Tabela de quatro estados antes do Run.

Explicação e modelo mental

• Modelo mental: Cada binding tem seu valor atual; inicializar com outro binding copia o valor daquele momento.
• Exemplo resolvido: const limite=4; let contador=limite; contador=contador+1 termina com 4 e 5.
• Armadilha comum: Antes da declaração, descreva o binding como ainda não inicializado; a causa é a ordem temporal.
• Por que importa: Rastrear estado por nome sustenta depuração.

Material/arquivo inicial

material.md: const base = 2;
let total = base;
total = total + 3;
console.log(base, total);

Exemplo e resultado público

após base → base=2,total não inicializado; após total=base -> base=2,total=2; após reatribuição -> base=2,total=5; saída -> 2 5

Bordas e erros publicados

const taxa=2; let custo=taxa; custo=custo+4 -> taxa=2,custo=2 e depois taxa=2,custo=6

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que muda em total=total+3?

 (Correta) Somente total
• base e total
• Nenhum binding

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

const taxa=2; let custo=taxa; custo=custo+4 → taxa=2,custo=2 e depois taxa=2,custo=6

Concepção errada e falha observável

Tratar inicialização como ligação permanente ou chamar um binding antes da inicialização de inexistente. / A tabela não acompanha o valor atual de total.

Pergunta diagnóstica

Qual binding muda na última atribuição?

Pistas graduais

1. Uma linha por instrução. 2. Leia o lado direito primeiro. 3. Atualize só o nome à esquerda.

Solução de referência explicada

base termina em 2 e total em 5; antes da declaração, total ainda não foi inicializado.

Transferência em outro contexto

Rastreie um custo que recebe duas reatribuições. -> resposta conforme a regra demonstrada

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.03 Organize dependências

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.declaration_order | Requer: bindings.reassignment | Introduz: bindings.declaration_order

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Registre duas ordens válidas e explique a dependência real.

Objetivo da atividade

Duas ordens válidas antes de mover blocos.

Explicação e modelo mental

• Modelo mental: Uma leitura precisa ocorrer depois da inicialização do nome lido.
• Exemplo resolvido: Duas declarações que não se usam podem trocar de lugar antes do log que lê ambas.
• Armadilha comum: Não invente dependência entre produtores independentes.
• Por que importa: Dependências reais orientam a ordem do código.

Material/arquivo inicial

material.md: E=console.log("diagnóstico"); C=const ativo=true; D=console.log(nome, idade, ativo); B=const idade=20; A=const nome="Lia"

Exemplo e resultado público

A,B,C,D ou qualquer ordem de A/B/C antes de D → Lia 20 true; E fica fora

Bordas e erros publicados

D antes de uma declaração que ele consulta causa erro de leitura antes da inicialização -> saída conforme a regra demonstrada

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que limita a posição de D?

• A ordem alfabética
 (Correta) As três declarações que D lê
• A obrigação de usar E

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

D antes de uma declaração que ele consulta causa erro de leitura antes da inicialização → saída conforme a regra demonstrada

Concepção errada e falha observável

Forçar uma ordem única entre declarações independentes. / O consumidor aparece antes de uma declaração necessária ou E entra na resposta.

Pergunta diagnóstica

Quais nomes D consulta?

Pistas graduais

1. Liste os nomes lidos. 2. Produtores precedem o consumidor. 3. A, B e C são independentes.

Solução de referência explicada

A, B e C podem trocar de ordem, desde que venham antes de D; E fica fora.

Transferência em outro contexto

Organize três declarações antes de um resumo. -> resposta conforme a regra demonstrada

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.04 Escolha const ou let

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.const_let_choice | Requer: bindings.reassignment | Introduz: bindings.const_let_choice

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Complete seis declarações pela convenção const quando não há reatribuição e let quando há.

Objetivo da atividade

Seis escolhas antes de completar o código.

Explicação e modelo mental

• Modelo mental: const protege o binding contra reatribuição; let permite outra atribuição.
• Exemplo resolvido: const sala=0; let ocupacao=0; ocupacao=3 usa const para o fixo e let para o que muda.
• Armadilha comum: O valor inicial não decide; a mudança prevista decide.
• Por que importa: A declaração comunica o estado que o programa terá.

Material/arquivo inicial

material.md: ___ nome="Ana";
___ pontos=0;
pontos=5;
___ ativo=false;
___ volume=0;
volume=7;
___ idioma="pt-BR";
___ tentativas=1;
tentativas=2;
console.log(nome, pontos, ativo, volume, idioma, tentativas);

Exemplo e resultado público

nome,ativo,idioma → const; pontos,volume,tentativas -> let; valores finais Ana,5,false,7,pt-BR,2

Bordas e erros publicados

Zero e false permanecem valores válidos; a escolha segue reatribuição prevista -> saída conforme a regra demonstrada

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual binding precisa de let?

• legendas, porque é false
• idioma, porque é texto
 (Correta) volume, porque muda

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Zero e false permanecem valores válidos; a escolha segue reatribuição prevista → saída conforme a regra demonstrada

Concepção errada e falha observável

Escolher pela aparência do valor inicial. / Uma reatribuição ocorre em const ou a convenção não é aplicada.

Pergunta diagnóstica

Quais nomes recebem nova atribuição?

Pistas graduais

1. Marque as três reatribuições. 2. Use let nesses nomes. 3. Zero e false não mudam a decisão.

Solução de referência explicada

Use let nos três bindings reatribuídos e const nos demais.

Transferência em outro contexto

Escolha const/let para uma agenda. -> resposta conforme a regra demonstrada

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.05 Classifique valores especiais

PREDICT · 8 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: primitives.special_values | Requer: primitives.typeof | Introduz: primitives.special_values

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Preveja quatro rótulos; operações numéricas ficam para a Etapa 02.

Objetivo da atividade

Quatro rótulos antes do Run.

Explicação e modelo mental

• Modelo mental: typeof classifica o valor recebido; não cria tipos pelo nome escrito.
• Exemplo resolvido: typeof NaN é number; typeof 1n é bigint.
• Armadilha comum: Não misture classificação com operações que produzem valores especiais.
• Por que importa: Separar tipo e comportamento evita sobrecarga de conceitos.

Material/arquivo inicial

material.md: console.log(typeof NaN);
console.log(typeof Infinity);
console.log(typeof 1n);
console.log(typeof null);

Exemplo e resultado público

NaN, Infinity, 1n, null → number, number, bigint, object

Bordas e erros publicados

-0, 5n, -Infinity -> number, bigint, number

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

typeof NaN?

 (Correta) "number"
• "NaN"
• "undefined"

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

-0, 5n, -Infinity → number, bigint, number

Concepção errada e falha observável

Supor que NaN seja um tipo separado ou que bigint seja apenas number. / Um rótulo é escolhido pela grafia do nome, não por typeof.

Pergunta diagnóstica

Qual rótulo typeof devolve?

Pistas graduais

1. Classifique somente. 2. NaN pertence a number. 3. n indica bigint.

Solução de referência explicada

NaN e Infinity são number; 1n é bigint; null retorna object.

Transferência em outro contexto

Classifique dois novos valores. -> resposta conforme a regra demonstrada

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.06 Complete uma template literal

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.template_literal | Requer: bindings.values | Introduz: bindings.template_literal

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Preveja e complete a única expressão de saída.

Objetivo da atividade

Uma linha prevista antes de editar.

Explicação e modelo mental

• Modelo mental: Crases delimitam a string e ${...} insere o valor avaliado.
• Exemplo resolvido: codigo=R7 e estado=pronto formam R7 — pronto.
• Armadilha comum: Aspas comuns não interpolam.
• Por que importa: Strings derivadas acompanham os dados atuais.

Material/arquivo inicial

material.md: const nome="Nara";
const nivel=3;
console.log(/* uma expressão */);

Exemplo e resultado público

nome=Nara,nivel=3 → Nara: nível 3

Bordas e erros publicados

nome=Ivo,nivel=1 -> Ivo: nível 1

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como inserir nome?

• "${nome}"
 (Correta) ${nome} entre crases
• copiar Nara

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

nome=Ivo,nivel=1 → Ivo: nível 1

Concepção errada e falha observável

Usar aspas comuns ou copiar os valores em vez de interpolar bindings. / A saída mostra ${nome} literalmente ou não forma uma string.

Pergunta diagnóstica

Que string os bindings atuais devem formar?

Pistas graduais

1. Preveja antes de editar. 2. Use crases. 3. Interpole nome e nivel.

Solução de referência explicada

Use uma única expressão ` ${nome}: nível ${nivel} `; o contrato é Nara: nível 3.

Transferência em outro contexto

Monte uma expressão textual com dois bindings. -> resposta conforme a regra demonstrada

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.07 Escreva um resumo

MAKE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.template_literal | Requer: bindings.template_literal | Introduz: bindings.template_literal

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Crie do zero a declaração do resumo e sua impressão.

Objetivo da atividade

Saída prevista e plano de duas instruções.

Explicação e modelo mental

• Modelo mental: MAKE pede produzir as instruções derivadas dos dados fornecidos.
• Exemplo resolvido: Com dias=2 e horas=3, ${dias}d${horas}h produz 2d3h.
• Armadilha comum: Não cole a saída como texto fixo.
• Por que importa: Produzir uma string curta consolida bindings e expressão.

Material/arquivo inicial

material.md: const horas=2;
const minutos=15;

Exemplo e resultado público

horas=2,minutos=15 → declara resumo com ${horas}h${minutos} e imprime 2h15

Bordas e erros publicados

horas=4,minutos=0 -> 4h0

Teste público e aprovação visível

Critérios desta atividade: Declarar e imprimir resumo derivado de horas e minutos. Preservar zero nos dois campos.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que falta?

• Somente mudar horas
• Somente escrever uma string fixa
 (Correta) Declarar e imprimir resumo

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

horas=4,minutos=0 → 4h0

Concepção errada e falha observável

Continuar tratando como lacuna em vez de escrever o programa pedido. / Falta a declaração, a impressão ou os bindings não são usados.

Pergunta diagnóstica

Quais duas instruções faltam?

Pistas graduais

1. Crie resumo. 2. Leia horas e minutos. 3. Passe resumo a console.log.

Solução de referência explicada

Declare resumo com template literal e execute console.log(resumo); zero é preservado.

Transferência em outro contexto

Crie e imprima duração de dias e horas. -> resposta conforme a regra demonstrada

Evidência de domínio

Critérios desta atividade: Declarar e imprimir resumo derivado de horas e minutos. Preservar zero nos dois campos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.08 Transfira const e let para configuração

TRANSFER · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.configuration | Requer: bindings.const_let_choice, bindings.template_literal | Introduz: bindings.configuration

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Complete as cinco declarações e produza os dois resumos exatos.

Objetivo da atividade

Tabela de cinco decisões e duas linhas previstas.

Explicação e modelo mental

• Modelo mental: A configuração combina bindings estáveis com um valor ajustável.
• Exemplo resolvido: Arena com volume 0 e depois 7 preserva legendas false nos dois resumos.
• Armadilha comum: Não substitua escolhas explícitas por outros valores.
• Por que importa: Configurações precisam preservar preferências válidas entre estados.

Material/arquivo inicial

material.md: ___ jogo="Arena";
___ volume=0;
___ dificuldade="normal";
___ legendas=false;
___ idioma="pt-BR";
// resumo; volume=7; // segundo resumo

Exemplo e resultado público

Arena | volume 0 | normal | legendas false | pt-BR; depois Arena | volume 7 | normal | legendas false | pt-BR → saída conforme o contrato publicado

Bordas e erros publicados

agenda: duração 0 -> duração 3, mantendo lembretes false

Teste público e aprovação visível

Compare competência estrutural em domínio diferente e sem ajuda.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual binding usa let?

 (Correta) volume, porque recebe 7
• legendas, porque é false
• idioma, porque é texto

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

agenda: duração 0 → duração 3, mantendo lembretes false

Concepção errada e falha observável

Tratar 0 ou false como ausência e escolher let/const sem olhar reatribuição. / Um resumo perde 0/false ou não muda apenas volume.

Pergunta diagnóstica

Qual binding realmente muda?

Pistas graduais

1. Marque volume. 2. Os demais podem ser const. 3. Preserve os valores publicados.

Solução de referência explicada

Use let em volume e const nos quatro fixos; derive os dois resumos dos bindings atuais.

Transferência em outro contexto

Modele e atualize uma configuração de agenda. -> resposta conforme a regra demonstrada

Evidência de domínio

Compare competência estrutural em domínio diferente e sem ajuda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 01.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

01.09 Projeto: ficha em três estados

PROJECT · 16 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: bindings.textual_profile | Requer: bindings.reassignment, bindings.const_let_choice, bindings.template_literal | Introduz: bindings.textual_profile

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 2 · depuração 1 · arquivos 1 · leitura 1 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Complete o arquivo e produza exatamente as três linhas na ordem publicada.

Objetivo da atividade

Três linhas previstas antes da primeira edição.

Explicação e modelo mental

• Modelo mental: Cada execução da expressão cria um snapshot textual do estado atual.
• Exemplo resolvido: Lume | energia 8 e depois Lume | energia 3 mostram a mesma expressão reavaliada.
• Armadilha comum: Não fixe vida no texto e não omita vida=0.
• Por que importa: Uma ficha curta integra binding, reatribuição e apresentação.

Material/arquivo inicial

material.md: const nome="Iris";
const nivel=1;
let vida=10;
const status="ativa";
// três checkpoints

Exemplo e resultado público

Iris | nível 1 | vida 10 | ativa; Iris | nível 1 | vida 6 | ativa; Iris | nível 1 | vida 0 | ativa → saída conforme o contrato publicado

Bordas e erros publicados

modelo=Lume, energia=8 -> Lume | energia 8; energia=3 -> Lume | energia 3

Teste público e aprovação visível

Critérios desta atividade: Imprimir três fichas atuais, incluindo vida zero. Resolver as duas transferências sem dicas e explicar as mudanças.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que vida usa let?

• Porque todo número usa let
□ (Correta) Porque recebe 6 e 0
• Porque zero não pode estar em const

Por que a resposta está certa

Correto: a resposta acompanha o binding e a execução descrita.

Armadilha da alternativa incorreta

Releia a explicação e acompanhe o valor atual na ordem das instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

modelo=Lume, energia=8 → Lume | energia 8; energia=3 -> Lume | energia 3

Concepção errada e falha observável

Copiar uma ficha literal, não reatribuir vida ou apagar o zero. / Uma linha não corresponde ao valor atual ou omite o zero.

Pergunta diagnóstica

Qual binding muda nos três checkpoints?

Pistas graduais

1. Use os quatro bindings. 2. Altere somente vida. 3. Imprima o zero como valor atual.

Solução de referência explicada

Use uma template literal, imprima vida=10, reatribua 6, imprima, reatribua 0 e imprima. O contrato tem exatamente três linhas.

Transferência em outro contexto

Faça uma ficha de veículo com energia 8, 3 e 0. -> resposta conforme a regra demonstrada

Evidência de domínio

Critérios desta atividade: Imprimir três fichas atuais, incluindo vida zero. Resolver as duas transferências sem dicas e explicar as mudanças. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JavaScript introdutório: bindings, primitivos, typeof e template literals | Próxima sugestão: 02.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02 Operadores, Math e Number

12 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 02.01 · 02.02 · 02.03 · 02.04 · 02.05 · 02.06 · 02.07 · 02.08 · 02.09 · 02.10 · 02.11 · 02.12

02.01 Expressões por família: aritmética, comparação e lógica

PREDICT · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: operators.predict | Requer: bindings.values | Introduz: operators.predict, operators.arithmetic, operators.comparison, operators.logical, operators.basic

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 1 · depuração 1 · arquivos 1 · leitura 4 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Resolva primeiro o bloco A (+ - * / % ** e parênteses), registre valores e tipos e confira antes de avançar. Depois resolva B (=== > >= ! && || e concatenação). Por último compare 0 === "0" com 0 == "0" e teste também !==: explique a coerção de == e por que a trilha prefere ===. O operador ?? entra em 02.02.

Objetivo da atividade

Duas listas separadas (A e B) de valor e tipo, registradas antes de executar cada bloco; depois a comparação curta entre ==, === e !==.

Explicação e modelo mental

• Modelo mental: Uma expressão combina valores e produz outro valor. + soma, - subtrai, * multiplica, / divide, % encontra o resto e ** calcula potência. Resolva parênteses, depois potências, multiplicação/divisão/resto e, por último, soma/subtração. Comparações devolvem booleanos. > significa maior, >= inclui igualdade, < significa menor e <= inclui igualdade. Truthiness é a forma como um valor é tratado numa condição: 0 é falsy; 2 e textos não vazios são truthy. ! inverte essa condição. && exige os dois lados quando eles são booleanos; || aceita qualquer um. === exige o mesmo tipo e o mesmo valor; !== responde o oposto dessa comparação. == pode converter tipos, e por isso exige cuidado ao ler código antigo.
• Exemplo resolvido: console.log(5 + 2 * 3);
console.log((5 + 2) * 3);
console.log(7 % 3, 3 ** 2);
2 * 3 vale 6; 5 + 6 vale 11. Com parênteses, 5 + 2 vale 7; 7 * 3 vale 21. 7 dividido por 3 deixa resto 1. 3 ** 2 é 3 vezes 3, portanto 9. Cada resultado tem tipo number.

console.log(7 >= 7, !2);
console.log(true && "ok", false || "reserva");
console.log("4" + 1);
console.log(false && nomeNaoDeclarado, true || nomeNaoDeclarado);
7 >= 7 é true; 2 é truthy e !2 é false. && retorna o primeiro falsy ou o último operando. || retorna o primeiro truthy ou o último operando. Assim true && "ok" retorna uma string e false || "reserva" também. Com uma string, + concatena: "4" + 1 produz "41", não 5. Curto-circuito: nas duas últimas expressões, o primeiro lado já decide o resultado. Ler nomeNaoDeclarado causaria erro, mas o lado direito é pulado. Execute o exemplo: ele produz false e true sem erro. Se o primeiro lado de && fosse true, o nome seria lido e ocorreria erro.

console.log(1 === "1", 1 == "1");
console.log(1 !== "1", 2 !== 2);
O número 1 e a string "1" têm tipos diferentes: === dá false, mas == faz coerção e dá true. !== não converte os tipos: 1 !== "1" é true; 2 !== 2 é false. Os quatro resultados têm tipo boolean.
• Armadilha comum: Não resolva tudo da esquerda para a direita. % devolve o resto, não uma porcentagem. && e || nem sempre devolvem booleanos. = atribui; === compara. Prefira === e !==. Aprender este exemplo não significa conhecer todas as regras de ==.
• Por que importa: Separar cálculo, comparação, lógica e concatenação permite escrever e ler condições sem confundir valor com tipo.

Material/arquivo inicial

material.md: Bloco A — aritmética e precedência: 2+3*4; (2+3)*4; 9-4; 6%4; 8/2; 2**3.
Bloco B — comparação e lógica: 0===false; 5>3; 5>=5; !0; true&&false; false||"x"; "2"+3.
COMPARE posterior aos dois blocos: 0==="0"; 0=="0"; 0!=="0"; 3!==3.

Exemplo e resultado público

Bloco A → 14 (number), 20 (number), 5 (number), 2 (number), 4 (number), 8 (number); Bloco B -> false (boolean), true (boolean), true (boolean), true (boolean), false (boolean), "x" (string), "23" (string); COMPARE -> false, true, true, false

Bordas e erros publicados

3+1*2 -> 5 (number); false&&"x" -> false (boolean); true&&"x" -> "x" (string)

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

MDN operator precedence: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_precedence; nullish coalescing: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que valor e tipo false || "x" e "2" + 3 devolvem?

• true e 5, ambos convertidos em números.
• false e 23, porque + sempre soma numericamente.
 (Correta) "x" e "23", ambos strings.

Por que a resposta está certa

|| devolve um operando; + concatena quando há string. Aspas deixam o tipo explícito.

Armadilha da alternativa incorreta

Anote valor e tipo. O fallback pode devolver string e + concatena texto.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

3+1*2 → 5 (number); false&&"x" -> false (boolean); true&&"x" -> "x" (string)

Concepção errada e falha observável

Misturar precedência, igualdade estrita e fallback em uma regra única. / Primeira divergência indica se falhou aritmética, comparação ou nullish.

Pergunta diagnóstica

Qual família de operadores você está resolvendo e qual valor e tipo ela devolve?

Pistas graduais

1. Resolva uma família por vez; parênteses mudam a precedência. 2. && e || devolvem operandos; anote também o tipo. 3. Em 1==="1" o resultado é false; 1=="1" é true por coerção. Prefira === na trilha.

Solução de referência explicada

A: 14,20,5,2,4,8. B: false,true,true,true,false,"x","23". COMPARE: false,true,true,false. === não converte tipos; == aplica regras adicionais de coerção e só deve ser usado com intenção explícita. ?? será praticado em 02.02.

Transferência em outro contexto

Preveja (3 + 1) * 2 e true && "ok"; registre valor e tipo sem executar.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Operadores aritméticos, de comparação e lógicos (**, ===, &&) da linguagem ECMAScript 2026; sem API nova. | Próxima sugestão: 02.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.02 Matriz falsy/nullish

PREDICT · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: operators.falsy_nullish | Requer: bindings.values, operators.logical | Introduz: operators.falsy_nullish

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 4 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Preencha a matriz para cada valor sob x || "padrão" e x ?? "padrão". Antes de conferir, marque quais valores são falsy e quais são nullish e explique onde os operadores divergem.

Objetivo da atividade

Uma matriz com duas saídas por valor e colunas separadas para truthy/falsy e nullish/não-nullish.

Explicação e modelo mental

• Modelo mental: Falsy significa tratado como falso numa condição; nullish significa ausência null ou undefined. Os seis valores desta tabela são falsy, mas só dois são nullish. || usa um padrão para qualquer falsy; ?? usa um padrão somente para nullish.
• Exemplo resolvido: const tentativas = 0;
console.log(tentativas || 3, tentativas ?? 3);
console.log(null || 3, null ?? 3);
Zero é falsy, mas não é nullish: || devolve 3 e ?? preserva 0. null é falsy e nullish: os dois operadores devolvem 3. Na matriz, classifique a entrada e depois preveja cada operador.
• Armadilha comum: "" é uma resposta (string vazia), não uma célula sem resposta. NaN pode ser inválido e mesmo assim não ser ausência. Esta tabela não lista todos os falsy de JavaScript.
• Por que importa: Escolher a regra de ausência correta preserva escolhas válidas de preferências e formulários.

Material/arquivo inicial

material.md: tabela: x em [false,0,"",null,undefined,NaN]; colunas x||"padrão" e x??"padrão"

Exemplo e resultado público

false/0/""/NaN → || dá padrão, ?? preserva x; null/undefined -> ambos dão padrão

Bordas e erros publicados

x=true -> ambos preservam true; x='ok' -> ambos preservam ok

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

MDN logical OR: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_OR; nullish coalescing: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual diferença entre || e ?? explica o tratamento de valores falsy versus valores ausentes?

□ (Correta) || substitui qualquer valor falsy; ?? mantém falsy válidos e substitui somente null ou undefined.
• ?? substitui false, 0 e texto vazio, enquanto || substitui só null e undefined.
• Ambos testam exatamente truthiness e diferem apenas na forma de escrever o fallback.

Por que a resposta está certa

Correto. || usa truthiness, mas ?? limita o fallback aos dois valores nullish.

Armadilha da alternativa incorreta

Classifique primeiro o valor. False, 0 e vazio podem ser escolhas válidas; somente null e undefined ativam ??.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

x=true → ambos preservam true; x='ok' -> ambos preservam ok

Concepção errada e falha observável

Tratar falso, zero e vazio como ausência em ??. / Coluna ?? substitui um valor válido que é falsy.

Pergunta diagnóstica

O valor é null/undefined ou apenas falsy?

Pistas graduais

1. Liste os dois valores nullish. 2. || testa truthiness; ?? testa só nullish. 3. Para volume 0, 0||50 dá 50 e 0??50 dá 0.

Solução de referência explicada

|| substitui os seis valores falsy desta tabela; ?? substitui apenas null e undefined. Exiba "" entre aspas, nunca como uma célula sem resposta.

Transferência em outro contexto

Compare 0 e undefined em duas preferências numéricas opcionais; escolha o operador que preserva uma escolha explícita e justifique a política para ausência.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Operadores || e ?? (ES2020) e a lista de valores falsy; verificado no Node.js 24 LTS. | Próxima sugestão: 02.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.03 Parsons de validação

PARSONS · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: operators.boolean_validation | Requer: bindings.values, operators.comparison, operators.logical | Introduz: operators.boolean_validation

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Monte a expressão de validação ordenando os blocos. Registre quais duas condições são obrigatórias e quais blocos devem anteceder o resultado; não use if, que será ensinado depois.

Objetivo da atividade

Uma lista ordenada dos blocos e uma frase dizendo por que cada condição obrigatória precisa ser combinada com a outra.

Explicação e modelo mental

• Modelo mental: Calcule cada requisito separadamente e combine os booleanos. O pagamento exige saldo suficiente E conta ativa; saldo igual ao valor já é suficiente. Um nome precisa ser declarado antes de ser usado.
• Exemplo resolvido: const estoque = 6;
const pedido = 6;
const aberto = true;
const suficiente = estoque >= pedido;
const autorizado = aberto === true;
const podeAtender = suficiente && autorizado;
console.log(podeAtender);
6 >= 6 é true; aberto === true também. true && true autoriza o pedido. Se qualquer requisito for false, && devolve false. A e B são independentes, mas C depende dos dois e D depende de C.
• Armadilha comum: || autorizaria com apenas um requisito. Retirar um bloco obrigatório exige restaurar a montagem.
• Por que importa: Uma regra de autorização precisa exigir todos os requisitos, incluindo a borda em que saldo e valor são iguais.

Material/arquivo inicial

material.md: entradas: saldo=10, valor=8, ativa=true.
A: const suficiente = saldo >= valor;
B: const autorizada = ativa === true;
C: const podePagar = suficiente && autorizada;
D: console.log(podePagar);
E (distrator): const podePagar = suficiente || autorizada;

Exemplo e resultado público

Saldo e conta suficientes: saldo=10,valor=8,ativa=true → true; Saldo exatamente igual: saldo=8,valor=8,ativa=true -> true; Saldo insuficiente: saldo=8,valor=10,ativa=true -> false; Conta inativa: saldo=10,valor=8,ativa=false -> false; Dois requisitos ausentes: saldo=2,valor=8,ativa=false -> false

Bordas e erros publicados

saldo=8,valor=10,ativa=true -> false; saldo=10,valor=8,ativa=false -> false

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

MDN logical AND: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_AND

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando saldo suficiente e conta ativa são requisitos simultâneos, que operador combina os resultados?

• Use ||: basta saldo suficiente ou conta ativa, porque qualquer condição válida autoriza o pedido.
 (Correta) Use &&: a validação só é true se as duas condições forem true.
• Use ! para inverter cada condição e aceite o pedido sempre que uma delas for false.

Por que a resposta está certa

Correto. A regra pede ambas as condições; conjunção impede autorização quando qualquer uma falha.

Armadilha da alternativa incorreta

Observe a palavra “e”. Uma condição alternativa não garante que os dois requisitos estejam satisfeitos ao mesmo tempo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

saldo=8,valor=10,ativa=true → false; saldo=10,valor=8,ativa=false -> false

Concepção errada e falha observável

Usar || para uma regra que exige as duas condições. / A resposta aprova conta inativa ou saldo insuficiente.

Pergunta diagnóstica

As duas condições são necessárias ou basta uma?

Pistas graduais

1. Calcule suficiente e autorizada separadamente. 2. Una com && porque ambas são obrigatórias. 3. Em entrada permitida e senha correta, só as duas verdadeiras autorizam.

Solução de referência explicada

A e B podem trocar de ordem; ambos devem vir antes de C e D. O distrator E viola a regra.

Transferência em outro contexto

Defina quando uma reserva de sala pode ser confirmada: a cobrança foi recebida e a sala está disponível. Escreva a condição e explique por que ambos os requisitos precisam valer.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Operadores lógicos &&, ||, ! e comparações estritas; sem API nova. | Próxima sugestão: 02.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.04 Normalizador de entrada

FADED · 15 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: coercion.number_validation | Requer: bindings.values, operators.boolean_validation | Introduz: coercion.number_validation, number.model

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Preveja numero e invalido para cada entrada pública, incluindo a flag vazio. O contrato aceita apenas números finitos: Number("Infinity") e 1e309 produzem Infinity e são inválidos. Um campo vazio é inválido pela flag; o texto "0" com vazio=false continua válido. Depois preencha somente as duas lacunas do starter e explique a validação.

Objetivo da atividade

Uma tabela de entrada → numero → invalido para os casos públicos, antes de preencher o código.

Explicação e modelo mental

• Modelo mental: Number(texto) converte texto em número. Number.isFinite(numero) responde se é um número finito; Number.isNaN só identifica NaN. A flag vazio já vem da entrada e vale true para campo vazio ou só espaços.
• Exemplo resolvido: const texto = "42.5";
const vazio = false;
const numero = Number(texto);
const invalido = vazio || !Number.isFinite(numero);
console.log(numero, invalido);
"42.5" vira 42.5; é finito. !true é false e false || false resulta em invalido=false. Number("") produz 0, mas vazio=true torna a entrada inválida. Já o texto "0" com vazio=false representa zero válido. "abc" vira NaN e "Infinity" vira Infinity: ambos são rejeitados por isFinite. Complete apenas as duas expressões ausentes.
• Armadilha comum: Number.isNaN(Infinity) é false. Usar só isNaN deixaria infinitos passarem.
• Por que importa: Separar conversão e validade evita tratar um campo vazio como zero legítimo ou aceitar Infinity.

Material/arquivo inicial

main.js: const texto = "12";
const vazio = false;
const numero = /* complete */;
const invalido = /* complete */;
console.log(invalido);

Exemplo e resultado público

"12" → invalido false; "0" -> false; "" com vazio=true -> true; "abc" -> true

Bordas e erros publicados

"NaN" -> true; "Infinity" -> true porque Number("Infinity") não é finito; 1e309 -> true; espaços com vazio=true -> true

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

As fixtures substituem apenas os literais texto e vazio declarados no arquivo. texto é string; vazio é boolean e vale true para texto vazio ou só espaços. Não há identificadores injetados.; O contrato exige finitude. Number.isNaN detecta somente NaN; Number.isFinite também rejeita Infinity e -Infinity. Não exigir if, throw ou funções.. MDN Number: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number; Number.isFinite: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isFinite

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que Number(texto) não basta para decidir se uma entrada de formulário é válida?

• Number rejeita automaticamente toda string vazia e toda palavra, então uma validação adicional não acrescenta informação.
• Qualquer resultado de Number é válido; NaN e Infinity representam quantidades que devem ser aceitas como zero.
 (Correta) A conversão pode produzir um número que ainda precisa ser finito, e a flag de campo vazio informa uma ausência que não se deve confundir com zero.

Por que a resposta está certa

Correto. Converter fornece um valor, mas a flag e a finitude determinam se esse valor atende à regra da entrada.

Armadilha da alternativa incorreta

Separe parsing da validação. Number("") pode virar zero; a flag vazio e Number.isFinite preservam as distinções do contrato.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

"NaN" → true; "Infinity" -> true porque Number("Infinity") não é finito; 1e309 -> true; espaços com vazio=true -> true

Concepção errada e falha observável

Usar isNaN global que converte entrada implicitamente ou aceitar vazio como zero. / Aceitar Infinity como quantidade válida ou tratar vazio como zero, embora o contrato exija um número finito e preserve a flag de ausência.

Pergunta diagnóstica

O site já forneceu vazio; depois da conversão, o número é finito? Zero continua válido quando o campo não está vazio.

Pistas graduais

1. Use a flag vazio declarada na fixture. 2. Converta com Number(texto); isNaN sozinho não rejeita Infinity. 3. Um valor é inválido quando está vazio ou não é finito: vazio || !Number.isFinite(numero).

Solução de referência explicada

const numero=Number(texto); const invalido=vazio || !Number.isFinite(numero). Number("Infinity") e 1e309 resultam em Infinity e são inválidos porque o contrato exige valor finito; vazio continua distinto do número 0.

Transferência em outro contexto

Valide a quantidade textual "0" e o texto "x", ambos com a flag de campo vazio em false; registre o valor convertido e a decisão de validade antes de executar.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Number(), Number.isFinite e Number.isNaN (ES2015); Node.js 24 LTS e navegadores evergreen. | Próxima sugestão: 02.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.05 Coercao explicita

MODIFY · 17 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: coercion.explicit | Requer: bindings.values, coercion.number_validation | Introduz: coercion.explicit

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Refatore a multiplicação usando Number() em cada entrada e String() na apresentação. Declare entradaValida combinando flags de vazio e finitude. Exiba entradaValida, total e etiqueta. Aqui você apenas detecta entrada inválida: o cálculo continua e seus resultados diagnósticos estão nos casos públicos. Rejeição com if e mensagens de erro será ensinada depois. Não é exigido Boolean().

Objetivo da atividade

O fluxo texto → números → entradaValida → total → etiqueta, com previsão das saídas válidas e inválidas antes da edição.

Explicação e modelo mental

• Modelo mental: Separe três decisões: converter as duas entradas, detectar validade e apresentar o cálculo como texto. Nesta prática, entradaValida sinaliza um problema; ainda não há if nem rejeição de entrada.
• Exemplo resolvido: const unidades = "4";
const valor = "2.5";
const unidadesVazias = false;
const valorVazio = false;
const unidadesNumero = Number(unidades);
const valorNumero = Number(valor);
const valida = !unidadesVazias && !valorVazio && Number.isFinite(unidadesNumero) && Number.isFinite(valorNumero);
const soma = unidadesNumero * valorNumero;
console.log(valida, soma, String(soma) + " reais");
As conversões produzem 4 e 2.5. As flags são false e os dois números são finitos, então valida=true. 4 * 2.5 vale 10. String(10) vira "10"; a apresentação é "10 reais". Com entrada vazia, o cálculo diagnóstico pode ser zero e a validade continua false. Isso não autoriza uma compra.
• Armadilha comum: A multiplicação converte strings implicitamente; aqui use Number nas duas entradas e String na apresentação para deixar a intenção explícita.
• Por que importa: Conversões explícitas tornam visível a passagem de texto recebido a cálculo numérico e apresentação, sem confundir diagnóstico com autorização.

Material/arquivo inicial

main.js: const quantidade = "3";
const preco = "10";
const quantidadeVazia = false;
const precoVazio = false;
const total = quantidade * preco;
const etiqueta = total + " reais";
console.log(total, etiqueta);

Exemplo e resultado público

quantidade="3",preco="10",flags=false → entradaValida=true, total=30, etiqueta="30 reais";
quantidade="0",preco="10",flags=false -> entradaValida=true, total=0, etiqueta="0 reais";
quantidade="",quantidadeVazia=true,preco="10",precoVazio=false -> entradaValida=false, total=0, etiqueta="0 reais";
quantidade="3",preco="x",flags=false -> entradaValida=false, total=NaN, etiqueta="NaN reais";
quantidade="3",preco="Infinity",flags=false -> entradaValida=false, total=Infinity, etiqueta="Infinity reais"

Bordas e erros publicados

quantidade=" ",quantidadeVazia=true,preco="10",precoVazio=false -> entradaValida=false, total=0, etiqueta="0 reais";
quantidade="3",preco="NaN",flags=false -> entradaValida=false, total=NaN, etiqueta="NaN reais"

Teste público e aprovação visível

Confira os casos publicados antes e depois da mudança; explique o que alterou e por que as saídas cumprem o contrato.

Restrições e documentação

Fixtures substituem apenas os quatro literais declarados. quantidade e preco são strings; quantidadeVazia e precoVazio são booleanos verdadeiros para campos vazios ou só espaços.; Não lançar erros ou usar if; entradaValida é o comportamento de detecção. total e etiqueta são calculados mesmo quando entradaValida=false.. MDN Number: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number; addition operator: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em que ordem uma entrada textual deve virar um valor calculado e uma apresentação?

 (Correta) Converta o texto, detecte validade, calcule os resultados diagnósticos e só então formate a saída.
• Concatene a entrada e a unidade primeiro; deixe a coerção implícita decidir como multiplicar a string.
• Formate a entrada como moeda antes de calcular e faça a função de domínio retornar esse texto localizado.

Por que a resposta está certa

Conversão, detecção, cálculo e apresentação têm papéis separados. Entrada inválida é marcada como false; rejeição com if vem depois.

Armadilha da alternativa incorreta

Confira os tipos em cada fronteira. Coerção implícita e formatação antecipada escondem a intenção e podem impedir cálculos posteriores.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

quantidade=" ",quantidadeVazia=true,preco="10",precoVazio=false → entradaValida=false, total=0, etiqueta="0 reais"; quantidade="3",preco="NaN",flags=false -> entradaValida=false, total=NaN, etiqueta="NaN reais"

Concepção errada e falha observável

Achar que coerção implícita acidental expressa claramente a intenção do domínio. / O código produz número por conversão escondida ou aceita entrada vazia sem decisão.

Pergunta diagnóstica

Onde o dado externo vira número e onde o número vira texto de apresentação?

Pistas graduais

1. Separe conversão, detecção de validade e cálculo. 2. Combine flags de vazio e Number.isFinite nos dois números. 3. Para texto "4" e preço "2", calcule com Number e só depois formate com String.

Solução de referência explicada

const quantidadeNumero=Number(quantidade); const precoNumero=Number(preco); const entradaValida=!quantidadeVazia && !precoVazio && Number.isFinite(quantidadeNumero) && Number.isFinite(precoNumero); const total=quantidadeNumero*precoNumero; const etiqueta=String(total)+" reais"; console.log(entradaValida,total,etiqueta); Para entradas inválidas, só detectar: o cálculo diagnóstico permanece observável; não emitir erros nem bloquear execução ainda.

Transferência em outro contexto

Refatore uma tarifa de quantidade "6" e preço "2.25", ambos não vazios. Exiba validade, total e etiqueta, separando conversão e apresentação.

Evidência de domínio

Confira os casos publicados antes e depois da mudança; explique o que alterou e por que as saídas cumprem o contrato. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Number() e String() como conversões explícitas; ES2015 em diante. | Próxima sugestão: 02.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.06 Bug de fallback

DEBUG · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: operators.nullish_fallback | Requer: bindings.values, operators.falsy_nullish | Introduz: operators.nullish_fallback

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 4 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Execute o starter sem mudanças e registre qual valor configurado desaparece no fallback. Corrija usando a distinção nullish apropriada, teste a ausência separadamente e explique por que a validação de tipo fica fora desta etapa.

Objetivo da atividade

O valor inicial observado, a previsão para um valor presente e a previsão para ausência, antes de alterar a expressão.

Explicação e modelo mental

• Modelo mental: Depurar é observar uma saída que contradiz a regra, localizar a causa e conferir a correção. Volume zero é uma escolha explícita; o padrão 50 deve servir apenas para ausência.
• Exemplo resolvido: const limiteEntrada = 0;
console.log(limiteEntrada || 3);
console.log(limiteEntrada ?? 3);
|| trata zero como falsy e mostra 3. ?? preserva zero. Execute o código original da atividade: ele mostra 50 com volumeEntrada=0. Depois registre sua previsão, corrija o operador e compare todos os casos.
• Armadilha comum: Trocar somente o número 50 por 0 não resolve os casos de null e undefined.
• Por que importa: Observar a contradição, localizar o operador e testar as ausências ensina a corrigir a causa de um erro de fallback.

Material/arquivo inicial

main.js: const volumeEntrada = 0;
const volume = volumeEntrada || 50;
console.log(volume);

Exemplo e resultado público

volume=0 → 0 após correção com ??; volume=null/undefined -> 50

Bordas e erros publicados

volume=false -> false preservado pelo operador ??, mas UI de volume deve validar tipo em etapa posterior

Teste público e aprovação visível

Mostre o caso que falha, explique o operador escolhido, corrija e confira novamente os casos publicados.

Restrições e documentação

Altere somente o literal volumeEntrada entre casos. O domínio desta atividade inclui number, false, null e undefined; validação de volume pertence a uma etapa posterior.. MDN nullish coalescing: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando um default só deve substituir uma configuração ausente, qual regra preserva uma escolha numérica igual a zero?

• Use ||, pois zero é falsy e portanto sempre deve significar ausência.
 (Correta) Use ??, que mantém zero e usa o valor padrão somente para null ou undefined.
• Use ?? e trate o resultado como automaticamente válido para qualquer faixa e tipo de configuração.

Por que a resposta está certa

Correto. Nullish fallback escolhe um default para ausência; ele não transforma zero em ausência nem valida a regra de domínio.

Armadilha da alternativa incorreta

Compare o critério de cada operador. || substitui zero; ?? preserva zero. Validação de faixa e tipo é uma checagem separada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

volume=false → false preservado pelo operador ??, mas UI de volume deve validar tipo em etapa posterior

Concepção errada e falha observável

Substituir zero explícito por default. / O teste com volume 0 retorna 50.

Pergunta diagnóstica

Zero significa ausência ou uma escolha legítima?

Pistas graduais

1. Compare || e ?? com 0. 2. Use fallback apenas para null/undefined. 3. 0??50 é 0; null??50 é 50.

Solução de referência explicada

const volume = volumeEntrada ?? 50; em exercício posterior, validar faixa e tipo separadamente.

Transferência em outro contexto

Investigue um contador de novas tentativas cujo fallback é 5 e que perde a escolha explícita 0; corrija a regra e compare com o caso ausente.

Evidência de domínio

Mostre o caso que falha, explique o operador escolhido, corrija e confira novamente os casos publicados. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Operador ?? (ES2020) e a diferença para ||; Node.js 24 LTS. | Próxima sugestão: 02.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.07 Regra de desconto

MAKE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: operators.discount_expression | Requer: bindings.values, operators.arithmetic | Introduz: operators.discount_expression

Eixos: novidade 2 · sintaxe 3 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Com preco finito não negativo e taxa finita entre 0 e 1, escreva o cálculo do total e exiba-o. O starter fornece apenas as entradas; você declara o resultado. Não implemente validação, if ou funções. Entradas fora desse domínio não são avaliadas aqui.

Objetivo da atividade

A expressão JavaScript que calcula o total depois do desconto e a anotação da fração que permanece.

Explicação e modelo mental

• Modelo mental: Uma taxa é uma fração: 25% corresponde a 25 / 100 = 0.25. Um desconto de 25% deixa 75% do preço. O domínio fornecido já garante preço finito não negativo e taxa entre 0 e 1.
• Exemplo resolvido: const valor = 84;
const desconto = 0.25;
const fracaoRestante = 1 - desconto;
const final = valor * fracaoRestante;
console.log(final);
1 - 0.25 vale 0.75. Multiplique 84 por 0.75: o resultado é 63. Sem desconto, taxa=0 deixa a fração 1. Desconto integral, taxa=1, deixa a fração 0. Escreva sua regra no binding total.
• Armadilha comum: preco * taxa calcula o desconto, não o preço final. Não use 20 para representar 20%.
• Por que importa: Transformar uma porcentagem em fração e testar os extremos permite transferir a fórmula para preços diferentes.

Material/arquivo inicial

main.js: const preco = 100;
const taxa = 0.2;
// Escreva o cálculo do total e exiba-o com console.log.

Exemplo e resultado público

preço=100,taxa=0.2 → total=80; preço=50,taxa=0 -> total=50

Bordas e erros publicados

preco=0,taxa=0.5 -> 0; preco=100,taxa=1 -> 0

Teste público e aprovação visível

Aceite qualquer cálculo que cumpra as entradas e saídas publicadas; não exigir tratamento de erro fora do domínio declarado.

Restrições e documentação

Fixtures alteram apenas preco e taxa declarados; preço >= 0 e taxa em [0,1], ambos finitos. Não há validação fornecida e nenhuma rejeição é exigida.. MDN multiplication operator:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Multiplication

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se taxa representa a fração removida do preço, qual expressão produz o total restante?

• preco * taxa, porque a fração removida é também o total final a pagar.
• preco + taxa, porque a taxa é um valor monetário fixo que sempre se soma ao preço.
 (Correta) preco * (1 - taxa), porque 1 - taxa é a proporção que permanece.

Por que a resposta está certa

Correto. A subtração encontra a fração remanescente; multiplicá-la pelo preço aplica o desconto.

Armadilha da alternativa incorreta

Identifique primeiro se está calculando a parte removida ou a parte restante. O contrato pede o total após o desconto, não o desconto em si.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

preco=0,taxa=0.5 → 0; preco=100,taxa=1 -> 0

Concepção errada e falha observável

Aplicar taxa como valor absoluto ou somá-la ao preço. / Desconto de 20% em 100 produz 20 ou 120 em vez de 80.

Pergunta diagnóstica

Que fração do preço permanece depois de descontar taxa?

Pistas graduais

1. A fração restante é 1 - taxa. 2. Multiplique o preço pela fração restante. 3. Se taxa=0, o resultado deve ser o preço original.

Solução de referência explicada

const total = preco * (1 - taxa); console.log(total); As fixtures garantem preço finito não negativo e taxa finita em 0..1; não existe validação oculta no starter.

Transferência em outro contexto

Calcule o total de um frete de 40 após aplicar a taxa de 10% já validada. Antes de executar, escreva a expressão que usa a fração do preço que permanece.

Evidência de domínio

Aceite qualquer cálculo que cumpra as entradas e saídas publicadas; não exigir tratamento de erro fora do domínio declarado. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Aritmética de ponto flutuante e operadores básicos; sem API nova. | Próxima sugestão: 02.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.08 Transferencia - preferencia de usuario

TRANSFER · 16 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: operators.preferences | Requer: bindings.values, operators.nullish_fallback | Introduz: operators.preferences

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Corrija o fallback das três preferências. Antes de editar, registre quais valores falsy são escolhas válidas; depois preserve-os sem misturar fallback de ausência com validação de tipo.

Objetivo da atividade

Uma tabela com os três valores recebidos, se cada um é escolha válida ou ausência, e qual fallback deve ser aplicado.

Explicação e modelo mental

• Modelo mental: Aplique a regra de ausência a cada preferência, de forma independente. Nesta atividade, 0, false e "" são escolhas aceitas, e qualquer outro valor presente também deve permanecer.
• Exemplo resolvido: const tentativasEntrada = 0;
const avisoEntrada = null;
const tentativas = tentativasEntrada ?? 3;
const aviso = avisoEntrada ?? false;
console.log(tentativas, aviso);
tentativas mantém 0; aviso recebe false porque sua entrada é null. Transfira essa regra para volume, tema e notificacoes. Observe os casos em que apenas uma preferência está ausente.
• Armadilha comum: Não grave sempre os valores do primeiro caso. Um tema "escuro" precisa continuar "escuro".
• Por que importa: Aplicar a mesma política em outro domínio demonstra que ausência não significa zero, falso ou texto vazio.

Material/arquivo inicial

main.js: const volumeEntrada = 0;
const temaEntrada = "";
const notificacoesEntrada = false;
const volume = volumeEntrada || 50;
const tema = temaEntrada || "claro";
const notificacoes = notificacoesEntrada || true;
console.log(volume, tema, notificacoes);

Exemplo e resultado público

Escolhas falsy válidas: volumeEntrada=0,temaEntrada="",notificacoesEntrada=false → 0,"",false; Três ausências: volumeEntrada=null,temaEntrada=undefined,notificacoesEntrada=null -> 50,"claro",true; Escolhas presentes: volumeEntrada=20,temaEntrada="escuro",notificacoesEntrada=true -> 20,"escuro",true; Ausência só no volume: volumeEntrada=undefined,temaEntrada="escuro",notificacoesEntrada=false -> 50,"escuro",false; Ausência só no tema: volumeEntrada=10,temaEntrada=null,notificacoesEntrada=true -> 10,"claro",true

Bordas e erros publicados

entradas null,undefined,null -> 50,"claro",true

Teste público e aprovação visível

Compare competência estrutural em domínio diferente e sem ajuda.

Restrições e documentação

Fixtures substituem apenas os três literais de entrada. Zero, false e "" são escolhas aceitas; null/undefined significam ausência. String vazia aparece como "" na tabela.. MDN nullish coalescing: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que operador preserva escolhas falsy quando o default só deve atuar para ausência?

 (Correta) Use ?? nos campos cujos valores 0, string vazia ou false são escolhas válidas; valide tipo e domínio separadamente.
• Use || em todos os campos, porque qualquer valor falsy representa que o usuário não fez escolha.
• Use ?? e dispense qualquer validação: valores diferentes de null e undefined são sempre válidos para o produto.

Por que a resposta está certa

Correto. Nullish fallback preserva escolhas falsy; a validação continua responsável por tipos e regras do domínio.

Armadilha da alternativa incorreta

Confirme a semântica do campo antes do fallback. || troca valores falsy válidos, e ?? não substitui a validação de produto.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entradas null,undefined,null → 50,"claro",true

Concepção errada e falha observável

Aplicar || a preferências falsy que têm significado. / Ao carregar preferências, zero, vazio ou false são apagados.

Pergunta diagnóstica

Quais campos podem ser falsy por escolha do usuário?

Pistas graduais

1. Teste o objeto com todas as escolhas falsy. 2. Troque fallback de ausência por ??. 3. Validação de tipo é regra separada da escolha de default.

Solução de referência explicada

Use ?? nos três campos; preserve 0, string vazia e false. O contrato aceita vazio como tema personalizado nesta atividade.

Transferência em outro contexto

Revise defaults de um player com volume 0, repetição false e legenda vazia; determine quando cada campo deve receber um padrão.

Evidência de domínio

Compare competência estrutural em domínio diferente e sem ajuda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: ?? (ES2020) para preservar 0, '' e false; Node.js 24 LTS. | Próxima sugestão: 02.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.09 Checkpoint: placar com entradas opcionais

TRANSFER · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: operators.scoreboard | Requer: bindings.values, operators.arithmetic, operators.preferences | Introduz: operators.scoreboard

Eixos: novidade 1 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Construa um placar de duas equipes que aplique o default a campos ausentes e some os valores. Antes de codificar, escreva os dois valores usados e a estratégia para ausência; depois rode os casos públicos.

Objetivo da atividade

Um plano curto de normalização dos dois campos e a primeira expressão/implementação que produz o total.

Explicação e modelo mental

• Modelo mental: Checkpoint: normalize cada placar ausente para 0 e some os dois números. As entradas são números finitos não negativos, null ou undefined. Retome 02.02 e 02.08 se precisar; tente escrever sem abrir dicas.
• Exemplo resolvido: const pontos = 13;
const visitante = null;
console.log(pontos + (visitante ?? 0));
visitante é null e recebe 0; pontos continua 13. A soma é 13. Na sua solução, declare a e b separadamente antes de total. Escolha casos que distingam zero presente, ausência e dois placares presentes.
• Armadilha comum: Não converta texto: este checkpoint não recebe strings. Na próxima etapa, uma condição como total >= 10 poderá escolher um caminho; aqui você aprende a calculá-la.
• Por que importa: Combinar normalização e soma consolida operadores antes de usar condições para controlar o fluxo na Etapa 03.

Material/arquivo inicial

main.js: const aEntrada = 0;
const bEntrada = 5;
// Declare a e b com padrão 0 para ausência, calcule total e exiba a, b, total.

Exemplo e resultado público

Zero e cinco: aEntrada=0,bEntrada=5 → 0,5,5; Ambos ausentes: aEntrada=undefined,bEntrada=undefined -> 0,0,0; Null e zero: aEntrada=null,bEntrada=0 -> 0,0,0; Três e ausência: aEntrada=3,bEntrada=undefined -> 3,0,3; Dois placares presentes: aEntrada=7,bEntrada=2 -> 7,2,9

Bordas e erros publicados

aEntrada=null,bEntrada=0 -> 0,0,0; aEntrada=3,bEntrada=undefined -> 3,0,3

Teste público e aprovação visível

Compare competência estrutural em domínio diferente e sem ajuda.

Restrições e documentação

As fixtures substituem apenas aEntrada e bEntrada. Cada entrada é um Number finito não negativo, null ou undefined. Parsing de strings e rejeição de dados externos não fazem parte deste checkpoint.. MDN nullish coalescing: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing; addition operator: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em que ordem normalizar entradas opcionais para montar um placar sem trocar um zero válido por default?

• Some aEntrada e bEntrada diretamente, pois undefined participa da soma como zero.
 (Correta) Aplique ?? 0 a cada entrada ausente, preserve os valores presentes e depois some os dois números normalizados.
• Some os campos primeiro e troque NaN por zero ao final, sem definir a política por entrada.

Por que a resposta está certa

Correto. A política por campo explicita a ausência; aplicar o default antes da soma preserva zero e evita propagar undefined.

Armadilha da alternativa incorreta

Normalize antes de somar: undefined não é zero na soma e null/undefined devem receber o padrão por campo. || 0 dá o mesmo resultado para este domínio, mas não explica a política nullish.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

aEntrada=null,bEntrada=0 → 0,0,0; aEntrada=3,bEntrada=undefined -> 3,0,3

Concepção errada e falha observável

Somar entradas opcionais diretamente e propagar NaN quando uma delas é undefined. / Uma entrada undefined faz a soma virar NaN em vez de usar zero.

Pergunta diagnóstica

Você distingue ausência de valor zero?

Pistas graduais

1. Use ?? para ausência. 2. Não substitua zero por default. 3. Some os dois valores após aplicar o fallback.

Solução de referência explicada

const a=aEntrada??0; const b=bEntrada??0; const total=a+b; a validação de entradas externas vem após controle de fluxo.

Transferência em outro contexto

Monte o saldo de duas cotas de orçamento: uma contribuição explícita é zero e a outra está ausente. Defina o padrão só para a ausência antes de somar.

Evidência de domínio

Compare competência estrutural em domínio diferente e sem ajuda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Parâmetros padrão (ES2015), ?? (ES2020) e aritmética básica; Node.js 24 LTS. | Próxima sugestão: 02.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.10 Math: arredondamento, utilitários e sorteios

MODIFY · 28 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: math.rounding | Requer: bindings.values, operators.arithmetic, coercion.number_validation | Introduz: math.rounding, math.basic, math.random

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Bloco A: preveja e complete floor/ceil/round/trunc para 89/60 e -89/60. Bloco B: escreva min(x,y), max(x,y) e abs(deslocamento). Bloco C: execute Math.random(), explique 0 <= sorteio < 1 e escreva a transformação para um dado de 1 a 6. Teste também sorteio 0, 0.5 e 0.9999999999999999, substituindo somente a inicialização. Não espere uma saída aleatória fixa. Math.random não serve a senhas ou tokens. Curiosidade opcional: Math.round(-0.5) retorna -0.

Objetivo da atividade

Tabelas de arredondamento e utilitários; previsão do intervalo de sorteio e das faces para 0, 0.5 e 0.9999999999999999 antes de executar cada bloco.

Explicação e modelo mental

• Modelo mental: Math é um conjunto de ferramentas numéricas. A chamada Math.floor(x) recebe um número e devolve outro: floor vai ao inteiro abaixo, ceil ao inteiro acima, trunc remove a parte fracionária e round escolhe o inteiro mais próximo (empates vão em direção a +Infinity). Math.min(a,b) devolve o menor argumento e Math.max(a,b) o maior. Math.abs(x) mede magnitude: a distância até zero, sem sinal. Math.random() não recebe argumentos e gera um número em [0,1): inclui zero e exclui um. Para N faces, multiplique por N, tire a parte fracionária com floor e some 1.
• Exemplo resolvido: const minutos = 149 / 60;
console.log(Math.floor(minutos), Math.ceil(minutos), Math.round(minutos), Math.trunc(minutos));
console.log(Math.floor(-1.8), Math.trunc(-1.8));
149 / 60 é aproximadamente 2.483: floor=2, ceil=3, round=2 e trunc=2. Para -1.8, o inteiro abaixo é -2; remover a fração dá -1. Na atividade, compare as quatro regras para os dois sinais.

console.log(Math.min(2, 8), Math.max(2, 8));
console.log(Math.abs(-4));
Entre 2 e 8, min=2 e max=8. A magnitude de -4 é 4. Use as entradas x, y e deslocamento para que a regra continue funcionando em cada caso.

const amostra = 0.5;
const faceDeQuatro = Math.floor(amostra * 4) + 1;
console.log(amostra, faceDeQuatro);
Exemplo paralelo com quatro faces: 0.5 * 4 vale 2; floor(2) + 1 vale 3. Zero vira 1; valores perto de 1 viram 4. O exercício pede SEIS faces: adapte N para 6 e mantenha Math.random() no starter. Os casos substituem sorteio por números fixos para testar a regra. O botão de sorteio real pode mostrar qualquer amostra em [0,1), mas a face deve ser inteira de 1 a 6.
• Armadilha comum: Arredondar um número negativo não é simplesmente apagar o sinal. A distinção entre -0 e 0 é opcional neste bloco. abs não arredonda. Em -2 e 7, o menor é -2, mesmo com magnitude positiva. Multiplicar por quatro resolveria o exemplo, não a tarefa. Math.random não serve para senhas ou tokens.
• Por que importa: Escolher arredondamento e mapear intervalos evita erros de limites em medidas, contagens e sorteios.

Material/arquivo inicial

main.js: const segundos = 89;
const segundosNegativos = -89;
console.log(Math.floor(segundos / 60));
// Complete ceil, round e trunc; depois as quatro operações para segundosNegativos.
const x = 3;
const y = 5;
const deslocamento = -3;
// Exiba min(x,y), max(x,y) e abs(deslocamento).
const sorteio = Math.random();
// Declare face usando sorteio para gerar um inteiro de 1 a 6 e exiba sorteio, face.

Exemplo e resultado público

s=89 → floor 1, ceil 2, round 1, trunc 1; s=-89 -> floor -2, ceil -1, round -1, trunc -1; x=3,y=5,deslocamento=-3 -> min 3,max 5,abs 3; sorteio=0 -> face 1; sorteio=0.5 -> face 4; sorteio=0.9999999999999999 -> face 6

Bordas e erros publicados

s=30 -> round 1; s=-30 -> round -0 (curiosidade opcional); x=-2,y=7,deslocamento=4 -> min -2,max 7,abs 4; sorteio=0.25 -> face 2

Teste público e aprovação visível

Confira os casos publicados antes e depois da mudança; explique o que alterou e por que as saídas cumprem o contrato.

Restrições e documentação

Fixtures alteram apenas segundos, segundosNegativos, x, y e deslocamento; são números finitos. Para o sorteio, substituem Math.random() por literais no intervalo [0,1).; Avalie o intervalo da saída aleatória e os casos determinísticos da fórmula; nunca exija um sorteio específico. A distinção de -0 é opcional.. MDN Math methods: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual fórmula transforma um sorteio x em [0,1) em faces inteiras de 1 a 6?

• Math.round(x * 6); gera apenas 1 a 6 com chances iguais.
• Math.ceil(x * 6); sempre inclui 1 e nunca devolve 0.
 (Correta) Math.floor(x * 6) + 1; inclui 1 e 6 e não serve para segredos.

Por que a resposta está certa

Floor produz 0..5; somar 1 desloca a faixa para 1..6. Para segurança, Math.random é inadequado.

Armadilha da alternativa incorreta

Teste x=0 e x perto de 1. Round muda a distribuição; ceil(0) é 0.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

s=30 → round 1; s=-30 -> round -0 (curiosidade opcional); x=-2,y=7,deslocamento=4 -> min -2,max 7,abs 4; sorteio=0.25 -> face 2

Concepção errada e falha observável

Tratar floor e trunc como equivalentes para negativos. / Com -89 segundos, o resultado de floor foi -1.

Pergunta diagnóstica

Qual operação arredonda em direção a -Infinity e qual apenas remove a fração?

Pistas graduais

1. Compare arredondamentos para números negativos. 2. Use min/max para extremos e abs para magnitude. 3. Para sortear 1 a 4, Math.floor(sorteio * 4) + 1 transforma [0,1) em inteiros 1..4.

Solução de referência explicada

Complete floor/ceil/round/trunc para ambos os quocientes e console.log(Math.min(x,y),Math.max(x,y),Math.abs(deslocamento)). const face=Math.floor(sorteio*6)+1; console.log(sorteio,face). Não compare uma execução aleatória com uma saída exata; confira 0<=sorteio && sorteio<1 e face inteira em 1..6. As fixtures determinísticas testam a transformação nos extremos.

Transferência em outro contexto

Gere um inteiro de 1 a 4 usando um sorteio em [0,1). Explique as saídas para 0 e um valor perto de 1; depois escolha a operação para acomodar 15 itens em páginas de 7.

Evidência de domínio

Confira os casos publicados antes e depois da mudança; explique o que alterou e por que as saídas cumprem o contrato. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Math.floor, Math.ceil, Math.round, Math.trunc, Math.min, Math.max, Math.abs e Math.random; disponíveis em todos os runtimes do curso. | Próxima sugestão: 02.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.11 Number não representa todo decimal exatamente

PREDICT · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: number.ieee754 | Requer: bindings.values, number.model, operators.comparison | Introduz: number.ieee754, number.safe_integers

Eixos: novidade 4 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Resolva e confira o bloco A antes de avançar: alguns decimais como 0.5 e 0.25 são exatos; 0.1 + 0.2 é 0.30000000000000004 e sua igualdade com 0.3 é false. Distinga NaN de Infinity. Depois preveja o bloco B: inteiros seguros e typeof 1n. Explique onde BigInt ajuda e por que não resolve frações. -0 é uma curiosidade opcional.

Objetivo da atividade

Previsões separadas para blocos A e B, com valor e tipo, antes de executar cada bloco.

Explicação e modelo mental

• Modelo mental: Number representa números em ponto flutuante binário. Frações como 0.5 e 0.25 têm representação exata; muitas outras são aproximações. Isso pode aparecer na soma e na igualdade estrita. Number.isSafeInteger(x) confirma que x é um inteiro dentro de -(2 ** 53 - 1) até 2 ** 53 - 1. Fora desse intervalo, Number pode perder diferenças entre inteiros. O sufixo n cria um BigInt.
• Exemplo resolvido: console.log(0.5 + 0.25 === 0.75);
console.log(0.2 + 0.7, 0.2 + 0.7 === 0.9);
console.log(Number.isNaN(NaN), Number.isFinite(Infinity));
A primeira comparação é true. A segunda soma produz 0.8999999999999999 e a igualdade é false. O resultado é determinístico. NaN indica resultado numérico inválido; Infinity não é finito. Use os predicados explícitos para distingui-los.

console.log(Number.isSafeInteger(9007199254740991));
console.log(9007199254740992 === 9007199254740993);
console.log(9007199254740992n + 1n);
console.log(typeof 2n);
O limite 9007199254740991 é seguro. Os dois Number da comparação seguinte acabam com a mesma representação: a igualdade dá true. BigInt preserva o inteiro seguinte: a soma dá 9007199254740993n e typeof devolve "bigint".
• Armadilha comum: Nem todo decimal é inexato. Não use a impressão arredondada como prova de igualdade. BigInt não representa frações; 1n + 1 mistura tipos e causa erro. Use operandos do mesmo tipo.
• Por que importa: O modelo numérico orienta a comparação de medidas e a escolha de tipos para inteiros grandes.

Material/arquivo inicial

main.js: // Bloco A: decimais e valores especiais
console.log(0.5 + 0.25 === 0.75);
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log(Number.isNaN(NaN));
console.log(Number.isFinite(Infinity));
// Bloco B: inteiros seguros e BigInt
console.log(Number.isSafeInteger(2 ** 53 - 1));
console.log(Number.isSafeInteger(2 ** 53));
console.log(typeof 1n);

Exemplo e resultado público

Bloco A → true, 0.30000000000000004, false, true, false; Bloco B -> true, false, "bigint" (string)

Bordas e erros publicados

0.2+0.4===0.6 -> false; Number.isSafeInteger(-(2**53-1)) -> true; Object.is(-0,0) -> false (opcional)

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

MDN Number: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number; Number.isNaN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isNaN; Object.is: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/is; BigInt: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que Number pode falhar numa comparação decimal exata e que tipo usar para inteiros além da precisão segura?

 (Correta) Number é binário e aproxima muitos decimais; BigInt permite aritmética inteira precisa, mas precisa ser usado sem misturar implicitamente com Number.

• Number armazena todos os decimais exatamente; uma comparação false só pode significar que o operador === está quebrado.

• BigInt é uma forma de Number com decimais exatos e pode ser somado diretamente a qualquer Number.

Por que a resposta está certa

Correto. IEEE-754 explica as aproximações de decimal, e BigInt é indicado para inteiros que exigem precisão além da faixa segura.

Armadilha da alternativa incorreta

Considere a representação binária e o tipo da operação. BigInt é exato para inteiros, mas não converte Number em decimal exato nem se mistura implicitamente.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

0.2+0.4===0.6 → false; Number.isSafeInteger(-(2**53-1)) -> true; Object.is(-0,0) -> false (opcional)

Concepção errada e falha observável

Supor que Number representa todo decimal/inteiro sem perda. / O aluno prevê igualdade exata ou classifica 2**53 como inteiro seguro.

Pergunta diagnóstica

Qual bloco avalia aproximação decimal e qual avalia inteiros seguros?

Pistas graduais

1. Compare um decimal binário exato (0.5) com outro aproximado (0.1). 2. NaN é inválido; Infinity não é finito. Inteiros seguros têm limite próprio. 3. BigInt é para inteiros; typeof 2n produz a string "bigint".

Solução de referência explicada

Bloco A: true,0.30000000000000004,false,true,false. Bloco B: true,false,"bigint". 0.5 e 0.25 são exatos; 0.1 e 0.2 são aproximados em binário. Inteiros seguros estão entre -(2**53-1) e 2**53-1. BigInt é para aritmética inteira, sem mistura implícita com Number.

Transferência em outro contexto

Preveja 0.2 + 0.4 === 0.6 e Number.isSafeInteger(2 ** 53 - 1). Explique por que o primeiro resultado é determinístico e por que BigInt serve apenas a inteiros.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Número de ponto flutuante de dupla precisão (IEEE 754); Number.EPSILON (ES2015) para comparação com tolerância. | Próxima sugestão: 02.12

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

02.12 Escolha a tolerância pela política do sensor

TRANSFER · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: number.tolerance | Requer: bindings.values, math.rounding, number.ieee754 | Introduz: number.tolerance

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 1 · depuração 1 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

O sensor informa resolução 0.01 unidade. A política deste exercício aceita diferença absoluta de até metade dessa resolução, inclusive. Declare sua tolerancia a partir de resolucao; depois escreva e exiba a comparação aproximada. Antes de executar, registre limiar, diferença e decisão. Não use Number.EPSILON como limite universal; compare também os casos publicados perto de 1 e de 2000.

Objetivo da atividade

O limiar derivado de cada resolução, a diferença absoluta prevista e a decisão true/false antes de escrever a comparação.

Explicação e modelo mental

• Modelo mental: Comparar medidas aproximadas exige uma política do domínio. Aqui ela é explícita: aceitar diferença de ATÉ metade da resolução, inclusive. Math.abs(a - b) mede a distância; <= inclui o limite.
• Exemplo resolvido: const leituraA = 0;
const leituraB = 0.125;
const passo = 0.25;
const limite = passo / 2;
console.log(limite, Math.abs(leituraA - leituraB) <= limite);
0.25 / 2 vale 0.125. A distância também é 0.125, então <= devolve true no limite. Com <, o mesmo caso falharia. No starter, 0.01 / 2 vale 0.005; a distância das leituras é aproximadamente 0.004 e é aceita. Declare tolerancia a partir de resolucao em cada caso.
• Armadilha comum: Number.EPSILON descreve uma escala perto de 1, não a política deste sensor. Resolução sozinha não define uma tolerância universal.
• Por que importa: Uma tolerância definida pelo domínio torna a comparação de sensores justificável e testável, inclusive na borda.

Material/arquivo inicial

main.js: const a = 2000.3;
const b = 2000.304;
const resolucao = 0.01;
// A política aceita diferença de até metade da resolução, inclusive.
// Declare tolerancia a partir de resolucao e exiba a comparação aproximada.

Exemplo e resultado público

a=2000.3,b=2000.304,resolucao=0.01 → tolerancia=0.005, true; a=1,b=1.1,resolucao=0.02 -> tolerancia=0.01, false; a=0.1+0.2,b=0.3,resolucao=2e-15 -> tolerancia=1e-15, true; a=1000.1+1000.2,b=2000.3,resolucao=2e-12 -> tolerancia=1e-12, true; a=0,b=0.125,resolucao=0.25 -> tolerancia=0.125, true

Bordas e erros publicados

a=0,b=0.25,resolucao=0.25 -> tolerancia=0.125, false; a=4,b=4,resolucao=0.01 -> tolerancia=0.005, true

Teste público e aprovação visível

Compare competência estrutural em domínio diferente e sem ajuda.

Restrições e documentação

Fixtures substituem a, b e resolucao declarados: a/b são finitos e resolucao é finita positiva. A política é tolerancia=resolucao/2, com limite inclusivo.; Não exigir função, rejeição ou if. Resolução sozinha não determina política de comparação; aqui a regra de metade está publicada.. MDN Math.abs: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/abs; Number: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Com resolução 0.02 e política que aceita até metade da resolução, qual limiar e comparação usar?

• tolerancia = Number.EPSILON sempre, independentemente da política.
 (Correta) tolerancia = 0.02 / 2; Math.abs(a - b) <= tolerancia.
• tolerancia = 0.02; a === b, pois a resolução elimina aproximações.

Por que a resposta está certa

A política determina 0.01 e inclui a borda. A diferença absoluta aplica o limiar.

Armadilha da alternativa incorreta

Derive o limiar da política publicada. EPSILON e igualdade estrita não expressam esse requisito.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

a=0,b=0.25,resolucao=0.25 → tolerancia=0.125, false; a=4,b=4,resolucao=0.01 -> tolerancia=0.005, true

Concepção errada e falha observável

Usar Number.EPSILON fixo para todo domínio e escala. / O caso em torno de 2000 falha apesar de tolerância publicada.

Pergunta diagnóstica

Qual tolerância a política de metade da resolução determina?

Pistas graduais

1. A política pede metade da resolução; isso é uma escolha deste domínio, não uma lei universal. 2. Declare o limiar antes da comparação e use Math.abs(a-b). 3. Em outro sensor com resolução 0.2, metade é 0.1; diferença 0.08 fica dentro desse limite.

Solução de referência explicada

const tolerancia=resolucao/2; console.log(Math.abs(a-b)<=tolerancia); O limiar foi derivado da política publicada. Não inferir tolerância apenas da resolução sem política; Number.EPSILON não é limite universal.

Transferência em outro contexto

Outro sensor informa resolução 0.02 e política de metade da resolução. Declare o limiar e compare 400 com 400.004; justifique usando a política.

Evidência de domínio

Compare competência estrutural em domínio diferente e sem ajuda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Math.abs e Number.EPSILON (ES2015) para tolerância; a política de aceitação pertence ao exercício, não a uma API. | Próxima sugestão: 03.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03 Decisões e repetição

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

A unidade 03A cobre decisão; a 03B cobre repetição. A ordem dos IDs nesta seção é intencional.

Ordem: 03.04 · 03.05 · 03.01 · 03.02 · 03.03 · 03.06 · 03.07 · 03.08 · 03.09 · 03.10

03.04 Classificador de faixa

FADED · 14 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: control.range_classifier | Requer: operators.basic, control.branch | Introduz: control.range_classifier

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Complete as faixas de nota depois da guarda inicial. A nota válida vai de 0 a 100, inclusive; valores menores que 0 ou maiores que 100 recebem “inválida”. Preserve a guarda e classifique os válidos em E, D, C, B ou A sem lacunas nos cortes.

Objetivo da atividade

Escreva os intervalos em palavras e diga a qual faixa pertence cada corte antes de preencher o código.

Explicação e modelo mental

• Modelo mental: Uma decisão começa com uma condição entre parênteses. Se ela for verdadeira, o bloco logo depois é executado; se for falsa, o programa procura o else ou a próxima condição da cadeia. Em uma cadeia if/else if, os testes são avaliados de cima para baixo e somente o primeiro ramo verdadeiro é escolhido. Uma guarda trata valores inválidos antes das faixas válidas; depois, cada teste pode usar um limite superior e o else recebe o restante. Escreva a condição em palavras antes de escolher < ou <=. Teste um valor imediatamente abaixo, o valor no corte e um valor imediatamente acima. Assim você descobre a qual intervalo a igualdade pertence. As chaves { e } delimitam as instruções do ramo. if (condição) { ação; } pode ter else { alternativa; }; else if acrescenta outro teste. Você já aprendeu comparações e || na Etapa 02: nota < 0 || nota > 100 aceita qualquer uma das duas situações inválidas.
• Exemplo resolvido: temperatura vale 28, então o teste temperatura < 10 falha. O teste temperatura < 25 também falha; o programa chega ao else. aviso recebe "quente" e o console imprime essa string. O else não precisa de uma nova condição.
• Armadilha comum: Trocar a ordem das faixas ou usar <= sem reavaliar o corte desloca exatamente os valores de fronteira.
• Por que importa: Depois de prever cada corte, complete o classificador e confira 0, 20, 40, 60, 80, 100 e um valor inválido.

Material/arquivo inicial

main.js: const nota = 20;
let faixa = "";
if (nota < 0 || nota > 100) {
faixa = "inválida";
} else if (nota < 20) {
faixa = "E";
} else if (/* complete a próxima faixa */) {
faixa = "D";
} else {
faixa = "A";
}
console.log(faixa);

Exemplo e resultado público

notas 0,20,40,60,80 → E,D,C,B,A; nota 79 -> B

Bordas e erros publicados

nota 100 -> A; nota -1 ou >100 -> inválida segundo guarda inicial fornecida

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em limites <20, <40, <60, em qual ramo cai exatamente 40?

• No ramo do segundo limite, pois 40 é igual a 40.
• Em nenhum ramo, pois 40 não é menor que os três cortes.
 (Correta) No terceiro ramo: falha em <20 e <40, mas satisfaz <60.

Por que a resposta está certa

Correto: 40 falha nos testes <20 e <40, mas satisfaz <60; por isso entra no terceiro ramo.

Armadilha da alternativa incorreta

40 não é menor que 20 nem que 40; a cadeia continua até o teste <60, cujo ramo recebe esse valor.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

nota 100 → A; nota -1 ou >100 -> inválida segundo guarda inicial fornecida

Concepção errada e falha observável

Misturar < e <= e criar buracos/sobreposição nos limites. / Valor exatamente no limite entra na faixa anterior.

Pergunta diagnóstica

O que acontece em 19/20, 39/40, 59/60 e 79/80?

Pistas graduais

1. Escreva intervalos sem sobreposição. 2. Use <20,<40,<60,<80 e else. 3. Teste imediatamente abaixo/no/acima de cada fronteira.

Solução de referência explicada

Após guarda 0..100, if nota<20 E; else if <40 D; else if <60 C; else if <80 B; else A.

Transferência em outro contexto

Classifique idades inteiras nas faixas 0–12, 13–17 e 18 ou mais. Escreva condições sem lacunas/sobreposição e teste explicitamente 12, 13, 17 e 18.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.05 Teste de limites

TEST · 15 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: control.boundary_tests | Requer: operators.basic, control.branch | Introduz: control.boundary_tests

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 1 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Crie casos abaixo, no corte e acima para as regras de desconto, idade, peso e a regra adicional x>0. Registre resultado esperado e explique como o caso de igualdade distingue o operador.

Objetivo da atividade

Preencha primeiro uma matriz com abaixo/no/acima para cada uma das quatro regras, incluindo x>0; anote o operador que a igualdade deve confirmar.

Explicação e modelo mental

• Modelo mental: Um teste de fronteira escolhe entradas que distinguem operadores vizinhos. Para uma regra >= L, use L-1, L e L+1: o valor anterior deve falhar, a igualdade deve passar e o seguinte também deve passar. Para <= L, a igualdade passa e L+1 falha. Para > L, L ainda falha e L+1 passa. O resultado no corte é a evidência que revela se a implementação incluiu a igualdade. Registre a entrada e o resultado esperado antes de rodar. Um caso distante da fronteira pode passar mesmo quando o operador foi trocado.
• Exemplo resolvido: tentativas e limite valem 3, portanto tentativas <= limite é true. A igualdade também é true, confirmando que o limite está incluído. A comparação estrita tentativas > limite é false no próprio corte.
• Armadilha comum: Escolher apenas valores aprovados ou apenas valores muito distantes não distingue >= de > nem <= de <.
• Por que importa: Monte uma linha abaixo/no/acima para cada regra e escreva o resultado booleano de cada célula.

Material/arquivo inicial

matriz.md: const total = 100;
const idade = 18;
const peso = 5;
// escreva os casos de teste abaixo, no corte e acima
console.log(total, idade, peso);

Exemplo e resultado público

total 99/100/101 → não/sim/sim; idade 17/18/19 -> não/sim/sim; peso 4/5/6 -> sim/sim/não

Bordas e erros publicados

para regra x>0 -> -1/0/1 dá não/não/sim

Teste público e aprovação visível

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Para testar valor >=18 na fronteira, qual conjunto é mais informativo?

 (Correta) 17, 18 e 19; a igualdade precisa passar.
• 16, 17 e 19; sem testar 18.
• 18, 19 e 20; todos aprovados.

Por que a resposta está certa

Correto: abaixo, igualdade e acima distinguem >= de > e cobrem os dois lados.

Armadilha da alternativa incorreta

Sem testar o valor exato ou um lado falso, não dá para confirmar se a fronteira está incluída.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

para regra x>0 → -1/0/1 dá não/não/sim

Concepção errada e falha observável

Testar apenas valores centrais e ignorar igualdade no limite. / Conjunto de casos não distingue >= de > ou <= de <.

Pergunta diagnóstica

Qual entrada faria uma implementação com operador errado falhar?

Pistas graduais

1. Marque o valor exato do limite. 2. Escolha um abaixo e um acima. 3. Em >=100, 100 separa o operador certo de >100.

Solução de referência explicada

Use os nove casos públicos e cubra também x>0 com entradas -1, 0 e 1 e resultados false, false, true; explique por que a igualdade falha.

Transferência em outro contexto

Escreva testes para o contrato prazo < 7 usando os valores 6, 7 e 8. Para cada entrada, registre o booleano esperado e justifique o caso da igualdade.

Evidência de domínio

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.01 Tabela de execução

TRACE · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: control.loop_trace | Requer: operators.basic, control.loop | Introduz: control.loop_trace

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 3 · depuração 2 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Trace as quatro passagens: preencha i e total, depois explique por que o próximo i não executa o corpo. Não altere o código.

Objetivo da atividade

Registre uma linha por passagem e o valor após a última atualização de i.

Explicação e modelo mental

• Modelo mental: Um for reúne inicialização, teste, corpo e atualização. A inicialização ocorre uma vez; antes de cada passagem o teste é avaliado; depois do corpo vem a atualização; só então o teste decide se haverá outra passagem. Um contador responde quantas vezes ou quais valores visitamos. Um acumulador conserva um resultado parcial, como uma soma, e recebe o estado anterior mais a contribuição da passagem atual. A condição falsa também é um estado importante. Se i chega a 4 e o corpo termina, i++ produz 5; como 5 <= 4 é falso, não existe uma quinta linha de saída. Aqui os loops usam números e estado simples. for...of será apresentado depois, junto com coleções, para não misturar repetição com a habilidade de percorrer arrays. No cabeçalho for (let i = 1; i <= 4; i++), os dois pontos e vírgulas separam início, teste e atualização. i++ soma uma unidade; total += i equivale a total = total + i. As chaves contêm o corpo repetido. i existe apenas dentro desse for; total, declarado fora, conserva a soma depois dele.
• Exemplo resolvido: A inicialização cria i=2 e soma=0. A primeira passagem soma 2 e imprime 2, 2; a segunda soma 3 e imprime 3, 5; a terceira soma 4 e imprime 4, 9. Depois da terceira passagem i vira 5. O teste 5 <= 4 falha e o corpo não executa de novo.
• Armadilha comum: Ler i++ como se acontecesse antes do primeiro teste ou esquecer que o acumulador recebe o total anterior muda toda a tabela.
• Por que importa: Preencha a tabela das quatro passagens do desafio e inclua a verificação final que não entra no corpo.

Material/arquivo inicial

main.js: let total = 0;
for (let i = 1; i <= 4; i++) {
total += i;
console.log(i, total);
}

Exemplo e resultado público

linhas i,total → 1,1; 2,3; 3,6; 4,10

Bordas e erros publicados

limite i<=3 -> 1,1; 2,3; 3,6, sem quarta linha

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

No for, quando ocorre i++ em relação à condição seguinte?

• Antes da primeira condição e de todo corpo.
 (Correta) Depois do corpo e antes da próxima condição.
• Só quando o corpo usa continue.

Por que a resposta está certa

Correto: a atualização vem após o corpo; então a condição seguinte decide se haverá outra passagem.

Armadilha da alternativa incorreta

continue não é necessário: a atualização faz parte do cabeçalho do for e ocorre após o corpo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

limite i<=3 → 1,1; 2,3; 3,6, sem quarta linha

Concepção errada e falha observável

Atualizar acumulador só no final ou executar a condição uma vez a mais. / A tabela diverge na primeira iteração ou contém i=5.

Pergunta diagnóstica

Quando i é incrementado e quando total recebe i?

Pistas graduais

1. Faça colunas i e total. 2. Em cada iteração, some i antes do log. 3. Após i=4, o incremento dá 5 e a condição falha.

Solução de referência explicada

Os quatro estados são 1/1, 2/3, 3/6, 4/10; i=5 não executa corpo.

Transferência em outro contexto

Trace manualmente a soma em um for de 2 a 4. Registre o par (i,total) após cada passagem e explique a condição de parada.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.02 Trace sem arrays

TRACE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: control.continue_trace | Requer: operators.basic, control.loop, control.branch | Introduz: control.continue_trace

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Rastreie i de 1 a 5; marque em quais passagens a soma é pulada. Escreva a soma prevista e diga o que continue pula e o que continua ocorrendo.

Objetivo da atividade

Faça uma tabela i / condição / soma ou pula para as cinco passagens.

Explicação e modelo mental

• Modelo mental: continue encerra somente a passagem atual. Ele pula o restante do corpo e segue para a atualização prevista pelo for; o teste da próxima passagem ainda será feito. break tem outro efeito: encerra o laço inteiro. Compare os dois comandos em uma tabela de estado antes de usá-los em código. Em um while, continue não atualiza nada automaticamente. Se a condição depende de uma variável que não muda antes do continue, o laço pode ficar preso.
• Exemplo resolvido: i=1 e i=2 entram no acumulador; soma chega a 3. Em i=3, continue pula soma += i. O for ainda atualiza i para 4. i=4 e i=5 entram; a soma final é 12. Se fosse break em i=3, o laço terminaria com soma 3.
• Armadilha comum: Dizer que continue “para o loop” confunde uma passagem pulada com a saída completa e costuma esconder a atualização do contador.
• Por que importa: Marque cada passagem como soma, pula ou encerra e depois compare continue com break.

Material/arquivo inicial

main.js: let soma = 0;
for (let i = 1; i <= 5; i++) {
if (i % 2 === 0) continue;
soma += i;
}
console.log(soma);

Exemplo e resultado público

i=1,3,5 somados → 9; pares pulados

Bordas e erros publicados

limite i<=4 -> 4; continue em i=2/4 pula só soma, não o incremento do for

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em for, o que acontece com a atualização após continue?

• O contador fica parado até executar o corpo inteiro.
• O laço todo termina.
 (Correta) Ela ainda ocorre antes da próxima condição.

Por que a resposta está certa

Correto: continue pula as instruções restantes do corpo, não a atualização prevista no for.

Armadilha da alternativa incorreta

continue não congela nem encerra o loop; no for, a atualização ainda prepara a próxima condição.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

limite i<=4 → 4; continue em i=2/4 pula só soma, não o incremento do for

Concepção errada e falha observável

Pensar que continue encerra o loop ou impede atualização automática de i. / A resposta inclui pares ou para em i=2.

Pergunta diagnóstica

Qual instrução do corpo é pulada quando a condição é verdadeira?

Pistas graduais

1. Liste i de 1 a 5. 2. Marque pares como pulados. 3. O incremento do for ainda ocorre após continue.

Solução de referência explicada

Some apenas 1+3+5=9; o loop termina normalmente depois de i=5.

Transferência em outro contexto

Percorra 1..6 e use continue para pular os valores que não são múltiplos de 3; some os demais e explique em que ponto cada passagem segue para a próxima.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.03 Monte o FizzBuzz restrito

PARSONS · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: control.fizzbuzz | Requer: operators.basic, control.branch, control.loop | Introduz: control.fizzbuzz

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 3 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Reordene os blocos para classificar os inteiros de 1 a 30. A combinação das regras deve ser verificada antes das condições individuais. Depois confira casos em que há sobreposição.

Objetivo da atividade

Anote a ordem das quatro regras da mais específica ao caso restante antes de mover qualquer bloco.

Explicação e modelo mental

• Modelo mental: Em if/else if, o primeiro ramo verdadeiro impede que os ramos seguintes sejam avaliados para aquele valor. Por isso a condição mais específica deve vir antes da condição mais ampla. Uma regra combinada, como ser múltiplo de 3 e de 5, precisa aparecer antes de “múltiplo de 3” e “múltiplo de 5”. Caso contrário, 15 recebe apenas uma etiqueta e nunca chega à combinação. Depois de ordenar, confira um caso de cada ramo e um caso de sobreposição. O restante precisa continuar explícito no else.
• Exemplo resolvido: 15 é divisível por 3 e por 5, então a primeira condição é verdadeira. O ramo “ambos” executa e os demais ramos não são tentados para esse número. Se o ramo de 3 viesse primeiro, 15 seria capturado cedo demais.
• Armadilha comum: Ordenar as condições pela ordem do enunciado, em vez de pela especificidade, esconde os casos combinados.
• Por que importa: Reordene os blocos do desafio e escreva quais valores testam a combinação e cada caso individual.

Material/arquivo inicial

main.js: // Os blocos abaixo devem ser ordenados antes da execução.
// Registre a ordem escolhida no campo de reflexão.

Exemplo e resultado público

i=1,3,5,15 → 1,Fizz,Buzz,FizzBuzz

Bordas e erros publicados

i=30 -> FizzBuzz; i=10 -> Buzz; quantidade de linhas -> 30

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual condição deve vir primeiro numa cadeia de regras sobrepostas?

 (Correta) A mais específica, antes das que também a incluem.
• A mais ampla, para terminar cedo.
• A última citada no enunciado.

Por que a resposta está certa

Correto: somente o primeiro ramo verdadeiro executa, então a combinação precisa ser checada antes das condições individuais.

Armadilha da alternativa incorreta

Uma condição ampla pode esconder um caso combinado; por isso a mais específica vem antes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

i=30 → FizzBuzz; i=10 -> Buzz; quantidade de linhas -> 30

Concepção errada e falha observável

Testar múltiplo de 3 antes da combinação e nunca chegar a FizzBuzz. / 15 produz Fizz em vez de FizzBuzz.

Pergunta diagnóstica

Qual condição mais específica deve vir primeiro?

Pistas graduais

1. Liste as quatro possibilidades. 2. Verifique i=15 antes de ordenar ramos. 3. Uma condição específica deve preceder a condição que já a inclui.

Solução de referência explicada

A ordem correta é combinação, 3, 5, caso restante; gere exatamente 1..30.

Transferência em outro contexto

Organize regras para Ping e Pong de 1 a 24: múltiplos de 4, múltiplos de 6 e a combinação. Escreva a ordem das condições para que a regra combinada não seja escondida por uma individual.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.06 Loop que não termina

DEBUG · 16 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: control.loop_termination | Requer: operators.basic, control.loop, control.branch | Introduz: control.loop_termination

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 4 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Investigue por que a mensagem se repete: trace fila em cada ramo, encontre o caminho sem progresso e faça correção mínima também para continue.

Objetivo da atividade

Trace fila nas duas primeiras passagens e indique qual linha deveria fazê-la progredir.

Explicação e modelo mental

• Modelo mental: while testa a condição antes de cada passagem e não tem atualização automática. Cada caminho que pode repetir precisa mudar o estado usado pela condição. Se um continue acontece antes de fila--, a execução volta ao teste com fila igual a 2. A condição fila > 0 continua verdadeira e a mensagem se repete para sempre. break encerra o while imediatamente quando uma regra de parada antecipada foi encontrada. Ele não substitui a atualização necessária nos caminhos que continuam. No starter com erro: a primeira passagem muda fila de 3 para 2; a segunda entra no if, imprime "pausa" e pula fila--; a terceira começa outra vez em 2. O controle de repetição depende da mudança de estado, não da impressão de mensagens.
• Exemplo resolvido: O teste inicial permite a passagem com saldo 3. saldo-- produz 2; como ainda não é 1, o while testa novamente. A próxima passagem produz 1 e break termina o laço; o console imprime 1.
• Armadilha comum: Mover a atualização para depois de um continue cria um caminho que repete sem progresso; em while, isso precisa ser prevenido explicitamente.
• Por que importa: Trace fila nos caminhos normal e continue, corrija o starter e prove que a fila chega a zero.

Material/arquivo inicial

main.js: let fila = 3;
while (fila > 0) {
if (fila === 2) {
console.log("pausa");
continue;
}
fila--;
}
console.log(fila);

Exemplo e resultado público

starter → repete pausa sem terminar em fila=2; corrigido -> fila chega a 0 e loop termina

Bordas e erros publicados

fila inicial 0 -> zero iterações; fila inicial 1 -> uma redução e termina

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que um caminho de while precisa garantir para permitir término?

• Que continue diminua o contador automaticamente.
 (Correta) Que o estado possa progredir até tornar falsa a condição.
• Que a condição seja falsa antes da primeira passagem.

Por que a resposta está certa

Correto: o código deve fazer a condição mudar nos caminhos repetidos; while não avança contador sozinho.

Armadilha da alternativa incorreta

continue não altera estado. Um caminho que repete sem progresso pode manter a condição verdadeira para sempre.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

fila inicial 0 → zero iterações; fila inicial 1 -> uma redução e termina

Concepção errada e falha observável

Achar que continue atualiza condição ou contador automaticamente no while. / Timeout do runner encontra fila fixa em 2.

Pergunta diagnóstica

Qual variável controla a condição e em qual ramo ela deixa de mudar?

Pistas graduais

1. Trace fila em 3,2,2. 2. Atualize fila também antes do continue, ou reestruture o ramo. 3. Em while, só seu código modifica a variável de controle.

Solução de referência explicada

Mova fila-- para início/fim comum antes de continue; cada iteração reduz fila exatamente uma vez.

Transferência em outro contexto

Corrija loop de reembolso que pula a redução do saldo em um ramo; mostre como todo caminho progride.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.07 Contador por intervalo

MAKE · 14 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: control.interval_count | Requer: operators.basic, control.loop, control.branch | Introduz: control.interval_count

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Complete o loop que percorre os inteiros de 1 até n inclusive. Incremente contagem quando i>limite && i%divisor===0 e imprima o total. Os valores são inteiros válidos, n é não negativo e divisor é diferente de zero; a validação de argumentos fica fora desta atividade.

Objetivo da atividade

Antes de editar, escreva a condição composta e identifique o primeiro e o último valor visitados.

Explicação e modelo mental

• Modelo mental: O loop visita cada inteiro de 1 até n, inclusive. O contador i representa o candidato atual; contagem representa quantos candidatos já satisfizeram a regra. A expressão i > limite && i % divisor === 0 exige as duas condições. O operador % produz resto zero quando i é divisível por divisor. A inicialização contagem=0 é o estado antes de qualquer candidato. Só some uma unidade quando o candidato passar pelos dois testes.
• Exemplo resolvido: i=20 falha em i > limite, mesmo sendo divisível por 4. i=24 e i=28 passam nos dois testes e aumentam contagem para 1 e depois 2. i=30 é visitado porque o intervalo é inclusivo, mas não altera contagem.
• Armadilha comum: Usar >= inclui o próprio limite; usar i < n deixa de visitar o último candidato.
• Por que importa: Escreva a condição composta e identifique o primeiro e o último valor visitados antes de editar.

Material/arquivo inicial

main.js: const n = 10;
const limite = 4;
const divisor = 2;
let contagem = 0;
for (let i = 1; i <= n; i++) {
// conte quando i é maior que limite e divisível por divisor
}
console.log(contagem);

Exemplo e resultado público

n=10, limite=4, divisor=2 → 3 (6,8,10) | n=10, limite=5, divisor=2 → 3 (6,8,10) | n=10, limite=6, divisor=2 → 2 (8,10)

Bordas e erros publicados

n=0 -> 0; limite=10 -> 0; n=15,limite=8,divisor=5 -> 2 (10,15)

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual expressão exige valor maior que limite e divisível por divisor?

• i >= limite || i % divisor === 0
• i > limite || i / divisor === 0
□ (Correta) i > limite && i % divisor === 0

Por que a resposta está certa

Certo: && exige ambas as regras e resto zero expressa divisibilidade.

Armadilha da alternativa incorreta

O contrato requer simultaneamente comparação estrita e resto zero, então use && e %.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

n=0 → 0; limite=10 -> 0; n=15,limite=8,divisor=5 -> 2 (10,15)

Concepção errada e falha observável

Usar >= em vez de > ou esquecer o último valor N. / A contagem inclui i igual ao limite ou deixa de fora o valor n.

Pergunta diagnóstica

O limite é estrito? O intervalo inclui N?

Pistas graduais

1. Percorra i=1 até i<=n. 2. Exija i>limite e resto zero. 3. Em 1..10, maior que 4 e par: 6,8,10.

Solução de referência explicada

Complete o loop mantendo os valores já fornecidos. Para cada i de 1 até n inclusive, incremente contagem somente quando i>limite && i%divisor===0. Neste exercício, n é inteiro não negativo e limite/divisor são inteiros com divisor diferente de zero; o avaliador fornece apenas entradas válidas. Imprima a contagem ao final.

Transferência em outro contexto

Na inspeção de uma linha de embalagens, os IDs dos pacotes vão de 1 a 15. Em um loop, conte quantos são maiores que 8 e divisíveis por 5.

Evidência de domínio

Resultado observável: n=10,limite=4,divisor=2 -> 3 (6,8,10); limite=5 -> 3 (6,8,10). Borda a conferir: n=0 -> 0; limite=10 -> 0; n=15,limite=8,divisor=5 -> 2 (10,15). Entrega esperada: Complete o loop mantendo os valores já fornecidos. Para cada i de 1 até n inclusive, incremente contagem somente quando i>limite && i%divisor===0. Neste exercício, n é inteiro não negativo e limite/divisor são inteiros com divisor diferente de zero; o avaliador fornece apenas entradas válidas. Imprima a contagem ao final. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.08 Transferencia - simulador de fila

MODIFY · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: control.queue_state | Requer: operators.basic, control.loop, control.branch | Introduz: control.queue_state

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Complete a simulação sem arrays. No cenário-base chega uma pessoa por tick e o atendimento é agendado nos ticks pares. A agenda pode informar zero chegadas em um tick; some as chegadas do tick antes do serviço e, se houver atendimento agendado e fila>0, atualize fila e atendidos. Antes de codificar, trace os ticks 1–4 do cenário-base.

Objetivo da atividade

Para o cenário-base, preencha quatro linhas com tick, uma chegada, se há atendimento agendado, fila final e total atendido; escreva a previsão antes de executar.

Explicação e modelo mental

• Modelo mental: Uma simulação é uma sequência de transições: o estado atual recebe uma entrada, aplica a regra do tick e produz o próximo estado. fila mede pessoas esperando; atendidos mede serviços efetivos. A ordem é parte do contrato. Primeiro some as chegadas do tick; depois, se houver atendimento agendado e fila > 0, retire uma pessoa e incremente atendidos. Use uma tabela para separar tick, chegada, agenda, fila final e total atendido. Isso permite conferir que a fila nunca fica negativa e que atendimento vazio não conta.
• Exemplo resolvido: O estado começa com fila 1 e recebe duas chegadas, ficando com fila 3. Como há atendimento e fila > 0, uma pessoa sai: fila fica 2 e atendidos fica 1. Um atendimento agendado só é efetivo se, depois de somar as chegadas, fila > 0. Num cenário alternativo com fila=0 e chegada=0, atendidos permanece zero.
• Armadilha comum: Atender antes de somar a chegada muda o resultado do mesmo tick; subtrair sem testar fila > 0 cria um estado impossível.
• Por que importa: Trace os quatro ticks do cenário-base e só depois escreva o loop da simulação. Use chegadasPorTick para a quantidade de chegadas: o cenário-base usa 1 e os casos também conferem 0. Imprima fila e atendidos após todos os ticks.

Material/arquivo inicial

main.js: const ticks = 20;
const chegadasPorTick = 1;
let fila = 0;
let atendidos = 0;
for (let tick = 1; tick <= ticks; tick++) {
// uma chegada por tick; atendimento nos ticks pares
}
console.log(fila, atendidos);

Exemplo e resultado público

chega 1 por tick e atende 1 em ticks pares → após 4 ticks fila=2, atendidos=2

Bordas e erros publicados

sem chegada em tick 5 e atendimento programado -> fila nunca negativa; atendimento vazio não aumenta atendidos

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se há chegada e atendimento no mesmo tick, qual ordem o contrato pede?

 (Correta) Somar chegada e atender se houver alguém.
• Atender primeiro e ignorar a chegada do tick.
• Subtrair mesmo se a fila estiver vazia.

Por que a resposta está certa

Correto: a chegada acontece primeiro; então se verifica se há alguém para atender.

Armadilha da alternativa incorreta

O contrato determina chegada seguida de serviço efetivo, que não reduz fila vazia.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

sem chegada em tick 5 e atendimento programado → fila nunca negativa; atendimento vazio não aumenta atendidos

Concepção errada e falha observável

Atender pessoa inexistente ou perder chegada quando atendimento ocorre no mesmo tick. / Fila fica negativa ou total atendidos excede chegadas.

Pergunta diagnóstica

Em cada tick, em que ordem o contrato aplica chegada e atendimento?

Pistas graduais

1. Registre fila antes/depois de cada tick. 2. Primeiro some chegadas, depois atenda se houver alguém. 3. Com fila 0 e atendimento, nenhum atendimento efetivo ocorre.

Solução de referência explicada

Para cada tick, acrescente chegadas; se tick par e fila>0, reduza fila e aumente atendidos. Não use arrays.

Transferência em outro contexto

Implemente uma simulação de seis ticks de chamados: estado inicial fila=0 e atendidos=0; chegam chamados nos ticks 1, 2, 4 e 6; há atendimento agendado nos ticks 2, 3, 5 e 6. Em cada tick, some as chegadas antes do serviço e só atenda se fila>0. Verifique ao longo da simulação que a fila nunca fica negativa e que atendidos só aumenta quando um chamado é efetivamente atendido.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.09 Microprojeto - batalha por turnos

PROJECT · 55 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: control.turn_battle | Requer: operators.basic, control.loop, control.branch | Introduz: control.turn_battle

Eixos: novidade 3 · sintaxe 3 · composição 5 · estado 5 · depuração 4 · arquivos 4 · leitura 4 · autonomia 5 · ferramentas 3

VISÃO DO ALUNO

Implemente os turnos: heroi ataca primeiro, monstro só responde se ainda estiver vivo. HP nunca fica abaixo de zero. Limite a 20 turnos e, ao parar, indique o vencedor ou empate se ambos ainda estiverem vivos pelo limite. O dano pode ser zero.

Objetivo da atividade

Liste a ordem das ações e condições de parada; anote quando um derrotado deixa de agir e o que ocorre se os danos forem zero.

Explicação e modelo mental

• Modelo mental: Um turno é uma sequência de mudanças de estado. Neste jogo, o herói ataca primeiro; somente se o monstro continuar com HP positivo ele pode responder. Após subtrair dano, se o HP ficar menor que zero, atribua zero: if (hp < 0) hp = 0. Isso mantém a regra de HP não negativo usando uma condição já aprendida. Reavalie se o monstro está vivo antes de permitir a resposta. O limite de 20 turnos garante parada mesmo quando dano é zero. Se ambos continuarem vivos ao atingir o limite, o resultado é empate.
• Exemplo resolvido: Estado inicial: herói=5, monstro=4. No turno 1 o ataque deixa monstro=2; ele responde e deixa herói=4. turnos passa a 1. No turno 2 o ataque deixa monstro=0. O teste monstro > 0 falha, então não há resposta; herói continua 4 e turnos passa a 2. O próximo teste do while falha porque monstro não está vivo. A saída é 4, 0, 2. Se ninguém causasse dano, o limite ainda encerraria o jogo.
• Armadilha comum: Testar HP apenas no começo do turno permite contra-ataque de um monstro que acabou de ser derrotado; esquecer o limite pode prender dano zero.
• Por que importa: Escreva primeiro a ordem das ações e as três condições de parada; depois implemente e teste vitória, derrota e empate.

Material/arquivo inicial

main.js: let heroi = 12;
let monstro = 9;
let turnos = 0;
const maxTurnos = 20;
const danoHeroi = 3;
const danoMonstro = 2;
// implemente a batalha e produza heroi, monstro, turnos e resultado
let resultado = "";
// escolha "heroi", "monstro" ou "empate" ao encerrar
console.log(heroi, monstro, turnos, resultado);

Exemplo e resultado público

HP heroi=12/monstro=9 → após 3 ataques do heroi: heroi=8, monstro=0, monstro não contra-ataca no último turno; turnos=3

Bordas e erros publicados

heroi=2,monstro=9,danoHeroi=3,danoMonstro=2 -> heroi chega a 0 no primeiro contra-ataque e monstro vence;
heroi=12,monstro=9,danoHeroi=0,danoMonstro=0 -> após 20 turnos HP permanece 12/9 e resultado é empate por limite

Teste público e aprovação visível

Confira o comportamento em cada caso; explique a primeira divergência.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Após o ataque do herói levar monstro a zero, o que ocorre?

• O monstro contra-ataca antes de verificar HP.
 (Correta) A luta termina sem contra-ataque do derrotado.
• HP fica negativo para indicar excesso.

Por que a resposta está certa

Correto: reavalie estado terminal entre as ações; quem foi derrotado não age.

Armadilha da alternativa incorreta

Verifique a condição após ataque e antes de permitir a resposta do monstro.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

heroi=2,monstro=9,danoHeroi=3,danoMonstro=2 → heroi chega a 0 no primeiro contra-ataque e monstro vence; heroi=12,monstro=9,danoHeroi=0,danoMonstro=0 -> após 20 turnos HP permanece 12/9 e resultado é empate por limite

Concepção errada e falha observável

Permitir contra-ataque de monstro já derrotado ou loop infinito com dano zero. / HP abaixo de zero, contra-ataque após derrota ou empate omitido quando chegam a 20 turnos.

Pergunta diagnóstica

O monstro ainda está vivo antes de contra-atacar?

Pistas graduais

1. Defina ordem de cada turno. 2. Após ataque do herói, teste monstro>0 antes da resposta. 3. Inclua limite de turnos e clamp de HP em zero.

Solução de referência explicada

Enquanto ambos vivos e turnos<20: monstro=max(0,monstro-danoHeroi); se monstro>0, heroi=max(0,heroi-danoMonstro); incremente turnos. Ao parar, informe vitória de quem permanece vivo ou empate se turnos=20 e ambos ainda vivos.

Transferência em outro contexto

No simulador de robôs, A começa com energia 8 e B com 6; A causa 3 de dano por golpe e B causa 2. Em cada rodada A ataca primeiro; B responde somente se ainda tiver energia positiva. Reduza energia sem passar de zero e encerre ao derrotar um robô ou após no máximo 5 rodadas. Simule a luta e informe o estado final, vencedor ou empate e rodada de encerramento.

Evidência de domínio

Resultado observável: HP heroi=12/monstro=9 -> após 3 ataques do heroi: heroi=8, monstro=0, monstro não contra-ataca no último turno; turnos=3. Borda a conferir: heroi=2,monstro=9,danoHeroi=3,danoMonstro=2 -> heroi chega a 0 no primeiro contra-ataque e monstro vence; heroi=12,monstro=9,danoHeroi=0,danoMonstro=0 -> após 20 turnos HP permanece 12/9 e resultado é empate por limite. Entrega esperada: Enquanto ambos vivos e turnos<20: monstro=max(0,monstro-danoHeroi); se monstro>0, heroi=max(0,heroi-danoMonstro); incremente turnos. Ao parar, informe vitória de quem permanece vivo ou empate se turnos=20 e ambos ainda vivos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe padrão disponível no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 03.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

03.10 Ternário ou if?

MAKE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: control.conditional_expression | Requer: operators.basic, control.branch | Introduz: control.conditional_expression

Eixos: novidade 2 · sintaxe 3 · composição 2 · estado 1 · depuração 2 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Implemente resumirNota(pontos). Use uma expressão ternária para faixa ("aprovar" quando pontos>=60, senão "revisar"). Use if/else if/else para situacao: "inválida" abaixo de 0, "em revisão" abaixo de 60 e "liberada" nos demais casos. Compare qual forma deixa cada decisão mais clara.

Objetivo da atividade

Antes de codificar, marque qual decisão tem duas alternativas e qual tem três; preveja os resultados para -1, 0, 45, 60 e 100.

Explicação e modelo mental

• Modelo mental: O ternário condicao ? valorA : valorB é uma expressão: escolhe um valor entre duas alternativas. if/else organiza caminhos; quando há várias condições, etapas ou efeitos, os blocos costumam deixar a decisão mais legível. Escolha pela clareza da regra, não pela quantidade de caracteres.
• Exemplo resolvido: Para uma nota, const faixa = pontos >= 60 ? "aprovar" : "revisar" nomeia uma escolha simples. Para três estados, if (pontos < 0) ... else if (pontos < 60) ... else ... mostra a ordem das faixas sem aninhar ternários.
• Armadilha comum: Ternários aninhados podem comprimir várias decisões e ocultar a ordem dos casos. Um if longo também pode ser desnecessário quando só se escolhe entre dois valores.
• Por que importa: Código legível facilita revisar regras de negócio e perceber limites ou casos que ficaram de fora.

Material/arquivo inicial

main.js: export function resumirNota(pontos) {
const faixa = "";
let situacao = "";
return { faixa, situacao };
}

Exemplo e resultado público

pontos=75 → {faixa:"aprovar", situacao:"liberada"}; pontos=45 -> {faixa:"revisar", situacao:"em revisão"}

Bordas e erros publicados

pontos=0 -> {faixa:"revisar", situacao:"em revisão"}; pontos=-1 -> {faixa:"revisar", situacao:"inválida"}; pontos=100 -> {faixa:"aprovar", situacao:"liberada"}

Teste público e aprovação visível

Critérios desta atividade: faixa usa uma expressão ternária legível com duas alternativas. situacao distingue as três faixas com if/else if/else na ordem correta. A solução preserva os limites 0, 60 e 100 e explica qual forma comunica melhor cada regra.

Restrições e documentação

Mantenha exatamente os nomes e os valores de retorno do contrato.; Não use ternários aninhados para a situação com três resultados.. Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

export function resumirNota(pontos) é um invólucro pronto para que o avaliador importe a função; preserve export, o nome e o parâmetro e altere somente o corpo indicado. Funções exportadas para módulos são aprofundadas na etapa 14. | { faixa, situacao } cria um objeto com duas propriedades; a forma curta usa cada variável também como nome da propriedade. Objetos são aprofundados na etapa 06; aqui, mantenha os nomes pedidos pelo contrato.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual forma costuma comunicar melhor uma escolha de três estados com condições em ordem?

• Ternários aninhados mesmo quando cada nível tem uma condição diferente
• Escolher sempre a forma com menos linhas
 (Correta) if/else if/else com um ramo por estado

Por que a resposta está certa

Correto: os blocos expõem a ordem dos três casos; ternário cabe melhor numa escolha curta entre dois valores.

Armadilha da alternativa incorreta

A quantidade de linhas não determina legibilidade; compare quantos casos existem e se a regra fica fácil de seguir.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

pontos=0 → {faixa:"revisar", situacao:"em revisão"}; pontos=-1 -> {faixa:"revisar", situacao:"inválida"}; pontos=100 -> {faixa:"aprovar", situacao:"liberada"}

Concepção errada e falha observável

Usar ternários aninhados para esconder uma sequência de três decisões; escolher if por ser mais legível quando há vários ramos. / faixa deve ter duas alternativas; situacao precisa distinguir inválida, em revisão e liberada, sem deixar pontos negativos passar.

Pergunta diagnóstica

Quantos resultados cada decisão tem, e a escolha é um valor ou uma sequência de ações?

Pistas graduais

1. Para faixa, há apenas duas alternativas e uma expressão curta. 2. Para situacao, ordene primeiro pontos<0, depois pontos<60 e por fim o restante. 3. Use ternário para produzir faixa; use if/else if/else para situacao e devolva as duas propriedades.

Solução de referência explicada

Um ternário expressa pontos>=60 ? "aprovar" : "revisar"; o if/else if/else torna explícitas as três faixas de situacao.

Transferência em outro contexto

No painel de entregas, use escolha binária pronta=true e atraso=0. Resultado esperado -> {texto:"pronta", situacao:"no prazo"}; com atraso=-1 -> situacao:"inválido"; com atraso=2 -> situacao:"atrasado". Use ternário na escolha binária, if/else na classificação e justifique a legibilidade.

Evidência de domínio

Critérios desta atividade: faixa usa uma expressão ternária legível com duas alternativas. situacao distingue as três faixas com if/else if/else na ordem correta. A solução preserva os limites 0, 60 e 100 e explica qual forma comunica melhor cada regra. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Ternário e if/else fazem parte do JavaScript padrão no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04 Funções e decomposição

13 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 04.01 · 04.02 · 04.03 · 04.04 · 04.05 · 04.06 · 04.07 · 04.08 · 04.09 · 04.10 · 04.11 · 04.12 · 04.13

04.01 Preveja retornos

PREDICT · 13 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: functions.return | Requer: control.branch | Introduz: functions.return

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 4 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Antes da tabela, compare duas formas de declarar uma função: function dobro(x){return x*2} e const dobro = x => x*2. Ambas recebem x e devolvem o dobro; uma declaração de função tem nome após function e uma arrow é atribuída a um binding. Para cada chamada do starter, registre o valor devolvido e diferencie ausência de return de retorno explícito de null. Siga o caminho executado; não trate console ou expressão solta como retorno.

Objetivo da atividade

Preencha para cada chamada o valor previsto e marque se alcança return.

Explicação e modelo mental

• Modelo mental: A chamada produz o valor indicado por return. Se o caminho termina sem executar return, JavaScript devolve undefined; null é um valor explícito.
• Exemplo resolvido: Em function area(lado){return lado*lado}, area(4) calcula 4*4 e vale 16. A forma const area = lado => lado*lado também devolve 16. O valor pode compor outra expressão sem ser impresso.
• Armadilha comum: Expressão solta em função com chaves não vira retorno. Arrow com corpo de expressão retorna implicitamente; com chaves, precisa de return.
• Por que importa: Prever retornos permite compor funções, verificar resultados e separar cálculo de apresentação.

Material/arquivo inicial

main.js: export function preverRetornos() {
function a() { return 2; }
function b() { 2; }
function c(x) { return x * 2; }
function d(x) { if (x > 0) return x; }
function e() { return null; }
function f() { return; }
const g = x => x + 1;
const h = x => { x + 1; };
return {a: a(), b: b(), c: c(3), dPositivo: d(4), dNegativo: d(-1), e: e(), f: f(), g: g(1), h: h(1)};
}

const resultado = preverRetornos();
console.log(JSON.stringify(Object.fromEntries(Object.entries(resultado).map(([chave, valor]) => [chave, valor === undefined ? "undefined" : valor]))));

Exemplo e resultado público

a() → 2; b() -> undefined; c(3) -> 6; d(0) -> undefined; e() -> null; f() -> undefined; g(2) -> 3; h(2) -> undefined

Bordas e erros publicados

d(-1) -> undefined; d(2) -> 2; função que retorna null -> null, não undefined

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao depurar uma função chamada dentro de outra expressão, qual evidência mostra o valor que a expressão consumirá?

 (Correta) O valor retornado pela chamada, distinguindo-o de mensagens escritas como efeito colateral.
• A última mensagem exibida no console, independentemente de onde foi produzida.
• O valor da variável local com nome mais parecido com o da função.

Por que a resposta está certa

Correto: a expressão recebe o retorno da chamada; efeitos de console são observações separadas.

Armadilha da alternativa incorreta

Uma mensagem exibida não é automaticamente o valor retornado. Siga o valor da chamada até o consumidor.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

d(-1) → undefined; d(2) -> 2; função que retorna null -> null, não undefined

Concepção errada e falha observável

Confundir expressão no corpo com return ou assumir retorno em todo ramo. / O aluno prevê 2 para b() ou 0 para d(0).

Pergunta diagnóstica

Há return no caminho realmente executado?

Pistas graduais

1. Sublinhe cada return. 2. Siga o ramo para a entrada específica. 3. Uma função sem return explícito devolve undefined.

Solução de referência explicada

a devolve 2; b/f/h não retornam valor; c/g calculam; d só retorna quando x>0; e retorna null explicitamente.

Transferência em outro contexto

Preveja o retorno de uma função de conversão que termina sem return fora de um if.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.02 Retorno vs efeito

READ · 14 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: functions.effects_purity | Requer: control.branch | Introduz: functions.effects_purity

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 3 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Classifique cada função em dois eixos: retorno útil e pureza. Justifique pela dependência de estado externo, mutação ou efeito observável; return não prova pureza.

Objetivo da atividade

Monte uma tabela função/retorno/efeito/pureza e justifique cada linha com evidência.

Explicação e modelo mental

• Modelo mental: Função pura depende só dos argumentos, repete o resultado para as mesmas entradas e não causa efeitos observáveis. Retorno e pureza são eixos diferentes.
• Exemplo resolvido: Uma função calcularTaxa(base, percentual) que só usa os dois argumentos e devolve o cálculo é pura: mesmas entradas, mesmo resultado, sem alterar estado. Já registrarPedido(id) que acrescenta o id a um histórico compartilhado causa um efeito observável.
• Armadilha comum: Ter return não basta para ser puro. Ler estado externo, imprimir ou alterar argumento/estado também caracteriza efeito.
• Por que importa: Identificar efeitos torna testes e comportamento mais previsíveis e localiza dependências do ambiente.

Material/arquivo inicial

main.js: const dobro=x=>x*2; let n=0; const conta=()=>++n; const imprime=x=>console.log(x); const copia=a=>[...a]; const muda=a=>{a.push(1);return a.length}; const aleatorio=()=>Math.random();

Exemplo e resultado público

dobro → puro/retorna; conta -> impuro/retorna; imprime -> impuro/retorna undefined; copia -> puro; muda -> impuro; aleatorio -> impuro/não determinístico

Bordas e erros publicados

função que altera array recebido -> impura; função que chama Math.random -> impura quanto ao resultado

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

[...a] cria um array novo com os mesmos itens de a (spread, aprofundado na etapa 06); o array original não muda. | a.push(1) acrescenta 1 ao final do próprio array a e devolve o novo tamanho; quem tiver referência a a vê a mudança (arrays na etapa 05). | ++n soma 1 a n e devolve o valor já somado; n++ também soma, mas devolve o valor anterior.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

quadrado(n) retorna n*n usando só n, sem efeitos. Como classificar?

• Impura porque recebe argumento.
 (Correta) Pura e retorna um valor.
• Sem retorno porque não usa console.log.

Por que a resposta está certa

Correto: é determinística a partir da entrada e não altera nem observa estado externo.

Armadilha da alternativa incorreta

Pureza não depende de haver argumento ou log; esse cálculo retorna um valor e não produz efeitos externos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

função que altera array recebido → impura; função que chama Math.random -> impura quanto ao resultado

Concepção errada e falha observável

Tratar console.log como valor retornado ou pureza apenas como ausência de mutação visível. / Classificação diz que imprime retorna x ou que Math.random é determinístico.

Pergunta diagnóstica

A função muda/observa estado externo, ou só depende de argumentos?

Pistas graduais

1. Separe retorno de efeito. 2. Um log é efeito observável. 3. Pureza exige mesmo resultado para as mesmas entradas e nenhum efeito observável.

Solução de referência explicada

Classifique os seis snippets pelos dois eixos; explicite estado externo, mutação e saída de console.

Transferência em outro contexto

Compare uma função que recebe uma semente como argumento com outra que chama Math.random; justifique efeitos e determinismo.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.03 Parsons de decomposição

PARSONS · 14 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: functions.decomposition | Requer: control.branch | Introduz: functions.decomposition

Eixos: novidade 2 · sintaxe 3 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Organize os blocos reconhecendo a diferença entre criar uma arrow function e executar seu corpo. A, B e C podem ser declarados em qualquer ordem; a chamada D deve acontecer depois das três declarações. Indique duas ordens válidas para A, B e C.

Objetivo da atividade

Desenhe setas de cada função usada para quem a chama; escreva uma ordem válida antes de mover os blocos.

Explicação e modelo mental

• Modelo mental: Uma função composta usa outras funções disponíveis no contexto. Dependências formam uma ordem parcial: blocos independentes não têm precedência entre si.
• Exemplo resolvido: Num gerador de mensagem, validarCodigo e buscarDescricao são auxiliares independentes; montarMensagem usa as duas quando é chamada. A declaração de montarMensagem pode vir antes das auxiliares, mas ambas precisam estar inicializadas antes da chamada; entre si, podem trocar de ordem.
• Armadilha comum: Ordenar pela posição no enunciado ou misturar cálculo e apresentação esconde as dependências reais.
• Por que importa: Decompor torna regras testáveis e permite mudar cálculo ou formato sem reescrever a composição.

Material/arquivo inicial

main.js: // A reconstrução precisa aplicar a conversão antes de formatar o valor.
export const exibir = celsius => formatar(celsius);
export const cParaF = celsius => celsius * 9 / 5 + 32;
export const formatar = fahrenheit => ${fahrenheit} °F;

Exemplo e resultado público

A, B e C podem ser declarados em qualquer ordem; execute D depois das três inicializações → 32 °F

Bordas e erros publicados

Declare C antes de A e B; A e B podem trocar de ordem; D depois dos três -> exibir(100) retorna 212 °F

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é a diferença entre declarar uma arrow function e executar o corpo dela?

• O corpo executa imediatamente ao final da declaração, mesmo sem nenhuma chamada.
• A declaração executa e guarda o resultado, que as chamadas futuras apenas reutilizam.
 (Correta) A declaração disponibiliza a função; o corpo só executa quando alguma chamada a invoca.

Por que a resposta está certa

Isso: a declaração cria a função; a execução das instruções acontece quando ela é chamada.

Armadilha da alternativa incorreta

Separe a criação da função da execução do corpo. Declarar uma arrow, por si só, não executa suas instruções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Declare C antes de A e B; A e B podem trocar de ordem; D depois dos três → exibir(100) retorna 212 °F

Concepção errada e falha observável

Achar que blocos independentes precisam de ordem única ou misturar cálculo com formatação. / D executa exibir antes de C, A ou B estar inicializado, ou a conversão não preserva a unidade esperada.

Pergunta diagnóstica

O que precisa estar inicializado no momento em que D chama exibir(0)?

Pistas graduais

1. Ao criar a arrow C, seu corpo ainda não chama as auxiliares. 2. A, B e C podem ser declaradas em qualquer ordem. 3. D chama C; deixe D depois de A, B e C.

Solução de referência explicada

A, B e C podem ser declaradas em qualquer ordem, pois seus corpos de arrow só usam as referências durante a chamada; D precisa vir depois das três. A execução imprime 32 °F.

Transferência em outro contexto

No módulo de rótulos de distância, crie const arrows para converter quilômetros em milhas (1 km equivale a 0.621371 milha), formatar o valor e compor as auxiliares. Demonstre duas ordens diferentes para as três declarações; execute a chamada final somente depois que todas estiverem inicializadas.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.04 Complete a assinatura

FADED · 15 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: functions.parameters_contract | Requer: control.branch | Introduz: functions.parameters_contract

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Complete as funções para que cada chamada devolva um valor sem console.log. Respeite os parâmetros de cada contrato e mantenha clamp dentro do intervalo se min<=max.

Objetivo da atividade

Anote para cada função as entradas e o tipo/forma da saída; depois escreva um corpo por vez.

Explicação e modelo mental

• Modelo mental: Parâmetros são nomes locais para argumentos recebidos. A função calcula usando esses valores e return entrega o resultado ao chamador.
• Exemplo resolvido: areaRetangulo(largura, altura) pode retornar largura*altura. Na chamada (3,4), os parâmetros recebem 3 e 4, a expressão vale 12 e return entrega 12.
• Armadilha comum: Imprimir não substitui retorno em função composta. Em clamp, trocar limites ou aplicar a ordem errada pode deixar valor fora da faixa.
• Por que importa: Contratos claros permitem reutilizar e compor funções sem depender do console.

Material/arquivo inicial

main.js: // 04.04 · Complete cada função para que ela DEVOLVA o valor, sem console.log.
// Troque cada ___ por uma expressão. export deixa a função disponível para os testes importarem.

```
export function media(a, b) {
return ___;
}

// Mantém x dentro do intervalo [min, max] (considere min <= max).
export function clamp(x, min, max) {
return ___;
}

// "Olá, " seguido do nome.
export function saudacao(nome) {
return ___;
}
```

Exemplo e resultado público

media(2,4) → 3; clamp(10,0,5) -> 5; saudacao("Ana") -> "Olá, Ana"

Bordas e erros publicados

media(0,0) -> 0; clamp(-1,0,5) -> 0; nome vazio -> Olá,

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que diferencia console.log(resultado) de return resultado para quem chamou?

 (Correta) return entrega o valor; console.log apenas exibe.
• Ambos entregam o mesmo valor.
• console.log converte automaticamente a saída em retorno.

Por que a resposta está certa

Exato: retorno é parte do contrato; log é um efeito de apresentação.

Armadilha da alternativa incorreta

A saída visível não é o valor da chamada. Para composição, a função precisa retornar.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

media(0,0) → 0; clamp(-1,0,5) -> 0; nome vazio -> Olá,

Concepção errada e falha observável

Usar console.log em vez de return ou trocar ordem de min/max. / Função retorna undefined ou clamp deixa valor fora do intervalo.

Pergunta diagnóstica

Qual valor deve ser devolvido sem efeitos de console?

Pistas graduais

1. media soma e divide por 2. 2. clamp limita primeiro por baixo e depois por cima. 3. saudacao devolve string, inclusive para nome vazio.

Solução de referência explicada

return (a+b)/2; return Math.min(max,Math.max(min,x)); return "Olá, "+nome. O contrato fornece min<=max.

Transferência em outro contexto

Complete triplo(x) e clamp(x,min,max) para limitar entre -2 e 2. Faça ambas devolverem seus valores sem console.log e respeite o contrato de cada função.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.05 Intl em contexto

MAKE · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: functions.intl_format | Requer: control.branch | Introduz: functions.intl_format

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 1 · arquivos 2 · leitura 3 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Mantenha calcularTotal como cálculo numérico. Crie formatadores Intl.NumberFormat separados para BRL, percentual e notação compacta; formate apenas na apresentação.

Objetivo da atividade

Escreva os tipos de entrada e saída de calcularTotal e dos três formatadores antes de implementar.

Explicação e modelo mental

• Modelo mental: Cálculo e apresentação têm contratos diferentes: o primeiro mantém Number; o formatador recebe número e devolve texto localizado. Intl.NumberFormat aplica locale e opções.
• Exemplo resolvido: Para apresentar um valor decimal em pt-BR, crie um NumberFormat com estilo decimal e formate 4321.5. O resultado é texto localizado para exibição; o dado original continua Number para outras operações.
• Armadilha comum: Concatenar R$ manualmente ou retornar texto da função de cálculo mistura domínio e apresentação. Formatação não resolve precisão monetária por si só.
• Por que importa: Separar formato de cálculo permite mudar locale ou apresentação sem alterar as regras do domínio.

Material/arquivo inicial

main.js: // 04.05 · Cálculo e apresentação são contratos diferentes.
// calcularTotal devolve um Number. Os três formatadores recebem um número e devolvem texto em pt-BR.
// Substitua cada null por um new Intl.NumberFormat("pt-BR", { ...opções }) (consulte a documentação das opções).

```
export function calcularTotal(preco, quantidade) {
return preco * quantidade;
}

// moeda brasileira: R$ 20,00
export const formatadorBRL = null;

// percentual: 0,25 vira 25%
export const formatadorPercentual = null;

// notação compacta: 1200 vira algo como "1,2 mil"
export const formatadorCompacto = null;
```

Exemplo e resultado público

calcularTotal(10,2) → Number 20; formatador BRL de 20 -> texto pt-BR com símbolo R$ e 20,00

Bordas e erros publicados

Intl.NumberFormat("pt-BR",{style:"percent"}).format(0.25) -> 25%; compacto de 1200 -> representação local com milhar

Teste público e aprovação visível

Resultado observável: calcularTotal(10,2) -> Number 20; formatador BRL de 20 -> texto pt-BR com símbolo R$ e 20,00. Borda a conferir: Intl.NumberFormat("pt-BR",{style:"percent"}).format(0.25) -> 25%; compacto de 1200 -> representação local com milhar. Entrega esperada: calcularTotal retorna Number; formatadores separados para BRL, percentual e notation compact. Não concatene símbolo manualmente.

Restrições e documentação

Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

{ style: 'currency', currency: 'BRL' } é um objeto literal usado como pacote de opções nomeadas: cada chave configura um aspecto do formatador. Objetos são estudados na etapa 06; aqui basta escrever as chaves que a documentação de Intl.NumberFormat lista.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual deve ser a saída de calcularTotal quando sua responsabilidade é multiplicar preço e quantidade?

• Uma string com símbolo de moeda concatenado.
 (Correta) Um Number; texto localizado fica para um formatador.

• O resultado de console.log do valor.

Por que a resposta está certa

Correto: manter o dado numérico permite outros cálculos; formatação localizada é outra responsabilidade.

Armadilha da alternativa incorreta

Misturar texto ao cálculo dificulta composição; a função de domínio deve devolver o Number calculado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Intl.NumberFormat("pt-BR",{style:"percent"}).format(0.25) → 25%; compacto de 1200 -> representação local com milhar

Concepção errada e falha observável

Deixar função de domínio retornar texto localizado e perder valor numérico. / calcularTotal retorna string em vez de Number ou formatação usa código de moeda inválido.

Pergunta diagnóstica

A função calcula dado ou apresenta dado?

Pistas graduais

1. Mantenha cálculo numérico. 2. Use currency: 'BRL' para formato monetário. 3. Intl.NumberFormat pode ter instâncias separadas para moeda, percentual e compacto.

Solução de referência explicada

calcularTotal retorna Number; formatadores separados para BRL, percentual e notation compact. Não concatene símbolo manualmente.

Transferência em outro contexto

Use Intl.NumberFormat com locale pt-BR para apresentar uma taxa 0.1 como percentual. Preserve a taxa de domínio como Number e explique a separação entre cálculo e formatação.

Evidência de domínio

Resultado observável: calcularTotal(10,2) -> Number 20; formatador BRL de 20 -> texto pt-BR com símbolo R$ e 20,00. Borda a conferir: Intl.NumberFormat("pt-BR",{style:"percent"}).format(0.25) -> 25%; compacto de 1200 -> representação local com milhar. Entrega esperada: calcularTotal retorna Number; formatadores separados para BRL, percentual e notation compact. Não concatene símbolo manualmente. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.06 Bug de retorno

DEBUG · 17 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: functions.return_vs_log | Requer: control.branch | Introduz: functions.return_vs_log

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 4 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Localize o primeiro retorno ausente no starter. Faça a menor correção para que somaDobros combine números e mantenha efeitos de apresentação fora do cálculo.

Objetivo da atividade

Anote separadamente o que dobro exibe e o que devolve; então determine o que somaDobros recebe.

Explicação e modelo mental

• Modelo mental: Uma função usada numa expressão precisa devolver o valor esperado. Imprimir esse valor não ocupa o lugar do retorno.
• Exemplo resolvido: Se triplo(x) só imprime x*3, triplo(4) mostra 12, mas a chamada vale undefined. Outra função que soma esse resultado não recebe 12; triplo precisa retornar x*3.
• Armadilha comum: Ver números corretos no console não prova que a função os retornou: saída e retorno são canais distintos.
• Por que importa: Retornos consistentes permitem compor e testar cálculos, deixando apresentação numa borda explícita.

Material/arquivo inicial

main.js:
```
// 04.06 · Bug de retorno.
// somaDobros(2, 3) deveria devolver 10, mas devolve NaN e imprime 4 e 6.
// Faça a menor correção: o cálculo devolve valores; imprimir não faz parte do cálculo.

export function dobro(x) {
console.log(x * 2);
}

export function somaDobros(a, b) {
return dobro(a) + dobro(b);
}
```

Exemplo e resultado público

somaDobros(2,3) no starter → NaN e logs 4,6; corrigido -> 10 sem logs internos

Bordas e erros publicados

somaDobros(0,0) -> 0 após correção; dobro(-1) -> -2

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Uma função imprime 12, mas não usa return; qual valor entra numa soma?

• 12, pois foi impresso.
• null, por falta de retorno explícito.
□ (Correta) undefined

Por que a resposta está certa

Isso mesmo: console exibe 12, mas a chamada sem return vale undefined.

Armadilha da alternativa incorreta

O valor impresso não é retornado; sem return executado, outra expressão recebe undefined.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

somaDobros(0,0) → 0 após correção; dobro(-1) -> -2

Concepção errada e falha observável

Tomar saída de console como retorno da função. / Composição soma undefined+undefined.

Pergunta diagnóstica

O que dobro devolve para somaDobros?

Pistas graduais

1. Inspecione o valor retornado por dobro. 2. Troque efeito por return no cálculo. 3. Log só deve aparecer no ponto de apresentação, não na função de domínio.

Solução de referência explicada

function dobro(x){return x*2}; somaDobros devolve dobro(a)+dobro(b).

Transferência em outro contexto

Corrija uma função de comissão que só imprime o valor, mas precisa compor o cálculo do pedido.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.07 API de carrinho

MAKE · 30 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: functions.cart_api | Requer: control.branch | Introduz: functions.cart_api

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 2 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

As assinaturas usam ESM, formato de módulos usado pelo executor deste curso: export function publica uma função para o avaliador importar. Implemente as três funções para entradas válidas: valores monetários são centavos inteiros seguros não negativos (zero é válido); taxa é Number finito não negativo e fração (0.05 representa 5%). Calcule impostoCentavos=Math.round(subtotalCentavos*taxa), com meio centavo arredondado para cima; some o imposto e aplique depois o desconto fixo em centavos, com piso zero. O avaliador fornece casos válidos em que os valores monetários e os resultados permanecem inteiros seguros; não implemente tratamento de entradas fora do contrato nesta atividade.

Objetivo da atividade

Escreva contrato, unidade e saída de cada função; calcule o caso de 105 centavos com taxa 0.1 e indique o arredondamento antes de compor o desconto.

Explicação e modelo mental

• Modelo mental: Uma API pequena divide regra em funções com entradas/saídas claras. A função final compõe operações na ordem de negócio; auxiliares podem ser compreendidas e testadas isoladamente.
• Exemplo resolvido: Um programa de fidelidade pode calcular pontosBase e bonusDeCampanha separadamente; a função pontuacaoFinal chama ambas e soma os resultados. Se a regra da campanha mudar, o cálculo base permanece isolado.
• Armadilha comum: Tratar desconto como fração ou aplicá-lo antes de uma taxa sobre o subtotal muda a regra. O cupom não deve tornar total negativo.
• Por que importa: Composição mantém regra comercial consistente e permite revisar uma etapa sem duplicar a lógica.

Material/arquivo inicial

main.js: // ESM do curso: export disponibiliza nomes que o avaliador importa. Entradas válidas: valores em centavos são inteiros seguros >=0; taxa é fração finita >=0. Imposto em centavos = Math.round(subtotalCentavos*taxa), arredondando meio centavo para cima; desconto fixo em centavos vem depois; piso do total=0.
export function adicionarTaxa(subtotalCentavos,taxa){} export function aplicarCupom(valorCentavos,descontoCentavos){} export function totalFinal(subtotalCentavos,taxa,descontoCentavos){}

Exemplo e resultado público

subtotalCentavos=10000,taxa=0.1,descontoCentavos=500 → imposto 1000, adicionarTaxa 11000, aplicarCupom(11000,500) 10500, totalFinal 10500

Bordas e erros publicados

subtotalCentavos=105,taxa=0.1,descontoCentavos=11 -> imposto=Math.round(10.5)=11 e total=105;
subtotal=0,taxa=0,desconto=0 -> 0; desconto maior que valor após taxa -> 0

Teste público e aprovação visível

Critérios desta atividade: Três funções públicas: adicionarTaxa, aplicarCupom e totalFinal; cada uma retorna centavos inteiros seguros para os casos válidos publicados. Imposto arredondado com Math.round antes do desconto fixo; piso zero.

Restrições e documentação

Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

export function nome(...) deixa a função disponível para outro arquivo, como o avaliador, importá-la com import { nome } from './main.js'. Módulos são estudados na etapa 14; até lá, mantenha o export que o arquivo inicial já traz.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como reduzir erros ao combinar uma taxa percentual e um cupom de valor fixo?

 (Correta) Documentar unidades distintas e definir a base e a ordem de cada operação.
• Usar nomes genéricos para ambos e combinar os valores em uma única soma.
• Tratar os dois parâmetros como porcentagens, pois ambos alteram o preço.

Por que a resposta está certa

Correto: unidades e ordem explícitas evitam aplicar uma taxa ou desconto com significado errado.

Armadilha da alternativa incorreta

Parâmetros monetários podem ter unidades diferentes. Registre o que cada um mede, a base de cálculo e a ordem contratada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

subtotalCentavos=105,taxa=0.1,descontoCentavos=11 → imposto=Math.round(10.5)=11 e total=105; subtotal=0,taxa=0,desconto=0 -> 0; desconto maior que valor após taxa -> 0

Concepção errada e falha observável

Misturar percentual e valor fixo, arredondar imposto de modo implícito ou deixar total negativo. / Para 105 centavos e taxa 0.1 o imposto arredonda de 10.5 para 11; valores fora do domínio declarado não fazem parte desta atividade.

Pergunta diagnóstica

Em qual ordem taxa percentual e cupom fixo são aplicados?

Pistas graduais

1. Represente valores monetários como centavos inteiros seguros. 2. Taxa é fração; arredonde o imposto com Math.round(subtotalCentavos*taxa), que leva meio centavo para cima para entradas não negativas. 3. Subtraia o desconto fixo em centavos depois do imposto e limite o total a zero.

Solução de referência explicada

Valide valores monetários como centavos inteiros seguros e taxa como Number finito não negativo. impostoCentavos=Math.round(subtotalCentavos*taxa); adicionarTaxa=subtotalCentavos+impostoCentavos; aplicarCupom=Math.max(0,valorCentavos-descontoCentavos); totalFinal compõe nessa ordem e devolve centavos inteiros.

Transferência em outro contexto

Em uma API de bilheteria, crie funções para taxa percentual e voucher fixo usando centavos inteiros. Arredonde o imposto por Math.round antes de subtrair o voucher; para base=4000 centavos, taxa=0.05 e voucher=200 centavos, informe o imposto e total final.

Evidência de domínio

Critérios desta atividade: Três funções públicas: adicionarTaxa, aplicarCupom e totalFinal; cada uma retorna centavos inteiros seguros para os casos válidos publicados. Imposto arredondado com Math.round antes do desconto fixo; piso zero. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.08 Transferencia - regra por funcoes

TRANSFER · 27 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: functions.shipping_decomposition | Requer: control.branch | Introduz: functions.shipping_decomposition

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 2 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Implemente custoPeso e custoDistancia separadamente e componha frete sem duplicar fórmulas. kg e km são Numbers finitos não negativos; zero é válido. Use tarifas R$2/kg e R$0,10/km. Entradas inválidas não fazem parte desta atividade.

Objetivo da atividade

Descreva entrada, unidade e saída das três funções; desenhe como frete usa as contribuições antes de codificar.

Explicação e modelo mental

• Modelo mental: Decompor dá nome a contribuições independentes; a função de alto nível coordena e soma resultados.
• Exemplo resolvido: Um resumo diário pode chamar minutosDeLeitura e paginasConcluidas, duas funções independentes, e apresentar ambos os valores. O resumo coordena as saídas sem repetir como cada contagem é obtida.
• Armadilha comum: Copiar as fórmulas dentro de frete pode criar versões divergentes. Também é importante não confundir as unidades associadas aos parâmetros.
• Por que importa: Funções com uma responsabilidade são mais fáceis de testar e revisar quando tarifas mudam.

Material/arquivo inicial

main.js: // Entradas válidas: kg e km são Numbers finitos >=0; zero é válido. Tarifas: R$2/kg e R$0,10/km.
export function custoPeso(kg){} export function custoDistancia(km){} export function frete(kg,km){}

Exemplo e resultado público

custoPeso(2) → 4; custoDistancia(10) -> 1; frete(2,10) -> 5, com regras R$2/kg e R$0,10/km

Bordas e erros publicados

kg=0,km=0 -> 0; kg=1.5,km=2 -> 3.2

Teste público e aprovação visível

Compare competência estrutural em domínio diferente e sem ajuda.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que frete deve chamar as funções auxiliares em vez de copiar fórmulas?

• A função final não pode chamar outra função.
 (Correta) Cada regra fica em um lugar; frete compõe os resultados.
• Copiar fórmulas garante tarifas sincronizadas.

Por que a resposta está certa

Exatamente: composição reutiliza regras nomeadas e evita cópias divergentes.

Armadilha da alternativa incorreta

Uma função composta pode delegar contribuições; duplicar as regras cria mais pontos de manutenção.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

kg=0,km=0 → 0; kg=1.5,km=2 -> 3.2

Concepção errada e falha observável

Colocar tudo em uma função e esconder unidades/regras independentes. / Uma mudança de preço por km exige alterar cálculo de peso.

Pergunta diagnóstica

Quais são as duas contribuições independentes e onde se compõem?

Pistas graduais

1. Escreva função de peso primeiro. 2. Escreva função de distância depois. 3. A função final apenas soma as duas saídas validadas.

Solução de referência explicada

custoPeso=kg*2; custoDistancia=km*0.1; frete soma ambas; valida cada entrada finita e não negativa.

Transferência em outro contexto

Decomponha uma tarifa de táxi: bandeirada de 6 mais 2 por quilômetro. Escreva uma função para cada parcela e uma função total que componha ambas sem duplicar fórmulas.

Evidência de domínio

Compare competência estrutural em domínio diferente e sem ajuda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.09 Microprojeto - gerador de ficha

PROJECT · 55 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: functions.rpg_derived | Requer: control.branch | Introduz: functions.rpg_derived

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 2 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente ataque e defesa com as fórmulas ataque=base+nivel e defesa=armadura+Math.floor(nivel/2). As entradas fornecidas são números finitos não negativos cujas somas permanecem finitas; nome é string e pode ser vazio. resumo deve chamar ambas e devolver exatamente o formato nome | ataque X | defesa Y, substituindo nome e X/Y pelos valores recebidos/calculados.

Objetivo da atividade

Anote entradas/saídas de ataque e defesa, depois enumere as chamadas que resumo precisa fazer.

Explicação e modelo mental

• Modelo mental: Funções derivadas encapsulam regras numéricas; resumo compõe seus valores para apresentar. Reutilizar auxiliares evita versões diferentes do mesmo conceito.
• Exemplo resolvido: A ficha de uma estação de pesquisa pode chamar energiaDisponivel (geração base mais módulos) e capacidadeDeCarga (estrutura base mais reforços). Cada função derivada é a fonte única do valor que a ficha apresenta.
• Armadilha comum: Repetir as fórmulas no resumo cria uma segunda versão que pode ficar desatualizada ao mudar a regra.
• Por que importa: Composição ajuda relatórios a permanecer consistentes e prepara para modelos de domínio mais ricos.

Material/arquivo inicial

main.js: // Entradas válidas: base, nivel e armadura são Numbers finitos >=0; nome é string, inclusive vazia.
export function ataque(base,nivel){} export function defesa(armadura,nivel){} export function resumo(nome,base,nivel,armadura){} | test/main.test.js: import test from 'node:test';

// Escreva os casos de ataque, defesa e resumo, incluindo nível zero e nome vazio.
test.todo('resumo de ficha');

Exemplo e resultado público

base=3,nivel=2,armadura=4,nome="Lia" → ataque=5,defesa=5,resumo="Lia | ataque 5 | defesa 5"

Bordas e erros publicados

nivel=0,base=0,armadura=0,nome="" -> ataque=0,defesa=0,resumo=" | ataque 0 | defesa 0" (nome vazio mantém o espaço antes da barra)

Teste público e aprovação visível

Critérios desta atividade: As três funções públicas são ataque, defesa e resumo; resumo chama ataque e defesa e apresenta nome e ambos os resultados. Preserve as fórmulas declaradas e aceite nome string vazio.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é a responsabilidade correta de resumo se ataque e defesa já existem?

• Repetir fórmulas para evitar chamadas internas.
• Alterar parâmetros para que as funções retornem texto.
 (Correta) Chamá-las e apresentar os resultados.

Por que a resposta está certa

Correto: resumo compõe resultados e conserva uma única fonte para cada regra.

Armadilha da alternativa incorreta

Repetir fórmula dentro de resumo pode fazê-la divergir da API; prefira compor as funções.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

nivel=0,base=0,armadura=0,nome="" → ataque=0,defesa=0,resumo=" | ataque 0 | defesa 0" (nome vazio mantém o espaço antes da barra)

Concepção errada e falha observável

Duplicar cálculo no resumo e nas funções derivadas. / Resumo diverge da API de ataque/defesa após mudança de regra.

Pergunta diagnóstica

Qual função é fonte de cada atributo derivado?

Pistas graduais

1. Defina regras numéricas: ataque=base+nivel, defesa=armadura+floor(nivel/2). 2. Chame funções de domínio dentro do resumo. 3. Teste nível zero e nome vazio, ambos permitidos.

Solução de referência explicada

ataque devolve base+nivel; defesa devolve armadura+Math.floor(nivel/2). resumo recebe nome (string vazia permitida) e os três números válidos, chama as funções derivadas e devolve texto com ambos os resultados.

Transferência em outro contexto

Crie um resumo de nave espacial com potência = potênciaBase + módulos e blindagem = blindagemBase + floor(nível/2). Implemente funções separadas para cada atributo e faça o resumo chamá-las para apresentar ambos.

Evidência de domínio

Critérios desta atividade: As três funções públicas são ataque, defesa e resumo; resumo chama ataque e defesa e apresenta nome e ambos os resultados. Preserve as fórmulas declaradas e aceite nome string vazio. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.10 Renomear para explicar

MODIFY · 24 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: readability.naming | Requer: control.branch | Introduz: readability.naming

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Renomeie os identificadores internos do módulo de pedidos para explicitar o subtotal e o resultado final. Preserve a função exportada, os parâmetros, as fórmulas e o comportamento para entradas válidas.

Objetivo da atividade

Liste cada identificador interno, seu papel e novo nome; identifique quais pertencem à API que deve ficar estável.

Explicação e modelo mental

• Modelo mental: Um bom nome comunica intenção e unidade. Refatorar nomes é mudança estrutural: para as mesmas entradas, comportamento observável deve permanecer igual.
• Exemplo resolvido: Num módulo de biblioteca, x pode virar quantidadeDePaginas e uma função privada contar pode virar contarPaginas. Faça as trocas sem alterar operações nem o nome exportado usado por consumidores.
• Armadilha comum: Misturar renomeação e mudança de algoritmo torna difícil saber a causa de divergência e pode quebrar API pública.
• Por que importa: Nomes claros reduzem esforço para entender legado e ajudam revisão e manutenção.

Material/arquivo inicial

main.js: export function totalPedido(precoUnitario, quantidade, desconto) {
const data = precoUnitario * quantidade;
const thing = d(data - desconto);
return thing;
}
function d(x) {
return Math.max(0, x);
}

Exemplo e resultado público

totalPedido(10,2,3) → 17; totalPedido(0,4,0) -> 0

Bordas e erros publicados

totalPedido(4,3,20) -> 0; totalPedido(4,3,0) -> 12; API pública inalterada; entradas numéricas não negativas

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Documentação oficial indicada por link e seção

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que evidência sustenta uma refatoração que deveria mudar apenas nomes internos?

 (Correta) Os nomes ficaram mais claros e as verificações existentes preservam interface e comportamento.
• O programa inicia sem erro, mesmo que as saídas e a interface pública não sejam comparadas.
• Comentários descrevem a intenção, então não é necessário executar verificações.

Por que a resposta está certa

Correto: testes de regressão e a interface preservada mostram que a mudança foi apenas de legibilidade.

Armadilha da alternativa incorreta

Iniciar sem erro ou adicionar comentários não demonstra equivalência. Compare comportamento observável e contrato público.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

totalPedido(4,3,20) → 0; totalPedido(4,3,0) -> 12; API pública inalterada; entradas numéricas não negativas

Concepção errada e falha observável

Alterar nomes e algoritmo juntos, impedindo saber qual mudança afetou o resultado. / Na entrada totalPedido(4,3,20), a saída observável não corresponde a 0; a divergência é compatível com esta hipótese: Alterar nomes e algoritmo juntos, impedindo saber qual mudança afetou o resultado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Alterar nomes e algoritmo juntos, impedindo saber qual mudança afetou o resultado.

Pistas graduais

1. Descubra a finalidade de cada valor antes de renomear. 2. Escolha nomes que expressem regra e unidade, não tipo genérico. 3. Faça apenas renomeação e compare testes/diff.

Solução de referência explicada

Renomeie data para subtotal, thing para valorFinal, d para limitarPisoZero e x para valorCalculado. Preserve a API exportada e Math.max(0, subtotal-desconto); explique como cada nome comunica estado ou resultado.

Transferência em outro contexto

Em um módulo de pedidos com nomes genéricos, renomeie o identificador do subtotal intermediário e o do resultado final para comunicar estado e resultado; preserve a API pública e o comportamento.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e APIs padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.11 Validar percentual

MAKE · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: errors.validation_guards | Requer: control.branch | Introduz: errors.validation_guards

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 2 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente validarPercentual(valor). Exija Number finito entre 0 e 100, inclusive. Use TypeError se o argumento não for Number e RangeError se o número for não finito ou estiver fora da faixa. Retorne o percentual válido sem conversão.

Objetivo da atividade

Classifique texto, -1, 0, 25, 100, 101 e NaN como TypeError, RangeError ou retorno.

Explicação e modelo mental

• Modelo mental: Uma função pode devolver um resultado quando o contrato é válido e lançar um erro quando não consegue cumprir esse contrato.
• Exemplo resolvido: validarPercentual(25) devolve 25. Uma guard clause rejeita texto antes das comparações; depois, finitude e faixa são verificadas antes do retorno.
• Armadilha comum: Comparações com NaN não indicam uma faixa válida e podem passar despercebidas sem Number.isFinite.
• Por que importa: Validar na fronteira protege cálculos e informa ao chamador se o problema é o tipo ou o domínio do valor.

Material/arquivo inicial

main.js: export function validarPercentual(valor) {}

Exemplo e resultado público

validarPercentual(25) → 25; validarPercentual(0) -> 0; validarPercentual(100) -> 100

Bordas e erros publicados

validarPercentual("25") -> TypeError; validarPercentual(-1) -> RangeError; validarPercentual(101) -> RangeError; validarPercentual(NaN) -> RangeError; validarPercentual(Infinity) -> RangeError

Teste público e aprovação visível

Critérios desta atividade: Usa guard clauses e lança TypeError quando o argumento não é Number. Lança RangeError para não finito ou fora de 0..100 e devolve sem conversão os valores válidos, inclusive os limites.

Restrições e documentação

Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

export function validarPercentual(...) deixa a função disponível ao avaliador; preserve export e a assinatura que o starter já apresenta. | throw new TypeError('mensagem') encerra a chamada e entrega o erro ao chamador. TypeError descreve um tipo incompatível; RangeError descreve um Number fora da faixa permitida. Uma guard clause valida primeiro e retorna ou lança antes do cálculo principal.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual erro descreve 101 quando a faixa permitida é de 0 a 100?

• TypeError, pois todo valor rejeitado tem tipo incorreto.
 (Correta) RangeError, pois o tipo é Number mas o valor está fora da faixa.
• Não lance erro; limite o resultado a 100.

Por que a resposta está certa

Isso mesmo: Number é o tipo certo, mas 101 não pertence ao domínio permitido.

Armadilha da alternativa incorreta

TypeError descreve um tipo incompatível; RangeError descreve um valor de tipo correto que está fora do domínio.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

validarPercentual("25") → TypeError; validarPercentual(-1) -> RangeError; validarPercentual(101) -> RangeError; validarPercentual(NaN) -> RangeError; validarPercentual(Infinity) -> RangeError

Concepção errada e falha observável

Usar uma comparação antes de validar o tipo permite coerção inesperada; aceitar números sem finitude pode propagar um valor sem significado. / Use uma guard clause para rejeitar o tipo que não é Number.

Pergunta diagnóstica

Depois, valide finitude e faixa; tipo errado é TypeError e valor fora do domínio é RangeError.

Pistas graduais

1. R 2. e 3. t

Solução de referência explicada

Faça retornos antecipados para os dois tipos de falha e devolva o percentual apenas após validar o contrato.

Transferência em outro contexto

Implemente validarDesconto: aceite Number finito entre 0 e 0.5, inclusive; lance TypeError para tipo incorreto e RangeError para valor fora da faixa.

Evidência de domínio

Critérios desta atividade: Usa guard clauses e lança TypeError quando o argumento não é Number. Lança RangeError para não finito ou fora de 0..100 e devolve sem conversão os valores válidos, inclusive os limites. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Sintaxe e classes de erro padrão do Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 04.12

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.12 Falha com contexto e limpeza

MAKE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: errors.catch_finally | Requer: control.branch | Introduz: errors.catch_finally

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 2 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente lerJsonSeguro(ler, liberar). Chame ler() e interprete o texto com JSON.parse. Se ocorrer SyntaxError, lance new Error com a mensagem 'JSON inválido: ' seguida da mensagem original. Se ler() lançar outro erro, relance o mesmo objeto. Chame liberar() exatamente uma vez em finally, tanto no sucesso como na falha. O contrato considera que liberar() conclui sem lançar; falha própria da limpeza fica fora desta atividade.

Objetivo da atividade

Trace o caminho de sucesso, o SyntaxError do parse e um erro de leitura; marque quando liberar roda.

Explicação e modelo mental

• Modelo mental: try/catch/finally separa operação, tratamento de uma falha conhecida e limpeza que precisa ocorrer em todos os caminhos.
• Exemplo resolvido: ler e liberar são funções recebidas como argumentos e chamadas dentro da operação. Se JSON.parse falhar com SyntaxError, o catch pode lançar Error com mensagem de contexto; finally roda antes de a função terminar, mesmo quando há throw.
• Armadilha comum: Um catch amplo que retorna {} trata entrada inválida como sucesso e pode também esconder uma falha do leitor.
• Por que importa: Recursos como arquivos, conexões e cursores precisam ser liberados mesmo quando leitura ou interpretação falha.

Material/arquivo inicial

main.js: export function lerJsonSeguro(ler, liberar) {}

Exemplo e resultado público

lerJsonSeguro(()=>'{}',()=>{}) → {}

Bordas e erros publicados

JSON inválido -> Error com mensagem de contexto; erro de ler() -> mesmo erro original; liberar() é chamado uma vez em sucesso ou falha

Teste público e aprovação visível

Critérios desta atividade: Transforma apenas SyntaxError em Error com mensagem contextualizada e relança o erro de leitura original. Retorna o objeto parseado no sucesso e chama liberar exatamente uma vez em todos os caminhos.

Restrições e documentação

Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

export function nome(...) mantém a função importável pelo avaliador; preserve export e os parâmetros do starter. | JSON.parse(texto) transforma texto JSON em valor JavaScript; texto malformado lança SyntaxError. A expressão erro instanceof SyntaxError identifica essa classe para contextualizá-la. | A forma é try { operação } catch (erro) { tratamento } finally { limpeza }. catch recebe o erro lançado no try; finally executa antes do retorno ou da propagação do erro. Deixe return e throw fora de finally para não substituir o resultado nem esconder a falha original. | ler e liberar são funções recebidas como argumentos: a primeira é chamada para obter o texto, a segunda para limpar o recurso em todos os caminhos.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual bloco garante a limpeza se o parse ou a leitura lançar?

• catch, que só roda quando uma exceção ocorre.
• return, que encerra o try antes da limpeza.
 (Correta) finally, que executa antes da saída normal ou excepcional.

Por que a resposta está certa

Correto: finally é o lugar para a limpeza que não depende do resultado.

Armadilha da alternativa incorreta

catch roda apenas em falhas; finally roda depois do sucesso ou da falha, antes da saída.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

JSON inválido → Error com mensagem de contexto; erro de ler() -> mesmo erro original; liberar() é chamado uma vez em sucesso ou falha

Concepção errada e falha observável

Capturar todo erro e devolver um valor padrão esconde falhas de leitura; esquecer finally deixa o recurso aberto quando JSON.parse lança. / Coloque leitura e parse dentro do try.

Pergunta diagnóstica

No catch, contextualize SyntaxError e relance outros erros sem substituí-los.

Pistas graduais

1. U 2. s 3. e

Solução de referência explicada

Inclua a mensagem original ao lançar Error novo para JSON inválido e use finally para garantir a limpeza.

Transferência em outro contexto

Implemente lerCsvSeguro(ler, fechar): converta erro de formato em Error com mensagem de contexto, preserve o mesmo objeto de erro de leitura e feche o recurso uma vez em sucesso ou falha.

Evidência de domínio

Critérios desta atividade: Transforma apenas SyntaxError em Error com mensagem contextualizada e relança o erro de leitura original. Retorna o objeto parseado no sucesso e chama liberar exatamente uma vez em todos os caminhos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Error, try/catch/finally e JSON.parse disponíveis no Node.js 24 LTS. | Próxima sugestão: 04.13

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

04.13 Retorno esperado ou erro

MAKE · 16 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: errors.return_or_throw | Requer: control.branch | Introduz: errors.return_or_throw

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 2 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente taxaDeEntrega(tipo). Exija string não vazia com TypeError para chamada inválida. Retorne 0.1 para 'comum', 0.25 para 'expressa' e null para qualquer outra opção bem formada ainda não cadastrada.

Objetivo da atividade

Classifique cada entrada como taxa, null esperado ou TypeError por argumento inválido.

Explicação e modelo mental

• Modelo mental: Ausência esperada pode ser representada por um valor de retorno; erro indica que a chamada não respeitou o contrato ou que a operação não pôde ser concluída.
• Exemplo resolvido: A string 'comum' devolve 0.1 e 'expressa' devolve 0.25. Uma modalidade futura como 'noturna' retorna null; um argumento que não é string não cumpre a API e lança TypeError.
• Armadilha comum: Lançar para toda opção desconhecida força o chamador a tratar uma possibilidade comum como exceção; retornar null para argumentos inválidos mascara erro de programação.
• Por que importa: Uma API previsível permite que o chamador trate resultados opcionais com um ramo normal e reserve exceções para falhas reais.

Material/arquivo inicial

main.js: export function taxaDeEntrega(tipo) {}

Exemplo e resultado público

taxaDeEntrega('comum') → 0.1; taxaDeEntrega('expressa') -> 0.25; taxaDeEntrega('noturna') -> null

Bordas e erros publicados

taxaDeEntrega('') -> TypeError; taxaDeEntrega(4) -> TypeError; opção não cadastrada e não vazia -> null

Teste público e aprovação visível

Critérios desta atividade: Valida tipo e string não vazia com guard clauses e lança TypeError quando a chamada viola o contrato. Retorna 0.1 e 0.25 para as opções cadastradas e null para opção válida sem cadastro.

Restrições e documentação

Documentação oficial indicada por link e seção

Conceitos prévios (Primers)

export function taxaDeEntrega(...) deixa a função disponível ao avaliador; preserve export e a assinatura do starter. | Uma guard clause rejeita argumento inválido antes das decisões normais. Use throw new TypeError('mensagem') quando a chamada viola o contrato; retorne null quando a opção está bem formada, mas não tem resultado cadastrado.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que deve ocorrer quando a modalidade é uma string não vazia ainda não cadastrada?

 (Correta) Retornar null, pois a ausência é resultado esperado da consulta.
• Lançar TypeError, pois nenhuma taxa foi cadastrada.
• Retornar a taxa comum como padrão.

Por que a resposta está certa

Isso mesmo: null comunica ausência sem tratar uma consulta válida como falha excepcional.

Armadilha da alternativa incorreta

O chamador forneceu argumento válido; uma opção desconhecida segue o contrato normal e retorna null.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

taxaDeEntrega('') → TypeError; taxaDeEntrega(4) -> TypeError; opção não cadastrada e não vazia -> null

Concepção errada e falha observável

Lançar erro para uma opção válida que ainda não foi cadastrada confunde ausência esperada com uso inválido da função. / Use uma guard clause para exigir string não vazia.

Pergunta diagnóstica

Depois da validação, trate opções conhecidas como resultados normais.

Pistas graduais

1. U 2. m 3. a

Solução de referência explicada

Retorne null para opção desconhecida e lance TypeError somente quando o chamador viola o contrato.

Transferência em outro contexto

Implemente tarifaDeAcesso(tipo): strings não vazias conhecidas retornam sua tarifa, opções ainda não registradas retornam null e argumentos inválidos lançam TypeError.

Evidência de domínio

Critérios desta atividade: Valida tipo e string não vazia com guard clauses e lança TypeError quando a chamada viola o contrato. Retorna 0.1 e 0.25 para as opções cadastradas e null para opção válida sem cadastro. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Array.isArray e classes de erro padrão disponíveis no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 05.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05 Strings, arrays, regex e Unicode

11 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

A unidade 05A cobre strings/arrays/Unicode; a 05B trata regex e parsing após a base.

Ordem: 05.01 · 05.03 · 05.04 · 05.06 · 05.07 · 05.10 · 05.11 · 05.02 · 05.05 · 05.08 · 05.09

05.01 Estado do array

PREDICT · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: arrays.mutation_alias | Requer: functions.return | Introduz: arrays.mutation_alias

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 2 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Rastreie o código linha por linha: desenhe quais nomes compartilham o mesmo array, preveja a saída e explique qual operação criou uma lista independente.

Objetivo da atividade

Desenhe uma caixa por identidade e anote o conteúdo de cada nome após cada linha.

Explicação e modelo mental

• Modelo mental: Atribuir uma variável cria outro nome para o mesmo array; uma cópia precisa de nova identidade.
• Exemplo resolvido: Numa agenda, agendaB=agendaA faz ambos apontarem à mesma lista. Adicionar pausa por B aparece ao ler A. agendaC=agendaA.slice() cria lista independente; adicionar almoço a C não muda A.
• Armadilha comum: Confundir alias com cópia; acompanhar identidade antes e depois de cada mutação.
• Por que importa: Alias acidental muda estado compartilhado sem que o chamador espere.

Material/arquivo inicial

main.js: export function observarReferencias() {
const a = [1, 2];
const b = a;
const c = a.slice();
a.push(3);
b.pop();
return {a, b, c, mesmaRaiz: a === b, copiaIndependente: a !== c};
}

console.log(JSON.stringify(observarReferencias()));

Exemplo e resultado público

saída → [1,2],[1,2],[1,2]; antes do pop, a e b são [1,2,3]

Bordas e erros publicados

c.push(4) depois -> c=[1,2,4], a/b continuam [1,2]

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se um módulo não deve expor sua coleção mutável interna, que decisão de API reduz acoplamento?

• Devolver o contêiner interno e presumir que todo chamador o tratará como imutável.
 (Correta) Expor operações ou uma representação controlada, em vez de entregar diretamente o contêiner interno.
• Guardar os dados numa variável global para que os consumidores compartilhem o mesmo estado.

Por que a resposta está certa

Correto: uma API controlada protege a propriedade do estado e deixa explícitas as operações permitidas.

Armadilha da alternativa incorreta

Uma referência mutável compartilhada permite que consumidores acoplem seus efeitos ao estado interno. Delimite a interface.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

c.push(4) depois → c=[1,2,4], a/b continuam [1,2]

Concepção errada e falha observável

Achar que atribuição cria cópia ou que slice modifica entrada. / Previsão diverge após push/pop por não acompanhar identidade a===b.

Pergunta diagnóstica

Quais nomes apontam para o mesmo array?

Pistas graduais

1. Desenhe uma caixa por array real. 2. b=a cria outro nome para a mesma caixa. 3. slice cria nova caixa rasa; push/pop mudam a caixa a/b.

Solução de referência explicada

a e b partilham identidade; c é cópia. push adiciona 3 a a/b, pop remove 3; c não muda.

Transferência em outro contexto

Numa agenda de playlists, rastreie um alias e uma cópia após inserção e remoção. Explique qual versão cada tela observa e escreva a operação que mantém uma versão independente.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Array e slice no baseline dependências fora do ECMAScript. | Próxima sugestão: 05.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.03 Parsons de slug

PARSONS · 13 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: strings.slug_basic | Requer: functions.return | Introduz: strings.slug_basic

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 2 · depuração 1 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Complete a função slug(titulo) usando os blocos. filter chama o callback para cada parte e mantém só as partes não vazias; registre os valores intermediários e explique a ordem.

Objetivo da atividade

Escreva a ordem das etapas e a forma dos arrays intermediários antes de ordenar os blocos.

Explicação e modelo mental

• Modelo mental: Separe responsabilidades: limpar bordas, padronizar caixa, dividir, filtrar vazios e juntar.
• Exemplo resolvido: Para converter ' Norte Sul ' em palavras com ponto: trim remove bordas; toLowerCase padroniza; split(' ') revela tokens vazios; filter recebe cada parte em um callback e mantém as não vazias; join('.') forma a saída.
• Armadilha comum: Juntar logo após split conserva tokens vazios como separadores duplicados. O callback de filter é chamado para cada parte; não acrescente remoção de acentos sem requisito.
• Por que importa: Pipelines legíveis tornam regras de formatação mais fáceis de revisar e mudar.

Material/arquivo inicial

main.js: export function slug(titulo) {
const limpo = titulo.trim();
const minusculo = limpo.toLowerCase();
const partes = minusculo.split(" ");
const validas = partes.filter(parte => parte !== "");
return validas.join(" ");
}

console.log(slug("Dragão Azul"));

Exemplo e resultado público

" Olá Mundo " → "olá-mundo"; ordem A,B,C,D,E

Bordas e erros publicados

"" -> ""; " A B " -> "a-b"; acentos são preservados

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

partes.filter(parte => parte !== '') chama a função entre parênteses uma vez para cada item e monta um array novo só com os itens cujo callback retorna um valor truthy — neste exemplo, true. É o mesmo que um for...of com if e push; funções como argumento são estudadas na etapa 07.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao dividir por um espaço literal, de onde podem surgir tokens vazios?

• Da conversão para minúsculas
• Do método join executado antes do split
 (Correta) De espaços repetidos entre palavras

Por que a resposta está certa

Separadores adjacentes deixam campos vazios no array.

Armadilha da alternativa incorreta

trim só limpa as bordas; delimitadores adjacentes no interior ainda criam posições vazias.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

"" → ""; " A B " -> "a-b"; acentos são preservados

Concepção errada e falha observável

Unir antes de remover partes vazias criadas por espaços repetidos. / Resultado contém -- ou espaços nas extremidades.

Pergunta diagnóstica

Que etapa gera strings vazias quando há dois espaços?

Pistas graduais

1. Faça trim antes de split. 2. Depois de split(' '), observe partes vazias. 3. Filtre partes vazias antes de join('-').

Solução de referência explicada

A→B→C→D→E. Neste exercício não há remoção de acentos ou pontuação.

Transferência em outro contexto

Defina etapas para transformar rótulos de salas com espaços irregulares em slugs separados por ponto. Registre dois valores intermediários e implemente sem normalizações extras.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: trim, split, filter e join estão no baseline ECMAScript. O callback de filter recebe cada parte; esta atividade apresenta essa chamada antes de exigir que o aluno escreva outros callbacks. | Próxima sugestão: 05.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.04 Complete a busca

FADED · 14 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: arrays.first_last_index | Requer: functions.return | Introduz: arrays.first_last_index

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Implemente função que devolve primeira e última posição do termo; registre sua previsão para ocorrências múltiplas, posição inicial e ausência.

Objetivo da atividade

Preencha uma tabela com primeiro índice, último índice e sentido de ausência nos casos públicos.

Explicação e modelo mental

• Modelo mental: indexOf busca da esquerda, lastIndexOf da direita; ambos usam -1 para ausência. Zero é posição válida.
• Exemplo resolvido: Em ['sol','lua','sol','mar'], indexOf('lua') aponta a primeira posição; lastIndexOf('sol') a ocorrência final. Buscar 'terra' retorna -1.
• Armadilha comum: if(indice) trata zero como falso; use indice !== -1.
• Por que importa: Posição e ausência precisam ser distintas antes de destacar ou editar elementos.

Material/arquivo inicial

main.js: // 05.04 · Primeira e última posição de um termo na lista.
// Devolve um objeto { primeira, ultima }; ausência é -1 nas duas posições. Lembre: 0 é uma posição válida.
// Registre a previsão para ocorrências múltiplas, posição inicial e ausência antes de executar.

export function posicoes(lista, termo) {
return { primeira: ___, ultima: ___ };
}

Exemplo e resultado público

["a","b","a"],"a" → {primeira:0,ultima:2}; ["a"],"a" -> {0,0}

Bordas e erros publicados

[],"x" -> {-1,-1}; ["a"],"x" -> {-1,-1}

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

return { primeira: 0, ultima: 2 } devolve um objeto com duas propriedades nomeadas; quem chama lê resultado.primeira e resultado.ultima. Objetos são estudados na etapa 06; aqui basta preencher os dois valores.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se uma busca precisa apenas responder se um elemento está presente, qual abordagem expressa melhor essa intenção?

 (Correta) Usar uma operação de pertinência em vez de depender do valor de uma posição.
• Converter o resultado da busca em booleano por truthiness.
• Inferir presença comparando o tamanho do array antes e depois da busca.

Por que a resposta está certa

Correto: uma operação de pertinência expressa presença diretamente e não confunde posição com existência.

Armadilha da alternativa incorreta

Posições são dados diferentes de uma resposta de presença; truthiness pode confundir uma posição válida com ausência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[],"x" → {-1,-1}; ["a"],"x" -> {-1,-1}

Concepção errada e falha observável

Retornar só a última ocorrência ou confundir índice zero com ausente. / primeira=2 para lista repetida ou ausente=0.

Pergunta diagnóstica

Qual valor de indexOf e lastIndexOf representa não encontrado?

Pistas graduais

1. Use indexOf para primeira ocorrência. 2. Use lastIndexOf para última. 3. Ambos devolvem -1 quando não encontram o termo.

Solução de referência explicada

return {primeira:lista.indexOf(termo),ultima:lista.lastIndexOf(termo)}; não muta a lista.

Transferência em outro contexto

Localize primeira e última ocorrência de um tipo numa sequência de eventos, incluindo ausência. Implemente e explique zero versus -1.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: indexOf e lastIndexOf estáveis no baseline. | Próxima sugestão: 05.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.06 Bug da mutacao - legado vs moderno

DEBUG · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: arrays.copy_by_change | Requer: functions.return | Introduz: arrays.copy_by_change

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Corrija ordenarSemMutar sem alterar a entrada e apresente duas implementações: a versão compatível com cópia explícita [...numeros].sort((a,b)=>a-b).reverse() e a versão moderna numeros.toSorted((a,b)=>a-b).toReversed(). Use a entrada congelada como evidência e compare clareza e compatibilidade do runtime.

Objetivo da atividade

Rastreie a mutação de sort/reverse, implemente duas versões não mutantes — cópia explícita e métodos copy-by-change — e compare clareza e compatibilidade.

Explicação e modelo mental

• Modelo mental: sort e reverse mutam o array receptor. Para preservar origem, copie antes ou use APIs copy-by-change compatíveis.
• Exemplo resolvido: Para exibir [8,3,6] em ordem inversa: copie, ordene a cópia com comparador numérico e reverta a cópia; compare identidade e conteúdo das duas listas.
• Armadilha comum: reverse também muta; sort sem comparador ordena como texto.
• Por que importa: Preservar entrada evita efeitos colaterais em chamadores e facilita comparar estados.

Material/arquivo inicial

main.js: // 05.06 · Bug da mutação: ordem decrescente SEM alterar a entrada.
// 1. Corrija ordenarSemMutar com cópia explícita (compatível com runtimes antigos).
// 2. Escreva ordenarSemMutarModerno com os métodos que devolvem cópia (toSorted e toReversed).
// 3. Teste as duas com Object.freeze([2, 1]) (entrada congelada) e [] e compare clareza e compatibilidade.

```
export function ordenarSemMutar(numeros) {
return numeros.sort((a, b) => a - b).reverse();
}

export function ordenarSemMutarModerno(numeros) {
return numeros; // troque pela versão com toSorted e toReversed
}
```

Exemplo e resultado público

[3,1,2] → [3,2,1] e entrada permanece [3,1,2] | Object.freeze([2,1]) e [] → [2,1] sem erro e []

Bordas e erros publicados

Object.freeze([2,1]) como entrada -> saída [2,1] sem erro; [] -> []

Teste público e aprovação visível

Exija reprodução e causa do bug, regressão em todos os casos, as duas estratégias não mutantes e comparação explicada de clareza/compatibilidade.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

Object.freeze(array) impede alterar as propriedades do array, mas é superficial: objetos dentro dele não são congelados. Neste starter, sort e reverse tentam reescrever posições e lançam TypeError mesmo fora do modo estrito.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Antes de depender de um método moderno de cópia por mudança em um pacote, o que verificar?

• Se sort é sempre não mutável, pois o método moderno só muda o nome.
 (Correta) Se os runtimes de destino oferecem o método ou se será preciso fornecer uma alternativa compatível.
• Se o método está disponível em todos os runtimes por executar JavaScript.

Por que a resposta está certa

Correto: verificar suporte no ambiente de destino orienta a escolha entre a API moderna e uma alternativa compatível.

Armadilha da alternativa incorreta

APIs da linguagem variam por suporte do runtime, e métodos mutáveis não se tornam imutáveis por trocar de nome. Confira a compatibilidade.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Object.freeze([2,1]) como entrada → saída [2,1] sem erro; [] -> []

Concepção errada e falha observável

Achar que sort/reverse criam arrays novos. / Entrada muda ou função falha com array congelado.

Pergunta diagnóstica

Qual método modifica o array recebido?

Pistas graduais

1. Faça cópia antes de sort e reverse. 2. Compare com toSorted().toReversed() em runtime compatível. 3. Teste a entrada congelada para revelar mutação.

Solução de referência explicada

Versão legada: [...numeros].sort((a,b)=>a-b).reverse(); moderna: numeros.toSorted((a,b)=>a-b).toReversed().

Transferência em outro contexto

Ordene tempos de corrida em ordem decrescente sem modificar a coleção de outra tela. Implemente no runtime declarado e mostre como confirmou que a origem ficou intacta.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 oferece toSorted/toReversed; em runtime menor copie antes de métodos mutáveis. | Próxima sugestão: 05.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.07 Estatísticas de lista

MAKE · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: arrays.statistics | Requer: functions.return | Introduz: arrays.statistics

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente métricas sem mutar a entrada: preserve o retorno publicado para [], lance TypeError se qualquer valor não for finito e escreva uma tabela de acumuladores antes de executar.

Objetivo da atividade

Escreva o retorno para lista vazia e uma tabela de acumuladores para cada valor público.

Explicação e modelo mental

• Modelo mental: A lista vazia tem contrato próprio. Em lista não vazia, extremos começam num valor observado; cada entrada deve ser finita; a varredura acumula sem escrever no array.
• Exemplo resolvido: Em [11,14,9], comece min/max em 11; 14 atualiza max; 9 atualiza min; a soma 34 dividida por 3 dá média. Antes disso, valide finitude; a função lê os valores, não os reordena.
• Armadilha comum: Zero como sentinela falha em faixas positivas/negativas; NaN contamina comparações; filtrar/coagir Infinity muda dados em vez de rejeitá-los; sort muta entrada.
• Por que importa: Métricas previsíveis precisam rejeitar dado numérico inválido e preservar a ordem que outros consumidores usam.

Material/arquivo inicial

main.js: export function metricas(numeros){ /* retorne [min,max,soma,media] */ }

Exemplo e resultado público

[2,4,6] → [2,6,12,4]; [-3] -> [-3,-3,-3,-3]

Bordas e erros publicados

[] -> [null,null,0,null]; valores não finitos -> TypeError; entrada não mutada

Teste público e aprovação visível

Resultado observável: [2,4,6] -> [2,6,12,4]; [-3] -> [-3,-3,-3,-3]. Borda a conferir: [] -> [null,null,0,null]; valores não finitos -> TypeError; entrada não mutada. Entrega esperada: Se vazio, retorne [null,null,0,null]. Antes de acumular, se algum valor não for finito lance TypeError. Caso contrário varra sem sort ou escrita na entrada, atualizando soma/min/max; compute média pelo tamanho.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

throw new TypeError('mensagem') interrompe a função e entrega o erro a quem chamou; o código depois do throw não roda. TypeError indica um valor do tipo ou formato errado. Retome a unidade Falhas previsíveis (04.11–04.13) para lançamento e tratamento com try/catch/finally; a etapa 13 aprofunda depuração, cause e testes.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que estratégia reduz memória extra ao calcular métricas básicas de uma coleção?

• Criar várias cópias ordenadas para cada métrica antes de combinar os resultados.
• Armazenar uma segunda lista de todos os estados intermediários mesmo quando não serão usados.
 (Correta) Atualizar um conjunto pequeno de acumuladores durante a passagem pelos dados.

Por que a resposta está certa

Correto: acumuladores mantêm o estado necessário sem construir coleções auxiliares proporcionais à entrada.

Armadilha da alternativa incorreta

Cópias e históricos completos aumentam memória sem necessidade. Mantenha apenas o estado exigido pelas métricas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → [null,null,0,null]; valores não finitos -> TypeError; entrada não mutada

Concepção errada e falha observável

Inicializar extremos em zero, não definir política para coleção vazia, aceitar não finitos ou mutar a entrada durante o cálculo. / Lista vazia gera média indefinida; NaN/Infinity não causam TypeError; ordenação destrói a ordem recebida.

Pergunta diagnóstica

Qual é o contrato explícito da lista vazia?

Pistas graduais

1. Trate [] conforme o contrato antes de dividir. 2. Valide cada valor com Number.isFinite e lance TypeError para qualquer não finito; não descarte nem converta. 3. Para dados válidos, inicialize min/max pelo primeiro item, some em uma passagem e não altere a entrada.

Solução de referência explicada

Se vazio, retorne [null,null,0,null]. Antes de acumular, se algum valor não for finito lance TypeError. Caso contrário varra sem sort ou escrita na entrada, atualizando soma/min/max; compute média pelo tamanho.

Transferência em outro contexto

Calcule extremos, total e média de medições de ruído. Defina [] conforme seu contrato, lance TypeError para NaN/Infinity e preserve a lista de origem; registre uma tabela de acumulação.

Evidência de domínio

Resultado observável: [2,4,6] -> [2,6,12,4]; [-3] -> [-3,-3,-3,-3]. Borda a conferir: [] -> [null,null,0,null]; valores não finitos -> TypeError; entrada não mutada. Entrega esperada: Se vazio, retorne [null,null,0,null]. Antes de acumular, se algum valor não for finito lance TypeError. Caso contrário varra sem sort ou escrita na entrada, atualizando soma/min/max; compute média pelo tamanho. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Number.isFinite e arrays no baseline; rejeitar não finitos pelo contrato. | Próxima sugestão: 05.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.10 Desaninhe grupos com flat()

MAKE · 16 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: arrays.flat_one | Requer: functions.return | Introduz: arrays.flat_one

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Implemente umNivel usando flat() padrão, sem mutar a entrada, e explique a profundidade aplicada.

Objetivo da atividade

Desenhe entrada e saída após remover exatamente um nível; marque o que permanece aninhado.

Explicação e modelo mental

• Modelo mental: flat() sem argumento remove um nível e devolve outro array; não achata toda a profundidade nem altera a origem.
• Exemplo resolvido: [[5],[6,[7]]] vira [5,6,[7]]: grupos externos somem, subarray mais interno permanece.
• Armadilha comum: Supor profundidade infinita ou mutação da entrada.
• Por que importa: Escolher profundidade mantém estrutura que consumidores esperam.

Material/arquivo inicial

main.js: export function umNivel(grupos){ /* use flat */ }

Exemplo e resultado público

[[1,2],[3,[4]]] → [1,2,3,[4]]; entrada não muda

Bordas e erros publicados

[] -> []; [[],[1]] -> [1]; nested [[[[1]]]] -> [[[1]]]

Teste público e aprovação visível

Resultado observável: [[1,2],[3,[4]]] -> [1,2,3,[4]]; entrada não muda. Borda a conferir: [] -> []; [[],[1]] -> [1]; nested [[[[1]]]] -> [[[1]]]. Entrega esperada: return grupos.flat(); a profundidade padrão é 1 e a entrada é preservada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao achatar arrays aninhados, que decisão deve orientar o uso de flat?

 (Correta) Determinar a profundidade estrutural esperada e escolher uma operação compatível com ela.
• Presumir que flat percorre recursivamente qualquer estrutura, sem limite.
• Usar flat apenas para remover elementos vazios, pois arrays internos permanecem iguais.

Por que a resposta está certa

Correto: flat segue uma profundidade definida; a estrutura dos dados determina se esse comportamento basta.

Armadilha da alternativa incorreta

flat não significa necessariamente percorrer todos os níveis. Compare a estrutura esperada à profundidade configurada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → []; [[],[1]] -> [1]; nested [[[[1]]]] -> [[[1]]]

Concepção errada e falha observável

Supor que flat() achata recursivamente ou muta a origem. / Quarto elemento vira 4 em vez de [4].

Pergunta diagnóstica

Quantos níveis a chamada sem argumento remove?

Pistas graduais

1. flat() sem argumento remove um nível. 2. Faça previsão antes de executar. 3. Compare grupos e resultado; eles são arrays distintos.

Solução de referência explicada

return grupos.flat(); a profundidade padrão é 1 e a entrada é preservada.

Transferência em outro contexto

Achate uma camada de equipes mantendo subgrupos internos. Implemente, explique a profundidade e confirme que a origem permanece intacta.

Evidência de domínio

Resultado observável: [[1,2],[3,[4]]] -> [1,2,3,[4]]; entrada não muda. Borda a conferir: [] -> []; [[],[1]] -> [1]; nested [[[[1]]]] -> [[[1]]]. Entrega esperada: return grupos.flat(); a profundidade padrão é 1 e a entrada é preservada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: flat() no baseline Node.js 24.x. | Próxima sugestão: 05.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.11 Unicode visto de perto

COMPARE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: unicode.codepoints_normalization | Requer: functions.return | Introduz: unicode.codepoints_normalization

Eixos: novidade 5 · sintaxe 3 · composição 3 · estado 1 · depuração 2 · arquivos 3 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Investigue as contagens e igualdades do snippet para UTF-16, code points e NFC. Registre previsões e explique o limite de cada medida.

Objetivo da atividade

Preveja length, igualdade pré/pós-normalização e iteração por code point; nomeie cada unidade.

Explicação e modelo mental

• Modelo mental: length conta unidades UTF-16, não caracteres percebidos. Iterar code points e normalizar tratam problemas diferentes; NFC não segmenta grafemas.
• Exemplo resolvido: ñ pode ser U+00F1 ou n + U+0303; comparar pode diferir antes e coincidir após NFC. As quantidades de unidades e símbolos visuais não são a mesma medida.
• Armadilha comum: Tratar length como contagem visual universal ou dizer que NFC resolve toda segmentação.
• Por que importa: Busca e limites de texto internacional precisam declarar qual unidade contam.

Material/arquivo inicial

main.js: const emoji="□"; const a="é"; const b="e\u0301";
console.log(emoji.length,[...emoji].length,a===b,a.normalize("NFC")===b.normalize("NFC"));

Exemplo e resultado público

saída → 2,1,false,true; for...of em emoji percorre um code point

Bordas e erros publicados

"e\u0301".length -> 2; codePointAt(0) de emoji -> 0x1F600

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se uma interface precisa contar caracteres percebidos pela pessoa, qual recurso é mais apropriado para textos Unicode?

• Contar bytes e tratar cada byte como um caractere visível.
□ (Correta) Segmentar o texto em grafemas, em vez de contar diretamente unidades de representação.
• Somar os comprimentos internos das strings sem considerar combinações Unicode.

Por que a resposta está certa

Correto: grafemas representam unidades percebidas na interface; unidades internas e bytes medem outra coisa.

Armadilha da alternativa incorreta

Representação em memória não corresponde necessariamente a caracteres visíveis. Escolha uma segmentação adequada à experiência da interface.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

"e\u0301".length → 2; codePointAt(0) de emoji -> 0x1F600

Concepção errada e falha observável

Confundir code units com caracteres percebidos ou igualdade visual com mesma sequência Unicode. / O aluno prevê emoji.length=1 ou a===b true antes de normalize.

Pergunta diagnóstica

O que length conta em UTF-16 e o que normalize altera?

Pistas graduais

1. Compare length com [...texto].length. 2. Unicode pode combinar base e acento em duas unidades/códigos. 3. normalize('NFC') aproxima sequências equivalentes; não resolve toda segmentação visual.

Solução de referência explicada

length conta duas code units no emoji; spread/for...of percorre code point; a e b diferem antes de NFC e coincidem depois.

Transferência em outro contexto

Compare formas composta/decomposta de outro texto internacional. Explique NFC, conte unidades e code points e identifique o que falta para contar grafemas.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: normalize e iteração por code point no baseline; grafemas completos fora do escopo. | Próxima sugestão: 05.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.02 Regex minima

FADED · 13 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: regex.identifier | Requer: functions.return | Introduz: regex.identifier

Eixos: novidade 3 · sintaxe 3 · composição 2 · estado 1 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Implemente a validação ASCII da string inteira. Antes de executar, preveja cada caso, inclusive a entrada com newline final, e explique o que ^ e $ ancoram em JavaScript sem a flag m.

Objetivo da atividade

Escreva a regra da primeira posição e das seguintes; preveja os casos antes de testar a expressão.

Explicação e modelo mental

• Modelo mental: Valide desde o início e exija fim absoluto. Em JavaScript, sem a flag m, ^ e $ casam só no início e no fim absolutos da entrada, então um newline final é rejeitado (Python e PCRE aceitam antes dele). A flag m move as âncoras para cada linha.
• Exemplo resolvido: Para uma letra maiúscula seguida por dois dígitos, exija ^ no início, [A-Z], [0-9]{2} e $ no fim. Compare o código completo, o mesmo código seguido de newline (rejeitado) e o mesmo padrão com a flag m (que passa a aceitar a linha isolada).
• Armadilha comum: Esquecer ^ ou $ e aceitar um trecho, ou ligar a flag m e descobrir que as âncoras passam a valer por linha.
• Por que importa: Validação parcial permite valores inválidos atravessarem limites de configuração ou identificação.

Material/arquivo inicial

main.js: // 05.02 · Valide a string INTEIRA como identificador ASCII.
// Regra: começa com letra ASCII ou underscore; depois só letras ASCII, dígitos ou underscore.
// Preveja cada caso antes de testar: "abc_2", "_x", "2abc", "", "a-b", "á", "A0" e "abc_2\n".

```
export function identificadorValido(entrada) {
const padrao = /___/;
return padrao.test(entrada);
}
```

Exemplo e resultado público

"abc_2" → true; "_x" -> true; "2abc" -> false; "" -> false; "abc_2\n" -> false

Bordas e erros publicados

"a-b" -> false; "á" -> false neste contrato ASCII; "A0" -> true

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é a diferença entre validar uma entrada e normalizá-la antes de validar?

• Normalizar sempre preserva exatamente o texto recebido, enquanto validar sempre o modifica.
• As duas operações são equivalentes e podem ser trocadas sem alterar o comportamento.
 (Correta) Validar decide se a entrada satisfaz o contrato; normalizar a transforma e só deve ocorrer se isso fizer parte do contrato.

Por que a resposta está certa

Correto: validação aceita ou rejeita conforme o contrato; normalização altera a representação recebida.

Armadilha da alternativa incorreta

Transformar silenciosamente pode aceitar algo diferente do que foi fornecido. Declare se a transformação é permitida antes de validar.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

"a-b" → false; "á" -> false neste contrato ASCII; "A0" -> true

Concepção errada e falha observável

Sem as âncoras ^ e $ (ou com a flag m, que as move para cada linha), a regex aceita só um trecho da string; em JavaScript, ao contrário de Python e PCRE, $ sem a flag m não tolera um newline final. / Uma busca sem âncoras (ou com a flag m) aceita um trecho: um texto que só contém um identificador válido em alguma linha passa.

Pergunta diagnóstica

Como exigir a string inteira, de modo que um newline final também seja rejeitado?

Pistas graduais

1. Use ^ no início e $ no fim, sem a flag m: em JavaScript, $ só casa no fim absoluto da entrada. 2. Primeiro caractere: letra ASCII ou underscore. 3. Depois: letras ASCII, dígitos ou underscore; nenhum outro caractere pode sobrar.

Solução de referência explicada

Use /^[A-Za-z_][A-Za-z0-9_]*$/ sem a flag m. As duas âncoras exigem a string inteira, então um newline final é rejeitado; com a flag m, ^ e $ passariam a casar em cada linha e o resultado mudaria.

Transferência em outro contexto

Valide uma chave de ambiente com regra ASCII definida por você. Exija correspondência inteira, inclua um caso com newline final que deve falhar e explique a escolha da âncora final.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Regex literal e test() no baseline; contrato deliberadamente ASCII. | Próxima sugestão: 05.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.05 RegExp.escape em busca

MAKE · TRILHA OPCIONAL · 19 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: regex.escape_literal | Requer: functions.return | Introduz: regex.escape_literal

Eixos: novidade 5 · sintaxe 4 · composição 4 · estado 1 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Implemente busca literal em texto: declare o contrato para metacaracteres, construa o padrão com escape compatível e explique como tratou suporte do runtime.

Objetivo da atividade

Declare se a busca aceita regex ou texto literal e escreva um caso de pontuação que diferencie as escolhas.

Explicação e modelo mental

• Modelo mental: Ao construir regex dinâmica, decida se entrada é sintaxe regex ou texto literal. Para texto literal, escape o termo inteiro.
• Exemplo resolvido: Uma busca por 'item?' precisa tratar ? como dado; RegExp.escape prepara o termo antes do construtor RegExp, mantendo pontuação literal.
• Armadilha comum: Escapar manualmente só alguns metacaracteres deixa lacunas. Não escape quando a interface promete linguagem regex.
• Por que importa: A separação entre dado e sintaxe evita padrões inválidos e resultados surpreendentes.

Material/arquivo inicial

main.js: export function contemLiteral(texto,termo){ const re=new RegExp(/* termo escapado */); return re.test(texto) }

Exemplo e resultado público

texto "a.b", termo "." → true; texto "axb", termo "." -> false; termo vazio -> true por contrato

Bordas e erros publicados

termo "[x]" em texto literal -> true; termo "*" isolado -> não lança SyntaxError e busca asterisco literal

Teste público e aprovação visível

Resultado observável: texto "a.b", termo "." -> true; texto "axb", termo "." -> false; termo vazio -> true por contrato. Borda a conferir: termo "[x]" em texto literal -> true; termo "*" isolado -> não lança SyntaxError e busca asterisco literal. Entrega esperada: const re=new RegExp(RegExp.escape(termo)); return re.test(texto). Runtime precisa declarar suporte ou polyfill verificado.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao usar uma API de escape de regex em uma biblioteca, que questão de compatibilidade deve ser verificada?

 (Correta) Se todos os runtimes declarados como alvo oferecem a API ou recebem uma alternativa compatível.
• Se qualquer implementação de JavaScript disponibiliza automaticamente toda API recente.
• Se a API pode ser omitida porque o construtor sempre interpreta texto dinamicamente como literal.

Por que a resposta está certa

Correto: a superfície da linguagem disponível depende dos runtimes suportados pela biblioteca.

Armadilha da alternativa incorreta

Declarar JavaScript como plataforma não garante suporte uniforme a APIs recentes. Verifique os alvos e planeje uma alternativa.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

termo "[x]" em texto literal → true; termo "*" isolado -> não lança SyntaxError e busca asterisco literal

Concepção errada e falha observável

Interpolar texto do usuário como fonte regex e dar significado a metacaracteres. / A busca por ponto casa qualquer letra ou por colchete quebra o padrão.

Pergunta diagnóstica

O termo deve ser linguagem regex ou caracteres literais?

Pistas graduais

1. Este contrato é busca literal. 2. Use RegExp.escape quando o runtime oferecer. 3. Se não oferecer, use polyfill revisado declarado, não escape manual improvisado.

Solução de referência explicada

const re=new RegExp(RegExp.escape(termo)); return re.test(texto). Runtime precisa declarar suporte ou polyfill verificado.

Transferência em outro contexto

Implemente busca literal num filtro de nomes de arquivo. Escolha estratégia compatível com o runtime, defina fallback e teste metacaractere.

Evidência de domínio

Resultado observável: texto "a.b", termo "." -> true; texto "axb", termo "." -> false; termo vazio -> true por contrato. Borda a conferir: termo "[x]" em texto literal -> true; termo "*" isolado -> não lança SyntaxError e busca asterisco literal. Entrega esperada: const re=new RegExp(RegExp.escape(termo)); return re.test(texto). Runtime precisa declarar suporte ou polyfill verificado. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Usar RegExp.escape quando disponível; senão somente polyfill mantido e revisado, sem escape parcial improvisado. | Próxima sugestão: 05.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.08 Transferencia - log parser pequeno

MAKE · 28 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: strings.log_parse | Requer: functions.return | Introduz: strings.log_parse

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Antes de escrever o parser, classifique cada linha pública como válida ou rejeitada e registre os campos que justificam a decisão. Em seguida implemente a separação pelos dois primeiros delimitadores, rode os casos e explique como preservou o restante da mensagem.

Objetivo da atividade

Classifique linhas públicas como válidas/rejeitadas e justifique com campos/separadores antes de implementar.

Explicação e modelo mental

• Modelo mental: Defina separadores estruturais, campos obrigatórios e se o restante é texto livre antes de parsear.
• Exemplo resolvido: Num registro categoria;horário;detalhes, localize dois primeiros ';', valide categoria/horário e preserve tudo após o segundo como detalhes, inclusive outros ';'.
• Armadilha comum: split irrestrito separa o texto livre; aceitar campo faltante mascara registro malformado.
• Por que importa: Importadores precisam conservar mensagens e rejeitar estrutura incompleta previsivelmente.

Material/arquivo inicial

main.js: export function parseLinhas(linhas){ /* retorne [validas,rejeitadas]; linha NIVEL|data|mensagem */ }

Exemplo e resultado público

["INFO|2026-01-02|ok","WARN|2026-01-03|lento"] → [["INFO|2026-01-02|ok","WARN|2026-01-03|lento"],[]]

Bordas e erros publicados

["ERRO","INFO||vazio","INFO|2026-01-02|"] -> [[],["ERRO","INFO||vazio","INFO|2026-01-02|"]]; data é texto ISO, sem Date

Teste público e aprovação visível

Resultado observável: ["INFO|2026-01-02|ok","WARN|2026-01-03|lento"] -> [["INFO|2026-01-02|ok","WARN|2026-01-03|lento"],[]]. Borda a conferir: ["ERRO","INFO||vazio","INFO|2026-01-02|"] -> [[],["ERRO","INFO||vazio","INFO|2026-01-02|"]]; data é texto ISO, sem Date. Entrega esperada: Para cada linha, localize os dois primeiros |, valide partes não vazias e nível INFO/WARN/ERROR; preserve a linha original no array válido ou rejeitado. Date vem depois.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando um parser rejeita uma linha, que informação ajuda a pessoa a investigar o problema?

• Somente a posição da linha, descartando o conteúdo que falhou.
 (Correta) A linha original junto de um motivo de rejeição claro, sem alterar silenciosamente a entrada.
• Uma versão corrigida automaticamente, sem registrar que a entrada original era inválida.

Por que a resposta está certa

Correto: preservar a entrada rejeitada e explicar o motivo torna o diagnóstico reproduzível.

Armadilha da alternativa incorreta

Sem conteúdo original ou motivo, fica difícil localizar e reproduzir a falha. Registre ambos conforme o contrato do parser.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

["ERRO","INFO||vazio","INFO|2026-01-02|"] → [[],["ERRO","INFO||vazio","INFO|2026-01-02|"]]; data é texto ISO, sem Date

Concepção errada e falha observável

Aceitar linha com campo ausente ou partir mensagem que contém delimitador sem contrato. / Registro inválido aparece em válidos ou mensagem perde conteúdo após terceiro campo.

Pergunta diagnóstica

Quantos delimitadores são obrigatórios e a mensagem pode conter |?

Pistas graduais

1. Separe no máximo os dois primeiros delimitadores. 2. Exija nível, data e mensagem não vazios. 3. Preserve o restante da mensagem como texto, inclusive |.

Solução de referência explicada

Para cada linha, localize os dois primeiros |, valide partes não vazias e nível INFO/WARN/ERROR; preserve a linha original no array válido ou rejeitado. Date vem depois.

Transferência em outro contexto

Projete leitor de setor|instante|observação, onde observação pode conter '|'. Defina campos ausentes/vazios, crie casos de borda e implemente preservando o resto.

Evidência de domínio

Resultado observável: ["INFO|2026-01-02|ok","WARN|2026-01-03|lento"] -> [["INFO|2026-01-02|ok","WARN|2026-01-03|lento"],[]]. Borda a conferir: ["ERRO","INFO||vazio","INFO|2026-01-02|"] -> [[],["ERRO","INFO||vazio","INFO|2026-01-02|"]]; data é texto ISO, sem Date. Entrega esperada: Para cada linha, localize os dois primeiros |, valide partes não vazias e nível INFO/WARN/ERROR; preserve a linha original no array válido ou rejeitado. Date vem depois. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: indexOf/slice estáveis; sem rede e sem interpretação de data. | Próxima sugestão: 05.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

05.09 Microprojeto - inventário textual

PROJECT · 45 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: arrays.inventory_text | Requer: functions.return | Introduz: arrays.inventory_text

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente as operações de adicionar, remover e listar sobre o array. Registre a sequência de comandos, política de repetidos/ausentes e mudança de estado após cada ação.

Objetivo da atividade

Registre estado, mudança e retorno de cada operação pública antes de executar.

Explicação e modelo mental

• Modelo mental: Modele coleção e operações explicitamente; defina efeitos e retornos para tornar estado observável.
• Exemplo resolvido: Numa estante, push adiciona título e join lista itens. Ao remover título ausente, não faça splice com -1: mantenha estado e informe false.
• Armadilha comum: splice(-1,1) remove o último item; não deduplique se repetição for permitida.
• Por que importa: A disciplina de estado se transfere a carrinhos, listas de tarefa e ferramentas pequenas.

Material/arquivo inicial

main.js: // 05.09 · Inventário textual.
// Cada operação é uma string: "adicionar NOME", "remover NOME" ou "listar".
// aplicar(operacoes) executa em ordem sobre um inventário (array) que começa vazio e devolve
// um array com o resultado de cada operação:
// adicionar -> true (repetidos são permitidos: duas linhas iguais)
// remover -> true se removeu a primeira ocorrência; false se o item não existe (estado inalterado)
// listar -> texto com um item por linha, na ordem de inserção ("" se vazio)
// Registre a sequência de comandos e o estado depois de cada ação.

export function aplicar(operacoes) {
const itens = [];
const resultados = [];
// implemente
return resultados;
}

Exemplo e resultado público

adicionar espada, adicionar poção, listar → 'espada\npoção'; remover espada, listar -> 'poção'

Bordas e erros publicados

adicionar espada duas vezes -> duas linhas espada; remover ausente -> estado inalterado e retorno false; lista vazia -> texto vazio

Teste público e aprovação visível

Resultado observável: adicionar espada, adicionar poção, listar -> 'espada\npoção'; remover espada, listar -> 'poção'. Borda a conferir: adicionar espada duas vezes -> duas linhas espada; remover ausente -> estado inalterado e retorno false; lista vazia -> texto vazio. Entrega esperada: Guarde itens em array; adicionar faz push; remover procura primeira ocorrência e só faz splice se encontrada; listar usa join('\n').

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual divisão de responsabilidades facilita manter consistente um inventário exibido como texto?

• Usar o texto renderizado como armazenamento principal e reconstruir a coleção depois.
• Misturar regras de formatação e alteração de estado em uma única string fixa.
 (Correta) Atualizar o estado da coleção separadamente de formatar sua representação para exibição.

Por que a resposta está certa

Correto: separar estado e apresentação deixa a coleção como fonte de dados e permite formatar a saída de forma consistente.

Armadilha da alternativa incorreta

Texto exibido é uma representação, não um bom substituto para o estado estruturado. Separe a atualização da coleção da formatação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

adicionar espada duas vezes → duas linhas espada; remover ausente -> estado inalterado e retorno false; lista vazia -> texto vazio

Concepção errada e falha observável

Tratar duplicata/removida ausente sem política explícita. / Remover item ausente apaga outro item por índice -1.

Pergunta diagnóstica

O que splice(-1,1) faria se indexOf não achasse o item?

Pistas graduais

1. Use array como estado interno. 2. Antes de remover, verifique índice >=0. 3. O contrato conserva duplicatas e remove só a primeira ocorrência.

Solução de referência explicada

Guarde itens em array; adicionar faz push; remover procura primeira ocorrência e só faz splice se encontrada; listar usa join('\n').

Transferência em outro contexto

Crie lista de materiais de oficina com inclusão, remoção da primeira ocorrência e listagem. Declare política de repetidos/ausentes e demonstre uma sequência de uso.

Evidência de domínio

Resultado observável: adicionar espada, adicionar poção, listar -> 'espada\npoção'; remover espada, listar -> 'poção'. Borda a conferir: adicionar espada duas vezes -> duas linhas espada; remover ausente -> estado inalterado e retorno false; lista vazia -> texto vazio. Entrega esperada: Guarde itens em array; adicionar faz push; remover procura primeira ocorrência e só faz splice se encontrada; listar usa join('\n'). Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Arrays, indexOf, splice e join no baseline; estado local determinístico. | Próxima sugestão: 06.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06 Objetos, referências e JSON

11 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 06.01 · 06.02 · 06.03 · 06.04 · 06.05 · 06.06 · 06.07 · 06.08 · 06.09 · 06.10 · 06.11

06.01 Leitura de referências

PREDICT · 13 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: objects.identity | Requer: arrays.basic | Introduz: objects.identity

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 4 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Rastreie as identidades do objeto e da propriedade aninhada antes/depois de cada atribuição; escreva a previsão e explique o alcance de spread raso.

Objetivo da atividade

Desenhe referências e preveja identidades e campos alterados, linha a linha.

Explicação e modelo mental

• Modelo mental: Atribuição compartilha identidade; spread cria novo externo, mas copia referências internas.
• Exemplo resolvido: Dois nomes apontam à mesma ficha de reserva; mudança é visível por ambos. Uma cópia rasa separa campo externo, mas ainda compartilha objeto contato.
• Armadilha comum: Supor que spread clona a árvore inteira; comparar referências externas e aninhadas com ===.
• Por que importa: O alcance real da cópia evita efeitos laterais em dados aninhados.

Material/arquivo inicial

main.js: export function observarAliasing() {
const base = {pontos: 1, meta: {ativo: true}};
const alias = base;
const copia = {...base};
alias.pontos = 2;
copia.meta.ativo = false;
return {pontos: base.pontos, ativo: base.meta.ativo, mesmaRaiz: base === copia, mesmoMeta: base.meta === copia.meta};
}

console.log(JSON.stringify(observarAliasing()));

Exemplo e resultado público

a.pontos/b.pontos/c.pontos → 2,2,1; a.meta.ativo/c.meta.ativo -> false,false

Bordas e erros publicados

c.pontos=9 -> a.pontos continua 2; b.meta={ativo:true} -> a.meta muda por alias b=a

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que um diagrama de referências ajuda a investigar objetos compartilhados?

 (Correta) Ele separa os nomes das variáveis dos objetos que esses nomes podem referenciar.
• Ele substitui a necessidade de acompanhar atribuições feitas durante a execução.
• Ele prova igualdade profunda sempre que dois nomes aparecem perto um do outro.

Por que a resposta está certa

Correto: nomes são bindings e objetos são identidades; desenhar essa relação torna a leitura do fluxo mais clara.

Armadilha da alternativa incorreta

Proximidade de nomes não mostra identidade. Acompanhe quais objetos são criados e a quais deles cada binding se liga.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

c.pontos=9 → a.pontos continua 2; b.meta={ativo:true} -> a.meta muda por alias b=a

Concepção errada e falha observável

Achar que spread copia recursivamente objetos aninhados. / A previsão diz que c.meta.ativo não afeta a.meta.

Pergunta diagnóstica

Quais referências apontam para o mesmo objeto interno meta?

Pistas graduais

1. Desenhe o objeto externo e o interno. 2. b é o mesmo objeto de a. 3. Spread cria externo novo, mas mantém meta compartilhado.

Solução de referência explicada

a===b; c!==a; c.meta===a.meta. Assim, mudança de pontos em b afeta a; mudança interna via c afeta ambos.

Transferência em outro contexto

Modele perfil de assinante com contato aninhado; compare alias e spread, depois proponha mudança mínima para isolar edição do contato.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Referências e spread são ECMAScript; sem pressupor layout físico stack/heap. | Próxima sugestão: 06.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.02 Aliasing visual

TRACE · 15 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: objects.alias_graph | Requer: arrays.basic | Introduz: objects.alias_graph

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 4 · depuração 1 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Complete o grafo de referências após spread, mutação e reatribuição. Anote quais caminhos compartilham alvo em cada etapa e justifique.

Objetivo da atividade

Desenhe o grafo antes/depois da reatribuição e rotule objetos compartilhados.

Explicação e modelo mental

• Modelo mental: Spread copia arestas da camada externa; reatribuir propriedade na cópia muda sua aresta, não o alvo da origem.
• Exemplo resolvido: Origem e cópia de uma configuração apontam ao mesmo objeto permissões. Alterar permissão afeta ambas; atribuir outro objeto à cópia cria nova aresta.
• Armadilha comum: Misturar mutação do aninhado compartilhado com substituição da propriedade. Desenhe arestas por etapa.
• Por que importa: Dados aninhados explicam por que alterações atravessam fronteiras inesperadas.

Material/arquivo inicial

main.js: export function observarReferencias() {
const base = {usuario: {nome: "Ana"}, itens: [1]};
const alias = base;
const copia = {...base};
copia.usuario.nome = "Bia";
alias.itens.push(2);
return {base, alias, copia, mesmaRaiz: base === copia, mesmoUsuario: base.usuario === copia.usuario};
}

console.log(JSON.stringify(observarReferencias()));

Exemplo e resultado público

base.usuario.nome → Bia; copia.usuario.nome -> Bia; base.itens -> [1,2]; copia.itens -> [1,2]

Bordas e erros publicados

copia.usuario={nome:'Cris'} depois -> base.usuario.nome continua Bia; copia.usuario.nome Cris

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Num diagrama de referências, o que uma seta entre dois nós representa?

• Uma cópia automática de todos os campos do objeto de destino.
 (Correta) Uma propriedade que aponta para a identidade do objeto de destino.
• Uma garantia de que as propriedades dos dois objetos serão sempre iguais.

Por que a resposta está certa

A seta representa uma referência. Duas setas podem apontar para o mesmo nó sem criar uma cópia.

Armadilha da alternativa incorreta

No grafo, a seta indica ligação por referência; igualdade dos campos e cópia são ideias diferentes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

copia.usuario={nome:'Cris'} depois → base.usuario.nome continua Bia; copia.usuario.nome Cris

Concepção errada e falha observável

Confundir troca da propriedade externa com mutação do objeto compartilhado. / Diagrama mostra duas instâncias de usuario após spread antes da reatribuição.

Pergunta diagnóstica

Qual aresta do grafo é copiada e qual objeto permanece compartilhado?

Pistas graduais

1. Marque identidades externas. 2. Spread copia valores das propriedades, inclusive referências. 3. Reatribuir copia.usuario troca uma aresta; não muda base.usuario.

Solução de referência explicada

Desenhe dois externos e um usuário/array interno compartilhado até a reatribuição; depois a cópia aponta a novo usuário.

Transferência em outro contexto

Desenhe cópia rasa de configuração de relatório com filtro aninhado compartilhado e depois substituído. Inclua previsão por etapa.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Objetos, spread e referências padrão; grafo representa semântica, não endereços. | Próxima sugestão: 06.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.03 Parsons de normalização

PARSONS · 15 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: objects.normalize_fields | Requer: arrays.basic | Introduz: objects.normalize_fields

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Ordene os blocos que constroem o objeto de domínio somente com campos permitidos. Explique por que a propriedade extra não atravessa a fronteira.

Objetivo da atividade

Liste campos finais, transformações e valores descartados; depois ordene os blocos dentro do corpo de normalizar(entrada).

Explicação e modelo mental

• Modelo mental: Normalizar entrada é construir modelo interno com campos autorizados e transformações explícitas.
• Exemplo resolvido: Formulário envia codigo, rotulo e token. Modelo da interface usa só codigo e rotulo aparado; o token fica fora do objeto final.
• Armadilha comum: Espalhar entrada inteira carrega propriedades extras e acopla domínio ao formato externo.
• Por que importa: Construção explícita reduz vazamento de campos e facilita auditoria.

Material/arquivo inicial

main.js: export function normalizar(entrada) {
const {id, nome} = entrada;
const usuario = {id, nome: nome.trim(), senha: entrada.senha};
return usuario;
}

console.log(JSON.stringify(normalizar({id: 1, nome: " Ana ", senha: "ignorada"})));

Exemplo e resultado público

entrada {id:1,nome:" Ana ",senha:"x"} → {id:1,nome:"Ana"}; usar A,B,C sem D

Bordas e erros publicados

campo extra admin na entrada -> não aparece no resultado; nome vazio após trim -> string vazia neste contrato

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o principal benefício de converter uma entrada externa para um modelo interno explícito?

• Garantir que qualquer campo recebido permaneça disponível sem revisão.
• Fazer com que a entrada externa e o modelo interno compartilhem sempre a mesma identidade.
 (Correta) Separar o formato recebido da interface de dados que o restante do programa usa.

Por que a resposta está certa

A conversão cria uma fronteira clara entre o formato de entrada e o modelo usado pela aplicação.

Armadilha da alternativa incorreta

Uma normalização serve para controlar essa fronteira; não precisa manter formato, campos ou identidade da entrada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

campo extra admin na entrada → não aparece no resultado; nome vazio após trim -> string vazia neste contrato

Concepção errada e falha observável

Espalhar entrada inteira e vazar campo não pertencente ao domínio. / Resultado contém senha/admin ou nome sem trim.

Pergunta diagnóstica

Quais campos compõem o modelo de domínio publicado?

Pistas graduais

1. Selecione somente id e nome. 2. Normalize nome antes de criar objeto final. 3. Não use spread de entrada externa quando campos extras devem ficar fora.

Solução de referência explicada

Dentro de normalizar(entrada): A→B→C; D é distrator e violaria a fronteira de dados.

Transferência em outro contexto

Uma importação de fornecedores tem identificador, nome e campo legado interno. Defina objeto de domínio, implemente transformação e justifique inclusão/omissão.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Destructuring, trim e objetos literais no baseline. | Próxima sugestão: 06.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.04 Complete o merge

FADED · 13 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: objects.merge_defaults | Requer: arrays.basic | Introduz: objects.merge_defaults

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 1 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente atualização rasa com patch prevalecendo, sem mutar os argumentos. Preveja os campos mantidos/substituídos e explique o caso de valor falsy.

Objetivo da atividade

Indique a origem de cada propriedade no resultado e quem vence colisão.

Explicação e modelo mental

• Modelo mental: Em spreads sucessivos, propriedade posterior vence; zero e string vazia podem ser valores fornecidos válidos.
• Exemplo resolvido: Preferências antigas idioma pt e zoom 110 recebem patch zoom 125. Espalhar antigas e depois patch preserva idioma e altera zoom sem mutar fontes.
• Armadilha comum: Inverter ordem desfaz patch; usar || como default pode apagar zero.
• Por que importa: Atualizações previsíveis preservam campos não alterados e valores válidos de fronteira.

Material/arquivo inicial

main.js: export function atualizar(antiga,patch){ return {___}; }

Exemplo e resultado público

antiga={tema:"escuro",volume:5},patch={volume:0} → {tema:"escuro",volume:0}

Bordas e erros publicados

patch={tema:""} -> tema vazio preservado; patch={} -> cópia rasa com valores iguais; entradas não mutadas

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que manter uma função de transformação de configurações separada da persistência?

 (Correta) Permite revisar e testar a transformação sem misturar acesso externo e regras de atualização.
• Garante que todo campo armazenado seja válido sem precisar de um contrato.
• Faz qualquer mudança de configuração ser salva automaticamente no local correto.

Por que a resposta está certa

Correto: separar transformação e I/O delimita efeitos colaterais e facilita verificar a regra isoladamente.

Armadilha da alternativa incorreta

Uma transformação pura não define validação nem persistência. Mantenha cada responsabilidade e seu contrato explícitos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

patch={tema:""} → tema vazio preservado; patch={} -> cópia rasa com valores iguais; entradas não mutadas

Concepção errada e falha observável

Aplicar defaults após patch e apagar novos valores ou usar || contra zero/vazio. / Patch volume 0 volta a 5.

Pergunta diagnóstica

Qual objeto deve prevalecer quando a mesma chave aparece nos dois?

Pistas graduais

1. Espalhe antiga primeiro. 2. Depois espalhe patch para sobrepor. 3. Spread é raso; campos internos precisam de política separada.

Solução de referência explicada

return {...antiga,...patch}; esta atividade cobre merge raso apenas.

Transferência em outro contexto

Atualize preferências de notificação preservando campos ausentes e permitindo false explícito. Demonstre precedência e valor falsy.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Spread e ?? no baseline; merge definido é raso. | Próxima sugestão: 06.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.05 structuredClone comparativo

INVESTIGATE · TRILHA OPCIONAL · 24 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: objects.clone_comparison | Requer: arrays.basic | Introduz: objects.clone_comparison

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Compare spread, structuredClone e JSON round-trip com objetos e arrays aninhados. Registre identidade e estrutura de cada resultado antes de escolher uma estratégia.

Objetivo da atividade

Preveja identidade, tipo e se o resultado é texto interoperável em cada estratégia.

Explicação e modelo mental

• Modelo mental: Spread copia referências internas; structuredClone copia muitos dados estruturados; JSON serializa num formato restrito.
• Exemplo resolvido: Um objeto de configuração contém tema e itens aninhados. Spread compartilha essas referências; structuredClone e JSON round-trip criam objetos e arrays internos novos. Compare identidade e depois altere um campo.
• Armadilha comum: Tratar JSON round-trip como clone universal ou presumir que structuredClone suporta funções.
• Por que importa: Escolha errada perde tipo silenciosamente ou deixa mutações compartilhadas.

Material/arquivo inicial

main.js: const fonte={interno:{n:1},itens:[1,2]}; /* comparar spread, structuredClone e JSON */

Exemplo e resultado público

spread → objeto externo novo, mas interno e itens compartilhados; structuredClone -> objetos e arrays internos independentes; JSON round-trip -> árvore compatível com novas referências

Bordas e erros publicados

fonte com ciclo -> structuredClone preserva o ciclo e JSON.stringify não consegue serializar; fonte com função -> structuredClone falha

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que diferença entre structuredClone e JSON round-trip fica visível com os dados deste exercício?

• JSON preserva funções e ciclos automaticamente
 (Correta) Ambos criam árvores independentes para objetos e arrays compatíveis; structuredClone também suporta ciclos
• Spread cria cópia profunda de todas as referências

Por que a resposta está certa

JSON round-trip serve para dados compatíveis e acíclicos; structuredClone suporta um conjunto maior de valores estruturados.

Armadilha da alternativa incorreta

JSON não é clone universal e spread é raso. Escolha conforme finalidade e tipo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

fonte com ciclo → structuredClone preserva o ciclo e JSON.stringify não consegue serializar; fonte com função -> structuredClone falha

Concepção errada e falha observável

Tratar cópia rasa, clone estruturado e serialização JSON como equivalentes. / Teste de identidade/tipo revela perdas diferentes.

Pergunta diagnóstica

A operação está copiando memória local ou serializando dados portáveis?

Pistas graduais

1. Compare fonte.interno === copia.interno. 2. Mude uma propriedade aninhada em cada cópia. 3. JSON copia dados compatíveis, mas não é clone universal.

Solução de referência explicada

Crie três resultados e registre identidade e tipos. Escolha structuredClone para grafo suportado; serialização requer esquema explícito.

Transferência em outro contexto

Compare cópias de uma configuração com objetos e arrays aninhados. Registre se a identidade interna é compartilhada e recomende qual estratégia serve para copiar ou persistir.

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: structuredClone suporta dados estruturados compatíveis; não clona todo valor executável. JSON tem perdas próprias. | Próxima sugestão: 06.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.06 JSON quebrado e tipos distorcidos

COMPARE · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: serialization.parse_failures | Requer: arrays.basic | Introduz: serialization.parse_failures

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Depure lerSave para retornar null em texto JSON sintaticamente inválido e aceitar um objeto JSON válido, como {"vida":0}. Em uma investigação separada no mesmo starter, serialize e restaure a amostra fornecida; registre o que acontece com Date, undefined, NaN, Infinity, Map e Set. Não implemente validação de esquema neste cartão: essa etapa vem depois do parse.

Objetivo da atividade

Diferencie falha de sintaxe JSON de JSON válido; investigue perdas e conversões no round-trip de Date, undefined, NaN, Infinity, Map e Set.

Explicação e modelo mental

• Modelo mental: try executa o parse; se JSON.parse lançar por sintaxe inválida, catch recebe o erro e retorna null. O catch não valida esquema e não captura um retorno normal inesperado.
• Exemplo resolvido: Texto com aspas simples em volta de chave faz JSON.parse lançar e o catch retorna null. Texto válido com tema numérico retorna um objeto normalmente; só uma validação de esquema posterior pode rejeitá-lo.
• Armadilha comum: Confundir o catch da sintaxe com validação: JSON válido não entra no catch só por ter campos errados. Date, Map, NaN e undefined têm limitações próprias na serialização.
• Por que importa: Dados persistidos podem ser válidos como JSON e incompatíveis com o modelo atual.

Material/arquivo inicial

main.js: export function lerSave(texto){ try{return JSON.parse(texto)}catch(e){return null} }
const amostra={data:new Date("2026-01-01T00:00:00.000Z"),ausente:undefined,nan:NaN,infinito:Infinity,mapa:new Map([["a",1]]),conjunto:new Set([1])};
const textoRoundTrip=JSON.stringify(amostra);
console.log(textoRoundTrip);
console.log(lerSave(textoRoundTrip));

Exemplo e resultado público

texto '{invalido' e texto '{"vida":0}' → null; objeto parseado com vida 0 | JSON.stringify({data:Date UTC,ausente:undefined,nan:NaN,infinito:Infinity,mapa:Map,conjunto:Set}) seguido de JSON.parse → data é string ISO; ausente não aparece; nan e infinito são null; mapa e conjunto são {}; os tipos especiais não são reconstruídos

Bordas e erros publicados

JSON.stringify({x:NaN,y:Infinity,z:undefined}) -> {"x":null,"y":null}; Date vira string ISO; Map/Set viram objetos vazios

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

JSON.parse lança SyntaxError quando o texto não é JSON válido. try { ... } catch (erro) { ... } executa o bloco catch quando algo dentro do try lança, e o programa continua a partir dali. | Map e Set são coleções da etapa 07: JSON.stringify não os converte sozinho. Converta antes: [...mapa] (pares [chave, valor]) e new Map(pares) na volta; [...set] e new Set(valores) fazem o mesmo para um Set.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

JSON.parse terminou sem erro. O que isso garante?

• Todos campos obrigatórios corretos
• Datas e mapas preservados
 (Correta) Sintaxe JSON válida, não conformidade ao esquema

Por que a resposta está certa

Parse valida o formato textual; a aplicação ainda verifica tipo, campos e invariantes.

Armadilha da alternativa incorreta

JSON válido ainda pode ter forma inesperada. Faça validação estrutural após o parse.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

JSON.stringify({x:NaN,y:Infinity,z:undefined}) → {"x":null,"y":null}; Date vira string ISO; Map/Set viram objetos vazios

Concepção errada e falha observável

Achar que parse válido prova esquema correto ou round-trip preserva todos os tipos. / Save parseado pode ter tipo inesperado ou campo silenciosamente perdido.

Pergunta diagnóstica

O problema é sintaxe inválida, dado válido com forma errada ou perda na serialização?

Pistas graduais

1. Separe try/catch de validação de esquema. 2. Inspecione saída de stringify para valores especiais. 3. Não restaure Map/Date sem política explícita.

Solução de referência explicada

Retorne null para JSON inválido; para JSON válido, valide campos mínimos em etapa seguinte. Documente perdas de NaN, Infinity, undefined, Date, Map e Set.

Transferência em outro contexto

Projete leitura de configuração de painel distinguindo texto corrompido de objeto válido com versão/tipo incompatível; descreva validação e resposta segura.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JSON.parse/stringify no baseline; validação estrutural é etapa do app. | Próxima sugestão: 06.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.07 Indexador por id

MAKE · 24 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: objects.index_by_id | Requer: arrays.basic | Introduz: objects.index_by_id

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 2 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente indexar(entidades) com o contrato visível: o último registro de um id duplicado vence, a lista vazia produz um objeto vazio e id "__proto__" é uma chave própria comum sem alterar o protótipo nem poluí-lo. Não altere os itens recebidos; escolha uma estrutura que cumpra os três casos.

Objetivo da atividade

Converta entidades em objeto indexado por id; duplicatas usam o último registro, lista vazia retorna objeto vazio e id __proto__ permanece chave de dados própria sem alterar o protótipo.

Explicação e modelo mental

• Modelo mental: Índice agrupa entidades por chave; estrutura define política de chave e duplicata. Objeto sem protótipo evita colisão com propriedades herdadas.
• Exemplo resolvido: Catalogando espécimes por código, percorra com Object.create(null); ao repetir código, atribuição posterior substitui a anterior.
• Armadilha comum: Objeto comum pode confundir chave externa com comportamento herdado; declarar política de duplicata é necessário.
• Por que importa: Chaves externas precisam continuar sendo dados ao criar índices.

Material/arquivo inicial

main.js: export function indexar(entidades){ /* retorne objeto simples por id */ }

Exemplo e resultado público

[{id:"a",v:1},{id:"b",v:2}] → índice com a.v=1 e b.v=2 | [{id:"a",v:1},{id:"a",v:3}], [], [{id:"__proto__",v:1}] → a.v=3 (último vence); objeto vazio; __proto__ é chave própria com v=1 e o protótipo não muda

Bordas e erros publicados

[{id:"a",v:1},{id:"a",v:3}] -> a.v=3 (último vence); [] -> objeto vazio; id "__proto__" -> dado comum sem poluir protótipo

Teste público e aprovação visível

Verifique saída, política de duplicata, ausência de mutação dos itens e ausência de efeito no protótipo para todos os casos publicados.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

Objetos comuns herdam propriedades do protótipo (etapa 10). Em { __proto__: valor }, a forma especial do literal define o protótipo se valor for objeto ou null; obj.__proto__ = valor pode acionar o setter herdado de um objeto comum. Já { ['__proto__']: valor } cria uma chave de dados. Este contrato pede um objeto indexado: use Object.create(null), sem protótipo; Map só serviria se a API mudasse para retornar um Map.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao construir um índice de registros por um campo identificador, o que cada entrada do índice deve associar?

 (Correta) O identificador do registro àquele registro, para permitir busca direta.
• A posição atual do registro como única informação, descartando o próprio registro.
• Uma lista de todos os campos do registro transformados em nomes de propriedades.

Por que a resposta está certa

Um índice relaciona a chave escolhida ao dado que será recuperado por ela.

Armadilha da alternativa incorreta

Pense no que a consulta precisa devolver: o índice deve preservar a associação entre a chave e o registro.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[{id:"a",v:1},{id:"a",v:3}] → a.v=3 (último vence); [] -> objeto vazio; id "__proto__" -> dado comum sem poluir protótipo

Concepção errada e falha observável

Usar objeto comum e atribuição direta sem considerar chave especial e duplicata. / Chave __proto__ altera protótipo ou entrada duplicada tem resultado indefinido.

Pergunta diagnóstica

Qual é a política pública para duplicatas e chaves especiais?

Pistas graduais

1. Crie índice com Object.create(null). 2. Percorra em ordem; atribuição posterior vence. 3. Object.hasOwn serve para consultar sem confundir protótipo.

Solução de referência explicada

const out=Object.create(null); for(const e of entidades) out[e.id]=e; return out. Último id igual prevalece.

Transferência em outro contexto

Indexe remessas por código de rastreio. Defina duplicatas e explique como tratar chaves externas como dados.

Evidência de domínio

Resultado observável: [{id:"a",v:1},{id:"b",v:2}] -> índice com a.v=1,b.v=2. Borda a conferir: [{id:"a",v:1},{id:"a",v:3}] -> a.v=3 (último vence); [] -> objeto vazio; id "__proto__" -> dado comum sem poluir protótipo. Entrega esperada: const out=Object.create(null); for(const e of entidades) out[e.id]=e; return out. Último id igual prevalece. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Object.create(null) e Object.hasOwn no baseline. | Próxima sugestão: 06.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.08 Transferencia - config versionada

MODIFY · 30 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: storage.migration_v1_v2 | Requer: arrays.basic | Introduz: storage.migration_v1_v2

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente a migração por versão: trate dados antigos/atuais, ausência, corrupção e versão futura. Faça primeiro a matriz de decisões e preserve valores válidos.

Objetivo da atividade

Preencha matriz de versão e estado: atual, anterior, texto inválido, campo ausente, futuro.

Explicação e modelo mental

• Modelo mental: Separe parse, detecção de versão, migração, defaults ausentes e recusa de formato futuro.
• Exemplo resolvido: Preferência v1 guarda escala na raiz; v2 agrupa em exibição. Parseie, reconheça v1 e construa v2 preservando valor; default só para campo ausente.
• Armadilha comum: || substitui zero ou string vazia válidos; interpretar versão futura como antiga pode destruir dados.
• Por que importa: Migração explícita preserva dados ao atualizar o app.

Material/arquivo inicial

main.js: // 06.08 · Configuração versionada.
// Entrada: texto JSON. Saída: configuração na versão atual (2).
// v1 = { version: 1, volume } -> { version: 2, audio: { volume } }
// v2 = { version: 2, audio: { volume } } -> mantida
// texto corrompido (JSON inválido) ou sem versão conhecida -> { version: 2, audio: { volume: 50 } }
// v1 sem volume -> volume padrão 50
// versão futura (por exemplo 3) -> { status: "unsupported", version: 3 } (não tente migrar)
// Preserve valores válidos, inclusive volume 0. Faça primeiro a matriz de decisões.

export function migrar(texto) {
// implemente
}

Exemplo e resultado público

JSON de {version:1,volume:0} → {version:2,audio:{volume:0}}; v2 válido -> mantido semanticamente

Bordas e erros publicados

JSON corrompido -> defaults {version:2,audio:{volume:50}}; v1 sem volume -> default 50; version futura 3 -> {status:"unsupported",version:3}; texto original preservado

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

JSON.parse lança SyntaxError quando o texto não é JSON válido; envolva a leitura em try { ... } catch (erro) { ... } para escolher o que fazer com a falha.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o objetivo central de uma migração de formato persistido?

• Tentar processá-la como a versão antiga removendo campos desconhecidos
 (Correta) Recusar com status de versão não suportada e não regravar o save
• Carregar defaults atuais e sobrescrever o save futuro

Por que a resposta está certa

Migração adapta a estrutura persistida sem mudar o sentido dos dados que o programa já conhece.

Armadilha da alternativa incorreta

Migrar ou regravar formato desconhecido pode descartar dados. Retorne status de recusa sem alterar o save.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

JSON corrompido → defaults {version:2,audio:{volume:50}}; v1 sem volume -> default 50; version futura 3 -> {status:"unsupported",version:3}; texto original preservado

Concepção errada e falha observável

Usar || e perder zero ou sobrescrever versão futura desconhecida. / Migração muda volume 0 para 50 ou destrói save de versão 3.

Pergunta diagnóstica

O valor está ausente, é zero válido ou a versão é desconhecida?

Pistas graduais

1. Faça parse seguro. 2. Trate v1 e v2 separadamente. 3. Use ?? para volume e retorne status unsupported para versão futura.

Solução de referência explicada

Parse seguro; v1 constrói novo objeto v2 com volume ?? 50; v2 valida campo; corrupto usa defaults; version >2 retorna {status:'unsupported',version} sem gravar nem alterar o texto.

Transferência em outro contexto

Migre preferências de leitura para formato novo que agrupa modo e fonte. Defina versões antiga, atual, corrompida e futura, preservando valores válidos.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: JSON e ?? no baseline; versão futura retorna status de recusa sem regravação. | Próxima sugestão: 06.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.09 Microprojeto - save de jogo robusto

PROJECT · 70 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: serialization.game_save | Requer: arrays.basic | Introduz: serialization.game_save

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 3 · leitura 5 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Antes do código, registre o contrato de cada caso público: V2, migração V1 sem itens, vida negativa, campo extra e texto inválido (que restaura defaults V2). Implemente salvar/restaurar em passos. Em nota separada, sem alterar o esquema V2, compare JSON para persistência/interoperabilidade com globalThis.structuredClone para cópia local e diga como futuras datas, Map e Set seriam representados.

Objetivo da atividade

Registre casos e invariantes; acrescente uma decisão curta sobre Date, Map/Set e por que cópia estruturada local não substitui formato de persistência.

Explicação e modelo mental

• Modelo mental: Persistir ou exportar exige representação serializável e versionada; structuredClone é cópia local de dados estruturados suportados, não formato textual interoperável.
• Exemplo resolvido: Se uma sessão de leitura guardar updatedAt: Date, tags: Set e posições: Map, JSON.stringify transforma Date em string ISO e não registra as entradas de Map/Set automaticamente. Defina schema JSON (Date como ISO; Map/Set como arrays explícitos), depois valide e reconstrua. structuredClone preserva Date/Map/Set suportados para cópia em memória, sem produzir texto de intercâmbio.
• Armadilha comum: Esperar round-trip JSON que devolva Date/Map/Set originais ou usar structuredClone como arquivo exportável. O esquema atual do save continua limitado aos campos V2 publicados.
• Por que importa: Persistência durável e intercâmbio precisam de contratos estáveis; cópia profunda local atende outra finalidade.

Material/arquivo inicial

main.js: // 06.09 · Save de jogo robusto.
// Formato V2 (texto JSON): { "version": 2, "nome": string, "vida": número, "itens": [string] }
// Formato V1 (save antigo, sem itens): { "version": 1, "nome": string, "vida": número }
// Estado em memória: { nome, vida, itens } (sem o campo version)
//
// salvar(estado) -> texto JSON V2 do estado.
// restaurar(texto):
// V2 válido -> estado { nome, vida, itens }
// V1 válido -> estado migrado, com itens = []
// campo extra no JSON -> ignorado (não aparece no estado)
// vida negativa -> { status: "invalid" } (não é aceita como estado)
// texto que não é JSON -> defaults V2: { nome: "Aventureiro", vida: 10, itens: [] }
// restaurar(salvar(estado)) devolve os mesmos campos e valores.
//
// Em nota separada (sem mudar o esquema): compare JSON para persistência com structuredClone para cópia local,
// e diga como datas futuras, Map e Set seriam representados.

```
export function salvar(estado) {
// implemente
}

export function restaurar(texto) {
// implemente
}
```

Exemplo e resultado público

estado {nome:"Ana",vida:5,itens:["chave"]} → JSON v2; restaurar(salvar(estado)) -> mesmos campos/valores

Bordas e erros publicados

texto inválido -> defaults v2; save v1 sem itens -> migra para itens=[]; vida negativa -> {status:"invalid"}; não é aceita como estado restaurado; dado extra ignorado

Teste público e aprovação visível

Resultado observável: estado {nome:"Ana",vida:5,itens:["chave"]} -> JSON v2; restaurar(salvar(estado)) -> mesmos campos/valores. Borda a conferir: texto inválido -> defaults v2; save v1 sem itens -> migra para itens=[]; vida negativa -> {status:"invalid"}; não é aceita como estado restaurado; dado extra ignorado. Entrega esperada: salvar serializa apenas version, nome, vida e itens do esquema V2. restaurar retorna defaults V2 para texto inválido, migra V1, valida tipos e vida>=0; para dados inválidos retorna {status:"invalid"} sem expor um estado restaurado. JSON é a representação persistente/interoperável; se versões futuras incluírem Date/Map/Set, converta-os em ISO/arrays explícitos e valide ao reconstruir. globalThis.structuredClone copia Date/Map/Set suportados em memória, mas não é serialização.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

try { ... } catch (erro) { ... } captura o erro lançado por JSON.parse com texto inválido. | Map e Set (etapa 07) não viram JSON diretamente: salve [...colecao] e reconstrua com new Map(...) ou new Set(...).

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Um save versionado precisa sobreviver a reinícios e ser exportado; uma futura versão contém Date, Map e Set. Qual decisão distingue persistência de cópia local?

• Chamar structuredClone e tratar o resultado como texto interoperável sem conversão
• Fazer JSON.stringify/parse e presumir que Date volta a Date e Map/Set preservam entradas
 (Correta) Definir JSON versionado, serializar Date como ISO, Map/Set como arrays explícitos e validar ao reconstruir; structuredClone fica para cópia local

Por que a resposta está certa

JSON versionado define representação exportável; tipos especiais exigem encoding e reconstrução. structuredClone copia localmente dados suportados.

Armadilha da alternativa incorreta

structuredClone não é formato textual; JSON não restaura tipos especiais automaticamente. Declare encoding e validação no schema.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

texto inválido → defaults v2; save v1 sem itens -> migra para itens=[]; vida negativa -> {status:"invalid"}; não é aceita como estado restaurado; dado extra ignorado

Concepção errada e falha observável

Confiar que JSON parseado já é estado válido ou ignorar validação de vida. / Restaurar aceita vida negativa ou perde os itens ausentes de um save antigo.

Pergunta diagnóstica

Como decidir entre JSON e structuredClone ao persistir um estado e representar datas ou coleções especiais?

Pistas graduais

1. Separe serialização, parse, migração e validação de domínio. 2. Preserve o esquema V2 atual; não acrescente campos ao save sem nova versão e contrato. 3. Em nota separada, compare JSON explícito (Date para ISO, Map/Set para arrays/entradas e validação ao reconstruir) com structuredClone para cópia local; structuredClone não cria texto interoperável.

Solução de referência explicada

salvar serializa apenas version, nome, vida e itens do esquema V2. restaurar retorna defaults V2 para texto inválido, migra V1, valida tipos e vida>=0; para dados inválidos retorna {status:"invalid"} sem expor um estado restaurado. JSON é a representação persistente/interoperável; se versões futuras incluírem Date/Map/Set, converta-os em ISO/arrays explícitos e valide ao reconstruir. globalThis.structuredClone copia Date/Map/Set suportados em memória, mas não é serialização.

Transferência em outro contexto

Implemente migração de uma lista de tarefas persistida: V1 não tem campo concluídas e V2 adiciona essa coleção; para tarefas antigas, o resultado deve conter a coleção vazia. Preserve os outros dados, valide o resultado e explique como manteve o conteúdo existente.

Evidência de domínio

Resultado observável: estado {nome:"Ana",vida:5,itens:["chave"]} -> JSON v2; restaurar(salvar(estado)) -> mesmos campos/valores. Borda a conferir: texto inválido -> defaults v2; save v1 sem itens -> migra para itens=[]; vida negativa -> {status:"invalid"}; não é aceita como estado restaurado; dado extra ignorado. Entrega esperada: salvar serializa apenas version, nome, vida e itens do esquema V2. restaurar retorna defaults V2 para texto inválido, migra V1, valida tipos e vida>=0; para dados inválidos retorna {status:"invalid"} sem expor um estado restaurado. JSON é a representação persistente/interoperável; se versões futuras incluírem Date/Map/Set, converta-os em ISO/arrays explícitos e valide ao reconstruir. globalThis.structuredClone copia Date/Map/Set suportados em memória, mas não é serialização. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: structuredClone (Node.js 17+ e navegadores evergreen) e JSON APIs no baseline; validação de inteiros; fallback de texto inválido preserva o contrato PDF. | Próxima sugestão: 06.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.10 Propriedade própria ou herdada

READ · 17 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: objects.own_inherited | Requer: arrays.basic | Introduz: objects.own_inherited

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Compare in, Object.hasOwn, Object.keys e descritor nas propriedades indicadas. Preencha a matriz prevista antes de rodar e explique próprio versus herdado.

Objetivo da atividade

Preencha matriz própria/herdada, in, hasOwn e keys para as propriedades antes de executar.

Explicação e modelo mental

• Modelo mental: in inclui propriedades herdadas; hasOwn só próprias; Object.keys lista próprias enumeráveis.
• Exemplo resolvido: Sessão herda idioma do protótipo e define apelido próprio: in encontra ambos, hasOwn só apelido e Object.keys enumera só o próprio.
• Armadilha comum: Tratar in como hasOwn ou keys como cadeia completa; método herdado pode existir sem pertencer ao objeto.
• Por que importa: Validação e enumeração corretas dependem da distinção entre dado próprio e herdado.

Material/arquivo inicial

main.js: const proto={cor:"azul"}; const obj=Object.create(proto); obj.nome="Ana"; /* compare hasOwn, in, keys e descriptor */

Exemplo e resultado público

"cor" in obj → true; Object.hasOwn(obj,"cor") -> false; Object.keys(obj) -> ["nome"]

Bordas e erros publicados

Object.getOwnPropertyDescriptor(obj,"nome").enumerable -> true; "toString" in obj -> true por cadeia de protótipos

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

Object.create(base) cria um objeto vazio cujo protótipo é base: ler uma chave que ele não tem consulta base. Object.hasOwn(obj, chave) diz se a chave é do próprio objeto. O modelo completo de protótipos é a etapa 10.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em um descritor de propriedade, o que controla a flag writable?

 (Correta) Se o valor de uma propriedade de dados pode ser substituído por atribuição.
• Se a propriedade aparece na enumeração de chaves próprias.
• Se uma propriedade pode ser removida ou redefinida.

Por que a resposta está certa

Correto: writable controla a possibilidade de atribuir outro valor à propriedade de dados.

Armadilha da alternativa incorreta

As flags de descritor têm papéis distintos: writable trata atribuição; outras flags controlam enumeração e reconfiguração.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Object.getOwnPropertyDescriptor(obj,"nome").enumerable → true; "toString" in obj -> true por cadeia de protótipos

Concepção errada e falha observável

Achar que in verifica apenas propriedade própria ou que keys inclui herdadas. / Resultado trata cor como own ou enumera cor via Object.keys.

Pergunta diagnóstica

A propriedade existe no objeto ou foi encontrada na cadeia?

Pistas graduais

1. Compare in com Object.hasOwn. 2. Object.keys mostra próprias enumeráveis. 3. Descriptor de nome mostra enumerable; descriptor próprio de cor é undefined.

Solução de referência explicada

cor é herdada: in true, hasOwn false, keys só nome; descritor próprio de cor não existe.

Transferência em outro contexto

Inspecione sessão com idioma herdado e apelido enumerável próprio. Registre in, hasOwn e keys e escolha teste apropriado para validar campo recebido.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Object.hasOwn, in, Object.keys e descritores no baseline. | Próxima sugestão: 06.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

06.11 Acessos opcionais e padrões precisos

MAKE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: objects.optional_access_defaults | Requer: arrays.basic | Introduz: objects.optional_access_defaults

Eixos: novidade 3 · sintaxe 3 · composição 2 · estado 2 · depuração 3 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Implemente resumirPerfil(usuario) com acesso opcional ao objeto perfil. Se perfil estiver presente, valide que seja um objeto não-array; retorne null para outro tipo. Quando cada campo estiver ausente/null, use nome:"Visitante", pontos:100, ativo:true e mensagem:"Sem mensagem". Uma configuração pode ser 0, false ou ""; conserve esses valores, aplicando padrão somente a null/undefined.

Objetivo da atividade

Preveja o resultado para usuario ausente e para perfil com 0, false, "" e tipo inválido antes de executar.

Explicação e modelo mental

• Modelo mental: Optional chaining obj?.campo interrompe o acesso quando obj é null ou undefined e devolve undefined. Nullish coalescing valor ?? padrao usa o padrão apenas para null ou undefined; 0, false e "" continuam sendo valores presentes. Esses operadores não validam se o valor tem o tipo esperado.
• Exemplo resolvido: Em config?.som?.volume ?? 5, caminho ausente produz 5, mas volume=0 permanece 0. Do mesmo modo, config?.avisos?.ativo ?? true conserva false. Um guard clause separado confere o tipo de config antes de ler suas propriedades.
• Armadilha comum: || escolhe o padrão para qualquer valor falsy, incluindo 0, false e ""; ?. evita erro de acesso quando algo é nullish, mas não confirma que o valor é um objeto válido.
• Por que importa: Preferências e dados recebidos podem omitir ramos inteiros; valores falsy também podem representar escolhas válidas do usuário.

Material/arquivo inicial

main.js: export function resumirPerfil(usuario) {
const perfil = usuario?.perfil;
if (perfil !== null && perfil !== undefined && (typeof perfil !== "object" || Array.isArray(perfil))) {
// Retorne null para um contêiner presente que não seja um registro.
}
return {
nome: "",
pontos: 0,
ativo: false,
mensagem: "",
};
}

Exemplo e resultado público

usuario={perfil:{nome:"Ana",pontos:0,ativo:false,mensagem:""}} → {nome:"Ana",pontos:0,ativo:false,mensagem:""}

Bordas e erros publicados

usuario=null -> {nome:"Visitante",pontos:100,ativo:true,mensagem:"Sem mensagem"}; usuario={perfil:{}} -> mesmos padrões; pontos=undefined usa 100, mas pontos=0 permanece 0; perfil="inválido" ou [] -> null

Teste público e aprovação visível

Critérios desta atividade: Encadeamento ?. protege usuario, perfil e cada leitura opcional. ?? aplica cada valor padrão somente a null/undefined. A saída preserva 0, false e string vazia. Uma guard clause retorna null para perfil que não é registro; ?. não substitui validação.

Restrições e documentação

Não use encadeamentos com && para acessar perfil.; Não use || para aplicar os padrões.; Perfil ausente/null é permitido; perfil presente deve ser objeto não-array, senão retorne null.; A função deve retornar as quatro propriedades no contrato quando perfil for válido.. Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

export function resumirPerfil(usuario) é o invólucro pronto para que o avaliador importe a função; preserve export, o nome e o parâmetro e implemente o retorno pedido. Módulos são aprofundados na etapa 14.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual expressão fornece um padrão quando pontos é 0, preservando esse zero?

• pontos || 100
 (Correta) pontos ?? 100
• pontos && 100

Por que a resposta está certa

Correto: ?? troca somente null e undefined; zero é mantido.

Armadilha da alternativa incorreta

|| considera 0 falsy e o substituiria; && também não é operador de padrão.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

usuario=null → {nome:"Visitante",pontos:100,ativo:true,mensagem:"Sem mensagem"}; usuario={perfil:{}} -> mesmos padrões; pontos=undefined usa 100, mas pontos=0 permanece 0; perfil="inválido" ou [] -> null

Concepção errada e falha observável

Usar && ou || para acessar campo opcional; usar || como padrão apaga valores válidos 0, false e string vazia; confiar em ?. como validação de tipo. / O acesso a usuario.perfil falha quando usuario é null; defaults com || substituem zero, false e ""; ?. acessa com segurança, mas não valida perfil como objeto.

Pergunta diagnóstica

Qual operador interrompe o acesso quando o caminho não existe, qual valor indica ausência e que guard clause rejeita perfil que não é um registro?

Pistas graduais

1. Encadeie usuario?.perfil?.campo para que cada acesso possa estar ausente. 2. Use ?? para fornecer padrão apenas se o resultado for null ou undefined. 3. Antes das leituras, retorne null se perfil presente não for objeto ou for array; ?. não valida tipos. Confira 0, false e "".

Solução de referência explicada

Optional chaining retorna undefined quando uma parte do caminho é null/undefined; nullish coalescing escolhe o padrão somente para null/undefined e conserva 0, false e "". Nenhum deles confirma o tipo dos valores.

Transferência em outro contexto

Leia preferências de conta opcional. usuario={perfil:{nome:"",notificacoes:false,atalho:""}} -> {nome:"", notificacoes:false, atalho:""}; usuario={perfil:"inválido"} -> null; usuario=null -> nome padrão. Explique por que || mudaria duas escolhas válidas.

Evidência de domínio

Critérios desta atividade: Encadeamento ?. protege usuario, perfil e cada leitura opcional. ?? aplica cada valor padrão somente a null/undefined. A saída preserva 0, false e string vazia. Uma guard clause retorna null para perfil que não é registro; ?. não substitui validação. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Optional chaining e nullish coalescing fazem parte do JavaScript padrão no Node.js 24 LTS; sem dependências externas. | Próxima sugestão: 07.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07 Callbacks, coleções e pipelines

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 07.01 · 07.02 · 07.03 · 07.04 · 07.05 · 07.06 · 07.07 · 07.10 · 07.08 · 07.09

07.01 Escolha o método

COMPARE · 15 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: arrays.method_choice | Requer: functions.return, arrays.basic | Introduz: arrays.method_choice

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Classifique as 12 intenções pelo formato pedido (coleção nova, subconjunto, primeiro item, sim/não ou acumulado), depois preencha método e justificativa. Por fim, resolva separadamente 'primeiro valor maior que 10' e 'quantos valores maiores que 10'.

Objetivo da atividade

Nos primeiros dois minutos, marque cada intenção como coleção, item, booleano ou acumulado; ainda não escolha métodos nem execute código.

Explicação e modelo mental

• Modelo mental: Escolha pelo formato da resposta. map produz um valor para cada entrada; filter conserva as entradas que passam; find devolve o primeiro elemento correspondente; some e every respondem com booleano; reduce carrega um acumulador até o fim.
• Exemplo resolvido: Exemplo análogo, fora da tabela: para [4, 7, 4], a pergunta “há algum 7?” exige sim/não, então some é apropriado; “qual é o primeiro 7?” exige o próprio elemento, então find é apropriado. A mesma entrada leva a operações diferentes porque muda a forma da resposta.
• Armadilha comum: find não devolve a coleção inteira e some não devolve o item encontrado: o primeiro retorna o item ou undefined; o segundo retorna somente booleano. filter é adequado quando todos os elementos correspondentes devem permanecer.
• Por que importa: O tipo de retorno determina o próximo passo: percorrer uma coleção, inspecionar um item, ramificar por booleano ou usar um total. Declarar essa forma torna pipelines legíveis e reduz erros ao compor métodos.

Material/arquivo inicial

main.js:
```
const metodos = ["map", "filter", "find", "some", "every", "reduce", "map", "filter", "find", "some", "every", "reduce"];

export function classificarIntencoes() {
return metodos;
}

console.log(JSON.stringify(classificarIntencoes()));
```

Exemplo e resultado público

respostas na ordem → map,filter,find,some,every,reduce,map,filter,find,some,every,reduce

Bordas e erros publicados

intenção 'primeiro maior que 10' -> find; 'quantos maiores que 10' -> filter(...).length ou reduce justificado

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

MDN Array: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual método é adequado quando cada item deve virar um valor correspondente, mantendo um resultado para cada posição?

• filter, que remove os itens cuja condição não passa.
• some, que resume a coleção a uma resposta booleana.
 (Correta) map, que transforma cada item e retorna uma nova coleção.

Por que a resposta está certa

map estabelece uma transformação item a item; filter seleciona e some responde se há correspondência.

Armadilha da alternativa incorreta

Escolha pelo formato do resultado: transformação por item pede uma coleção transformada, não seleção ou booleano.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

intenção 'primeiro maior que 10' → find; 'quantos maiores que 10' -> filter(...).length ou reduce justificado

Concepção errada e falha observável

Escolher método pela última aula, não pelo formato do resultado. / Usa map para boolean ou find para array completo.

Pergunta diagnóstica

A saída pedida é array, item, boolean ou acumulado?

Pistas graduais

1. Classifique o tipo de resultado. 2. Para um item, find; para boolean, some/every. 3. Use reduce quando realmente há acumulação.

Solução de referência explicada

A lista pública associa cada intenção à operação mais direta; alternativas equivalentes são aceitas quando a justificativa explica o trade-off.

Transferência em outro contexto

Examine logs e responda: existe algum ERROR? Qual é o primeiro ERROR? Para cada pergunta, identifique o formato da resposta e justifique sua escolha.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; usar APIs padrão deste runtime, preservar o contrato de saída e aplicar o fallback explicitamente descrito quando a API moderna não estiver garantida. | Próxima sugestão: 07.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.02 GroupBy moderno

MAKE · 24 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: arrays.groupby | Requer: functions.return, arrays.basic | Introduz: arrays.groupby

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Implemente agrupar(pedidos) com retorno Map. Mostre o caso público e compare Map.groupBy, Object.groupBy e fallback, sem supor suporte fora do runtime declarado.

Objetivo da atividade

Escreva o tipo exato do retorno e o que acontece quando uma chave aparece pela primeira vez.

Explicação e modelo mental

• Modelo mental: Quando categorias surgem conforme os dados, Map associa cada chave ao seu próprio bucket.
• Exemplo resolvido: Exemplo análogo: três livros chegam em ordem romance, poesia, romance. Crie cada bucket na primeira aparição e acrescente o livro; o bucket romance mantém os dois livros na ordem em que chegaram.
• Armadilha comum: Um objeto genérico pode confundir nomes especiais com propriedades herdadas; detecte suporte da API e mantenha o tipo Map prometido.
• Por que importa: Agrupar mensagens por nível ou pedidos por estado fica simples quando cada bucket é uma lista explícita.

Material/arquivo inicial

main.js: // 07.02 · Agrupar pedidos por status.
// pedidos: [{ id, status }]. Devolva um Map: status -> array dos pedidos desse status (objetos originais),
// na ordem em que aparecem. Lista vazia devolve um Map vazio. Não altere a entrada.
// Compare Map.groupBy, Object.groupBy e um laço de fallback para runtimes sem os métodos novos.

export function agrupar(pedidos) {
// implemente
}

Exemplo e resultado público

[{id:1,status:"novo"},{id:2,status:"pago"},{id:3,status:"novo"}] → novo ids [1,3], pago [2]

Bordas e erros publicados

[] -> grupos vazios; status repetido -> itens permanecem no grupo; ordem dentro de cada grupo preservada

Teste público e aprovação visível

Resultado observável: [{id:1,status:"novo"},{id:2,status:"pago"},{id:3,status:"novo"}] -> novo ids [1,3], pago [2]. Borda a conferir: [] -> grupos vazios; status repetido -> itens permanecem no grupo; ordem dentro de cada grupo preservada. Entrega esperada: Retorne Map para aceitar qualquer chave. Object.groupBy retorna objeto; Map.groupBy retorna Map. Use Map.groupBy quando disponível ou Map+loop como fallback; não substitua um contrato Map por objeto.

Restrições e documentação

MDN Map.groupBy(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/groupBy ; MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual diferença de retorno existe entre Map.groupBy e Object.groupBy, e que alternativa mantém um retorno Map quando a API não está disponível?

 (Correta) Map.groupBy devolve Map; Object.groupBy devolve objeto; Map com loop mantém o contrato Map.
• Map.groupBy e Object.groupBy devolvem sempre objetos simples.
• Object.groupBy retorna Map e aceita as mesmas chaves que Map.groupBy.
• O fallback deve converter buckets em objeto e descartar a ordem de inserção.

Por que a resposta está certa

Correto. O tipo de retorno faz parte da API pública: Map.groupBy produz Map, Object.groupBy produz objeto, e Map com loop é fallback coerente quando o contrato pede Map.

Armadilha da alternativa incorreta

Compare os tipos retornados: Object.groupBy cria objeto; Map.groupBy cria Map. A compatibilidade precisa preservar o tipo prometido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → grupos vazios; status repetido -> itens permanecem no grupo; ordem dentro de cada grupo preservada

Concepção errada e falha observável

Usar objeto comum de forma insegura ou depender de API indisponível sem fallback. / Pedido com status especial some ou implementação quebra no baseline.

Pergunta diagnóstica

O runtime oferece Object.groupBy/Map.groupBy? Que estrutura evita colisão de chave?

Pistas graduais

1. Escolha Map.groupBy quando as chaves podem ser qualquer valor. 2. Para objeto, use Object.groupBy quando disponível. 3. Fallback: Map e loop, preservando ordem.

Solução de referência explicada

Retorne Map para aceitar qualquer chave. Object.groupBy retorna objeto; Map.groupBy retorna Map. Use Map.groupBy quando disponível ou Map+loop como fallback; não substitua um contrato Map por objeto.

Transferência em outro contexto

Agrupe erros por nível, incluindo eventos cujo nível está vazio, e preserve a ordem de chegada dentro de cada grupo. Entregue o Map e um teste que torne observáveis as chaves e a ordem.

Evidência de domínio

Resultado observável: [{id:1,status:"novo"},{id:2,status:"pago"},{id:3,status:"novo"}] -> novo ids [1,3], pago [2]. Borda a conferir: [] -> grupos vazios; status repetido -> itens permanecem no grupo; ordem dentro de cada grupo preservada. Entrega esperada: Retorne Map para aceitar qualquer chave. Object.groupBy retorna objeto; Map.groupBy retorna Map. Use Map.groupBy quando disponível ou Map+loop como fallback; não substitua um contrato Map por objeto. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Feature-detect Map.groupBy/Object.groupBy; Map+loop fallback avoids assuming either API. Keep the required return type consistent. | Próxima sugestão: 07.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.03 Parsons de ranking

PARSONS · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: arrays.pipeline_ranking | Requer: functions.return, arrays.basic | Introduz: arrays.pipeline_ranking

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Ordene os blocos. Anote quando cada jogador já tem pontuação e escreva comparador completo, inclusive desempate, antes de conferir a ordem final.

Objetivo da atividade

Anote os critérios de ordenação na ordem em que devem ser aplicados.

Explicação e modelo mental

• Modelo mental: Ordenar objetos exige comparar pelos critérios definidos e na prioridade especificada. O comparador devolve sinal coerente: negativo quando o primeiro vem antes, positivo quando vem depois, zero quando empatam em todos os critérios.
• Exemplo resolvido: Imagine ordenar durações [14, 6]. Compare os dois valores, confira que trocar a ordem dos argumentos troca o sinal do comparador e verifique que cada duração continua aparecendo uma vez. Esse teste pequeno avalia a consistência da ordenação sem decidir critérios de um relatório.
• Armadilha comum: Não converta pontos numéricos em texto nem deixe o segundo critério sobrescrever uma diferença decisiva do primeiro. Confira também se a ordenação altera o array que deveria permanecer intacto.
• Por que importa: Relatórios ordenados influenciam o que aparece primeiro. Critérios explícitos e testes para empate deixam a regra revisável.

Material/arquivo inicial

main.js: export function ranking(partidas) {
return partidas
.filter(partida => partida.valida)
.map(partida => ({id: partida.id, jogador: partida.jogador, pontos: partida.pontos}))
.sort((a, b) => a.pontos - b.pontos || a.jogador.localeCompare(b.jogador))
.map(partida => partida.id);
}

console.log(JSON.stringify(ranking([
{id: "a", jogador: "Ana", valida: true, pontos: 8},
{id: "b", jogador: "Bia", valida: true, pontos: 5},
])));

Exemplo e resultado público

Ana 10, Bia 10, Cai 8 → Ana,Bia,Cai; partida inválida não gera pontos

Bordas e erros publicados

dois nomes iguais e pontos iguais -> ordem estável de entrada; entrada não mutada

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

MDN Array.prototype.sort(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual ordem de critérios produz o ranking e desempata os resultados?

• Ordenar nomes antes de calcular pontos e depois manter partidas inválidas.
 (Correta) Pontuar partidas válidas; ordenar pontos decrescentes; no empate ordenar nome crescente.
• Ordenar só pelos pontos e deixar o empate sem regra.
• Ordenar a entrada original antes de filtrar ou calcular pontos.

Por que a resposta está certa

Os pontos precisam existir antes da comparação; o segundo critério resolve empates e a cópia preserva a entrada.

Armadilha da alternativa incorreta

Os pontos precisam existir antes da comparação; o segundo critério resolve empates e a cópia preserva a entrada. Compare a saída pedida com o tipo e a ordem produzidos por cada opção.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

dois nomes iguais e pontos iguais → ordem estável de entrada; entrada não mutada

Concepção errada e falha observável

Ordenar antes de calcular pontos ou esquecer critério de desempate. / Jogadores empatados aparecem em ordem errada ou partida inválida afeta ranking.

Pergunta diagnóstica

Em que momento cada jogador já tem pontos e qual é a segunda chave da ordenação?

Pistas graduais

1. Filtre inválidos primeiro. 2. Calcule pontos antes de comparar. 3. No empate, compare nomes com localeCompare; use toSorted/cópia.

Solução de referência explicada

A→B→C→D; comparador pontos desc, depois nome asc, sem mutar dados originais.

Transferência em outro contexto

Produza ranking de equipes a partir de partidas válidas. Entregue o comparador, aplique desempate por nome e comprove que a entrada não foi alterada.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; usar APIs padrão deste runtime, preservar o contrato de saída e aplicar o fallback explicitamente descrito quando a API moderna não estiver garantida. | Próxima sugestão: 07.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.04 Complete callbacks

FADED · 14 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: callbacks.transform_orders | Requer: functions.return, arrays.basic | Introduz: callbacks.transform_orders

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Complete os callbacks. Para cada um, indique tipo recebido e devolvido; explique por que limite igual não satisfaz uma condição estrita maior que.

Objetivo da atividade

Marque cada callback como decisão, transformação ou pergunta booleana antes de preencher.

Explicação e modelo mental

• Modelo mental: Cada etapa de um pipeline recebe o resultado da etapa anterior. Antes de encadear, anote o tipo do valor: um filtro costuma receber registros inteiros; depois de projetar uma propriedade, a próxima função pode receber um escalar.
• Exemplo resolvido: Uma lista de consultas médicas começa com objetos de consulta. Uma etapa seleciona registros; outra os representa por minutos de atraso. Anote ao lado de cada etapa o tipo de entrada e de saída para não escrever a próxima condição como se ainda recebesse o objeto completo.
• Armadilha comum: Não use o nome de uma propriedade do objeto depois que a etapa anterior já transformou o registro em número. Confira o tipo em cada limite do pipeline.
• Por que importa: Pipelines legíveis ajudam a depurar transformações compostas e a detectar se uma pergunta booleana está sendo feita sobre os valores pretendidos.

Material/arquivo inicial

main.js: // 07.04 · Complete os callbacks.
// pedidos: [{ ativo: boolean, total: number }].
// ativos: só os pedidos ativos. totais: o total de cada pedido ativo, na mesma ordem.
// existeCaro: algum total é ESTRITAMENTE maior que o limite (limite igual não conta).
// Para cada callback, anote o tipo recebido e o tipo devolvido.

export function analisarPedidos(pedidos, limite) {
const ativos = pedidos.filter(___);
const totais = ativos.map(___);
const existeCaro = totais.some(___);
return { totais, existeCaro };
}

Exemplo e resultado público

pedidos [{ativo:true,total:5},{ativo:false,total:99},{ativo:true,total:20}] → totais [5,20], existeCaro true para limite 10

Bordas e erros publicados

[] -> totais [], existeCaro false; limite 20 estrito -> false quando maior que 20 é exigido

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

MDN Array methods: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual etapa cria a lista de totais e qual responde se algum total supera o limite?

• filter extrai totais; map responde true/false.
• some devolve o primeiro total; find responde se todos superam o limite.
 (Correta) map extrai totais; some responde se existe algum acima do limite.
• every cria uma lista; reduce pergunta se existe total acima do limite.

Por que a resposta está certa

map transforma cada item; some retorna booleano. Cada método corresponde a uma saída diferente.

Armadilha da alternativa incorreta

map transforma cada item; some retorna booleano. Cada método corresponde a uma saída diferente. Compare a saída pedida com o tipo e a ordem produzidos por cada opção.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → totais [], existeCaro false; limite 20 estrito -> false quando maior que 20 é exigido

Concepção errada e falha observável

Misturar predicado, transformação e pergunta booleana. / Pedido inativo entra em totais ou some retorna array.

Pergunta diagnóstica

Qual callback deve retornar boolean e qual deve retornar novo valor?

Pistas graduais

1. filter decide manter item. 2. map extrai total. 3. some pergunta se existe valor > limite.

Solução de referência explicada

filter(p=>p.ativo), map(p=>p.total), some(total=>total>limite).

Transferência em outro contexto

Monte um pipeline que filtre chamados abertos e transforme cada um em sua prioridade. Responda se existe algum chamado urgente; entregue a lista transformada e o valor booleano com testes para a pergunta.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; usar APIs padrão deste runtime, preservar o contrato de saída e aplicar o fallback explicitamente descrito quando a API moderna não estiver garantida. | Próxima sugestão: 07.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.05 Set operations

MAKE · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: sets.operations | Requer: functions.return, arrays.basic | Introduz: sets.operations

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente interseção, diferença direcional e subconjunto com conjuntos novos. Explique qual lado fornece A−B e cubra entrada vazia.

Objetivo da atividade

Complete: A−B contém membros que estão em ___ e não em ___.

Explicação e modelo mental

• Modelo mental: Em conjuntos, a interseção conserva membros presentes nos dois; A−B conserva membros de A que não estão em B; subconjunto verifica se todo membro de B também está em A.
• Exemplo resolvido: Considere duas listas de convites para sessões: manhã/noite e noite/fim de semana. Primeiro marque o item compartilhado; depois confira cada item da primeira lista contra a segunda, sem trocar a lista de origem. Esse procedimento ajuda a revisar uma comparação nova sem reutilizar os dados do exercício.
• Armadilha comum: Trocar A−B por B−A muda a origem dos membros. O conjunto vazio é subconjunto de qualquer conjunto porque não há membro que viole a condição.
• Por que importa: Comparações de permissões e recursos ficam mais seguras quando pertencimento e direção são explícitos.

Material/arquivo inicial

main.js: // 07.05 · Operações entre conjuntos de permissões.
// a e b são Sets. Devolva um objeto com conjuntos NOVOS (não altere a nem b):
// intersecao -> membros presentes em a e em b
// diferenca -> a − b (membros de a que NÃO estão em b; a ordem importa)
// subconjunto -> true quando todo membro de b também está em a (b □ a); o conjunto vazio é subconjunto de qualquer um
// Compare com os métodos nativos intersection, difference e isSubsetOf, se o runtime os tiver.

export function compararPermissoes(a, b) {
// implemente
}

Exemplo e resultado público

a={ler,editar}, b={ler} → intersecao={ler}, diferença(a-b)={editar}, b subconjunto de a=true

Bordas e erros publicados

b vazio -> subconjunto=true (o vazio é subconjunto de qualquer conjunto); a vazio -> intersecao e diferença vazias; entradas preservadas

Teste público e aprovação visível

Resultado observável: a={ler,editar}, b={ler} -> intersecao={ler}, diferença(a-b)={editar}, b subconjunto de a=true. Borda a conferir: b vazio -> subconjunto=true (o vazio é subconjunto de qualquer conjunto); a vazio -> intersecao e diferença vazias; entradas preservadas. Entrega esperada: Implemente primeiro com loops e has, retornando conjuntos novos e boolean. Como extensão opcional, compare com Set.intersection/difference/isSubsetOf se o runtime oferecer suporte.

Restrições e documentação

MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que pertence a A−B e quando B é subconjunto de A?

• A−B contém membros de B ausentes de A; B□A se A está em B.
• A−B guarda membros compartilhados; subconjunto exige conjuntos de tamanhos iguais.
• A−B remove exclusivos de B de B; o conjunto vazio não é subconjunto.
□ (Correta) A−B contém membros de A ausentes de B; B□A se todo membro de B está em A.

Por que a resposta está certa

A−B percorre A e exclui o que também está em B. Para B□A, procure um contraexemplo em B.

Armadilha da alternativa incorreta

A−B percorre A e exclui o que também está em B. Para B□A, procure um contraexemplo em B. Compare a saída pedida com o tipo e a ordem produzidos por cada opção.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

b vazio → subconjunto=true (o vazio é subconjunto de qualquer conjunto); a vazio -> intersecao e diferença vazias; entradas preservadas

Concepção errada e falha observável

Confundir diferença direcional ou supor que conjunto vazio não é subconjunto. / a-b contém permissão exclusiva de b.

Pergunta diagnóstica

De qual conjunto os elementos são removidos na diferença?

Pistas graduais

1. Interseção: pertence aos dois. 2. Diferença a-b: pertence a a e não a b. 3. Todo elemento do conjunto vazio satisfaz condição de subconjunto.

Solução de referência explicada

Implemente primeiro com loops e has, retornando conjuntos novos e boolean. Como extensão opcional, compare com Set.intersection/difference/isSubsetOf se o runtime oferecer suporte.

Transferência em outro contexto

Compare as tags de dois artigos. Entregue as tags de A que não aparecem em B e um teste que deixe explícita a direção da diferença A−B.

Evidência de domínio

Resultado observável: a={ler,editar}, b={ler} -> intersecao={ler}, diferença(a-b)={editar}, b subconjunto de a=true. Borda a conferir: b vazio -> subconjunto=true (o vazio é subconjunto de qualquer conjunto); a vazio -> intersecao e diferença vazias; entradas preservadas. Entrega esperada: Implemente primeiro com loops e has, retornando conjuntos novos e boolean. Como extensão opcional, compare com Set.intersection/difference/isSubsetOf se o runtime oferecer suporte. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Loops e has são o caminho principal; métodos modernos de Set são extensão opcional dependente do runtime. | Próxima sugestão: 07.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.06 Reduce ilegível

MODIFY · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: arrays.readable_reduction | Requer: functions.return, arrays.basic | Introduz: arrays.readable_reduction

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Reproduza a saída do reduce antes de refatorar. Reescreva em passos legíveis e compare os resultados nos mesmos dados; o objetivo é melhorar a leitura sem alterar o comportamento.

Objetivo da atividade

Registre a saída atual para categoria repetida e o estado do acumulador antes de refatorar.

Explicação e modelo mental

• Modelo mental: Uma redução atualiza um resultado à medida que percorre itens. Uma refatoração legível nomeia as decisões importantes e mantém o mesmo comportamento observável.
• Exemplo resolvido: Em outra refatoração, uma cadeia de ternários que escolhe o rótulo de um pedido pode virar uma função com um caso nomeado por estado. Compare alguns estados antes e depois; a leitura fica mais simples sem alterar a interface nem os rótulos apresentados.
• Armadilha comum: Uma expressão compacta pode esconder agrupamento, inicialização e comportamento de chaves especiais. Prefira um loop claro quando ele tornar essas regras auditáveis.
• Por que importa: Quem revisa precisa conseguir conferir a agregação, inclusive quando zero e chaves incomuns são dados válidos.

Material/arquivo inicial

main.js: export function resumirPorCategoria(itens) {
const resumo = itens.reduce((acumulado, item) => {
acumulado[item.categoria] = item.preco;
return acumulado;
}, {});
return resumo;
}

console.log(JSON.stringify(resumirPorCategoria([
{categoria: "A", preco: 2},
{categoria: "A", preco: 3},
{categoria: "B", preco: 4},
])));

Exemplo e resultado público

itens com categorias A:2,A:3,B:4 → totais A=5,B=4; entrada intacta

Bordas e erros publicados

preço 0 -> categoria ainda existe com total 0; categorias comuns mantêm totais independentes

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map ; MDN reduce(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que a função de redução precisa devolver em cada chamada do callback?

 (Correta) O próximo acumulador que será entregue à etapa seguinte.
• O array completo de entrada para reiniciar o percurso.
• Um efeito colateral, pois o valor retornado pelo callback é ignorado.

Por que a resposta está certa

Cada retorno atualiza o acumulador; a etapa seguinte recebe esse valor.

Armadilha da alternativa incorreta

O retorno do callback não é descartado: ele define o estado acumulado para o próximo passo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

preço 0 → categoria ainda existe com total 0; categorias comuns mantêm totais independentes

Concepção errada e falha observável

Achar que reduce em uma linha é sempre melhor que etapas nomeadas. / Leitor não consegue separar agrupamento e soma ou um acumulador não preserva o total anterior.

Pergunta diagnóstica

Que duas operações de domínio a expressão comprime?

Pistas graduais

1. Nomeie a etapa de agrupar e a de somar. 2. Use Map para chaves arbitrárias. 3. Aceite loop claro se ele expressa melhor a intenção.

Solução de referência explicada

Refatore para Map de totais com for...of ou funções agrupar/somar; preserve a saída e compare cada total com o reduce original.

Transferência em outro contexto

Refatore contagem de erros por serviço. Entregue versão legível e evidência de que totais zero e nomes especiais continuam representados.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS como base; use APIs padrão, preserve o contrato de saída e aplique o fallback descrito quando o suporte à API moderna não estiver garantido. | Próxima sugestão: 07.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.07 Contagem e agrupamento

MAKE · 28 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: map.frequency_grouping | Requer: functions.return, arrays.basic | Introduz: map.frequency_grouping

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente frequencias e porCategoria com Map e laços. Apresente os retornos lado a lado e diga por que o valor associado a uma chave tem tipo diferente em cada função.

Objetivo da atividade

Escreva o tipo de valor armazenado em cada Map: número ou lista de registros.

Explicação e modelo mental

• Modelo mental: Contagem e agrupamento usam chaves para reunir itens, mas respondem a perguntas diferentes: um resumo de quantidades ou a consulta dos próprios registros.
• Exemplo resolvido: Num catálogo de biblioteca, “quantos empréstimos cada filial registrou?” pede um resumo; “quais exemplares saíram de cada filial?” pede detalhes. Decida a representação pela pergunta que o próximo leitor precisa responder e mantenha os dois relatórios com responsabilidades claras.
• Armadilha comum: Um objeto sem política clara para chaves pode confundir contagem e detalhes ou tratar nomes especiais de modo inesperado.
• Por que importa: Logs frequentemente pedem totais e detalhes agrupados; separar os resultados evita descartar informação necessária.

Material/arquivo inicial

main.js: // 07.07 · Contagem e agrupamento com Map e laços (sem reduce).
// frequencias(palavras): Map palavra -> quantidade (Number), na ordem da primeira aparição.
// porCategoria(registros): registros = [{ categoria, id }]. Map categoria -> array de ids (na ordem de chegada).
// Diga por que o valor associado a uma chave tem tipo diferente em cada função (número x array).

```
export function frequencias(palavras) {
// implemente
}

export function porCategoria(registros) {
// implemente
}
```

Exemplo e resultado público

["a","b","a"] → Map(a:2,b:1); registros A1,B2,A3 -> grupos A:[1,3],B:[2]

Bordas e erros publicados

[] -> Map vazio/grupos vazios; chaves repetidas acumulam no mesmo grupo; ordem por primeira aparição preservada

Teste público e aprovação visível

Resultado observável: ["a","b","a"] -> Map(a:2,b:1); registros A1,B2,A3 -> grupos A:[1,3],B:[2]. Borda a conferir: [] -> Map vazio/grupos vazios; chaves repetidas acumulam no mesmo grupo; ordem por primeira aparição preservada. Entrega esperada: frequencias percorre palavras em Map. porCategoria usa Map+loop; Map.groupBy pode ser consultado como extensão se o runtime oferecer suporte.

Restrições e documentação

MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map ; MDN Map.groupBy(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/groupBy

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

No contador de frequências, que valor fica associado à palavra azul depois de processá-la duas vezes?

 (Correta) O número 2, guardado como contador no Map.
• A lista dos índices em que azul aparece.
• O booleano true, pois a palavra já existe.
• O primeiro objeto azul encontrado na entrada.

Por que a resposta está certa

Correto. A função de frequências associa cada chave ao total de ocorrências; o agrupamento por categoria associa uma lista de registros.

Armadilha da alternativa incorreta

Pergunte qual informação a função retorna: frequência é contagem numérica. Uma lista pertence ao agrupamento; booleano informa apenas existência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → Map vazio/grupos vazios; chaves repetidas acumulam no mesmo grupo; ordem por primeira aparição preservada

Concepção errada e falha observável

Usar uma única representação para contagens e grupos de registros. / Contagem duplica chave ou grupo perde registros/ordem.

Pergunta diagnóstica

As chaves são texto confiável e a ordem importa?

Pistas graduais

1. Use Map para frequência de palavras. 2. Antes de incrementar, consulte o contador atual no Map e diferencie chave ausente de contagem já registrada. 3. Agrupe por categoria com Map.groupBy ou loop.

Solução de referência explicada

frequencias percorre palavras em Map. porCategoria usa Map+loop; Map.groupBy pode ser consultado como extensão se o runtime oferecer suporte.

Transferência em outro contexto

Conte códigos de erro e agrupe requisições por serviço. Entregue os dois Maps e identifique o tipo de valor de cada um.

Evidência de domínio

Resultado observável: ["a","b","a"] -> Map(a:2,b:1); registros A1,B2,A3 -> grupos A:[1,3],B:[2]. Borda a conferir: [] -> Map vazio/grupos vazios; chaves repetidas acumulam no mesmo grupo; ordem por primeira aparição preservada. Entrega esperada: frequencias percorre palavras em Map. porCategoria usa Map+loop; Map.groupBy pode ser consultado como extensão se o runtime oferecer suporte. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; Map.groupBy (ES2024, Node.js 21+) é a API moderna e o agrupamento manual com Map é o fallback explicitamente descrito; preservar o contrato de saída. | Próxima sugestão: 07.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.10 Expandir resultados com flatMap()

MAKE · 21 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: arrays.flatmap | Requer: functions.return, arrays.basic | Introduz: arrays.flatmap

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Implemente linhas(pedidos) com flatMap. Mostre como cada pedido produz zero ou várias linhas e compare com map(...).flat(1), preservando a ordem.

Objetivo da atividade

Anote quantas linhas produzem pedido ativo com itens, ativo vazio e cancelado.

Explicação e modelo mental

• Modelo mental: Quando cada registro pode gerar várias linhas, transforme-o em uma lista de saídas e achate exatamente um nível, preservando a ordem dos registros e das linhas produzidas. Uma lista vazia representa zero linhas.
• Exemplo resolvido: Uma estante com dois livros contribui com duas linhas para um catálogo; uma estante sem livros não acrescenta linhas. Cada estante produz uma lista local de linhas, e o relatório junta essas listas em uma só, mantendo a ordem de visita.
• Armadilha comum: Retornar null para um registro sem linhas cria um valor na saída, e achatar um nível não percorre recursivamente objetos aninhados.
• Por que importa: Expansões de registros em linhas aparecem em relatórios e exportações. Tornar explícito o nível de achatamento evita linhas extras e perda de ordem.

Material/arquivo inicial

main.js: // 07.10 · Expandir pedidos em linhas com flatMap.
// pedidos: [{ id, status: "ativo" | "cancelado", itens: [...] }].
// Cada pedido ativo gera uma linha { pedido: id, item } por item; pedido cancelado ou sem itens gera zero linhas.
// Preserve a ordem de pedidos e itens. Compare com map(...).flat(1). Não achate o conteúdo de um item.

```
export function linhas(pedidos) {
// implemente
}
```

Exemplo e resultado público

pedido ativo id=1 com itens [A,B], cancelado id=2 com [C] → linhas [{pedido:1,item:A},{pedido:1,item:B}]

Bordas e erros publicados

[] -> []; ativo sem itens -> []; ordem de pedidos/itens preservada; não achatar objetos aninhados

Teste público e aprovação visível

Resultado observável: pedido ativo id=1 com itens [A,B], cancelado id=2 com [C] -> linhas [{pedido:1,item:A},{pedido:1,item:B}]. Borda a conferir: [] -> []; ativo sem itens -> []; ordem de pedidos/itens preservada; não achatar objetos aninhados. Entrega esperada: return pedidos.flatMap(p=>p.cancelado?[]:p.itens.map(item=>({pedido:p.id,item})));

Restrições e documentação

MDN Array.prototype.flatMap(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flatMap

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando cada item de entrada deve corresponder a exatamente um valor transformado, qual método expressa melhor essa relação?

• filter, que seleciona apenas itens que passam por uma condição.
 (Correta) map, que aplica uma transformação a cada item.
• reduce, que combina os itens em um único acumulado.

Por que a resposta está certa

map mantém a relação item a item; filter seleciona e reduce agrega.

Armadilha da alternativa incorreta

Escolha conforme a forma desejada da saída: transformação individual, seleção ou agregação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → []; ativo sem itens -> []; ordem de pedidos/itens preservada; não achatar objetos aninhados

Concepção errada e falha observável

Supor que flatMap achata arbitrariamente ou que pedido cancelado deve retornar null. / Saída contém subarrays ou linha de pedido cancelado.

Pergunta diagnóstica

O callback retorna zero, uma ou várias linhas para cada pedido?

Pistas graduais

1. Cancelado retorna []. 2. Ativo mapeia seus itens para array de linhas. 3. flatMap remove um nível do array produzido por pedido.

Solução de referência explicada

return pedidos.flatMap(p=>p.cancelado?[]:p.itens.map(item=>({pedido:p.id,item})));

Transferência em outro contexto

Expanda faturas em linhas de produto e ignore as anuladas. Entregue as linhas em ordem original e compare duas formas de achatar um nível.

Evidência de domínio

Resultado observável: pedido ativo id=1 com itens [A,B], cancelado id=2 com [C] -> linhas [{pedido:1,item:A},{pedido:1,item:B}]. Borda a conferir: [] -> []; ativo sem itens -> []; ordem de pedidos/itens preservada; não achatar objetos aninhados. Entrega esperada: return pedidos.flatMap(p=>p.cancelado?[]:p.itens.map(item=>({pedido:p.id,item}))); Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; usar APIs padrão deste runtime, preservar o contrato de saída e aplicar o fallback explicitamente descrito quando a API moderna não estiver garantida. | Próxima sugestão: 07.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.08 Transferencia - relatorio de vendas

MAKE · 24 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: arrays.aggregate_transfer | Requer: functions.return, arrays.basic | Introduz: arrays.aggregate_transfer

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Leia as doações e implemente três métricas: total por campanha, doadores únicos e campanha de maior arrecadação. Declare o desempate pela primeira campanha vista e o resultado para lista vazia; registre a previsão e confira que a entrada ficou intacta.

Objetivo da atividade

Preveja o Map de totais e o Set de doadores para a entrada publicada antes de implementar.

Explicação e modelo mental

• Modelo mental: Map associa cada campanha a um acumulador de valor; Set representa identidades únicas. As estruturas guardam métricas distintas da mesma lista.
• Exemplo resolvido: Duas doações para a campanha A, de 5 e 7, produzem Map A=>12; doadores ana e bia produzem Set {ana,bia}. Uma terceira doação de ana não aumenta o tamanho do Set.
• Armadilha comum: Não use Set para totais nem incremente um contador simples de doadores por linha. Valor zero pode ser uma doação válida e a entrada deve permanecer sem alteração.
• Por que importa: Escolher a coleção pelo significado do dado torna os acumuladores fáceis de conferir e transfere o padrão para relatórios de vendas ou presença.

Material/arquivo inicial

main.js: // 07.08 · Relatório de doações.
// doacoes: [{ campanha, doador, valor }]. Devolva um objeto:
// totais -> Map campanha -> soma dos valores (valor 0 é válido; campanha repetida acumula)
// doadores -> Set com cada doador uma única vez
// maiorCampanha -> nome da campanha de maior total; empate fica com a campanha vista primeiro; lista vazia -> null
// Não altere a entrada. Registre sua previsão antes de executar.

export function resumoDoacoes(doacoes) {
// implemente
}

Exemplo e resultado público

doações [{campanha:'A',doador:'ana',valor:5},{campanha:'A',doador:'bia',valor:7},{campanha:'B',doador:'ana',valor:0}] → totais Map {A=>12,B=>0}; doadores Set {ana,bia}; maior campanha A; entrada preservada

Bordas e erros publicados

lista vazia -> Map e Set vazios, maior campanha null; campanha repetida acumula; valor 0 continua válido; empate de total favorece a campanha vista primeiro; entrada não é mutada

Teste público e aprovação visível

Resultado observável: doações
[{campanha:'A',doador:'ana',valor:5},{campanha:'A',doador:'bia',valor:7},{campanha:'B',doador:'ana',valor:0}] -> totais Map {A=>12,B=>0}; doadores Set {ana,bia}; maior campanha A; entrada preservada. Borda a conferir: lista vazia -> Map e Set vazios, maior campanha null; campanha repetida acumula; valor 0 continua válido; empate de total favorece a campanha vista primeiro; entrada não é mutada. Entrega esperada: Percorra as doações uma vez, acumule valores em um Map e doadores em um Set. A campanha com maior total é a de maior soma; empate favorece a primeira campanha encontrada. Para lista vazia, retorne null. Não ordene nem mute a entrada.

Restrições e documentação

MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map ; MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set ; MDN sort(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual estrutura representa corretamente os doadores únicos?

• Um Map que soma o valor das doações da mesma pessoa.
 (Correta) Um Set de identificadores, que mantém uma ocorrência por pessoa.
• Um contador que aumenta para cada linha, mesmo quando o doador se repete.

• A campanha com o maior total, usada como identidade do doador.

Por que a resposta está certa

Correto. Set expressa identidade única; Map acumula valores por campanha.

Armadilha da alternativa incorreta

Diferencie identidade e soma: Set elimina identificadores repetidos, enquanto Map guarda o acumulado de cada campanha.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

lista vazia → Map e Set vazios, maior campanha null; campanha repetida acumula; valor 0 continua válido; empate de total favorece a campanha vista primeiro; entrada não é mutada

Concepção errada e falha observável

Somar valores em Set, contar o mesmo doador mais de uma vez ou deixar indefinido o que significa a campanha do topo. / A campanha escolhida não tem a maior soma, o empate ignora a primeira aparição, zero é tratado como ausência ou a entrada foi alterada.

Pergunta diagnóstica

Qual coleção representa totais por chave e qual representa identidades sem repetição?

Pistas graduais

1. Use Map para acumular valor por campanha. 2. Use Set para guardar cada identificador de doador uma vez. 3. Retorne ambos e compare a entrada antes/depois para verificar que não foi mutada.

Solução de referência explicada

Percorra as doações uma vez, acumule valores em um Map e doadores em um Set. A campanha com maior total é a de maior soma; empate favorece a primeira campanha encontrada. Para lista vazia, retorne null. Não ordene nem mute a entrada.

Transferência em outro contexto

Em um relatório de vendas, use Map para total por categoria e Set para clientes únicos. Entregue uma entrada pequena com uma categoria e um cliente repetidos e explique por que as duas estruturas produzem respostas diferentes.

Evidência de domínio

Resultado observável: doações [{campanha:'A',doador:'ana',valor:5},{campanha:'A',doador:'bia',valor:7},{campanha:'B',doador:'ana',valor:0}] -> totais Map {A=>12,B=>0}; doadores Set {ana,bia}; maior campanha A; entrada preservada. Borda a conferir: lista vazia -> Map e Set vazios, maior campanha null; campanha repetida acumula; valor 0 continua válido; empate de total favorece a campanha vista primeiro; entrada não é mutada. Entrega esperada: Percorra as doações uma vez, acumule valores em um Map e doadores em um Set. A campanha com maior total é a de maior soma; empate favorece a primeira campanha encontrada. Para lista vazia, retorne null. Não ordene nem mute a entrada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; usar APIs padrão deste runtime, preservar o contrato de saída e aplicar o fallback explicitamente descrito quando a API moderna não estiver garantida. | Próxima sugestão: 07.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

07.09 Microprojeto - analisador de sessão

PROJECT · 65 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: arrays.session_metrics | Requer: functions.return, arrays.basic | Introduz: arrays.session_metrics

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 4 · depuração 4 · arquivos 4 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Implemente analisar(eventos) com entrada temporalmente fora de ordem. Inclua um caso vazio e um empate de frequência; defina desempate pela primeira ação vista no array original, calcule duração sem depender da ordem de chegada e prove que a entrada não foi mutada.

Objetivo da atividade

Escreva como obter duração quando o primeiro evento não for o mais antigo.

Explicação e modelo mental

• Modelo mental: Duração é máximo menos mínimo dos tempos; contagem por ação, ações únicas e taxa de sucesso são métricas separadas. A ação mais frequente é o item mais comum; empate favorece primeira aparição no array original.
• Exemplo resolvido: Uma trilha de reprodução pode registrar eventos fora de ordem. Antes de calcular a duração, compare os instantes e não assuma que o primeiro registro é o mais antigo. Mantenha separadas a contagem de ações, a ação mais frequente e a proporção de eventos bem-sucedidos.
• Armadilha comum: Assuming timestamps are sorted can produce a negative or incomplete duration; ties in frequency need a declared rule.
• Por que importa: Telemetry and audit summaries should remain correct when events arrive out of order.

Material/arquivo inicial

main.js: // 07.09 · Analisador de sessão.
// eventos: [{ tempo: number, acao: string, resultado: "sucesso" | "falha" }], possivelmente fora de ordem.
// Devolva { duracao, unicas, top, taxa }:
// duracao -> maior tempo menos menor tempo (não depende da ordem de chegada); lista vazia -> 0
// unicas -> quantidade de ações diferentes
// top -> ação mais frequente; empate fica com a PRIMEIRA ação vista no array original; vazio -> null
// taxa -> sucessos / total de eventos (vazio -> 0)
// Não altere o array recebido: ordene uma cópia se precisar.

export function analisar(eventos) {
// implemente
}

Exemplo e resultado público

eventos tempo 10 ataque sucesso, tempo 20 ataque falha, tempo 25 cura sucesso; lista vazia; empate na entrada: tempo 5 abrir sucesso, tempo 6 fechar sucesso, tempo 7 fechar falha, tempo 8 abrir falha → duração 15, únicas 2, top ataque, taxa 2/3; lista vazia retorna duração 0, únicas 0, top null, taxa 0; empate retorna duração 3, únicas 2, top abrir por aparecer primeiro na entrada original, taxa 1/2

Bordas e erros publicados

[] -> duração 0, únicas 0, top null, taxa 0; tempos fora de ordem -> ordenar cópia; empates de frequência -> primeira ação vista

Teste público e aprovação visível

Resultado observável: eventos tempo 10 ataque sucesso, tempo 20 ataque falha, tempo 25 cura sucesso; lista vazia; empate na entrada: tempo 5 abrir sucesso, tempo 6 fechar sucesso, tempo 7 fechar falha, tempo 8 abrir falha -> duração 15, únicas 2, top ataque, taxa 2/3; lista vazia retorna duração 0, únicas 0, top null, taxa 0; empate retorna duração 3, únicas 2, top abrir por aparecer primeiro na entrada original, taxa 1/2. Borda a conferir: [] -> duração 0, únicas 0, top null, taxa 0; tempos fora de ordem -> ordenar cópia; empates de frequência -> primeira ação vista. Entrega esperada: Valide eventos e preserve a entrada. Para lista vazia, retorne duração 0, nenhuma ação única, top null e taxa 0. Nos demais casos, calcule mínimo/máximo dos tempos, contagem com Map, únicas com Set e sucessos/total; resolva empate de frequência pela primeira ação na entrada original, mesmo se usar uma cópia ordenada para duração.

Restrições e documentação

MDN Date: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date ; MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map ; MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual característica torna um analisador de sessões mais fácil de repetir em testes automatizados?

• Consultar a hora atual durante cada cálculo para ordenar as ações.
• Manter resultados anteriores em uma variável compartilhada entre execuções.
 (Correta) Receber os eventos como entrada e produzir o resultado sem depender de relógio ou estado externo.

Por que a resposta está certa

Entradas explícitas e ausência de dependências ocultas tornam a execução reproduzível.

Armadilha da alternativa incorreta

Relógio e estado compartilhado introduzem variação externa; um analisador determinístico depende dos dados recebidos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → duração 0, únicas 0, top null, taxa 0; tempos fora de ordem -> ordenar cópia; empates de frequência -> primeira ação vista

Concepção errada e falha observável

Calcular duração pela ordem original quando eventos vêm desordenados. / Duração negativa ou top muda arbitrariamente no empate.

Pergunta diagnóstica

Você separou ordenação temporal de contagem por ação?

Pistas graduais

1. Faça cópia ordenada para duração. 2. Map conta ações, Set conta únicas. 3. Taxa é sucessos/total e lista vazia tem taxa zero.

Solução de referência explicada

Valide eventos e preserve a entrada. Para lista vazia, retorne duração 0, nenhuma ação única, top null e taxa 0. Nos demais casos, calcule mínimo/máximo dos tempos, contagem com Map, únicas com Set e sucessos/total; resolva empate de frequência pela primeira ação na entrada original, mesmo se usar uma cópia ordenada para duração.

Transferência em outro contexto

Analise eventos de checkout fora de ordem. Entregue duração, ações distintas, ação mais frequente e taxa de sucesso sem mutar a entrada. Se ações empatarem em frequência, preserve a que apareceu primeiro na entrada original; inclua um caso de empate que permita verificar essa regra.

Evidência de domínio

Resultado observável: eventos tempo 10 ataque sucesso, tempo 20 ataque falha, tempo 25 cura sucesso; lista vazia; empate na entrada: tempo 5 abrir sucesso, tempo 6 fechar sucesso, tempo 7 fechar falha, tempo 8 abrir falha -> duração 15, únicas 2, top ataque, taxa 2/3; lista vazia retorna duração 0, únicas 0, top null, taxa 0; empate retorna duração 3, únicas 2, top abrir por aparecer primeiro na entrada original, taxa 1/2. Borda a conferir: [] -> duração 0, únicas 0, top null, taxa 0; tempos fora de ordem -> ordenar cópia; empates de frequência -> primeira ação vista. Entrega esperada: Valide eventos e preserve a entrada. Para lista vazia, retorne duração 0, nenhuma ação única, top null e taxa 0. Nos demais casos, calcule mínimo/máximo dos tempos, contagem com Map, únicas com Set e sucessos/total; resolva empate de frequência pela primeira ação na entrada original, mesmo se usar uma cópia ordenada para duração. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; usar APIs padrão deste runtime, preservar o contrato de saída e aplicar o fallback explicitamente descrito quando a API moderna não estiver garantida. | Próxima sugestão: 08.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08 Algoritmos e complexidade

8 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 08.01 · 08.02 · 08.03 · 08.04 · 08.05 · 08.06 · 08.07 · 08.08

08.01 Classifique a complexidade

PREDICT · 17 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: algorithms.big_o_read | Requer: control.loop, arrays.basic, mapset.basic | Introduz: algorithms.big_o_read

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Use o starter descrito no contrato (snippets: acesso a[0]; varredura n; dois loops aninhados n; divisão por 2 repetida; sort; busca binária; Set.has médio; cópia de n itens). Antes de executar, escreva a saída/ordem e um traço por passo que justifique a transição decisiva; então compare com a execução.

Objetivo da atividade

Classifique trechos sequenciais e aninhados antes de calcular crescimento.

Explicação e modelo mental

• Modelo mental: Trabalhos em sequência somam; só aninhamento repete o custo interno para cada item externo.
• Exemplo resolvido: Um importador percorre o lote para contar linhas e depois percorre-o outra vez para validar extensões. São duas passagens; comparar cada arquivo com todos os demais criaria pares.
• Armadilha comum: Loops consecutivos não se multiplicam; inclua sort e cópias no custo. Conte repetições em função de n.
• Por que importa: Classificar etapas ajuda estimar o impacto de dobrar o lote antes de escolher uma implementação.

Material/arquivo inicial

main.js: export const analises = [
{trecho: "acesso por índice", crescimento: "O(1)"},
{trecho: "uma varredura", crescimento: "O(n)"},
{trecho: "dois laços aninhados", crescimento: "O(n²)"},
{trecho: "dividir por dois", crescimento: "O(log n)"},
{trecho: "ordenar", crescimento: "O(n log n)"},
{trecho: "busca binária", crescimento: "O(log n)"},
{trecho: "Set.has médio", crescimento: "O(1)"},
{trecho: "copiar n itens", crescimento: "O(n)"},
];

console.log(JSON.stringify(analises));

Exemplo e resultado público

ordens na sequência → O(1),O(n),O(n²),O(log n),O(n log n),O(log n),O(1) médio,O(n)

Bordas e erros publicados

loop n com corpo constante extra -> O(n); dois loops consecutivos n+n -> O(n), não O(n²)

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

MDN loops: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration ; MDN sort(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Um algoritmo ordena uma coleção e depois faz uma única varredura. Qual termo representa o crescimento dominante?

 (Correta) O(n log n), determinado pela ordenação, que domina a varredura linear.
• O(n²), porque duas operações sempre multiplicam seus custos.
• O(log n), porque uma ordenação reduz pela metade a coleção.

Por que a resposta está certa

Operações sequenciais somam custos; para tamanhos grandes, a ordenação domina a varredura linear.

Armadilha da alternativa incorreta

A contagem sequencial não é automaticamente multiplicativa; compare as taxas de crescimento das operações.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

loop n com corpo constante extra → O(n); dois loops consecutivos n+n -> O(n), não O(n²)

Concepção errada e falha observável

Multiplicar custos de loops consecutivos ou ignorar custo de sort/cópia. / A resposta diz O(n²) para dois loops sequenciais.

Pergunta diagnóstica

Os loops estão aninhados ou um termina antes do outro começar?

Pistas graduais

1. Conte repetições em função de n. 2. Aninhamento multiplica; sequência soma. 3. Ignore constantes e termos menores ao dar ordem dominante.

Solução de referência explicada

A tabela pública contém as oito classes e distingue pior caso/esperado quando a operação depende de implementação.

Transferência em outro contexto

Analise um lote que passa por uma varredura de leitura/validação e depois por ordenação. Entregue a ordem dominante e explique, usando n, por que as etapas são sequenciais ou aninhadas.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract's stated fallback for newer APIs. | Próxima sugestão: 08.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.02 Parsons recursivo

PARSONS · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: algorithms.recursion_tree | Requer: control.loop, arrays.basic | Introduz: algorithms.recursion_tree

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Use o starter descrito no contrato (blocos: A=if(no===null)return 0; B=const esquerda=somar(no.esq); C=const direita=somar(no.dir); D=return no.valor+esquerda+direita). Ordene ou selecione os blocos, entregue a sequência final e explique uma dependência entre etapas.

Objetivo da atividade

Preveja resultado para árvore vazia e folha antes de ordenar blocos.

Explicação e modelo mental

• Modelo mental: Recursão em árvore combina caso base, chamadas menores e composição de resultados.
• Exemplo resolvido: Para contar documentos, pasta ausente contribui zero; pasta existente soma seu documento e as contagens das subpastas. Cada chamada desce até ausência.
• Armadilha comum: Sem caso base, a chamada passa da folha; combinar antes de receber descendentes perde dados. Trate null primeiro.
• Por que importa: O mesmo padrão se aplica a diretórios, categorias e comentários hierárquicos.

Material/arquivo inicial

main.js: export function somar(no) {
if (no === null || no === undefined) return undefined;
const esquerda = somar(no.esq ?? null);
const direita = somar(no.dir ?? null);
return no.valor + esquerda + direita;
}

console.log(somar({valor: 2, esq: {valor: 3}, dir: {valor: 5}}));

Exemplo e resultado público

árvore raiz 1, filhos 2 e 3 → soma 6; ordem A antes de B/C antes de D

Bordas e erros publicados

árvore null -> 0; árvore de um nó valor -2 -> -2; B/C podem trocar

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

MDN Functions: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao escrever uma função recursiva para agregar uma estrutura em árvore, o que torna o problema menor a cada chamada?

• Repetir a chamada com a árvore inteira até o contador global mudar.
 (Correta) Aplicar a mesma função às subárvores e combinar os resultados.
• Pular a verificação de término e esperar que a pilha de chamadas limite o percurso.

Por que a resposta está certa

A recursão progride quando cada chamada trata subproblemas menores e combina seus resultados.

Armadilha da alternativa incorreta

Uma chamada precisa aproximar-se do caso base; repetir o mesmo problema não garante término.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

árvore null → 0; árvore de um nó valor -2 -> -2; B/C podem trocar

Concepção errada e falha observável

Esquecer caso base e recursar em null. / Stack overflow ou tentativa de ler propriedade de null.

Pergunta diagnóstica

O que interrompe a recursão na folha?

Pistas graduais

1. Trate null primeiro. 2. Calcule subárvores antes de compor. 3. A ordem entre esquerda e direita é livre se ambas são puras.

Solução de referência explicada

A→B/C→D; retorne 0 para null e some valor com resultados recursivos.

Transferência em outro contexto

Some os pesos associados a arquivos em uma árvore de pastas. Entregue a implementação e testes para árvore vazia, pasta folha e subpastas com pesos acumulados.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract's stated fallback for newer APIs. | Próxima sugestão: 08.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.03 Complete busca binária

FADED · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: algorithms.binary_search | Requer: control.loop, arrays.basic | Introduz: algorithms.binary_search

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Use o starter descrito no contrato (main.js: export function busca(xs,alvo){let lo=0,hi=xs.length-1;while(lo<=hi){const mid=Math.floor((lo+hi)/2); if(xs[mid]===alvo)return mid;…). Complete os espaços do starter, entregue a função e explique qual caso de borda verifica cada papel preenchido.

Objetivo da atividade

Registre lo/hi iniciais e o primeiro mid antes de executar.

Explicação e modelo mental

• Modelo mental: Busca binária mantém um intervalo ordenado candidato; o meio comparado sai do intervalo.
• Exemplo resolvido: Na lista [2,6,11,18,25], procure 18. Compare o centro; como o alvo é maior, mova lo para depois do meio. Repita até achar o alvo ou cruzar os limites.
• Armadilha comum: Manter mid depois de descartá-lo pode impedir progresso em intervalos pequenos. Lista é ascendente e já ordenada.
• Por que importa: Índices de catálogo ordenado podem ser localizados com bem menos comparações que uma varredura.

Material/arquivo inicial

main.js: // 08.03 · Busca binária em lista ORDENADA crescente.
// Devolve o índice de alvo ou -1. Com duplicatas, qualquer índice que contenha o alvo é aceito.
// Complete os dois ___ e explique qual caso de borda cada um protege ([], primeiro, último, ausente).

```
export function busca(xs, alvo) {
let lo = 0, hi = xs.length - 1;
while (lo <= hi) {
const mid = Math.floor((lo + hi) / 2);
if (xs[mid] === alvo) return mid;
if (xs[mid] < alvo) lo = ___;
else hi = ___;
}
return -1;
}
```

Exemplo e resultado público

xs=[1,3,5,7],alvo=5 → 2; alvo=2 -> -1

Bordas e erros publicados

[] -> -1; alvo=1 -> 0; alvo=7 -> 3; duplicatas -> qualquer índice com alvo é aceito neste contrato

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

MDN loops: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration ; MDN Array: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual pré-condição permite à busca binária descartar metade do intervalo de busca com segurança?

• A coleção contém somente números positivos.
• A coleção tem tamanho que é potência de dois.
 (Correta) Os elementos estão ordenados segundo a mesma relação usada pela busca.

Por que a resposta está certa

A ordem permite inferir de que lado o alvo poderia estar depois de comparar o meio.

Armadilha da alternativa incorreta

Sem ordenação compatível, uma comparação não informa qual metade pode ser descartada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → -1; alvo=1 -> 0; alvo=7 -> 3; duplicatas -> qualquer índice com alvo é aceito neste contrato

Concepção errada e falha observável

Atualizar ponteiro para mid em vez de mid±1 e não progredir. / Loop fica preso quando lo/hi se aproximam.

Pergunta diagnóstica

Depois de descartar mid, ele ainda pode permanecer no intervalo?

Pistas graduais

1. Lista é ascendente e já ordenada. 2. Se valor no meio é menor, descarte até mid inclusive. 3. Use lo=mid+1 e hi=mid-1.

Solução de referência explicada

Preencha lo=mid+1, hi=mid-1; mantenha while(lo<=hi).

Transferência em outro contexto

Busque SKU em outra lista ordenada. Entregue índice/ausência e traço lo/hi/mid.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 08.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.04 Invariantes

READ · 21 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: algorithms.loop_invariants | Requer: control.loop, arrays.basic | Introduz: algorithms.loop_invariants

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 3 · depuração 2 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Use o starter descrito no contrato (trechos: busca binária em array ordenado; soma prefixo xs[0..i); dois ponteiros para par alvo). Registre uma hipótese e a evidência que precisa coletar antes de inspecionar o material; entregue observação, conclusão e fonte/ferramenta quando aplicável.

Objetivo da atividade

Escreva uma propriedade verdadeira no prefixo vazio.

Explicação e modelo mental

• Modelo mental: Invariante descreve o prefixo já processado e continua válida após cada iteração.
• Exemplo resolvido: Ao achar mínimo em [8,3,9], o mínimo do primeiro prefixo é 8; depois do segundo é 3; no fim continua 3. A variável resume exatamente o prefixo visitado.
• Armadilha comum: Dizer só o resultado final não demonstra que o estado estava correto entre passos. Verifique inicialização.
• Por que importa: Invariantes ajudam localizar a transição onde um índice ou acumulador deixa de representar o prefixo.

Material/arquivo inicial

main.js: export function buscaBinaria(valores, alvo) {
let inicio = 0;
let fim = valores.length;
while (inicio < fim) {
const meio = Math.floor((inicio + fim) / 2);
if (valores[meio] < alvo) inicio = meio + 1;
else fim = meio;
}
return inicio < valores.length && valores[inicio] === alvo ? inicio : -1;
}

export function prefixo(valores, fim) {
return valores.slice(0, fim).reduce((total, valor) => total + valor, 0);
}

console.log(buscaBinaria([1, 3, 5, 7], 5), prefixo([2, 4, 6], 2));

Exemplo e resultado público

busca binária → se alvo existe, está em [lo,hi]; soma -> total é soma de elementos já processados; ponteiros -> pares descartados não podem satisfazer alvo

Bordas e erros publicados

em array vazio, intervalo da busca é vazio e retorno -1; invariante da soma com i=0 -> total=0

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

MDN loops: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration ; MDN Array: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual afirmação vale em cada passo de uma soma por prefixo?

• total sempre soma o array inteiro antes do loop.
• O objetivo é encontrar qualquer número grande.
 (Correta) total é a soma dos itens já processados.
• A propriedade só precisa valer no final.

Por que a resposta está certa

A invariante acompanha o prefixo e precisa ser inicializada e preservada em cada iteração.

Armadilha da alternativa incorreta

A invariante acompanha o prefixo e precisa ser inicializada e preservada em cada iteração. Compare a regra ao significado dos limites e ao resultado pedido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

em array vazio, intervalo da busca é vazio e retorno -1; invariante da soma com i=0 → total=0

Concepção errada e falha observável

Descrever objetivo final em vez de propriedade preservada a cada iteração. / Explicação não vale antes da primeira iteração ou após atualização.

Pergunta diagnóstica

A frase vale antes, durante e depois de cada passo?

Pistas graduais

1. Verifique inicialização. 2. Verifique preservação após um passo. 3. Mostre como término mais invariante implica resultado.

Solução de referência explicada

Escreva as três invariantes públicas com domínio dos índices e justifique inicialização/preservação/terminação.

Transferência em outro contexto

Defina invariante para varredura de mínimo. Entregue inicialização, preservação e vínculo com o resultado.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 08.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.05 Bug off-by-one

DEBUG · 17 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: algorithms.off_by_one | Requer: control.loop, arrays.basic | Introduz: algorithms.off_by_one

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 4 · arquivos 2 · leitura 3 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Corrija os dois limites no starter. Antes de finalizar, escreva a previsão para listas de tamanho 0, 1 e 3 e compare com os índices realmente acessados.

Objetivo da atividade

Liste último índice e tamanhos 0, 1 e 3 antes da correção.

Explicação e modelo mental

• Modelo mental: Em arrays, os índices começam em zero, mas length conta elementos. Trace quais índices existem e compare essa sequência com as tentativas de acesso; a fronteira muda entre vazio, singleton e coleção maior.
• Exemplo resolvido: Para uma fila com três códigos, escreva sob cada elemento o índice que ocupa. Simule a posição inicial, o avanço e a primeira tentativa que já não encontra elemento. Repita com fila vazia e com um único código antes de decidir a condição do seu loop.
• Armadilha comum: Não confunda o número de elementos com o último índice válido; em array vazio não existe índice inicial a visitar.
• Por que importa: Um traço de índices torna erros de fronteira visíveis antes de processar dados reais, como registros da última página.

Material/arquivo inicial

main.js: // 08.05 · Bug de off-by-one nos dois laços.
// visitarTodos deve chamar visitar(xs[i]) para TODOS os itens, uma vez cada.
// lerTodos deve chamar ler(xs[j]) para os índices 0..length-1, sem ler fora da lista.
// Antes de corrigir, escreva a previsão para listas de tamanho 0, 1 e 3.

```
export function visitarTodos(xs, visitar) {
for (let i = 0; i < xs.length - 1; i++) visitar(xs[i]);
}

export function lerTodos(xs, ler) {
for (let j = 0; j <= xs.length; j++) ler(xs[j]);
}
```

Exemplo e resultado público

xs=[A,B,C] → primeiro loop deve visitar A,B,C; segundo deve ler índices 0,1,2 sem undefined

Bordas e erros publicados

xs=[] -> ambos zero acessos; xs=[A] -> ambos visitam A exatamente uma vez

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

MDN loops: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration ; MDN Array: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se um percurso precisa apenas visitar valores, sem usar posições, qual escolha reduz erros de controle manual do índice?

 (Correta) Usar for...of para obter cada valor diretamente.
• Incrementar manualmente um índice e calcular o limite em dois lugares.
• Começar do último índice e subtrair em uma condição independente do tamanho.

Por que a resposta está certa

Quando a posição não é necessária, for...of remove a aritmética do índice da tarefa.

Armadilha da alternativa incorreta

O controle manual pode ser útil quando posições importam, mas é desnecessário para simples visita dos valores.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

xs=[] → ambos zero acessos; xs=[A] -> ambos visitam A exatamente uma vez

Concepção errada e falha observável

Trocar limite sem testar lista vazia e de um item. / Último elemento omitido ou acesso xs[length] ocorre.

Pergunta diagnóstica

Qual é o último índice válido e quando o loop deve parar?

Pistas graduais

1. Último índice é length-1. 2. Para percorrer todos, condição i<length. 3. Teste tamanhos 0,1,3.

Solução de referência explicada

Nos dois loops, use índice inicial 0 e condição <xs.length; preserve incremento unitário.

Transferência em outro contexto

Revise uma paginação que pode omitir o último registro. Entregue a condição corrigida e um teste de visitas que cubra lista vazia, singleton e confirme que o último item de uma lista maior é alcançado.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 08.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.06 Deduplique eficientemente

MAKE · 26 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: algorithms.deduplicate | Requer: control.loop, arrays.basic, mapset.basic | Introduz: algorithms.deduplicate

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Use o starter descrito no contrato (main.js: export function unicosLento(xs){} export function unicosSet(xs){}). Implemente a mudança e entregue código e evidência dos casos público e de borda; explique a regra que preserva o contrato.

Objetivo da atividade

Declare a regra de igualdade antes de escolher includes ou Set.

Explicação e modelo mental

• Modelo mental: Deduplicar preservando primeira aparição exige manter a ordem enquanto se reconhece se um valor já apareceu. Defina também o que significa igualdade para as chaves do domínio.
• Exemplo resolvido: Ao importar um catálogo de livros, duas linhas podem compartilhar o mesmo ISBN mesmo que tenham títulos abreviados de modo diferente. Antes de remover repetições, decida qual identificador define uma ocorrência e registre como a ordem original será mantida.
• Armadilha comum: Comparar representações textuais de objetos pode tratar objetos distintos como iguais; deduplicar também pode perder a ordem se a estrutura escolhida não a preservar.
• Por que importa: Logs, eventos e listas de IDs frequentemente precisam manter a primeira ocorrência para que a ordem reflita a origem.

Material/arquivo inicial

main.js: export function unicosLento(xs){} export function unicosSet(xs){}

Exemplo e resultado público

[a,b,a,c] → [a,b,c] nas duas versões, preservando primeira ocorrência

Bordas e erros publicados

[] -> []; [NaN,NaN] -> [NaN] por SameValueZero; objetos distintos com mesma forma -> mantidos distintos

Teste público e aprovação visível

Resultado observável: [a,b,a,c] -> [a,b,c] nas duas versões, preservando primeira ocorrência. Borda a conferir: [] -> []; [NaN,NaN] -> [NaN] por SameValueZero; objetos distintos com mesma forma -> mantidos distintos. Entrega esperada: Lento: percorra e inclua se não existe; rápido: [...new Set(xs)]. Explique O(n²) versus O(n) médio.

Restrições e documentação

MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como um algoritmo de deduplicação pode preservar a ordem da primeira aparição?

• Ordenar a entrada antes de remover repetidos e devolver a ordem classificada.
 (Correta) Percorrer a entrada na ordem original e acrescentar um item somente quando ainda não foi visto.
• Percorrer a entrada ao contrário e substituir cada item já emitido.

Por que a resposta está certa

A saída mantém a ordem se os itens forem emitidos no percurso original, no primeiro encontro.

Armadilha da alternativa incorreta

Ordenar ou percorrer ao contrário pode mudar qual ocorrência aparece primeiro; emita durante o percurso original.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[] → []; [NaN,NaN] -> [NaN] por SameValueZero; objetos distintos com mesma forma -> mantidos distintos

Concepção errada e falha observável

Comparar objetos por forma sem contrato ou prometer complexidade absoluta sem modelo. / Uma versão muda a ordem ou remove objetos distintos.

Pergunta diagnóstica

A igualdade desejada é identidade/SameValueZero ou estrutura profunda?

Pistas graduais

1. Versão lenta usa includes no prefixo. 2. Versão Set usa novo Set(xs). 3. Compare crescimento n=100,1000 sem depender de tempo exato.

Solução de referência explicada

Lento: percorra e inclua se não existe; rápido: [...new Set(xs)]. Explique O(n²) versus O(n) médio.

Transferência em outro contexto

Remova IDs repetidos de um log preservando a primeira aparição. Entregue a saída e demonstre que as duas versões do exercício produzem o mesmo resultado.

Evidência de domínio

Resultado observável: [a,b,a,c] -> [a,b,c] nas duas versões, preservando primeira ocorrência. Borda a conferir: [] -> []; [NaN,NaN] -> [NaN] por SameValueZero; objetos distintos com mesma forma -> mantidos distintos. Entrega esperada: Lento: percorra e inclua se não existe; rápido: [...new Set(xs)]. Explique O(n²) versus O(n) médio. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 08.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.07 Trace BFS por largura

TRACE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: algorithms.bfs_trace_bridge | Requer: control.loop, arrays.basic, mapset.basic | Introduz: algorithms.bfs_trace_bridge

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Antes de implementar a rota completa em 08.08, acompanhe uma busca em largura: registre fila, visitados e predecessor em cada passo e reconstrua o caminho de A até D.

Objetivo da atividade

Escreva a ordem de remoção da fila e o predecessor de cada vértice antes de reconstruir a rota.

Explicação e modelo mental

• Modelo mental: Busca em largura mantém uma fila FIFO, registra o predecessor ao descobrir cada vértice e só então enfileira seus vizinhos não visitados.
• Exemplo resolvido: De A, enfileire B e C. Retire B e descubra D; como C já está na fila, D fica depois dela. Ao retirar C, D já está visitado. O predecessor de D continua B e a reconstrução retorna A,B,D.
• Armadilha comum: Marcar só ao retirar permite enfileirar D duas vezes. Trocar FIFO por pilha muda a exploração para profundidade e não garante a mesma rota mínima.
• Por que importa: Rastrear fila e predecessores torna visível por que BFS encontra a rota com menos arestas antes de pedir ao aluno que implemente o planejador completo.

Material/arquivo inicial

main.js: export function tracarBfs(grafo, inicio, destino) {
const fila = [inicio];
const visitados = new Set([inicio]);
const anterior = new Map();
const ordem = [];
while (fila.length) {
const atual = fila.shift();
ordem.push(atual);
if (atual === destino) break;
for (const vizinho of grafo.get(atual) ?? []) {
if (visitados.has(vizinho)) continue;
visitados.add(vizinho);
anterior.set(vizinho, atual);
fila.push(vizinho);
}
}
const caminho = [];
for (let no = destino; no !== undefined; no = anterior.get(no)) caminho.unshift(no);
return {ordem, caminho: caminho[0] === inicio ? caminho : []};
}

const grafo = new Map([["A", ["B", "C"]], ["B", ["D"]], ["C", ["D"]], ["D", []]]);
console.log(JSON.stringify(tracarBfs(grafo, "A", "D")));

Exemplo e resultado público

início A, destino D → removidos da fila A,B,C,D; fila após cada remoção [],[B,C],[C,D],[D],[]; anterior B<-A, C<-A, D<-B; caminho mínimo [A,B,D]

Bordas e erros publicados

início igual ao destino -> caminho [A]; destino X ausente ou inalcançável -> null; grafo com ciclo A<->B -> cada vértice entra na fila no máximo uma vez

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map ; MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que a busca deve marcar um vértice ao enfileirá-lo?

• Para impedir que ciclos ou vizinhos convergentes sejam enfileirados várias vezes.
• Para ordenar os vértices alfabeticamente antes da busca.
• Para transformar um grafo sem pesos em grafo ponderado.
 (Correta) Para visitar o destino antes de processar qualquer vizinho.

Por que a resposta está certa

Correto. Registrar na descoberta garante que cada vértice entre uma vez na fila.

Armadilha da alternativa incorreta

A descoberta ocorre antes da remoção posterior. Marcar nesse momento evita duplicação em ciclos e caminhos convergentes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

início igual ao destino → caminho [A]; destino X ausente ou inalcançável -> null; grafo com ciclo A<->B -> cada vértice entra na fila no máximo uma vez

Concepção errada e falha observável

Tratar o grafo como lista sem ordem, visitar apenas ao retirar ou escolher o caminho mais longo por profundidade. / A tabela repete vértice em ciclo, perde predecessor ou devolve uma rota que não segue arestas.

Pergunta diagnóstica

Em que momento um vizinho fica marcado e como o predecessor permite reconstruir a rota?

Pistas graduais

1. A fila FIFO visita todos os vértices de uma camada antes da próxima. 2. Marque cada vizinho como visitado quando o enfileirar para evitar duplicatas. 3. Guarde o vértice anterior ao descobrir o destino e reconstrua voltando até A.

Solução de referência explicada

Trace a fila e o Map de predecessores em um grafo pequeno. Ao descobrir D por B, reconstrua A→B→D; a fila FIFO garante o primeiro caminho mínimo em grafo sem pesos.

Transferência em outro contexto

Trace a rota entre estações de metrô representadas por um Map de adjacência. Registre fila, predecessores e o caminho mínimo; inclua um ciclo e mostre que cada estação é enfileirada uma vez.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS (ES modules); Map, Set e arrays padrão, sem dependências externas. | Próxima sugestão: 08.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

08.08 Microprojeto - planejador de rotas simplificado

PROJECT · 75 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: algorithms.bfs_path | Requer: control.loop, arrays.basic, mapset.basic | Introduz: algorithms.bfs_path

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 4 · depuração 4 · arquivos 4 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Use o starter descrito no contrato (main.js: export function caminho(grafo,inicio,fim){ /* Map de adjacência, BFS */ }). Implemente o comportamento e entregue código executável, uma verificação de sucesso e de borda, e uma decisão de comportamento visível ao usuário.

Objetivo da atividade

Anote ordem dos vizinhos e momento de visitar antes de traçar.

Explicação e modelo mental

• Modelo mental: BFS usa fila FIFO; visitar ao enfileirar evita duplicatas e predecessor reconstrói rota.
• Exemplo resolvido: Num mapa de estações, parta de Central e enfileire vizinhas na ordem anunciada. Ao descobrir Mercado, registre de onde veio; ao achar destino, siga predecessoras e inverta.
• Armadilha comum: Visitar só ao retirar duplica nós; sem predecessora não há rota para devolver. Use fila FIFO.
• Por que importa: Rota mínima sem pesos depende da exploração por camadas, não de qualquer caminho.

Material/arquivo inicial

main.js: export function caminho(grafo,inicio,fim){ /* Map de adjacência, BFS */ }

Exemplo e resultado público

grafo A:[B,C],B:[D],C:[D],D:[]; buscar A→D e consultar vértice X ausente; buscar A→A; buscar D→A → caminho [A,B,D] pela ordem de vizinhos publicada; consulta de X retorna null; início igual ao fim retorna [A]; sem caminho retorna null

Bordas e erros publicados

grafo A:[B],B:[A]; buscar A→X -> null e exploração encerra sem repetir vértice

Teste público e aprovação visível

Resultado observável: grafo A:[B,C],B:[D],C:[D],D:[]; buscar A→D e consultar vértice X ausente; buscar A→A; buscar D→A -> caminho [A,B,D] pela ordem de vizinhos publicada; consulta de X retorna null; início igual ao fim retorna [A]; sem caminho retorna null. Borda a conferir: grafo A:[B],B:[A]; buscar A→X -> null e exploração encerra sem repetir vértice. Entrega esperada: BFS com fila, Set visitados e Map predecessor; retorne primeiro caminho mínimo pela ordem dos vizinhos, ou null.

Restrições e documentação

MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map ; MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em um grafo sem pesos, por que a busca em largura encontra uma rota com o menor número de arestas?

• Ela escolhe sempre o vizinho com o menor nome.
• Ela aprofunda um único ramo até o fim antes de considerar os demais.
 (Correta) Ela explora os vértices em ordem de distância por meio de uma fila FIFO.

Por que a resposta está certa

A fila mantém a exploração por camadas; por isso uma rota mais curta é considerada antes das mais longas.

Armadilha da alternativa incorreta

A ordem alfabética não mede distância, e aprofundar um ramo primeiro é característica de outra estratégia.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

grafo A:[B],B:[A]; buscar A→X → null e exploração encerra sem repetir vértice

Concepção errada e falha observável

Não marcar visitado e repetir vértice em ciclo ou perder predecessor. / Busca não termina em ciclo ou devolve caminho que não segue arestas.

Pergunta diagnóstica

Quando um vértice entra em visitados e onde guardar seu predecessor?

Pistas graduais

1. Use fila FIFO. 2. Marque visitado ao enfileirar, não só ao retirar. 3. Ao achar fim, volte por predecessor e inverta o caminho.

Solução de referência explicada

BFS com fila, Set visitados e Map predecessor; retorne primeiro caminho mínimo pela ordem dos vizinhos, ou null.

Transferência em outro contexto

Encontre rota mínima entre estações sem pesos. Entregue caminho/null e trace da fila/predecessores.

Evidência de domínio

Resultado observável: grafo A:[B,C],B:[D],C:[D],D:[]; buscar A→D e consultar vértice X ausente; buscar A→A; buscar D→A -> caminho [A,B,D] pela ordem de vizinhos publicada; consulta de X retorna null; início igual ao fim retorna [A]; sem caminho retorna null. Borda a conferir: grafo A:[B],B:[A]; buscar A→X -> null e exploração encerra sem repetir vértice. Entrega esperada: BFS com fila, Set visitados e Map predecessor; retorne primeiro caminho mínimo pela ordem dos vizinhos, ou null. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 09.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09 Escopo e closures

8 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 09.01 · 09.02 · 09.03 · 09.04 · 09.05 · 09.06 · 09.07 · 09.08

09.01 Mapa de escopo

TRACE · 17 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: scope.lexical_map | Requer: functions.return | Introduz: scope.lexical_map

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Leia o snippet no editor e desenhe uma caixa para cada bloco. Antes de executar, classifique onde cada nome nasce e marque em cada posição se a referência encontra um binding ao subir pela cadeia léxica. Depois rode o snippet e anote evidências. Entregue o mapa de escopos e explique por que chamar a função depois não muda onde foi declarada.

Objetivo da atividade

Desenhe caixas com os bindings declarados e classifique as referências do snippet antes de rodar.

Explicação e modelo mental

• Modelo mental: Escopo léxico depende do lugar em que uma função ou variável foi escrita. Para resolver um nome, procure no bloco atual e suba pelos blocos que o contêm; a ordem das chamadas não reescreve essa cadeia.
• Exemplo resolvido: Em const taxa=2; function total(){ const parcial=3; return taxa+parcial }, marque taxa no bloco externo e parcial no corpo de total. A avaliação encontra parcial localmente e sobe um nível para encontrar taxa. Uma função irmã não recebe acesso a parcial só por ser chamada depois.
• Armadilha comum: Não confunda estar disponível no momento da chamada com estar no escopo léxico. Desenhe os blocos de declaração e siga apenas ligações de contenção.
• Por que importa: A regra permite inspecionar callbacks e funções aninhadas. Um mapa prévio ajuda a localizar referências inválidas sem tentativa e erro.

Material/arquivo inicial

main.js: export function tracarEscopo() {
const fora = 1;
let bloco;
let dentro;
{
bloco = 2;
function f() {
dentro = 3;
return fora + bloco + dentro;
}
f();
}
return {fora, bloco, dentro, soma: fora + bloco + dentro};
}

console.log(JSON.stringify(tracarEscopo()));

Exemplo e resultado público

f() dentro do bloco → 6; fora fora do bloco -> 1; bloco fora -> ReferenceError

Bordas e erros publicados

dentro fora de f -> ReferenceError; função aninhada lê binding ancestral, não irmão

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

MDN Closures: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures ; MDN let: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual afirmação descreve o alcance de let e const em relação a var?

 (Correta) let e const têm escopo de bloco; var tem escopo de função.
• let, const e var têm todos escopo de bloco.
• var tem escopo de módulo, enquanto let e const são sempre globais.

Por que a resposta está certa

As declarações léxicas respeitam blocos; var é delimitado pela função mais próxima.

Armadilha da alternativa incorreta

A diferença aparece ao comparar blocos internos com o escopo de função; as três declarações não têm o mesmo alcance.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

dentro fora de f → ReferenceError; função aninhada lê binding ancestral, não irmão

Concepção errada e falha observável

Confundir escopo léxico com a ordem em que a função é chamada. / Marca bloco como disponível fora das chaves.

Pergunta diagnóstica

Em qual bloco cada binding foi declarado?

Pistas graduais

1. Desenhe caixas de escopo. 2. Siga de dentro para fora até encontrar cada nome. 3. Uma chamada posterior não move a declaração para outro escopo.

Solução de referência explicada

Mapa: global contém fora; bloco contém bloco e f; corpo de f contém dentro; referências sobem na cadeia léxica.

Transferência em outro contexto

Desenhe o mapa de escopos de um callback declarado dentro de uma função, ela própria dentro de um bloco. Marque onde três nomes nascem e justifique quais referências o callback pode resolver; entregue o diagrama e uma previsão verificável.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract's stated fallback for newer APIs. | Próxima sugestão: 09.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.02 Parsons de factory

PARSONS · 24 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: closures.private_state | Requer: functions.return | Introduz: closures.private_state

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 4 · depuração 1 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Ordene os blocos do starter para completar criarConta e implemente os métodos. Antes de rodar, anote em que chamada nasce o saldo e o que deve acontecer quando duas contas forem criadas. Rode o exemplo e acrescente uma verificação de isolamento entre instâncias.

Objetivo da atividade

Antes de montar os blocos, anote onde o estado deve ser declarado e descreva um teste que detecte compartilhamento entre contas.

Explicação e modelo mental

• Modelo mental: Uma factory executada cria um novo ambiente léxico. Métodos retornados conservam acesso ao estado local desse ambiente; outra execução cria outro ambiente.
• Exemplo resolvido: Em makeLabel(prefix), retorne name => prefix + name. Uma chamada produz função que lê seu próprio prefixo; outra chamada com valor distinto produz outro binding capturado.
• Armadilha comum: Não declare estado mutável no escopo global se cada instância precisa dos próprios dados. Retorne métodos que fecham sobre o binding local.
• Por que importa: Closures criam módulos pequenos com estado privado e API explícita. A regra mantém instâncias independentes sem armazenamento externo.

Material/arquivo inicial

main.js: export function criarConta(inicial) {
let saldo = inicial;
return {
depositar(valor) {
if (!(valor >= 0)) return false;
saldo += valor;
return true;
},
consultar() {
return saldo;
},
};
}

console.log(criarConta(2).depositar(3));

Exemplo e resultado público

conta=criarConta(5); depositar(2) → true; consultar -> 7; depositar(-1) -> false; consulta permanece 7

Bordas e erros publicados

duas contas criadas com 1 e 9 -> saldos independentes

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

MDN Closures: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que caracteriza uma closure em JavaScript?

• Uma cópia serializada de todos os objetos visíveis no momento da chamada.
 (Correta) Uma função junto ao acesso que mantém ao ambiente léxico relevante.
• Uma função que só pode ser executada uma vez depois de retornar.

Por que a resposta está certa

A closure continua podendo acessar bindings do ambiente em que foi criada.

Armadilha da alternativa incorreta

Closure não é uma serialização nem limita a função a uma chamada; ela preserva acesso lexical.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

duas contas criadas com 1 e 9 → saldos independentes

Concepção errada e falha observável

Expor saldo como propriedade pública ou compartilhar uma variável global. / Alterar uma conta muda a outra.

Pergunta diagnóstica

Onde saldo deve existir para cada chamada à factory?

Pistas graduais

1. A factory executa uma vez por conta. 2. Declare saldo dentro dela e fora dos métodos. 3. Retorne métodos que fecham sobre esse binding.

Solução de referência explicada

A factory mantém saldo local e retorna métodos. depositar soma e retorna true para valor positivo; para zero/negativo retorna false sem mudar o saldo.

Transferência em outro contexto

Implemente uma factory de carteira com saque e consulta. Escreva testes mostrando que duas carteiras mantêm saldos independentes; entregue factory e testes.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract's stated fallback for newer APIs. | Próxima sugestão: 09.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.03 Complete closure

FADED · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: closures.parameter_capture | Requer: functions.return | Introduz: closures.parameter_capture

Eixos: novidade 2 · sintaxe 3 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 2 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Complete multiplicador no editor. Antes de escrever a função interna, registre qual argumento chega na primeira chamada e qual chega na segunda. Rode os exemplos e explique como a função retornada ainda acessa o fator escolhido antes.

Objetivo da atividade

Anote em duas colunas os parâmetros da chamada externa e da chamada interna. Registre o que prevê que permanece acessível.

Explicação e modelo mental

• Modelo mental: Uma closure mantém acesso aos bindings do ambiente léxico em que foi criada. A função interna pode receber novo argumento e consultar um parâmetro capturado da chamada externa.
• Exemplo resolvido: Em makePrefix(prefix), retorne text => prefix + text. A primeira chamada escolhe o prefixo; depois a função retornada recebe o texto. Outra chamada externa conserva seu próprio prefixo.
• Armadilha comum: Não tente ler parâmetro externo de chamada futura nem substituí-lo por variável compartilhada. Verifique que a função interna usa o binding capturado.
• Por que importa: Separar configuração e execução permite criar funções reutilizáveis com comportamentos independentes.

Material/arquivo inicial

main.js: export function multiplicador(fator){ return /* função */ }

Exemplo e resultado público

multiplicador(3)(4) → 12; triplo=multiplicador(3); triplo(-2) -> -6

Bordas e erros publicados

multiplicador(0)(99) -> 0; dois fatores distintos -> funções independentes

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

MDN Closures: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que significa currying na forma f(a)(b)?

• Uma função aceita todos os argumentos apenas como um array.
• Uma função executa seus argumentos em paralelo e combina as promessas.
 (Correta) Uma chamada retorna outra função, que recebe o próximo argumento.

Por que a resposta está certa

Currying divide a aplicação em chamadas encadeadas, cada uma recebendo parte dos argumentos.

Armadilha da alternativa incorreta

A notação encadeada representa funções retornadas entre chamadas, não array de argumentos nem concorrência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

multiplicador(0)(99) → 0; dois fatores distintos -> funções independentes

Concepção errada e falha observável

Executar a multiplicação na criação e retornar número em vez de função. / multiplicador(3) não pode ser chamado com (4).

Pergunta diagnóstica

Qual valor fica guardado entre as duas chamadas?

Pistas graduais

1. A primeira chamada recebe fator. 2. Retorne uma função que recebe valor. 3. Essa função usa fator do escopo anterior.

Solução de referência explicada

return function(valor){return valor*fator}; fator é capturado lexicalmente.

Transferência em outro contexto

Escreva somador(base)(valor) e demonstre duas bases distintas com pelo menos três chamadas. Entregue implementação, testes e uma frase indicando qual binding cada função conserva.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 09.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.04 Closure passo a passo

TRACE · 19 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: closures.binding_not_snapshot | Requer: functions.return | Introduz: closures.binding_not_snapshot

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 4 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Trace o starter em uma linha do tempo. Anote o valor do binding antes e depois de cada método; então rode o programa. Explique se as funções guardam uma cópia antiga ou consultam o binding local compartilhado por aquela instância.

Objetivo da atividade

Preencha uma tabela temporal com o valor do binding antes e depois de cada chamada, sem executar o programa.

Explicação e modelo mental

• Modelo mental: Uma closure conserva acesso a um binding, não necessariamente uma fotografia do valor. Se um método o modifica, outro método da mesma instância lê o valor atualizado.
• Exemplo resolvido: makeClock() cria let minute=0 e retorna advance() e read(). Após advance, read consulta o mesmo binding; não conserva a leitura existente quando foi criada. Outra execução tem binding próprio.
• Armadilha comum: Não desenhe um binding para cada método retornado. Eles foram criados no mesmo ambiente; use uma linha do tempo.
• Por que importa: Entender bindings mutáveis evita previsões erradas em contadores, listeners e estado encapsulado.

Material/arquivo inicial

main.js: export function criarContadores() {
function make() {
let n = 0;
return {inc() {n++;}, read() {return n;}};
}
const a = make();
const b = make();
a.inc();
return {a: a.read(), b: b.read()};
}

console.log(JSON.stringify(criarContadores()));

Exemplo e resultado público

a.read(),a.inc(),a.read(),b.read() → 0,1,1,0

Bordas e erros publicados

const r=a.read; r() -> 1 porque método não usa this; chamada posterior vê n atual

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

MDN Closures: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Métodos de leitura e alteração fecham sobre a mesma variável local. Depois da alteração, o que a leitura consulta?

 (Correta) O valor atual do mesmo binding capturado no ambiente compartilhado.
• Uma cópia imutável criada quando os métodos foram definidos.
• A variável local mais recente de qualquer outra instância.
• O valor do parâmetro da última chamada externa ao módulo.

Por que a resposta está certa

As closures compartilham acesso ao binding local; uma alteração nele é visível às outras.

Armadilha da alternativa incorreta

Diferencie binding e valor lido em um instante. Métodos da mesma factory consultam o ambiente compartilhado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

const r=a.read; r() → 1 porque método não usa this; chamada posterior vê n atual

Concepção errada e falha observável

Pensar que closure guarda uma cópia congelada do valor inicial. / Prevê sempre zero após inc.

Pergunta diagnóstica

A closure conserva um valor ou acesso ao binding?

Pistas graduais

1. Marque o escopo de n. 2. Execute cada alteração antes da leitura seguinte. 3. Criações diferentes executam make separadamente.

Solução de referência explicada

Cada invocação de make cria binding n próprio; inc altera o mesmo binding que read consulta.

Transferência em outro contexto

Adicione reset a uma factory de contador e escreva uma sequência que mostre como leitores já retornados observam o estado após o reset. Entregue implementação, linha do tempo e teste.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 09.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.05 Bug no loop e callback

DEBUG · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: closures.loop_capture | Requer: functions.return | Introduz: closures.loop_capture

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 4 · arquivos 1 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Investigue o starter antes de corrigir: escreva o resultado previsto para os callbacks e indique quando cada um executa. Faça a menor alteração que preserva quantidade e execução tardia. Rode testes e justifique o binding consultado por cada callback.

Objetivo da atividade

Antes de editar, escreva a saída prevista e desenhe quantos bindings do índice existem para os callbacks guardados.

Explicação e modelo mental

• Modelo mental: Callback lê os bindings de seu escopo quando executado. Se funções do loop compartilham um binding mutável, podem observar o valor final; um binding por iteração separa valores.
• Exemplo resolvido: Imagine um loop percorrendo três nomes e criando funções ()=>nome. Se o binding for atualizado pelo loop, funções guardadas e chamadas depois podem ler a última atribuição.
• Armadilha comum: Não confunda criação do callback com leitura do valor. Inspecione quantos bindings existem e onde o callback os encontra.
• Por que importa: Handlers de interface e tarefas assíncronas executam depois da configuração. Um teste tardio detecta erros que uma chamada imediata esconderia.

Material/arquivo inicial

main.js: export function criarHandlers(limite) {
const handlers = [];
for (var i = 0; i < limite; i++) {
handlers.push(() => i);
}
return handlers;
}

console.log(JSON.stringify(criarHandlers(3).map(handler => handler())));

Exemplo e resultado público

handlers.map(f=>f()) antes → [3,3,3]; com let i no for -> [0,1,2]

Bordas e erros publicados

quatro iterações -> [0,1,2,3]; callback executado depois do loop mantém valor da iteração

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

MDN Closures: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures ; MDN let: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let ; MDN var: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/var

Conceitos prévios (Primers)

var é a forma antiga de declarar variáveis: vale para a função inteira, não para o bloco, e sua declaração é içada. Os callbacks deste loop executam depois e leem o mesmo binding i, que terminou em 3; declarar i com let no for cria um binding por iteração. Em código novo, prefira const e let.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que um callback pode consultar um binding lexical associado à sua criação mesmo quando executado depois?

 (Correta) A função mantém acesso ao ambiente léxico capturado.
• O mecanismo reexecuta automaticamente o bloco que declarou a função.
• O valor é sempre copiado para uma variável global quando o callback é criado.

Por que a resposta está certa

Uma closure mantém acesso aos bindings capturados durante a vida da função.

Armadilha da alternativa incorreta

Execução tardia não recria o bloco nem exige variável global; a função conserva acesso lexical.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

quatro iterações → [0,1,2,3]; callback executado depois do loop mantém valor da iteração

Concepção errada e falha observável

Achar que a função é executada no momento em que entra no array. / Todos os callbacks retornam limite do loop.

Pergunta diagnóstica

Quando callback lê i e quantos bindings i existem?

Pistas graduais

1. Separe criação de execução. 2. var cria um binding compartilhado nessa função. 3. let na declaração do for cria binding por iteração.

Solução de referência explicada

Troque var i por let i no for; cada iteração fornece um binding distinto à closure.

Transferência em outro contexto

Crie handlers para quatro linhas chamados após a renderização. Corrija a captura para que cada clique identifique sua linha e entregue teste que execute os handlers tardiamente.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 09.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.06 Cache simples

MAKE · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: closures.memoization | Requer: functions.return, closures.basic, mapset.basic | Introduz: closures.memoization

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 2 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente memoize para função pura de um argumento primitivo. Antes de codificar, defina como distinguir entrada ausente de resultado falsy armazenado. Use função instrumentada para testar hits e misses, sem ampliar o contrato para objetos.

Objetivo da atividade

Anote o predicado para identificar cache hit e descreva um teste com resultado falsy antes de editar.

Explicação e modelo mental

• Modelo mental: Memoização reutiliza o resultado de uma função pura para a mesma entrada. O wrapper precisa distinguir se já calculou a entrada e manter o resultado junto da closure que ele retorna.
• Exemplo resolvido: Uma função pura que formata o título "Guia 7" sempre produz a mesma etiqueta. Se o mesmo título for pedido de novo, reutilizar o trabalho anterior pode evitar uma segunda formatação; um contador de chamadas torna essa economia observável sem mudar o texto produzido.
• Armadilha comum: Não compartilhe o cache entre funções com comportamento ou argumentos diferentes, e não memoize uma função cujo resultado dependa de estado externo mutável.
• Por que importa: Um cache pequeno pode reduzir trabalho repetido, mas só é correto quando a saída depende dos argumentos declarados e a política de armazenamento é explícita.

Material/arquivo inicial

main.js: export function memoize(fn) { const cache = new Map(); return function (x) { return fn(x); }; }

Exemplo e resultado público

double instrumentada; f(2),f(2),f(3) → resultados 4,4,6 e fn chamada 2 vezes

Bordas e erros publicados

entrada 0 -> cache hit mesmo resultado zero; NaN como chave Map -> hit; objetos fora do contrato

Teste público e aprovação visível

Resultado observável: double instrumentada; f(2),f(2),f(3) -> resultados 4,4,6 e fn chamada 2 vezes. Borda a conferir: entrada 0 -> cache hit mesmo resultado zero; NaN como chave Map -> hit; objetos fora do contrato. Entrega esperada: return x=>{if(cache.has(x))return cache.get(x);const v=fn(x);cache.set(x,v);return v};

Restrições e documentação

MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em que tipo de função a memoização tende a preservar o significado das chamadas?

• Uma função que depende de relógio, rede ou estado global não incluído na chave.
 (Correta) Uma função determinística cujo resultado depende dos argumentos considerados na chave.
• Uma função que modifica a entrada e retorna objetos diferentes a cada chamada.

Por que a resposta está certa

O cache só é seguro quando a chave representa todas as entradas relevantes e o resultado é repetível.

Armadilha da alternativa incorreta

Se dados externos mudam o resultado sem aparecer na chave, uma resposta armazenada pode ficar obsoleta.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada 0 → cache hit mesmo resultado zero; NaN como chave Map -> hit; objetos fora do contrato

Concepção errada e falha observável

Testar cache com if(cache.get(x)), perdendo valores falsy. / Valor armazenado 0 é recalculado.

Pergunta diagnóstica

Como distinguir ausência de um valor falsy já armazenado?

Pistas graduais

1. Map pertence à closure retornada. 2. Use cache.has(x) antes de cache.get(x). 3. Só compute e guarde no miss.

Solução de referência explicada

return x=>{if(cache.has(x))return cache.get(x);const v=fn(x);cache.set(x,v);return v};

Transferência em outro contexto

Memoize um normalizador puro de código textual. Inclua contador de execuções e testes para entrada repetida e outra cujo resultado seja falsy; explique o limite das chaves assumido.

Evidência de domínio

Resultado observável: double instrumentada; f(2),f(2),f(3) -> resultados 4,4,6 e fn chamada 2 vezes. Borda a conferir: entrada 0 -> cache hit mesmo resultado zero; NaN como chave Map -> hit; objetos fora do contrato. Entrega esperada: return x=>{if(cache.has(x))return cache.get(x);const v=fn(x);cache.set(x,v);return v}; Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 09.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.07 WeakMap de metadados

COMPARE · TRILHA OPCIONAL · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: weakmap.metadata | Requer: functions.return, mapset.basic | Introduz: weakmap.metadata

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Compare Map e WeakMap com o starter. Preencha tabela com tipo de chave, possibilidade de enumeração e requisito do caso; escolha estrutura e registre o motivo. Não use coleta de lixo como teste determinístico.

Objetivo da atividade

Antes de alterar o código, anote se o caso precisa enumerar entradas e que tipo de chave usa.

Explicação e modelo mental

• Modelo mental: Map é enumerável e aceita chaves primitivas e objetos. WeakMap associa metadados a chaves-objeto sem enumeração e sem sinal determinístico de liberação pelo coletor.
• Exemplo resolvido: Um mapa de abreviações usa códigos string e precisa percorrer entradas: Map atende aos requisitos. Metadados de um objeto editor podem usar o objeto como chave quando não é preciso listar todas as associações.
• Armadilha comum: Não escolha WeakMap por parecer mais leve nem teste um instante de coleta. Comece pelos requisitos de chave e enumeração.
• Por que importa: A estrutura define operações disponíveis e ciclo de vida observável. Explicitar requisitos evita falhas quando é preciso listar dados ou usar chaves não-objeto.

Material/arquivo inicial

main.js: export function compararMetadados() {
const a = {};
const b = {};
const map = new Map([[a, "visível"]]);
const weak = new WeakMap([[b, "privado"]]);
return {
mapTemA: map.has(a),
weakTemB: weak.has(b),
mapIteravel: [...map].length === 1,
weakTemSize: "size" in weak,
weakTemIterador: typeof weak[Symbol.iterator] === "function",
};
}

console.log(JSON.stringify(compararMetadados()));

Exemplo e resultado público

WeakMap.set(a,{visitas:1}); get(a) → {visitas:1}; Map.keys() -> iterável

Bordas e erros publicados

WeakMap.set(a,{visitas:1}) e WeakMap.get(a) -> {visitas:1}; typeof WeakMap.keys -> undefined; WeakMap.set("id",1) -> TypeError; GC não pode ser afirmado por teste determinístico

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

MDN WeakMap: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap ; MDN Map: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual caso de uso combina com WeakMap?

• Manter um inventário cujas chaves precisam ser listadas e ordenadas.
• Armazenar valores escalares independentes de qualquer objeto como chaves.
 (Correta) Associar metadados a objetos sem acrescentar propriedades ao objeto.

Por que a resposta está certa

WeakMap permite associação externa por identidade de objeto sem alterar a forma pública do objeto.

Armadilha da alternativa incorreta

WeakMap é apropriado para metadados ligados a objetos, não para coleções que precisam ser percorridas ou ordenadas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

WeakMap.set(a,{visitas:1}) e WeakMap.get(a) → {visitas:1}; typeof WeakMap.keys -> undefined; WeakMap.set("id",1) -> TypeError; GC não pode ser afirmado por teste determinístico

Concepção errada e falha observável

Achar que WeakMap aceita string ou que permite enumerar suas chaves. / Solução tenta exibir todas as chaves do WeakMap.

Pergunta diagnóstica

A aplicação precisa enumerar ou manter acesso por objeto vivo?

Pistas graduais

1. Ambos associam valores a chaves objeto. 2. Só Map oferece enumeração. 3. WeakMap não é observador determinístico do coletor de lixo.

Solução de referência explicada

Use WeakMap para metadados por identidade de objeto; use Map quando enumeração ou chave primitiva for requisito.

Transferência em outro contexto

Escolha estruturas para metadados de nós DOM e contagem indexada por strings. Entregue requisitos e um exemplo curto de leitura/enumeração que justifique cada escolha; não dependa do coletor.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 09.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

09.08 Microprojeto - store mínimo

PROJECT · 55 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: closures.observable_store | Requer: functions.return | Introduz: closures.observable_store

Eixos: novidade 3 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 4 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Complete a store no starter com estado privado, assinaturas e cancelamento. Antes de implementar, desenhe quais dados pertencem a cada chamada da factory. Depois crie dois assinantes, altere o estado e cancele uma assinatura; registre evidências textuais do contrato.

Objetivo da atividade

Liste os bindings privados e esboce uma sequência com inscrição, atualização, cancelamento e nova atualização.

Explicação e modelo mental

• Modelo mental: Uma store criada por factory mantém estado e listeners no mesmo ambiente léxico. Assinar adiciona função e devolve remoção; atualizar define quando notificar e percorre snapshot dos listeners.
• Exemplo resolvido: Uma lista de tarefas mantém items e listeners dentro de createListStore. subscribe(fn) registra callback e devolve unsubscribe; uma segunda store tem bindings próprios e não recebe mudanças da primeira.
• Armadilha comum: Não guarde listeners em array global nem mantenha inscrições canceladas ativas. Se callbacks alterarem inscrições durante emissão, defina política e itere por cópia estável.
• Por que importa: A store é um motor reutilizável para estado observável: encapsula mutações, fornece leituras e controla listeners. Isolamento e cancelamento tornam o contrato revisável.

Material/arquivo inicial

main.js: export function createStore(initial){return {getState(){},setState(next){},subscribe(fn){}}}

Exemplo e resultado público

store(0); subscribe(fn); setState(1) → getState=1, fn recebe 1 uma vez; unsubscribe(); setState(2) -> nenhuma chamada adicional

Bordas e erros publicados

setState(0) após 0 -> política sem notificação por Object.is; listener que desinscreve durante emissão não quebra iteração; duas stores isoladas

Teste público e aprovação visível

Critérios desta atividade: Cada chamada de createStore mantém estado e listeners próprios; getState devolve o valor atual. setState atualiza o estado e notifica uma única vez quando Object.is detecta mudança; estado igual não notifica. subscribe devolve cancelamento idempotente; listener removido não recebe atualizações posteriores. Dois testes de stores simultâneas demonstram isolamento, ordem das notificações e cancelamento de um listener durante emissão.

Restrições e documentação

MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set ; MDN Object.is(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/is

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

No padrão observável, qual é o papel principal de subscribe?

 (Correta) Registrar uma função para receber notificações sobre mudanças futuras.
• Devolver uma cópia do estado e encerrar os listeners existentes.
• Aplicar diretamente uma mudança ao estado sem executar a operação da store.

Por que a resposta está certa

subscribe registra um observador; a operação que altera o estado continua sendo responsabilidade da store.

Armadilha da alternativa incorreta

Inscrição estabelece uma relação de observação, não substitui leitura ou atualização do estado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

setState(0) após 0 → política sem notificação por Object.is; listener que desinscreve durante emissão não quebra iteração; duas stores isoladas

Concepção errada e falha observável

Guardar listeners globalmente ou não retornar unsubscribe idempotente. / Um listener recebe mudança de outra store.

Pergunta diagnóstica

Qual estado e lista de inscritos pertencem a cada chamada?

Pistas graduais

1. Mantenha state e Set de listeners na factory. 2. Compare next com state via Object.is. 3. Copie listeners ao notificar e retorne função que remove o callback.

Solução de referência explicada

A factory mantém state e Set locais; setState substitui valor e notifica snapshot dos listeners apenas quando mudou; subscribe retorna remoção idempotente.

Transferência em outro contexto

Crie uma store de filtro com dois assinantes independentes. Demonstre em testes leitura do estado, sequência exata de notificações e remoção de apenas uma assinatura; descreva a política para estado proposto igual ao atual.

Evidência de domínio

Critérios desta atividade: Cada chamada de createStore mantém estado e listeners próprios; getState devolve o valor atual. setState atualiza o estado e notifica uma única vez quando Object.is detecta mudança; estado igual não notifica. subscribe devolve cancelamento idempotente; listener removido não recebe atualizações posteriores. Dois testes de stores simultâneas demonstram isolamento, ordem das notificações e cancelamento de um listener durante emissão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 10.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10 this, protótipos e classes

8 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 10.01 · 10.02 · 10.03 · 10.04 · 10.05 · 10.06 · 10.07 · 10.08

10.01 Preveja this - metodo comum vs arrow

PREDICT · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: objects.this_arrows | Requer: objects.identity, functions.return | Introduz: objects.this_arrows

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Use o objeto do starter para comparar uma chamada por propriedade, uma chamada com call e a função separada. Antes de executar, anote quem fornece this em cada forma; depois confronte a previsão e explique por que a arrow não troca seu this pelo de call.

Objetivo da atividade

Faça uma tabela com as formas de chamada do starter e escreva quem fornece this em cada uma, sem executar.

Explicação e modelo mental

• Modelo mental: Em método comum, this vem da forma da chamada: obj.m() fornece obj; uma chamada solta não leva automaticamente o objeto original. Arrow não cria seu próprio this, então conserva o contexto léxico onde foi escrita.
• Exemplo resolvido: Considere const leitor={prefixo:"A", juntar(texto){return this.prefixo+texto}}. leitor.juntar.call({prefixo:"B"},"!") fornece outro receptor à chamada do método comum. O resultado depende da chamada, não de onde o método foi armazenado. Uma arrow definida fora desse método não se tornaria ligada ao novo receptor só por usar call.
• Armadilha comum: Não trate toda propriedade de função como método permanentemente ligado ao objeto. Observe o call site e distinga método comum de arrow.
• Por que importa: Callbacks retirados de objetos são comuns em timers e listeners. Identificar o receptor evita erros de estado sem esconder a decisão em uma solução ad hoc.

Material/arquivo inicial

main.js: const obj = {
x: 2,
metodo() {return this.x;},
arrow: () => this?.x,
};
const solto = obj.metodo;

export function compararThis() {
let soltoResultado;
try { soltoResultado = solto(); }
catch (error) { soltoResultado = error.name; }
return {
metodo: obj.metodo(),
call: obj.metodo.call({x: 7}),
solto: soltoResultado,
arrowCall: String(obj.arrow.call({x: 7})),
};
}

console.log(JSON.stringify(compararThis()));

Exemplo e resultado público

obj.metodo() → 2; obj.metodo.call({x:7}) -> 7

Bordas e erros publicados

solto() em módulo strict -> falha ao ler x porque não recebeu obj como this; arrow.call({x:7}) -> não devolve 7

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

MDN this: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this ; MDN arrow functions: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

Conceitos prévios (Primers)

obj?.x lê x só se obj não for null nem undefined; nesse caso o resultado é undefined em vez de um TypeError. obj?.metodo() chama o método só se obj existir.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que determina o this de um método comum chamado como obj.metodo()?

• O objeto em que a função foi escrita fica ligado para sempre.
 (Correta) A base à esquerda do ponto na chamada fornece o receptor.
• O primeiro argumento declarado pelo método vira automaticamente o receptor.
• O último objeto criado no mesmo módulo fornece o receptor.

Por que a resposta está certa

Em chamada por propriedade, a base da expressão é o receptor de this.

Armadilha da alternativa incorreta

Separe a definição da função do call site. Método comum não fica permanentemente ligado ao objeto que o contém.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

solto() em módulo strict → falha ao ler x porque não recebeu obj como this; arrow.call({x:7}) -> não devolve 7

Concepção errada e falha observável

Arrow recebe this dinâmico por call. / Na entrada solto() em módulo strict, a saída observável não corresponde a falha ao ler x porque não recebeu obj como this; a divergência é compatível com esta hipótese: Arrow recebe this dinâmico por call.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Arrow recebe this dinâmico por call.

Pistas graduais

1. Métodos comuns recebem this do call site. 2. Atribuir a variável remove a base da chamada. 3. Arrow conserva this do local de criação.

Solução de referência explicada

Compare chamada por propriedade, chamada solta e call; explicite que o resultado exato do this lexical depende do módulo e não use uma constante global.

Transferência em outro contexto

Passe um método de objeto a um callback que será executado fora do call site original. Entregue uma correção que preserve o comportamento associado ao objeto, um teste que execute a referência separadamente e uma justificativa para a decisão tomada.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract's stated fallback for newer APIs. | Próxima sugestão: 10.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.02 Parsons de classe

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: classes.encapsulation | Requer: objects.identity, functions.return | Introduz: classes.encapsulation

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 2 · arquivos 1 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Ordene os blocos da classe e complete a implementação. Antes de executar, escreva onde o campo privado é inicializado e quais operações podem alterar saldo. Depois rode o cenário e acrescente um teste que proteja o invariante da classe.

Objetivo da atividade

Marque nos blocos onde o campo privado nasce, onde muda e qual interface externa apenas o consulta.

Explicação e modelo mental

• Modelo mental: Campos privados com # pertencem à instância da classe e só podem ser acessados no corpo autorizado. O constructor estabelece estado inicial; métodos públicos validam mudanças e um getter pode expor leitura sem liberar mutação direta.
• Exemplo resolvido: Uma classe Vagas pode inicializar #livres no constructor, expor um getter e oferecer reservar(). A operação verifica a condição antes de diminuir o valor; código externo consulta a quantidade, mas não escreve diretamente no campo privado.
• Armadilha comum: Não atribua o campo privado fora da classe nem permita que uma mutação viole o invariante. Faça a validação antes de alterar o estado.
• Por que importa: Encapsular estado torna explícito quais operações são permitidas e protege invariantes locais sem precisar de arquitetura externa.

Material/arquivo inicial

main.js: export class Cofre {
#saldo;

constructor(inicial) {
this.#saldo = inicial;
}

depositar(valor) {
if (!(valor > 0)) return false;
this.#saldo += valor;
return true;
}

get saldo() {
return this.#saldo;
}
}

console.log(new Cofre(2).saldo);

Exemplo e resultado público

new Cofre(2).depositar(3); saldo → 5

Bordas e erros publicados

saldo inicial negativo -> saldo inicial 0; depósito zero/negativo -> retorna false e não altera o saldo; #saldo só pode ser acessado no corpo da classe

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

MDN private elements: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_elements

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que testes de encapsulamento devem preferir a API pública em vez de inspecionar diretamente um campo privado?

• Porque um campo privado não pode participar de nenhum cálculo dentro da classe.
 (Correta) Porque o contrato útil é o comportamento observável; detalhes privados podem mudar sem alterar a API.
• Porque testes só podem criar instâncias e nunca chamar métodos públicos.

Por que a resposta está certa

Testar pela interface protege o comportamento prometido e permite mudar a implementação interna.

Armadilha da alternativa incorreta

Campos privados participam da implementação; o teste deve verificar efeitos públicos sem acoplar-se a eles.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

saldo inicial negativo → saldo inicial 0; depósito zero/negativo -> retorna false e não altera o saldo; #saldo só pode ser acessado no corpo da classe

Concepção errada e falha observável

Declarar campo privado fora da instância ou omitir validação. / Na entrada saldo inicial negativo, a saída observável não corresponde a saldo inicial 0; a divergência é compatível com esta hipótese: Declarar campo privado fora da instância ou omitir validação.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Declarar campo privado fora da instância ou omitir validação.

Pistas graduais

1. Class tem declaração, constructor e métodos. 2. #saldo é inicializado no constructor. 3. Getter só lê; depositar valida antes de alterar.

Solução de referência explicada

A classe mantém #saldo e expõe getter de leitura. O constructor normaliza inicial negativo para 0; depositar(valor) retorna false sem mutar quando valor<=0 e retorna true após depósito positivo.

Transferência em outro contexto

Acrescente uma operação de saque à classe. Entregue a implementação e testes para operações aceitas e rejeitadas, mostrando que chamadas externas não podem alterar diretamente o estado encapsulado.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 10.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.03 Complete protótipo

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: prototypes.method_sharing | Requer: objects.identity, functions.return | Introduz: prototypes.method_sharing

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Complete criarPonto ligando cada objeto ao protótipo compartilhado. Antes de executar, preveja se mover será propriedade própria ou herdada e se mover um ponto afeta o outro. Depois teste identidade do protótipo e dados independentes.

Objetivo da atividade

Antes de codificar, escreva onde ficará o método e onde ficarão as coordenadas de cada ponto.

Explicação e modelo mental

• Modelo mental: Um protótipo pode guardar métodos compartilhados enquanto cada instância guarda seus próprios dados. A busca de propriedades sobe pela cadeia de protótipos; métodos comuns podem usar this para alterar os dados da instância que recebeu a chamada.
• Exemplo resolvido: Crie um único protótipo comando com ativar(){this.ligado=true}. Object.create(comando) produz dois aparelhos ligados ao mesmo método, mas cada um pode ter seu próprio ligado. A identidade do protótipo mostra o compartilhamento; os campos próprios mostram o isolamento.
• Armadilha comum: Não copie o método para cada objeto e chame isso de compartilhamento, nem guarde dados mutáveis do ponto no próprio protótipo.
• Por que importa: Compartilhar comportamento sem compartilhar estado evita duplicação e mantém instâncias independentes; testes de identidade e mutação tornam a estrutura observável.

Material/arquivo inicial

main.js: // 10.03 · Pontos que compartilham um protótipo.
// criarPonto(x, y) devolve um objeto com dados próprios x e y, ligado ao protótipo compartilhado proto.
// mover mora SÓ no protótipo (herdado): move o próprio ponto somando dx e dy.
// Antes de executar, preveja: mover é propriedade própria ou herdada? Mover um ponto afeta o outro?

```
const proto = {
mover(dx, dy) {
// implemente: altera x e y deste ponto
},
};

export function criarPonto(x, y) {
// implemente: use Object.create(proto) e dados próprios
}
```

Exemplo e resultado público

a=criarPonto(1,2); b=criarPonto(3,4); Object.getPrototypeOf(a)===Object.getPrototypeOf(b) → true

Bordas e erros publicados

a.mover(2,-1) -> a=(3,1), b=(3,4); a.hasOwnProperty('mover') -> false

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

MDN Object.create(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que Object.getPrototypeOf(obj) devolve?

• Uma cópia de todas as propriedades herdadas pelo objeto.
• A lista de todas as instâncias criadas pela mesma função.
 (Correta) O protótipo direto ligado ao objeto consultado.

Por que a resposta está certa

O método expõe o elo direto da cadeia, não uma cópia nem um inventário de instâncias.

Armadilha da alternativa incorreta

A cadeia de protótipos é formada por elos; getPrototypeOf retorna o elo direto.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

a.mover(2,-1) → a=(3,1), b=(3,4); a.hasOwnProperty('mover') -> false

Concepção errada e falha observável

Copiar método para cada instância e chamar isso de compartilhamento. / Na entrada a.mover(2,-1), a saída observável não corresponde a a=(3,1), b=(3,4); a divergência é compatível com esta hipótese: Copiar método para cada instância e chamar isso de compartilhamento.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Copiar método para cada instância e chamar isso de compartilhamento.

Pistas graduais

1. Crie um único objeto protótipo. 2. Object.create(proto) liga a instância a ele. 3. Método usa this para dados próprios x/y.

Solução de referência explicada

Criar ponto com Object.create(proto), atribuir x/y próprios e manter mover apenas em proto.

Transferência em outro contexto

Crie círculos com uma operação de área reutilizável. Entregue a factory e testes que inspecionem onde o método fica disponível e verifiquem que os dados geométricos de uma instância não alteram outra.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 10.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.04 Metodo extraido

DEBUG · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: objects.method_binding | Requer: objects.identity, functions.return, this.binding | Introduz: objects.method_binding

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

O starter prende fixo a um objeto separado com valor 4. Reproduza a divergência e corrija o receptor. Depois compare fixo e wrapper antes e depois da reatribuição de obj: bind(obj) mantém o objeto original atualizado; wrapper consulta a variável obj na chamada.

Objetivo da atividade

Preveja os resultados de fixo e wrapper antes e depois da reatribuição de obj; explique qual conserva o objeto original e qual consulta a variável na chamada.

Explicação e modelo mental

• Modelo mental: this em método comum é determinado pela expressão de chamada. Guardar só a função não conserva automaticamente a base da propriedade; numa chamada posterior, examine qual expressão a invoca.
• Exemplo resolvido: No starter, fixo está ligado a um objeto separado com valor 4, então retorna 4; wrapper consulta o objeto original atualizado e retorna 5. Ao corrigir para bind(obj), ambos retornam 5 antes da reatribuição. Depois dela, fixo continua no objeto original e retorna 5, enquanto wrapper consulta o novo obj e retorna 9.
• Armadilha comum: Não assuma que uma referência de método transporta seu objeto original, nem confunda uma arrow lexical com método comum.
• Por que importa: Callbacks são chamados por timers e componentes que não conhecem o objeto de origem. Rastrear o receptor no call site ajuda a escolher uma adaptação consciente.

Material/arquivo inicial

main.js: // 10.04 · Método extraído: qual objeto é o this?
// fixo deveria acompanhar o objeto ORIGINAL (bind), wrapper consulta a variável obj na hora da chamada.
// Hoje fixo está preso a um objeto separado com valor 4. Reproduza a divergência e corrija o receptor.
// executar(inicial, atualizado, trocado) devolve { antes, depois }:
// antes = [fixo(), wrapper()] depois de obj.valor = atualizado
// depois = [fixo(), wrapper()] depois de obj ser SUBSTITUÍDO por outro objeto com valor trocado
// Esperado com (4, 5, 9): antes [5, 5], depois [5, 9].

```
export function executar(inicial = 4, atualizado = 5, trocado = 9) {
let obj = { valor: inicial, ler() { return this.valor; } };
const fixo = obj.ler.bind({ valor: inicial });
const wrapper = () => obj.ler();
obj.valor = atualizado;
const antes = [fixo(), wrapper()];
obj = { valor: trocado, ler() { return this.valor; } };
return { antes, depois: [fixo(), wrapper()] };
}
```

Exemplo e resultado público

após correção: antes da troca → fixo=5, wrapper=5; após obj ser substituído -> fixo=5 (objeto original, propriedade atualizada), wrapper=9 (objeto atual no instante da chamada)

Bordas e erros publicados

obj.valor=5; console.log(fixo(),wrapper()); obj={valor:9,ler(){return this.valor}}; console.log(fixo(),wrapper()) -> first calls [5,5]; calls after reassignment [5,9]

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

MDN this: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this ; MDN Function.bind(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual característica de uma arrow function afeta o uso de this?

 (Correta) Ela não cria um this próprio e usa o contexto léxico externo.
• Ela sempre associa this ao objeto que contém a propriedade da função.
• Ela converte qualquer chamada em uma chamada com bind explícito.

Por que a resposta está certa

Arrow functions capturam this do ambiente léxico em vez de receber um receptor próprio pela chamada.

Armadilha da alternativa incorreta

Arrow não fixa automaticamente o objeto que guarda a função; seu this vem do contexto léxico.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

obj.valor=5; console.log(fixo(),wrapper()); obj={valor:9,ler(){return this.valor}}; console.log(fixo(),wrapper()) → first calls [5,5]; calls after reassignment [5,9]

Concepção errada e falha observável

bind deve receber o objeto original; prendê-lo a um objeto copiado deixa o callback com estado desatualizado. / Com o starter atual, fixo retorna 4 antes da reatribuição porque foi ligado a um objeto separado; o contrato exige 5. Corrija o receptor para ler o objeto original atualizado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: bind deve receber o objeto original; prendê-lo a um objeto copiado deixa o callback com estado desatualizado.

Pistas graduais

1. Compare obj.ler() com ler(). 2. bind fixa receptor; wrapper escolhe receptor ao executar. 3. Teste depois de alterar valor do objeto.

Solução de referência explicada

Use const fixo=obj.ler.bind(obj) e const wrapper=()=>obj.ler(). O bind conserva o objeto original e lê suas propriedades atuais; o wrapper consulta a variável obj quando é executado, inclusive depois de reatribuição.

Transferência em outro contexto

Passe um handler de classe a um timer simulado. Entregue uma correção e teste que execute o handler depois de alterar o estado relevante; explique como sua abordagem determina o receptor no momento da chamada.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 10.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.05 Bug de herança

DEBUG · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: classes.inheritance_super | Requer: objects.identity, functions.return, classes.basic, this.binding | Introduz: classes.inheritance_super

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

No starter, reproduza o ReferenceError causado por this antes de super(x). Corrija essa ordem; então execute novamente e perceba que descricao() ainda omite o texto da base. Preserve Base.descricao() na sobrescrita e confirme que new Filha(2, "A").descricao() retorna x=2; A. nome é uma string recebida pelo constructor.

Objetivo da atividade

Numere a sequência necessária no constructor e marque quais partes do método derivado precisam preservar a base.

Explicação e modelo mental

• Modelo mental: Uma subclasse pode estender o estado e comportamento herdado, mas seu constructor precisa inicializar a classe base com super(...) antes de usar this. Uma sobrescrita pode delegar à implementação base quando precisa manter o contrato anterior.
• Exemplo resolvido: class Evento{constructor(codigo){this.codigo=codigo} rotulo(){return evento ${this.codigo}}} pode ser estendida por Aviso: primeiro chama super(codigo), depois guarda seu título; o método rotulo pode usar super.rotulo() antes de acrescentar o título. Assim a parte base não é descartada.
• Armadilha comum: Não acesse this antes de super no constructor derivado nem substitua a implementação base sem verificar quem depende dela.
• Por que importa: Herança serve quando a subclasse preserva o contrato comportamental da base e acrescenta detalhes. Seguir a ordem de inicialização evita erros de execução.

Material/arquivo inicial

main.js: // 10.05 · Bug de herança.
// 1. new Filha(2, "A") lança ReferenceError: this é usado antes de super(x). Corrija a ordem.
// 2. Depois, descricao() ainda omite o texto da base: componha com super.descricao().
// Esperado: new Filha(2, "A").descricao() -> "x=2; A". nome é uma string recebida pelo constructor.

```
export class Base {
constructor(x) {
this.x = x;
}
descricao() {
return x=${this.x};
}
}

export class Filha extends Base {
constructor(x, nome) {
this.nome = nome;
super(x);
}
descricao() {
return this.nome;
}
}
```

Exemplo e resultado público

new Filha(2,'A').descricao() → 'x=2; A'

Bordas e erros publicados

new Filha(2,"A") antes da correção -> ReferenceError por acessar this antes de super(x); após corrigir a ordem e compor super.descricao() -> x=2; A

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

MDN extends: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/extends ; MDN Error.cause: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/cause

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que extends estabelece entre uma classe derivada e sua classe base?

• Uma cópia independente de todos os métodos base dentro de cada instância.
 (Correta) Uma relação de protótipos que permite herdar comportamento e sobrescrever partes da interface.
• Uma conexão que faz todas as instâncias compartilharem os mesmos campos mutáveis.

Por que a resposta está certa

A herança conecta protótipos para reutilizar comportamento; não copia métodos nem exige estado de instância compartilhado.

Armadilha da alternativa incorreta

extends cria uma relação de herança/delegação, não uma cópia de métodos nem um único estado global.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

new Filha(2,"A") antes da correção → ReferenceError por acessar this antes de super(x); após corrigir a ordem e compor super.descricao() -> x=2; A

Concepção errada e falha observável

Usar this antes de super e substituir completamente contrato da classe base. / Na entrada new Filha(2,"A") antes da correção, a saída observável não corresponde a ReferenceError por acessar this antes de super(x); a divergência é compatível com esta hipótese: Usar this antes de super e substituir completamente contrato da classe base.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar this antes de super e substituir completamente contrato da classe base.

Pistas graduais

1. Chame super(x) antes de this. 2. Armazene nome após super. 3. Ao sobrescrever, preserve descrição base com super.descricao().

Solução de referência explicada

constructor(x,nome){super(x);this.nome=nome} e descricao(){return ${super.descricao()}; ${this.nome}} preservam inicialização e descrição da base.

Transferência em outro contexto

Crie um erro de domínio que herde de Error e receba uma mensagem e uma causa original. Entregue o constructor e testes que confirmem que consumidores podem capturá-lo como erro JavaScript e consultar a mensagem e Error.cause.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS com ESM, salvo se o cartão comparar outro contexto; Error.cause (ES2022, Node.js 16.9+) e built-ins apenas; manter o fallback declarado para APIs mais recentes. | Próxima sugestão: 10.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.06 Refatore para composição

MODIFY · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: design.composition | Requer: objects.identity, functions.return | Introduz: design.composition

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 5 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Investigue a hierarquia do starter e liste dados mutáveis separados de capacidades. Substitua a combinação de subclasses por composição para somente as capacidades configuradas. Antes de executar, preveja quais operações cada entidade expõe; depois teste duas entidades sem compartilhar estado.

Objetivo da atividade

Faça duas listas: estado de cada entidade e funções de capacidade. Marque quais combinações o starter realmente precisa oferecer.

Explicação e modelo mental

• Modelo mental: Composição monta uma entidade a partir de dados próprios e capacidades escolhidas. Funções de capacidade podem receber a entidade explicitamente; compartilhar a função não significa compartilhar os dados mutáveis de cada instância.
• Exemplo resolvido: Um instrumento pode receber {bateria, capacidades} e ganhar emitirSom e piscar apenas quando configurados. As funções de capacidade podem ser reutilizadas enquanto a bateria fica em cada instância. Um instrumento sem capacidade de piscar não precisa herdar uma classe combinatória para escondê-la.
• Armadilha comum: Não crie uma subclasse para cada combinação nem guarde estado da entidade numa variável compartilhada pelo comportamento. Monte somente as capacidades declaradas e teste a ausência também.
• Por que importa: Composição reduz combinações artificiais e deixa explícito o que cada objeto pode fazer. A separação facilita estender o motor sem criar hierarquias explosivas.

Material/arquivo inicial

main.js: export const mover = (estado, distancia) => { estado.x += distancia; return estado; };
export const atacar = (estado, alvo) => { alvo.vida -= estado.forca; return alvo; };
export const curar = (estado, quantidade) => { estado.vida += quantidade; return estado; };

export function criarPersonagem(dados, capacidades) {
return {estado: {...dados}, capacidades: {...capacidades}};
}

const personagem = criarPersonagem({nome: "Mira", vida: 5, x: 0}, {mover, curar});
console.log(JSON.stringify(Object.keys(personagem.capacidades)));

Exemplo e resultado público

entidade com mover+curar → move e cura; entidade sem atacar -> não expõe atacar

Bordas e erros publicados

duas entidades compartilham função de capacidade -> funções compartilhadas, vida isolada por entidade

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

MDN Classes: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes ; MDN Functions: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual vantagem da composição sobre uma hierarquia com uma subclasse para cada combinação?

• Garantir que todos os objetos exponham todos os métodos disponíveis.
• Fazer com que instâncias diferentes usem obrigatoriamente o mesmo estado.
 (Correta) Combinar capacidades necessárias sem codificar cada combinação possível como tipo novo.

Por que a resposta está certa

Composição reúne comportamentos escolhidos e evita multiplicar subclasses para combinações.

Armadilha da alternativa incorreta

Composição busca combinações menores e explícitas; não obriga a expor toda capacidade nem compartilhar estado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

duas entidades compartilham função de capacidade → funções compartilhadas, vida isolada por entidade

Concepção errada e falha observável

Criar subclasses para cada combinação e compartilhar estado mutável entre entidades. / Na entrada duas entidades compartilham função de capacidade, a saída observável não corresponde a funções compartilhadas, vida isolada por entidade; a divergência é compatível com esta hipótese: Criar subclasses para cada combinação e compartilhar estado mutável entre entidades.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Criar subclasses para cada combinação e compartilhar estado mutável entre entidades.

Pistas graduais

1. Liste dados da entidade separadamente das capacidades. 2. Funções puras ou closures recebem entidade. 3. Monte só capacidades declaradas e teste combinações.

Solução de referência explicada

Crie makeEntity({vida,posicao,capacidades}) que compõe funções e mantém estado por entidade; não invente herança combinatória.

Transferência em outro contexto

Modele veículos com diferentes combinações de navegação e transporte de carga. Entregue factory e testes que identifiquem operações disponíveis e comprovem que o estado de dois veículos não se mistura.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 10.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.07 Transferencia - composicao vs heranca

TRANSFER · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: design.inheritance_tradeoff | Requer: objects.identity, functions.return | Introduz: design.inheritance_tradeoff

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 5 · autonomia 5 · ferramentas 2

VISÃO DO ALUNO

Analise os dois cenários do starter separadamente. Para cada um, identifique se a relação proposta preserva comportamento ou se uma política varia independentemente. Depois implemente a decisão do cenário de estratégia e escreva testes que expressem a troca sem depender de subclasses.

Objetivo da atividade

Escreva para cada cenário um requisito de substituição e um eixo de variação, antes de escolher a estrutura.

Explicação e modelo mental

• Modelo mental: Herança expressa substituição comportamental: uma instância derivada deve cumprir os contratos que os consumidores esperam da base. Composição é útil quando partes ou políticas variam independentemente e podem ser trocadas sem redefinir uma hierarquia.
• Exemplo resolvido: Em um relatório que pode ser exportado como CSV ou JSON, a política de formato varia independentemente do conteúdo do relatório. Um exportador injetado permite mudar a política sem criar uma classe filha para cada relatório e formato. Antes de herdar, pergunte se a substituição preserva invariantes e expectativas dos clientes.
• Armadilha comum: Não escolha herança apenas por campos ou nomes parecidos. Verifique contratos mutáveis e se o subtipo pode realmente ocupar o lugar da base em todos os usos.
• Por que importa: Avaliar substituição e eixos de variação ajuda a escolher entre extensão e composição. Testes tornam visíveis os trade-offs do desenho.

Material/arquivo inicial

main.js:
```
export class Retangulo {
constructor(largura, altura) { this.largura = largura; this.altura = altura; }
get area() { return this.largura * this.altura; }
}

export class Quadrado extends Retangulo {
set largura(valor) { this._largura = valor; this._altura = valor; }
get largura() { return this._largura; }
set altura(valor) { this._largura = valor; this._altura = valor; }
get altura() { return this._altura; }
}

export function criarNotificador(canal) {
return {enviar(mensagem) { return mensagem; }};
}

console.log(new Retangulo(3, 4).area);
```

Exemplo e resultado público

A → composição/contratos separados; B -> estratégia de canal; justificativa cita invariantes

Bordas e erros publicados

trocar canal em runtime -> sem nova subclasse; setter de largura do quadrado -> não satisfaz comportamento de retângulo geral

Teste público e aprovação visível

Compare competência estrutural em domínio diferente e sem ajuda.

Restrições e documentação

MDN Using classes: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_classes ; MDN extends: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/extends

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que custo de manutenção pode surgir quando subclasses representam cada combinação de políticas independentes?

 (Correta) A quantidade de tipos pode crescer conforme se multiplicam as combinações.
• As classes deixam de poder declarar métodos próprios.
• O runtime passa a escolher automaticamente a política mais adequada.

Por que a resposta está certa

Eixos independentes combinados em subclasses podem multiplicar tipos e duplicar decisões.

Armadilha da alternativa incorreta

O risco é o crescimento combinatório do modelo; subclasses continuam podendo ter métodos e não são escolhidas automaticamente.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

trocar canal em runtime → sem nova subclasse; setter de largura do quadrado -> não satisfaz comportamento de retângulo geral

Concepção errada e falha observável

Escolher herança apenas porque duas classes têm campos parecidos. / Na entrada trocar canal em runtime, a saída observável não corresponde a sem nova subclasse; a divergência é compatível com esta hipótese: Escolher herança apenas porque duas classes têm campos parecidos.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Escolher herança apenas porque duas classes têm campos parecidos.

Pistas graduais

1. Cheque substituição comportamental, não nomes. 2. Identifique capacidades que variam independentemente. 3. Implemente um cenário com teste de troca/contrato.

Solução de referência explicada

Implemente notificador com canal injetado e explique que retângulo mutável não é supertipo seguro de quadrado com setters independentes.

Transferência em outro contexto

Modele pagamentos cujo cálculo de taxa pode mudar sem alterar o fluxo principal. Entregue a interface de variação, testes com duas políticas e justificativa de como cada uma respeita o mesmo contrato de pagamento.

Evidência de domínio

Compare competência estrutural em domínio diferente e sem ajuda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 10.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

10.08 Microprojeto - motor de entidades

PROJECT · 65 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.5/5

Competência: design.entity_engine | Requer: objects.identity, functions.return | Introduz: design.entity_engine

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 4 · leitura 5 · autonomia 5 · ferramentas 3

VISÃO DO ALUNO

Implemente o motor de entidade do starter mantendo estado por entidade e validações nas ações. Antes de codificar, registre os invariantes de vida, posição e capacidades. Rode os testes existentes e acrescente um segundo caso que detecte estado compartilhado ou ação indisponível; então acrescente cura sem alterar o mover.

Objetivo da atividade

Liste os invariantes e as ações disponíveis antes de implementar; aponte uma hipótese de teste para detectar compartilhamento entre instâncias.

Explicação e modelo mental

• Modelo mental: Um motor reutilizável separa estado da entidade, operações e capacidades disponíveis. Cada instância mantém seu estado; funções do motor recebem a entidade, validam antes de mutar e produzem observação do novo estado.
• Exemplo resolvido: Uma coleção de sensores pode ter estado próprio de ligado e uma capacidade compartilhada ativar(sensor). A função valida o sensor e altera apenas a instância passada. Adicionar uma capacidade independente não exige reescrever a operação já estável.
• Armadilha comum: Não deixe entidades apontarem para o mesmo objeto de estado nem exponha ação que não foi configurada. Valide limites antes de mutar e teste instâncias lado a lado.
• Por que importa: Um motor pequeno concentra regras reaproveitáveis e permite extensão localizada. Evidências para duas instâncias detectam acoplamentos que um caso isolado não revela.

Material/arquivo inicial

main.js: // 10.08 · Motor de entidades.
// criarEntidade({ id, vida, posicao: { x, y }, capacidades: [string] }) -> entidade com ESTADO PRÓPRIO:
// copie posicao e capacidades; duas entidades criadas a partir do mesmo objeto não compartilham estado.
// mover(ent, dx, dy):
// sem a capacidade "mover" em ent.capacidades -> { status: "unsupported" } e nada muda
// com a capacidade -> soma dx e dy à posição e devolve { status: "ok" }
// receberDano(ent, n):
// n precisa ser Number finito >= 0; senão devolve false SEM alterar a entidade
// caso válido: vida = max(0, vida - n) e devolve true
// Registre os invariantes de vida, posição e capacidades. Depois acrescente cura sem alterar mover.

```
export function criarEntidade({ id, vida, posicao, capacidades }) {
// implemente
}

export function mover(ent, dx, dy) {
// implemente
}

export function receberDano(ent, n) {
// implemente
}
```

Exemplo e resultado público

cenários: entidade A vida=10 move (1,2) e recebe dano 3; entidade B vida=10 recebe dano -1; entidade C tenta executar ação sem capacidade declarada; entidade D vida=4 recebe dano 9 → resultados na ordem: A pos(1,2), vida 7; B retorno false sem mutação; C {status:unsupported}; D vida 0

Bordas e erros publicados

criar entidades A e B com posições próprias; mover A em (1,0) -> a posição de A muda; B conserva seu snapshot inicial

Teste público e aprovação visível

Resultado observável: cenários: entidade A vida=10 move (1,2) e recebe dano 3; entidade B vida=10 recebe dano -1; entidade C tenta executar ação sem capacidade declarada; entidade D vida=4 recebe dano 9 -> resultados na ordem: A pos(1,2), vida 7; B retorno false sem mutação; C {status:unsupported}; D vida 0. Borda a conferir: criar entidades A e B com posições próprias; mover A em (1,0) -> a posição de A muda; B conserva seu snapshot inicial. Entrega esperada: O motor cria entidades isoladas, aplica ações compostas com validação e retorna snapshot. Dano maior que a vida leva a vida a zero; dano negativo retorna false sem mutação. Uma ação sem capacidade retorna status unsupported. Testes cobrem duas entidades independentes.

Restrições e documentação

MDN Classes: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que um invariante de um motor de entidades descreve?

• Uma sequência exata de chamadas que toda entidade precisa executar.
 (Correta) Uma condição que o estado deve respeitar antes e depois das operações permitidas.
• Uma decisão interna de implementação que não pode ser expressa por comportamento observável.

Por que a resposta está certa

Invariantes definem condições de validade que as operações precisam preservar.

Armadilha da alternativa incorreta

Um invariante é uma regra sobre validade do estado, não uma ordem fixa de chamadas nem um detalhe interno.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

criar entidades A e B com posições próprias; mover A em (1,0) → a posição de A muda; B conserva seu snapshot inicial

Concepção errada e falha observável

Estado compartilhado ou ações disponíveis sem capacidade. / Na borda criar entidades A e B com posições próprias, observe a saída e compare-a com o contrato publicado; esta interpretação incorreta causa a falha: Estado compartilhado ou ações disponíveis sem capacidade.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Estado compartilhado ou ações disponíveis sem capacidade.

Pistas graduais

1. Defina invariantes: vida não negativa e posição finita. 2. Guarde estado por entidade e capacidades explícitas. 3. Valide ações antes de mutar e exponha snapshot.

Solução de referência explicada

O motor cria entidades isoladas, aplica ações compostas com validação e retorna snapshot. Dano maior que a vida leva a vida a zero; dano negativo retorna false sem mutação. Uma ação sem capacidade retorna status unsupported. Testes cobrem duas entidades independentes.

Transferência em outro contexto

Modele um motor pequeno para chamados de suporte com estado próprio e transições de status. Acrescente uma capacidade opcional de escalonamento sem alterar a operação que atualiza o status. Entregue a factory e testes que mostrem o escalonamento disponível apenas onde configurado e o estado independente entre chamados.

Evidência de domínio

Resultado observável: cenários: entidade A vida=10 move (1,2) e recebe dano 3; entidade B vida=10 recebe dano -1; entidade C tenta executar ação sem capacidade declarada; entidade D vida=4 recebe dano 9 -> resultados na ordem: A pos(1,2), vida 7; B retorno false sem mutação; C {status:unsupported}; D vida 0. Borda a conferir: criar entidades A e B com posições próprias; mover A em (1,0) -> a posição de A muda; B conserva seu snapshot inicial. Entrega esperada: O motor cria entidades isoladas, aplica ações compostas com validação e retorna snapshot. Dano maior que a vida leva a vida a zero; dano negativo retorna false sem mutação. Uma ação sem capacidade retorna status unsupported. Testes cobrem duas entidades independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS with ESM unless the card explicitly compares another context; use built-ins only and retain the contract’s stated fallback for newer APIs. | Próxima sugestão: 11.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11 DOM, eventos, formulário e Kanban

14 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Carregamento e módulos de navegador vêm antes do Kanban; migração e testes voltam na etapa 13.

Ordem: 11.09 · 11.01 · 11.11 · 11.14 · 11.02 · 11.03 · 11.04 · 11.06 · 11.05 · 11.07 · 11.12 · 11.08 · 11.10 · 11.13

11.09 Quando o script executa

PREDICT · 6 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.0/5

Competência: dom.script_loading | Requer: objects.identity, html.tree_reading | Introduz: dom.script_loading

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 2 · leitura 4 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Compare quando cada script pode executar em relação ao parse do documento. Separe garantias de ordem dos eventos que dependem do download.

Objetivo da atividade

Para cada script, escreva o que seu modo garante e o que depende da conclusão do download antes de ver a linha do tempo.

Explicação e modelo mental

• Modelo mental: Separe download, parse e execução. defer e módulo aguardam parse. async executa quando disponível e a ordem varia.
• Exemplo resolvido: Um script de métricas lento com async pode terminar depois de um script menor abaixo dele; a posição no HTML não garante ordem de execução.
• Armadilha comum: Considerar async ordenado como defer.
• Por que importa: Escolha o modo de carregamento para inicializar um formulário cujos elementos precisam ter sido interpretados.

Material/arquivo inicial

index.html: <!doctype html><html><head><script>window.ordem=["clássico"]</script><script defer src="defer.js"></script><script async src="async.js"></script><script type="module" src="module.js"></script></head><body><button id="salvar">Salvar</button></body></html>
| defer.js: window.ordem.push("defer");
| async.js: window.ordem.push("async");
| module.js: window.ordem.push("module");

Exemplo e resultado público

clássico no head sem defer → pode rodar antes do body; defer/module -> após parse e antes de DOMContentLoaded

Bordas e erros publicados

async -> ordem relativa ao parse/DOMContentLoaded não garantida; não afirmar que sempre vem antes/depois

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/HTML/Element/script

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual comportamento caracteriza um script clássico marcado com defer?

• Ele executa assim que o download termina e pode inverter a ordem de outros scripts defer.
• Ele só pode executar depois do evento load e sempre bloqueia o parse enquanto baixa.
 (Correta) Ele baixa sem bloquear o parse e executa após o parse, mantendo a ordem entre scripts defer do documento.

Por que a resposta está certa

Correto. Um script de métricas lento com async pode terminar depois de um script menor abaixo dele; a posição no HTML não garante ordem de execução.

Armadilha da alternativa incorreta

Revise esta regra: Considerar async ordenado como defer. Um script de métricas lento com async pode terminar depois de um script menor abaixo dele; a posição no HTML não garante ordem de execução.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

async → ordem relativa ao parse/DOMContentLoaded não garantida; não afirmar que sempre vem antes/depois

Concepção errada e falha observável

Considerar async ordenado como defer. / A previsão afirma ordem fixa para async ou executa script antes do parse sem garantia.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Considerar async ordenado como defer.

Pistas graduais

1. Separe download, parse e execução. 2. defer e módulo aguardam parse. 3. async executa quando disponível e a ordem varia.

Solução de referência explicada

Tabela explicita ordem garantida e não garantida; DOMContentLoaded espera deferred scripts e módulos relevantes do documento.

Transferência em outro contexto

Escolha o modo de carregamento para inicializar um formulário cujos elementos precisam ter sido interpretados.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.01 Rastreie eventos

TRACE · 9 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.0/5

Competência: dom.event_bubbling | Requer: objects.identity, html.tree_reading | Introduz: dom.event_bubbling

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Trace um clique do alvo até os ancestrais na fase bubble. Compare então stopPropagation e stopImmediatePropagation pelo efeito em listeners do mesmo nó.

Objetivo da atividade

Antes de executar, escreva o alvo e a cadeia de ancestrais na ordem visitada pelo evento.

Explicação e modelo mental

• Modelo mental: Identifique target e ancestralidade. Na fase bubble, suba um ancestral por vez. Distinga impedir próxima fase de impedir listener irmão.
• Exemplo resolvido: Em article > a > span, clicar em span começa nele; o listener bubble de a roda antes do listener de article porque o caminho sobe a partir do alvo.
• Armadilha comum: Supor que eventos sempre começam no elemento mais externo.
• Por que importa: Analise um menu aninhado com ação no item e no contêiner; escolha onde interromper a propagação sem bloquear controles irmãos.

Material/arquivo inicial

index.html: <!doctype html><article id="pai">Abrir</article>
| main.js: export function observarClique(root) {
const log = [];
for (const id of ['pai', 'filho', 'alvo']) root.querySelector('#'+id).addEventListener('click', event =>
log.push({target:event.target.id,currentTarget:event.currentTarget.id}));
return log;
}
export function observarControles(root, modo) {
const alvo=root.querySelector('#alvo'),log=[];
root.addEventListener('click',()=>log.push('capture'),{capture:true});
alvo.addEventListener('click',()=>log.push('target-1'));
alvo.addEventListener('click',event=>{log.push('target-controle');if(modo==='stop')event.stopPropagation();if(modo==='immediate')event.stopImmediatePropagation();});
alvo.addEventListener('click',()=>log.push('target-3'));
root.addEventListener('click',()=>log.push('bubble'));
return log;
}

Exemplo e resultado público

clique botão → logs botão,filho,pai

Bordas e erros publicados

stopPropagation em filho -> botão,filho; stopImmediatePropagation em primeiro listener do filho -> segundo listener do filho não roda

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em um listener de evento, o que distinguem target e currentTarget?

 (Correta) target identifica a origem do evento; currentTarget identifica o nó cujo listener está sendo executado.
• target sempre identifica document; currentTarget sempre identifica o elemento clicado.
• target e currentTarget são aliases e nunca diferem durante a propagação.

Por que a resposta está certa

Correto. Em article > a > span, clicar em span começa nele; o listener bubble de a roda antes do listener de article porque o caminho sobe a partir do alvo.

Armadilha da alternativa incorreta

Revise esta regra: Supor que eventos sempre começam no elemento mais externo. Em article > a > span, clicar em span começa nele; o listener bubble de a roda antes do listener de article porque o caminho sobe a partir do alvo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

stopPropagation em filho → botão,filho; stopImmediatePropagation em primeiro listener do filho -> segundo listener do filho não roda

Concepção errada e falha observável

Supor que eventos sempre começam no elemento mais externo. / O log começa no ancestral, altera target durante bubble ou continua subindo depois de stopPropagation.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Supor que eventos sempre começam no elemento mais externo.

Pistas graduais

1. Identifique target e ancestralidade. 2. Na fase bubble, suba um ancestral por vez. 3. Distinga impedir próxima fase de impedir listener irmão.

Solução de referência explicada

Trace target e caminho de propagação; exclua listeners após parada conforme método usado.

Transferência em outro contexto

Analise um menu aninhado com ação no item e no contêiner; escolha onde interromper a propagação sem bloquear controles irmãos.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.11 Propagação não é comportamento padrão

COMPARE · 12 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.0/5

Competência: dom.event_phases_default | Requer: objects.identity, html.tree_reading | Introduz: dom.event_phases_default

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 2 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Rastreie capture, target e bubble separadamente. Avalie preventDefault e stopPropagation como controles independentes.

Objetivo da atividade

Faça duas previsões independentes: ordem dos listeners e ocorrência da ação padrão.

Explicação e modelo mental

• Modelo mental: Capture desce, bubble sobe. preventDefault atua na ação padrão. stopPropagation atua no caminho do evento.
• Exemplo resolvido: Num link dentro de item de lista, impedir navegação não impede o evento de alcançar ancestrais; parar propagação não cancela sozinho a navegação.
• Armadilha comum: Confundir ação padrão com propagação.
• Por que importa: Escolha a política de evento para uma checkbox dentro de uma linha selecionável, preservando o comportamento de cada controle.

Material/arquivo inicial

index.html: <!doctype html> <button id="item" data-parar>Escolher</button>
| main.js: export function instalar(menu,item,{modo='none',cancelar=false}={}) {
const log=[],fora=menu.parentElement;
fora.addEventListener('click',()=>log.push('fora-capture'),{capture:true});
menu.addEventListener('click',()=>log.push('menu-capture'),{capture:true});
item.addEventListener('click',()=>log.push('item-1'));
item.addEventListener('click',event=>{log.push('item-controle');if(cancelar)event.preventDefault();if(modo==='stop')event.stopPropagation();if(modo==='immediate')event.stopImmediatePropagation();});
item.addEventListener('click',()=>log.push('item-3'));
menu.addEventListener('click',()=>log.push('menu-bubble'));
fora.addEventListener('click',()=>log.push('fora-bubble'));
return {log};
}

Exemplo e resultado público

capture ul, target link, bubble li, bubble ul → ordem de fases; preventDefault no link -> navegação impedida, propagação continua

Bordas e erros publicados

stopPropagation no link -> navegação padrão ainda pode ocorrer; passive listener -> preventDefault não cancela

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Events

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em qual direção a fase capture percorre a árvore antes de alcançar o alvo?

• Do alvo para os ancestrais depois que todos os listeners target terminaram.
 (Correta) Dos ancestrais mais externos em direção ao alvo.
• Somente entre elementos irmãos no mesmo nível.

Por que a resposta está certa

Correto. Num link dentro de item de lista, impedir navegação não impede o evento de alcançar ancestrais; parar propagação não cancela sozinho a navegação.

Armadilha da alternativa incorreta

Revise esta regra: Confundir ação padrão com propagação. Num link dentro de item de lista, impedir navegação não impede o evento de alcançar ancestrais; parar propagação não cancela sozinho a navegação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

stopPropagation no link → navegação padrão ainda pode ocorrer; passive listener -> preventDefault não cancela

Concepção errada e falha observável

Confundir ação padrão com propagação. / Parar propagação cancela a ação padrão indevidamente ou bloqueia listener que deveria continuar.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir ação padrão com propagação.

Pistas graduais

1. Capture desce, bubble sobe. 2. preventDefault atua na ação padrão. 3. stopPropagation atua no caminho do evento.

Solução de referência explicada

Trace fase/caminho e trate dois eixos separados: cancelamento padrão por cancelable e interrupção de propagação.

Transferência em outro contexto

Escolha a política de evento para uma checkbox dentro de uma linha selecionável, preservando o comportamento de cada controle.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.14

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.14 Listeners com ciclo de vida

COMPARE · 6 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.0/5

Competência: dom.listener_options | Requer: objects.identity, html.tree_reading | Introduz: dom.listener_options

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Em cada situação, decida como termina o ciclo de vida do listener e se ele pode cancelar a ação padrão. Justifique as três opções separadamente.

Objetivo da atividade

Antes de selecionar, anote para cada listener o que encerra seu ciclo e se ele pode cancelar a ação padrão.

Explicação e modelo mental

• Modelo mental: once remove após primeira invocação. passive promete que não cancelará a ação padrão. signal liga listener ao ciclo do AbortController.
• Exemplo resolvido: Um botão “Copiar” pode responder uma vez; um listener de rolagem que não cancela a ação padrão pode ser passivo; fechar um painel pode abortar os listeners ligados ao controller dele.
• Armadilha comum: Confundir once com cleanup geral ou usar preventDefault em listener passive.
• Por que importa: Projete instalação e limpeza de listeners para um painel de filtros que abre e fecha repetidamente, sem acumular callbacks.

Material/arquivo inicial

index.html: <!doctype html><button id="copiar">Copiar</button> <section id="painel"></section>
| main.js: export function instalar({botao,rolagem,painel,copiarTexto,observarRolagem,atualizarPainel,controller=new AbortController()}) {
botao.addEventListener('click',copiarTexto,{once:true}); rolagem.addEventListener('wheel',observarRolagem,{passive:true});
painel.addEventListener('click',atualizarPainel,{signal:controller.signal}); return controller;
}

Exemplo e resultado público

dois cliques em once → callback uma vez; controller.abort() -> listener do painel não executa mais

Bordas e erros publicados

passive listener chama preventDefault -> ação não é cancelada e aviso no console; signal já abortado -> listener não é instalado

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o efeito de registrar um listener com a opção once: true?

• O listener passa a executar uma vez por cada elemento descendente.
• A opção transforma o callback em execução assíncrona.
 (Correta) O navegador remove esse listener depois da primeira invocação.

Por que a resposta está certa

Correto. Um botão “Copiar” pode responder uma vez; um listener de rolagem que não cancela a ação padrão pode ser passivo; fechar um painel pode abortar os listeners ligados ao controller dele.

Armadilha da alternativa incorreta

Revise esta regra: Confundir once com cleanup geral ou usar preventDefault em listener passive. Um botão “Copiar” pode responder uma vez; um listener de rolagem que não cancela a ação padrão pode ser passivo; fechar um painel pode abortar os listeners ligados ao controller dele.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

passive listener chama preventDefault → ação não é cancelada e aviso no console; signal já abortado -> listener não é instalado

Concepção errada e falha observável

Confundir once com cleanup geral ou usar preventDefault em listener passive. / Listener once/passive/signal tem ciclo errado ou passive cancela a ação padrão.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir once com cleanup geral ou usar preventDefault em listener passive.

Pistas graduais

1. once remove após primeira invocação. 2. passive promete que não cancelará a ação padrão. 3. signal liga listener ao ciclo do AbortController.

Solução de referência explicada

Monte três casos independentes e demonstre cleanup por abort; declare que opção signal não cancela ação padrão por si.

Transferência em outro contexto

Projete instalação e limpeza de listeners para um painel de filtros que abre e fecha repetidamente, sem acumular callbacks.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.02 Parsons de componente

PARSONS · 10 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.5/5

Competência: dom.safe_creation | Requer: objects.identity, html.tree_reading | Introduz: dom.safe_creation

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Ordene os blocos para criar um item de lista seguro e removível. Explique por que o texto é texto e como o botão encontra apenas seu item.

Objetivo da atividade

Desenhe primeiro a árvore final com um li, seu texto e um botão; anote qual nó cada append recebe.

Explicação e modelo mental

• Modelo mental: Crie nós explícitos. textContent/createTextNode interpreta conteúdo como texto. Listener fecha sobre o li criado.
• Exemplo resolvido: Para exibir “Novo rascunho”, um nó de texto mantém as marcas literalmente; o navegador não cria um elemento a partir delas.
• Armadilha comum: Usar innerHTML com dado não confiável ou remover a lista inteira.
• Por que importa: Renderize mensagens externas de chat como texto literal e confira na árvore DOM que elas não criam elementos.

Material/arquivo inicial

main.js: export function montarItem(lista, mensagem) {
const item = document.createElement('li'); item.innerHTML = mensagem;
const botao = document.createElement('button'); botao.type = 'button'; botao.textContent = 'Remover';
botao.addEventListener('click', () => lista.remove()); item.append(botao); lista.append(item); return item;
}

Exemplo e resultado público

texto 'x' → aparece literalmente; clicar Remover -> só li alvo sai

Bordas e erros publicados

texto '<img src=x onerror=alert(1)>' -> nenhuma execução; dois itens iguais -> remover um

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual método cria um novo elemento HTML para montar uma interface no DOM?

 (Correta) document.createElement(tagName)
• document.createTextNode(tagName)
• element.setAttribute(tagName, valor)

Por que a resposta está certa

Correto. Para exibir “Novo rascunho”, um nó de texto mantém as marcas literalmente; o navegador não cria um elemento a partir delas.

Armadilha da alternativa incorreta

Revise esta regra: Usar innerHTML com dado não confiável ou remover a lista inteira. Para exibir “Novo rascunho”, um nó de texto mantém as marcas literalmente; o navegador não cria um elemento a partir delas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

texto '<img src=x onerror=alert(1)>' → nenhuma execução; dois itens iguais -> remover um

Concepção errada e falha observável

Usar innerHTML com dado não confiável ou remover a lista inteira. / A marcação da mensagem vira elemento, ou remover um item afeta um irmão da lista.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar innerHTML com dado não confiável ou remover a lista inteira.

Pistas graduais

1. Crie nós explícitos. 2. textContent/createTextNode interpreta conteúdo como texto. 3. Listener fecha sobre o li criado.

Solução de referência explicada

Monte li com texto e botão, anexe listener ao botão e depois li à lista; conteúdo de usuário nunca entra em innerHTML.

Transferência em outro contexto

Renderize mensagens externas de chat como texto literal e confira na árvore DOM que elas não criam elementos.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.03 Complete validação

FADED · 10 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.5/5

Competência: dom.form_validation | Requer: objects.identity, html.tree_reading | Introduz: dom.form_validation

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Complete a validação em etapas: normalize nome, converta quantidade e associe erros ao campo correto sem apagar o que a pessoa digitou.

Objetivo da atividade

Escreva as condições que tornam a quantidade convertida aceitável antes de abrir a execução.

Explicação e modelo mental

• Modelo mental: Trim no nome. Converta número explicitamente e cheque inteiro positivo. Associe mensagem ao campo sem apagar a entrada.
• Exemplo resolvido: Em um formulário de entrega, “ Lia ” vira “Lia” e “4” vira o número quatro; apresentar um erro não deve substituir os valores ainda editados.
• Armadilha comum: Validar apenas presença da string, aceitando quantidade inválida.
• Por que importa: Adapte a regra para um formulário de estoque com unidade e quantidade, preservando a entrada quando houver erro.

Material/arquivo inicial

validate.js: export function validar({ nome, quantidade }) {
const erros = {};
// TODO: normalize o nome com trim e registre erro se ficar vazio.
// TODO: converta quantidade e aceite somente inteiro positivo.
// TODO: retorne { ok: true, nome, quantidade } ou { ok: false, erros }.
}

Exemplo e resultado público

nome ' Ana ', quantidade '3' → {ok:true,nome:'Ana',quantidade:3}

Bordas e erros publicados

nome vazio -> erro nome; '2.5'/'0'/'abc' -> erro quantidade; não perder valor digitado

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Constraint_validation

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é uma vantagem de manter a validação de dados em uma função separada da manipulação do DOM?

• O navegador passa a validar automaticamente qualquer regra específica do produto.
 (Correta) A regra pode ser verificada deterministicamente sem montar a interface, e a apresentação dos erros fica independente.
• A função separada garante que nenhum dado vindo do usuário precise de verificação.

Por que a resposta está certa

Correto. Em um formulário de entrega, “ Lia ” vira “Lia” e “4” vira o número quatro; apresentar um erro não deve substituir os valores ainda editados.

Armadilha da alternativa incorreta

Revise esta regra: Validar apenas presença da string, aceitando quantidade inválida. Em um formulário de entrega, “ Lia ” vira “Lia” e “4” vira o número quatro; apresentar um erro não deve substituir os valores ainda editados.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

nome vazio → erro nome; '2.5'/'0'/'abc' -> erro quantidade; não perder valor digitado

Concepção errada e falha observável

Validar apenas presença da string, aceitando quantidade inválida. / '2.5', '0' ou 'abc' é aceito como quantidade, ou o valor digitado desaparece após erro.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Validar apenas presença da string, aceitando quantidade inválida.

Pistas graduais

1. Trim no nome. 2. Converta número explicitamente e cheque inteiro positivo. 3. Associe mensagem ao campo sem apagar a entrada.

Solução de referência explicada

Valide nome não vazio e Number(valor) inteiro positivo; retorne dados normalizados ou erros específicos por campo.

Transferência em outro contexto

Adapte a regra para um formulário de estoque com unidade e quantidade, preservando a entrada quando houver erro.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.04 FormData real

MAKE · 25 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.5/5

Competência: dom.formdata_repeated | Requer: objects.identity, html.tree_reading | Introduz: dom.formdata_repeated

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Leia os valores no submit. Diferencie um campo simples dos controles com mesmo name e converta números somente após ler a string.

Objetivo da atividade

Antes de preencher lacunas, escreva o tipo retornado por get/getAll e marque a etapa em que a conversão acontece.

Explicação e modelo mental

• Modelo mental: Leia FormData no submit preventDefault. Use getAll para campos repetidos. Valide tipo e converta quantidade só após leitura.
• Exemplo resolvido: Em um formulário de preferências, getAll("cor") pode retornar [azul, verde] e preservar as duas caixas marcadas; get("cor") retorna apenas uma ocorrência.
• Armadilha comum: Usar get no campo repetido ou tratar string numérica como number.
• Por que importa: Projete a leitura de um formulário de inscrição com múltiplas preferências e idade, validando o array e o número.

Material/arquivo inicial

form.js: export function lerInscricao(form) {
const dados = new FormData(form);
const nome = dados.get('nome');
const quantidadeTexto = dados.get('quantidade');
const interesses = dados.getAll('interesse');
// TODO: normalize e valide sem descartar os valores do formulário.
}

```
const area = document.createElement('div');
area.innerHTML = '<form><input name="nome"><input name="quantidade"><input type="checkbox" name="interesse" value="JS"><input type="checkbox" name="interesse" value="TS"></form>';
const form = area.querySelector('form');
form.elements.nome.value = 'Ana';
form.elements.quantidade.value = '2';
form.querySelectorAll('[name=interesse]')[0].checked = true;
form.querySelectorAll('[name=interesse]')[1].checked = true;
console.log(lerInscricao(form));
```

Exemplo e resultado público

nome Ana, quantidade 2, interesses JS+TS → nome Ana, quantidade 2, interesses ['JS','TS']

Bordas e erros publicados

sem interesse -> []; quantidade '2x' -> erro; FormData.get devolve string/File/null, nunca number

Teste público e aprovação visível

Resultado observável: nome Ana, quantidade 2, interesses JS+TS -> nome Ana, quantidade 2, interesses ['JS','TS']. Borda a conferir: sem interesse -> []; quantidade '2x' -> erro; FormData.get devolve string/File/null, nunca number. Entrega esperada: Extrair nome com get, interesses com getAll e quantidade com Number seguido de inteiro positivo; preserve UI e mostre erros.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/FormData

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual evento deve ser observado para tratar o envio de um formulário tanto por botão quanto pelo teclado?

• O evento click de cada botão de envio.
• O evento keydown de cada campo de texto.
 (Correta) O evento submit no próprio formulário.

Por que a resposta está certa

Correto. Em um formulário de preferências, getAll("cor") pode retornar [azul, verde] e preservar as duas caixas marcadas; get("cor") retorna apenas uma ocorrência.

Armadilha da alternativa incorreta

Revise esta regra: Usar get no campo repetido ou tratar string numérica como number. Em um formulário de preferências, getAll("cor") pode retornar [azul, verde] e preservar as duas caixas marcadas; get("cor") retorna apenas uma ocorrência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

sem interesse → []; quantidade '2x' -> erro; FormData.get devolve string/File/null, nunca number

Concepção errada e falha observável

Usar get no campo repetido ou tratar string numérica como number. / Apenas uma checkbox repetida aparece, ou quantidade sai como number antes da validação/conversão explícita.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar get no campo repetido ou tratar string numérica como number.

Pistas graduais

1. Leia FormData no submit preventDefault. 2. Use getAll para campos repetidos. 3. Valide tipo e converta quantidade só após leitura.

Solução de referência explicada

Extrair nome com get, interesses com getAll e quantidade com Number seguido de inteiro positivo; preserve UI e mostre erros.

Transferência em outro contexto

Projete a leitura de um formulário de inscrição com múltiplas preferências e idade, validando o array e o número.

Evidência de domínio

Resultado observável: nome Ana, quantidade 2, interesses JS+TS -> nome Ana, quantidade 2, interesses ['JS','TS']. Borda a conferir: sem interesse -> []; quantidade '2x' -> erro; FormData.get devolve string/File/null, nunca number. Entrega esperada: Extrair nome com get, interesses com getAll e quantidade com Number seguido de inteiro positivo; preserve UI e mostre erros. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.06 Delegação de eventos

MAKE · 25 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.5/5

Competência: dom.event_delegation | Requer: objects.identity, html.tree_reading | Introduz: dom.event_delegation

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Implemente delegação no ul: normalize o alvo com closest, confirme que o botão pertence à lista e só então leia o identificador.

Objetivo da atividade

Anote os alvos possíveis (botão ou filho) e o passo que os reduz ao controle válido.

Explicação e modelo mental

• Modelo mental: Comece no target. Use closest e cheque que botão pertence ao contêiner. Converta ou compare ID conforme contrato de dados.
• Exemplo resolvido: Numa lista de arquivos, clicar no ícone dentro de Arquivar deve localizar o botão ancestral antes de ler seu data-id.
• Armadilha comum: Ler dataset apenas de event.target e perder clique em filho.
• Por que importa: Use um único listener para ações de notificações adicionadas após a montagem da tela.

Material/arquivo inicial

app.js: export function montarDelegacao(lista, removerPorId) {
lista.addEventListener('click', (event) => {
// TODO: normalize o alvo clicado, inclusive quando for um span dentro do botão.
// TODO: ignore botão ausente, externo à lista ou com id desconhecido.
// TODO: chame removerPorId somente com o data-id do botão válido.
});
}

// A lista pode receber botões novos depois desta função ser chamada.
// <ul id="tarefas">
<button data-id="7">Remover</button>

Exemplo e resultado público

clique botão data-id=7 → remove tarefa 7

Bordas e erros publicados

clique span dentro do botão -> closest('button[data-id]') encontra botão; clique fora de botão -> ignora; ID inexistente -> sem alteração

Teste público e aprovação visível

Resultado observável: clique botão data-id=7 -> remove tarefa 7. Borda a conferir: clique span dentro do botão -> closest('button[data-id]') encontra botão; clique fora de botão -> ignora; ID inexistente -> sem alteração. Entrega esperada: Listener único usa target.closest('button[data-id]'), verifica ul.contains(botao), resolve ID e atualiza estado antes de render.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual definição corresponde a event.currentTarget dentro de um listener?

 (Correta) O elemento ao qual aquele listener foi associado.
• Sempre o elemento mais interno originalmente clicado.
• O primeiro ancestral que contém um atributo de dados.

Por que a resposta está certa

Correto. Numa lista de arquivos, clicar no ícone dentro de Arquivar deve localizar o botão ancestral antes de ler seu data-id.

Armadilha da alternativa incorreta

Revise esta regra: Ler dataset apenas de event.target e perder clique em filho. Numa lista de arquivos, clicar no ícone dentro de Arquivar deve localizar o botão ancestral antes de ler seu data-id.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

clique span dentro do botão → closest('button[data-id]') encontra botão; clique fora de botão -> ignora; ID inexistente -> sem alteração

Concepção errada e falha observável

Ler dataset apenas de event.target e perder clique em filho. / Clicar no span interno perde o botão, ou ação externa/ID inexistente altera o estado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Ler dataset apenas de event.target e perder clique em filho.

Pistas graduais

1. Comece no target. 2. Use closest e cheque que botão pertence ao contêiner. 3. Converta ou compare ID conforme contrato de dados.

Solução de referência explicada

Listener único usa target.closest('button[data-id]'), verifica ul.contains(botao), resolve ID e atualiza estado antes de render.

Transferência em outro contexto

Use um único listener para ações de notificações adicionadas após a montagem da tela.

Evidência de domínio

Resultado observável: clique botão data-id=7 -> remove tarefa 7. Borda a conferir: clique span dentro do botão -> closest('button[data-id]') encontra botão; clique fora de botão -> ignora; ID inexistente -> sem alteração. Entrega esperada: Listener único usa target.closest('button[data-id]'), verifica ul.contains(botao), resolve ID e atualiza estado antes de render. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.05 Bug de listeners duplicados

DEBUG · 22 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.0/5

Competência: dom.listener_lifecycle | Requer: objects.identity, html.tree_reading | Introduz: dom.listener_lifecycle

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 4 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Repare o ciclo de vida: mount instala uma delegação, render atualiza a árvore e destroy remove o listener. Demonstre que um elemento criado depois funciona.

Objetivo da atividade

Conte quantas vezes cada função de lifecycle roda em mount, render, render, destroy antes de alterar o código.

Explicação e modelo mental

• Modelo mental: Conte addEventListener por ciclo. Instale uma vez fora do render e delegue pelo contêiner. Guarde referência para removeEventListener no cleanup.
• Exemplo resolvido: Um painel de favoritos pode redesenhar sua lista a cada busca, enquanto um listener único no contêiner continua atendendo novos cartões.
• Armadilha comum: Confundir render de elementos com instalação de listeners.
• Por que importa: Corrija uma lista filtrável que redesenha várias vezes, mas deve manter uma ação de clique até ser desmontada.

Material/arquivo inicial

app.js: const lista = document.querySelector('#lista');
let acoes = 0;
const limpezas = [];

function render(itens) {
lista.replaceChildren();
for (const item of itens) {
const botao = document.createElement('button');
botao.textContent = item;
lista.append(botao);
}
const aoClicar = (event) => {
if (event.target.closest('button')) acoes += 1;
};
lista.addEventListener('click', aoClicar);
limpezas.push(() => lista.removeEventListener('click', aoClicar));
}

function destroy() {
lista.replaceChildren();
// BUG: limpezas nunca são executadas; renders também acumulam listeners.
}

export { render, destroy };
export const getAcoes = () => acoes;

Exemplo e resultado público

render 3 vezes; clicar uma vez → uma ação

Bordas e erros publicados

item incluído depois da primeira renderização -> ainda funciona; destroy -> listener removido

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que acontece quando addEventListener registra uma função para um tipo de evento?

• A função é executada imediatamente e removida após retornar.
 (Correta) A função fica associada para ser chamada quando aquele evento for despachado no alvo apropriado.
• A função passa a ser chamada em todo redesenho do DOM, mesmo sem evento.

Por que a resposta está certa

Correto. Um painel de favoritos pode redesenhar sua lista a cada busca, enquanto um listener único no contêiner continua atendendo novos cartões.

Armadilha da alternativa incorreta

Revise esta regra: Confundir render de elementos com instalação de listeners. Um painel de favoritos pode redesenhar sua lista a cada busca, enquanto um listener único no contêiner continua atendendo novos cartões.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

item incluído depois da primeira renderização → ainda funciona; destroy -> listener removido

Concepção errada e falha observável

Confundir render de elementos com instalação de listeners. / Um clique após render repetido executa múltiplas ações, ou destroy deixa listener ativo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir render de elementos com instalação de listeners.

Pistas graduais

1. Conte addEventListener por ciclo. 2. Instale uma vez fora do render e delegue pelo contêiner. 3. Guarde referência para removeEventListener no cleanup.

Solução de referência explicada

Separe mount que instala listener único, render que só atualiza DOM e destroy que remove listener.

Transferência em outro contexto

Corrija uma lista filtrável que redesenha várias vezes, mas deve manter uma ação de clique até ser desmontada.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.07 A11y de formulario

UI · 35 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.0/5

Competência: dom.form_accessibility | Requer: objects.identity, html.tree_reading | Introduz: dom.form_accessibility

Eixos: novidade 2 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Remodele o formulário com controles nativos, relações explícitas entre rótulos e erros e uma política de foco para falha e sucesso. Verifique sem mouse.

Objetivo da atividade

Percorra mentalmente localizar controle, entender erro, mover foco, corrigir e submeter por teclado antes de editar.

Explicação e modelo mental

• Modelo mental: Use controles HTML semânticos. Associe label e mensagem por id. Defina política de foco para erro e sucesso.
• Exemplo resolvido: Num formulário de entrega, ativar o rótulo Endereço deve focar o input; um erro ligado por id deve ser anunciado junto ao campo.
• Armadilha comum: Tratar cor/placeholder como substituto de nome acessível e gesto de mouse.
• Por que importa: Audite um formulário de reserva para que controles e erros possam ser encontrados e usados sem mouse.

Material/arquivo inicial

index.html: <form id="reserva"><input id="email" placeholder="E-mail"> Enviar Campo inválido </form> | main.js: const form = document.querySelector('#reserva');
const email = document.querySelector('#email');
form?.addEventListener('submit', event => { event.preventDefault(); if (!email.value.includes('@')) document.querySelector('#erro').textContent = 'Campo inválido'; });

Exemplo e resultado público

label for/id, button type=submit, erro aria-describedby, região aria-live → navegação e anúncio coerentes

Bordas e erros publicados

erro -> foco no primeiro campo inválido; sucesso -> resumo com tabindex=-1 recebe foco; Enter submete e Space ativa controles nativos

Teste público e aprovação visível

Teste DOM, teclado, foco, estados transitórios e fronteira de confiança.

Restrições e documentação

https://www.w3.org/WAI/tutorials/forms/labels/

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que conecta a mensagem de erro ao campo que ela descreve?

• Uma borda vermelha sem texto nem associação.
• Um placeholder que desaparece ao começar a digitar.
 (Correta) Um id na mensagem referenciado por aria-describedby, com label associada ao input.

Por que a resposta está certa

Correto. Num formulário de entrega, ativar o rótulo Endereço deve focar o input; um erro ligado por id deve ser anunciado junto ao campo.

Armadilha da alternativa incorreta

Revise esta regra: Tratar cor/placeholder como substituto de nome acessível e gesto de mouse. Num formulário de entrega, ativar o rótulo Endereço deve focar o input; um erro ligado por id deve ser anunciado junto ao campo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

erro → foco no primeiro campo inválido; sucesso -> resumo com tabindex=-1 recebe foco; Enter submete e Space ativa controles nativos

Concepção errada e falha observável

Tratar cor/placeholder como substituto de nome acessível e gesto de mouse. / Erro não é associado/anunciado e o foco não segue a política para falha ou sucesso.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar cor/placeholder como substituto de nome acessível e gesto de mouse.

Pistas graduais

1. Use controles HTML semânticos. 2. Associe label e mensagem por id. 3. Defina política de foco para erro e sucesso.

Solução de referência explicada

Associe label e erro por id, use button submit e aria-live; em falha foque o primeiro campo inválido, e em sucesso mova o foco para um resumo com tabindex=-1. Preserve os valores digitados.

Transferência em outro contexto

Audite um formulário de reserva para que controles e erros possam ser encontrados e usados sem mouse.

Evidência de domínio

Teste DOM, teclado, foco, estados transitórios e fronteira de confiança. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.12

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.12 Comentário como texto seguro

UI · 12 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.5/5

Competência: dom.untrusted_text | Requer: objects.identity, html.tree_reading | Introduz: dom.untrusted_text

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Implemente renderComentario e inspecione o texto visível e os nós realmente criados no DOM.

Objetivo da atividade

Antes de codificar, preveja o elemento estrutural e se o texto cria algum descendente.

Explicação e modelo mental

• Modelo mental: Crie um elemento contêiner. Atribua textContent, não innerHTML. Verifique DOM resultante além da aparência visual.
• Exemplo resolvido: Uma nota “2 < 5 & 7 > 3” pode aparecer como caracteres dentro de um parágrafo; a árvore continua tendo apenas o parágrafo esperado.
• Armadilha comum: Tratar texto externo como HTML confiável.
• Por que importa: Renderize títulos recuperados do storage como texto, preservando pontuação e sinais reservados literalmente.

Material/arquivo inicial

render.js: export function mostrarTitulo(alvo, titulo) {
alvo.innerHTML = titulo;
}

const titulo = '<img src=x onerror=alert(1)> Revisão';
// Alvo inicial: <h2 id="titulo"></h2>

Exemplo e resultado público

'olá' → texto literal com sinais < >; querySelector('b') -> null

Bordas e erros publicados

'<img src=x onerror=alert(1)>' -> nenhum img criado; string vazia -> nó de comentário vazio; quebra de linha preservada como texto

Teste público e aprovação visível

Teste DOM, teclado, foco, estados transitórios e fronteira de confiança.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que container.append(node) faz com um nó DOM?

 (Correta) Insere o nó como filho do contêiner.
• Converte automaticamente o nó em uma string HTML segura.
• Clona o nó e o adiciona a todos os elementos com o mesmo seletor.

Por que a resposta está certa

Correto. Uma nota “2 < 5 & 7 > 3” pode aparecer como caracteres dentro de um parágrafo; a árvore continua tendo apenas o parágrafo esperado.

Armadilha da alternativa incorreta

Revise esta regra: Tratar texto externo como HTML confiável. Uma nota “2 < 5 & 7 > 3” pode aparecer como caracteres dentro de um parágrafo; a árvore continua tendo apenas o parágrafo esperado.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

'<img src=x onerror=alert(1)>' → nenhum img criado; string vazia -> nó de comentário vazio; quebra de linha preservada como texto

Concepção errada e falha observável

Tratar texto externo como HTML confiável. / Texto de comentário cria elemento/evento em vez de permanecer como texto literal.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar texto externo como HTML confiável.

Pistas graduais

1. Crie um elemento contêiner. 2. Atribua textContent, não innerHTML. 3. Verifique DOM resultante além da aparência visual.

Solução de referência explicada

const p=document.createElement('p');p.textContent=texto;container.append(p); não use HTML parser para comentários.

Transferência em outro contexto

Renderize títulos recuperados do storage como texto, preservando pontuação e sinais reservados literalmente.

Evidência de domínio

Teste DOM, teclado, foco, estados transitórios e fronteira de confiança. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.08 Microprojeto - Kanban acessivel e persistente

PROJECT · 95 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 4.5/5

Competência: dom.kanban_v1 | Requer: objects.identity, html.tree_reading | Introduz: dom.kanban_v1

Eixos: novidade 3 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Construa a primeira versão do Kanban ponta a ponta: entrada validada, estado fonte da verdade, render seguro, persistência e feedback acessível.

Objetivo da atividade

Escreva a forma do objeto tarefa e as transições possíveis entre as três colunas antes de implementar a primeira ação.

Explicação e modelo mental

• Modelo mental: Estado em memória é fonte para render. FormData extrai entrada; valide antes de inserir. Persistência versionada e render seguro; revise foco/aria-live.
• Exemplo resolvido: Um quadro de leitura começa com “Revisar pauta”. Mover o status altera o objeto no estado; render reposiciona o cartão e a persistência guarda essa alteração.
• Armadilha comum: Escrever DOM e storage como duas fontes de verdade ou renderizar texto via HTML.
• Por que importa: Crie uma lista de reservas de equipamento com estado consistente, persistência local e controles operáveis por teclado, sem reaproveitar a estrutura visual do Kanban.

Material/arquivo inicial

index.html: <!doctype html><form id="nova-tarefa"><label for="texto">Nova tarefa</label><input id="texto" name="texto" required><button type="submit">Adicionar</button></form> <main class="quadro"><section aria-labelledby="todo-title"><h2 id="todo-title">A fazer</h2><ul id="todo" data-status="todo"> </section><section aria-labelledby="doing-title"><h2 id="doing-title">Em curso</h2><ul id="doing" data-status="doing"> </section><section aria-labelledby="done-title"><h2 id="done-title">Feito</h2><ul id="done" data-status="done"> </section></main> | style.css: .quadro { display: grid; grid-template-columns: repeat(3, minmax(12rem, 1fr)); gap: 1rem; } section { border: 1px solid #999; padding: 1rem; } | app.js: const STORAGE_KEY = 'kanban:v1';
const STATUSES = new Set(['todo', 'doing', 'done']);
const vazio = () => ({version: 1, tasks: []});
export function carregar(storage, warn = () => {}) { try { return JSON.parse(storage.getItem(STORAGE_KEY)) || vazio(); } catch { warn('JSON inválido'); return vazio(); } }
export function mover(estado, id, status) { return estado; }
export function renderizar(document, estado) { /* TODO: renderize cada status com textContent */ }
export function salvar(storage, estado) { storage.setItem(STORAGE_KEY, JSON.stringify(estado)); }
export function montarKanban(document, storage) { /* TODO: valide submit, mova por botões e anuncie */ }

Exemplo e resultado público

V1 {version:1,tasks:[{id:'t1',text:'Ler pauta',status:'todo'}]} → mover para done atualiza estado/storage; reload preserva tarefa

Bordas e erros publicados

reload -> V1 preservado; JSON inválido -> quadro vazio e aviso sem apagar o valor original; texto '<img src=x>' -> texto literal; status desconhecido -> rejeitado

Teste público e aprovação visível

Critérios desta atividade: Entrega 1 — contrato V1 registra {version:1,tasks:[{id,text,status}]}, estados permitidos e transições; o draft descreve add/move/remove. Entrega 2 — app.js mantém uma fonte de verdade, valida antes de mutar e renderiza texto do usuário com textContent; a prévia mostra as três colunas. Entrega 3 — localStorage salva após cada mudança e reload recupera o quadro; JSON inválido mostra aviso sem sobrescrever os bytes originais. Entrega 4 — checklist registra evidência de CRUD, status desconhecido, reload, JSON inválido e texto '<img>' literal; reflexão anota a sequência de teclado/foco verificada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual camada é a fonte de verdade quando renderização e storage precisam concordar?

• O DOM, porque sempre se deve recuperar estado lendo os textos dos cartões.
 (Correta) O estado em memória; a renderização deriva dele e storage persiste uma cópia serializada.
• localStorage, consultado em cada render para decidir posições visuais.

Por que a resposta está certa

Correto. Um quadro de leitura começa com “Revisar pauta”. Mover o status altera o objeto no estado; render reposiciona o cartão e a persistência guarda essa alteração.

Armadilha da alternativa incorreta

Revise esta regra: Escrever DOM e storage como duas fontes de verdade ou renderizar texto via HTML. Um quadro de leitura começa com “Revisar pauta”. Mover o status altera o objeto no estado; render reposiciona o cartão e a persistência guarda essa alteração.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

reload → V1 preservado; JSON inválido -> quadro vazio e aviso sem apagar o valor original; texto '<img src=x>' -> texto literal; status desconhecido -> rejeitado

Concepção errada e falha observável

Escrever DOM e storage como duas fontes de verdade ou renderizar texto via HTML. / Estado, renderização e storage divergem; entrada HTML vira nó ou JSON inválido é apagado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Escrever DOM e storage como duas fontes de verdade ou renderizar texto via HTML.

Pistas graduais

1. Estado em memória é fonte para render. 2. FormData extrai entrada; valide antes de inserir. 3. Persistência versionada e render seguro; revise foco/aria-live.

Solução de referência explicada

Implemente o Kanban V1 {version:1,tasks:[{id,text,status}]} com estado em memória como fonte, tarefas adicionadas/movidas/removidas, render seguro e storage local. Em JSON inválido mostre aviso e estado vazio sem sobrescrever o valor salvo.

Transferência em outro contexto

Crie uma lista de reservas de equipamento com estado consistente, persistência local e controles operáveis por teclado, sem reaproveitar a estrutura visual do Kanban.

Evidência de domínio

Critérios desta atividade: Entrega 1 — contrato V1 registra {version:1,tasks:[{id,text,status}]}, estados permitidos e transições; o draft descreve add/move/remove. Entrega 2 — app.js mantém uma fonte de verdade, valida antes de mutar e renderiza texto do usuário com textContent; a prévia mostra as três colunas. Entrega 3 — localStorage salva após cada mudança e reload recupera o quadro; JSON inválido mostra aviso sem sobrescrever os bytes originais. Entrega 4 — checklist registra evidência de CRUD, status desconhecido, reload, JSON inválido e texto '<img>' literal; reflexão anota a sequência de teclado/foco verificada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.10 Kanban em três módulos

MODIFY · 22 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.5/5

Competência: modules.browser_entry | Requer: objects.identity, html.tree_reading | Introduz: modules.browser_entry

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Separe o Kanban em estado, visualização e ponto de entrada. Defina dependências e dados entre fronteiras e valide o carregamento no navegador.

Objetivo da atividade

Desenhe setas entre state, view e main e anote qual dado/callback cruza cada uma antes de mover funções.

Explicação e modelo mental

• Modelo mental: State não importa view. View recebe snapshot e callbacks. main importa ambos e é entrada única do HTML.
• Exemplo resolvido: Num contador, state.js calcula o próximo valor, view.js desenha o número recebido e main.js conecta clique, atualização e renderização.
• Armadilha comum: Achar que separar arquivos sem imports/exports basta para executar no navegador.
• Por que importa: Extraia model/view/main de um app de lembretes mantendo uma fronteira testável para regras e persistência.

Material/arquivo inicial

state.js: const key = 'kanban:v1';
export function carregar(storage) { try { return JSON.parse(storage.getItem(key)); } catch { return {version: 1, tasks: []}; } }
export function mover(state, id, status) { return state; }
export function salvar(storage, state) { storage.setItem(key, JSON.stringify(state)); } | view.js: export function render(document, state, list) { list.replaceChildren(...state.tasks.map(task => { const item = document.createElement('li'); item.innerHTML = task.text; return item; })); } | main.js: import {carregar} from './state.js';
import {render} from './view.js';
export function boot(storage, document, list) { render(document, carregar(storage), list); }

Exemplo e resultado público

state exporta operações; view recebe estado; main conecta eventos; adicionar/mover/reload → V1 preservado

Bordas e erros publicados

import com caminho relativo e extensão .js; abrir via file:// -> servir por HTTP local; módulo sem globals implícitas

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual módulo deve compor view e estado sem fazer o domínio depender do DOM?

• O módulo state importa view para atualizar elementos diretamente.
• Todos os arquivos compartilham uma variável global em vez de import/export.
 (Correta) O ponto de entrada importa state/view e conecta dados e eventos.

Por que a resposta está certa

Correto. Num contador, state.js calcula o próximo valor, view.js desenha o número recebido e main.js conecta clique, atualização e renderização.

Armadilha da alternativa incorreta

Revise esta regra: Achar que separar arquivos sem imports/exports basta para executar no navegador. Num contador, state.js calcula o próximo valor, view.js desenha o número recebido e main.js conecta clique, atualização e renderização.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

import com caminho relativo e extensão .js; abrir via file:// → servir por HTTP local; módulo sem globals implícitas

Concepção errada e falha observável

Achar que separar arquivos sem imports/exports basta para executar no navegador. / O import não resolve no navegador ou a separação de módulos mantém responsabilidade acoplada.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que separar arquivos sem imports/exports basta para executar no navegador.

Pistas graduais

1. State não importa view. 2. View recebe snapshot e callbacks. 3. main importa ambos e é entrada única do HTML.

Solução de referência explicada

Separe domínio/persistência em state, DOM em view e composição em main; use imports ESM explícitos e servidor local.

Transferência em outro contexto

Extraia model/view/main de um app de lembretes mantendo uma fronteira testável para regras e persistência.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 11.13

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

11.13 Save antigo no Kanban

MODIFY · 22 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.5/5

Competência: storage.schema_migration | Requer: objects.identity, html.tree_reading | Introduz: storage.schema_migration

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 4 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Planeje o carregamento por etapas: parse, validação por registro, migração e gravação da versão atual. Decida como itens válidos sobrevivem a um item danificado.

Objetivo da atividade

Monte uma tabela com versão de entrada, função aplicada e formato final; marque onde falha por registro pode ser recuperada.

Explicação e modelo mental

• Modelo mental: Leia e parseie dentro de try/catch. Valide versão e cada registro separadamente. Converta done para status e gere IDs estáveis antes de salvar V2.
• Exemplo resolvido: Uma lista antiga de favoritos pode virar objetos com ids estáveis. Se uma entrada estiver incompleta, as demais podem ser convertidas e a interface pode reportar o descarte.
• Armadilha comum: Descartar todas as tarefas porque um único item antigo está inválido.
• Por que importa: Migre uma coleção antiga de favoritos para objetos identificáveis sem descartar entradas aproveitáveis.

Material/arquivo inicial

```
migration.js: export function migrarV1ParaV2(entrada) {
// V1 usado pelo Kanban de 11.08: { version: 1, tasks: [{ id, text, status }] }
// V2 esperado: { version: 2, tasks: [{ id, text, status, priority }] }
// TODO: preserve ids/text/status, use priority='normal' quando ausente e valide por tarefa.
}

export function carregar(entrada) {
// TODO: V1 passa pela migração uma vez; V2 não é migrado novamente.
}
```

Exemplo e resultado público

V1 {version:1,tasks:[{id:'t1',text:'Ler pauta',status:'todo'}]} → V2 {version:2,tasks:[{id:'t1',text:'Ler pauta',status:'todo',priority:'normal'}]}

Bordas e erros publicados

V1 -> adiciona priority='normal'; V2 -> não migra de novo; prioridade válida existente é preservada; registro parcial é ignorado com aviso; JSON inválido não sobrescreve storage

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que tarefas persistidas devem ter um identificador estável?

 (Correta) Para que uma tarefa continue reconhecível mesmo quando seu texto ou outros campos mudam.
• Para obrigar duas tarefas diferentes a serem tratadas como a mesma tarefa.
• Para codificar a posição atual da tarefa na lista como parte de sua identidade.

Por que a resposta está certa

Correto. Uma lista antiga de favoritos pode virar objetos com ids estáveis. Se uma entrada estiver incompleta, as demais podem ser convertidas e a interface pode reportar o descarte.

Armadilha da alternativa incorreta

Revise esta regra: Descartar todas as tarefas porque um único item antigo está inválido. Uma lista antiga de favoritos pode virar objetos com ids estáveis. Se uma entrada estiver incompleta, as demais podem ser convertidas e a interface pode reportar o descarte.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

V1 → adiciona priority='normal'; V2 -> não migra de novo; prioridade válida existente é preservada; registro parcial é ignorado com aviso; JSON inválido não sobrescreve storage

Concepção errada e falha observável

Descartar todas as tarefas porque um único item antigo está inválido. / A migração não produz V2 idempotente, perde itens válidos ou sobrescreve storage inválido.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Descartar todas as tarefas porque um único item antigo está inválido.

Pistas graduais

1. Leia e parseie dentro de try/catch. 2. Valide versão e cada registro separadamente. 3. Converta done para status e gere IDs estáveis antes de salvar V2.

Solução de referência explicada

Use o loader e os fixtures do Kanban 11.08. Migre V1 para V2 preservando id/text/status e adicionando priority='normal'; valide cada registro, reporte os descartados e nunca migre V2 novamente. Grave a versão atual só depois da conversão segura.

Transferência em outro contexto

Migre uma coleção antiga de favoritos para objetos identificáveis sem descartar entradas aproveitáveis.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Browser evergreen; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12 HTTP, assincronismo e tempo

14 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

HTTP e o corpo da Response vêm antes do dashboard com fetch. A API recente exige suporte ou fallback fiel.

Ordem: 12.10 · 12.11 · 12.01 · 12.04 · 12.02 · 12.03 · 12.05 · 12.06 · 12.07 · 12.09 · 12.12 · 12.13 · 12.14 · 12.08

12.10 Antes de fetch: HTTP

READ · 10 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 2.5/5

Competência: http.response_model | Requer: functions.return | Introduz: http.response_model

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 2 · depuração 1 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Em cada troca HTTP, identifique o que vem do status, dos headers e do corpo. Defina a política antes de escolher o parser.

Objetivo da atividade

Anote status, tipo de conteúdo e presença de corpo nessa ordem para cada troca.

Explicação e modelo mental

• Modelo mental: Leia método e URL da request. Leia status e headers da response antes do corpo. Escolha parser conforme status e Content-Type.
• Exemplo resolvido: Uma requisição de imagem em faixa pode responder 206 com bytes parciais e Content-Range; o cliente deve distinguir essa representação de uma resposta JSON.
• Armadilha comum: Confundir status, cabeçalho e corpo ou assumir JSON em toda resposta.
• Por que importa: Projete o tratamento de resposta para upload que pode ser aceito para processamento ou rejeitado.

Material/arquivo inicial

material.md: GET /itens -> 200, Content-Type application/json, body=[{"id":1}]
POST /itens -> 201, Location /itens/2, Content-Type application/json, body={"id":2}
GET /ausente -> 404, Content-Type text/plain, body=not found
GET /vazio -> 204, sem corpo

Para cada troca, separe status, headers e body. Não tente parsear JSON no 204 nem trate fetch como rejeição automática para 404.

Exemplo e resultado público

GET 200 JSON → parse JSON; POST 201 -> recurso criado + Location; 404 -> erro de domínio, não sucesso

Bordas e erros publicados

204 -> null sem chamar json(); 200/201 com Content-Type ausente ou diferente de application/json -> TypeError; 404 text/plain -> erro HTTP com status preservado

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Response

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que informação o status HTTP comunica, independentemente do formato do corpo?

• A estrutura exata dos campos do JSON que o cliente deve receber.
 (Correta) O resultado protocolar da solicitação, como sucesso, redirecionamento ou uma classe de falha.
• Se o corpo da resposta foi previamente consumido por JavaScript.

Por que a resposta está certa

Correto. Uma requisição de imagem em faixa pode responder 206 com bytes parciais e Content-Range; o cliente deve distinguir essa representação de uma resposta JSON.

Armadilha da alternativa incorreta

Revise esta regra: Confundir status, cabeçalho e corpo ou assumir JSON em toda resposta. Uma requisição de imagem em faixa pode responder 206 com bytes parciais e Content-Range; o cliente deve distinguir essa representação de uma resposta JSON.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

204 → null sem chamar json(); 200/201 com Content-Type ausente ou diferente de application/json -> TypeError; 404 text/plain -> erro HTTP com status preservado

Concepção errada e falha observável

Confundir status, cabeçalho e corpo ou assumir JSON em toda resposta. / 204 tenta ler JSON, status não-2xx vira sucesso ou Content-Type inválido não segue a política publicada.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir status, cabeçalho e corpo ou assumir JSON em toda resposta.

Pistas graduais

1. Leia método e URL da request. 2. Leia status e headers da response antes do corpo. 3. Escolha parser conforme status e Content-Type.

Solução de referência explicada

Verifique status antes do corpo; 204 retorna null sem parse. Para 200/201 exija Content-Type application/json e então faça parse; em status não-2xx ou tipo ausente/incompatível, devolva erro contextual que preserva o status.

Transferência em outro contexto

Projete o tratamento de resposta para upload que pode ser aceito para processamento ou rejeitado.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Browser (Fetch API); o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.11 Response tem corpo consumível

INVESTIGATE · 20 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.5/5

Competência: fetch.body_consumption | Requer: functions.return | Introduz: fetch.body_consumption

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Inspecione status e representação, escolha um leitor de corpo e registre bodyUsed antes/depois.

Objetivo da atividade

Escolha primeiro a representação pretendida e preveja se o corpo ficará consumido após essa leitura.

Explicação e modelo mental

• Modelo mental: Inspecione status/header primeiro. Escolha método de leitura conforme protocolo. Se precisar de duas leituras, clone antes da primeira.
• Exemplo resolvido: Um recibo em texto simples pode ser lido uma vez com text(); depois bodyUsed indica consumo, e outra leitura do mesmo stream não deve ser presumida reutilizável.
• Armadilha comum: Achar que header Content-Type consome/parseia corpo ou que body pode ser lido repetidamente.
• Por que importa: Escolha parser e política de leitura única para uma resposta CSV e uma resposta de imagem.

Material/arquivo inicial

client.js: export async function inspecionar(resposta) {
const antes=resposta.bodyUsed; const status=resposta.status; const tipo=resposta.headers.get('Content-Type'); let corpo;
try { corpo=await resposta.json(); } catch(erro) { return {status,tipo,antes,depois:resposta.bodyUsed,erro:erro.name}; }
return {status,tipo,antes,depois:resposta.bodyUsed,corpo};
}

Exemplo e resultado público

await res.json() → objeto e bodyUsed=true; segunda leitura res.text() -> TypeError

Bordas e erros publicados

204/no body -> tratamento explícito; Content-Type não força parser automaticamente; res.clone() antes de consumir permite duas leituras

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Response/json

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que a propriedade Response.bodyUsed informa?

• Se o servidor já encerrou a conexão TCP.
• Se os headers foram validados contra um schema JSON.
 (Correta) Se o stream do corpo dessa resposta já foi consumido.

Por que a resposta está certa

Correto. Um recibo em texto simples pode ser lido uma vez com text(); depois bodyUsed indica consumo, e outra leitura do mesmo stream não deve ser presumida reutilizável.

Armadilha da alternativa incorreta

Revise esta regra: Achar que header Content-Type consome/parseia corpo ou que body pode ser lido repetidamente. Um recibo em texto simples pode ser lido uma vez com text(); depois bodyUsed indica consumo, e outra leitura do mesmo stream não deve ser presumida reutilizável.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

204/no body → tratamento explícito; Content-Type não força parser automaticamente; res.clone() antes de consumir permite duas leituras

Concepção errada e falha observável

Achar que header Content-Type consome/parseia corpo ou que body pode ser lido repetidamente. / O mesmo corpo é consumido duas vezes ou a representação lida não corresponde ao Content-Type/status.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que header Content-Type consome/parseia corpo ou que body pode ser lido repetidamente.

Pistas graduais

1. Inspecione status/header primeiro. 2. Escolha método de leitura conforme protocolo. 3. Se precisar de duas leituras, clone antes da primeira.

Solução de referência explicada

Demonstre text/json/blob/arrayBuffer/formData em responses apropriadas e registre bodyUsed antes/depois; valide tipo retornado.

Transferência em outro contexto

Escolha parser e política de leitura única para uma resposta CSV e uma resposta de imagem.

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Browser (Fetch API); o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.01 Ordem do event loop

PREDICT · 6 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.0/5

Competência: async.event_loop | Requer: functions.return | Introduz: async.event_loop

Eixos: novidade 1 · sintaxe 1 · composição 1 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 1 · ferramentas 1

VISÃO DO ALUNO

Sem executar, preveja a ordem dos logs e explique por que atraso zero não torna um timer síncrono.

Objetivo da atividade

Escreva uma linha para pilha atual, outra para microtasks e outra para a próxima task antes de prever a ordem.

Explicação e modelo mental

• Modelo mental: Execute a pilha síncrona. Depois drene fila de microtasks. Então processe a próxima task.
• Exemplo resolvido: Um log síncrono antes e depois de queueMicrotask permanece na mesma pilha; a microtask roda após essa pilha e antes do callback setTimeout.
• Armadilha comum: Tratar timeout 0 como execução imediata.
• Por que importa: Preveja a ordem entre uma resposta já resolvida e uma notificação de interface agendada por timer; identifique cada fila.

Material/arquivo inicial

main.js: console.log('A');
setTimeout(() => console.log('T'), 0);
Promise.resolve().then(() => console.log('P'));
console.log('B');

Exemplo e resultado público

execução → A,B,P,T

Bordas e erros publicados

dois then encadeados e timer -> microtasks drenam antes do timer, com FIFO entre microtasks agendadas

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando a função passada ao construtor new Promise(executor) é executada?

 (Correta) Imediatamente, durante a chamada ao construtor, na pilha síncrona atual.
• Somente depois que todas as reações de then forem executadas.
• Na próxima task, como um callback de setTimeout.

Por que a resposta está certa

Correto. Um log síncrono antes e depois de queueMicrotask permanece na mesma pilha; a microtask roda após essa pilha e antes do callback setTimeout.

Armadilha da alternativa incorreta

Revise esta regra: Tratar timeout 0 como execução imediata. Um log síncrono antes e depois de queueMicrotask permanece na mesma pilha; a microtask roda após essa pilha e antes do callback setTimeout.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

dois then encadeados e timer → microtasks drenam antes do timer, com FIFO entre microtasks agendadas

Concepção errada e falha observável

Tratar timeout 0 como execução imediata. / O log coloca timer antes da continuação Promise ou trata atraso zero como execução síncrona.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar timeout 0 como execução imediata.

Pistas graduais

1. Execute a pilha síncrona. 2. Depois drene fila de microtasks. 3. Então processe a próxima task.

Solução de referência explicada

Ordem A,B,P,T; promessa agenda microtask e setTimeout agenda task, mesmo com atraso zero.

Transferência em outro contexto

Preveja a ordem entre uma resposta já resolvida e uma notificação de interface agendada por timer; identifique cada fila.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.04 Microtask vs task

TRACE · 9 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: async.microtasks_tasks | Requer: functions.return | Introduz: async.microtasks_tasks

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Trace pilha síncrona, fila FIFO de microtasks e task do timer, incluindo uma microtask agendada por outra.

Objetivo da atividade

Desenhe a fila após cada agendamento e só escreva a ordem final quando as instruções síncronas terminarem.

Explicação e modelo mental

• Modelo mental: Marque execução síncrona primeiro. Adicione microtasks à fila na ordem de agendamento. Task T só roda após esvaziar a fila.
• Exemplo resolvido: Se um callback de Promise enfileira outra microtask enquanto uma já espera, a que aguardava conserva sua posição; a nova é anexada ao fim.
• Armadilha comum: Supor que novas microtasks pulam à frente das que já estavam na fila.
• Por que importa: Trace uma atualização assíncrona que agenda uma segunda microtask antes de uma notificação atrasada.

Material/arquivo inicial

main.js: console.log('A');
Promise.resolve().then(() => { console.log('P1'); Promise.resolve().then(() => console.log('P2')); });
queueMicrotask(() => console.log('Q'));
setTimeout(() => console.log('T'), 0);
console.log('B');

Exemplo e resultado público

ordem → A,B,P1,Q,T

Bordas e erros publicados

P1 agenda P2 durante execução -> A,B,P1,Q,P2,T

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API/Microtask_guide

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual par classifica corretamente os callbacks de Promise.then e setTimeout?

• Promise.then agenda uma task; setTimeout agenda uma microtask.
 (Correta) Promise.then agenda uma microtask; setTimeout agenda uma task.
• Ambos agendam microtasks e diferem somente pelo atraso numérico.

Por que a resposta está certa

Correto. Se um callback de Promise enfileira outra microtask enquanto uma já espera, a que aguardava conserva sua posição; a nova é anexada ao fim.

Armadilha da alternativa incorreta

Revise esta regra: Supor que novas microtasks pulam à frente das que já estavam na fila. Se um callback de Promise enfileira outra microtask enquanto uma já espera, a que aguardava conserva sua posição; a nova é anexada ao fim.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

P1 agenda P2 durante execução → A,B,P1,Q,P2,T

Concepção errada e falha observável

Supor que novas microtasks pulam à frente das que já estavam na fila. / Uma microtask enfileirada durante outra fica depois do timer, apesar da fila ainda estar sendo drenada.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Supor que novas microtasks pulam à frente das que já estavam na fila.

Pistas graduais

1. Marque execução síncrona primeiro. 2. Adicione microtasks à fila na ordem de agendamento. 3. Task T só roda após esvaziar a fila.

Solução de referência explicada

Trace a fila FIFO de microtasks, incluindo as adicionadas por P1; timer é task posterior.

Transferência em outro contexto

Trace uma atualização assíncrona que agenda uma segunda microtask antes de uma notificação atrasada.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.02 Parsons de retry

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: async.retry | Requer: functions.return, errors.types | Introduz: async.retry

Eixos: novidade 2 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Ordene os blocos de retry e declare se max representa tentativas totais. Inclua validação e os pontos onde uma espera é permitida.

Objetivo da atividade

Defina o significado de max e marque no laço a única transição que pode aguardar antes de uma nova tentativa.

Explicação e modelo mental

• Modelo mental: Defina se max conta tentativas totais. Incremente antes de executar op. Só aguarde se ainda houver tentativa.
• Exemplo resolvido: Se uma gravação de arquivo puder ser tentada duas vezes, uma falha transitória na primeira chamada permite uma espera e uma última chamada; não há espera depois da falha final.
• Armadilha comum: Repetir infinitamente ou esperar após última falha.
• Por que importa: Especifique retry limitado para uma gravação temporariamente indisponível, com limites explícitos de chamadas e esperas.

Material/arquivo inicial

main.js: export async function tentar(operacao,max,esperar,atraso) {
if(!Number.isInteger(max)||max<1)return Promise.reject(new RangeError('max deve ser inteiro positivo')); let ultima;
for(let tentativa=0;tentativa<=max;tentativa++){try{return await operacao();}catch(erro){ultima=erro;await esperar(atraso);}}
return Promise.reject(ultima);
}

Exemplo e resultado público

op falha duas vezes e resolve na 3ª, max=3 → 3 chamadas, resultado

Bordas e erros publicados

max=1 com falha -> 1 chamada e erro original; delay só entre tentativas; max<=0 -> rejeição de validação antes da primeira chamada

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

Conceitos prévios (Primers)

Revise 04.11: TypeError descreve argumento de tipo incorreto e RangeError um Number fora do domínio. Em retry, rejeite max inválido com RangeError antes de chamar a operação. Dentro de uma função async, await numa Promise rejeitada lança o erro naquela linha. try { await ... } catch (erro) { ... } captura essa rejeição como captura um throw comum. throw interrompe a função e a Promise dela rejeita.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como retry se diferencia de fallback?

• Retry sempre troca a operação por outra; fallback repete a mesma chamada.
• São nomes diferentes para aguardar antes de qualquer operação assíncrona.
 (Correta) Retry tenta novamente a mesma operação; fallback oferece um caminho alternativo.

Por que a resposta está certa

Correto. Se uma gravação de arquivo puder ser tentada duas vezes, uma falha transitória na primeira chamada permite uma espera e uma última chamada; não há espera depois da falha final.

Armadilha da alternativa incorreta

Revise esta regra: Repetir infinitamente ou esperar após última falha. Se uma gravação de arquivo puder ser tentada duas vezes, uma falha transitória na primeira chamada permite uma espera e uma última chamada; não há espera depois da falha final.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

max=1 com falha → 1 chamada e erro original; delay só entre tentativas; max<=0 -> rejeição de validação antes da primeira chamada

Concepção errada e falha observável

Repetir infinitamente ou esperar após última falha. / A operação excede max, espera após a última tentativa ou troca o erro original por erro de retry.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Repetir infinitamente ou esperar após última falha.

Pistas graduais

1. Defina se max conta tentativas totais. 2. Incremente antes de executar op. 3. Só aguarde se ainda houver tentativa.

Solução de referência explicada

Loop for attempt de 1 a max, return await op no try; catch relança na última, senão await delay; valide max.

Transferência em outro contexto

Especifique retry limitado para uma gravação temporariamente indisponível, com limites explícitos de chamadas e esperas.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.03 Complete Promise.all

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: async.promise_all | Requer: functions.return | Introduz: async.promise_all

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Complete carregar iniciando juntas as requisições independentes e mantendo a ordem da API no objeto retornado. Explique o limite de rejeição.

Objetivo da atividade

Marque quais chamadas dependem de outras; inicie apenas as independentes antes do primeiro await.

Explicação e modelo mental

• Modelo mental: Inicie as três operações antes do primeiro await. Passe promises em array na ordem do contrato. Desestruture resultado após Promise.all.
• Exemplo resolvido: Uma tela pode pedir detalhes de conta e preferências ao mesmo tempo; se preferências terminar primeiro, cada valor ainda ocupa seu campo nomeado no resultado final.
• Armadilha comum: Fazer awaits sequenciais para recursos independentes ou pensar que rejeição cancela requests.
• Por que importa: Carregue seções de conta independentes ao mesmo tempo e monte a resposta por nome, ainda que terminem em ordens diferentes.

Material/arquivo inicial

main.js: export async function carregar(api) {
// api.produtos(), api.usuarios() e api.precos() são independentes.
// TODO: iniciar as três operações em paralelo e devolver os resultados na ordem acima.
}

Exemplo e resultado público

três promises resolvem 1,2,3 → Promise.all devolve [1,2,3] na ordem de entrada

Bordas e erros publicados

resolução fora de ordem -> array ainda [1,2,3]; uma rejeita -> Promise.all rejeita, outras operações não são automaticamente canceladas

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que Promise.all faz com as operações representadas pelos valores que recebe?

 (Correta) Coordena e aguarda os valores recebidos; não chama por conta própria as funções que iniciariam operações.
• Invoca cada função de serviço recebida e inicia todas as operações automaticamente.
• Converte qualquer valor recebido em uma operação de rede concorrente.

Por que a resposta está certa

Correto. Uma tela pode pedir detalhes de conta e preferências ao mesmo tempo; se preferências terminar primeiro, cada valor ainda ocupa seu campo nomeado no resultado final.

Armadilha da alternativa incorreta

Revise esta regra: Fazer awaits sequenciais para recursos independentes ou pensar que rejeição cancela requests. Uma tela pode pedir detalhes de conta e preferências ao mesmo tempo; se preferências terminar primeiro, cada valor ainda ocupa seu campo nomeado no resultado final.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

resolução fora de ordem → array ainda [1,2,3]; uma rejeita -> Promise.all rejeita, outras operações não são automaticamente canceladas

Concepção errada e falha observável

Fazer awaits sequenciais para recursos independentes ou pensar que rejeição cancela requests. / Resultados fora de ordem de conclusão mudam a ordem do array, ou uma rejeição é descrita como cancelamento automático.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Fazer awaits sequenciais para recursos independentes ou pensar que rejeição cancela requests.

Pistas graduais

1. Inicie as três operações antes do primeiro await. 2. Passe promises em array na ordem do contrato. 3. Desestruture resultado após Promise.all.

Solução de referência explicada

const [produtos,usuarios,precos]=await Promise.all([api.produtos(),api.usuarios(),api.precos()]); retorne objeto nomeado.

Transferência em outro contexto

Carregue seções de conta independentes ao mesmo tempo e monte a resposta por nome, ainda que terminem em ordens diferentes.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.05 Bug de await e captura local

DEBUG · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: async.await_catch | Requer: functions.return | Introduz: async.await_catch

Eixos: novidade 2 · sintaxe 4 · composição 4 · estado 4 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Repare as duas versões assíncronas para que usem valores resolvidos e o catch local receba a rejeição pretendida.

Objetivo da atividade

Marque onde a Promise é criada, liquidada e aguardada antes de editar try/catch.

Explicação e modelo mental

• Modelo mental: Promise é um valor até ser aguardada. Para tratar localmente, faça await dentro do try. Teste tanto resolve quanto reject com promise controlada.
• Exemplo resolvido: Se um carregador retorna Promise, use await dentro do try quando essa função precisa traduzir localmente uma rejeição para uma mensagem útil.
• Armadilha comum: Achar que try/catch captura rejeição de Promise retornada sem await.
• Por que importa: Repare um helper que converte erro de configuração assíncrono em mensagem de domínio sem perder a causa.

Material/arquivo inicial

profile.js: export async function carregar(fetchProfile) {
try {
return fetchProfile();
} catch (erro) {
return { nome: 'offline' };
}
}

// fetchProfile() devolve uma Promise que pode rejeitar depois da chamada.
// O fallback deveria ser usado quando essa rejeição ocorre.

Exemplo e resultado público

await asyncFunc() → valor resolvido; try{return await asyncFunc()}catch -> catch local intercepta rejeição

Bordas e erros publicados

return asyncFunc() sem await em try -> catch local não intercepta rejeição posterior; await faltante propaga Promise como valor

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

Conceitos prévios (Primers)

throw new Error('mensagem') dentro de uma função async faz a Promise devolvida rejeitar; quem chamou captura com try/catch em volta do await.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que uma função async retorna mesmo quando seu corpo usa return com um valor comum?

• O valor comum diretamente, sem envolver uma Promise.
 (Correta) Uma Promise que será cumprida com esse valor.
• Um callback que só pode ser executado por setTimeout.

Por que a resposta está certa

Correto. Se um carregador retorna Promise, use await dentro do try quando essa função precisa traduzir localmente uma rejeição para uma mensagem útil.

Armadilha da alternativa incorreta

Revise esta regra: Achar que try/catch captura rejeição de Promise retornada sem await. Se um carregador retorna Promise, use await dentro do try quando essa função precisa traduzir localmente uma rejeição para uma mensagem útil.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

return asyncFunc() sem await em try → catch local não intercepta rejeição posterior; await faltante propaga Promise como valor

Concepção errada e falha observável

Achar que try/catch captura rejeição de Promise retornada sem await. / A Promise rejeita depois do retorno e escapa do catch local, ou é usada como se fosse o valor resolvido.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que try/catch captura rejeição de Promise retornada sem await.

Pistas graduais

1. Promise é um valor até ser aguardada. 2. Para tratar localmente, faça await dentro do try. 3. Teste tanto resolve quanto reject com promise controlada.

Solução de referência explicada

Use const x=await asyncFunc() e return await asyncFunc() dentro do try quando a intenção é capturar localmente; preserve a causa.

Transferência em outro contexto

Repare um helper que converte erro de configuração assíncrono em mensagem de domínio sem perder a causa.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.06 Busca cancelável

MAKE · 25 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 4.0/5

Competência: fetch.abort_race | Requer: functions.return | Introduz: fetch.abort_race

Eixos: novidade 3 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 4 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Implemente a busca somente com o fetcher injetado e fixtures determinísticas. Cancele trabalho obsoleto e proteja a renderização se uma resposta antiga chegar tarde.

Objetivo da atividade

Escreva a invariável “só a geração atual pode renderizar” e indique onde incrementar e verificar o id.

Explicação e modelo mental

• Modelo mental: Guarde controller corrente. Abort anterior e gere id crescente. Antes de render, cheque id e trate AbortError separado.
• Exemplo resolvido: Busque uma cidade com fixture lenta e depois outra com fixture rápida. Mesmo que a primeira resolva por último apesar do abort, um id de geração impede que substitua o resultado novo.
• Armadilha comum: Confiar só na ordem de chegada ou mostrar abort como falha.
• Por que importa: Construa autocomplete latest-query-wins com fixtures que deixam liberar respostas em ordens diferentes.

Material/arquivo inicial

search.js: export function criarBusca(fetcher) {
let controllerAtual;
let sequencia = 0;
return async function buscar(query) {
// TODO: cancelar a busca anterior, iniciar a nova e ignorar resposta obsoleta.
// fetcher(query, { signal }) usa fixtures locais e pode resolver/rejeitar com atraso.
};
}

Exemplo e resultado público

fixture A lenta, depois B rápida → B visível; A abortada

Bordas e erros publicados

abort anterior ocorre antes de iniciar B; resposta A chega tardiamente apesar de abort -> token de sequência impede sobrescrita; AbortError não aparece como erro ao usuário; nenhuma chamada à rede externa

Teste público e aprovação visível

Resultado observável: fixture A lenta, depois B rápida -> B visível; A abortada. Borda a conferir: abort anterior ocorre antes de iniciar B; resposta A chega tardiamente apesar de abort -> token de sequência impede sobrescrita; AbortError não aparece como erro ao usuário; nenhuma chamada à rede externa. Entrega esperada: Injete fetcher controlado nos exemplos: cada busca cria controller e geração; nova busca aborta anterior; somente geração atual altera UI e erros de cancelamento são silenciosos. Não use endpoint real.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que operações independentes costumam usar um AbortController novo para cada tentativa?

• Um AbortController novo faz qualquer servidor responder mais rápido.
• O navegador aceita somente um AbortController durante toda a vida da página.
 (Correta) Depois de abortado, um AbortSignal continua abortado e não pode ser rearmado para uma nova tentativa.

Por que a resposta está certa

Correto. Busque uma cidade com fixture lenta e depois outra com fixture rápida. Mesmo que a primeira resolva por último apesar do abort, um id de geração impede que substitua o resultado novo.

Armadilha da alternativa incorreta

Revise esta regra: Confiar só na ordem de chegada ou mostrar abort como falha. Busque uma cidade com fixture lenta e depois outra com fixture rápida. Mesmo que a primeira resolva por último apesar do abort, um id de geração impede que substitua o resultado novo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

abort anterior ocorre antes de iniciar B; resposta A chega tardiamente apesar de abort → token de sequência impede sobrescrita; AbortError não aparece como erro ao usuário; nenhuma chamada à rede externa

Concepção errada e falha observável

Confiar só na ordem de chegada ou mostrar abort como falha. / Resposta antiga sobrescreve a nova, AbortError aparece como erro ao usuário ou a fixture não recebe sinal.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confiar só na ordem de chegada ou mostrar abort como falha.

Pistas graduais

1. Guarde controller corrente. 2. Abort anterior e gere id crescente. 3. Antes de render, cheque id e trate AbortError separado.

Solução de referência explicada

Injete fetcher controlado nos exemplos: cada busca cria controller e geração; nova busca aborta anterior; somente geração atual altera UI e erros de cancelamento são silenciosos. Não use endpoint real.

Transferência em outro contexto

Construa autocomplete latest-query-wins com fixtures que deixam liberar respostas em ordens diferentes.

Evidência de domínio

Resultado observável: fixture A lenta, depois B rápida -> B visível; A abortada. Borda a conferir: abort anterior ocorre antes de iniciar B; resposta A chega tardiamente apesar de abort -> token de sequência impede sobrescrita; AbortError não aparece como erro ao usuário; nenhuma chamada à rede externa. Entrega esperada: Injete fetcher controlado nos exemplos: cada busca cria controller e geração; nova busca aborta anterior; somente geração atual altera UI e erros de cancelamento são silenciosos. Não use endpoint real. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline browser; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.07 Combinadores de Promise

COMPARE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: async.combinators | Requer: functions.return | Introduz: async.combinators

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 1 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Escolha um combinador Promise para cada requisito e justifique sucesso, falha parcial e timeout.

Objetivo da atividade

Em cada cenário, sublinhe se precisa de todos os valores, todos os estados, primeiro settlement ou primeiro sucesso.

Explicação e modelo mental

• Modelo mental: Identifique critério: todos, parcial, primeiro settle ou primeiro sucesso. Especifique comportamento de rejeição. No timeout, planeje cleanup e cancelamento.
• Exemplo resolvido: Um resumo de viagem precisa tanto do horário de chegada quanto do portão: Promise.all entrega o par quando ambos cumprem; se um rejeita, o par não fica disponível.
• Armadilha comum: Tratar any e race como equivalentes ou pensar que all cancela tarefas.
• Por que importa: Localize o primeiro pacote de ícones de tema disponível entre cache local, pacote instalado e catálogo remoto; verifique o caso de sucesso e o resultado quando todas as fontes falharem.

Material/arquivo inicial

material.md: Requisito 1: esperar todas as operações obrigatórias (Promise.all).
Requisito 2: conservar sucessos parciais (Promise.allSettled).
Requisito 3: primeiro término (Promise.race).
Requisito 4: primeiro sucesso (Promise.any).
Requisito 5: timeout com limpeza explícita; um race sozinho não cancela o trabalho perdedor.
Requisito 6: consultar réplicas e aceitar o primeiro sucesso; cancelar/limpar as demais só se a API oferecer essa operação.

Exemplo e resultado público

todos necessários → all; parcial -> allSettled; primeiro settle -> race; primeiro sucesso -> any

Bordas e erros publicados

any com todas rejeitadas -> AggregateError; race timeout exige limpar timer/cancelar trabalho se possível; all não cancela entradas

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/any

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que diferença separa Promise.any de Promise.race?

 (Correta) any espera o primeiro cumprimento; race encerra no primeiro cumprimento ou rejeição.
• Ambos retornam o primeiro sucesso e ignoram rejeições anteriores.
• all cancela as operações restantes quando uma rejeita.

Por que a resposta está certa

Correto. Um resumo de viagem precisa tanto do horário de chegada quanto do portão: Promise.all entrega o par quando ambos cumprem; se um rejeita, o par não fica disponível.

Armadilha da alternativa incorreta

Revise esta regra: Tratar any e race como equivalentes ou pensar que all cancela tarefas. Um resumo de viagem precisa tanto do horário de chegada quanto do portão: Promise.all entrega o par quando ambos cumprem; se um rejeita, o par não fica disponível.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

any com todas rejeitadas → AggregateError; race timeout exige limpar timer/cancelar trabalho se possível; all não cancela entradas

Concepção errada e falha observável

Tratar any e race como equivalentes ou pensar que all cancela tarefas. / O combinador não corresponde ao requisito de sucesso/falha parcial, ou timer de timeout permanece ativo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar any e race como equivalentes ou pensar que all cancela tarefas.

Pistas graduais

1. Identifique critério: todos, parcial, primeiro settle ou primeiro sucesso. 2. Especifique comportamento de rejeição. 3. No timeout, planeje cleanup e cancelamento.

Solução de referência explicada

Matriz de escolha com justificativa e casos de falha; implemente exemplo pequeno de each combinator e compare saídas.

Transferência em outro contexto

Localize o primeiro pacote de ícones de tema disponível entre cache local, pacote instalado e catálogo remoto; verifique o caso de sucesso e o resultado quando todas as fontes falharem.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; AggregateError (ES2021, Node.js 15+); o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.09 Normalizar callbacks com Promise.try()

COMPARE · TRILHA OPCIONAL · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: async.promise_try | Requer: functions.return | Introduz: async.promise_try

Eixos: novidade 5 · sintaxe 4 · composição 3 · estado 3 · depuração 2 · arquivos 1 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Normalize um callback que pode retornar valor, Promise ou lançar erro. Compare Promise.try com fallback fiel e registre quando o callback é chamado.

Objetivo da atividade

Registre se fn roda na pilha atual ou em uma microtask para as duas abordagens antes de implementar feature detection.

Explicação e modelo mental

• Modelo mental: Promise.try chama callback sincronamente. Promise.resolve().then(fn) chama callback em microtask; não é igual. Fallback fiel usa try/catch ao chamar fn e converte retorno/erro em Promise.
• Exemplo resolvido: Um plugin pode retornar string simples e ser normalizado para Promise cumprida; se lançar durante a chamada, o wrapper transforma isso em rejeição em vez de deixar escapar sincronamente.
• Armadilha comum: Executar callback fora de try e deixar throw escapar síncrono, ou dizer que .then(fn) tem mesma ordem.
• Por que importa: Normalize um hook de plugin que pode devolver valor ou Promise preservando timing declarado e caminho de rejeição.

Material/arquivo inicial

material.md: Callback retorna valor, Promise ou lança. Compare:
Promise.try(callback)
try { Promise.resolve(callback()) } catch (erro) { Promise.reject(erro) }
Promise.resolve().then(callback)

Registre para cada forma quando o callback roda e como um throw síncrono aparece. Promise.try depende do runtime-alvo; não presuma disponibilidade universal.

Exemplo e resultado público

Promise.try(()=>3).then → 3; Promise.try(()=>Promise.resolve(4)) -> 4

Bordas e erros publicados

callback lança Error -> promise rejeita; fallback try{return Promise.resolve(fn())}catch(e){return Promise.reject(e)} preserva chamada síncrona

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/try

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Para que serve uma verificação de capacidade de runtime antes de usar um método recente?

• Alterar a versão instalada do runtime durante a execução.
 (Correta) Escolher o caminho compatível com a API disponível naquele runtime.
• Fazer o compilador adicionar a implementação ausente ao objeto global.

Por que a resposta está certa

Correto. Um plugin pode retornar string simples e ser normalizado para Promise cumprida; se lançar durante a chamada, o wrapper transforma isso em rejeição em vez de deixar escapar sincronamente.

Armadilha da alternativa incorreta

Revise esta regra: Executar callback fora de try e deixar throw escapar síncrono, ou dizer que .then(fn) tem mesma ordem. Um plugin pode retornar string simples e ser normalizado para Promise cumprida; se lançar durante a chamada, o wrapper transforma isso em rejeição em vez de deixar escapar sincronamente.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

callback lança Error → promise rejeita; fallback try{return Promise.resolve(fn())}catch(e){return Promise.reject(e)} preserva chamada síncrona

Concepção errada e falha observável

Executar callback fora de try e deixar throw escapar síncrono, ou dizer que .then(fn) tem mesma ordem. / O callback roda em microtask no fallback ou seu throw síncrono escapa como throw síncrono.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Executar callback fora de try e deixar throw escapar síncrono, ou dizer que .then(fn) tem mesma ordem.

Pistas graduais

1. Promise.try chama callback sincronamente. 2. Promise.resolve().then(fn) chama callback em microtask; não é igual. 3. Fallback fiel usa try/catch ao chamar fn e converte retorno/erro em Promise.

Solução de referência explicada

Use Promise.try(fn) sob feature detection; fallback try{return Promise.resolve(fn())}catch(e){return Promise.reject(e)}; declare diferença de criação de Promise e baseline.

Transferência em outro contexto

Normalize um hook de plugin que pode devolver valor ou Promise preservando timing declarado e caminho de rejeição.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; Promise.try (ES2025, Node.js 23+) com o fallback descrito pelo contrato da atividade. | Próxima sugestão: 12.12

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.12 Tempo, expiração e fuso

MODIFY · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: date.ttl_timezone | Requer: functions.return | Introduz: date.ttl_timezone

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Separe cálculo de frescor da formatação de exibição. Valide timestamp e TTL e use a condição estrita nowMs - savedMs < ttl; na igualdade, o cache está expirado. Converta e compare os instantes antes de formatar a saída para o fuso escolhido.

Objetivo da atividade

Escreva a expressão de tempo decorrido e a regra de limite estrito antes de escolher um formato de exibição.

Explicação e modelo mental

• Modelo mental: Guarde instante em ISO UTC. Converta para epoch ms e compare diferença com ttl. Intl formata só na interface com timeZone declarado.
• Exemplo resolvido: O mesmo instante salvo em Lisboa e exibido em Tóquio continua sendo um único valor de epoch; a conversão visual não altera tempo decorrido.
• Armadilha comum: Comparar strings locais ou exibição formatada para expiração.
• Por que importa: Implemente expiração de token com relógio UTC e exibição no fuso America/Sao_Paulo, preservando a identidade do instante.

Material/arquivo inicial

dates.js: export function parseIsoDate(text) { return Date.parse(text); }
export function isFresh(savedMs, nowMs, ttlMs) { return nowMs - savedMs <= ttlMs; }

// BUG: datas inválidas e TTL negativo não são tratados; o limite deveria ser exclusivo.

Exemplo e resultado público

savedAt 2026-01-01T00:00:00.000Z, now +59s, ttl 60s → fresh=true; ISO inválido -> stale; ttl negativo -> RangeError; now +60s, ttl 60s -> fresh=false (nowMs - savedMs < ttl)

Bordas e erros publicados

fuso de apresentação alterado -> mesmo instante e frescor em epoch ms

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/parse

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que validar a conversão de um timestamp antes de usar seu resultado em aritmética de tempo?

• A validação transforma automaticamente qualquer string em horário UTC correto.
• O timestamp convertido deixa de depender da unidade usada pelo relógio.
 (Correta) Uma data inválida pode produzir um valor numérico não finito e contaminar comparações posteriores.

Por que a resposta está certa

Correto. O mesmo instante salvo em Lisboa e exibido em Tóquio continua sendo um único valor de epoch; a conversão visual não altera tempo decorrido.

Armadilha da alternativa incorreta

Revise esta regra: Comparar strings locais ou exibição formatada para expiração. O mesmo instante salvo em Lisboa e exibido em Tóquio continua sendo um único valor de epoch; a conversão visual não altera tempo decorrido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

fuso de apresentação alterado → mesmo instante e frescor em epoch ms

Concepção errada e falha observável

Comparar strings locais ou exibição formatada para expiração. / Data inválida/limite do TTL é classificado como fresco ou conversão de fuso muda o instante comparado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Comparar strings locais ou exibição formatada para expiração.

Pistas graduais

1. Guarde instante em ISO UTC. 2. Converta para epoch ms e compare diferença com ttl. 3. Intl formata só na interface com timeZone declarado.

Solução de referência explicada

isFresh valida timestamps e ttl, usa Date.parse e nowMs-savedMs < ttl; formatação Intl separada do cálculo.

Transferência em outro contexto

Implemente expiração de token com relógio UTC e exibição no fuso America/Sao_Paulo, preservando a identidade do instante.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.13

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.13 Iteração assíncrona antes de materializar

COMPARE · TRILHA OPCIONAL · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: async.iteration | Requer: functions.return | Introduz: async.iteration

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Compare iteração async incremental com materialização completa. Marque quando cada valor fica disponível e quando a coleção completa pode ser usada.

Objetivo da atividade

Marque o momento de processar o primeiro item e o de finalizar a fonte antes de escolher uma estratégia.

Explicação e modelo mental

• Modelo mental: async generator implementa protocolo async iterator. for await entrega item por item. Array.fromAsync só resolve com coleção completa.
• Exemplo resolvido: Uma fonte async de registros pode deixar o loop resumir o primeiro item enquanto o segundo ainda carrega; uma coleção completa só é retornada quando a iteração termina.
• Armadilha comum: Achar que materializar mantém processamento incremental para o consumidor.
• Por que importa: Escolha streaming ou coleção completa para páginas de API quando memória e tempo até primeiro resultado têm custos diferentes.

Material/arquivo inicial

stream.js: export async function* valores() { yield 1; await Promise.resolve(); yield 2; }
export async function consumir(informar) { const vistos=[]; for await (const valor of valores()) { vistos.push(valor); informar([...vistos]); } return vistos; }

Exemplo e resultado público

for await...of → processa 1 e depois 2; Array.fromAsync(pagina()) -> [1,2] após fim

Bordas e erros publicados

fonte com dois itens -> o loop registra cada valor assim que chega; Array.fromAsync resolve a coleção completa somente depois do fim

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for-await...of

Conceitos prévios (Primers)

async function* produz valores um de cada vez e pode usar await entre eles; for await (const item of fonte) consome esses valores esperando cada um. Generators são estudados na etapa 20.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual protocolo um objeto precisa expor para ser consumido diretamente por for await...of?

 (Correta) O protocolo de iterável assíncrono, normalmente fornecido por Symbol.asyncIterator.
• Somente um método toJSON que produza um array.
• Uma propriedade length numérica e índices consecutivos.

Por que a resposta está certa

Correto. Uma fonte async de registros pode deixar o loop resumir o primeiro item enquanto o segundo ainda carrega; uma coleção completa só é retornada quando a iteração termina.

Armadilha da alternativa incorreta

Revise esta regra: Achar que materializar mantém processamento incremental para o consumidor. Uma fonte async de registros pode deixar o loop resumir o primeiro item enquanto o segundo ainda carrega; uma coleção completa só é retornada quando a iteração termina.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

fonte com dois itens → o loop registra cada valor assim que chega; Array.fromAsync resolve a coleção completa somente depois do fim

Concepção errada e falha observável

Achar que materializar mantém processamento incremental para o consumidor. / Valores são materializados antes do consumo ou erro parcial apaga valores já recebidos sem política.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que materializar mantém processamento incremental para o consumidor.

Pistas graduais

1. async generator implementa protocolo async iterator. 2. for await entrega item por item. 3. Array.fromAsync só resolve com coleção completa.

Solução de referência explicada

Compare log temporal de for await com Array.fromAsync quando suportado; fallback acumula array manualmente com for await.

Transferência em outro contexto

Escolha streaming ou coleção completa para páginas de API quando memória e tempo até primeiro resultado têm custos diferentes.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; Array.fromAsync (ES2026, Node.js 22+); o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.14

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.14 Falha agregada

PREDICT · 6 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: async.aggregate_error | Requer: functions.return | Introduz: async.aggregate_error

Eixos: novidade 2 · sintaxe 3 · composição 2 · estado 3 · depuração 1 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Preveja como Promise.any termina conforme cada entrada cumpre ou rejeita. Inspecione AggregateError apenas depois que todas falharem.

Objetivo da atividade

Monte uma tabela de settlement para cada entrada e só pare no evento que determina o resultado de any.

Explicação e modelo mental

• Modelo mental: any procura primeira satisfação. Rejeições são coletadas até todas falharem. Inspecione .errors e preserve causas originais.
• Exemplo resolvido: Uma página tenta duas fontes de fonte tipográfica; a primeira que cumpre permite renderizar, mesmo se uma falha anterior chegou antes.
• Armadilha comum: Tratar primeira rejeição como falha final de any.
• Por que importa: Modele uma busca em réplicas e relate falhas somente se nenhuma delas cumprir.

Material/arquivo inicial

main.js: export async function primeiroSucesso(promessas) { try { return {ok:true,valor:await Promise.any(promessas)}; } catch(erro) { return {ok:false,nome:erro.name,erros:erro.errors.map(item=>item.message)}; } }

Exemplo e resultado público

todas rejeitam → AggregateError com errors ['A','B'] na ordem de entradas

Bordas e erros publicados

uma resolve tardiamente -> sucesso, mesmo com rejeição anterior; array vazio -> AggregateError com errors vazio

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como Promise.any ordena os motivos no campo errors de um AggregateError?

• Na ordem temporal em que as rejeições chegaram.
 (Correta) Na ordem das entradas fornecidas, não na ordem em que as rejeições ocorreram.
• Em ordem alfabética pelo texto de cada motivo.

Por que a resposta está certa

Correto. Uma página tenta duas fontes de fonte tipográfica; a primeira que cumpre permite renderizar, mesmo se uma falha anterior chegou antes.

Armadilha da alternativa incorreta

Revise esta regra: Tratar primeira rejeição como falha final de any. Uma página tenta duas fontes de fonte tipográfica; a primeira que cumpre permite renderizar, mesmo se uma falha anterior chegou antes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

uma resolve tardiamente → sucesso, mesmo com rejeição anterior; array vazio -> AggregateError com errors vazio

Concepção errada e falha observável

Tratar primeira rejeição como falha final de any. / Promise.any rejeita na primeira entrada recusada em vez de aguardar o resultado agregado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar primeira rejeição como falha final de any.

Pistas graduais

1. any procura primeira satisfação. 2. Rejeições são coletadas até todas falharem. 3. Inspecione .errors e preserve causas originais.

Solução de referência explicada

try/await Promise.any; catch verifica AggregateError e mapeia errors sem apagar causas.

Transferência em outro contexto

Modele uma busca em réplicas e relate falhas somente se nenhuma delas cumprir.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline Node.js 24 LTS; AggregateError (ES2021, Node.js 15+); o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 12.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

12.08 Microprojeto - dashboard de API

PROJECT · 90 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 4.5/5

Competência: fetch.dashboard | Requer: functions.return | Introduz: fetch.dashboard

Eixos: novidade 3 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 5 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Complete o dashboard somente com fixtures injetadas. Trate status HTTP separado de rejeição, valide JSON, permita retry e ignore atualizações obsoletas.

Objetivo da atividade

Mapeie transporte, status, parse do corpo, validação da forma e renderização; associe uma falha possível a cada etapa.

Explicação e modelo mental

• Modelo mental: Cheque response.ok e Content-Type. Leia corpo uma vez e valide forma mínima. Identificador de geração impede race e try/finally fecha loading.
• Exemplo resolvido: Uma fixture local pode cumprir com status 503 e corpo JSON válido. A Promise de rede foi cumprida, mas o dashboard precisa classificar a resposta HTTP como malsucedida.
• Armadilha comum: Assumir que fetch rejeita em status HTTP de erro.
• Por que importa: Monte painel de clima para duas fontes com fixtures determinísticas, loading/erro/retry e proteção contra resposta obsoleta.

Material/arquivo inicial

dashboard.js: export function criarDashboard(fetcher) {
// fetcher('/profile') e fetcher('/stats') retornam Response de fixtures locais.
// TODO: modelar loading/error/success, validar status e JSON e ignorar resposta antiga.
}

Exemplo e resultado público

duas fixtures 200 com JSON válido → sucesso com dois painéis; retry após fixture de erro -> nova tentativa

Bordas e erros publicados

fixture HTTP 404 -> erro mesmo com Promise resolvida; JSON inválido -> erro; fixture antiga lenta -> não sobrescreve nova; loading finaliza; nenhuma requisição externa

Teste público e aprovação visível

Critérios desta atividade: Entrega 1 — API aceita fetcher injetado para /profile e /stats e valida response.ok, JSON e campos mínimos antes de atualizar a tela. Entrega 2 — estados loading, error e success são observáveis; erro HTTP/JSON preserva a mensagem e retry inicia uma nova geração. Entrega 3 — timeout/cancelamento encerra a espera e limpa o indicador; resposta antiga que termina por último não substitui a atual. Entrega 4 — checklist registra sucesso, 404 cumprido, JSON inválido, retry, timeout e resposta fora de ordem; reflexão anota o comando e confirma que não houve rede real.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual benefício principal um fetcher injetado oferece a um dashboard testado localmente?

• Garante que toda resposta de rede tenha o formato esperado pela interface.
• Faz o navegador armazenar respostas em cache sem configuração adicional.
 (Correta) Permite substituir a fronteira de rede por uma implementação controlada nos testes.

Por que a resposta está certa

Correto. Uma fixture local pode cumprir com status 503 e corpo JSON válido. A Promise de rede foi cumprida, mas o dashboard precisa classificar a resposta HTTP como malsucedida.

Armadilha da alternativa incorreta

Revise esta regra: Assumir que fetch rejeita em status HTTP de erro. Uma fixture local pode cumprir com status 503 e corpo JSON válido. A Promise de rede foi cumprida, mas o dashboard precisa classificar a resposta HTTP como malsucedida.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

fixture HTTP 404 → erro mesmo com Promise resolvida; JSON inválido -> erro; fixture antiga lenta -> não sobrescreve nova; loading finaliza; nenhuma requisição externa

Concepção errada e falha observável

Assumir que fetch rejeita em status HTTP de erro. / HTTP 404 é marcado como sucesso, JSON inválido escapa ou resposta antiga altera a tela atual.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Assumir que fetch rejeita em status HTTP de erro.

Pistas graduais

1. Cheque response.ok e Content-Type. 2. Leia corpo uma vez e valide forma mínima. 3. Identificador de geração impede race e try/finally fecha loading.

Solução de referência explicada

Camada api verifica HTTP e body; controlador inicia requisições independentes, atualiza estados, permite retry e descarta resposta obsoleta. Todas as respostas vêm de fixtures injetadas; não dependa da rede.

Transferência em outro contexto

Monte painel de clima para duas fontes com fixtures determinísticas, loading/erro/retry e proteção contra resposta obsoleta.

Evidência de domínio

Critérios desta atividade: Entrega 1 — API aceita fetcher injetado para /profile e /stats e valida response.ok, JSON e campos mínimos antes de atualizar a tela. Entrega 2 — estados loading, error e success são observáveis; erro HTTP/JSON preserva a mensagem e retry inicia uma nova geração. Entrega 3 — timeout/cancelamento encerra a espera e limpa o indicador; resposta antiga que termina por último não substitui a atual. Entrega 4 — checklist registra sucesso, 404 cumprido, JSON inválido, retry, timeout e resposta fora de ordem; reflexão anota o comando e confirma que não houve rede real. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline browser; o comportamento de cada API está delimitado pelo contrato desta atividade. | Próxima sugestão: 13.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13 Teste, debugger e manutenção do Kanban

14 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 13.01 · 13.02 · 13.03 · 13.04 · 13.05 · 13.06 · 13.07 · 13.08 · 13.09 · 13.10 · 13.11 · 13.12 · 13.13 · 13.14

13.01 Leia uma falha de teste

READ · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 2.0/5

Competência: testing.failure_reading | Requer: functions.return | Introduz: testing.failure_reading

Eixos: novidade 2 · sintaxe 1 · composição 2 · estado 1 · depuração 2 · arquivos 1 · leitura 4 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Leia o diff do teste sem declarar causa antes da hora. Separe esperado, recebido e hipótese; peça uma reprodução mínima que possa refutar a hipótese.

Starter de trabalho:
Vitest: expect(paginar([1,2,3],2,2)).toEqual([3]); Received: [2,3]

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Escreva esperado/recebido e formule uma hipótese como pergunta que um caso pequeno possa confirmar ou refutar.

Explicação e modelo mental

• Modelo mental: Uma falha de teste é uma observação que restringe hipóteses; a causa precisa de evidência adicional antes de ser afirmada.
• Exemplo resolvido: Um teste de contagem esperava quatro registros e recebeu cinco. A diferença revela uma observação e sugere duplicação, mas um recorte menor ainda precisa confirmar a causa.
• Armadilha comum: Armadilha: Interpretar diff como prova definitiva do bug. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Ler falha de ordenação com item extra. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: Vitest: expect(paginar([1,2,3],2,2)).toEqual([3]).
Recebido: [2,3].
A divergência localiza comportamento diferente; não prova a causa. Escreva uma hipótese refutável e a menor reprodução que separaria erro de índice de erro de normalização.

Exemplo e resultado público

esperado → [3]; recebido -> [2,3]; hipótese: offset da página começa cedo

Bordas e erros publicados

hipótese não prova causa -> pedir menor reprodução e inspecionar fórmula de início

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

https://vitest.dev/guide/learn/writing-tests

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que repetir uma falha antes de começar a alterá-la?

 (Correta) Para confirmar que o comportamento é reproduzível e separar um defeito consistente de uma ocorrência transitória.
• Para provar que a primeira hipótese sobre a causa está correta.
• Para dispensar a leitura do teste e do contexto da implementação.

Por que a resposta está certa

Correto. Reproduzir a observação fortalece a investigação, mas não determina sozinho a causa.

Armadilha da alternativa incorreta

Uma reprodução confirma o fenômeno; a explicação ainda precisa de evidência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

hipótese não prova causa → pedir menor reprodução e inspecionar fórmula de início

Concepção errada e falha observável

Interpretar diff como prova definitiva do bug. / A resposta afirma causa apenas pela diferença entre esperado e recebido, sem reduzir e inspecionar a expressão.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Interpretar diff como prova definitiva do bug.

Pistas graduais

1. Separe esperado de recebido. 2. Ache primeira diferença observável. 3. Formule hipótese testável sem declarar certeza.

Solução de referência explicada

Registre entrada/esperado/recebido e hipótese de offset; verifique com caso página 1 antes de corrigir.

Transferência em outro contexto

Analise uma falha de ordenação com elemento extra e registre hipótese mais contraexemplo mínimo antes de editar.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.02 Parsons de validação

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 1.5/5

Competência: errors.validation_order | Requer: functions.return | Introduz: errors.validation_order

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 2 · autonomia 2 · ferramentas 1

VISÃO DO ALUNO

Ordene as guardas para rejeitar entradas inválidas antes do cálculo. Escolha tipo de erro estável e indique por que a operação só vem depois.

Starter de trabalho:
blocos: validar número finito, validar divisor, calcular a/b, return, throw TypeError/RangeError

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Agrupe as pré-condições pelo tipo de erro e coloque todas antes da primeira operação aritmética.

Explicação e modelo mental

• Modelo mental: Pré-condições definem quando uma operação pertence ao domínio válido; erros previsíveis dependem de respeitar essa fronteira.
• Exemplo resolvido: Ao calcular percentual concluído, valide que o total de tarefas é positivo antes de dividir a quantidade concluída pelo total.
• Armadilha comum: Armadilha: Calcular primeiro e validar depois, deixando NaN/Infinity escapar. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Validador de média com array vazio. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

main.js: export function dividir(dividendo,divisor) { return dividendo/divisor; }

Exemplo e resultado público

dividir(6,2) → 3

Bordas e erros publicados

dividir('6',2) -> TypeError; dividir(6,0) -> RangeError; nenhum cálculo antes de validar

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RangeError

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que uma anotação de tipo TypeScript não substitui validação na fronteira de dados?

• Toda anotação de tipo executa uma conversão obrigatória em runtime.
 (Correta) Tipos estáticos não verificam automaticamente valores externos recebidos durante a execução.
• Valores externos são sempre confiáveis depois de passar pelo parser JSON.

Por que a resposta está certa

Correto. Dados externos precisam de verificações executadas antes de serem tratados como válidos.

Armadilha da alternativa incorreta

TypeScript orienta o typecheck; ele não inspeciona automaticamente os valores em execução.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

dividir('6',2) → TypeError; dividir(6,0) -> RangeError; nenhum cálculo antes de validar

Concepção errada e falha observável

Calcular primeiro e validar depois, deixando NaN/Infinity escapar. / Divisão ocorre antes da validação; string/Infinity e divisor zero não produzem os erros publicados.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Calcular primeiro e validar depois, deixando NaN/Infinity escapar.

Pistas graduais

1. Cheque tipos e finitude. 2. Cheque divisor zero. 3. Só então calcule e retorne.

Solução de referência explicada

Ordene guardas antes de a/b e escolha erros estáveis: TypeError para tipo/finito, RangeError para divisor zero.

Transferência em outro contexto

Defina o tratamento de array vazio num cálculo de média, lançando o erro previsto antes de dividir.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.03 Complete a suíte

TEST · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 2.5/5

Competência: testing.boundary_cases | Requer: functions.return | Introduz: testing.boundary_cases

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Amplie a suíte de paginação com limites, página parcial, além do fim, valores inválidos e garantia de não mutação; mantenha intacta a implementação fornecida.

Starter de trabalho:
paginar(items,page,size) 1-based; testes existentes só page=1,size=2

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Liste condições de page e size, depois cubra a última janela parcial e o caso além do final.

Explicação e modelo mental

• Modelo mental: Contratos de funções incluem limites, entradas inválidas, efeitos sobre argumentos e o comportamento após o intervalo comum.
• Exemplo resolvido: Um leitor de lotes processa cinco códigos em grupos de três: o último grupo é parcial e pedir outro grupo não deve alterar a lista fonte.
• Armadilha comum: Armadilha: Testar somente caminho feliz e confundir página 1 com offset 1. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Completar testes de fatiamento por janela. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

pagination.test.js: import { paginar } from './pagination.js';

```
describe('paginar', () => {
it('retorna a primeira página', () => {
expect(paginar([1, 2, 3], 1, 2)).toEqual([1, 2]);
});
});

// Complete a suíte para contratos de borda; page e size são inteiros 1-based.
```

Exemplo e resultado público

[1,2,3], page=2,size=2 → [3]

Bordas e erros publicados

page=0/size=0 -> RangeError; além do fim -> []; entrada não mutada; size não inteiro -> RangeError

Teste público e aprovação visível

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual vantagem testes parametrizados oferecem ao cobrir diferentes casos de uma mesma regra?

• Fazem cada caso herdar automaticamente o resultado esperado do primeiro.
• Eliminam a necessidade de observar o comportamento da implementação.
 (Correta) Reutilizam a mesma estrutura de verificação e deixam explícitas as variações dos dados de entrada.

Por que a resposta está certa

Correto. Uma tabela de casos reduz repetição sem esconder quais entradas estão sendo exercitadas.

Armadilha da alternativa incorreta

Parametrizar compartilha o formato do teste, mas cada caso ainda precisa de dados e expectativa próprios.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

page=0/size=0 → RangeError; além do fim -> []; entrada não mutada; size não inteiro -> RangeError

Concepção errada e falha observável

Testar somente caminho feliz e confundir página 1 com offset 1. / Página/tamanho inválido não falha ou última página, além do fim e não mutação divergem do contrato.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Testar somente caminho feliz e confundir página 1 com offset 1.

Pistas graduais

1. Liste limites de page/size. 2. Cubra última página parcial e além do fim. 3. Cheque erros e não mutação da entrada.

Solução de referência explicada

Adicione asserts para segunda/última página, lista vazia, inválidos, além do fim e referência/ordem da entrada; não altere implementação fornecida.

Transferência em outro contexto

Complete testes para janelas de fatiamento, cobrindo limites inválidos, última janela e preservação do array de entrada.

Evidência de domínio

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.04 Prever-executar-debugger

INVESTIGATE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 2.5/5

Competência: debugging.predict_run_step | Requer: functions.return | Introduz: debugging.predict_run_step

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 3 · depuração 4 · arquivos 1 · leitura 3 · autonomia 2 · ferramentas 4

VISÃO DO ALUNO

Siga o protocolo prever→executar→inspecionar em cada iteração. Registre a primeira divergência e qual evidência do debugger resolveu sua dúvida.

Starter de trabalho:
função somaPositivos([1,-2,3]) com loop e acumulador; prever, executar, breakpoint a cada iteração

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Escreva resultado previsto e confiança, depois escolha a variável e o ponto de parada que tornariam cada passo observável.

Explicação e modelo mental

• Modelo mental: Previsão e execução são fontes diferentes de conhecimento: uma revela expectativa, outra fornece evidência sobre o estado percorrido.
• Exemplo resolvido: Ao procurar a primeira letra “r” em “arara”, marque índice e posição encontrada a cada iteração; o traço mostra quando a condição encerra o laço.
• Armadilha comum: Armadilha: Rodar antes de prever e perder diagnóstico do modelo mental. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Mesmo protocolo para buscar máximo. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: Protocolo: preveja os valores de total ao entrar e depois de cada iteração para [1,-2,3]; execute; avance o debugger e registre a primeira divergência entre previsão e valor observado. Explique qual evidência do watch/breakpoint resolveu a dúvida.

Exemplo e resultado público

previsão registrada → 4; execução -> 4; passos mostram acc 0,1,1,4

Bordas e erros publicados

entrada [0,-1] -> 0; registrar se previsão divergiu e qual passo resolveu dúvida

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

https://developer.chrome.com/docs/devtools/javascript/

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que deve ser registrado antes de iniciar o debugger para diagnosticar o modelo mental?

 (Correta) Uma previsão com confiança; depois ela pode ser comparada com o traço de cada iteração.
• Só o valor final observado após executar várias entradas.
• A correção esperada copiada antes de registrar qualquer hipótese.

Por que a resposta está certa

Correto: Uma previsão com confiança; depois ela pode ser comparada com o traço de cada iteração. Previsão e execução são fontes diferentes de conhecimento: uma revela expectativa, outra fornece evidência sobre o estado percorrido.

Armadilha da alternativa incorreta

Reveja o conceito: Previsão e execução são fontes diferentes de conhecimento: uma revela expectativa, outra fornece evidência sobre o estado percorrido. A resposta conceitual esperada é: Uma previsão com confiança; depois ela pode ser comparada com o traço de cada iteração.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada [0,-1] → 0; registrar se previsão divergiu e qual passo resolveu dúvida

Concepção errada e falha observável

Rodar antes de prever e perder diagnóstico do modelo mental. / Total previsto deixa de acompanhar o acumulador em uma iteração sem que a primeira divergência seja localizada.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Rodar antes de prever e perder diagnóstico do modelo mental.

Pistas graduais

1. Escreva previsão com confiança. 2. Execute com entrada fixa. 3. Inspecione acc e índice a cada parada.

Solução de referência explicada

Entregue tabela previsão/resultado/traço e explique qual ferramenta forneceu evidência em cada fase.

Transferência em outro contexto

Aplique o mesmo protocolo de previsão/execução a uma rotina de busca do maior valor e registre a primeira divergência.

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.05 Bug fantasma

DEBUG · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 3.5/5

Competência: debugging.minimal_repro | Requer: functions.return | Introduz: debugging.minimal_repro

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 3 · depuração 5 · arquivos 3 · leitura 3 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Reduza o módulo até a falha caber num caso mínimo. Confirme que o defeito persiste sem UI/rede, faça uma correção pequena e preserve a reprodução como regressão.

Starter de trabalho:
módulo 40 linhas com filtros, formatação e bug: item zero desaparece por if(valor); reduzir

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Remova uma responsabilidade de cada vez e execute a reprodução após cada redução; pare quando o menor caso ainda falhar.

Explicação e modelo mental

• Modelo mental: Um defeito pode depender de várias camadas; reduzir contexto ajuda a separar a condição causal do cenário incidental.
• Exemplo resolvido: Um formatador de datas pode deslocar um registro para o dia anterior. Remover rede e interface e reduzir a duas datas próximas à meia-noite revela se a conversão é a causa.
• Armadilha comum: Armadilha: Corrigir código grande sem isolar uma entrada que falha. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Reduzir bug de preço zero em carrinho. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

filtro.js: export function manterDefinidos(valores) {
const resultado = [];
for (const valor of valores) {
if (valor) resultado.push(valor); // BUG: remove valores válidos falsy, como 0 e false.
}
return resultado;
}

// Contrato: remover somente undefined; preservar 0, false, null, strings e sua ordem.

Exemplo e resultado público

caso mínimo [0,1] → deve preservar 0; versão bug devolve [1]

Bordas e erros publicados

remover UI/rede -> falha persiste; caso [false] também denuncia teste de truthiness conforme contrato

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

https://nodejs.org/en/learn/getting-started/debugging

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o propósito principal de uma reprodução mínima?

• Refatorar todo o módulo antes de confirmar que o defeito continua.
 (Correta) Isolar uma entrada que ainda demonstra a falha removendo partes irrelevantes.
• Apagar os testes e manter apenas a saída que parece correta.

Por que a resposta está certa

Correto: Isolar uma entrada que ainda demonstra a falha removendo partes irrelevantes. Um defeito pode depender de várias camadas; reduzir contexto ajuda a separar a condição causal do cenário incidental.

Armadilha da alternativa incorreta

Reveja o conceito: Um defeito pode depender de várias camadas; reduzir contexto ajuda a separar a condição causal do cenário incidental. A resposta conceitual esperada é: Isolar uma entrada que ainda demonstra a falha removendo partes irrelevantes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

remover UI/rede → falha persiste; caso [false] também denuncia teste de truthiness conforme contrato

Concepção errada e falha observável

Corrigir código grande sem isolar uma entrada que falha. / 0/false/null são descartados ou undefined permanece, reproduzindo o filtro por truthiness.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Corrigir código grande sem isolar uma entrada que falha.

Pistas graduais

1. Liste entradas/saídas observadas. 2. Remova partes irrelevantes conservando falha. 3. Só depois ajuste condição e guarde regressão.

Solução de referência explicada

Reduza a função a filtro com if(valor !== undefined) conforme contrato; teste 0 e mantenha diff mínimo.

Transferência em outro contexto

Reduza um defeito em que preço zero desaparece no carrinho a uma reprodução pequena antes de corrigir.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.06 TDD de parser

PROJECT · 90 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 3.0/5

Competência: testing.tdd_parser | Requer: functions.return | Introduz: testing.tdd_parser

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 3 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente parseLinha em incrementos guiados pelos lotes de testes. Faça cada lote falhar primeiro, passe com a menor mudança e refatore mantendo os anteriores verdes.

Starter de trabalho:
parseLinha('nome=Ana;idade=3') com lotes de testes progressivos: formato, escape, erro

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Leia o próximo teste antes de escolher a menor regra que o faz passar; só avance quando os casos anteriores continuarem passando.

Explicação e modelo mental

• Modelo mental: Desenvolvimento guiado por testes é uma sequência de hipóteses pequenas que evoluem junto com a compreensão do formato aceito.
• Exemplo resolvido: Um parser de pares separados por vírgula pode começar aceitando uma chave e valor, depois adicionar repetição conforme a política documentada; não precisa antecipar escapes ainda não pedidos.
• Armadilha comum: Armadilha: Implementar formato completo imaginado antes de ler o próximo teste. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Parser de cabeçalho simples com separador diferente. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

parser.js: export function parseLinha(texto) {
// Formato: chave=valor;chave=valor. \; representa ; literal e \\ representa \ literal.
// Chave repetida: último valor vence. Segmento sem '=' ou escape incompleto: SyntaxError.
// Entrada vazia retorna {}.
}

Exemplo e resultado público

'nome=Ana;idade=3' → {nome:'Ana',idade:'3'}

Bordas e erros publicados

chave repetida -> último valor; \; decodifica como ; e \\ como \; segmento sem '=' ou barra invertida final -> SyntaxError; entrada vazia -> {}

Teste público e aprovação visível

Resultado observável: 'nome=Ana;idade=3' -> {nome:'Ana',idade:'3'}. Borda a conferir: chave repetida -> último valor; \; decodifica como ; e \\ como \; segmento sem '=' ou barra invertida final -> SyntaxError; entrada vazia -> {}. Entrega esperada: Parser incremental separa segmentos não escapados, valida chave/igual, aplica política de repetição; registre ciclos vermelho/verde/refatoração.

Restrições e documentação

https://vitest.dev/guide/

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual decisão combina com uma evolução TDD por lotes?

• Antecipar todos os formatos imagináveis antes de ler os próximos testes.
• Remover os testes antigos quando um novo formato aparece.
 (Correta) Implementar o menor comportamento que passa o lote atual e preservar os testes anteriores.

Por que a resposta está certa

Correto: Implementar o menor comportamento que passa o lote atual e preservar os testes anteriores. Desenvolvimento guiado por testes é uma sequência de hipóteses pequenas que evoluem junto com a compreensão do formato aceito.

Armadilha da alternativa incorreta

Reveja o conceito: Desenvolvimento guiado por testes é uma sequência de hipóteses pequenas que evoluem junto com a compreensão do formato aceito. A resposta conceitual esperada é: Implementar o menor comportamento que passa o lote atual e preservar os testes anteriores.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

chave repetida → último valor; \; decodifica como ; e \\ como \; segmento sem '=' ou barra invertida final -> SyntaxError; entrada vazia -> {}

Concepção errada e falha observável

Implementar formato completo imaginado antes de ler o próximo teste. / Separador escapado divide o valor ou chave repetida/escape incompleto diverge da gramática publicada.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Implementar formato completo imaginado antes de ler o próximo teste.

Pistas graduais

1. Faça primeiro teste falhar. 2. Implemente menor passo que passa o lote. 3. Refatore sem perder testes anteriores.

Solução de referência explicada

Parser incremental separa segmentos não escapados, valida chave/igual, aplica política de repetição; registre ciclos vermelho/verde/refatoração.

Transferência em outro contexto

Implemente por ciclos TDD um parser de cabeçalho simples com outro separador, lendo cada teste antes de ampliar a gramática.

Evidência de domínio

Resultado observável: 'nome=Ana;idade=3' -> {nome:'Ana',idade:'3'}. Borda a conferir: chave repetida -> último valor; \; decodifica como ; e \\ como \; segmento sem '=' ou barra invertida final -> SyntaxError; entrada vazia -> {}. Entrega esperada: Parser incremental separa segmentos não escapados, valida chave/igual, aplica política de repetição; registre ciclos vermelho/verde/refatoração. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.07 Teste de regressao

TEST · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 2.0/5

Competência: testing.regression | Requer: functions.return | Introduz: testing.regression

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 2 · arquivos 2 · leitura 2 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Demonstre a regressão com um teste que falha no código atual, corrija o limite inferior e preserve verificações do limite superior e dos extremos.

Starter de trabalho:
bug: clamp(-5,0,10) retorna -5; escrever teste Vitest antes de corrigir

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Registre primeiro a saída defeituosa e escreva o assert que a contradiz; só então faça a mudança de código.

Explicação e modelo mental

• Modelo mental: Um defeito corrigido pode reaparecer; uma reprodução automatizada transforma a descoberta em proteção duradoura.
• Exemplo resolvido: Um controle de volume limitado entre 2 e 8 deve retornar 8 quando recebe 11; esse teste é uma afirmação separada para a outra fronteira.
• Armadilha comum: Armadilha: Mudar código primeiro e só depois criar teste sem demonstrar regressão. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Bug de desconto acima do teto. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

clamp.js: export function clamp(valor, minimo, maximo) {
if (minimo > maximo) throw new RangeError('intervalo inválido');
return valor; // BUG: deveria limitar o valor aos extremos.
}

// Primeiro escreva um teste que falhe para clamp(-5, 0, 10); depois corrija. | tests/clamp.test.js: import {clamp} from '../clamp.js'; describe('clamp regression', () => { test('lower-bound regression', () => { expect(clamp(-5, 0, 10)).toBe(0); }); });

Exemplo e resultado público

clamp(-5,0,10) → 0; teste inicialmente falha e depois passa

Bordas e erros publicados

clamp(12,0,10) -> 10; min>max -> RangeError; min/max exatos preservados

Teste público e aprovação visível

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos.

Restrições e documentação

https://vitest.dev/guide/learn/writing-tests

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que faz do teste uma prova de regressão útil?

 (Correta) Ele reproduz o defeito antes da correção e permanece na suíte depois dela.
• Ele é escrito somente após corrigir, sem demonstrar que falhava.
• Ele testa apenas um caso já correto e apaga a saída defeituosa.

Por que a resposta está certa

Correto: Ele reproduz o defeito antes da correção e permanece na suíte depois dela. Um defeito corrigido pode reaparecer; uma reprodução automatizada transforma a descoberta em proteção duradoura.

Armadilha da alternativa incorreta

Reveja o conceito: Um defeito corrigido pode reaparecer; uma reprodução automatizada transforma a descoberta em proteção duradoura. A resposta conceitual esperada é: Ele reproduz o defeito antes da correção e permanece na suíte depois dela.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

clamp(12,0,10) → 10; min>max -> RangeError; min/max exatos preservados

Concepção errada e falha observável

Mudar código primeiro e só depois criar teste sem demonstrar regressão. / O caso regressivo não falha antes do fix ou clamp continua abaixo do mínimo/acima do máximo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Mudar código primeiro e só depois criar teste sem demonstrar regressão.

Pistas graduais

1. Registre saída defeituosa. 2. Escreva expect que falha no bug real. 3. Corrija e rode antiga+nova suíte.

Solução de referência explicada

Teste reproduz limite inferior, corrige com Math.min(Math.max(x,min),max) após validar intervalo e conserva teste.

Transferência em outro contexto

Proteja uma regra de desconto máximo com um teste que falha antes da correção e cobre as fronteiras relacionadas.

Evidência de domínio

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.08 Microprojeto - biblioteca confiável

PROJECT · 90 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 3.5/5

Competência: testing.library_contract | Requer: functions.return | Introduz: testing.library_contract

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Documente contratos de três funções e escreva testes para exemplos, bordas, erros e não mutação. Explique por que cobertura percentual não substitui semântica.

Starter de trabalho:
utilitários clamp, unique, paginar em módulo ESM; API documentada com erros

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Para cada export, escreva pré-condições, resultado, erro possível e se a entrada pode mudar antes de escolher testes.

Explicação e modelo mental

• Modelo mental: Uma API pública é definida por resultados, erros e efeitos observáveis, não apenas pela quantidade de linhas exercitadas.
• Exemplo resolvido: Uma função unique para rótulos ["x","x","y"] pode preservar a primeira ocorrência e a ordem; um teste deve afirmar ambos, não apenas contar linhas executadas.
• Armadilha comum: Armadilha: Cobertura percentual sem verificar semântica de bordas. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Módulo de moeda/intervalos com API. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

library.js: export function clamp(valor, minimo, maximo) {}
export function unique(valores) {}
export function paginar(valores, pagina, tamanho) {}

// API ESM pública; registre entrada, saída, erros, mutação e ordem para cada função.

Exemplo e resultado público

clamp(2,0,3), unique([1,1,2]), paginar([1,2,3],2,2) → 2,[1,2],[3]

Bordas e erros publicados

clamp min>max -> RangeError; unique preserva ordem e SameValueZero; paginar page inválida -> RangeError; sem mutação

Teste público e aprovação visível

Resultado observável: clamp(2,0,3), unique([1,1,2]), paginar([1,2,3],2,2) -> 2,[1,2],[3]. Borda a conferir: clamp min>max -> RangeError; unique preserva ordem e SameValueZero; paginar page inválida -> RangeError; sem mutação. Entrega esperada: Entregue três exports com JSDoc, Vitest cobrindo contratos e README de entradas/erros; revisão avalia cenários além da métrica.

Restrições e documentação

https://vitest.dev/guide/learn/writing-tests

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que uma alta cobertura percentual demonstra diretamente?

• Que todos os resultados, erros e efeitos da API tenham sido testados.
 (Correta) Que muitas das linhas medidas foram executadas, sem provar que todos os contratos foram verificados.
• Que o código coberto não contenha defeitos.

Por que a resposta está certa

Correto: Que muitas das linhas medidas foram executadas, sem provar que todos os contratos foram verificados. Uma API pública é definida por resultados, erros e efeitos observáveis, não apenas pela quantidade de linhas exercitadas.

Armadilha da alternativa incorreta

Reveja o conceito: Uma API pública é definida por resultados, erros e efeitos observáveis, não apenas pela quantidade de linhas exercitadas. A resposta conceitual esperada é: Que muitas das linhas medidas foram executadas, sem provar que todos os contratos foram verificados.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

clamp min>max → RangeError; unique preserva ordem e SameValueZero; paginar page inválida -> RangeError; sem mutação

Concepção errada e falha observável

Cobertura percentual sem verificar semântica de bordas. / A API passa exemplos comuns, mas viola erro, borda, ordenação ou não mutação explicitamente contratados.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Cobertura percentual sem verificar semântica de bordas.

Pistas graduais

1. Declare contratos públicos e erros. 2. Teste exemplo, borda, erro e não mutação por função. 3. Separe implementação de suíte e documentação curta.

Solução de referência explicada

Entregue três exports com JSDoc, Vitest cobrindo contratos e README de entradas/erros; revisão avalia cenários além da métrica.

Transferência em outro contexto

Crie contrato e suíte para um módulo de moeda e intervalos, cobrindo erros, bordas e invariantes.

Evidência de domínio

Resultado observável: clamp(2,0,3), unique([1,1,2]), paginar([1,2,3],2,2) -> 2,[1,2],[3]. Borda a conferir: clamp min>max -> RangeError; unique preserva ordem e SameValueZero; paginar page inválida -> RangeError; sem mutação. Entrega esperada: Entregue três exports com JSDoc, Vitest cobrindo contratos e README de entradas/erros; revisão avalia cenários além da métrica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.09 Dublês e relógio falso

TEST · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 3.0/5

Competência: testing.doubles_fake_time | Requer: functions.return | Introduz: testing.doubles_fake_time

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 4 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Controle a resposta assíncrona e o tempo sem rede real ou espera real. Verifique tentativas/atrasos e restaure spies e timers após cada teste.

Starter de trabalho:
retry(op,max,delay) com timer; testes vi.fn, vi.spyOn, vi.useFakeTimers; fake fetch injetado

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Identifique dependências externas de tempo/operação e escolha injeção ou relógio falso antes de montar os asserts.

Explicação e modelo mental

• Modelo mental: Dependências substituíveis e tempo controlado permitem observar trabalho assíncrono sem depender de serviços ou relógios variáveis.
• Exemplo resolvido: Um aviso de sucesso desaparece após alguns segundos; avançar o relógio virtual permite conferir que ele permanece antes do prazo e expira depois, sem aguardar tempo real.
• Armadilha comum: Armadilha: Usar rede real ou timer real e criar teste lento/instável. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Testar debounce sem esperar relógio real. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

retry.test.js: import { retry } from './retry.js';
import { vi, expect, it } from 'vitest';

it('retorna o resultado de uma operação bem-sucedida', async () => {
const operation = vi.fn(async () => 'ok');
const timer = (fn, ms) => { fn(); return 1; };
expect(await retry(operation, 2, 10, timer)).toBe('ok');
});

// Amplie para falha transitória, limite de tentativas, atraso e limpeza entre testes.

Exemplo e resultado público

op falha uma vez, resolve → duas chamadas e um atraso virtual

Bordas e erros publicados

falha até max -> erro original; após teste vi.useRealTimers/vi.restoreAllMocks; spy não altera código de produção

Teste público e aprovação visível

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos.

Restrições e documentação

https://vitest.dev/guide/mocking/timers

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que usar relógio falso num teste com atraso?

• Para fazer a função de produção executar com um relógio diferente.
• Para dispensar restauração de timer e spy após o teste.
 (Correta) Para controlar a ordem e o tempo virtual sem deixar o teste lento ou instável.

Por que a resposta está certa

Correto: Para controlar a ordem e o tempo virtual sem deixar o teste lento ou instável. Dependências substituíveis e tempo controlado permitem observar trabalho assíncrono sem depender de serviços ou relógios variáveis.

Armadilha da alternativa incorreta

Reveja o conceito: Dependências substituíveis e tempo controlado permitem observar trabalho assíncrono sem depender de serviços ou relógios variáveis. A resposta conceitual esperada é: Para controlar a ordem e o tempo virtual sem deixar o teste lento ou instável.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

falha até max → erro original; após teste vi.useRealTimers/vi.restoreAllMocks; spy não altera código de produção

Concepção errada e falha observável

Usar rede real ou timer real e criar teste lento/instável. / O teste depende de relógio/rede real ou deixa timers e mocks ativos para o próximo teste.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar rede real ou timer real e criar teste lento/instável.

Pistas graduais

1. Injete op/clock ou use fake timers. 2. Avance relógio controladamente. 3. Restaure spies e timers no teardown.

Solução de referência explicada

Suíte Vitest controla promise e tempo, verifica número de chamadas/intervalos e restaura estado global em afterEach.

Transferência em outro contexto

Teste debounce sem aguardar o relógio real, provando a quantidade de chamadas após os intervalos.

Evidência de domínio

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.10 Preserve a causa do erro

MODIFY · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 3.0/5

Competência: errors.cause | Requer: functions.return | Introduz: errors.cause

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Traduza falha externa para erro de domínio conservando cause. Separe mensagem pública de detalhe técnico e decida como normalizar valores que não são Error.

Starter de trabalho:
capturar erro do adaptador externo e lançar erro de domínio com cause; UI mostra mensagem pública

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Separe em duas colunas o dado registrado no log técnico e a mensagem apresentada à pessoa antes de escrever o catch.

Explicação e modelo mental

• Modelo mental: Erros atravessam fronteiras de camadas. Contexto útil para diagnóstico e informação apropriada à interface têm necessidades distintas.
• Exemplo resolvido: Um adaptador de compactação pode lançar uma falha técnica; a camada de domínio pode acrescentar contexto funcional mantendo a causa original para diagnóstico interno.
• Armadilha comum: Armadilha: Perder stack/cause ao substituir erro ou exibir detalhes sensíveis ao usuário. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Erro de storage indisponível. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

service.js: export async function carregarConfiguracao(readFile) {
try {
return await readFile();
} catch (erro) {
throw new Error(String(erro));
}
}

// BUG: a causa original foi descartada e o texto externo pode expor detalhes indevidos.

Exemplo e resultado público

falha externa 'ECONNRESET' → Error('Não foi possível salvar',{cause:erroOriginal})

Bordas e erros publicados

cause mantém identidade do erro; UI não mostra token/URL sensível; erro não-Error lançado -> normalização explícita

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/cause

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao traduzir uma falha técnica para um erro de domínio, o que a mensagem pública deve priorizar?

 (Correta) Uma descrição estável e compreensível da condição relevante para quem usa o domínio.
• Todos os detalhes internos e valores locais capturados durante a falha.
• O nome da biblioteca externa, mesmo quando não ajuda a pessoa consumidora.

Por que a resposta está certa

Correto. A mensagem de domínio descreve o problema no vocabulário da aplicação.

Armadilha da alternativa incorreta

A mensagem pública deve servir ao domínio; detalhes de implementação têm outro público e outro canal.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

cause mantém identidade do erro; UI não mostra token/URL sensível; erro não-Error lançado → normalização explícita

Concepção errada e falha observável

Perder stack/cause ao substituir erro ou exibir detalhes sensíveis ao usuário. / O erro externo perde cause ou sua mensagem interna/segredo aparece ao usuário.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Perder stack/cause ao substituir erro ou exibir detalhes sensíveis ao usuário.

Pistas graduais

1. Capture falha de fronteira. 2. Relance com mensagem pública e cause original. 3. Separe logging técnico da apresentação.

Solução de referência explicada

try/catch na borda; throw new Error('Não foi possível salvar',{cause:e}); camada UI lê só mensagem pública.

Transferência em outro contexto

Trate falha de armazenamento: mantenha a causa técnica para diagnóstico e mostre mensagem genérica à interface.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício; Error.cause (ES2022, Node.js 16.9+) para encadear causas nos testes. | Próxima sugestão: 13.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.11 Leia a documentação para implementar

DOCS · 18 min (estimativa, sem nota por velocidade) · Navegador e documentação oficial · dificuldade multidimensional 2.5/5

Competência: docs.official_lookup | Requer: functions.return | Introduz: docs.official_lookup

Eixos: novidade 4 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Consulte a documentação oficial de Array.prototype.at, registre assinatura/retorno/compatibilidade e aplique o que aprendeu numa função pequena.

Starter de trabalho:
documentação oficial de Array.prototype.at: assinatura, índice negativo, retorno e compatibilidade; usar em tarefa

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Encontre a página oficial, anote assinatura, retorno e limite relevante, e registre a data/fonte antes de codificar.

Explicação e modelo mental

• Modelo mental: Documentação de API registra contrato e compatibilidade em uma fonte verificável; versões de runtime determinam o que está disponível.
• Exemplo resolvido: A busca oficial de findLast documenta que o retorno pode ser undefined quando nenhum elemento satisfaz a condição; esse caso precisa constar no contrato da função que a usa.
• Armadilha comum: Armadilha: Copiar exemplo de blog sem checar retorno/compatibilidade oficial. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Ler docs de Object.hasOwn e aplicar em lookup de propriedade. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

indices.js: export function ler(itens,indice){return itens.at(indice);}
// Consulte a documentação oficial de Array.prototype.at. Registre assinatura, retorno,
// compatibilidade do runtime-alvo e compare índice positivo, negativo e fora dos limites.

Exemplo e resultado público

[10,20,30].at(-1) → 30; fonte oficial identificada com URL/data

Bordas e erros publicados

[].at(-1) -> undefined; índice fora -> undefined; fallback arr[arr.length-1] quando baseline exigir

Teste público e aprovação visível

Rubrica de assinatura, parâmetros, retorno, erros, compatibilidade e aplicação.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/at

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual evidência torna uma decisão de compatibilidade verificável depois?

• Um exemplo de blog sem checar assinatura nem retorno.
 (Correta) Fonte oficial identificada com URL, ambiente alvo e data da consulta.
• O ano em que a proposta foi aprovada, sem olhar suporte do runtime.

Por que a resposta está certa

Correto: Fonte oficial identificada com URL, ambiente alvo e data da consulta. Documentação de API registra contrato e compatibilidade em uma fonte verificável; versões de runtime determinam o que está disponível.

Armadilha da alternativa incorreta

Reveja o conceito: Documentação de API registra contrato e compatibilidade em uma fonte verificável; versões de runtime determinam o que está disponível. A resposta conceitual esperada é: Fonte oficial identificada com URL, ambiente alvo e data da consulta.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

[].at(-1) → undefined; índice fora -> undefined; fallback arr[arr.length-1] quando baseline exigir

Concepção errada e falha observável

Copiar exemplo de blog sem checar retorno/compatibilidade oficial. / Índice negativo/out-of-range diverge de Array.prototype.at ou a decisão ignora compatibilidade do runtime.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Copiar exemplo de blog sem checar retorno/compatibilidade oficial.

Pistas graduais

1. Localize página oficial da API. 2. Extraia assinatura, retorno, limites e compatibilidade. 3. Use a API em uma função curta e cite a decisão de fallback.

Solução de referência explicada

Entregue ficha de documentação com link oficial e lastItem(arr) que retorna arr.at(-1) ou fallback declarado.

Transferência em outro contexto

Leia a documentação de Object.hasOwn e aplique-a numa busca por propriedade própria, registrando retorno e suporte.

Evidência de domínio

Rubrica de assinatura, parâmetros, retorno, erros, compatibilidade e aplicação. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: navegador atual e documentação oficial acessível; registre a fonte e a data da consulta. | Próxima sugestão: 13.12

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.12 Run, debugger, teste ou leitura?

INVESTIGATE · 20 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · Vitest (versão estável atual) · dificuldade multidimensional 3.0/5

Competência: debugging.tool_choice | Requer: functions.return | Introduz: debugging.tool_choice

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 2 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Associe cada falha à ferramenta que produz a evidência necessária. Escreva hipótese, evidência observada, limite da ferramenta e alternativa.

Starter de trabalho:
quatro falhas: sintaxe; valor inesperado em loop; regressão; requisito desconhecido de API

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Classifique primeiro a pergunta: estrutura, execução, regressão, tempo ou requisito de API; só então escolha a ferramenta.

Explicação e modelo mental

• Modelo mental: Cada ferramenta torna certos fatos observáveis e deixa outros fora de alcance; a escolha depende da pergunta investigada.
• Exemplo resolvido: Uma falha de sintaxe fica visível num diagnóstico de parse; um valor inesperado dentro de um loop pede observar índice/acumulador com uma entrada fixa.
• Armadilha comum: Armadilha: Usar sempre Run sem hipótese ou sempre debugger para dúvida documental. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Diagnóstico de bug de Promise. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: Falha de sintaxe; valor inesperado num loop; regressão; requisito sem critério.
Para cada caso escolha Run, debugger, teste ou leitura de documentação. Registre hipótese, evidência que a ferramenta oferece e o que ela não prova.

Exemplo e resultado público

sintaxe → leitura/typecheck; loop -> debugger; regressão -> teste; API -> docs oficial

Bordas e erros publicados

caso intermitente dependente de tempo -> fake timer/log controlado; registrar evidência e ferramenta alternativa

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

https://nodejs.org/en/learn/getting-started/debugging

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como escolher uma ferramenta de diagnóstico?

• Sempre usar Run, qualquer que seja a pergunta.
• Sempre usar debugger mesmo para confirmar requisito documental.
 (Correta) Pela evidência necessária para testar a hipótese, considerando também seus limites.

Por que a resposta está certa

Correto: Pela evidência necessária para testar a hipótese, considerando também seus limites. Cada ferramenta torna certos fatos observáveis e deixa outros fora de alcance; a escolha depende da pergunta investigada.

Armadilha da alternativa incorreta

Reveja o conceito: Cada ferramenta torna certos fatos observáveis e deixa outros fora de alcance; a escolha depende da pergunta investigada. A resposta conceitual esperada é: Pela evidência necessária para testar a hipótese, considerando também seus limites.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

caso intermitente dependente de tempo → fake timer/log controlado; registrar evidência e ferramenta alternativa

Concepção errada e falha observável

Usar sempre Run sem hipótese ou sempre debugger para dúvida documental. / A ferramenta escolhida não mostra a evidência necessária ou a conclusão excede o que a observação prova.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar sempre Run sem hipótese ou sempre debugger para dúvida documental.

Pistas graduais

1. Classifique a dúvida antes da ferramenta. 2. Escolha ferramenta que torna evidência observável. 3. Compare custo/limite com alternativa.

Solução de referência explicada

Para cada falha registre hipótese, ferramenta, evidência, conclusão e alternativa; não declare correção sem reprodução.

Transferência em outro contexto

Investigue uma falha de Promise com tempo controlado e registre contexto, traço e alternativa à ferramenta escolhida.

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e runner Vitest provisionado para o exercício. | Próxima sugestão: 13.13

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.13 Teste DOM com teclado e foco

TEST · 25 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.5/5

Competência: testing.dom_accessibility | Requer: functions.return | Introduz: testing.dom_accessibility

Eixos: novidade 4 · sintaxe 3 · composição 3 · estado 4 · depuração 2 · arquivos 3 · leitura 4 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Escreva testes para uso por teclado, associação de rótulo/erro, foco após submit, anúncio de resultado e inserção segura de texto. Separe o que o teste automatizado prova do que requer revisão assistiva.

Starter de trabalho:
form com label,input,button,ul e região aria-live; escrever testes de DOM e teclado

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Liste o comportamento observável e a ferramenta adequada antes de escrever cada assert.

Explicação e modelo mental

• Modelo mental: Testes de interface podem observar estrutura, foco e eventos, mas cada ferramenta tem limites sobre a experiência assistiva completa.
• Exemplo resolvido: Num acordeão, um teste pode verificar que Enter expande a seção e atualiza aria-expanded; revisar anúncio e ordem de foco com tecnologia assistiva ainda exige conferir experiência real.
• Armadilha comum: Armadilha: Testar só querySelector e deixar foco/teclado/sink inseguro passar. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Testar diálogo de confirmação acessível. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

app.js: export function montarWidget(document) { /* TODO: associe submit, foco, texto literal e delegação */ } | tests/13.13.test.js: import {montarWidget} from '../app.js';
describe('suíte DOM do widget', () => {
test('mantém associação de rótulo e botão de submit', () => { montarWidget(document);
expect(document.querySelector('label[for=text]')).toBeTruthy();
expect(document.querySelector('#add[type=submit]')).toBeTruthy(); });
});

Exemplo e resultado público

label ligada e submit Enter → item criado e foco conforme contrato

Bordas e erros publicados

erro inválido -> foco no campo e texto anunciado; string '<img>' -> nenhum img criado; clique de botão filho por delegação -> um item removido

Teste público e aprovação visível

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Keyboard-navigable_JavaScript_widgets

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que um teste DOM automatizado consegue verificar diretamente?

 (Correta) Relações do DOM, estado de foco e resposta a eventos de teclado simulados.
• Toda a qualidade do anúncio para qualquer leitor de tela e dispositivo.
• A experiência completa sem verificar ordem de foco ou texto anunciado.

Por que a resposta está certa

Correto: Relações do DOM, estado de foco e resposta a eventos de teclado simulados. Testes de interface podem observar estrutura, foco e eventos, mas cada ferramenta tem limites sobre a experiência assistiva completa.

Armadilha da alternativa incorreta

Reveja o conceito: Testes de interface podem observar estrutura, foco e eventos, mas cada ferramenta tem limites sobre a experiência assistiva completa. A resposta conceitual esperada é: Relações do DOM, estado de foco e resposta a eventos de teclado simulados.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

erro inválido → foco no campo e texto anunciado; string '<img>' -> nenhum img criado; clique de botão filho por delegação -> um item removido

Concepção errada e falha observável

Testar só querySelector e deixar foco/teclado/sink inseguro passar. / O teste não detecta falha de teclado/foco/associação ou confunde checagem DOM com experiência completa de leitor de tela.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Testar só querySelector e deixar foco/teclado/sink inseguro passar.

Pistas graduais

1. Use ações de teclado e asserts de foco. 2. Inspecione nome/associação de label e região de feedback. 3. Verifique DOM real para texto não confiável.

Solução de referência explicada

Suíte DOM testa entrada por teclado, foco, mensagens, delegação e ausência de elemento injetado; diferencie checks automáticos de revisão manual com leitor de tela.

Transferência em outro contexto

Teste um diálogo de confirmação acessível: abertura por teclado, foco inicial, fechamento e retorno de foco ao acionador.

Evidência de domínio

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: navegador atual com APIs DOM; execução local, sem serviço externo. | Próxima sugestão: 13.14

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

13.14 Kanban V2: migração e regressões

MODIFY · 75 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 4.5/5

Competência: testing.evolving_project | Requer: functions.return | Introduz: testing.evolving_project

Eixos: novidade 4 · sintaxe 3 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Evolua o Kanban conforme a change request: a V3 acrescenta tags, um array de strings que vale [] quando ausente. Reuse a migração V1→V2 de 11.13 (que já acrescenta priority='normal') e crie migrarV2ParaV3 como função pura. Saves V1 percorrem V1→V2→V3; saves V2 percorrem só V2→V3; saves V3 não migram de novo. Preserve id, text, status e priority. Tags inválidas viram [] com aviso; registros parciais são descartados com aviso sem perder os válidos; JSON totalmente inválido segue o contrato anterior e não sobrescreve o storage. Mantenha texto HTML literal, teclado, foco e persistência. Entregue a alteração localizada e os testes de domínio, DOM e regressão.

Objetivo da atividade

Liste o que V1 (11.08) e V2 (11.13) já garantem e o que a V3 acrescenta (tags) antes de alterar o loader.

Explicação e modelo mental

• Modelo mental: Uma mudança de projeto convive com comportamento já utilizado. Evoluir exige preservar compatibilidade e tornar as novas regras observáveis.
• Exemplo resolvido: Uma lista de compras V2 pode ganhar categoria na V3. A migração V2→V3 preenche um padrão; um save V1 primeiro passa pela migração V1→V2 já existente e só depois pela nova, sem reescrever a anterior.
• Armadilha comum: Armadilha: converter V1 direto para V3, reimplementar V1→V2 ou migrar de novo um save que já está na V3. Cada etapa tem contrato próprio e testes próprios.
• Por que importa: Esse raciocínio reaparece em Catálogo de estoque V2→V3 com reorderThreshold. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

migration.js: export function migrarV1ParaV2(value) { /* TODO: reutilize a migração existente de 11.13 */ }
export function migrarV2ParaV3(value) { /* TODO: acrescente tags sem perder prioridade */ }
export function carregar(storage, warn = () => {}) { /* TODO: validar por registro e gravar somente V3 válido */ } | migration.test.js: import { migrarV1ParaV2, migrarV2ParaV3, carregar } from './migration.js';

```
// V1 (11.08): { version: 1, tasks: [{ id, text, status }] }
// V2 (11.13): acrescenta priority ('normal' por padrão)
// V3 (esta change request): acrescenta tags, um array de strings ([] por padrão)

const v1 = { version: 1, tasks: [{ id: 't1', text: 'Ler pauta', status: 'todo' }] };
const v2 = { version: 2, tasks: [{ id: 't1', text: 'Ler pauta', status: 'todo', priority: 'high' }] };

// TODO: mantenha os testes de 11.13 e acrescente V1→V3, V2→V3, V3 sem nova migração,
// tags inválidas, registro parcial e JSON inválido.
```

Exemplo e resultado público

Save V1 {version:1,tasks:[{id:'t1',text:'Ler pauta',status:'todo'}]} → Passa por V1→V2 e V2→V3: {version:3,tasks:[{id:'t1',text:'Ler pauta',status:'todo',priority:'normal',tags:[]}]}. | Save V2 {version:2,tasks:[{id:'t1',text:'Ler pauta',status:'todo',priority:'high'}]} → Passa só por V2→V3: mantém priority='high' e o id, acrescenta tags:[]. | Save V3 com tags ['urgente'] carregado duas vezes → Nenhuma migração é aplicada; o resultado é idêntico nas duas leituras.

Bordas e erros publicados

V1 do Kanban 11.08 -> passa por V1→V2 de 11.13 e então V2→V3; V2 com priority='high' -> V3 mantém high e recebe tags:[]; tags inválidas -> [] com aviso; registro parcial é reportado sem perder outros itens; V3 não migra novamente; JSON inválido não sobrescreve storage

Teste público e aprovação visível

A suíte de 11.13 continua verde e a cadeia V1→V2→V3 é coberta por teste.

Restrições e documentação

https://vitest.dev/guide/learn/writing-tests

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como acrescentar tags na V3 sem quebrar o contrato V1→V2 de 11.13?

• Substituir o migrador por uma conversão direta V1→V3 e remover o caminho V2.
• Recriar ids e descartar saves V2 que ainda não tenham tags.
 (Correta) Criar migrarV2ParaV3 pura e encadear V1→V2→V3 no loader, sem migrar de novo saves que já estão na V3.
• Sobrescrever registros parciais com uma lista vazia para simplificar a nova versão.

Por que a resposta está certa

A cadeia preserva cada contrato anterior e torna a nova migração testável isoladamente.

Armadilha da alternativa incorreta

Preserve a migração de 11.13 e encadeie a nova. Converter direto, recriar ids ou descartar registros perde dados.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

V1 do Kanban 11.08 → passa por V1→V2 de 11.13 e então V2→V3; V2 com priority='high' -> V3 mantém high e recebe tags:[]; tags inválidas -> [] com aviso; registro parcial é reportado sem perder outros itens; V3 não migra novamente; JSON inválido não sobrescreve storage

Concepção errada e falha observável

Reimplementar V1→V2, converter V1 direto para V3 ou reaplicar uma migração a um save que já está na versão atual; perder dados ou acessibilidade anterior. / A versão V1 não percorre V1→V2→V3, prioridade existente se perde ou V3 é migrada outra vez.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Reimplementar V1→V2, converter V1 direto para V3 ou reaplicar uma migração a um save que já está na versão atual; perder dados ou acessibilidade anterior.

Pistas graduais

1. Cubra com teste a migração V1→V2 de 11.13 antes de mudar qualquer coisa. 2. Implemente migrarV2ParaV3 como função pura: tags ausente vira [], priority não muda. 3. Componha V1→V2→V3 no loader e rode as regressões de domínio, DOM e persistência.

Solução de referência explicada

Implemente migrarV2ParaV3 como função pura: acrescente tags:[] quando ausente, preserve tags válidas e todos os campos V2, inclusive priority. O loader aceita V1, V2 e V3, aplica a migração V1→V2 de 11.13 a entradas V1 e depois V2→V3; valida por registro e só grava V3 após conversão segura. Preserve os testes e a interface existentes.

Transferência em outro contexto

Evolua o catálogo de estoque V2 para V3 adicionando reorderThreshold; preserve a migração encadeada para entradas V1/V2, os dados e os controles acessíveis.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: navegador atual com APIs DOM; execução local, sem serviço externo. | Próxima sugestão: 14.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14 ESM, Node, npm e Git

13 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 14.01 · 14.02 · 14.03 · 14.04 · 14.05 · 14.06 · 14.07 · 14.08 · 14.09 · 14.10 · 14.11 · 14.12 · 14.13

14.01 Mapa de imports

READ · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.0/5

Competência: modules.dependency_graph | Requer: functions.return | Introduz: modules.dependency_graph

Eixos: novidade 3 · sintaxe 1 · composição 2 · estado 1 · depuração 3 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 1

VISÃO DO ALUNO

Identifique exatamente os nós que pertencem ao ciclo e proponha uma fronteira comum que quebre a dependência sem criar um novo ciclo.

Starter de trabalho:
grafo A->B->C->A, D->B, E isolado; desenhe dependências

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Siga cada aresta até retornar ao ponto inicial e marque separadamente os módulos que apenas apontam para o ciclo.

Explicação e modelo mental

• Modelo mental: Um grafo mostra relações de dependência. Pertencer a um ciclo requer participar do caminho que retorna ao ponto de partida.
• Exemplo resolvido: Se módulos carrinho, preço e estoque dependem em círculo, um módulo de tipos/regras compartilhadas pode receber contrato estável desde que não importe os consumidores.
• Armadilha comum: Armadilha: Marcar qualquer módulo que aponta ao ciclo como parte dele. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Ciclo service/repository/model. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: Grafo de imports de valor: A -> B, B -> C, C -> A, D -> B, E isolado.
Identifique somente os membros do ciclo e proponha uma fronteira comum sem novo ciclo. Um import de tipo isolado em D não cria aresta de runtime.

Exemplo e resultado público

ciclo → A,B,C; extrair contrato comum X -> A/B/C dependem de X sem volta

Bordas e erros publicados

D aponta ao ciclo -> D não pertence a ele; import de tipo isolado não apaga import runtime de valor

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em um grafo dirigido, qual critério define que um nó pertence a um ciclo?

• O nó tem uma aresta para qualquer nó que faça parte de um ciclo.
 (Correta) Existe um caminho dirigido que sai do nó e retorna a ele.
• O nó está conectado por arestas a pelo menos outro componente do grafo.

Por que a resposta está certa

Pertencer ao ciclo exige que o próprio nó esteja em um caminho dirigido de retorno.

Armadilha da alternativa incorreta

Uma ligação com um ciclo não basta; verifique se o caminho retorna ao próprio nó.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

D aponta ao ciclo → D não pertence a ele; import de tipo isolado não apaga import runtime de valor

Concepção errada e falha observável

Marcar qualquer módulo que aponta ao ciclo como parte dele. / D é classificado como parte do ciclo só por apontar para B, ou o import de tipo é confundido com dependência de runtime.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Marcar qualquer módulo que aponta ao ciclo como parte dele.

Pistas graduais

1. Siga arestas até voltar ao ponto inicial. 2. Separe dependente do componente cíclico. 3. Extraia abstração estável sem fazer X depender dos consumidores.

Solução de referência explicada

Identifique ciclo A-B-C-A e proponha módulo X de interface/função compartilhada com grafo acíclico.

Transferência em outro contexto

Desenhe dependências entre service/repository/model e proponha uma fronteira comum que deixe o grafo acíclico.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 LTS com módulos ES. | Próxima sugestão: 14.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.02 Parsons de CLI

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 3.0/5

Competência: node.cli_entry | Requer: functions.return | Introduz: node.cli_entry

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Ordene a entrada CLI para separar leitura de argumentos, validação, chamada ao domínio e apresentação. Mantenha o domínio invocável sem process.argv.

Starter de trabalho:
blocos: process.argv.slice(2), validar comando, parse args, chamar função de domínio, imprimir e definir exitCode

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Marque o ponto em que dados externos se tornam entrada validada e só depois conecte a função de domínio.

Explicação e modelo mental

• Modelo mental: A linha de comando é uma fronteira entre convenções do processo e operações reutilizáveis da aplicação.
• Exemplo resolvido: Um comando de exportação pode validar o formato solicitado na borda e então chamar uma função de domínio recebendo uma opção já estruturada, sem ler argumentos de processo.
• Armadilha comum: Armadilha: Espalhar leitura do processo dentro da lógica de domínio. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em CLI para listar tarefas com argumento opcional. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

main.js: const comandos = new Set(['somar']);
export function executarDominio(comando,args){if(comando==='somar')return String(Number(args[0])+Number(args[1]));throw new Error('comando inválido');}
export function main(argv,io){const argumentos=argv.slice(2);io.write(executarDominio(argumentos[0],argumentos.slice(1))+'\n');return 0;}

Exemplo e resultado público

node cli.js add 'A' → domínio recebe add/A e saída confirma A

Bordas e erros publicados

comando desconhecido -> mensagem de uso e exitCode=2; função de domínio não lê process.argv diretamente

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

https://nodejs.org/api/process.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que process.argv.slice(2) fornece em um programa Node iniciado pela linha de comando?

• Todas as variáveis de ambiente que começam com letras minúsculas.
• O texto de saída que o processo já escreveu no terminal.
 (Correta) Os argumentos informados pelo usuário, sem os prefixos do executável e do script.

Por que a resposta está certa

process.argv começa com informações de execução; o recorte após elas contém os argumentos do comando.

Armadilha da alternativa incorreta

argv descreve argumentos do processo; variáveis de ambiente e saída do terminal são interfaces diferentes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

comando desconhecido → mensagem de uso e exitCode=2; função de domínio não lê process.argv diretamente

Concepção errada e falha observável

Espalhar leitura do processo dentro da lógica de domínio. / Domínio acessa process.argv diretamente, ou comando ausente/desconhecido não imprime uso e define o status contratado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Espalhar leitura do processo dentro da lógica de domínio.

Pistas graduais

1. Entrada CLI fica na borda. 2. Valide antes de invocar domínio. 3. Formate saída/erro depois do resultado.

Solução de referência explicada

Reordene parse/validação/chamada/apresentação e deixe domínio invocável sem processo.

Transferência em outro contexto

Adapte a CLI para listar tarefas com um argumento opcional mantendo a regra independente do processo.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.03 Complete package scripts

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 2.0/5

Competência: npm.scripts | Requer: functions.return | Introduz: npm.scripts

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 1 · depuração 1 · arquivos 3 · leitura 3 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Complete os scripts dev/test/start usando caminhos que existem, formato de módulo coerente e comando apropriado para cada ciclo de trabalho.

Starter de trabalho:
package.json incompleto; src/main.js; test/*.test.js; scripts dev,test,start

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Inspecione arquivos/dependências antes de escrever os valores em scripts e depois confira quais comandos rodam uma vez ou observam mudanças.

Explicação e modelo mental

• Modelo mental: Scripts de pacote são nomes ligados a comandos concretos e caminhos do projeto; sua utilidade depende dessa configuração corresponder aos arquivos e ambiente.
• Exemplo resolvido: Um projeto pode ter um script check que aponta para src/check.js; o nome bonito do script não ajuda se o arquivo ou executável não existe.
• Armadilha comum: Armadilha: Copiar scripts que apontam para arquivo inexistente ou misturar CJS/ESM. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Scripts para app com src/server.js. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

package.json: {"type":"module","scripts":{"dev":"TODO","test":"TODO","start":"TODO"}}
| package-lock.json: {"name":"devpath-14-03","lockfileVersion":3,"requires":true,"packages":{"":{"name":"devpath-14-03"}}}
| src/main.js: // Implemente o ponto de entrada ESM do app.
export {};
| test/app.test.js: import test from 'node:test';
test.todo('app');
| README.md: Complete dev/test/start apontando para os arquivos existentes e usando node:test; confirme no Node.js 24 local.

Exemplo e resultado público

npm run dev → node --watch src/main.js; npm test -> vitest run; npm start -> node src/main.js

Bordas e erros publicados

script ausente -> npm run falha; package type module combina com imports ESM; lockfile versionado

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

https://docs.npmjs.com/cli/v11/using-npm/scripts

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que precisa ser verificado além do nome do script no package.json?

 (Correta) O caminho de entrada, as dependências e a compatibilidade com o formato de módulos do pacote.
• Que dev, test e start sejam todos aliases do mesmo comando.
• Que o lockfile não seja versionado para permitir versões variadas.

Por que a resposta está certa

Correto: O caminho de entrada, as dependências e a compatibilidade com o formato de módulos do pacote. Scripts de pacote são nomes ligados a comandos concretos e caminhos do projeto; sua utilidade depende dessa configuração corresponder aos arquivos e ambiente.

Armadilha da alternativa incorreta

Reveja o conceito: Scripts de pacote são nomes ligados a comandos concretos e caminhos do projeto; sua utilidade depende dessa configuração corresponder aos arquivos e ambiente. A resposta conceitual esperada é: O caminho de entrada, as dependências e a compatibilidade com o formato de módulos do pacote.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

script ausente → npm run falha; package type module combina com imports ESM; lockfile versionado

Concepção errada e falha observável

Copiar scripts que apontam para arquivo inexistente ou misturar CJS/ESM. / Script aponta para caminho inexistente, usa módulo incompatível ou não devolve o status do teste.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Copiar scripts que apontam para arquivo inexistente ou misturar CJS/ESM.

Pistas graduais

1. Liste arquivos de entrada reais. 2. Associe um comando a cada modo. 3. Cheque package type e dependência vitest.

Solução de referência explicada

Preencha scripts com caminhos reais e verifique que test termina, dev observa e start executa uma vez.

Transferência em outro contexto

Configure scripts para uma aplicação com src/server.js, distinguindo execução única e modo de desenvolvimento.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.04 ESM vs CJS

COMPARE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 3.0/5

Competência: modules.esm_cjs | Requer: functions.return | Introduz: modules.esm_cjs

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 4 · leitura 3 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Converta o pacote CommonJS para ESM de modo consistente: formato declarado, exports, imports relativos com extensão e execução.

Starter de trabalho:
CJS: module.exports={soma}; require('./math'); converter pacote para ESM

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Faça uma tabela do que o arquivo exporta e do que cada consumidor importa antes de mudar package.json ou sintaxe.

Explicação e modelo mental

• Modelo mental: CommonJS e ES modules definem formas de ligação entre módulos; o formato precisa ser coerente ao longo da fronteira entre consumidor e fornecedor.
• Exemplo resolvido: Um módulo de validação pode trocar module.exports/require por export nomeado/import relativo; tanto produtor quanto consumidor precisam usar o mesmo formato.
• Armadilha comum: Armadilha: Achar que trocar uma palavra em package.json converte imports automaticamente. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Converter módulo de formatter CJS para ESM. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: math.cjs exporta soma com module.exports; app.cjs importa por require.
Converta o projeto para ESM: declare type: module, use export/import nomeados e inclua extensão nos imports relativos. Confirme a resolução no Node do projeto; o browser não simula package.json.

Exemplo e resultado público

package.json type=module; math.js export function soma; main.js import {soma} from './math.js' → 3

Bordas e erros publicados

require direto em ESM -> ReferenceError; extensão .js em import relativo Node ESM; exportações padrão e nomeadas consistentes

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://nodejs.org/api/esm.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual afirmação descreve o escopo de cada arquivo ES module?

• Todos os módulos compartilham automaticamente um escopo global de declarações locais.
 (Correta) Cada módulo tem seu próprio escopo; seus nomes só atravessam arquivos pela interface de módulos.
• As declarações dentro de um módulo ficam visíveis a qualquer arquivo sem importação.

Por que a resposta está certa

ES modules encapsulam declarações por arquivo e expõem explicitamente os nomes exportados.

Armadilha da alternativa incorreta

A fronteira do módulo impede visibilidade implícita; comunicação entre arquivos usa a interface de importação/exportação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

require direto em ESM → ReferenceError; extensão .js em import relativo Node ESM; exportações padrão e nomeadas consistentes

Concepção errada e falha observável

Achar que trocar uma palavra em package.json converte imports automaticamente. / Somente package.json ou só um lado da fronteira muda; import relativo ESM não resolve no Node.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que trocar uma palavra em package.json converte imports automaticamente.

Pistas graduais

1. Defina o formato do pacote. 2. Converta export e import em conjunto. 3. Execute entrada com caminhos/extension corretos.

Solução de referência explicada

Edite package type, math export nomeado e main import relativo com extensão; documente fronteira com dependência CJS se houver.

Transferência em outro contexto

Converta um módulo de formatação de data de CommonJS para ESM preservando a API que o consumidor espera.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.05 Bug de caminho

DEBUG · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 3.5/5

Competência: node.module_relative_paths | Requer: functions.return | Introduz: node.module_relative_paths

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 4 · depuração 4 · arquivos 4 · leitura 3 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Corrija a resolução do arquivo para que o CLI encontre seu recurso ao ser chamado de qualquer diretório de trabalho. Contextualize erro ausente com causa.

Starter de trabalho:
readFileSync('./data/config.json') quebra ao chamar CLI de outro cwd; módulo ESM

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Escreva a diferença entre diretório de execução e URL do módulo antes de trocar a expressão de caminho.

Explicação e modelo mental

• Modelo mental: O diretório de execução e a localização do módulo têm origens diferentes; caminhos devem expressar qual deles define o recurso.
• Exemplo resolvido: Um módulo de importação pode localizar schema.json relativo à própria URL ao ser chamado por um teste iniciado em outra pasta.
• Armadilha comum: Armadilha: Pensar que caminho relativo de fs se refere ao arquivo fonte. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em CLI instalado globalmente lê template ao lado do módulo. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

config.js: import {readFile} from 'node:fs/promises';
export async function carregarConfig(){return JSON.parse(await readFile('./data/config.json','utf8'));}
// BUG: o caminho depende do cwd e quebra ao chamar o CLI de outro diretório.

Exemplo e resultado público

executar no diretório do projeto → carrega; executar de /tmp -> carrega o mesmo arquivo junto ao módulo

Bordas e erros publicados

caminho com espaço -> new URL('./data/config.json',import.meta.url) resolve; arquivo ausente -> erro contextual com cause

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

https://nodejs.org/api/fs.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que usar os helpers de node:path em vez de concatenar fragmentos com uma barra escrita manualmente?

• A concatenação manual transforma qualquer caminho relativo em URL de módulo.
• Os helpers verificam que o arquivo existe e carregam seu conteúdo.
 (Correta) Os helpers compõem e normalizam caminhos segundo as convenções da plataforma.

Por que a resposta está certa

node:path lida com composição e separadores do sistema; leitura de arquivo é outra operação.

Armadilha da alternativa incorreta

Helpers de caminho não carregam arquivos nem resolvem automaticamente o contexto do módulo; eles normalizam segmentos de caminho.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

caminho com espaço → new URL('./data/config.json',import.meta.url) resolve; arquivo ausente -> erro contextual com cause

Concepção errada e falha observável

Pensar que caminho relativo de fs se refere ao arquivo fonte. / A execução de outro cwd procura data/config.json no diretório do usuário em vez do diretório do módulo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Pensar que caminho relativo de fs se refere ao arquivo fonte.

Pistas graduais

1. process.cwd aponta diretório de execução. 2. import.meta.url aponta módulo. 3. new URL(relative,import.meta.url) produz URL aceitável por fs.

Solução de referência explicada

Use readFile(new URL('./data/config.json', import.meta.url), 'utf8') e envolva erro com causa contextual.

Transferência em outro contexto

Faça um CLI instalado globalmente localizar o template distribuído ao lado de seu módulo, qualquer que seja o cwd.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.06 Config por ambiente

MAKE · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 3.0/5

Competência: node.environment_config | Requer: functions.return | Introduz: node.environment_config

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente loadConfig(env) como função pura: leia defaults, converta e valide porta e nível, exija segredo no ambiente adequado e nunca o exponha em mensagens.

Starter de trabalho:
loadConfig(env) recebe PORT, LOG_LEVEL, TOKEN; defaults PORT=3000, LOG_LEVEL=info

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Liste cada variável como texto de entrada, tipo/intervalo de saída e regra de segredo antes de codificar.

Explicação e modelo mental

• Modelo mental: Variáveis de ambiente chegam como texto externo; a aplicação precisa atribuir significado, domínio válido e política de exposição.
• Exemplo resolvido: Uma configuração em ambiente de teste pode usar um timeout padrão; em produção, ausência de um identificador obrigatório deve gerar erro contextual sem incluir seu valor secreto.
• Armadilha comum: Armadilha: Usar || sobre strings sem validar ou publicar segredo em erro. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Config de timeout e ambiente. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

config.js: export function loadConfig(env) {
const portTexto = env.PORT || '3000';
const logLevel = env.LOG_LEVEL || 'info';
const token = env.TOKEN;
// TODO: trate defaults, conversão/intervalo da porta e segredo exigido em produção.
// Nunca inclua token em erro ou log.
}

Exemplo e resultado público

env vazio → port 3000/logLevel info; PORT='8080' -> 8080

Bordas e erros publicados

PORT ausente/vazio/espaços -> 3000; PORT='8080' -> 8080; PORT='abc'/'0'/'65536' -> RangeError; TOKEN ausente ou vazio em produção -> erro sem revelar segredo

Teste público e aprovação visível

Resultado observável: env vazio -> port 3000/logLevel info; PORT='8080' -> 8080. Borda a conferir: PORT ausente/vazio/espaços -> 3000; PORT='8080' -> 8080; PORT='abc'/'0'/'65536' -> RangeError; TOKEN ausente ou vazio em produção -> erro sem revelar segredo. Entrega esperada: Trate PORT ausente ou em branco como default 3000; converta texto restante e aceite somente inteiro de 1 a 65535. LOG_LEVEL ausente ou em branco vira info e valor desconhecido é rejeitado. Em produção, TOKEN ausente/em branco falha sem expor o valor.

Restrições e documentação

https://nodejs.org/api/process.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que injetar env em loadConfig?

 (Correta) Para testar combinações de configuração de forma determinística sem depender do processo real.
• Para registrar todas as variáveis, inclusive segredos, no console.
• Para deixar strings numéricas sem validação porque o ambiente já é tipado.

Por que a resposta está certa

Correto: Para testar combinações de configuração de forma determinística sem depender do processo real. Variáveis de ambiente chegam como texto externo; a aplicação precisa atribuir significado, domínio válido e política de exposição.

Armadilha da alternativa incorreta

Reveja o conceito: Variáveis de ambiente chegam como texto externo; a aplicação precisa atribuir significado, domínio válido e política de exposição. A resposta conceitual esperada é: Para testar combinações de configuração de forma determinística sem depender do processo real.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

PORT ausente/vazio/espaços → 3000; PORT='8080' -> 8080; PORT='abc'/'0'/'65536' -> RangeError; TOKEN ausente ou vazio em produção -> erro sem revelar segredo

Concepção errada e falha observável

Usar || sobre strings sem validar ou publicar segredo em erro. / Porta em branco/inválida recebe valor fora do intervalo ou segredo aparece em erro/log.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar || sobre strings sem validar ou publicar segredo em erro.

Pistas graduais

1. Injete env para testar. 2. Parseie porta inteira dentro de 1..65535. 3. Separe segredo da saída e valide por modo.

Solução de referência explicada

Trate PORT ausente ou em branco como default 3000; converta texto restante e aceite somente inteiro de 1 a 65535. LOG_LEVEL ausente ou em branco vira info e valor desconhecido é rejeitado. Em produção, TOKEN ausente/em branco falha sem expor o valor.

Transferência em outro contexto

Defina defaults e validação para timeout em diferentes modos, mantendo segredos fora da saída.

Evidência de domínio

Resultado observável: env vazio -> port 3000/logLevel info; PORT='8080' -> 8080. Borda a conferir: PORT ausente/vazio/espaços -> 3000; PORT='8080' -> 8080; PORT='abc'/'0'/'65536' -> RangeError; TOKEN ausente ou vazio em produção -> erro sem revelar segredo. Entrega esperada: Trate PORT ausente ou em branco como default 3000; converta texto restante e aceite somente inteiro de 1 a 65535. LOG_LEVEL ausente ou em branco vira info e valor desconhecido é rejeitado. Em produção, TOKEN ausente/em branco falha sem expor o valor. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.07 Compatibilidade de feature

DOCS · TRILHA OPCIONAL · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: compatibility.feature_decision | Requer: functions.return | Introduz: compatibility.feature_decision

Eixos: novidade 4 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Avalie Promise.try no runtime-alvo Node.js 24 LTS consultando uma fonte oficial. Registre versão exata, data da consulta, assinatura, parâmetros, retorno e tratamento de erros. Execute promise-try.js e compare três caminhos: A (Promise.try nativo), B (fallback fiel: chama fn dentro de try e converte retorno/exceção com Promise.resolve/Promise.reject) e C (Promise.resolve().then(fn), que adia fn para uma microtask). Decida entre usar a API, usar o fallback B ou adiar, e explique por que C não é equivalente.

Objetivo da atividade

Consulte documentação oficial para decidir se Promise.try está disponível no runtime baseline definido; registre runtime, versão, data da consulta, assinatura, parâmetros, retorno e erros antes de escolher uso ou fallback.

Explicação e modelo mental

• Modelo mental: Uma especificação descreve uma capacidade da plataforma; disponibilidade concreta depende do runtime e da versão escolhida.
• Exemplo resolvido: Uma API moderna pode existir na especificação antes de estar disponível no runtime implantado; uma tabela oficial por versão responde à pergunta que o código sozinho não resolve.
• Armadilha comum: Armadilha: Confundir ano da proposta com suporte em navegador/Node alvo ou tratar Promise.resolve().then como fallback semanticamente idêntico. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Avaliar Iterator helpers em runtime alvo. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

promise-try.js: // Node.js 24 LTS: registre versão, fonte oficial e data da consulta. Compare Promise.try, fallback síncrono fiel e Promise.resolve().then(callback).
const callback=()=>{throw new Error("falhou");};
console.log(typeof Promise.try);

Exemplo e resultado público

Promise.try está documentada como disponível na versão do runtime-alvo → Registrar fonte, versão e data; usar Promise.try e mostrar que o callback roda antes de 'antes de B' e que o throw síncrono vira rejeição. | Promise.try não está disponível no runtime-alvo → Usar o fallback B, que chama fn imediatamente; registrar que C adia fn para uma microtask e por isso não é equivalente.

Bordas e erros publicados

callback lança -> fallback retorna rejeição sem deixar throw síncrono escapar; Promise.resolve().then(fn) posterga a chamada para microtask e não é equivalente; transpilador transforma sintaxe mas não garante API runtime; registrar versão/data da consulta

Teste público e aprovação visível

A fonte oficial e o runtime/versão sustentam a decisão; o registro mostra quando o callback roda.

Restrições e documentação

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/try

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que evidência sustenta melhor a escolha de uma API moderna para um runtime específico?

• A data em que a proposta da API foi discutida, sem conferir o runtime.
 (Correta) Documentação oficial compatível com o runtime-alvo e confirmação da disponibilidade e assinatura da API.
• O fato de um editor não mostrar erro de sintaxe, mesmo sem verificar a API em execução.

Por que a resposta está certa

A compatibilidade depende do runtime declarado e do contrato real da API.

Armadilha da alternativa incorreta

Uma proposta ou ausência de erro no editor não confirma suporte no runtime escolhido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

callback lança → fallback retorna rejeição sem deixar throw síncrono escapar; Promise.resolve().then(fn) posterga a chamada para microtask e não é equivalente; transpilador transforma sintaxe mas não garante API runtime; registrar versão/data da consulta

Concepção errada e falha observável

Confundir ano da proposta com suporte em navegador/Node alvo ou tratar Promise.resolve().then como fallback semanticamente idêntico. / A decisão usa ano da proposta/transpilação sem verificar a API no runtime alvo em fonte oficial.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir ano da proposta com suporte em navegador/Node alvo ou tratar Promise.resolve().then como fallback semanticamente idêntico.

Pistas graduais

1. Declare runtime e versão antes da consulta. 2. Leia tabela/nota oficial de suporte. 3. Escolha fallback e compare quando fn é chamado e como throws são tratados.

Solução de referência explicada

Ficha registra recurso, baseline, fonte oficial, data, decisão e alternativa. Fallback fiel chama fn dentro de try, converte retorno com Promise.resolve e exceção com Promise.reject; documente que trocar o momento da chamada muda semântica.

Transferência em outro contexto

Avalie Iterator helpers para o runtime alvo e decida entre uso, fallback ou adiamento com fonte verificável.

Evidência de domínio

Rubrica de assinatura, parâmetros, retorno, erros, compatibilidade e aplicação. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS com módulos ES; Promise.try (ES2025, Node.js 23+) está disponível no runtime do curso, e o cartão exige registrar a fonte oficial antes de adotá-lo. | Próxima sugestão: 14.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.08 Microprojeto - CLI de notas

PROJECT · 100 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 4.5/5

Competência: node.notes_cli | Requer: functions.return | Introduz: node.notes_cli

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Entregue uma CLI modular de notas: parse de argumentos na borda, domínio puro e repositório JSON com leitura e validação seguras; ID inexistente retorna not-found; JSON inválido gera erro contextual sem sobrescrever storage; para gravar atomicamente, escreva em arquivo temporário no mesmo diretório e depois renomeie-o.

Starter de trabalho:
CLI modular add/list/remove; storage JSON local; domínio puro e adaptador de fs

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Desenhe dependências CLI→serviço→repositório e identifique uma operação de cada fronteira antes de implementar.

Explicação e modelo mental

• Modelo mental: Aplicações de linha de comando combinam entrada do processo, regras de domínio e persistência, que têm responsabilidades distintas.
• Exemplo resolvido: Uma CLI de relatórios pode validar --format na borda, chamar uma função pura para montar o conteúdo e só então imprimir o texto.
• Armadilha comum: Armadilha: Misturar process.argv, regras e fs num arquivo difícil de testar. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em CLI de inventário com comandos equivalentes. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

cli.js: export async function executar(comando, repositorio, argumentos = []) {
if (!repositorio || typeof repositorio[comando] !== 'function') return {status: 'usage', exitCode: 2};
}
export async function salvarAtomico(fs, destino, conteudo) { await fs.writeFile(destino, conteudo); }

// BUG: a gravação direta pode deixar dados parciais; o contrato exige temp e rename. | test/cli.test.js: import test from 'node:test';

// Escreva os testes de adicionar, listar, remover e das falhas de storage em diretório temporário.
test.todo('CLI de notas');

Exemplo e resultado público

add A; list → [A]; remove ID de A; list -> []; remove ID inexistente -> not-found; list com JSON inválido -> erro contextual sem sobrescrever storage

Bordas e erros publicados

list com JSON inválido -> erro contextual sem sobrescrever storage; remove ID inexistente -> not-found; escrita atômica via temp+rename para evitar truncamento

Teste público e aprovação visível

Critérios desta atividade: Entrega 1 — parse de add/list/remove fica na borda da CLI; serviço recebe argumentos explícitos e domínio não lê process.argv nem importa fs. Entrega 2 — repository é injetável; testes comprovam ordem, remoção, not-found e rejeição antes de alterar o estado. Entrega 3 — JSON inválido produz erro contextual e preserva os bytes; gravação escreve temporário no mesmo diretório e só então renomeia. Entrega 4 — testes de domínio e adapter usam diretório temporário; checklist e reflexão registram comando, formato de dados e política de falha observada.

Restrições e documentação

https://nodejs.org/api/fs.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que manter process.argv e fs fora do domínio puro?

• Para deixar cada função de domínio abrir seu próprio arquivo.
• Para que o serviço armazene argumentos globais e os leia depois.
 (Correta) Para poder testar regras de negócio sem processo ou I/O e trocar adaptadores sem reescrevê-las.

Por que a resposta está certa

Correto: Para poder testar regras de negócio sem processo ou I/O e trocar adaptadores sem reescrevê-las. Aplicações de linha de comando combinam entrada do processo, regras de domínio e persistência, que têm responsabilidades distintas.

Armadilha da alternativa incorreta

Reveja o conceito: Aplicações de linha de comando combinam entrada do processo, regras de domínio e persistência, que têm responsabilidades distintas. A resposta conceitual esperada é: Para poder testar regras de negócio sem processo ou I/O e trocar adaptadores sem reescrevê-las.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

list com JSON inválido → erro contextual sem sobrescrever storage; remove ID inexistente -> not-found; escrita atômica via temp+rename para evitar truncamento

Concepção errada e falha observável

Misturar process.argv, regras e fs num arquivo difícil de testar. / Regra de domínio depende de argv/fs, JSON inválido é sobrescrito ou gravação parcial destrói notas.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Misturar process.argv, regras e fs num arquivo difícil de testar.

Pistas graduais

1. Modele comandos puros e IDs. 2. Injete repositório para teste. 3. Persistência lê/valida e grava com estratégia segura.

Solução de referência explicada

Separe parse CLI, serviço de notas e repositório JSON; teste domínio sem I/O e teste adaptador com diretório temporário.

Transferência em outro contexto

Crie CLI de inventário com comandos equivalentes, serviço isolado e persistência em repositório substituível.

Evidência de domínio

Critérios desta atividade: Entrega 1 — parse de add/list/remove fica na borda da CLI; serviço recebe argumentos explícitos e domínio não lê process.argv nem importa fs. Entrega 2 — repository é injetável; testes comprovam ordem, remoção, not-found e rejeição antes de alterar o estado. Entrega 3 — JSON inválido produz erro contextual e preserva os bytes; gravação escreve temporário no mesmo diretório e só então renomeia. Entrega 4 — testes de domínio e adapter usam diretório temporário; checklist e reflexão registram comando, formato de dados e política de falha observada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.09 Git antes do PR

MODIFY · 22 min (estimativa, sem nota por velocidade) · Git (versão estável atual) · dificuldade multidimensional 3.5/5

Competência: git.workflow | Requer: functions.return | Introduz: git.workflow

Eixos: novidade 3 · sintaxe 2 · composição 4 · estado 3 · depuração 4 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Use o repositório descartável para inspecionar status/diff, criar commits pequenos e resolver o conflito preservando as duas intenções. Revise base/head antes de concluir.

Starter de trabalho:
repositório local descartável com main e feature, histórico curto e conflito preparado em arquivo de configuração; sem remoto real

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Escreva quais comandos revelam estado, conteúdo staged e diferença entre base/head antes de iniciar merge.

Explicação e modelo mental

• Modelo mental: Git registra estados e diferenças entre versões. Revisar essas evidências torna mudanças e integrações mais compreensíveis.
• Exemplo resolvido: Ao revisar uma mudança sem conflito que renomeia um arquivo, compare o status e o diff staged para confirmar exatamente qual caminho será incluído no commit.
• Armadilha comum: Armadilha: Fazer commit sem ler diff ou resolver conflito aceitando cegamente um lado. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Conflito em configuração de script. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: Repositório descartável: main tem README 'uso'; feature acrescenta 'exemplo'; working tree modifica config.json com segredo fictício. Inspecione status, diff staged, base e resultado combinado antes de concluir. Faça a atividade em Git local isolado.

Exemplo e resultado público

status,diff,commit,merge → resolução preserva intenção das duas mudanças

Bordas e erros publicados

arquivos não rastreados exigem add explícito; segredo fictício no diff -> remover antes de commit; diff de PR simulado compara base/head

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

https://git-scm.com/docs/git-status

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que deve ser feito antes de confirmar uma resolução de conflito?

 (Correta) Ler as duas versões e revisar o diff resultante para confirmar que ambas as intenções foram preservadas.
• Aceitar sempre o lado atual para evitar ler o conflito.
• Fazer commit sem olhar status/diff porque o merge reporta qualquer problema.

Por que a resposta está certa

Correto: Ler as duas versões e revisar o diff resultante para confirmar que ambas as intenções foram preservadas. Git registra estados e diferenças entre versões. Revisar essas evidências torna mudanças e integrações mais compreensíveis.

Armadilha da alternativa incorreta

Reveja o conceito: Git registra estados e diferenças entre versões. Revisar essas evidências torna mudanças e integrações mais compreensíveis. A resposta conceitual esperada é: Ler as duas versões e revisar o diff resultante para confirmar que ambas as intenções foram preservadas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

arquivos não rastreados exigem add explícito; segredo fictício no diff → remover antes de commit; diff de PR simulado compara base/head

Concepção errada e falha observável

Fazer commit sem ler diff ou resolver conflito aceitando cegamente um lado. / O commit inclui segredo/modificação não revisada ou a resolução descarta uma das intenções do conflito.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Fazer commit sem ler diff ou resolver conflito aceitando cegamente um lado.

Pistas graduais

1. Inspecione status e diff. 2. Crie branch/commit pequeno. 3. No conflito, leia base e as duas versões antes de editar.

Solução de referência explicada

Roteiro executável status,diff,add,commit,switch,merge; registre resolução sem perder conteúdo e revise diff do PR simulado.

Transferência em outro contexto

Resolva um conflito em configuração de scripts juntando as mudanças e revise o diff antes do commit.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Git CLI local em repositório descartável; sem remoto, conta ou credenciais. | Próxima sugestão: 14.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.10 Grafo ESM em execução

TRACE · 9 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · npm e ferramentas locais · dificuldade multidimensional 3.5/5

Competência: modules.esm_graph | Requer: functions.return | Introduz: modules.esm_graph

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 3 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Desenhe o grafo e a linha do tempo de avaliação dos módulos. Aguarde o import dinâmico e analise ciclo apenas a partir dos bindings realmente lidos.

Starter de trabalho:
A importa B; B tem top-level await; C usa import('./A.js'); log ordem; import.meta.url

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Separe vinculação, avaliação e resolução de Promise numa cronologia antes de prever os logs.

Explicação e modelo mental

• Modelo mental: Carregamento de módulos envolve ligação de dependências e avaliação de código; a ordem dessas etapas importa para efeitos e valores.
• Exemplo resolvido: Um módulo de tema pode importar tokens e aguardar configuração assíncrona; o módulo importador só continua a avaliação após a dependência terminar sua etapa assíncrona.
• Armadilha comum: Armadilha: Achar que import() devolve namespace síncrono ou que ciclo sempre funciona. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Módulos config/data/UI com await na inicialização. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: A importa binding nomeado de B; B exporta valor via top-level await; C faz import('./A.js'). Traceie linking, avaliação assíncrona, resolução de import() e logs separadamente. Registre runtime/formato e não una os snippets em um arquivo.

Exemplo e resultado público

B com await → avaliação de A espera B; import() -> Promise de namespace

Bordas e erros publicados

ciclo com leitura de binding antes da inicialização -> ReferenceError possível; import.meta.url refere módulo atual; import com efeito colateral executa uma vez

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

https://nodejs.org/api/esm.html

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que import() devolve imediatamente?

• O objeto namespace síncrono, pronto para acessar sem await.
 (Correta) Uma Promise que se resolve com o namespace do módulo.
• Uma Promise que cancela a avaliação se o importador fechar.

Por que a resposta está certa

Correto: Uma Promise que se resolve com o namespace do módulo. Carregamento de módulos envolve ligação de dependências e avaliação de código; a ordem dessas etapas importa para efeitos e valores.

Armadilha da alternativa incorreta

Reveja o conceito: Carregamento de módulos envolve ligação de dependências e avaliação de código; a ordem dessas etapas importa para efeitos e valores. A resposta conceitual esperada é: Uma Promise que se resolve com o namespace do módulo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

ciclo com leitura de binding antes da inicialização → ReferenceError possível; import.meta.url refere módulo atual; import com efeito colateral executa uma vez

Concepção errada e falha observável

Achar que import() devolve namespace síncrono ou que ciclo sempre funciona. / A ordem ignora espera assíncrona, ciclo concreto ou o fato de import() retornar uma Promise.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que import() devolve namespace síncrono ou que ciclo sempre funciona.

Pistas graduais

1. Separe vinculação de avaliação. 2. Trace dependências e awaits. 3. Marque leitura em zona temporal antes de inicialização.

Solução de referência explicada

Diagrama de grafo e cronologia de logs, com resultado de import() aguardado e explicação do ciclo concreto.

Transferência em outro contexto

Trace módulos config/data/UI com inicialização assíncrona e explique a ordem de avaliação entre dependências.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js atual conforme runtime indicado; código local, sem credenciais. | Próxima sugestão: 14.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.11 Devo instalar esta dependência?

COMPARE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: npm.dependency_decision | Requer: functions.return | Introduz: npm.dependency_decision

Eixos: novidade 4 · sintaxe 2 · composição 4 · estado 2 · depuração 2 · arquivos 3 · leitura 5 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Compare implementação local e bibliotecas com critérios de licença, manutenção, dependências transitivas, lockfile, testes e custo futuro. Baseie a escolha em evidência verificável.

Starter de trabalho:
opções biblioteca A, B ou função local para parse de 20 linhas; dados de licença/manutenção/dependências/lockfile

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Registre requisito mínimo e risco de manutenção local antes de avaliar fontes primárias e versões.

Explicação e modelo mental

• Modelo mental: Dependências carregam custo e responsabilidade ao longo do tempo; popularidade é só uma dimensão da decisão.
• Exemplo resolvido: Para formatar uma duração curta, uma função local minúscula pode ter custo de manutenção menor que adicionar dependência; para regras amplas, a política de licença e atualizações pesa na comparação.
• Armadilha comum: Armadilha: Escolher apenas por estrelas ou downloads sem verificar licença e manutenção. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Escolher parser de CSV para produto pequeno. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

material.md: Dossiê comparativo de duas bibliotecas CSV: licença, manutenção, tamanho, dependências transitivas, lockfile, testes e custo futuro. Escolha com evidência verificável; execução local de pacote não decide adequação.

Exemplo e resultado público

A ativa mas pesada, B leve porém abandonada, local pequena → decisão justificada por critérios e riscos

Bordas e erros publicados

licença incompatível -> descartar opção; lockfile sem versão pinada -> build não reproduzível; teste/atualização considerados

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://docs.npmjs.com/cli/v11/configuring-npm/package-json

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como decidir com responsabilidade entre manter uma função pequena no projeto e adotar uma dependência?

• Escolher sempre a opção mais popular, sem avaliar se atende ao caso de uso.
• Escolher sempre a opção com mais recursos, mesmo que o projeto use apenas uma parte.
 (Correta) Comparar o ajuste ao problema e o custo total de manter cada alternativa ao longo do tempo.

Por que a resposta está certa

A decisão deve considerar a necessidade real e as responsabilidades que cada opção traz.

Armadilha da alternativa incorreta

Popularidade ou quantidade de recursos, isoladamente, não demonstram que a opção seja adequada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

licença incompatível → descartar opção; lockfile sem versão pinada -> build não reproduzível; teste/atualização considerados

Concepção errada e falha observável

Escolher apenas por estrelas ou downloads sem verificar licença e manutenção. / Popularidade/tamanho isolado decide a dependência sem licença, manutenção, transitive tree ou lockfile.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Escolher apenas por estrelas ou downloads sem verificar licença e manutenção.

Pistas graduais

1. Defina requisito e risco de implementação local. 2. Cheque fonte primária: repositório/licença/changelog. 3. Registre versão, dependências transitivas e plano de atualização.

Solução de referência explicada

Matriz compara solução local/A/B com evidências datadas; escolha justificada e breve política de lockfile/atualização.

Transferência em outro contexto

Escolha uma estratégia para parsing de CSV em produto pequeno e registre evidências e tradeoffs de implementação/dependência.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 LTS com módulos ES. | Próxima sugestão: 14.12

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.12 Por que existe um bundler?

COMPARE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS e navegadores evergreen · dificuldade multidimensional 4.0/5

Competência: tooling.bundler_model | Requer: functions.return | Introduz: tooling.bundler_model

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 5 · leitura 4 · autonomia 4 · ferramentas 5

VISÃO DO ALUNO

Analise o que o bundler transforma e o que continua sendo responsabilidade do runtime. Compare os arquivos de origem com a saída e registre os limites da ferramenta.

Starter de trabalho:
projeto ESM main.js importa math.js e style.css; configuração mínima de bundler; comparar source/output

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Antes do build, anote os arquivos esperados e quais evidências mostrariam transformação, divisão e depuração do código.

Explicação e modelo mental

• Modelo mental: Ferramentas de build transformam e organizam arquivos para distribuição; isso é uma etapa de produção, separada da semântica do programa em runtime.
• Exemplo resolvido: Um mapa interativo pode importar uma imagem e um módulo de dados; o build pode emitir arquivos de saída e manter um mapa de origem que aponta para o módulo original.
• Armadilha comum: Armadilha: Achar que bundler é requisito da linguagem ESM ou que substitui testes. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Empacotar widget pequeno com CSS. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

src/index.html: <!doctype html><html><head><meta charset="UTF-8"><script type="module" src="/main.js"></script></head><body><button id="details">Detalhes</button> </body></html>
| src/main.js: import { soma } from './math.js';
import './style.css';
import { mostrarDetalhe } from './details.js';
document.querySelector('#result').textContent = String(soma(2, 3));
document.querySelector('#details').addEventListener('click', mostrarDetalhe);
| src/math.js: export function soma(a, b) { return a + b; }
export function multiplicar(a, b) { return a * b; }
| src/details.js: export function mostrarDetalhe() { document.querySelector('#result').textContent = '5 detalhes'; }
| src/style.css: body { font: 1rem sans-serif; }
button { color: navy; }

Exemplo e resultado público

build → entrada processada, chunks/assets gerados e source map apontando ao original

Bordas e erros publicados

import dinâmico -> chunk separado se ferramenta/config permitir; build sem sourcemap -> depuração menos clara; bundle não valida dados runtime

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://vite.dev/guide/

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o papel principal de um bundler no fluxo de módulos de um projeto?

 (Correta) Analisar módulos e recursos e produzir arquivos de saída conforme a configuração do projeto.
• Decidir a lógica de negócio que será executada em cada interação do usuário.
• Substituir o runtime-alvo, tornando desnecessário considerar o ambiente em que a saída será usada.

Por que a resposta está certa

O bundler transforma e organiza entradas para produzir artefatos de build.

Armadilha da alternativa incorreta

A saída do build ainda será usada por um runtime; a ferramenta não escolhe a lógica de negócio.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

import dinâmico → chunk separado se ferramenta/config permitir; build sem sourcemap -> depuração menos clara; bundle não valida dados runtime

Concepção errada e falha observável

Achar que bundler é requisito da linguagem ESM ou que substitui testes. / O bundle é confundido com o runtime e a resposta atribui ao bundler suporte a API que ele não implementa.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que bundler é requisito da linguagem ESM ou que substitui testes.

Pistas graduais

1. Execute ESM nativo primeiro. 2. Compare grafo fonte e arquivos de saída. 3. Registre transformação, assets, source map e limites.

Solução de referência explicada

Experimento documenta por que usar bundler no projeto e quando módulos nativos bastam; nenhuma framework exigida.

Transferência em outro contexto

Empacote um widget pequeno com CSS, compare a saída gerada e explique o que o source map permite depurar.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js e navegador atual; compare os contextos separadamente. | Próxima sugestão: 14.13

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

14.13 Event loop em dois runtimes

COMPARE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS e navegadores evergreen · dificuldade multidimensional 4.0/5

Competência: runtime.event_loop_context | Requer: functions.return | Introduz: runtime.event_loop_context

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 5 · depuração 3 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Faça previsões separadas para navegador e Node, anotando versão, formato do módulo e contexto de execução. Não transforme uma observação contextual em regra universal.

Starter de trabalho:
dois snippets rotulados: browser Promise.then,setTimeout; Node ESM process.nextTick,Promise.then; prever com contexto

Registre sua primeira decisão antes de executar; depois compare previsão, evidência e justificativa.

Objetivo da atividade

Registre primeiro ambiente, módulo e fase de execução para cada snippet; depois monte sua previsão e execute no mesmo contexto.

Explicação e modelo mental

• Modelo mental: Ordem de callbacks depende do runtime e do contexto de execução; uma observação local não estabelece uma regra universal.
• Exemplo resolvido: Quando um timer do navegador executa, ele pode agendar uma Promise; essa reação só roda depois que o callback atual termina e antes da próxima task de timer.
• Armadilha comum: Armadilha: Dizer sempre nextTick antes de Promise sem informar CJS/ESM e fase. Volte à regra pertinente e explique qual evidência a sustenta.
• Por que importa: Esse raciocínio reaparece em Investigar ordem de callback I/O e microtasks em Node. Reconhecer o mesmo mecanismo permite reutilizá-lo sem copiar a implementação.

Material/arquivo inicial

ordem-browser.js: // Browser evergreen
console.log('A síncrono');setTimeout(()=>console.log('T timer'),0);Promise.resolve().then(()=>console.log('P promise'));console.log('B síncrono');
// Node ESM/CJS usa também process.nextTick: execute localmente e registre versão/contexto.

Exemplo e resultado público

browser síncrono A, Promise P, timer T → A,P,T

Bordas e erros publicados

Node nextTick versus Promise em ESM/top-level -> ordem depende de contexto; executar no runtime declarado e registrar versão; não aplicar regra de CJS universalmente

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que torna uma observação sobre a ordem do event loop reproduzível e interpretável?

• Anotar apenas a ordem observada uma vez e tratá-la como universal.
 (Correta) Registrar o ambiente e o contexto do experimento e comparar execuções feitas sob as mesmas condições.
• Combinar resultados de runtimes diferentes sem identificar qual gerou cada observação.

Por que a resposta está certa

Resultados interpretáveis precisam estar associados às condições em que foram observados.

Armadilha da alternativa incorreta

Uma observação isolada ou sem contexto não justifica generalização entre ambientes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Node nextTick versus Promise em ESM/top-level → ordem depende de contexto; executar no runtime declarado e registrar versão; não aplicar regra de CJS universalmente

Concepção errada e falha observável

Dizer sempre nextTick antes de Promise sem informar CJS/ESM e fase. / A previsão ignora bloqueio síncrono ou transfere ordem/temporização do Node para o navegador sem delimitar contexto.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Dizer sempre nextTick antes de Promise sem informar CJS/ESM e fase.

Pistas graduais

1. Comece com ordem síncrona e filas do navegador. 2. Em Node declare versão e formato de módulo. 3. Compare saída observada e fonte oficial para a situação.

Solução de referência explicada

Tabela separa garantias do browser de detalhes de Node; previsão Node inclui contexto e verificação, sem regra universal falsa.

Transferência em outro contexto

Investigue callbacks de I/O e microtasks em Node, registrando versão, formato e fase antes de propor a ordem.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Compatibilidade: Node.js 24 e navegador atual, com exemplos executados no ambiente indicado. | Próxima sugestão: 15.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15 TypeScript: contratos e runtime

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 15.01 · 15.02 · 15.03 · 15.04 · 15.06 · 15.07 · 15.08 · 15.09 · 15.10 · 15.11

15.01 Leia o erro do compilador

READ · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.compiler_messages | Requer: functions.return, objects.identity | Introduz: ts.compiler_messages

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 2 · depuração 2 · arquivos 3 · leitura 5 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Em um formulário de reserva, classifique os seis diagnósticos do starter. Entregue a matriz completa e escreva um exemplo mínimo que mostre por que o tipo sozinho não comprova a forma de um JSON recebido.

Objetivo da atividade

Antes de consultar as dicas, registre uma hipótese para um dos seis diagnósticos e a evidência de código que espera ver.

Explicação e modelo mental

• Modelo mental: Um erro do compilador aponta uma afirmação que não foi provada pelo tipo; um valor externo ainda exige observação e validação em execução.
• Exemplo resolvido: Uma anotação de tipo para Booking orienta o checker, mas não analisa a string recebida do formulário; código de validação precisa observar o valor em runtime.
• Armadilha comum: Silenciar com any esconde a obrigação e pode deixar um acesso inválido ocorrer em Node.
• Por que importa: Ler o diagnóstico como evidência ausente ajuda a corrigir o contrato sem mascarar falhas de dados de clientes.

Material/arquivo inicial

diagnosticos.ts: // 15.01 · Formulário de reserva: seis diagnósticos para classificar.
// Configuração: strict (padrão desde o TypeScript 6) + noUncheckedIndexedAccess.
// Numa pasta sem tsconfig.json, rode npx tsc --noEmit --strict --noUncheckedIndexedAccess diagnosticos.ts
// (com tsconfig.json presente, o TypeScript 6+ exige --ignoreConfig). Depois preencha matriz.md.

type Reserva = { readonly id: string; nome: string; noites: number };
type EntradaFormulario = { nome: string; noites: string };

const reservas: Reserva[] = [{ id: "r1", nome: "Ana", noites: 2 }];

// 1. Leitura de posição de array
export const primeiroNome = reservas[0].nome;

// 2. Valor do formulário usado como número
export function noitesDoFormulario(entrada: EntradaFormulario): number {
const noites: number = entrada.noites;
return noites;
}

// 3. JSON recebido
export function checkIn(texto: string): string {
const dados: unknown = JSON.parse(texto);
return dados.checkIn;
}

// 4. Alteração de identificador
export function renomear(reserva: Reserva): void {
reserva.id = "r2";
}

// 5. Objeto literal com campo a mais
export const nova: Reserva = { id: "r3", nome: "Bia", noites: 1, cafe: true };

// 6. Função com caminho sem retorno
export function preco(noites: number): number {
if (noites > 0) {
return noites * 200;
}
}
| matriz.md: # Matriz de diagnósticos

Nº	Código TS	Linha	Causa	Prova que falta	Correção sem cast
1					
2					
3					
4					
5					
6					

O tipo não comprova o JSON

Escreva um exemplo mínimo em que o código compila e o valor recebido em runtime não tem a forma declarada.
| tsconfig.json: {"compilerOptions":{"strict":true,"noEmit":true,"noUncheckedIndexedAccess":true},"files":["diagnosticos.ts"]}

Exemplo e resultado público

para cada diagnóstico → linha/causa/prova ausente/correção sem cast

Bordas e erros publicados

erro em acesso array com noUncheckedIndexedAccess -> validar elemento; compilação não valida JSON runtime

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que evidência distingue uma incompatibilidade de tipos de um problema causado pelo valor recebido em execução?

• Um valor externo que não satisfaz a forma assumida pelo programa
• Uma propriedade ausente em um objeto recebido
 (Correta) Uma relação entre tipos que falha durante a checagem estática

Por que a resposta está certa

O typechecker compara tipos antes de executar o programa.

Armadilha da alternativa incorreta

Os outros casos dependem de dados em runtime e precisam de tratamento na fronteira.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

erro em acesso array com noUncheckedIndexedAccess → validar elemento; compilação não valida JSON runtime

Concepção errada e falha observável

Suprimir tudo com as any sem provar contrato. / Na entrada erro em acesso array com noUncheckedIndexedAccess, a saída observável não corresponde a validar elemento; a divergência é compatível com esta hipótese: Suprimir tudo com as any sem provar contrato.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Suprimir tudo com as any sem provar contrato.

Pistas graduais

1. Leia tipo conhecido na linha. 2. Identifique o caso que o compilador não consegue excluir. 3. Prove via guarda ou ajuste modelo, não esconda erro.

Solução de referência explicada

Tabela para seis mensagens com tipo real, condição não provada e correção mínima; distinção de runtime explícita.

Transferência em outro contexto

Analise um diagnóstico novo na integração do formulário de reserva. Registre o que o compilador conhece, qual evidência falta e um caso autoral que diferencie a correção estática de uma falha runtime.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, executed examples on Node.js 24 LTS. Separate a compiler diagnostic from a runtime check: tsc does not execute or validate external data. | Próxima sugestão: 15.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.02 Parsons tipado

PARSONS · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.typed_config | Requer: functions.return, objects.identity | Introduz: ts.typed_config

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Implemente a fronteira de configuração de uma página de catálogo: reordene parseConfig para aceitar unknown, validar e converter port, aplicar o default documentado e só então devolver Config. Inclua seu próprio caso inválido e o resultado observado.

Objetivo da atividade

Liste quais fatos observáveis sobre uma configuração externa precisam ser verificados antes de confiar no tipo declarado.

Explicação e modelo mental

• Modelo mental: Uma anotação descreve um valor já provado; ela não converte nem valida a entrada desconhecida.
• Exemplo resolvido: Ao validar preferências de notificação, confirme o tipo do campo e converta um intervalo textual para número antes de devolver a configuração.
• Armadilha comum: Tratar as Config ou um parâmetro anotado como Config como se validassem o JSON.
• Por que importa: Configuração malformada deve falhar na borda com motivo compreensível, antes de afetar navegação ou limites da página.

Material/arquivo inicial

config.ts: // 15.02 · Fronteira de configuração da página de catálogo.
// Contrato: port ausente -> 3000 (default documentado); texto com inteiro de 1 a 65535 -> number;
// qualquer outro valor -> RangeError. Config é o resultado; não é o parser.
// Monte parseConfig com os blocos abaixo. Um bloco não deve ser usado.

export type Config = { port: number };

export function parseConfig(raw: unknown): Config {
// cole aqui os blocos, na ordem escolhida
}

/* Blocos (embaralhados)
[P] const texto = "port" in raw ? raw.port : undefined;
[Q] return { port };
[R] if (typeof raw !== "object" || raw === null) throw new TypeError("config precisa ser um objeto");
[S] return raw as Config;
[T] if (!Number.isInteger(port) || port < 1 || port > 65535) throw new RangeError(porta inválida: ${String(texto)});
[U] if (texto === undefined) return { port: 3000 };
[V] const port = typeof texto === "string" && /^\d+$/.test(texto) ? Number(texto) : Number.NaN;
*/

// Casos para conferir depois de montar:
// parseConfig({ port: "8080" }) -> { port: 8080 }
// parseConfig({}) -> { port: 3000 }
// parseConfig({ port: "abc" }) -> RangeError
// Seu caso inválido: ___ -> resultado observado: ___
| tsconfig.json: {"compilerOptions":{"target":"es2022","strict":true,"noEmit":true},"files":["config.ts"]}

Exemplo e resultado público

parseConfig({port:'8080'}) → {port:8080}; tsc strict -> sem erros

Bordas e erros publicados

port='abc' -> erro de validação runtime; valor ausente -> default; tipo Config não deve ser tratado como parser

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que uma fronteira que recebe dados externos deve começar tratando o valor como unknown?

 (Correta) Porque é preciso verificar e converter o valor em runtime antes de tratá-lo como um tipo de domínio.
• Porque unknown converte automaticamente os dados para o formato esperado.
• Porque anotar o parâmetro com um tipo de domínio garante que a entrada externa já tenha esse formato.

Por que a resposta está certa

A checagem estática não valida dados externos; uma fronteira precisa estabelecer a forma do valor.

Armadilha da alternativa incorreta

Uma anotação não transforma a entrada. A aplicação precisa inspecioná-la e validá-la em execução.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

port='abc' → erro de validação runtime; valor ausente -> default; tipo Config não deve ser tratado como parser

Concepção errada e falha observável

Declarar Config antes e achar que anotação valida entrada externa. / Na entrada port='abc', a saída observável não corresponde a erro de validação runtime; a divergência é compatível com esta hipótese: Declarar Config antes e achar que anotação valida entrada externa.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Declarar Config antes e achar que anotação valida entrada externa.

Pistas graduais

1. Entrada começa como unknown. 2. Valide forma e converta porta. 3. Anote saída pública Config após prova.

Solução de referência explicada

Ordem: definir contrato, aceitar unknown, guards, converter/validar, retornar Config; tsc strict e casos runtime.

Transferência em outro contexto

Projete a fronteira de configuração para limites de paginação: escreva o tipo de saída, uma entrada aceita e uma entrada rejeitada escolhida por você, com evidência da validação.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict em Node.js 24 LTS. Config é o tipo de saída; validação, defaults e conversão acontecem em runtime. | Próxima sugestão: 15.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.03 Complete as anotações mínimas

FADED · 16 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.0/5

Competência: ts.inference_public_api | Requer: functions.return, objects.identity | Introduz: ts.inference_public_api

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 1 · depuração 1 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Revise a API exportada criarUsuario de uma tela de perfil. Escreva assinatura e uma chamada válida e uma inválida; explique por que uma variável local simples pode continuar inferida.

Objetivo da atividade

Registre a assinatura exportada que você espera antes de decidir quais locais precisam de anotação.

Explicação e modelo mental

• Modelo mental: Anotações são mais úteis nas fronteiras públicas e entradas sem contexto; o compilador infere muitos valores locais.
• Exemplo resolvido: Em uma função exportada de endereços, documentar a fronteira ajuda consumidores a chamá-la; valores locais derivados de expressões simples ainda podem ser inferidos.
• Armadilha comum: Anotar tudo como any remove a proteção; anotar cada local simples acrescenta ruído sem melhorar a fronteira.
• Por que importa: Um contrato explícito torna os módulos consumidores mais previsíveis e reduz divergências na API de perfil.

Material/arquivo inicial

usuario.ts: // 15.03 · API exportada da tela de perfil.
// Anote só o contrato público: os parâmetros e o retorno de criarUsuario.
// A constante local pode continuar inferida; explique por quê.

```
export type Usuario = { nome: string; idade: number; criadoEm: string };

const IDADE_MINIMA = 13;

export function criarUsuario(nome /* : ___ */, idade /* : ___ */) /* : ___ */ {
const limpo = nome.trim();
if (idade < IDADE_MINIMA) {
throw new RangeError("idade abaixo do mínimo");
}
return { nome: limpo, idade, criadoEm: new Date().toISOString() };
}

// Uma chamada válida:
// criarUsuario(___);
// Uma chamada que deve falhar no compilador (por exemplo, idade como texto):
// criarUsuario(___);
```

| tsconfig.json: {"compilerOptions":{"target":"es2022","strict":true,"noEmit":true},"files":["usuario.ts"]}

Exemplo e resultado público

anotar parâmetros e retorno público → função chama com string/number; locais inferidos

Bordas e erros publicados

chamada com idade string -> erro TS; retorno declarado incompatível -> erro TS; não anotar cada let trivial

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao declarar uma variável local sem tipo explícito, de onde TypeScript obtém a maior parte da informação para inferir seu tipo estático?

• Do nome escolhido para a variável, conforme palavras como count ou user.
 (Correta) Da expressão inicializadora e das operações de fluxo de controle aplicáveis à variável.

• De inspecionar os valores que a variável terá em produção depois da compilação.

Por que a resposta está certa

Correto. A inferência usa informação disponível no código-fonte e no fluxo de tipos.

Armadilha da alternativa incorreta

O compilador não deduz contratos pelo nome nem observa a execução futura do programa.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

chamada com idade string → erro TS; retorno declarado incompatível -> erro TS; não anotar cada let trivial

Concepção errada e falha observável

Anotar tudo como any ou confundir inferência de local com contrato de export. / Na entrada chamada com idade string, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Anotar tudo como any ou confundir inferência de local com contrato de export.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Anotar tudo como any ou confundir inferência de local com contrato de export.

Pistas graduais

1. Marque fronteiras exportadas. 2. Anote entradas sem contexto e saída quando clarifica API. 3. Deixe literais/intermediários inferidos.

Solução de referência explicada

export function criarUsuario(nome:string,idade:number):Usuario, validando invariantes runtime quando exigidas; locais inferidos.

Transferência em outro contexto

Defina a assinatura pública de uma função de paginação consumida por outro módulo. Entregue a chamada válida, uma chamada incompatível e a razão para manter inferidos os detalhes locais.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; Node.js 24 LTS. tsc checa chamadas e contrato exportado; invariantes de domínio que dependem do valor requerem validação runtime. | Próxima sugestão: 15.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.04 Any contagioso

MODIFY · 20 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.unknown_narrowing | Requer: functions.return, objects.identity | Introduz: ts.unknown_narrowing

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Corrija o leitor de perfil para receber payload desconhecido. Entregue o trecho com guardas e crie seus próprios casos de sucesso e falha, explicando a evidência de cada resultado.

Objetivo da atividade

Escolha um payload próprio e anote a sequência mínima de provas necessária antes de usar uma propriedade.

Explicação e modelo mental

• Modelo mental: unknown exige uma prova antes do acesso; cada guarda reduz possibilidades tanto para o compilador quanto para o caminho executado.
• Exemplo resolvido: Para uma resposta de estoque, confirme que o valor é um objeto não nulo e que quantidade é número antes de calcular disponibilidade.
• Armadilha comum: Trocar any por unknown e imediatamente fazer cast apenas esconde o mesmo acesso inseguro.
• Por que importa: Respostas de API mudam e podem estar incompletas; uma fronteira explícita evita que a tela de perfil quebre em dados inesperados.

Material/arquivo inicial

perfil.ts: // 15.04 · Leitor de perfil que recebe payload externo.
// Hoje any desliga a verificação: o compilador aceita qualquer acesso.
// Troque any por unknown, prove cada passo com guardas e lance TypeError
// com mensagem clara quando a forma não for a esperada.

export function ler(payload: any): string {
return payload.user.name.toUpperCase();
}

// Casos para conferir:
// ler({ user: { name: "Ana" } }) -> "ANA"
// ler(null) -> TypeError com a sua mensagem, não "Cannot read properties of null"
// ler({}) -> TypeError
// ler({ user: { name: 42 } }) -> TypeError
// Seus casos: ___
| tsconfig.json: {"compilerOptions":{"target":"es2022","strict":true,"noEmit":true},"files":["perfil.ts"]}

Exemplo e resultado público

{user:{name:'Ana'}} → 'ANA'; tsc strict -> sem as any

Bordas e erros publicados

null, user ausente, name número -> erro de validação; não explode por acesso prematuro

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que acontece em runtime quando um tipo TypeScript como unknown é adicionado ou alterado?

• O JavaScript percorre automaticamente o objeto e valida as propriedades descritas pelo tipo.
• O compilador insere uma conversão que transforma o valor em um objeto do tipo declarado.
 (Correta) Nada por causa da anotação em si; tipos são apagados, e só código de verificação explícito altera o comportamento runtime.

Por que a resposta está certa

Correto. Anotações TypeScript orientam o typecheck; elas não criam validação runtime.

Armadilha da alternativa incorreta

Separe as garantias estáticas do código JavaScript que realmente executa.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

null, user ausente, name número → erro de validação; não explode por acesso prematuro

Concepção errada e falha observável

Trocar any por unknown e imediatamente usar cast sem verificar. / Na entrada null, user ausente, name número, a saída observável não corresponde a erro de validação; a divergência é compatível com esta hipótese: Trocar any por unknown e imediatamente usar cast sem verificar.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Trocar any por unknown e imediatamente usar cast sem verificar.

Pistas graduais

1. typeof payload === 'object' e payload !== null. 2. Cheque propriedade com in e valide objeto interno. 3. Valide name como string antes de usar método.

Solução de referência explicada

Receba unknown, use guardas de objeto/propriedade/typeof e só depois retorne uppercase; erro definido para forma inválida.

Transferência em outro contexto

Valide uma resposta de API que contém email antes de construir o modelo de domínio. Entregue um guard e casos autorais para campo ausente, tipo incompatível e valor aceito.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict em Node.js 24 LTS. Narrowing é verificado no compile-time; o guard também precisa testar o valor em runtime. | Próxima sugestão: 15.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.06 Estrutural vs nominal

COMPARE · 20 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.structural_typing | Requer: functions.return, objects.identity | Introduz: ts.structural_typing

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 2 · depuração 3 · arquivos 3 · leitura 5 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Compare interface A { id: string } com interface B { id: string }: preveja e confirme atribuições nos dois sentidos. Em seguida, altere B.id para number e registre o diagnóstico. Por fim, compare um objeto literal fresco com propriedade extra a uma variável intermediária com a mesma propriedade.

Objetivo da atividade

Preveja a compatibilidade entre duas interfaces de nomes diferentes e mesma forma; depois altere um membro e compare com a regra de excesso em objeto literal.

Explicação e modelo mental

• Modelo mental: TypeScript compara principalmente a forma estrutural: nomes distintos não impedem atribuição quando os membros requeridos são compatíveis. Uma propriedade incompatível quebra essa relação. A verificação de propriedade extra para literal fresco é uma checagem adicional de contexto; atribuição via variável intermediária segue a compatibilidade estrutural.
• Exemplo resolvido: interface Produto { id: string } e interface Registro { id: string } podem descrever formas compatíveis apesar dos nomes distintos. Se um membro obrigatório mudar para um tipo incompatível, a atribuição deixa de funcionar; um literal fresco também pode receber um diagnóstico contextual por propriedade extra.
• Armadilha comum: Concluir que nomes distintos tornam interfaces nominais, ou tratar a verificação de propriedade extra em literal fresco como regra geral de compatibilidade.
• Por que importa: Compatibilidade estrutural permite integrar modelos de APIs diferentes quando suas formas coincidem; as checagens contextuais do compilador ajudam a detectar objetos escritos com campos inesperados.

Material/arquivo inicial

formas.ts: // 15.06 · Tipagem estrutural versus nominal.
// Para cada passo, escreva a previsão antes de rodar npx tsc --noEmit formas.ts.

```
interface A { id: string }
interface B { id: string }

declare const a: A;
declare const b: B;

// Passo 1: atribuição nos dois sentidos. Previsão: ___
export const deAparaB: B = a;
export const deBparaA: A = b;

// Passo 2: mude B.id para number e anote o diagnóstico. Previsão: ___

// Passo 3: literal fresco versus variável intermediária. Previsão: ___
export const fresco: A = { id: "x", extra: true };
const intermediario = { id: "x", extra: true };
export const viaVariavel: A = intermediario;

// Passo 4: membro private compara a declaração de origem, não só a forma. Previsão: ___
class MedidorA { private tag = "medidor"; valor = 0; }
class MedidorB { private tag = "medidor"; valor = 0; }
class Filho extends MedidorA {}
export const cruzado: MedidorB = new MedidorA();
export const herdado: MedidorA = new Filho();
// private só existe na verificação; para esconder o campo em runtime, use #tag.
```

Exemplo e resultado público

A{id:string} e B{id:string}; depois B{id:number} → As formas iniciais são compatíveis nos dois sentidos; após a mudança, a propriedade id incompatível causa erro de atribuição. | Atribua {id:'x',extra:true} diretamente a A e faça a mesma atribuição via variável intermediária → O literal fresco recebe excess-property diagnostic; a variável intermediária permanece atribuível por compatibilidade estrutural.

Bordas e erros publicados

MeterA -> MeterB -> erro TS pois private _tag tem declarações de origem distintas; Child compartilha a origem; private TS não vira #private de runtime

Teste público e aprovação visível

Exija previsão e confirmação de cada caso no typecheck, com justificativa que distinga forma estrutural de excess-property checking.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

As interfaces A e B com a mesma propriedade pública são compatíveis pela forma (tipagem estrutural). Em classes, membros private só são compatíveis quando vêm da mesma declaração de origem; uma classe derivada preserva essa origem. Interfaces e aliases type também podem descrever a mesma forma; suas diferenças ficam para a etapa 17.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

No modelo estrutural do TypeScript, o que geralmente permite compatibilidade entre dois tipos de objeto?

 (Correta) A compatibilidade dos membros públicos requeridos, mesmo que os tipos tenham nomes diferentes.
• A igualdade dos nomes dos tipos, independentemente dos membros declarados.
• A declaração de ambos os objetos pela mesma interface nomeada.

Por que a resposta está certa

A compatibilidade estrutural considera a forma dos valores, não apenas o nome do tipo.

Armadilha da alternativa incorreta

No modelo estrutural, o nome por si só não determina a compatibilidade dos objetos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

MeterA → MeterB -> erro TS pois private _tag tem declarações de origem distintas; Child compartilha a origem; private TS não vira #private de runtime

Concepção errada e falha observável

Achar que toda classe TypeScript é nominal, ou que implements muda a compatibilidade estrutural. / Na entrada MeterA, a saída observável não corresponde a MeterB -> erro TS pois private _tag tem declarações de origem distintas; a divergência é compatível com esta hipótese: Achar que toda classe TypeScript é nominal, ou que implements muda a compatibilidade estrutural.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que toda classe TypeScript é nominal, ou que implements muda a compatibilidade estrutural.

Pistas graduais

1. Compare propriedades públicas antes de nomes. 2. Membros private/protected de classe precisam ter a mesma origem de declaração para compatibilidade. 3. implements checa o contrato sem mudar o tipo; private TS é apagado, #private é runtime.

Solução de referência explicada

Compare duas interfaces e classes pequenas: interfaces seguem forma; classes com private/protected só são compatíveis quando o membro vem da mesma classe-base. Diga que classe implements A continua compatível com A e que o modificador TS private não é privacidade runtime.

Transferência em outro contexto

Compare dois DTOs JavaScript que têm os mesmos membros públicos mas representam domínios diferentes. Proponha uma marca de domínio e demonstre separadamente o que muda no typecheck e na validação runtime.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Target: TypeScript 7.0 strict; examples run on Node.js 24 LTS. Structural assignability and private/protected origin are compile-time rules. TypeScript private is erased; ECMAScript #private enforces runtime privacy. | Próxima sugestão: 15.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.07 Flags strict extras

MODIFY · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.strict_flags | Requer: functions.return, objects.identity | Introduz: ts.strict_flags

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Aplique as flags estritas a um formulário de catálogo que contém listas e campos opcionais. Entregue correções compiláveis e crie casos próprios que demonstrem o contrato de ausência escolhido pelo produto.

Objetivo da atividade

Preveja separadamente o tipo de array[0] e se {apelido: undefined} satisfaz o tipo opcional do starter.

Explicação e modelo mental

• Modelo mental: Uma leitura indexada pode faltar; uma propriedade opcional ausente difere de uma propriedade presente cujo valor é undefined.
• Exemplo resolvido: As flags tornam explícitas suposições sobre presença; modele os estados que o produto aceita e prove uma leitura segura antes de usar seu valor.
• Armadilha comum: Desligar as flags ou espalhar ! remove o aviso sem provar que o item existe.
• Por que importa: Formulários e listas reais contêm campos opcionais e vazios; o tipo precisa descrever a semântica que o produto aceita.

Material/arquivo inicial

catalogo.ts: // 15.07 · Formulário de catálogo sob flags estritas extras (veja tsconfig.json).
// Rode npx tsc -p ., corrija sem ! nem cast e decida o contrato de ausência do produto.

```
export type Produto = { nome: string; apelido?: string };

export function primeiroNome(lista: string[]): string {
return lista[0].toUpperCase();
}

export function semApelido(produto: Produto): Produto {
return { ...produto, apelido: undefined };
}

export function rotulo(produto: Produto): string {
// Compila, mas ! só esconde a ausência: troque por uma decisão explícita.
return produto.apelido!.trim();
}

// Casos próprios para o contrato de ausência:
// primeiroNome([]) -> ___
// semApelido({ nome: "Caneca", apelido: "C" }) -> ___
// rotulo({ nome: "Caneca" }) -> ___
```

| tsconfig.json: {

```
"compilerOptions": {
"target": "es2024",
"module": "preserve",
"strict": true,
"noUncheckedIndexedAccess": true,
"exactOptionalPropertyTypes": true,
"noEmit": true
},
"files": ["catalogo.ts"]
}
```

Exemplo e resultado público

array[0] → string|undefined e guard; apelido ausente -> válido

Bordas e erros publicados

{apelido:undefined} -> erro com exactOptionalPropertyTypes sem union explícita; ! indiscriminado -> rejeitado

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que as flags estritas ajudam a expressar sobre dados opcionais e indexados?

• Que propriedades opcionais sempre existem em runtime
 (Correta) Quais casos de ausência precisam ser tratados ou modelados pelo contrato
• Que o compilador converterá automaticamente valores ausentes

Por que a resposta está certa

As flags pedem que o contrato e o código reconheçam formas que podem estar ausentes.

Armadilha da alternativa incorreta

Flags não inserem dados nem convertem valores durante a execução.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

{apelido:undefined} → erro com exactOptionalPropertyTypes sem union explícita; ! indiscriminado -> rejeitado

Concepção errada e falha observável

Desligar flags ou espalhar ! para silenciar erros. / Na entrada {apelido:undefined}, a saída observável não corresponde a erro com exactOptionalPropertyTypes sem union explícita; a divergência é compatível com esta hipótese: Desligar flags ou espalhar ! para silenciar erros.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Desligar flags ou espalhar ! para silenciar erros.

Pistas graduais

1. Trate acesso indexado como possivelmente ausente. 2. Ausência de propriedade difere de presente com undefined. 3. Modele opcionalidade real ou forneça fallback.

Solução de referência explicada

Corrija acesso com guard/default; omita propriedade opcional quando ausente ou declare string|undefined se presença com undefined for válida. O curso compila em TypeScript 7.0; noUncheckedIndexedAccess existe desde TypeScript 4.1 e exactOptionalPropertyTypes desde TypeScript 4.4.

Transferência em outro contexto

Ative as flags estritas num formulário de preferências. Escolha os casos de ausência que o produto permite, modele-os e mostre evidência para um item indexado e um campo opcional.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 é o baseline deste conjunto e inclui ambas as flags; noUncheckedIndexedAccess requer 4.1+, exactOptionalPropertyTypes 4.4+. | Próxima sugestão: 15.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.08 Microprojeto - migrar utilitários

PROJECT · 90 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.5/5

Competência: ts.migration_strict | Requer: functions.return, objects.identity | Introduz: ts.migration_strict

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 4 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Migre o módulo de utilitários de paginação do produto mantendo sua API. Registre uma função por vez, os diagnósticos resolvidos, as chamadas externas preservadas e evidência de execução para entradas válidas e inválidas.

Objetivo da atividade

Antes de alterar a API, registre um comportamento observável e o caso runtime que comparará as versões antiga e nova.

Explicação e modelo mental

• Modelo mental: Migração segura congela comportamento e assinaturas antes de adicionar tipos; cada anotação documenta uma fronteira real.
• Exemplo resolvido: Ao migrar um filtro de relatórios, preserve a assinatura pública e compare a quantidade de itens e o erro conhecido antes e depois das anotações.
• Armadilha comum: Mudar semântica ou aceitar casts para fazer o typecheck passar pode deixar consumidores existentes com outro comportamento.
• Por que importa: Uma migração incremental protege a tela que já usa esses utilitários e deixa claro quais garantias vêm do compilador e quais vêm de testes de execução.

Material/arquivo inicial

paginacao.js: // 15.08 · Utilitários de paginação (versão JavaScript da etapa 13).
// Migre para paginacao.ts com TypeScript strict, uma função por vez, mantendo exports e erros.

```
export function clamp(valor, minimo, maximo) {
if (minimo > maximo) throw new RangeError("minimo maior que maximo");
return Math.min(Math.max(valor, minimo), maximo);
}

export function unique(valores) {
return [...new Set(valores)];
}

export function paginar(valores, pagina, tamanho) {
if (!Number.isInteger(pagina) || pagina < 1) throw new RangeError("pagina deve ser inteiro >= 1");
if (!Number.isInteger(tamanho) || tamanho < 1) throw new RangeError("tamanho deve ser inteiro >= 1");
const inicio = (pagina - 1) * tamanho;
return valores.slice(inicio, inicio + tamanho);
}
```

| paginacao.test.js: // Suíte de runtime: deve passar antes e depois da migração.
// Antes: node --test paginacao.test.js. Depois: troque o import para "./paginacao.ts" e rode de novo.

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { clamp, unique, paginar } from "./paginacao.js";

test("exemplos publicados", () => {
assert.equal(clamp(2, 0, 3), 2);
assert.deepEqual(unique([1, 1, 2]), [1, 2]);
assert.deepEqual(paginar([1, 2, 3], 2, 2), [3]);
});

test("erros documentados", () => {
assert.throws(() => clamp(1, 5, 0), RangeError);
assert.throws(() => paginar([1], 0, 2), RangeError);
assert.throws(() => paginar([1], 1, 0), RangeError);
});

test("não muta a entrada e preserva a ordem", () => {
const entrada = [3, 1, 3, Number.NaN, Number.NaN];
assert.deepEqual(unique(entrada), [3, 1, Number.NaN]);
assert.deepEqual(entrada, [3, 1, 3, Number.NaN, Number.NaN]);
});
```

| tsconfig.json: {

```
"compilerOptions": {
"target": "es2022",
"module": "nodenext",
"strict": true,
"noUncheckedIndexedAccess": true,
"erasableSyntaxOnly": true,
"verbatimModuleSyntax": true,
"allowImportingTsExtensions": true,
"noEmit": true,
"types": ["node"]
}
}
```

| package.json: {

```
"name": "devpath-15-08",
```

"version": "1.0.0",
"private": true,
"type": "module",
"engines": {
"node": ">=24.0.0"
},
"scripts": {
"test": "node --test paginacao.test.js",
"typecheck": "tsc --noEmit -p tsconfig.json"
},
"devDependencies": {
"@types/node": "24.19.1",
"typescript": "6.0.3"
}
}
| assessment.tsconfig.json:
{"compilerOptions":{"target":"es2022","strict":true,"noEmit":true,"noUncheckedIndexedAccess":true},"files":["paginacao.ts"]}
| package-lock.json: {
"name": "devpath-15-08",
"version": "1.0.0",
"lockfileVersion": 3,
"packages": {
"": {
"name": "devpath-15-08",
"version": "1.0.0",
"devDependencies": {
"@types/node": "24.19.1",
"typescript": "6.0.3"
}
},
"node_modules/@types/node": {
"version": "24.19.1",
"resolved": "https://registry.npmjs.org/@types/node/-/node-24.19.1.tgz",
"integrity":
"sha512-aS3/DG0oM05K0RIXXP+hKjinGG5IgSSVGzswZxW3O0sS3pH4/fycXundUC9XsszgKCk4gHXylTEK6hyFxVxnoQ==",
"dev": true,
"license": "MIT",
"dependencies": {
"undici-types": ">=7.24.0 <7.24.7"
}
},
"node_modules/typescript": {
"version": "6.0.3",
"resolved": "https://registry.npmjs.org/typescript/-/typescript-6.0.3.tgz",
"integrity":
"sha512-y2TvuxSZPDyQakkFRPZHKFm+KKVqIisdg9/CZwm9ftvKXLP8NRWj38/ODjNbr43SsoXqNuAisEf1GdCxqWcdBw==",
"dev": true,
"license": "Apache-2.0",
"bin": {
"tsc": "bin/tsc",
"tsserver": "bin/tsserver"
},
"engines": {
"node": ">=14.17"
}
},

"node_modules/undici-types": {
"version": "7.24.6",
"resolved": "https://registry.npmjs.org/undici-types/-/undici-types-7.24.6.tgz",
"integrity":
"sha512-WRNW+sJgj5OBN4/0JpHFqtqzhpbnV0GuB+OozA9gCL7a993SmU+1JBZCzLNxYsbMfIeDL+ITsphD5jN5N+n0zg==",
"dev": true,
"license": "MIT"
}
}
}

Exemplo e resultado público

mesma suíte de runtime passa; tsc --noEmit strict → sem erro; API pública equivalente

Bordas e erros publicados

entrada inválida -> mesmos erros documentados; nenhum as any; declarations representam exports

Teste público e aprovação visível

Resultado observável: mesma suíte de runtime passa; tsc --noEmit strict -> sem erro; API pública equivalente. Borda a conferir: entrada inválida -> mesmos erros documentados; nenhum as any; declarations representam exports. Entrega esperada: Migre utilitários por função, preserve semântica/erros e publique tipos; relatório lista casts eliminados e mudanças de API se inevitáveis.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que demonstra progresso controlado ao migrar um módulo de utilitários?

• Converter todos os arquivos juntos e avaliar o resultado somente no final.
• Ocultar os diagnósticos restantes sem relacioná-los aos módulos ou às chamadas.
 (Correta) Fazer mudanças localizadas, revisar as chamadas afetadas e registrar evidência em cada etapa.

Por que a resposta está certa

Etapas pequenas tornam as mudanças e seus efeitos mais fáceis de acompanhar.

Armadilha da alternativa incorreta

Uma conversão em lote ou sem rastrear diagnósticos dificulta localizar problemas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada inválida → mesmos erros documentados; nenhum as any; declarations representam exports

Concepção errada e falha observável

Modificar comportamento para satisfazer tipo e quebrar consumidores. / Na entrada entrada inválida, a saída observável não corresponde a mesmos erros documentados; a divergência é compatível com esta hipótese: Modificar comportamento para satisfazer tipo e quebrar consumidores.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Modificar comportamento para satisfazer tipo e quebrar consumidores.

Pistas graduais

1. Congele testes existentes. 2. Anote fronteiras e resolva inferência interna. 3. Rode typecheck e regressões de runtime.

Solução de referência explicada

Migre utilitários por função, preserve semântica/erros e publique tipos; relatório lista casts eliminados e mudanças de API se inevitáveis.

Transferência em outro contexto

Migre um módulo de datas sem alterar seu contrato externo. Registre uma assinatura antes/depois, decisões de tipos e evidência runtime para casos existentes.

Evidência de domínio

Resultado observável: mesma suíte de runtime passa; tsc --noEmit strict -> sem erro; API pública equivalente. Borda a conferir: entrada inválida -> mesmos erros documentados; nenhum as any; declarations representam exports. Entrega esperada: Migre utilitários por função, preserve semântica/erros e publique tipos; relatório lista casts eliminados e mudanças de API se inevitáveis. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Migration target: TypeScript 7.0 strict, executed with Node.js 24 LTS. Type declarations and tsc validate static contracts; runtime regression cases validate behavior and error semantics. | Próxima sugestão: 15.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.09 Readonly observa mutação

PREDICT · 16 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.readonly_alias | Requer: functions.return, objects.identity | Introduz: ts.readonly_alias

Eixos: novidade 3 · sintaxe 1 · composição 2 · estado 4 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Uma tela de inventário mantém duas referências para dados compartilhados. Preveja o efeito de uma mutação feita por uma delas e escreva uma observação reproduzível que diferencie tipo, identidade e runtime.

Objetivo da atividade

Antes de rodar o exemplo, anote quantos itens a tela verá após dados.itens.push(2).

Explicação e modelo mental

• Modelo mental: Readonly limita escrita pela referência tipada; não clona nem congela profundamente o objeto compartilhado.
• Exemplo resolvido: Readonly limita escrita pela referência tipada; aliases ainda podem observar o mesmo objeto mutável. Isolamento requer uma decisão explícita sobre cópia ou ownership.
• Armadilha comum: Ler Readonly como garantia de imutabilidade transitiva ou acreditar que Object.freeze é profundo.
• Por que importa: Uma API que promete visão somente leitura ainda pode observar alterações feitas por outro dono do estado; ownership precisa ser explícito.

Material/arquivo inicial

inventario.ts: // 15.09 · Duas referências para o mesmo inventário.
// Escreva cada previsão antes de rodar node inventario.ts (o Node.js 24 remove os tipos).

```
const dados = { itens: [1] };
const vista: { readonly itens: number[] } = dados;

// Previsão 1: o que a linha seguinte mostra depois do push feito por dados? ___
dados.itens.push(2);
console.log("vista.itens", vista.itens);

// Previsão 2: esta linha compila? O que acontece em runtime? ___
vista.itens.push(3);

// Previsão 3: esta linha compila? ___
// vista.itens = [];

// Previsão 4: as duas referências apontam para o mesmo objeto? ___
console.log("mesmo objeto?", dados === vista, dados.itens);

// Experimento: compare com Object.freeze(dados) (também raso) e explique a diferença
// entre a visão tipada, a identidade e o objeto em runtime.
```

Exemplo e resultado público

vista.itens após push → [1,2]; readonly superficial não congela objeto runtime

Bordas e erros publicados

vista.itens.push(3) -> compila porque readonly é superficial: protege a propriedade itens, não o array; Object.freeze também é raso

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Duas variáveis apontam para o mesmo objeto. O que se deve prever sobre uma mutação feita por uma delas?

 (Correta) As duas referências continuam apontando para o mesmo objeto, então a alteração pode ser observada pela outra.
• Atribuir o objeto à segunda variável cria automaticamente uma cópia independente.
• A identidade do objeto muda sempre que uma propriedade é alterada.

Por que a resposta está certa

Atribuir uma referência não implica clonar o objeto.

Armadilha da alternativa incorreta

Considere identidade e aliasing: duas referências podem apontar para o mesmo valor mutável.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

vista.itens.push(3) → compila porque readonly é superficial: protege a propriedade itens, não o array; Object.freeze também é raso

Concepção errada e falha observável

Achar que readonly cria cópia ou freeze profundo. / Na entrada vista.itens.push(3), a saída observável não corresponde a compila porque readonly é superficial: protege a propriedade itens, não o array; a divergência é compatível com esta hipótese: Achar que readonly cria cópia ou freeze profundo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que readonly cria cópia ou freeze profundo.

Pistas graduais

1. Compare referência, tipo e comportamento runtime. 2. Readonly bloqueia escrita via aquela visão, conforme nível tipado. 3. Teste mutação via alias gravável e Object.freeze separadamente.

Solução de referência explicada

Predição demonstra alias compartilhado; explique superficialidade e escolha ReadonlyArray/deep policy ou cópia quando isolamento é requerido.

Transferência em outro contexto

Uma API recebe uma lista somente leitura enquanto outro módulo mantém uma referência mutável. Escreva uma previsão própria e uma observação que a confirme sem assumir cópia.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, Node.js 24 LTS. Readonly is a compile-time view; aliases and runtime mutation remain observable unless copying/freezing is deliberately applied. | Próxima sugestão: 15.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.10 Literal fresco versus variável

COMPARE · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.excess_property | Requer: functions.return, objects.identity | Introduz: ts.excess_property

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Use um tipo de cartão de produto e invente um campo adicional de extensão. Compare sua autoria como literal e como variável, depois explique como a aplicação deve tratar dados recebidos por fetch.

Objetivo da atividade

Escreva sua previsão para as duas chamadas do starter antes de checar o diagnóstico.

Explicação e modelo mental

• Modelo mental: A atribuição é estrutural; literais frescos recebem uma verificação extra de propriedades conhecidas para detectar enganos de autoria.
• Exemplo resolvido: Uma verificação de autoria pode ser mais estrita para um literal recém-escrito; a compatibilidade estrutural e a validação runtime são questões distintas.
• Armadilha comum: Tratar o diagnóstico do literal como schema universal que remove ou bloqueia campos externos.
• Por que importa: Entender a diferença evita depender de uma checagem de conveniência para proteger a API de dados não confiáveis.

Material/arquivo inicial

cartao.ts: // 15.10 · Cartão de produto: literal fresco versus variável.
// Troque destaque por um campo de extensão inventado por você antes de rodar npx tsc --noEmit cartao.ts.

```
type CartaoProduto = { nome: string; preco: number };

function exibir(cartao: CartaoProduto): string {
return ${cartao.nome}: R$ ${cartao.preco};
}

// A: literal fresco com campo extra. Previsão: ___
exibir({ nome: "Caneca", preco: 39.9, destaque: true });

// B: o mesmo objeto guardado antes numa variável. Previsão: ___
const emVariavel = { nome: "Caneca", preco: 39.9, destaque: true };
exibir(emVariavel);

// C: satisfies verifica o literal e mantém o tipo inferido. Previsão: ___
const verificado = { nome: "Caneca", preco: 39.9 } satisfies CartaoProduto;
exibir(verificado);

// D: dados de fetch. O compilador não enxerga o JSON; como a aplicação deve tratá-los?
// const recebido: unknown = await (await fetch("/api/produtos/1")).json();
```

Exemplo e resultado público

literal fresco com role extra → erro TS; variável v -> aceita pela compatibilidade estrutural

Bordas e erros publicados

entrada JSON externa com role -> TS não valida em runtime; usar satisfies no literal pode manter checagem da forma

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que comparar a checagem de um objeto literal direto com a atribuição de uma variável contendo o objeto?

• Para demonstrar que o TypeScript remove propriedades extras do objeto em runtime.
 (Correta) Para observar como a checagem de propriedades extras em literais frescos difere da compatibilidade estrutural de

uma variável.
• Para provar que variáveis sempre precisam declarar exatamente o mesmo nome de tipo.

Por que a resposta está certa

O contexto da expressão afeta a checagem de propriedades extras; isso não significa que o objeto seja filtrado.

Armadilha da alternativa incorreta

A comparação trata de regras de compatibilidade estática, não de remoção de propriedades em runtime.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada JSON externa com role → TS não valida em runtime; usar satisfies no literal pode manter checagem da forma

Concepção errada e falha observável

Tratar excess property check como schema ou proibição universal de campo extra. / Na entrada entrada JSON externa com role, a saída observável não corresponde a TS não valida em runtime; a divergência é compatível com esta hipótese: Tratar excess property check como schema ou proibição universal de campo extra.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar excess property check como schema ou proibição universal de campo extra.

Pistas graduais

1. Compare literal direto e variável. 2. TS é estrutural e usa checagem extra em literal fresco. 3. Valide dado externo em runtime se fronteira não confiável.

Solução de referência explicada

Mostre as duas compilações e explique que a checagem é auxílio de autoria, não filtro de propriedades em execução.

Transferência em outro contexto

Projete uma fronteira para um DTO obtido por fetch que pode conter campos adicionais. Separe o diagnóstico de autoria do modo como o produto valida dados não confiáveis.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; run on Node.js 24 LTS. Excess-property checking and satisfies are compile-time authoring checks; JSON values need runtime validation. | Próxima sugestão: 15.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

15.11 Compilou, mas falhou

DEBUG · 20 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.0/5

Competência: ts.runtime_boundary | Requer: functions.return, objects.identity | Introduz: ts.runtime_boundary

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 4 · depuração 5 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Investigue uma falha na tela de perfil causada por uma resposta externa. Remova o cast, escreva a validação de fronteira e anexe resultados para payloads próprios escolhidos por você.

Objetivo da atividade

Preveja qual erro acontece quando name é número apesar do cast as User.

Explicação e modelo mental

• Modelo mental: JSON começa como dado sem confiança; uma assertion altera a visão estática, mas não transforma o valor recebido.
• Exemplo resolvido: Parseie para unknown, confirme objeto e name string; só então construa User e chame toUpperCase.
• Armadilha comum: Concluir que tsc sem erros prova que uma resposta de servidor tem os campos declarados.
• Por que importa: A fronteira de rede precisa tratar separadamente sintaxe JSON, forma dos dados e uso de domínio para falhar com clareza.

Material/arquivo inicial

perfil.ts: // 15.11 · O código compila, mas a tela de perfil quebra.
// Reproduza a falha, remova o cast e valide o JSON na fronteira.

```
type User = { name: string };

export function nomeEmCaixaAlta(texto: string): string {
const user = JSON.parse(texto) as User;
return user.name.toUpperCase();
}

// Reprodução (descomente e rode node perfil.ts):
// console.log(nomeEmCaixaAlta('{"name":"Ana"}')); // esperado: ANA
// console.log(nomeEmCaixaAlta('{"name":42}')); // observado: ___
// console.log(nomeEmCaixaAlta('{')); // observado: ___
// Seus payloads: ___
```

| tsconfig.json: {"compilerOptions":{"target":"es2022","strict":true,"noEmit":true},"files":["perfil.ts"]}

Exemplo e resultado público

cast compila → runtime TypeError; guard name string -> erro de validação antes do método

Bordas e erros publicados

JSON.parse inválido -> SyntaxError; type assertion não transforma valor; tsc sem erros não prova dado externo

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como vários chamadores podem consumir dados externos sem duplicar suposições sobre o tipo de domínio?

• Fazer cada chamador declarar localmente que os dados externos já têm o tipo esperado.
• Confiar na compilação do projeto como evidência de que qualquer resposta externa segue o tipo.
 (Correta) Usar uma fronteira compartilhada que valida os dados e só então fornece o tipo de domínio aos chamadores.

Por que a resposta está certa

Uma fronteira explícita centraliza a validação antes do uso pelo restante do programa.

Armadilha da alternativa incorreta

Suposições locais e compilação não estabelecem a forma de um valor recebido em runtime.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

JSON.parse inválido → SyntaxError; type assertion não transforma valor; tsc sem erros não prova dado externo

Concepção errada e falha observável

Pensar que as User checa ou converte a resposta. / Na entrada JSON.parse inválido, a saída observável não corresponde a SyntaxError; a divergência é compatível com esta hipótese: Pensar que as User checa ou converte a resposta.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Pensar que as User checa ou converte a resposta.

Pistas graduais

1. Reproduza o erro com payload concreto. 2. Inspecione valor runtime e origem do dado. 3. Substitua cast por validação de fronteira.

Solução de referência explicada

Receba unknown após parse, valide objeto/name string e só então construa User; registre distinção compile/runtime.

Transferência em outro contexto

Um valor de preço chega do storage como texto. Proponha conversão e validação explícitas e entregue casos próprios para sucesso, formato inválido e valor fora das regras do produto.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, Node.js 24 LTS. as User is compile-time only; JSON.parse may throw at runtime and external fields require guards before use. | Próxima sugestão: 16.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16 Unions, narrowing e validação

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 16.01 · 16.02 · 16.03 · 16.09 · 16.04 · 16.05 · 16.10 · 16.06 · 16.07 · 16.08

16.01 Ache estados impossíveis

READ · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.discriminated_state | Requer: ts.basic | Introduz: ts.discriminated_state

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Modele fases mutuamente exclusivas para uma busca no catálogo. Entregue a union, exemplos autorais válidos e uma combinação que seu desenho deve impedir.

Objetivo da atividade

Liste uma combinação contraditória que o modelo antigo podia representar.

Explicação e modelo mental

• Modelo mental: Uma tag discrimina estados exclusivos; dados que só existem em um estado ficam junto daquela variante.
• Exemplo resolvido: Uma tag literal identifica a variante; campos obrigatórios ficam associados à fase que os usa, reduzindo combinações incoerentes no modelo.
• Armadilha comum: Manter booleans e propriedades opcionais independentes recria estados contraditórios.
• Por que importa: A UI de resultados pode renderizar cada fase sem adivinhar qual combinação de flags prevalece.

Material/arquivo inicial

search.ts: export type SearchState<T> = {kind:'idle'}|{kind:'loading';data?:T;message?:string}|{kind:'success';data:T}|{kind:'error';message:string}; export const idle:SearchState<never>={kind:'idle'}; export const loading:SearchState<never>={kind:'loading'}; export const success:SearchState<string[]>={kind:'success',data:['a']}; export const failure:SearchState<never>={kind:'error',message:'offline'};

Exemplo e resultado público

loading=true,error='x',data presente → inválido no modelo antigo; union impede combinação

Bordas e erros publicados

success sem data -> erro TS; error sem message -> erro TS; transição tipada mantém estados válidos

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que característica de uma propriedade permite que uma comparação de igualdade estreite uma união discriminada?

 (Correta) Ela tem valores literais distintos entre as variantes e está presente no contrato de cada uma.
• Ela é opcional em todas as variantes e recebe qualquer string em runtime.
• Ela é declarada como private em uma classe que implementa cada variante.

Por que a resposta está certa

Correto. Um discriminante comum com valores literais permite ao compilador identificar cada alternativa.

Armadilha da alternativa incorreta

Pense numa propriedade compartilhada cuja união de valores represente as alternativas possíveis.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

success sem data → erro TS; error sem message -> erro TS; transição tipada mantém estados válidos

Concepção errada e falha observável

Achar que booleans independentes representam estados mutuamente exclusivos. / Na entrada success sem data, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Achar que booleans independentes representam estados mutuamente exclusivos.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que booleans independentes representam estados mutuamente exclusivos.

Pistas graduais

1. Enumere combinações impossíveis. 2. Use tag discriminante literal. 3. Associe campos apenas à variante que os possui.

Solução de referência explicada

type State<T>={status:'idle'}|{status:'loading'}|{status:'success';data:T}|{status:'error';message:string}.

Transferência em outro contexto

Modele estados mutuamente exclusivos para uma busca do catálogo. Escolha campos próprios para cada fase e registre uma combinação inválida criada por você com a evidência do typecheck.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, Node.js 24 LTS. A union restringe combinações no compile-time; resposta HTTP ainda exige validação runtime. | Próxima sugestão: 16.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.02 Parsons de narrowing

PARSONS · 16 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.0/5

Competência: ts.narrowing_guards | Requer: ts.basic | Introduz: ts.narrowing_guards

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 1 · leitura 2 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Complete format para uma tela de preço usando string, number e null. Entregue código e saídas para valores escolhidos por você, incluindo um caso que teste sua escolha de guarda.

Objetivo da atividade

Para uma função de rótulo, anote como separar uma entrada textual, uma numérica e a ausência sem usar truthiness.

Explicação e modelo mental

• Modelo mental: Guards e retornos antecipados removem possibilidades do union com base em fatos observáveis.
• Exemplo resolvido: Em um campo de quantidade, compare null explicitamente antes de tratar o valor como número; assim a alternativa de ausência fica distinta de qualquer quantidade válida.
• Armadilha comum: Um teste de truthiness trata 0 como se fosse ausência.
• Por que importa: Preço zero, campo vazio e dado ausente têm significados de produto diferentes e não devem colapsar em uma ramificação.

Material/arquivo inicial

preco.ts: // 16.02 · Rótulo de preço com narrowing.
// Monte format com os blocos. Um bloco usa truthiness e quebra o contrato: deixe-o de fora.
// Contrato: null -> "—"; number -> String(x) (0 vira "0"); string -> texto sem espaços nas bordas.

export function format(x: string | number | null): string {
// cole aqui os blocos, na ordem escolhida
}

/* Blocos (embaralhados)
[K] return x.trim();
[L] if (!x) return "—";
[M] if (typeof x === "number") return String(x);
[N] if (x === null) return "—";
*/

// Saídas para conferir:
// format(null) -> "—"
// format(2) -> "2"
// format(0) -> "0"
// format(" a ") -> "a"
// Seus valores: ___

Exemplo e resultado público

format(null) → '—'; format(2) -> '2'; format(' a ') -> 'a'

Bordas e erros publicados

x=0 -> '0'; sem as/cast; typecheck confirma cada ramo

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao formatar uma união de tipos primitivos, por que ramificar com guardas de tipo explícitas?

• Para converter todas as variantes em uma só representação antes de verificar qual valor chegou.
 (Correta) Para estreitar cada variante e aplicar a lógica apropriada sem colapsar os casos em uma coerção genérica.
• Para dispensar a análise do caso de ausência porque o compilador escolhe a variante em runtime.

Por que a resposta está certa

Guardas explícitas permitem tratar cada alternativa segundo seu tipo.

Armadilha da alternativa incorreta

O compilador não escolhe nem valida a variante em runtime; a lógica precisa examinar o valor.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

x=0 → '0'; sem as/cast; typecheck confirma cada ramo

Concepção errada e falha observável

Usar truthiness e tratar 0 como ausente. / Na entrada x=0, a saída observável não corresponde a '0'; a divergência é compatível com esta hipótese: Usar truthiness e tratar 0 como ausente.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar truthiness e tratar 0 como ausente.

Pistas graduais

1. Primeiro cheque x === null. 2. typeof distingue number/string. 3. Após return, o tipo restante estreita sozinho.

Solução de referência explicada

Ordene null guard, number return String(x), string return x.trim(); compilação sem assertion.

Transferência em outro contexto

Estenda format para string | boolean | null numa tela de status. Defina saídas para exemplos escolhidos por você e mostre como cada guarda estreita o tipo.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; run examples on Node.js 24 LTS. Guards are checked statically and execute against the real input; avoid relying on truthiness for valid zero. | Próxima sugestão: 16.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.03 Complete a união

FADED · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.payment_union | Requer: ts.basic | Introduz: ts.payment_union

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Modele pagamentos Pix, cartão e boleto como uma união discriminada pela propriedade metodo. Mantenha obrigatórios os campos exclusivos de cada variante e escreva um handler switch que só leia os campos válidos daquele método.

Objetivo da atividade

Escolha um campo que só pertence ao Pix e anote em qual ramo ele pode ser lido.

Explicação e modelo mental

• Modelo mental: Cada alternativa de domínio é uma variante; seu discriminante permite acessar somente os dados válidos daquele método.
• Exemplo resolvido: Uma tag discriminante deixa o handler identificar a variante atual e acessar somente os campos definidos para aquela alternativa.
• Armadilha comum: Uma interface com todos os campos opcionais permite boleto sem linha ou Pix sem chave.
• Por que importa: Contratos específicos impedem que a tela de checkout apresente ou processe detalhes do método errado.

Material/arquivo inicial

pagamento.ts: // 16.03 · Pagamentos como union discriminada por metodo.
// Complete as lacunas: cada variante tem o discriminante literal e o campo exclusivo obrigatório.

```
type Pix = { metodo: "pix"; chave: string };
type Cartao = { metodo: ___; ultimos4: string };
type Boleto = { metodo: ___; ___: string };

export type Pagamento = Pix | Cartao | Boleto;

export function resumo(pagamento: Pagamento): string {
switch (pagamento.metodo) {
case "pix":
return Pix para ${pagamento.chave};
case ___:
return Cartão final ${pagamento.___};
case ___:
return Boleto ${pagamento.___};
}
}

// Depois de completar, estas linhas devem falhar no typecheck (descomente uma por vez):
// const semChave: Pagamento = { metodo: "pix" };
// const misturado: Pagamento = { metodo: "boleto", linha: "123", ultimos4: "4242" };
```

Exemplo e resultado público

pix com chave → válido; cartão com ultimos4 -> válido; boleto com linha -> válido

Bordas e erros publicados

pix sem chave -> erro TS; boleto com ultimos4 literal -> erro TS; handler switch estreita campos

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Antes de narrowing, o que o tipo A | B garante sobre um valor?

• O valor precisa conter simultaneamente todos os membros requeridos por A e por B.
• O valor não pode compartilhar membros entre A e B.

 (Correta) O valor é atribuível a pelo menos uma das alternativas, então só membros seguros para todas ficam disponíveis sem narrowing.

Por que a resposta está certa

Correto. Uma união representa alternativas; uma interseção exige satisfazer as duas formas.

Armadilha da alternativa incorreta

Compare o sentido de união com o de interseção e considere quais membros são seguros em qualquer alternativa.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

pix sem chave → erro TS; boleto com ultimos4 literal -> erro TS; handler switch estreita campos

Concepção errada e falha observável

Criar interface com todos os campos opcionais. / Na entrada pix sem chave, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Criar interface com todos os campos opcionais.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Criar interface com todos os campos opcionais.

Pistas graduais

1. Cada método é uma variante. 2. Campos exclusivos ficam obrigatórios só nela. 3. Switch em metodo dá narrowing sem cast.

Solução de referência explicada

type Pagamento=... union das três variantes; função descrever usa switch e campos próprios.

Transferência em outro contexto

Modele opções de entrega para retirada, courier e locker. Escolha campos específicos de cada variante e mostre uma função de resumo e um caso autoral que o contrato rejeita.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, Node.js 24 LTS. Discriminated unions guide compile-time narrowing; values arriving over a network still need runtime validation. | Próxima sugestão: 16.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.09 Exaustividade com assertNever

TYPE · 16 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.assert_never | Requer: ts.basic | Introduz: ts.assert_never

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Complete o handler de eventos de pesquisa com assertNever. Invente uma mudança no conjunto de eventos e registre como o diagnóstico identifica um handler que ainda precisa ser revisto.

Objetivo da atividade

Preveja o erro de compilação que surgirá ao adicionar a quarta variante sem handler.

Explicação e modelo mental

• Modelo mental: Depois de um switch exaustivo, não sobra nenhum valor; never representa essa ausência e denuncia alterações incompletas.
• Exemplo resolvido: Num parser de tokens, um ramo final encaminha o caso ainda não discriminado a uma função que recebe never; isso transforma uma suposição estática em diagnóstico quando o modelo fica incompleto.
• Armadilha comum: Retornar string vazia no default esconde eventos novos e aparenta sucesso.
• Por que importa: Novos eventos de produto forçam a revisão dos pontos de apresentação e processamento que dependem deles.

Material/arquivo inicial

eventos.ts: // 16.09 · Handler de eventos da pesquisa com verificação de exaustividade.

```
export function assertNever(valor: never): never {
throw new Error(Evento não tratado: ${JSON.stringify(valor)});
}

export type EventoPesquisa =
| { tipo: "digitou"; termo: string }
| { tipo: "enviou"; termo: string }
| { tipo: "limpou" };

export function descrever(evento: EventoPesquisa): string {
switch (evento.tipo) {
case "digitou":
return digitando "${evento.termo}";
case "enviou":
return buscando "${evento.termo}";
case "limpou":
return "busca limpa";
default:
// complete: o compilador deve provar que nenhum evento sobrou aqui
}
}

// Depois de completar, invente um evento novo (por exemplo { tipo: "filtrou"; filtro: string }),
// acrescente-o à union sem mexer no switch e registre o diagnóstico que aparece.
```

Exemplo e resultado público

três variantes tratadas → compila e retorna descrição

Bordas e erros publicados

adicionar quarta variante -> erro TS no argumento de assertNever; runtime inesperado -> lança Error contextual

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em um switch JavaScript, o que pode ocorrer se um case terminar sem break, return ou outra saída?

 (Correta) A execução pode continuar no corpo do próximo case, fenômeno chamado fallthrough.
• O runtime encerra automaticamente o switch assim que encontra o próximo rótulo case.
• TypeScript converte automaticamente o switch em uma cadeia de if com escopos isolados.

Por que a resposta está certa

Correto. Rótulos case não encerram o fluxo por si sós; uma saída explícita evita fallthrough não intencional.

Armadilha da alternativa incorreta

Lembre-se de que o comportamento básico do switch vem do JavaScript e requer uma saída explícita quando desejada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

adicionar quarta variante → erro TS no argumento de assertNever; runtime inesperado -> lança Error contextual

Concepção errada e falha observável

Usar default retorna vazio e perder exaustividade. / Na entrada adicionar quarta variante, a saída observável não corresponde a erro TS no argumento de assertNever; a divergência é compatível com esta hipótese: Usar default retorna vazio e perder exaustividade.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar default retorna vazio e perder exaustividade.

Pistas graduais

1. Escreva helper com parâmetro never. 2. No default do switch chame helper. 3. Acrescente variante para provar erro de typecheck.

Solução de referência explicada

assertNever lança erro; função switch cobre variantes; exercício registra compilação falha antes de incluir novo case.

Transferência em outro contexto

Adicione uma nova ação de catálogo ao union de eventos. Registre o diagnóstico que aparece nos handlers incompletos e como sua implementação o resolve.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; run snippets on Node.js 24 LTS. never exposes missing cases during typecheck; assertNever also chooses explicit runtime behavior for an unexpected value. | Próxima sugestão: 16.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.04 Cast perigoso

MODIFY · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.0/5

Competência: ts.external_validation | Requer: ts.basic | Introduz: ts.external_validation

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 4 · depuração 4 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Corrija userFromResponse para o cartão de perfil. Escreva o parser/guard e use payloads próprios para mostrar uma resposta válida e formas diferentes de falha, identificando a etapa de cada uma.

Objetivo da atividade

Preveja qual dos quatro casos falha no parse e qual só falha na validação de campos.

Explicação e modelo mental

• Modelo mental: Parse válido só prova sintaxe; o formato de objeto e os tipos de campos são provas distintas.
• Exemplo resolvido: Parsear JSON só estabelece sintaxe; a aplicação ainda precisa justificar que o valor pode ser tratado como o modelo de domínio.
• Armadilha comum: Usar as User no retorno do fetch transforma uma hipótese em tipo sem verificar os bytes recebidos.
• Por que importa: Separar parse, validação e construção dá erros úteis e mantém dados incorretos fora do modelo de perfil.

Material/arquivo inicial

perfil.ts:
```
// 16.04 · Cartão de perfil: o cast promete um User que ninguém verificou.
// Substitua o cast por um parser que valide id (number) e name (string).
// Falhas: forma errada -> erro de validação que diga qual campo falhou;
// JSON inválido -> erro de parse que preserve o erro original em cause.

export type User = { id: number; name: string };

// Forma mínima de uma resposta HTTP, para testar sem rede.
// (No navegador, Response.json() devolve Promise<any>; aqui o retorno é unknown de propósito.)
export type RespostaHttp = { json(): Promise<unknown> };

export async function userFromResponse(res: RespostaHttp): Promise<User> {
return (await res.json()) as User;
}

export function respostaFalsa(corpo: string): RespostaHttp {
return { json: async () => JSON.parse(corpo) };
}

// Payloads para conferir (crie os seus também):
// await userFromResponse(respostaFalsa('{"id":1,"name":"Ana"}')) -> User válido
// await userFromResponse(respostaFalsa('{"id":"1","name":"Ana"}')) -> erro de validação em id
// await userFromResponse(respostaFalsa('null')) -> erro de validação
// await userFromResponse(respostaFalsa('{')) -> erro de parse com cause
```

Exemplo e resultado público

{id:1,name:'Ana'} → User válido

Bordas e erros publicados

{id:'1',name:'Ana'} ou null -> erro de validação com caminho; JSON inválido -> erro de parse com cause

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

Promise<User> é um tipo com parâmetro: a Promise que, quando resolvida, entrega um User. Array<number> é a mesma ideia para listas (igual a number[]). Escrever seus próprios tipos com parâmetro é o assunto da etapa 17.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como vários componentes devem obter um tipo de domínio confiável a partir de dados de API?

• Deixar cada componente presumir independentemente a forma esperada do valor.
 (Correta) Passar o valor por uma fronteira comum que verifica sua forma antes de entregá-lo aos componentes.
• Usar o status da resposta como substituto para verificar a estrutura dos dados.

Por que a resposta está certa

Uma fronteira compartilhada evita suposições divergentes entre os consumidores.

Armadilha da alternativa incorreta

Informações de transporte ou suposições locais não verificam a estrutura do valor.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

{id:'1',name:'Ana'} ou null → erro de validação com caminho; JSON inválido -> erro de parse com cause

Concepção errada e falha observável

Tratar type assertion como prova de shape. / Na entrada {id:'1',name:'Ana'} ou null, a saída observável não corresponde a erro de validação com caminho; a divergência é compatível com esta hipótese: Tratar type assertion como prova de shape.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar type assertion como prova de shape.

Pistas graduais

1. Receba JSON como unknown. 2. Valide objeto/id/name no runtime. 3. Só construa User após guards completos.

Solução de referência explicada

Faça parse em unknown, guard record, id number inteiro e name string; lance erro de domínio com causa quando parse falhar.

Transferência em outro contexto

Valide um DTO lido do storage antes de aceitá-lo no domínio. Escreva um guard e use payloads próprios para demonstrar parse, validação de campos e construção tipada.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; Node.js 24 LTS. JSON parsing and field validation happen at runtime; the compiler checks the guards and resulting contract. | Próxima sugestão: 16.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.05 Type predicate

MAKE · 20 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.type_predicate | Requer: ts.basic | Introduz: ts.type_predicate

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implemente isUser para filtrar registros externos. Entregue a função e quatro valores de teste escolhidos por você, com justificativa de cada retorno segundo os campos que seu contrato exige.

Objetivo da atividade

Antes de codar, anote quais fatos observáveis um predicate precisa estabelecer para que consumidores confiem no tipo prometido.

Explicação e modelo mental

• Modelo mental: Um predicate x is User promete que todo valor true satisfaz User; a implementação precisa provar essa promessa.
• Exemplo resolvido: Para isPoint(x), confirme que x é objeto não nulo e que latitude e longitude são números antes de prometer esse tipo.
• Armadilha comum: Uma assinatura type predicate não faz o compilador verificar se o corpo realmente checou cada propriedade.
• Por que importa: Filtrar conteúdo externo com guardas honestas evita que renderização posterior confie em campos que não existem.

Material/arquivo inicial

usuarios.ts: // 16.05 · Type predicate para filtrar registros externos.
// O predicate é uma promessa: se ele checar menos do que promete, o compilador confia e o erro aparece em runtime.

```
export type User = { id: number; name: string };

export function isUser(x: unknown): x is User {
// implemente: objeto não nulo, não array, id number e name string como propriedades próprias
}

const registros: unknown[] = [
{ id: 1, name: "A" },
null,
{ id: "2", name: "B" },
{ id: 3, name: 4 },
[1, "A"],
];

export const usuarios: User[] = registros.filter(isUser);
console.log(usuarios); // esperado: [{ id: 1, name: "A" }]

// Escreva mais quatro valores de teste e justifique cada retorno:
// 1. ___
// 2. ___
// 3. ___
// 4. ___ (dica: um objeto que só herda id e name de outro: Object.create({ id: 1, name: "X" }))
```

Exemplo e resultado público

[{id:1,name:'A'},null] → [{id:1,name:'A'}] com tipo User[]

Bordas e erros publicados

id string/name número -> false; array como objeto -> false; propriedade herdada -> política declarada de own fields

Teste público e aprovação visível

Resultado observável: [{id:1,name:'A'},null] -> [{id:1,name:'A'}] com tipo User[]. Borda a conferir: id string/name número -> false; array como objeto -> false; propriedade herdada -> política declarada de own fields. Entrega esperada: isUser combina guards e Object.hasOwn para id/name, confirma número inteiro e string; filter estreita para User[].

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que TypeScript infere para os elementos retornados por array.filter(isSpecificType) quando isSpecificType declara um type predicate?

• Um array com todos os elementos originais, pois type predicates só existem na documentação.
• Um array serializado para JSON com as propriedades verificadas pelo predicate.
 (Correta) Um array estreitado para o subtipo declarado pelo predicate; em runtime, filter ainda conserva os itens cujo callback retorna true.

Por que a resposta está certa

Correto. A assinatura informa narrowing estático, enquanto filter continua aplicando seu callback em runtime.

Armadilha da alternativa incorreta

Separe a informação de tipo usada pelo compilador do comportamento real de Array.filter.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

id string/name número → false; array como objeto -> false; propriedade herdada -> política declarada de own fields

Concepção errada e falha observável

Assinar type predicate verdadeiro sem verificar todos os campos. / Na entrada id string/name número, a saída observável não corresponde a false; a divergência é compatível com esta hipótese: Assinar type predicate verdadeiro sem verificar todos os campos.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Assinar type predicate verdadeiro sem verificar todos os campos.

Pistas graduais

1. Cheque não null e não array. 2. Valide campos runtime. 3. Assinatura x is User deve refletir a prova real.

Solução de referência explicada

isUser combina guards e Object.hasOwn para id/name, confirma número inteiro e string; filter estreita para User[].

Transferência em outro contexto

Filtre eventos externos desconhecidos com um predicate honesto. Escolha registros de teste variados, documente a regra de aceitação e compare o array antes/depois.

Evidência de domínio

Resultado observável: [{id:1,name:'A'},null] -> [{id:1,name:'A'}] com tipo User[]. Borda a conferir: id string/name número -> false; array como objeto -> false; propriedade herdada -> política declarada de own fields. Entrega esperada: isUser combina guards e Object.hasOwn para id/name, confirma número inteiro e string; filter estreita para User[]. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; examples run on Node.js 24 LTS. A user-written predicate must check every claimed field at runtime; filter then uses that compile-time claim. | Próxima sugestão: 16.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.10 Catch recebe unknown

MODIFY · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.catch_unknown | Requer: ts.basic | Introduz: ts.catch_unknown

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 3 · depuração 4 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Corrija a captura de falhas ao salvar um pedido. Entregue o tratamento de Error e de valor não-Error, a mensagem pública escolhida e a evidência de que o valor original permanece disponível como cause.

Objetivo da atividade

Antes de implementar, escreva a guarda necessária antes de ler uma propriedade de um valor capturado como unknown.

Explicação e modelo mental

• Modelo mental: O valor capturado não tem contrato confiável de domínio; mantenha separadas a mensagem apresentada ao usuário e a informação técnica útil para diagnóstico.
• Exemplo resolvido: Uma fronteira que converte falhas em um objeto de relatório pode manter um texto de apresentação e o valor original em campos distintos; ainda é preciso definir quais entradas merecem confiança.
• Armadilha comum: Assumir que tudo lançado implementa Error e acessar e.message diretamente.
• Por que importa: Normalizar a mensagem protege a UI contra valores arbitrários sem perder contexto técnico para diagnóstico.

Material/arquivo inicial

pedido.ts: // 16.10 · Salvar pedido: em modo strict, o valor capturado é unknown.
// Corrija a captura: trate Error e valor que não é Error, mostre uma mensagem pública segura
// e relance um novo Error que preserve o valor original em cause.

```
export async function salvar(pedido: { id: string }, falha: unknown): Promise<void> {
if (falha !== undefined) throw falha;
console.log("salvo", pedido.id);
}

export async function salvarPedido(pedido: { id: string }, falha?: unknown): Promise<string> {
try {
await salvar(pedido, falha);
return "Pedido salvo.";
} catch (e) {
return Falhou: ${e.message};
}
}

// Casos para conferir:
// salvarPedido({ id: "p1" }) -> "Pedido salvo."
// salvarPedido({ id: "p1" }, new Error("disco")) -> mensagem com "disco"; cause preservada
// salvarPedido({ id: "p1" }, "falhou") -> mensagem pública segura
// salvarPedido({ id: "p1" }, { codigo: 503 }) -> não acessar .message de um objeto qualquer
```

Exemplo e resultado público

throw new Error('x') → mensagem x e cause preservada; throw 'falhou' -> mensagem pública segura

Bordas e erros publicados

objeto arbitrário sem message -> não acessar e.message; cause original guardada em novo Error quando relançar

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em runtimes JavaScript comuns, qual propriedade de Error é usada para inspecionar o caminho de chamadas associado a uma falha?

□ (Correta) stack
• name
• message

Por que a resposta está certa

Correto. stack costuma expor contexto de chamadas; seu formato exato depende do runtime.

Armadilha da alternativa incorreta

name e message descrevem a classe e o texto do erro; procure a propriedade de contexto de chamadas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

objeto arbitrário sem message → não acessar e.message; cause original guardada em novo Error quando relançar

Concepção errada e falha observável

Fazer catch(e:any) e assumir que tudo lançado é Error. / Na entrada objeto arbitrário sem message, a saída observável não corresponde a não acessar e.message; a divergência é compatível com esta hipótese: Fazer catch(e:any) e assumir que tudo lançado é Error.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Fazer catch(e:any) e assumir que tudo lançado é Error.

Pistas graduais

1. catch recebe unknown em strict. 2. Use instanceof Error para ler message/cause. 3. Trate outros valores sem expor objeto não confiável.

Solução de referência explicada

Normalize erro com função que testa instanceof Error e relança Error público com cause:e; logs técnicos separados.

Transferência em outro contexto

Um callback assíncrono pode rejeitar com um número. Defina a apresentação segura na UI, a informação técnica retida para diagnóstico e a guarda que fundamenta a mensagem.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict; Node.js 24 LTS. catch unknown and instanceof narrow statically; thrown values and safe presentation are runtime concerns. | Próxima sugestão: 16.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.06 as const derivado

TYPE · 16 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.as_const_derived | Requer: ts.basic | Introduz: ts.as_const_derived

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 1 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Derive um tipo de nível de acesso a partir de uma lista literal do produto. Registre a derivação e exemplos próprios aceitos e rejeitados pelo typecheck, além da diferença entre contrato e array runtime.

Objetivo da atividade

Preveja o tipo de STATUS[0] e o resultado do push em tsc antes de observar o array runtime.

Explicação e modelo mental

• Modelo mental: Tipos podem ser derivados de valores para manter uma fonte de verdade; literais preservados permitem que o tipo represente as opções exatas.
• Exemplo resolvido: typeof captura o tipo da lista e indexed access seleciona o tipo de seus elementos, formando uma union reutilizável.
• Armadilha comum: Achar que readonly de as const chama Object.freeze ou impede mutação por todos os aliases em runtime.
• Por que importa: Uma fonte única mantém a lista de estados e o tipo sincronizados sem prometer proteção que JavaScript não executa.

Material/arquivo inicial

acesso.ts: // 16.06 · Nível de acesso derivado da lista do produto.
// Hoje a lista runtime e a union de tipos repetem os mesmos textos: uma mudança atualiza um e esquece o outro.

export const NIVEIS = ["leitor", "editor", "admin"];
export type Nivel = "leitor" | "editor" | "admin";

// Tarefa: derive Nivel de NIVEIS com as const e (typeof NIVEIS)[number].

// Depois de derivar, registre o que o typecheck faz com cada linha:
// const ok: Nivel = "editor"; // ___
// const fora: Nivel = "dono"; // ___
// NIVEIS.push("dono"); // ___
// Em runtime, o array ficou congelado? Teste com Object.isFrozen(NIVEIS): ___

Exemplo e resultado público

'idle' atribui a Status → compila; 'other' -> erro TS

Bordas e erros publicados

STATUS.push -> erro TS; array runtime não é congelado por as const; derive em vez de duplicar string union

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

typeof STATUS (numa posição de tipo) pega o tipo da constante STATUS; [number] pega o tipo de qualquer item desse array. Com as const, o resultado é a união dos literais: 'idle' | 'loading' | 'done'. Indexed access é aprofundado na etapa 18.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em uma posição de tipo, o que a consulta typeof binding permite referenciar?

• O texto produzido pelo operador JavaScript typeof durante a execução.
 (Correta) O tipo estático associado a uma variável ou propriedade existente no código.
• Uma cópia congelada do valor da variável no heap.

Por que a resposta está certa

Correto. O operador de consulta de tipo reaproveita um tipo existente; não é uma operação runtime.

Armadilha da alternativa incorreta

A posição sintática importa: em um tipo, typeof é uma consulta estática, não uma execução JavaScript.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

STATUS.push → erro TS; array runtime não é congelado por as const; derive em vez de duplicar string union

Concepção errada e falha observável

Repetir união à mão e deixá-la divergir do array. / Na entrada STATUS.push, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Repetir união à mão e deixá-la divergir do array.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Repetir união à mão e deixá-la divergir do array.

Pistas graduais

1. as const preserva literais da expressão. 2. typeof STATUS indexado por number dá união. 3. Teste valor aceito e rejeitado no typecheck.

Solução de referência explicada

Use o array como fonte única, derive Status e documente que readonly é checagem TS, não freeze runtime.

Transferência em outro contexto

Derive um union para os níveis de acesso de uma equipe a partir de uma lista literal. Apresente a derivação e exemplos autorais aceitos e rejeitados pelo typecheck.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, Node.js 24 LTS. as const and the derived union are compile-time readonly/literal typing; the array is not frozen at runtime. | Próxima sugestão: 16.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.07 Mudanca de requisito

MODIFY · 24 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.exhaustive_change | Requer: ts.basic | Introduz: ts.exhaustive_change

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Adicione a variante triangle { kind: 'triangle'; base: number; height: number } à união Shape. Registre o diagnóstico de assertNever antes de atualizar os pontos afetados, implemente área como base * height / 2, confirme que height ausente é rejeitado no typecheck e que nenhum default silencioso retorna zero. Rode os casos anteriores e um caso de triângulo.

Objetivo da atividade

Antes de adicionar case, anote qual tipo você espera que assertNever receba depois de incluir triangle.

Explicação e modelo mental

• Modelo mental: A discriminante estreita a union dentro de cada ramo; cada cálculo deve usar os campos e regras próprios da variante selecionada.
• Exemplo resolvido: Quando o domínio ganha uma variante, o ramo exaustivo deixa de provar que não há valores restantes; implemente o caso usando a regra do domínio.
• Armadilha comum: Um default que retorna zero transforma uma variante não tratada em resultado aparentemente válido.
• Por que importa: Mudanças futuras no domínio apontam para cada cálculo dependente e reduzem erros silenciosos na tela.

Material/arquivo inicial

formas.ts: // 16.07 · Mudança de requisito: acrescentar triângulo.
// 1. Acrescente a variante { kind: "triangle"; base: number; height: number } à union Shape.
// 2. Antes de mexer em area, registre o diagnóstico de assertNever.
// 3. Implemente a área como base * height / 2, sem default que devolva 0.

```
export function assertNever(valor: never): never {
throw new Error(Forma não tratada: ${JSON.stringify(valor)});
}

export type Shape =
| { kind: "circle"; radius: number }
| { kind: "square"; side: number };

export function area(shape: Shape): number {
switch (shape.kind) {
case "circle":
return Math.PI * shape.radius ** 2;
case "square":
return shape.side ** 2;
default:
return assertNever(shape);
}
}

// Casos anteriores e novo:
// area({ kind: "square", side: 3 }) -> 9
// area({ kind: "circle", radius: 1 }) -> Math.PI
// area({ kind: "triangle", base: 6, height: 4 }) -> 12
// area({ kind: "triangle", base: 6 }) -> deve falhar no typecheck
```

Exemplo e resultado público

Adicione triangle com base 6 e height 4 → Antes de tratar a nova variante, assertNever acusa valor restante; após implementar a variante, área é 12. Omitir height falha no typecheck.

Bordas e erros publicados

triangle base/height -> área base*height/2; variante sem campo exigido -> erro TS; default silencioso rejeitado

Teste público e aprovação visível

Exija evidência do compilador antes/depois, regra de domínio correta, erro de campo ausente e regressão das variantes existentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando uma união fechada recebe uma nova variante, por que revisar os handlers que já a consomem?

• Para permitir que os handlers antigos escolham qualquer variante automaticamente.
• Para remover a nova alternativa dos tipos até todos os consumidores ignorá-la.
 (Correta) Para decidir explicitamente como cada consumidor trata a nova alternativa.

Por que a resposta está certa

Uma alteração no conjunto de variantes pode exigir uma decisão em cada ponto de consumo.

Armadilha da alternativa incorreta

Uma variante nova não recebe comportamento automaticamente; os consumidores precisam manter intenção clara.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

triangle base/height → área base*height/2; variante sem campo exigido -> erro TS; default silencioso rejeitado

Concepção errada e falha observável

Usar default que retorna 0 e esconder variante nova. / Na entrada triangle base/height, a saída observável não corresponde a área base*height/2; a divergência é compatível com esta hipótese: Usar default que retorna 0 e esconder variante nova.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar default que retorna 0 e esconder variante nova.

Pistas graduais

1. Adicione tag e campos. 2. Deixe o compilador apontar switches incompletos. 3. Atualize cada caso e mantenha assertNever.

Solução de referência explicada

Inclua triangle, corrija area e outros switches indicados, prove erro de compilação antes da correção e execução depois.

Transferência em outro contexto

Adicione uma nova opção de pagamento ao sistema tipado. Registre os pontos que o compilador sinaliza e a evidência de execução depois de completar os handlers.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Baseline: TypeScript 7.0 strict, Node.js 24 LTS. Exhaustiveness errors are compile-time; executing the updated cases verifies the formula and output. | Próxima sugestão: 16.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

16.08 Microprojeto - máquina de estados

PROJECT · 90 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.5/5

Competência: ts.upload_state_machine | Requer: ts.basic | Introduz: ts.upload_state_machine

Eixos: novidade 5 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 4 · leitura 5 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Implemente reducer de upload para anexos de recibos. Entregue unions de estados/ações, tabela de transições e casos próprios para fluxo feliz, uma fronteira de progress e uma ação incompatível.

Objetivo da atividade

Desenhe em texto uma transição válida e uma ação que seu produto deve rejeitar ou ignorar.

Explicação e modelo mental

• Modelo mental: Um reducer total transforma um estado válido por ação; union expõe os casos e runtime valida os números recebidos.
• Exemplo resolvido: Num editor, a fase da janela (closed/open) e a persistência do documento (saved/unsaved) são dimensões distintas; fechar pode deixar conteúdo não salvo, a menos que o produto bloqueie ou descarte essa mudança.
• Armadilha comum: Booleans independentes deixam progress coexistir com success e escondem transições sem sentido.
• Por que importa: A tela de upload pode mostrar progresso e resultado sem combinar fases incompatíveis nem aceitar percentuais impossíveis.

Material/arquivo inicial

upload.ts: // 16.08 · Máquina de estados do upload de recibos.
// Estados: idle, selecting, uploading(progress 0..100), success(url), error(message).
// Ações: select, start, progress(valor), resolve(url), fail(message), reset.
// Política para ação incompatível com o estado: devolva o mesmo estado (documente no fim do arquivo).

```
export type Estado =
| { status: "idle" }
| { status: "selecting" }
| { status: "uploading"; progress: number }
| { status: "success"; url: string }
| { status: "error"; message: string };

export type Acao =
| { type: "select" }
| { type: "start" }
| { type: "progress"; valor: number }
| { type: "resolve"; url: string }
| { type: "fail"; message: string }
| { type: "reset" };

export function reducer(estado: Estado, acao: Acao): Estado {
// implemente as transições; progress fora de 0..100 lança RangeError
}

// Tabela de transições (preencha):
// estado | ação | próximo estado
// idle | select | selecting
// selecting | start | uploading 0
// uploading | progress | ___
// uploading | resolve | ___
// ___ | ___ | ___
```

| upload.test.ts: // Rode com node --test upload.test.ts (Node.js 24 remove os tipos).

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { reducer } from "./upload.ts";

test("fluxo feliz", () => {
const selecionando = reducer({ status: "idle" }, { type: "select" });
assert.deepEqual(selecionando, { status: "selecting" });
const enviando = reducer(selecionando, { type: "start" });
assert.deepEqual(enviando, { status: "uploading", progress: 0 });
assert.deepEqual(reducer(enviando, { type: "resolve", url: "/r/1.png" }), { status: "success", url: "/r/1.png" });
});

test("fronteira de progress", () => {
const enviando = { status: "uploading", progress: 0 } as const;
assert.deepEqual(reducer(enviando, { type: "progress", valor: 100 }), { status: "uploading", progress: 100 });
assert.throws(() => reducer(enviando, { type: "progress", valor: 101 }), RangeError);
});

test("ação incompatível mantém o estado", () => {
const inicial = { status: "idle" } as const;
assert.equal(reducer(inicial, { type: "progress", valor: 10 }), inicial);
});
```

| tsconfig.json: {

```
"compilerOptions": {
```

"target": "es2024",
"module": "nodenext",
"strict": true,
"noUncheckedIndexedAccess": true,
"erasableSyntaxOnly": true,
"verbatimModuleSyntax": true,
"allowImportingTsExtensions": true,
"noEmit": true,
"types": ["node"]
}
}
| package.json: {
"name": "devpath-16-08",
"version": "1.0.0",
"type": "module"
}
| package-lock.json: {
"name": "devpath-16-08",
"version": "1.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "devpath-16-08",
"version": "1.0.0"
}
}
}

Exemplo e resultado público

idle+select → selecting; selecting+start -> uploading 0; uploading+resolve(url) -> success

Bordas e erros publicados

progress em idle -> erro/estado inalterado por política; progress fora 0..100 -> RangeError; success sem URL impossível pelo tipo

Teste público e aprovação visível

Resultado observável: idle+select -> selecting; selecting+start -> uploading 0; uploading+resolve(url) -> success. Borda a conferir: progress em idle -> erro/estado inalterado por política; progress fora 0..100 -> RangeError; success sem URL impossível pelo tipo. Entrega esperada: Reducer exaustivo por estado/ação, invariantes de progresso, sem mutação; suíte cobre fluxo feliz e ações inválidas.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que responsabilidades pertencem à union/reducer e quais dependem de validar um número recebido em runtime?

 (Correta) A union estrutura estados/transições; uma regra numérica concreta também exige checagem do valor
• A union verifica automaticamente cada valor numérico vindo da rede
• A validação runtime substitui a definição de estados mutuamente exclusivos

Por que a resposta está certa

O tipo restringe formas e o reducer organiza transições; limites numéricos dependem do dado executado.

Armadilha da alternativa incorreta

Tipos não inspecionam valores da rede, e guards numéricos não definem sozinhos os estados válidos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

progress em idle → erro/estado inalterado por política; progress fora 0..100 -> RangeError; success sem URL impossível pelo tipo

Concepção errada e falha observável

Modelar transições apenas com booleans e permitir progress após sucesso. / Na entrada progress em idle, a saída observável não corresponde a erro/estado inalterado por política; a divergência é compatível com esta hipótese: Modelar transições apenas com booleans e permitir progress após sucesso.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Modelar transições apenas com booleans e permitir progress após sucesso.

Pistas graduais

1. Defina union de estados e ações. 2. Tabela de transições válidas antes de codar reducer. 3. Valide payload runtime e trate ação inválida explicitamente.

Solução de referência explicada

Reducer exaustivo por estado/ação, invariantes de progresso, sem mutação; suíte cobre fluxo feliz e ações inválidas.

Transferência em outro contexto

Amplie a máquina de pedidos com cancelamento. Entregue a tabela de transições, o reducer atualizado e casos próprios para transição válida e ação incompatível.

Evidência de domínio

Resultado observável: idle+select -> selecting; selecting+start -> uploading 0; uploading+resolve(url) -> success. Borda a conferir: progress em idle -> erro/estado inalterado por política; progress fora 0..100 -> RangeError; success sem URL impossível pelo tipo. Entrega esperada: Reducer exaustivo por estado/ação, invariantes de progresso, sem mutação; suíte cobre fluxo feliz e ações inválidas. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Project target: TypeScript 7.0 strict and Node.js 24 LTS. Union/reducer checks are compile-time; guards enforce action payload and progress bounds at runtime. | Próxima sugestão: 17.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17 Generics e APIs reutilizáveis

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 17.01 · 17.02 · 17.03 · 17.04 · 17.05 · 17.06 · 17.07 · 17.11 · 17.08 · 17.09

17.01 Generic ou union?

PREDICT · 40 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.0/5

Competência: ts.generic_choice | Requer: ts.basic | Introduz: ts.generic_choice

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 1 · depuração 1 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Classifique as APIs do starter como genérica, union ou concreta. Para cada escolha, explique que informação a assinatura precisa preservar; não adicione parâmetro genérico sem uma relação útil.

Objetivo da atividade

Invente uma API que preserve o tipo de uma entrada e outra com saída fixa; classifique essas assinaturas antes de analisar as opções do cartão.

Explicação e modelo mental

• Modelo mental: Generics preservam correspondências entre tipos; unions enumeram alternativas fechadas; tipos concretos descrevem contratos fixos.
• Exemplo resolvido: Uma lista de estados de sincronização representa alternativas finitas; em runtime, os valores continuam sendo dados que a aplicação precisa observar.
• Armadilha comum: Um parâmetro T sem relação entre argumentos e retorno só torna a assinatura mais difícil de ler; unions tampouco validam dados em runtime.
• Por que importa: Uma assinatura bem escolhida melhora inferência e diagnósticos sem prometer uma dependência inexistente.

Material/arquivo inicial

apis.ts:
```
// 17.01 · Generic, union ou concreto?
// Antes de olhar as respostas, classifique cada assinatura e diga que informação ela precisa preservar.
// Regra: só use parâmetro de tipo quando ele liga duas posições (entrada e saída, chave e valor...).

type Status = "ativo" | "pausado";
type Usuario = { id: string; nome: string };

// 1. identity(x) -> devolve o próprio x
// 2. parseStatus(texto) -> "ativo" | "pausado" ou erro
// 3. first(xs) -> primeiro item ou undefined
// 4. area(raio) -> número
// 5. pair(a, b) -> tupla com os dois valores
// 6. log(mensagem) -> nada
// 7. getById(usuarios, id) -> usuário ou undefined
// 8. selectKey(obj, chave) -> valor da propriedade escolhida

// Escreva a assinatura escolhida para cada uma (corpo pode ser throw new Error("todo")):
// export function identity___
// export function parseStatus___
// export function first___
// export function area___
// export function pair___
// export function log___
// export function getById___
// export function selectKey___

// Classificação e justificativa:
// 1. ___ 2. ___ 3. ___ 4. ___ 5. ___ 6. ___ 7. ___ 8. ___
export type { Status, Usuario };
```

Exemplo e resultado público

identity<T>(x:T):T → generic; parseStatus -> union; area(r:number) -> concreto

Bordas e erros publicados

log(message:string) -> concreto; first<T>(xs:readonly T[]):T|undefined -> generic; não usar T se retorno não depende da entrada

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual forma preserva o tipo exato do valor que entra e sai?

• function identity(x:string|number):string|number
• function identity(x:unknown):unknown
• function identity(x:string):string
 (Correta) function identity<T>(x:T):T

Por que a resposta está certa

Correto: T preserva a relação entre argumento e retorno.

Armadilha da alternativa incorreta

As demais formas generalizam ou restringem tipos e não preservam a identidade do tipo específico.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

log(message:string) → concreto; first<T>(xs:readonly T[]):T|undefined -> generic; não usar T se retorno não depende da entrada

Concepção errada e falha observável

Colocar generic em toda função mesmo sem relação entre tipos. / Na entrada log(message:string), a saída observável não corresponde a concreto; a divergência é compatível com esta hipótese: Colocar generic em toda função mesmo sem relação entre tipos.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Colocar generic em toda função mesmo sem relação entre tipos.

Pistas graduais

1. Procure relação de tipos entre entrada e saída. 2. Conjunto fechado de alternativas pede union. 3. Tipo fixo e conhecido pede concreto.

Solução de referência explicada

Tabela de oito assinaturas com justificativa; generic só quando preserva relação que concrete/union perderiam.

Transferência em outro contexto

Aplique a escolha de assinatura a uma API de cache com dois tipos independentes de entrada e resultado, justificando sua modelagem.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. As assinaturas são checadas estaticamente; tipos e unions são apagados no JavaScript emitido. Node executa somente JavaScript emitido; anote separadamente typecheck e comportamento runtime. | Próxima sugestão: 17.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.02 Parsons de repository

PARSONS · 55 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.generic_repository | Requer: ts.basic | Introduz: ts.generic_repository

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Reordene e complete pair<K,V>, wrap<T> e Repository<T extends Entity>. Preserve o tipo específico da entidade em save/get e mantenha o ID como papel de tipo separado; não substitua ambos por um único parâmetro genérico.

Objetivo da atividade

Construir pair<K,V> e wrap<T> antes de Repository<T>, preservando tipos separados para ID e entidade.

Explicação e modelo mental

• Modelo mental: Cada parâmetro genérico representa um papel de tipo; uma constraint declara capacidades mínimas e deixa T conservar a forma específica da entidade.
• Exemplo resolvido: pair('a', 2) preserva K=string e V=number em posições distintas. Um Repository<Produto> usa o id:string como chave e mantém Produto como tipo do valor retornado; uma busca ausente pode retornar undefined.
• Armadilha comum: Usar um único parâmetro para representar tanto o ID quanto a entidade mistura dois papéis. A constraint Entity limita o requisito estrutural, enquanto T preserva os demais campos da entidade concreta.
• Por que importa: Tipos relacionais precisos deixam APIs reutilizáveis sem ocultar quais partes são escolhas de design.

Material/arquivo inicial

repository.ts: export type Entity={id:string};
export function pair(key:string,value:string){return [key,value];}
export function wrap<T>(value:T){return {value};}
export interface Repository<T extends Entity,ID= T['id']>{save(value:T):void;get(id:ID):T|undefined;}

Exemplo e resultado público

pair('a',2); repo.save({id:'u1',name:'A'}); get('u1') → pair preserva [string,number]; Repository aceita entidade com id:string e get devolve a entidade específica ou undefined.

Bordas e erros publicados

get ausente -> undefined; pair('a',2) não vira [string,string]; ID e entidade não são mesmo tipo genérico; union/composição tipada é expressa por type, extensão/merging pode favorecer interface

Teste público e aprovação visível

Avalie reordenação e typecheck de cada helper/repository; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual benefício um repositório genérico oferece ao trabalhar com diferentes tipos de entidade?

• Converte automaticamente cada entidade em uma instância de uma classe-base em runtime.
 (Correta) Permite reutilizar as operações mantendo o tipo escolhido pelo consumidor ao longo da API.
• Substitui o modelo de dados por um tipo amplo, evitando declarar relações entre tipos.

Por que a resposta está certa

Um parâmetro de tipo torna a implementação reutilizável sem abandonar informação de tipo.

Armadilha da alternativa incorreta

Generics apoiam reutilização estática; não convertem objetos nem eliminam o contrato dos dados.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

get ausente → undefined; pair('a',2) não vira [string,string]; ID e entidade não são mesmo tipo genérico; union/composição tipada é expressa por type, extensão/merging pode favorecer interface

Concepção errada e falha observável

Usar apenas um T para ID e entidade, perdendo relação; escolher interface ou type como dogma sem requisito. / Na entrada get ausente, a saída observável não corresponde a undefined; a divergência é compatível com esta hipótese: Usar apenas um T para ID e entidade, perdendo relação; escolher interface ou type como dogma sem requisito.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar apenas um T para ID e entidade, perdendo relação; escolher interface ou type como dogma sem requisito.

Pistas graduais

1. Comece com pair de dois tipos. 2. wrap conserva T sem constrangimento. 3. Repository restringe entidade a ter id e get retorna T|undefined; para a forma básica ambos servem: decida por composição, extensão ou declaration merging, não por runtime.

Solução de referência explicada

Organize helpers antes de Repository; Map armazena T por id string e API preserva entidade específica. Compare alias e interface para a forma Entity: ambas são apagadas e ambas validam o mesmo shape aqui; type também nomeia unions, enquanto interface pode participar de declaration merging.

Transferência em outro contexto

Desenhe um repositório de inventário e explique quando seu contrato se beneficia de composição ou declaration merging.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. TypeScript verifica constraint e inferência; interfaces, aliases e parâmetros de tipo não criam validação runtime. Node executa somente JavaScript emitido; anote separadamente typecheck e comportamento runtime. | Próxima sugestão: 17.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.03 Complete callbacks

FADED · 40 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.callback_generics | Requer: ts.basic | Introduz: ts.callback_generics

Eixos: novidade 2 · sintaxe 4 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Complete map<T,U> e filter<T> com callbacks tipados e entrada readonly. Mostre tipos inferidos em pelo menos um mapeamento e um filtro, sem recorrer a Function nem any.

Objetivo da atividade

Escreva a assinatura do callback antes da implementação e explique por que map precisa de dois papéis de tipo e filter pode preservar um.

Explicação e modelo mental

• Modelo mental: T identifica os elementos da entrada, U identifica o valor produzido; um filtro apenas seleciona membros e conserva T.
• Exemplo resolvido: Em um relatório, um callback recebe um registro e seu índice para anexar a posição de origem a uma mensagem de diagnóstico; os tipos dos argumentos ajudam a detectar acessos inválidos antes da execução.
• Armadilha comum: Anotar callback como Function apaga seus parâmetros e retorno. Predicate boolean comum não estreita sozinho uma union discriminada.
• Por que importa: Callbacks genéricos permitem composição e mantêm inferência útil em transformações de dados.

Material/arquivo inicial

colecoes.ts: // 17.03 · Callbacks tipados sem Function nem any.
// Complete as assinaturas. A entrada é readonly: a função promete não alterar o array recebido.

```
export function map<T, U>(xs: readonly T[], fn: (x: T, index: number) => ___): ___ {
const saida: U[] = [];
for (let i = 0; i < xs.length; i++) saida.push(fn(xs[i] as T, i));
return saida;
}

export function filter<T>(xs: readonly T[], predicate: (x: ___, index: number) => boolean): ___ {
const saida: T[] = [];
xs.forEach((x, i) => { if (predicate(x, i)) saida.push(x); });
return saida;
}

// Confira o tipo inferido de cada resultado no editor:
// const textos = map([1, 2], (n) => String(n)); // esperado: string[]
// const positivos = filter([0, 1], (n) => n > 0); // esperado: number[]
// Por que map precisa de dois parâmetros de tipo e filter de um? ___
```

Exemplo e resultado público

map([1,2],n=>String(n)) → string[]; filter([0,1],n=>n>0) -> number[]

Bordas e erros publicados

callback e readonly input -> índice number, sem mutação pela assinatura, sem Function/any

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em map<T,U>, qual tipo representa o valor produzido pelo callback?

 (Correta) U
• T
• T | U
• Promise<U>

Por que a resposta está certa

Correto: o callback transforma um elemento T em um resultado U.

Armadilha da alternativa incorreta

T é a entrada; T | U e Promise<U> só se aplicariam a contratos diferentes do callback descrito.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

callback e readonly input → índice number, sem mutação pela assinatura, sem Function/any

Concepção errada e falha observável

Anotar callback como Function e apagar tipos de entrada/retorno. / Na entrada callback e readonly input, a saída observável não corresponde a índice number, sem mutação pela assinatura, sem Function/any; a divergência é compatível com esta hipótese: Anotar callback como Function e apagar tipos de entrada/retorno.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Anotar callback como Function e apagar tipos de entrada/retorno.

Pistas graduais

1. Separe T da entrada e U da saída. 2. Descreva parâmetros do callback. 3. filter comum preserva T; predicate de tipo usa overload/assinatura própria.

Solução de referência explicada

Assinaturas genéricas inferem T/U do array e callback, implementação percorre sem mutar entrada.

Transferência em outro contexto

Implemente um find<T> para lista somente leitura: o predicado recebe T e índice, o retorno é T | undefined, e casos próprios cobrem encontrado e ausência.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. O compilador verifica os contratos e inferência dos callbacks; somente JavaScript emitido executa a transformação. Node executa somente JavaScript emitido; anote separadamente typecheck e comportamento runtime. | Próxima sugestão: 17.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.04 Constraint excessiva

MODIFY · 35 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.constraint_minimal | Requer: ts.basic | Introduz: ts.constraint_minimal

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Refatore primeiro<T> para aceitar qualquer elemento quando sua implementação só acessa a coleção; mantenha uma constraint apenas no helper cujo corpo usa essa capacidade. Demonstre assinaturas separadas.

Objetivo da atividade

Leia as operações que o corpo faz sobre T e separe utilidade geral de busca orientada a entidade.

Explicação e modelo mental

• Modelo mental: Uma constraint se justifica pelas operações feitas sobre T; indexar e devolver um elemento não requer propriedades de domínio.
• Exemplo resolvido: Uma constraint como T extends {rank:number} limita chamadas aceitas pelo typechecker; ela não converte a entrada nem cria um wrapper em JavaScript.
• Armadilha comum: Adicionar constraint por costume exclui usos válidos sem proteger uma operação que não lê o campo.
• Por que importa: Constraints mínimas tornam utilitários amplos e mantêm contratos específicos onde são necessários.

Material/arquivo inicial

busca.ts: // 17.04 · Constraint maior do que o corpo precisa.
// primeiro não lê id, mas exige que todo item tenha id. Refatore e mantenha a constraint só onde ela é usada.

```
export function primeiro<T extends { id: string }>(xs: T[]): T | undefined {
return xs[0];
}

export function buscarPorId<T extends { id: string }>(xs: T[], id: string): T | undefined {
return xs.find((x) => x.id === id);
}

// Hoje falha no typecheck; depois da refatoração deve compilar com tipo number | undefined:
export const n = primeiro([1, 2]);

// Continua funcionando:
export const objeto = primeiro([{ id: "x" }]);
export const achado = buscarPorId([{ id: "x", nome: "A" }], "x");
```

Exemplo e resultado público

primeiro([1,2]) → number|undefined; primeiro([{id:'x'}]) -> objeto|undefined

Bordas e erros publicados

buscarPorId -> mantém constraint {id:string}; primeiro sem uso de id não exige

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que um tipo genérico bem escolhido oferece a um helper reutilizável em comparação com any?

• Força todas as chamadas do helper a usar um único tipo concreto escolhido na primeira chamada.
• Converte em runtime qualquer valor recebido para o tipo declarado pelo chamador.
 (Correta) Preserva relações de tipo entre entradas e resultados para que o checker acompanhe cada uso.

Por que a resposta está certa

Generics permitem reutilização mantendo informação estática específica de cada chamada.

Armadilha da alternativa incorreta

Parâmetros de tipo não convertem valores em runtime nem precisam fixar todos os usos ao mesmo tipo.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

buscarPorId → mantém constraint {id:string}; primeiro sem uso de id não exige

Concepção errada e falha observável

Adicionar constraint por costume em vez de necessidade do corpo. / Na entrada buscarPorId, a saída observável não corresponde a mantém constraint {id:string}; a divergência é compatível com esta hipótese: Adicionar constraint por costume em vez de necessidade do corpo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Adicionar constraint por costume em vez de necessidade do corpo.

Pistas graduais

1. Leia operações feitas sobre T. 2. Se só indexa array, não precisa de id. 3. Deixe constraint no helper que realmente lê id.

Solução de referência explicada

primeiro<T>(xs:readonly T[]):T|undefined; buscarPorId<T extends {id:string}> separado.

Transferência em outro contexto

Implemente um helper ultimo<T> para qualquer coleção somente leitura; devolva o último elemento ou undefined e demonstre com valores primitivos e objetos sem impor constraint de domínio.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. A constraint é uma garantia estática sobre membros disponíveis; ela não valida que um objeto externo realmente os contenha. Node executa somente JavaScript emitido; anote separadamente typecheck e comportamento runtime. | Próxima sugestão: 17.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.05 Readonly por contrato

TYPE · 35 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.0/5

Competência: ts.readonly_parameter | Requer: ts.basic | Introduz: ts.readonly_parameter

Eixos: novidade 2 · sintaxe 3 · composição 2 · estado 1 · depuração 2 · arquivos 2 · leitura 2 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Altere a assinatura de soma para que ela aceite entradas somente de leitura e implemente o cálculo sem solicitar capacidade de escrita. Demonstre chamada com duas formas de entrada aceitas.

Objetivo da atividade

Preveja se ordenar a coleção compilaria pela referência recebida e se uma alias gravável ainda poderia alterar o mesmo objeto em runtime.

Explicação e modelo mental

• Modelo mental: readonly restringe as operações permitidas pela referência dentro do código checado; não é cópia, congelamento nem garantia contra outros aliases.
• Exemplo resolvido: Um painel de horários que apenas lista escalas não precisa oferecer controles de edição ou ordenação; a API pode comunicar somente as operações que o consumidor usa.
• Armadilha comum: Confundir a garantia local do typechecker com imutabilidade global em runtime leva a contratos incorretos.
• Por que importa: Parâmetros somente leitura aceitam mais fontes de dados e tornam explícita a intenção da implementação.

Material/arquivo inicial

soma.ts: // 17.05 · O contrato pede só leitura.
// Mude a assinatura para aceitar entradas somente de leitura sem pedir capacidade de escrita.

```
export function soma(xs: number[]): number {
let total = 0;
for (const x of xs) total += x;
return total;
}

const fixos = [1, 2] as const; // readonly [1, 2]
const lista: readonly number[] = [3, 4];

// Hoje as duas chamadas abaixo falham no typecheck; depois da mudança devem compilar:
export const a = soma(fixos);
export const b = soma(lista);
export const c = soma([1, 2]); // esperado: 3

// Previsão: se alguém acrescentar xs.push(0) dentro de soma depois da mudança, o que acontece? ___
// readonly copia o array? Um alias externo gravável ainda pode alterá-lo? ___
```

Exemplo e resultado público

soma([1,2]) → 3; aceita readonly [1,2]

Bordas e erros publicados

xs.push dentro da função -> erro TS; alias externo ainda pode mutar durante runtime, readonly não copia

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao projetar uma função utilitária para coleções, por que é útil declarar apenas as operações necessárias ao algoritmo?

 (Correta) Reduz acoplamento e torna a intenção da função mais clara para quem a chama.
• Garante que todos os chamadores usem a mesma implementação interna da coleção.
• Faz o compilador substituir a coleção por uma cópia otimizada.

Por que a resposta está certa

Um contrato enxuto deixa explícitas as necessidades reais do algoritmo.

Armadilha da alternativa incorreta

A assinatura comunica requisitos da função; ela não escolhe nem copia a implementação de cada coleção.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

xs.push dentro da função → erro TS; alias externo ainda pode mutar durante runtime, readonly não copia

Concepção errada e falha observável

Achar que readonly congela entrada ou exigir array gravável sem necessidade. / Na entrada xs.push dentro da função, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Achar que readonly congela entrada ou exigir array gravável sem necessidade.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que readonly congela entrada ou exigir array gravável sem necessidade.

Pistas graduais

1. Anote parâmetro readonly number[]. 2. Use apenas leitura/reduce. 3. Teste chamada com array e tupla readonly.

Solução de referência explicada

function soma(xs:readonly number[]):number{return xs.reduce((a,b)=>a+b,0)}; contrato garante só acesso local.

Transferência em outro contexto

Projete uma função que calcula métricas de uma coleção sem modificá-la e indique seu contrato mínimo.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. A restrição readonly é estática e local à referência; JavaScript executado em Node ainda permite mutações por outras referências. Separe checagem estática da execução do JavaScript emitido. | Próxima sugestão: 17.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.06 Overload, union ou generic?

COMPARE · 50 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.api_signature_choice | Requer: ts.basic | Introduz: ts.api_signature_choice

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 2 · arquivos 2 · leitura 5 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Escolha assinaturas para as oito APIs e justifique a forma mais simples que preserva seus contratos de chamada. Compare generic, union, concreto e overloads com exemplos válidos e inválidos.

Objetivo da atividade

Escolha uma chamada do trecho em que o consumidor precisa conservar uma relação entre opção e retorno; escreva como confirmaria esse tipo inferido no compilador.

Explicação e modelo mental

• Modelo mental: Uma assinatura descreve quais argumentos o consumidor pode fornecer e qual garantia recebe; preserve só as relações que o contrato realmente promete.
• Exemplo resolvido: Antes de escolher a forma pública de uma API de exportação, registre chamadas representativas e os tipos que seus consumidores precisam receber; a tabela pode virar type tests.
• Armadilha comum: Overloads excessivos duplicam contratos; uma union pode ser larga demais quando o retorno depende da entrada escolhida.
• Por que importa: A forma adequada equilibra legibilidade e informação útil na chamada sem inventar relações.

Material/arquivo inicial

assinaturas.ts: // 17.06 · Overload, union, generic ou concreto?
// Para cada API, escolha a forma mais simples que preserva o contrato de chamada.
// Escreva a assinatura, uma chamada válida e uma inválida.

```
type Config = { porta: number; host: string };

// 1. identity: devolve o mesmo valor recebido.
// 2. parse(texto): devolve Config ou lança erro.
// 3. format(valor): aceita string ou number e sempre devolve string.
// 4. get(config, chave): devolve o valor da chave escolhida, com o tipo daquela chave.
// 5. create(): devolve uma Config com valores padrão.
// 6. map(xs, fn): transforma cada item de T em U.
// 7. toJSON(valor): devolve string; a entrada é qualquer valor serializável.
// 8. ler(tipo): ler("numero") devolve number; ler("texto") devolve string.

// Exemplo (8) com overloads: a relação entre o literal e o retorno não cabe numa union simples.
export function ler(tipo: "numero"): number;
export function ler(tipo: "texto"): string;
export function ler(tipo: "numero" | "texto"): number | string {
return tipo === "numero" ? 0 : "";
}

// Suas assinaturas para 1 a 7:
// ___

// Comparação: quando um generic resolveria a 8? Quando uma union bastaria? ___
export type { Config };
```

Exemplo e resultado público

identity → generic; format simples -> union; map -> generic T/U; get por chave -> generic K extends keyof T

Bordas e erros publicados

API com pares de entrada/saída distintos -> overload quando generic/union não representam claramente; any não resolve

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Se entradas de dois tipos diferentes sempre produzem uma string, qual assinatura costuma bastar?

• Uma assinatura genérica que promete devolver o mesmo tipo da entrada
 (Correta) Uma entrada union e retorno string
• Uma overload para cada valor literal possível
• Um retorno any para aceitar qualquer chamada

Por que a resposta está certa

Correto: a union descreve as entradas permitidas e a string é o retorno comum.

Armadilha da alternativa incorreta

As outras alternativas adicionam uma relação falsa, multiplicam casos desnecessariamente ou removem a informação estática.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

API com pares de entrada/saída distintos → overload quando generic/union não representam claramente; any não resolve

Concepção errada e falha observável

Criar overloads longos para relação simples capturada por generic. / Na entrada API com pares de entrada/saída distintos, a saída observável não corresponde a overload quando generic/union não representam claramente; a divergência é compatível com esta hipótese: Criar overloads longos para relação simples capturada por generic.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Criar overloads longos para relação simples capturada por generic.

Pistas graduais

1. Identifique relação entre input/output. 2. Prefira union quando saída não depende da variante. 3. Use overload quando chamadas distintas têm contratos distintos.

Solução de referência explicada

Matriz das oito assinaturas com código mínimo e justificativa verificável por exemplos de compilação.

Transferência em outro contexto

Escolha uma assinatura para uma API de busca por identificador que pode não encontrar resultado; justifique se o contrato deve ser genérico ou concreto e mostre chamadas válidas.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. Overloads e relações de tipos são verificadas estaticamente; Node executa apenas a implementação JavaScript emitida e não seleciona tipos. | Próxima sugestão: 17.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.07 Const type parameter

TYPE · TRILHA OPCIONAL · 45 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.const_type_parameter | Requer: ts.basic | Introduz: ts.const_type_parameter

Eixos: novidade 4 · sintaxe 4 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Crie um helper que preserve valores literais em uma configuração declarada diretamente na chamada. Explique qual comportamento de inferência quer do compilador e a versão mínima compatível.

Objetivo da atividade

Implemente o helper e escreva uma chamada de exemplo; depois descreva que garantia existe no tipo e qual não existe em runtime.

Explicação e modelo mental

• Modelo mental: Um const type parameter pede inferência mais específica para certos valores escritos diretamente na chamada; a função continua sendo JavaScript comum em runtime.
• Exemplo resolvido: Uma API que recebe um mapa de opções pode inferir chaves e valores mais específicos em vez de ampliar tudo para string.
• Armadilha comum: Tratar const como uma anotação que congela dados ou como um mecanismo que substitui qualquer uso de as const mistura inferência com comportamento runtime.
• Por que importa: Inferência literal pode produzir APIs de configuração mais precisas sem casts repetidos quando o uso se alinha ao contrato.

Material/arquivo inicial

rotas.ts: // 17.07 · Preservar literais na chamada com const no parâmetro de tipo (TypeScript 5.0 ou posterior).

```
export function defineRoutesComum<T extends Record<string, string>>(rotas: T): T {
return rotas;
}

// Implemente: mesmo corpo, mas com <const T extends ...>.
// export function defineRoutes___

// Compare os tipos inferidos no editor:
export const comum = defineRoutesComum({ home: "/", sobre: "/sobre" });
// const preciso = defineRoutes({ home: "/", sobre: "/sobre" }); // esperado: home: "/"
// const noCallSite = defineRoutesComum({ home: "/" } as const); // alternativa no call site
// const jaAlargada = { home: "/" };
// const tarde = defineRoutes(jaAlargada); // literais voltam? ___

// Que garantia existe no tipo e qual não existe em runtime (o objeto fica congelado?) ___
```

Exemplo e resultado público

defineRoutes({home:'/'}) → tipo literal '/' preservado no contexto apropriado

Bordas e erros publicados

variável já alargada -> const generic não recupera literais; runtime não congela; as const no call site é alternativa

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual versão mínima do compilador oferece const type parameters?

• TypeScript 4.9
• TypeScript 5.1
 (Correta) TypeScript 5.0
• TypeScript 4.8

Por que a resposta está certa

Correto: o recurso foi introduzido em TypeScript 5.0.

Armadilha da alternativa incorreta

As outras versões não são a versão inicial desse recurso de inferência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

variável já alargada → const generic não recupera literais; runtime não congela; as const no call site é alternativa

Concepção errada e falha observável

Achar que const type parameter transforma qualquer variável prévia em literal profundo. / Na entrada variável já alargada, a saída observável não corresponde a const generic não recupera literais; a divergência é compatível com esta hipótese: Achar que const type parameter transforma qualquer variável prévia em literal profundo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que const type parameter transforma qualquer variável prévia em literal profundo.

Pistas graduais

1. Passe literal diretamente. 2. Compare tipo inferido com T normal. 3. Separe inferência TS de imutabilidade runtime.

Solução de referência explicada

Helper usa const T para inferir literais de expressão direta; type tests mostram preservação e limite com variável alargada. O recurso exige TypeScript 5.0+.

Transferência em outro contexto

Aplique inferência literal a uma função que registra categorias de comando e derive um tipo para validação de chamadas.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. Const type parameters requerem TypeScript 5.0+. O typecheck verifica inferência; o modificador não altera runtime nem congela o valor recebido. Separe checagem estática da execução do JavaScript emitido. | Próxima sugestão: 17.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.11 Classes: visibilidade e propriedades apagáveis

PROJECT · 50 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.class_contracts | Requer: ts.basic | Introduz: ts.class_contracts

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Corrija as declarações da classe em main.ts: implemente Capacidade, torne limiteBytes private readonly e nome readonly. Preserve as propriedades explícitas e atribuições no constructor. Complete podeGuardar para aceitar bytes finitos entre zero e o limite, inclusive.

Objetivo da atividade

Classifique private, #, readonly e implements como checagem estática, runtime ou declaração estrutural; depois indique a forma de constructor compatível com erasableSyntaxOnly.

Explicação e modelo mental

• Modelo mental: private restringe acesso pelo TypeScript; #campo oferece privacidade do JavaScript em runtime. readonly bloqueia reatribuição na checagem, mas não congela o objeto. implements verifica o contrato estrutural da interface e não cria comportamento runtime.
• Exemplo resolvido: Declare os campos no corpo da classe e faça this.limiteBytes = limiteBytes dentro do constructor. Com erasableSyntaxOnly, a remoção dos tipos não precisa transformar parameter properties.
• Armadilha comum: constructor(private limiteBytes: number) declara e inicializa uma propriedade durante a transformação. erasableSyntaxOnly proíbe essa sintaxe que não pode ser apenas apagada.
• Por que importa: Separar as garantias estáticas das garantias JavaScript ajuda a escolher encapsulamento sem prometer proteção que não existe em runtime.

Material/arquivo inicial

main.ts: export interface Capacidade {
readonly nome: string;
podeGuardar(bytes: number): boolean;
}

export class Plano {
limiteBytes: number;
nome: string;

constructor(nome: string, limiteBytes: number) {
this.nome = nome;
this.limiteBytes = limiteBytes;
}

podeGuardar(bytes: number): boolean {
// Implemente a regra sem acessar estado global.
return false;
}
}
| package.json: {"type":"module"}
| tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"erasableSyntaxOnly": true,
"verbatimModuleSyntax": true,
"allowImportingTsExtensions": true,
"noEmit": true
}
}

Exemplo e resultado público

Plano('Básico',100).podeGuardar(60) → true; checagem rejeita acesso externo ao limite e reatribuição ao nome

Bordas e erros publicados

bytes acima do limite, negativos, NaN e Infinity -> false; constructor(private limiteBytes: number) é parameter property e não passa erasableSyntaxOnly

Teste público e aprovação visível

Resultado observável: Plano('Básico',100).podeGuardar(60) -> true; checagem rejeita acesso externo ao limite e reatribuição ao nome. Borda a conferir: bytes acima do limite, negativos, NaN e Infinity -> false; constructor(private limiteBytes: number) é parameter property e não passa erasableSyntaxOnly. Entrega esperada: Plano implementa interface, mantém campos privados/somente leitura, usa atribuições explícitas compatíveis com erasableSyntaxOnly e passa casos de limite.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual declaração pode ser apagada sem transformação sob erasableSyntaxOnly?

• constructor(private campo: number)
• constructor(readonly campo: number)
• constructor(public campo: number)
 (Correta) Campo explícito e atribuição this.campo = campo no constructor

Por que a resposta está certa

Correto: a propriedade e a atribuição são JavaScript comum após apagar a anotação de tipo.

Armadilha da alternativa incorreta

Parameter properties precisam emitir declaração e atribuição no corpo da classe.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

bytes acima do limite, negativos, NaN e Infinity → false; constructor(private limiteBytes: number) é parameter property e não passa erasableSyntaxOnly

Concepção errada e falha observável

Confundir private TS com # runtime, readonly com congelamento do objeto ou parameter property com sintaxe apagável. / Na entrada bytes acima do limite, negativos, NaN e Infinity, a saída observável não corresponde a false; a divergência é compatível com esta hipótese: Confundir private TS com # runtime, readonly com congelamento do objeto ou parameter property com sintaxe apagável.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir private TS com # runtime, readonly com congelamento do objeto ou parameter property com sintaxe apagável.

Pistas graduais

1. Implemente Capacidade com retorno booleano. 2. Adicione implements, marque o limite private readonly e o nome readonly. 3. Mantenha campos explícitos e atribuições no constructor; private/readonly são estáticos, # protege em runtime e implements não emite código.

Solução de referência explicada

Plano implementa interface, mantém campos privados/somente leitura, usa atribuições explícitas compatíveis com erasableSyntaxOnly e passa casos de limite.

Transferência em outro contexto

Modele outro limite com interface e propriedades explícitas. Mostre um consumidor que usa a interface, uma atribuição readonly inválida e uma chamada runtime na fronteira.

Evidência de domínio

Resultado observável: Plano('Básico',100).podeGuardar(60) -> true; checagem rejeita acesso externo ao limite e reatribuição ao nome. Borda a conferir: bytes acima do limite, negativos, NaN e Infinity -> false; constructor(private limiteBytes: number) é parameter property e não passa erasableSyntaxOnly. Entrega esperada: Plano implementa interface, mantém campos privados/somente leitura, usa atribuições explícitas compatíveis com erasableSyntaxOnly e passa casos de limite. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: A configuração do exercício usa erasableSyntaxOnly: true; o código não precisa de transformação para propriedades do constructor. private e readonly são estáticos; # tem privacidade runtime. | Próxima sugestão: 17.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.08 Microprojeto - cache genérico

PROJECT · 85 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.5/5

Competência: ts.generic_cache | Requer: ts.basic | Introduz: ts.generic_cache

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Implemente Cache<K,V> com Map, get/set/has/delete e relógio injetável. Uma entrada expira quando now >= expiresAt: get retorna undefined e a remove; has retorna false para a entrada expirada. TTL negativo lança RangeError. Suporte valores armazenados como 0 ou undefined sem usar truthiness; has deve permitir distinguir presença de ausência quando get retorna undefined.

Objetivo da atividade

Liste os tipos de entrada e retorno que cada método expõe e indique como consumidores distinguem ausência de um valor armazenado.

Explicação e modelo mental

• Modelo mental: K e V descrevem papéis independentes no armazenamento; um relógio injetável separa a política de tempo da dependência global do processo.
• Exemplo resolvido: Um registro de templates pode usar Map<Locale, Template>: tipos de chave e valor são independentes; essa escolha de armazenamento não define a política temporal.
• Armadilha comum: Usar truthiness para distinguir presença confunde valores falsy com ausência; generic types não implementam relógio, expiração ou limpeza automática.
• Por que importa: Testabilidade e contratos explícitos evitam estados ambíguos numa abstração temporal.

Material/arquivo inicial

cache.ts: // 17.08 · Cache genérico com expiração e relógio injetável.
// Contrato:
// - set(chave, valor, ttlMs): ttl negativo lança RangeError; expiresAt = agora + ttl.
// - Uma entrada expira quando agora >= expiresAt: get devolve undefined e a remove; has devolve false.
// - Valores 0 e undefined são armazenáveis: não use truthiness; has distingue presença de ausência.

```
export type Relogio = () => number;
type Entrada<V> = { valor: V; expiresAt: number };

export class Cache<K, V> {
#entradas = new Map<K, Entrada<V>>();
#agora: Relogio;

constructor(agora: Relogio = () => Date.now()) {
this.#agora = agora;
}

set(chave: K, valor: V, ttlMs: number): void {
// implemente
}

get(chave: K): V | undefined {
// implemente
}

has(chave: K): boolean {
// implemente
}

delete(chave: K): boolean {
return this.#entradas.delete(chave);
}
}
```

| cache.test.ts: // Rode com node --test cache.test.ts.

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { Cache } from "./cache.ts";

function relogio(inicio = 0) {
let agora = inicio;
return { agora: () => agora, avancar: (ms: number) => { agora += ms; } };
}

test("expira exatamente em expiresAt", () => {
const tempo = relogio();
const cache = new Cache<string, number>(tempo.agora);
cache.set("a", 1, 100);
tempo.avancar(99);
assert.equal(cache.get("a"), 1);
tempo.avancar(1);
assert.equal(cache.get("a"), undefined);
assert.equal(cache.has("a"), false);
});

test("0 e undefined são valores válidos", () => {
const tempo = relogio();
```

```
const cache = new Cache<string, number | undefined>(tempo.agora);
cache.set("zero", 0, 10);
cache.set("vazio", undefined, 10);
assert.equal(cache.get("zero"), 0);
assert.equal(cache.has("vazio"), true);
assert.equal(cache.has("nunca"), false);
});

test("ttl negativo", () => {
assert.throws(() => new Cache<string, number>().set("a", 1, -1), RangeError);
});
| tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"noUncheckedIndexedAccess": true,
"erasableSyntaxOnly": true,
"verbatimModuleSyntax": true,
"allowImportingTsExtensions": true,
"noEmit": true,
"types": ["node"]
}
}
| package.json: {
"name": "devpath-17-08",
"version": "1.0.0",
"type": "module"
}
| package-lock.json: {
"name": "devpath-17-08",
"version": "1.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "devpath-17-08",
"version": "1.0.0"
}
}
}
```

Exemplo e resultado público

set('a',1,ttl=100) em t=0; get em t=99; avançar para t=100 → Em t=99, get devolve 1; em t=100, get devolve undefined e remove a entrada, e has devolve false.

Bordas e erros publicados

get em t=100 -> undefined e remove; has expirado -> false; V pode ser 0/undefined conforme contrato; ttl negativo -> RangeError

Teste público e aprovação visível

Avalie typecheck, coerência de get/has/delete e cada fronteira temporal; nenhum timer global é necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

A atividade anterior distinguiu private, readonly e implements. Neste Cache, #entradas e #agora usam privacidade runtime do JavaScript; o constructor recebe uma função normal e atribui this.#agora = agora. O Cache não usa readonly nem parameter properties.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que injetar uma fonte de tempo em uma abstração com comportamento temporal?

• Para definir automaticamente a política que decide quando um dado deixa de estar disponível.
 (Correta) Para tornar a leitura do tempo controlável e reproduzível ao exercitar a política temporal.
• Para sincronizar os relógios de todos os processos que usam a abstração.

Por que a resposta está certa

A dependência de tempo explícita pode ser controlada; a regra temporal ainda precisa ser definida pela implementação.

Armadilha da alternativa incorreta

Injetar um relógio não escolhe a política nem sincroniza relógios entre processos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

get em t=100 → undefined e remove; has expirado -> false; V pode ser 0/undefined conforme contrato; ttl negativo -> RangeError

Concepção errada e falha observável

Testar get truthy e considerar valor falsy como ausência. / Na entrada get em t=100, a saída observável não corresponde a undefined e remove; a divergência é compatível com esta hipótese: Testar get truthy e considerar valor falsy como ausência.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Testar get truthy e considerar valor falsy como ausência.

Pistas graduais

1. Map<K,Entry<V>> armazena prazo. 2. Use Map.has/entry, não truthiness. 3. Cheque now>=expiresAt e expurgue.

Solução de referência explicada

Cache<K,V> usa Map e relógio injetado, expira no limite, has/get coerentes, sem timer global; typecheck e casos de borda.

Transferência em outro contexto

Crie uma abstração tipada de memoização com uma chave composta e defina como suas consultas tratam validade.

Evidência de domínio

Resultado observável: set('a',1,ttl=100) em t=0; get em t=99 -> 1. Borda a conferir: get em t=100 -> undefined e remove; has expirado -> false; V pode ser 0/undefined conforme contrato; ttl negativo -> RangeError. Entrega esperada: Cache<K,V> usa Map e relógio injetado, expira no limite, has/get coerentes, sem timer global; typecheck e casos de borda. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. TypeScript 7.0 strict; Node.js 24 LTS. tsc --noEmit verifica relações K/V; relógio, expiração e limpeza são lógica JavaScript que precisa executar em Node. Diferencie typecheck de JavaScript executado. | Próxima sugestão: 17.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

17.09 Enum, objeto e código emitido

COMPARE · 45 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.enum_emit | Requer: ts.basic | Introduz: ts.enum_emit

Eixos: novidade 3 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Compile declarações enum e alternativas baseadas em valores; inspecione o JavaScript emitido e separe o que existe como valor runtime do que serve apenas ao typecheck.

Objetivo da atividade

Faça uma previsão sobre os artefatos emitidos para as formas do starter; confirme cada afirmação lendo o JavaScript resultante.

Explicação e modelo mental

• Modelo mental: Tipos e valores ocupam papéis diferentes: algumas declarações tipadas também emitem estruturas JavaScript; aliases de tipos e unions são apagados.
• Exemplo resolvido: Compare o texto de saída do compilador com as declarações-fonte: o output é a evidência da representação disponível ao processo Node.
• Armadilha comum: Concluir o formato runtime apenas pela aparência no editor ignora que parte da sintaxe TypeScript não sobrevive à emissão.
• Por que importa: Decisões de API precisam considerar tanto a experiência estática quanto os valores disponíveis em runtime.

Material/arquivo inicial

formas.ts: // 17.09 · enum, objeto as const e o JavaScript emitido.
// Compile com npx tsc formas.ts --target es2024 --outDir saida e leia saida/formas.js.
// Para cada declaração, marque: existe em runtime? só existe no typecheck?

```
export enum Numerico { A, B }
export enum Texto { A = "A", B = "B" }

export const Objeto = { A: "A", B: "B" } as const;
export type Objeto = (typeof Objeto)[keyof typeof Objeto];

// Previsões:
// Numerico emitido: ___ (há mapeamento reverso Numerico[0] === "A"?)
// Texto emitido: ___
// Objeto emitido: ___
// type Objeto emitido: ___
// Node.js 24 roda este arquivo direto (node formas.ts)? E com erasableSyntaxOnly ligado, o tsc aceita? ___
```

Exemplo e resultado público

enum numérico → objeto runtime com reverse mapping; enum string -> objeto sem reverse mapping numérico

Bordas e erros publicados

obj as const -> objeto runtime normal, type derivado some na emissão; const enum pode depender de configuração e exige cuidado de pacote

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao escolher uma representação para um conjunto finito de valores nomeados, qual critério deve orientar a decisão?

• Qual forma tem a menor quantidade de caracteres no arquivo-fonte.
• Qual forma cria mais declarações, independentemente de como a API será usada.
 (Correta) Como os consumidores usarão a API e quais convenções do projeto favorecem seu entendimento.

Por que a resposta está certa

A escolha deve servir ao uso da API e ser compreensível no contexto do projeto.

Armadilha da alternativa incorreta

Tamanho do texto-fonte ou volume de declarações, isoladamente, não definem a melhor representação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

obj as const → objeto runtime normal, type derivado some na emissão; const enum pode depender de configuração e exige cuidado de pacote

Concepção errada e falha observável

Achar que enum e union de literais têm mesmo runtime emitido. / Na entrada obj as const, a saída observável não corresponde a objeto runtime normal, type derivado some na emissão; a divergência é compatível com esta hipótese: Achar que enum e union de literais têm mesmo runtime emitido.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que enum e union de literais têm mesmo runtime emitido.

Pistas graduais

1. Compile trechos com config declarada. 2. Compare JS resultante, não só tipos. 3. Escolha conforme necessidade de objeto runtime e interoperabilidade.

Solução de referência explicada

Relatório traz JS emitido, reverse mapping do enum numérico, ausência no string e decisão contextual por API.

Transferência em outro contexto

Compare uma API de estados enumerados em TypeScript com uma alternativa de valores JavaScript e registre a evidência que orienta sua escolha.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. TypeScript 7.0 strict; Node.js 24 LTS. Typecheck avalia assinaturas; a representação runtime deve ser confirmada no JavaScript emitido e executada em Node. Diferencie typecheck de JavaScript executado. | Próxima sugestão: 18.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18 Utility types e relações entre tipos

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 18.01 · 18.02 · 18.03 · 18.04 · 18.05 · 18.06 · 18.07 · 18.08 · 18.09 · 18.10

18.01 Derive, não replique

READ · 40 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.derive_types | Requer: ts.basic, ts.generics | Introduz: ts.derive_types

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 1 · depuração 2 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Derive os aliases pedidos a partir das declarações canônicas. Para cada um, indique se a fonte é um valor, uma propriedade, uma união de chaves ou uma função; diferencie o tipo estático do dado runtime.

Objetivo da atividade

Antes de alterar o código, preveja quais declarações ainda existem no JavaScript emitido e quais são somente tipos.

Explicação e modelo mental

• Modelo mental: Escolha a operação conforme a fonte: indexed access lê o tipo de uma propriedade; typeof nomeia o tipo de um valor; keyof extrai as chaves; ReturnType deriva o retorno declarado.
• Exemplo resolvido: Quando um mapa de regiões em execução é a fonte oficial dos rótulos, seu conjunto de chaves pode alimentar o contrato estático para evitar cópias divergentes.
• Armadilha comum: Uma consulta de tipo não converte nem valida um valor runtime; copiar manualmente o tipo derivado pode deixá-lo desatualizado.
• Por que importa: Derivações tornam as dependências entre contratos explícitas e reduzem duplicação suscetível a divergências.

Material/arquivo inicial

tipos.ts: // 18.01 · Derive, não replique.
// Cada alias abaixo copia à mão algo que já existe. Troque cada cópia por uma derivação
// e indique a fonte: valor runtime, propriedade de tipo, união de chaves ou função.

```
export type User = { id: string; nome: string };
export const routes = { home: "/", perfil: "/perfil" };
export function criarPedido(itens: string[]) {
return { id: crypto.randomUUID(), itens, criadoEm: Date.now() };
}
export const STATUS = ["aberto", "pago", "enviado"] as const;

// Cópias manuais
export type UserId = string; // fonte: ___
export type RouteName = "home" | "perfil"; // fonte: ___
export type Pedido = { id: string; itens: string[]; criadoEm: number }; // fonte: ___
export type Status = "aberto" | "pago" | "enviado"; // fonte: ___

// Experimento: mude User.id para number. Quais aliases acompanham a mudança depois da sua refatoração? ___
// Quais desses nomes existem em runtime e quais somem no JavaScript emitido? ___
```

Exemplo e resultado público

type UserId=User['id']; type RouteName=keyof typeof routes → acompanha fonte

Bordas e erros publicados

mudar User.id para number -> UserId muda; cópia manual string fica incorreta; fonte runtime vs tipo identificada

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual operador forma a união dos nomes de propriedades de um tipo?

 (Correta) keyof
• typeof
• ReturnType
• Awaited

Por que a resposta está certa

Correto: keyof extrai as chaves disponíveis no tipo de objeto.

Armadilha da alternativa incorreta

typeof descreve o tipo de um valor; ReturnType e Awaited atuam sobre relações diferentes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

mudar User.id para number → UserId muda; cópia manual string fica incorreta; fonte runtime vs tipo identificada

Concepção errada e falha observável

Repetir contratos manualmente e aceitar divergência silenciosa. / Na entrada mudar User.id para number, a saída observável não corresponde a UserId muda; a divergência é compatível com esta hipótese: Repetir contratos manualmente e aceitar divergência silenciosa.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Repetir contratos manualmente e aceitar divergência silenciosa.

Pistas graduais

1. Localize fonte canônica. 2. Use indexed access/typeof/keyof/ReturnType conforme fonte. 3. Altere a fonte e observe o tipo derivado.

Solução de referência explicada

Substitua quatro duplicações por derivações com type tests positivos/negativos; declare onde uma cópia intencional é mais clara.

Transferência em outro contexto

Derive uma union de rótulos a partir de um mapa constante e decida qual declaração deve permanecer como fonte canônica.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. Derivações são contratos estáticos verificados pelo TypeScript; nenhum operador de tipo valida valor externo em runtime. Diferencie typecheck da execução do JavaScript emitido. | Próxima sugestão: 18.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.02 Parsons de patch

PARSONS · 45 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.partial_patch | Requer: ts.basic, ts.generics | Introduz: ts.partial_patch

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Implemente uma atualização parcial tipada com cópia de primeiro nível. Descreva como um objeto aninhado é tratado e que validação deve ocorrer se o patch vier de JSON.

Objetivo da atividade

Escreva o resultado esperado para uma alteração comum e uma propriedade composta antes de escolher a implementação de merge.

Explicação e modelo mental

• Modelo mental: Partial<T> torna propriedades opcionais no contrato estático; não altera recursivamente os tipos aninhados e não valida objetos recebidos.
• Exemplo resolvido: Um patch de preferências de apresentação pode deixar configurações superiores sem alteração; a função precisa documentar se um valor composto informado substitui ou combina sua contraparte.
• Armadilha comum: Assumir que uma utility faz merge profundo ou valida dados runtime promete comportamento que não foi implementado.
• Por que importa: Semântica explícita evita confundir opcionalidade de tipos com regras runtime de merge.

Material/arquivo inicial

patch.ts: export type Perfil={nome:string;tema:'claro'|'escuro'}; export type Usuario={id:number;nome:string;perfil:Perfil}; export function aplicarPatch<T extends object>(base:T,patch:any):T{return {...base,...patch};}

Exemplo e resultado público

update({a:1,b:'x'},{a:2}) → {a:2,b:'x'}

Bordas e erros publicados

patch em objeto aninhado -> substituição superficial; base intacta; exactOptionalPropertyTypes exige política de undefined

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que Partial<T> altera por si só?

• O merge recursivo de objetos aninhados
 (Correta) A possibilidade de omitir propriedades no tipo
• A validação de dados JSON
• O congelamento do objeto em runtime

Por que a resposta está certa

Correto: Partial altera a opcionalidade no contrato estático.

Armadilha da alternativa incorreta

Merge profundo, validação e congelamento requerem implementação runtime separada.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

patch em objeto aninhado → substituição superficial; base intacta; exactOptionalPropertyTypes exige política de undefined

Concepção errada e falha observável

Achar que Partial faz deep merge e permite qualquer JSON externo. / Na entrada patch em objeto aninhado, a saída observável não corresponde a substituição superficial; a divergência é compatível com esta hipótese: Achar que Partial faz deep merge e permite qualquer JSON externo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que Partial faz deep merge e permite qualquer JSON externo.

Pistas graduais

1. Partial só torna propriedades de T opcionais. 2. Spread faz merge de primeiro nível. 3. Retorne objeto novo e deixe runtime validation na fronteira.

Solução de referência explicada

return {...base,...patch}; documente merge superficial e assine patch:Partial<T> sem any.

Transferência em outro contexto

Projete uma atualização para preferências de notificação e especifique a política para uma configuração composta.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. TypeScript verifica os campos opcionais; spread executa em JavaScript e não faz merge profundo nem valida entrada externa. Diferencie typecheck da execução do JavaScript emitido. | Próxima sugestão: 18.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.03 Complete chave/valor

FADED · 40 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.keyof_indexed_access | Requer: ts.basic, ts.generics | Introduz: ts.keyof_indexed_access

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 1 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Complete a API genérica de acesso indexado do starter e use-a com um novo objeto de registro. Explique o limite que a relação entre chave e tipo de propriedade oferece.

Objetivo da atividade

Liste as chaves válidas do seu objeto e escreva o tipo de retorno sem recorrer a any.

Explicação e modelo mental

• Modelo mental: Uma chave genérica pode ser limitada ao conjunto de propriedades de T; o acesso indexado recupera o tipo associado à chave escolhida.
• Exemplo resolvido: Em um registro com priority:number e label:string, os acessos apontam para tipos de propriedade diferentes, de acordo com a chave selecionada.
• Armadilha comum: Trocar essa relação por string e any aceita erros de digitação estaticamente; ainda é necessário validar uma chave obtida de fonte externa.
• Por que importa: Relações entre chave e valor reduzem casts e mantêm o acesso alinhado ao shape declarado.

Material/arquivo inicial

acesso.ts: // 18.03 · Chave e valor relacionados pelo tipo.
// Complete a assinatura para que o retorno acompanhe a chave escolhida.

```
export function get<T, K extends ___>(obj: T, key: K): ___ {
return obj[key];
}

// Esperado depois de completar:
// get({ n: 2, s: "a" }, "n") -> number
// get({ n: 2, s: "a" }, "s") -> string
// get({ n: 2, s: "a" }, "x") -> erro de typecheck

// Use com um registro novo criado por você:
// const registro = ___;
// const valor = get(registro, ___);

// Limite: se registro veio de JSON.parse com um cast, o que este tipo garante de verdade? ___
```

Exemplo e resultado público

get({n:2,s:'a'},'n') → number; get(...,'s') -> string

Bordas e erros publicados

get(...,'x') -> erro TS; chave dinâmica keyof T retorna union conforme chave; runtime ainda pode ter dado externo inválido

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que escrever um helper genérico para acessar campos de diferentes registros em vez de usar um tipo amplo para todos?

 (Correta) Para reutilizar a operação mantendo a informação sobre a forma do registro fornecido pelo chamador.
• Para obrigar todos os registros a conter exatamente as mesmas propriedades.
• Para permitir que qualquer texto seja tratado como propriedade sem relação com o registro.

Por que a resposta está certa

Generics ajudam um helper a trabalhar com formas variadas sem perder a informação estática recebida.

Armadilha da alternativa incorreta

Um tipo amplo perde informação, enquanto um helper genérico pode preservar a forma de cada registro.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

get(...,'x') → erro TS; chave dinâmica keyof T retorna union conforme chave; runtime ainda pode ter dado externo inválido

Concepção errada e falha observável

Tipar key como string e retorno como any, perdendo relação. / Na entrada get(...,'x'), a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Tipar key como string e retorno como any, perdendo relação.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tipar key como string e retorno como any, perdendo relação.

Pistas graduais

1. K deve ser chave de T. 2. Retorno indexa T por K. 3. Teste chamada válida/inválida no typecheck.

Solução de referência explicada

Assinatura genérica relaciona chave e valor via T[K]; implementação retorna obj[key].

Transferência em outro contexto

Desenhe um setter para um registro de preferências cujo valor precisa corresponder à chave escolhida.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. O compilador verifica a relação chave/valor; entradas externas ainda precisam de validação quando não são confiáveis. Diferencie typecheck da execução do JavaScript emitido. | Próxima sugestão: 18.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.04 Utility type inadequado

MODIFY · 45 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.utility_readability | Requer: ts.basic, ts.generics | Introduz: ts.utility_readability

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 2 · arquivos 2 · leitura 5 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Substitua um alias excessivamente aninhado por um resumo público nomeado que represente o contrato de domínio. Compare exemplos aceitos e identifique quais informações ficam fora desse contrato.

Objetivo da atividade

Descreva os campos públicos em linguagem simples e dê um nome ao shape antes de decidir se precisa derivá-lo.

Explicação e modelo mental

• Modelo mental: Utility types ajudam quando a relação derivada faz parte do contrato; um alias nomeado é mais claro quando a forma pública é uma decisão deliberada.
• Exemplo resolvido: Um MemberBadge pode nomear só o identificador e o rótulo de exibição de que o consumidor precisa, independente do modelo interno completo.
• Armadilha comum: Encadear utilities só para evitar escrever uma forma curta pode ocultar a intenção; derivar um contrato público de um modelo interno amplo pode expor campos futuros acidentalmente.
• Por que importa: Nomes de domínio ajudam consumidores a entender a API e deixam modelos internos evoluírem sem alterações incidentais no resumo.

Material/arquivo inicial

perfil.ts: // 18.04 · Um alias que ninguém consegue ler.
// Substitua ProfileCard por um tipo nomeado de domínio, ProfileSummary, com só os campos públicos decididos.

```
type UserRecord = {
id: string;
name: string;
email: string;
passwordHash: string;
avatarUrl: string;
lastLoginIp: string;
createdAt: string;
};

export type ProfileCard = Pick<Omit<Partial<UserRecord>, "passwordHash" | "lastLoginIp">, "id" | "name" | "avatarUrl"> &
Required<Pick<UserRecord, "id" | "name">>;

// Escreva:
// export type ProfileSummary = { ___ };

// Teste de tipo: um resumo com campo secreto deve ser rejeitado.
// const vazado: ProfileSummary = { id: "u1", name: "Ana", passwordHash: "x" }; // esperado: erro

// Se UserRecord ganhar o campo phone, o que acontece com ProfileCard e com ProfileSummary? ___
```

Exemplo e resultado público

tipo nomeado com id,name,avatarUrl opcional → mesmo uso público claro

Bordas e erros publicados

mudar campo privado em origem -> resumo mantém apenas contrato decidido; teste de tipo rejeita campo secreto

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quando um alias de domínio nomeado pode ser a escolha mais clara?

• Quando qualquer utility type existir, ela deve ser substituída
• Quando o alias precisa acompanhar todo campo interno futuro
 (Correta) Quando o shape público é um contrato intencional que vale a pena nomear
• Quando o alias é usado para validar valores em runtime

Por que a resposta está certa

Correto: um contrato público deliberado pode ser mais legível com um nome de domínio.

Armadilha da alternativa incorreta

Utilities continuam úteis quando a relação importa; um alias estático não filtra nem valida objetos em runtime.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

mudar campo privado em origem → resumo mantém apenas contrato decidido; teste de tipo rejeita campo secreto

Concepção errada e falha observável

Usar utility types em cascata mesmo quando contrato de domínio fica ilegível. / Na entrada mudar campo privado em origem, a saída observável não corresponde a resumo mantém apenas contrato decidido; a divergência é compatível com esta hipótese: Usar utility types em cascata mesmo quando contrato de domínio fica ilegível.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar utility types em cascata mesmo quando contrato de domínio fica ilegível.

Pistas graduais

1. Escreva a forma desejada em linguagem de domínio. 2. Extraia aliases para relações reais. 3. Cheque equivalência de exemplos válidos e inválidos.

Solução de referência explicada

Refatore para alias legíveis por domínio e poucas derivações justificadas; type tests provam shape público.

Transferência em outro contexto

Defina um tipo resumido para um item de catálogo que possa evoluir independentemente do registro interno.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. O alias é verificado estaticamente; não remove propriedades em runtime nem valida objetos recebidos. Diferencie typecheck da execução do JavaScript emitido. | Próxima sugestão: 18.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.05 Mapa de handlers

TYPE · 40 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.record_exhaustiveness | Requer: ts.basic, ts.generics | Introduz: ts.record_exhaustiveness

Eixos: novidade 2 · sintaxe 3 · composição 3 · estado 2 · depuração 3 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Crie uma tabela completa de handlers para uma nova união finita de comandos. Mostre que mudança causa diagnóstico de handler ausente e explique quando um callback comum não expressa payloads diferentes.

Objetivo da atividade

Antes do typecheck, relacione cada nome da união a uma chave do mapa de handlers; preveja o erro se uma nova variante entrar na união sem handler.

Explicação e modelo mental

• Modelo mental: Record sobre uma união finita exige uma propriedade por membro e associa a cada chave o tipo de valor declarado.
• Exemplo resolvido: Um mapa de atalhos pode associar cada tecla a um callback uniforme; se eventos carregam dados com formas diferentes, a relação chave-payload precisa de uma modelagem mais específica.
• Armadilha comum: Partial<Record<...>> enfraquece intencionalmente a cobertura. Um callback uniforme também não representa payloads diferentes sem uma relação mapeada.
• Por que importa: Tabelas exaustivas tornam explícito quando uma API finita muda e falta trabalho a completar.

Material/arquivo inicial

handlers.ts: // 18.05 · Tabela de handlers completa por construção.
// Hoje todos os handlers recebem unknown. Crie uma union de comandos própria e um mapa em que
// cada comando tenha o payload correto.

```
type EventName = "open" | "close" | "error";
type Handler = (payload: unknown) => void;

export const handlers: Record<EventName, Handler> = {
open: (payload) => console.log("abriu", payload),
close: () => console.log("fechou"),
error: (payload) => console.log("erro", payload),
};

export function handle(nome: EventName, payload: unknown): void {
handlers[nome](payload);
}

// Tarefa:
// 1. Descreva os payloads: type Payloads = { open: { id: string }; close: undefined; error: { message: string } };
// 2. Tipar o mapa: { [K in keyof Payloads]: (payload: Payloads[K]) => void }
// 3. Remova error do mapa e anote o diagnóstico. Depois acrescente "retry" a Payloads e anote de novo.
// Quando um callback comum (payload: unknown) => void não basta? ___
```

Exemplo e resultado público

três chaves presentes → compila; handle('open') -> callback correspondente

Bordas e erros publicados

omitir error -> erro TS; adicionar variante retry -> novo handler exigido; payload diferente por evento pede mapped type específico

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que vantagem de manutenção uma tabela central de handlers pode oferecer?

• Espalhar decisões de despacho por módulos sem relação para reduzir a leitura do ponto de entrada.
 (Correta) Deixar a lógica de despacho visível em um único lugar para que a estrutura possa ser revista como unidade.
• Substituir nomes de comandos por strings arbitrárias em cada chamada.

Por que a resposta está certa

Uma tabela central pode tornar a organização das ações mais fácil de inspecionar.

Armadilha da alternativa incorreta

Dispersar ou tornar arbitrárias as decisões de despacho dificulta entender a estrutura da API.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

omitir error → erro TS; adicionar variante retry -> novo handler exigido; payload diferente por evento pede mapped type específico

Concepção errada e falha observável

Usar Partial<Record> e perder obrigação de cobertura. / Na entrada omitir error, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Usar Partial<Record> e perder obrigação de cobertura.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar Partial<Record> e perder obrigação de cobertura.

Pistas graduais

1. Record exige todas as chaves da union. 2. Inclua callback para cada evento. 3. Teste variante nova e observe erro de compilação.

Solução de referência explicada

Mapeie três handlers com Record, sem índice genérico string; documente limite para payloads heterogêneos.

Transferência em outro contexto

Crie uma Record completa de handlers para start, stop e pause; prove com um teste de tipo que cada comando tem handler e explique quando payloads diferentes exigem mapeamento por chave.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. A cobertura de Record é checada no typecheck; a execução ainda precisa tratar valores runtime não confiáveis. Diferencie typecheck da execução do JavaScript emitido. | Próxima sugestão: 18.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.06 satisfies vs annotation

COMPARE · 45 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.satisfies_literals | Requer: ts.basic, ts.generics | Introduz: ts.satisfies_literals

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 2 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Compare uma anotação ampla de mapa com a forma as const satisfies para uma configuração criada no starter. Derive as chaves em cada caso e explique qual sintaxe conserva informação literal.

Objetivo da atividade

Preveja o resultado de keyof para as duas declarações e só depois compare os diagnósticos do typecheck.

Explicação e modelo mental

• Modelo mental: satisfies verifica compatibilidade com um contrato sem substituir o tipo inferido; as const muda a inferência para literais readonly e são mecanismos distintos.
• Exemplo resolvido: Uma tabela de operações pode ser checada contra um formato comum e ainda conservar seus próprios nomes literais quando declarada adequadamente.
• Armadilha comum: Afirmar que satisfies sempre produz tipos literais é impreciso: a inferência depende também de como a expressão foi declarada.
• Por que importa: Preservar as chaves específicas permite validar nomes sem descartar informação útil do objeto.

Material/arquivo inicial

rotas.ts:
```
// 18.06 · satisfies versus anotação.
// Derive RouteName nas duas versões e compare.

type Rota = { path: string };

export const rotasAnotadas: Record<string, Rota> = {
home: { path: "/" },
about: { path: "/about" },
};

export const rotas = {
home: { path: "/" },
about: { path: "/about" },
} as const satisfies Record<string, Rota>;

export type RouteNameAnotada = keyof typeof rotasAnotadas; // previsão: ___
export type RouteName = keyof typeof rotas; // previsão: ___

export function dispatch(nome: RouteName): string {
return rotas[nome].path;
}

// Experimentos (descomente um por vez):
// dispatch("home"); // ___
// dispatch("missing"); // ___
// const ruim = { home: { path: 1 } } as const satisfies Record<string, Rota>; // ___
```

Exemplo e resultado público

RouteName → 'home'|'about'; dispatch('home') compila

Bordas e erros publicados

rota com path:number -> erro TS; dispatch('missing') -> erro TS; annotation Record<string,...> alarga keyof para string

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que as const acrescenta quando combinado com satisfies em uma configuração literal?

• Validação profunda dos dados em runtime
• Substituição do tipo inferido pelo tipo amplo alvo
• Aceitação irrestrita de chaves desconhecidas
 (Correta) Inferência literal readonly para a expressão

Por que a resposta está certa

Correto: as const influencia a inferência literal; satisfies verifica compatibilidade com o shape.

Armadilha da alternativa incorreta

Nenhuma das outras propriedades decorre de as const ou satisfies; runtime precisa de validação própria.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

rota com path:number → erro TS; dispatch('missing') -> erro TS; annotation Record<string,...> alarga keyof para string

Concepção errada e falha observável

Achar que anotação ampla e satisfies preservam as mesmas chaves literais. / Na entrada rota com path:number, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Achar que anotação ampla e satisfies preservam as mesmas chaves literais.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que anotação ampla e satisfies preservam as mesmas chaves literais.

Pistas graduais

1. Compare tipo inferido de cada declaração. 2. satisfies verifica forma sem substituir tipo inferido. 3. Use keyof typeof rotas para nomes.

Solução de referência explicada

Mostre ambas variantes e type tests; dispatcher aceita apenas keyof do objeto verificado.

Transferência em outro contexto

Valide uma tabela de operações e derive uma união de nomes sem transformar a tabela em uma API aberta.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. satisfies foi introduzido no TypeScript 4.9; const assertions preservam literais. Typecheck verifica a declaração, sem validar runtime. Diferencie typecheck e execução do JavaScript emitido. | Próxima sugestão: 18.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.07 Indice seguro

TYPE · 40 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.safe_indexing | Requer: ts.basic, ts.generics | Introduz: ts.safe_indexing

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Com noUncheckedIndexedAccess habilitado, escreva uma consulta a um índice por chave desconhecida e trate ausência sem usar non-null assertion. Indique a configuração que produz essa garantia.

Objetivo da atividade

Compare o JavaScript emitido com a flag ligada e desligada; anote o que muda na análise estática e o que permanece igual em execução.

Explicação e modelo mental

• Modelo mental: Um index signature descreve tipos de valores possíveis, mas não prova que qualquer chave dinâmica exista. A flag adiciona undefined a acessos não comprovados.
• Exemplo resolvido: A opção altera diagnósticos do typechecker para leituras indexadas; JavaScript emitido não ganha validação automática, então a fronteira de entrada ainda escolhe o comportamento.
• Armadilha comum: Record<string,T> não preenche todas as strings possíveis; ! apenas silencia a checagem e não cria a entrada ausente.
• Por que importa: Representar ausência no contrato torna consultas por chaves dinâmicas mais seguras.

Material/arquivo inicial

usuarios.ts: // 18.07 · Índice por chave desconhecida, com noUncheckedIndexedAccess ligado.
// Rode npx tsc --noEmit --strict --noUncheckedIndexedAccess usuarios.ts (numa pasta sem tsconfig.json).
// Trate a ausência sem !.

```
type User = { id: string; nome: string };

const porId: Record<string, User> = {
u1: { id: "u1", nome: "Ana" },
};

export function nomeDe(id: string): string {
const usuario = porId[id];
return usuario.nome;
}

// Escolha a política para id ausente (fallback ou erro explícito) e registre:
// nomeDe("u1") -> "Ana"
// nomeDe("u9") -> ___
// Compile com e sem a flag. O JavaScript emitido muda? ___
```

Exemplo e resultado público

id existente → User; id ausente -> undefined tratado sem crash

Bordas e erros publicados

acesso ausente -> fallback/erro explícito; noUncheckedIndexedAccess impede assumir User; sem !

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual vantagem há em registrar uma política de checagem de tipos no tsconfig compartilhado do projeto?

• O compilador passa a validar em runtime todos os dados fornecidos por usuários.
• As regras do projeto deixam de se aplicar quando o código é compilado em CI.
 (Correta) A equipe e os builds aplicam a mesma análise sem depender de escolhas locais de cada editor.

Por que a resposta está certa

A configuração compartilhada torna a checagem consistente entre ambientes de desenvolvimento e build.

Armadilha da alternativa incorreta

Configuração de tipos não insere validação runtime, e deve ser aplicada também nos builds do projeto.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

acesso ausente → fallback/erro explícito; noUncheckedIndexedAccess impede assumir User; sem !

Concepção errada e falha observável

Achar que Record<string,T> garante que toda string existe em runtime. / Na entrada acesso ausente, a saída observável não corresponde a fallback/erro explícito; a divergência é compatível com esta hipótese: Achar que Record<string,T> garante que toda string existe em runtime.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que Record<string,T> garante que toda string existe em runtime.

Pistas graduais

1. Ative flag declarada. 2. Leia retorno como User|undefined. 3. Cheque ausência antes de acessar nome.

Solução de referência explicada

function find(id:string):User|undefined{return dict[id]}; caller guarda undefined ou lança erro contextual.

Transferência em outro contexto

Projete uma consulta segura a um catálogo indexado por slug e decida como apresentar uma chave desconhecida.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. A opção precisa estar habilitada no typecheck para adicionar undefined; Node executa o acesso e pode devolver undefined. Diferencie typecheck e execução do JavaScript emitido. | Próxima sugestão: 18.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.08 Microprojeto - formulario tipado com satisfies

PROJECT · 85 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.5/5

Competência: ts.form_satisfies | Requer: ts.basic, ts.generics | Introduz: ts.form_satisfies

Eixos: novidade 4 · sintaxe 5 · composição 5 · estado 4 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Construa uma configuração de filtros tipada e derive nomes e tipos dos campos. Implemente leitura de valores submetidos com conversão e validação explícitas, sem confiar só na configuração estática.

Objetivo da atividade

Derive as chaves e identifique cada conversão exigida antes de retornar um valor tipado.

Explicação e modelo mental

• Modelo mental: Uma configuração literal pode ser fonte de chaves e discriminantes; mapped/conditional types expressam relações estáticas. Dados de formulário são externos e exigem análise em runtime.
• Exemplo resolvido: Uma configuração com filtro textual e filtro de seleção pode orientar o tipo esperado; o texto submetido só vira esse tipo depois de parse e validação.
• Armadilha comum: satisfies e tipos derivados verificam a configuração no código, mas não convertem nem validam automaticamente dados recebidos de formulário.
• Por que importa: Derivação estática evita duplicar nomes; validação runtime protege a fronteira que recebe dados reais.

Material/arquivo inicial

filtros.ts: // 18.08 · Formulário de filtros tipado a partir da configuração.
// O tipo derivado diz o que cada campo deve ser; a leitura do formulário continua recebendo texto.

```
type Field = { kind: "text" } | { kind: "number"; min?: number };

export const config = {
nome: { kind: "text" },
idade: { kind: "number", min: 0 },
} as const satisfies Record<string, Field>;

export type FieldName = keyof typeof config;

// Complete: number para campos "number", string para campos "text".
export type Valores = { [K in FieldName]: ___ };

// Implemente: recebe os valores submetidos (sempre texto) e devolve Valores,
// convertendo e validando cada campo. Valor inválido lança RangeError com o nome do campo.
export function lerFormulario(enviado: Record<string, string>): Valores {
// implemente
}

// Casos para conferir:
// lerFormulario({ nome: " Ana ", idade: "30" }) -> { nome: "Ana", idade: 30 }
// lerFormulario({ nome: "Ana", idade: "-1" }) -> RangeError (idade)
// lerFormulario({ nome: "Ana" }) -> ___ (decida e documente)
```

Exemplo e resultado público

FieldName → 'nome'|'idade'; idade recebe número em mapeamento tipado

Bordas e erros publicados

kind inválido -> erro TS; chave inexistente -> erro TS; valor de nome não aceita número; runtime form data continua string antes de conversão

Teste público e aprovação visível

Resultado observável: FieldName -> 'nome'|'idade'; idade recebe número em mapeamento tipado. Borda a conferir: kind inválido -> erro TS; chave inexistente -> erro TS; valor de nome não aceita número; runtime form data continua string antes de conversão. Entrega esperada: Config é fonte dos nomes; derive FieldName e FieldValue<K>, implemente leitura validada por kind com testes de tipo e runtime.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que derivar nomes e tipos de campos a partir de uma única configuração de formulário?

 (Correta) Para reduzir declarações duplicadas que poderiam divergir e manter as relações estáticas próximas à configuração.
• Para garantir que valores enviados pelo usuário já estejam convertidos sem código adicional.
• Para tornar cada campo independente do restante da configuração.

Por que a resposta está certa

Uma fonte comum reduz repetição e ajuda a manter a configuração e seus tipos alinhados.

Armadilha da alternativa incorreta

Derivação de tipos não executa conversão, e separá-los sem necessidade pode criar divergência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

kind inválido → erro TS; chave inexistente -> erro TS; valor de nome não aceita número; runtime form data continua string antes de conversão

Concepção errada e falha observável

Validar shape de config e perder relação chave->tipo do campo. / Na entrada kind inválido, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Validar shape de config e perder relação chave->tipo do campo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Validar shape de config e perder relação chave->tipo do campo.

Pistas graduais

1. Use satisfies para validar forma sem alargar chaves. 2. Mapeie kind para tipo de valor por conditional type simples. 3. FormData ainda precisa parse runtime.

Solução de referência explicada

Config é fonte dos nomes; derive FieldName e FieldValue<K>, implemente leitura validada por kind com testes de tipo e runtime.

Transferência em outro contexto

Descreva a API de leitura de um filtro configurável e identifique o parser necessário para uma categoria nova.

Evidência de domínio

Resultado observável: FieldName -> 'nome'|'idade'; idade recebe número em mapeamento tipado. Borda a conferir: kind inválido -> erro TS; chave inexistente -> erro TS; valor de nome não aceita número; runtime form data continua string antes de conversão. Entrega esperada: Config é fonte dos nomes; derive FieldName e FieldValue<K>, implemente leitura validada por kind com testes de tipo e runtime. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. tsc verifica relações derivadas; Node executa parse e validação, que satisfies não fornece. Diferencie typecheck e execução do JavaScript emitido. | Próxima sugestão: 18.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.09 Import de tipo não roda

TRACE · 45 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.import_type_runtime | Requer: ts.basic, ts.generics | Introduz: ts.import_type_runtime

Eixos: novidade 4 · sintaxe 3 · composição 3 · estado 2 · depuração 3 · arquivos 4 · leitura 5 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Compare os módulos do exercício antes e depois da compilação. Identifique quais dependências existem para o typecheck, quais permanecem como valores e como confirmar no artefato emitido o que Node carregará.

Objetivo da atividade

Classifique cada dependência dos exemplos pela função que desempenha e confirme no output emitido, sem presumir comportamento pela anotação de tipo.

Explicação e modelo mental

• Modelo mental: Uma dependência usada só como tipo pode desaparecer do JavaScript; dependências de valor permanecem conforme a sintaxe e o uso. Um efeito lateral intencional deve ser expresso claramente.
• Exemplo resolvido: Observe as importações que restam no arquivo JavaScript e os efeitos registrados ao carregá-lo; o resultado emitido é a evidência do programa runtime.
• Armadilha comum: Inferir execução a partir da presença de um nome no código-fonte pode confundir uma dependência estática com carregamento de módulo.
• Por que importa: Sintaxe explícita ajuda a prever e revisar a inicialização do programa executado.

Material/arquivo inicial

consumidor.ts: // 18.09 · O que roda e o que some.
// Compile com npx tsc consumidor.ts --outDir saida --target es2024 --module nodenext --verbatimModuleSyntax --rewriteRelativeImportExtensions
// (package.json com "type": "module") e compare consumidor.ts com saida/consumidor.js.
// O Node.js 24 também roda o .ts direto: compare node consumidor.ts com node saida/consumidor.js.

```
import type { Schema } from "./side.ts";
import { init } from "./side.ts";

const schema: Schema = { campos: ["nome"] };
init(schema);

// Perguntas:
// 1. Quais imports aparecem em saida/consumidor.js? ___
// 2. Quantas vezes "side carregado" aparece ao rodar node saida/consumidor.js? ___
// 3. Troque import type { Schema } por import { Schema }. O que verbatimModuleSyntax diz? ___
// 4. Se um módulo só tivesse tipos e efeito colateral, como manter o efeito? (dica: import "./side.ts") ___
```

| side.ts: // Módulo com efeito colateral: avaliar este arquivo incrementa um contador global.

```
const global = globalThis as { carregamentos?: number };
global.carregamentos = (global.carregamentos ?? 0) + 1;
console.log("side carregado", global.carregamentos);

export type Schema = { campos: string[] };

export function init(schema: Schema): void {
console.log("init", schema.campos.join(","));
}
```

Exemplo e resultado público

import type Schema → apagado no JS, side effect ausente; import init -> módulo avaliado

Bordas e erros publicados

import de valor usado só como tipo -> diagnóstico sob verbatimModuleSyntax; import './side.js' explícito mantém efeito

Teste público e aprovação visível

Compare estados e ordem; peça explicação curta sobre a primeira divergência.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como verificar se a organização dos módulos compilados pode ser consumida pelo runtime Node declarado?

• Confiar apenas na navegação de tipos do editor antes da compilação.
 (Correta) Testar o artefato compilado em um consumidor compatível com o formato de módulo e o runtime previstos.
• Examinar somente os nomes dos arquivos-fonte, sem consultar a saída gerada.

Por que a resposta está certa

O consumidor deve exercitar a saída que o runtime realmente receberá.

Armadilha da alternativa incorreta

Informação do editor e nomes de origem não confirmam por si sós como o artefato compilado será consumido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

import de valor usado só como tipo → diagnóstico sob verbatimModuleSyntax; import './side.js' explícito mantém efeito

Concepção errada e falha observável

Achar que import type executa arquivo para inicialização. / Na entrada import de valor usado só como tipo, a saída observável não corresponde a diagnóstico sob verbatimModuleSyntax; a divergência é compatível com esta hipótese: Achar que import type executa arquivo para inicialização.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que import type executa arquivo para inicialização.

Pistas graduais

1. Compile e leia JS emitido. 2. Separe tipo de valor na sintaxe. 3. Adicione import de efeito só quando efeito for intencional.

Solução de referência explicada

Demonstre emit e logs com três importações; ajuste consumer conforme intenção sem depender de elisão implícita.

Transferência em outro contexto

Audite a dependência de um módulo de registro e escreva como declarar explicitamente a intenção de inicialização.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. TypeScript 7.0 strict; Node.js 24 LTS. tsc confere posições de tipo e valor; o output e a avaliação devem ser observados em Node. Diferencie typecheck e execução do JavaScript emitido. | Próxima sugestão: 18.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

18.10 Utility types com propósito

MODIFY · 50 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.utility_purpose | Requer: ts.basic, ts.generics | Introduz: ts.utility_purpose

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 2 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Refatore os tipos derivados da API de usuários do starter: loadUser assíncrona, handle(input:string) e a união de eventos. Use Awaited, NonNullable, Exclude, Extract, ReturnType e Parameters apenas onde cada transformação preserva uma relação útil com a fonte; explique cada alias e mantenha claro que tipos não validam valores em runtime.

Objetivo da atividade

Para cada tipo pedido, anote sua fonte e explique em uma frase o que a transformação preserva.

Explicação e modelo mental

• Modelo mental: Utility types aplicam transformações comuns: derivar parâmetros e retornos, obter o tipo resolvido de Promise, remover nullish e selecionar ou excluir membros de união.
• Exemplo resolvido: Uma função pública de normalização fornece uma fonte para seu tipo de retorno; um handler nomeado fornece uma fonte separada para o tipo de argumento.
• Armadilha comum: Composições sem uma pergunta clara escondem a relação em vez de documentá-la; tipos derivados não validam valores reais em runtime.
• Por que importa: Derivações acompanham contratos públicos quando a relação é real, e aliases intermediários podem manter composições legíveis.

Material/arquivo inicial

api.ts:
```
// 18.10 · Utility types com propósito.
// Os aliases abaixo foram copiados à mão. Refatore usando Awaited, ReturnType, NonNullable,
// Parameters, Exclude e Extract só onde a transformação preserva uma relação com a fonte.
// Explique cada alias em uma frase.

type User = { id: string; nome: string };
type Result = { ok: true; valor: number } | { ok: false; erro: string };
export type Evento = "ok" | "error" | "retry";

export async function loadUser(id: string): Promise<User | null> {
return id === "u1" ? { id, nome: "Ana" } : null;
}

export function handle(input: string): Result {
const n = Number(input);
return Number.isFinite(n) ? { ok: true, valor: n } : { ok: false, erro: não é número: ${input} };
}

// Cópias manuais
export type LoadedUser = User | null; // Awaited<ReturnType<typeof loadUser>> ?
export type PresentUser = User; // ___
export type HandleInput = string; // ___
export type SemErro = "ok" | "retry"; // ___
export type SoErro = "error"; // ___

// Lembrete: nenhum desses tipos valida o que chega em runtime. Onde a validação precisa acontecer? ___
```

Exemplo e resultado público

Awaited<ReturnType<typeof loadUser>> → User|null; NonNullable remove null

Bordas e erros publicados

Exclude<'ok'|'error','error'> -> 'ok'; Extract seleciona 'error'; Parameters<typeof handle>[0] -> string; types não validam runtime

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual raciocínio ajuda a escolher uma utility type sem empilhar transformações sem propósito?

• Aplicar o maior número possível de utilities para deixar o alias mais genérico.
• Escolher uma utility pelo nome mais curto, sem relacioná-la à API de origem.
 (Correta) Identificar o tipo de origem e a relação desejada, escolhendo a transformação que expressa essa relação.

Por que a resposta está certa

Cada alias deve partir de uma fonte identificável e representar uma transformação útil para a API.

Armadilha da alternativa incorreta

Empilhar utilities sem uma relação definida torna o alias difícil de justificar e manter.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Exclude<'ok'|'error','error'> → 'ok'; Extract seleciona 'error'; Parameters<typeof handle>[0] -> string; types não validam runtime

Concepção errada e falha observável

Empilhar utilities sem relação clara com API de origem. / Na entrada Exclude<'ok'|'error','error'>, a saída observável não corresponde a 'ok'; a divergência é compatível com esta hipótese: Empilhar utilities sem relação clara com API de origem.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Empilhar utilities sem relação clara com API de origem.

Pistas graduais

1. Escolha uma fonte pública para cada derivação. 2. Anote que relação cada utility preserva. 3. Type tests devem quebrar quando origem mudar.

Solução de referência explicada

Refatore aliases de resultado/parâmetro/evento com Awaited, NonNullable, Exclude, Extract, ReturnType, Parameters e explique por que cada um existe.

Transferência em outro contexto

Derive um alias de resultado para uma chamada assíncrona de cobrança e escolha nomes intermediários que deixem a intenção clara.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Node.js 24 LTS. Utilities transformam tipos estáticos; não verificam os valores reais retornados em runtime. Diferencie typecheck e execução do JavaScript emitido. | Próxima sugestão: 19.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19 Tipos avançados com propósito

9 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 19.01 · 19.02 · 19.03 · 19.04 · 19.05 · 19.06 · 19.07 · 19.08 · 19.09

19.01 Expanda à mão

PREDICT · 18 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 1.5/5

Competência: ts.mapped_expand | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.mapped_expand

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 1 · arquivos 1 · leitura 2 · autonomia 1 · ferramentas 2

VISÃO DO ALUNO

Antes do typecheck, expanda Flags<User> no papel e escreva um type test para as três propriedades e uma atribuição incompatível.

Objetivo da atividade

Anote a expansão manual de Flags<User> para id, name e active; indique um teste que prove rejeição de uma chave ausente.

Explicação e modelo mental

• Modelo mental: Mapped types percorrem as chaves de um tipo no sistema estático e produzem outro tipo; nenhum laço é emitido para JavaScript.
• Exemplo resolvido: Para Source={readonly path:string;retry?:number}, cada chave deriva seu boolean sem perder necessariamente readonly/optional no mapeamento homomórfico.
• Armadilha comum: Ler {[K in keyof T]: boolean} como transformação em execução ou esquecer modificadores preservados.
• Por que importa: Expansões pequenas ajudam a revisar tipos gerados e decidir se a abstração representa relação útil.

Material/arquivo inicial

flags.ts: export type User={id:number;name:string;active:boolean}; export type Flags<T>={[K in keyof T]:unknown}; export type UserFlags=Flags<User>;

Exemplo e resultado público

Flags<User> → {id:boolean;name:boolean;active:boolean}

Bordas e erros publicados

User com readonly/optional -> mapped type homomórfico preserva modificadores salvo remoção explícita

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

condicao ? valorSeVerdadeiro : valorSeFalso é o operador ternário do JavaScript: uma expressão que escolhe um entre dois valores. Tipos condicionais usam a mesma forma, A extends B ? X : Y, só que a pergunta é "A cabe em B?".

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Em Flags<T> = {[K in keyof T]: boolean}, o que ocorre a id e name em User={id:number;name:string}?

 (Correta) O resultado tem id:boolean e name:boolean.
• O objeto recebe dois booleanos em runtime.
• O resultado conserva id:number e name:string.
• O resultado é sempre Record<string,boolean>.

Por que a resposta está certa

O mapeamento transforma estaticamente cada chave existente e conserva o conjunto de chaves.

Armadilha da alternativa incorreta

A expressão mapeada deriva propriedades para o checker; ela não percorre nem altera objeto em runtime.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

User com readonly/optional → mapped type homomórfico preserva modificadores salvo remoção explícita

Concepção errada e falha observável

Achar que mapped type executa loop em runtime. / Na entrada User com readonly/optional, a saída observável não corresponde a mapped type homomórfico preserva modificadores salvo remoção explícita; a divergência é compatível com esta hipótese: Achar que mapped type executa loop em runtime.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que mapped type executa loop em runtime.

Pistas graduais

1. Liste keyof User. 2. Substitua T[K] por boolean por chave. 3. Compare modificadores preservados no tipo resultante.

Solução de referência explicada

Expansão manual e type tests; explique que JS emitido não contém o tipo.

Transferência em outro contexto

Derive flags de permissão para um tipo Pedido e declare um caso de propriedade ausente.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; mapped types e keyof estão no baseline. Tipos são apagados do JS; exemplo executável usa Node.js 24 LTS. | Próxima sugestão: 19.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.02 Parsons condicional

PARSONS · 20 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.conditional_array | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.conditional_array

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Ordene ElementOrSelf<T>. Antes de compilar, escreva os resultados para array mutável, tupla readonly, escalar e união; valide com type tests.

Objetivo da atividade

Escreva os quatro tipos esperados, incluindo ElementOrSelf<readonly [1,2]> e ElementOrSelf<readonly number[]|boolean>.

Explicação e modelo mental

• Modelo mental: infer U captura o elemento quando T casa com um padrão de array; conditional type escolhe um dos dois ramos.
• Exemplo resolvido: ElementOrSelf<readonly bigint[]> captura bigint porque o padrão extrai elemento; uma tupla readonly usa o mesmo ramo.
• Armadilha comum: Esquecer readonly e rejeitar tuplas imutáveis, ou esquecer que parâmetro genérico nu pode distribuir sobre união.
• Por que importa: Ler transformações pequenas ajuda a entender tipos de retorno sem transformar o sistema de tipos em enigmas.

Material/arquivo inicial

element.ts: export type ElementOrSelf<T>=T extends readonly (infer U)[]?U:unknown;

Exemplo e resultado público

ElementOrSelf<readonly string[]> → string; ElementOrSelf<number> -> number

Bordas e erros publicados

ElementOrSelf<string[]|number> -> string|number por distributividade; tupla readonly também casa

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como verificar se um tipo condicional genérico representa corretamente as categorias de entrada previstas?

 (Correta) Escrever type tests para os casos incluídos no contrato e comparar cada resultado com o comportamento pretendido.
• Executar exemplos em runtime, pois isso comprova diretamente todos os resultados do typechecker.
• Testar apenas uma entrada feliz e presumir que as outras categorias seguem a mesma regra.

Por que a resposta está certa

Type tests verificam as relações estáticas previstas para as diferentes categorias do contrato.

Armadilha da alternativa incorreta

Execução runtime não substitui checagem de tipos, e um único caso não cobre todas as categorias declaradas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

ElementOrSelf<string[]|number> → string|number por distributividade; tupla readonly também casa

Concepção errada e falha observável

Esquecer readonly e falhar para array imutável. / Na entrada ElementOrSelf<string[]|number>, a saída observável não corresponde a string|number por distributividade; a divergência é compatível com esta hipótese: Esquecer readonly e falhar para array imutável.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Esquecer readonly e falhar para array imutável.

Pistas graduais

1. Teste primeiro T como readonly array. 2. infer U extrai item. 3. Ramo falso preserva T.

Solução de referência explicada

Ordene conditional type com readonly (infer U)[]; type tests para array, tuple e escalar.

Transferência em outro contexto

Derive o tipo de elemento de uma tupla readonly de códigos e mantenha o tipo de um escalar.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; conditional types/readonly tuples são recursos estáticos, sem conversão runtime. | Próxima sugestão: 19.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.03 Complete infer

FADED · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.infer_return | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.infer_return

Eixos: novidade 3 · sintaxe 4 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Complete Unwrap<T> e FnResult<T>. Preserve a diferença entre retorno Promise<Date> e o valor depois de Awaited; acrescente type tests positivos e negativos.

Objetivo da atividade

Registre tipos previstos para Unwrap<number>, FnResult<()=>string> e Awaited<FnResult<()=>Promise<Date>>>.

Explicação e modelo mental

• Modelo mental: infer nomeia uma parte de um padrão de tipo existente; Unwrap extrai de Promise e FnResult extrai retorno de função.
• Exemplo resolvido: type Box<T>={item:T}; type Payload<T>=T extends Box<infer Item>?Item:never; Payload<Box<boolean>> é boolean; infer nomeia uma parte do padrão estrutural.
• Armadilha comum: Achar que infer ou o tipo de função executa ou aguarda Promise em runtime.
• Por que importa: Extrair relações de uma API evita repetir assinaturas e deixa explícita a diferença entre resposta assíncrona e valor resolvido.

Material/arquivo inicial

unwrap.ts: // 19.03 · Complete os tipos com infer.

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true : false;
type Expect<T extends true> = T;

export type Unwrap<T> = T extends Promise<infer ___> ? ___ : T;
// any[] aqui só descreve "qualquer lista de parâmetros"; o resultado não vira any.
export type FnResult<T> = T extends (...args: any[]) => infer ___ ? ___ : never;

async function buscarData(): Promise<Date> {
return new Date();
}

// Type tests positivos (complete as previsões):
export type T1 = Expect<Equal<Unwrap<Promise<number>>, number>>;
export type T2 = Expect<Equal<Unwrap<number>, ___>>;
export type T3 = Expect<Equal<FnResult<() => string>, ___>>;
export type T4 = Expect<Equal<FnResult<typeof buscarData>, ___>>; // antes de Awaited
export type T5 = Expect<Equal<Awaited<FnResult<typeof buscarData>>, ___>>; // depois de Awaited

// Type test negativo: escreva um que deve falhar e deixe-o com @ts-expect-error.
// // @ts-expect-error ___
// export type T6 = Expect<Equal<FnResult<() => string>, number>>;
```

Exemplo e resultado público

Unwrap<Promise<number>> → number; FnResult<()=>string> -> string

Bordas e erros publicados

Unwrap<number> -> number; função async retorna Promise<string> até aplicar Unwrap; any[] aqui isolado à posição de parâmetros de tipo

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é a principal função de type tests para utilitários genéricos de tipos?

• Executar as funções JavaScript associadas aos tipos sem iniciar o runtime.
 (Correta) Registrar resultados esperados para tipos representativos e detectar mudanças acidentais no contrato estático.
• Substituir testes de comportamento mesmo quando o utilitário também altera código executável.

Por que a resposta está certa

Type tests preservam o comportamento de checagem que a API de tipos promete.

Armadilha da alternativa incorreta

Testes de tipos não executam funções e não substituem testes de runtime quando há comportamento executável.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Unwrap<number> → number; função async retorna Promise<string> até aplicar Unwrap; any[] aqui isolado à posição de parâmetros de tipo

Concepção errada e falha observável

Confundir retorno async com valor já aguardado. / Na entrada Unwrap<number>, a saída observável não corresponde a number; a divergência é compatível com esta hipótese: Confundir retorno async com valor já aguardado.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir retorno async com valor já aguardado.

Pistas graduais

1. infer captura parte de um padrão de tipo. 2. Primeiro extraia ReturnType, depois Awaited se necessário. 3. Type tests verificam ambos os níveis.

Solução de referência explicada

Implemente duas aliases com infer e compare a versão de biblioteca Awaited<ReturnType<F>>.

Transferência em outro contexto

Extraia o valor resolvido de uma função assíncrona de busca e compare o tipo antes e depois de aguardar.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; infer e Awaited estão no baseline. O resultado de FnResult continua sendo um tipo estático. | Próxima sugestão: 19.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.04 Distributividade surpresa

DEBUG · 25 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.conditional_distribution | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.conditional_distribution

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 2 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Investigue por que IsString<string|number> produz uma união. Compare parâmetro nu com forma em tupla; escreva type tests para string, união e never e explique a pergunta que cada versão responde.

Objetivo da atividade

Preveja IsString<string|number>, IsString<never> e WholeString<string|number> antes de editar.

Explicação e modelo mental

• Modelo mental: Conditional type distributivo testa cada membro quando parâmetro genérico nu está à esquerda de extends; tuplas fazem o teste agregado.
• Exemplo resolvido: IsString<'draft'|'sent'> resulta true para cada membro textual; a versão agregada pergunta se a união inteira cabe em string.
• Armadilha comum: Supor que todo conditional type trata a união como unidade ou mudar distributividade sem checar never.
• Por que importa: Semântica diferente altera contrato de biblioteca; previsões e type tests deixam a mudança revisável.

Material/arquivo inicial

is-string.ts: // 19.04 · Por que IsString<string | number> virou uma união?

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true : false;
type Expect<T extends true> = T;

export type IsString<T> = T extends string ? true : false;

// Este type test falha hoje. Leia o diagnóstico antes de mexer:
export type Bug = Expect<Equal<IsString<string | number>, false>>;

// Crie WholeString<T> com o parâmetro "vestido" numa tupla: [T] extends [string] ? true : false
// export type WholeString<T> = ___;

// Type tests para as duas versões (complete as previsões):
// IsString<string> -> ___ WholeString<string> -> ___
// IsString<string | number> -> ___ WholeString<string | number> -> ___
// IsString<never> -> ___ WholeString<never> -> ___

// Que pergunta cada versão responde? Qual o requisito do produto pede? ___
```

Exemplo e resultado público

IsString<string|number> → true|false; [T] extends [string] -> false

Bordas e erros publicados

never na versão distributiva -> never; com tupla o comportamento deve ser testado separadamente; não substituir sem analisar requisito

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que melhora a manutenção de uma transformação escrita com tipos condicionais complexos?

• Comprimir todas as condições em uma expressão sem exemplos que documentem seu uso.
• Adicionar casts em cada consumidor para evitar explicar a transformação.
 (Correta) Usar nomes descritivos e type tests que mostrem a intenção da transformação.

Por que a resposta está certa

Nomes e exemplos de tipos tornam a regra mais fácil de revisar e preservar.

Armadilha da alternativa incorreta

Expressões opacas e casts espalhados escondem a intenção em vez de documentá-la.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

never na versão distributiva → never; com tupla o comportamento deve ser testado separadamente; não substituir sem analisar requisito

Concepção errada e falha observável

Achar que conditional type sempre trata union como bloco. / Na entrada never na versão distributiva, a saída observável não corresponde a never; a divergência é compatível com esta hipótese: Achar que conditional type sempre trata union como bloco.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que conditional type sempre trata union como bloco.

Pistas graduais

1. Parâmetro nu à esquerda de extends distribui. 2. Envolva ambos lados em tupla para bloquear. 3. Teste union e never antes de escolher.

Solução de referência explicada

Crie IsStringWhole<T>=[T] extends [string]?true:false e explique mudança de semântica nos type tests.

Transferência em outro contexto

Especifique duas consultas de tipos para descobrir se todos os membros de uma união são datas.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; conditional types distributivos são de compile time e não têm comportamento runtime. | Próxima sugestão: 19.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.05 Eventos por template literal

TYPE · 25 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.template_event_names | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.template_event_names

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Complete a assinatura genérica para que ${K}Changed preserve a chave do campo e o callback receba Model[K]. Inclua uma chamada aceita e outra rejeitada.

Objetivo da atividade

Registre o valor que o parâmetro K deve preservar e crie uma chamada com chave fora do modelo.

Explicação e modelo mental

• Modelo mental: Template literal types constroem nomes a partir das chaves; K conecta nome do evento a Model[K].
• Exemplo resolvido: Em Product={title:string;price:number}, priceChanged liga o callback a number; K é a mesma chave que seleciona o payload.
• Armadilha comum: Gerar nomes de evento mas perder a relação entre cada nome e payload, ou achar que o tipo registra listeners em runtime.
• Por que importa: Uma API detecta erros de nome/payload sem duplicar contratos; mensagem externa ainda precisa de validação runtime.

Material/arquivo inicial

on.ts: // 19.05 · Eventos por template literal.
// Complete a assinatura para ligar cada evento ao tipo do campo correspondente.
// O nome do evento segue a forma ${K}Changed.

```
type Model = { name: string; age: number };

declare function on(event: string, listener: (value: unknown) => void): void;

// Devem compilar depois da correção:
on("nameChanged", (value) => value.toUpperCase());
on("ageChanged", (value) => value.toFixed());

// Devem ser rejeitadas (descomente e confirme o diagnóstico):
// on("otherChanged", (value) => value);
// on("ageChanged", (value: string) => value.trim());
```

Exemplo e resultado público

on('nameChanged',v=>v.toUpperCase()) → v string; on('ageChanged',v=>v.toFixed()) -> v number

Bordas e erros publicados

on('otherChanged') -> erro TS; age callback tratando string -> erro TS; chave K restrita a string & keyof Model

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Para Product com K=price e price:number, que tipo Product[K] recebe o callback?

• string.
 (Correta) number.
• number | string porque o modelo tem mais de uma chave.
• any porque o evento é construído por texto.

Por que a resposta está certa

K conecta a chave price ao indexed access Product[K].

Armadilha da alternativa incorreta

Uma union de nomes sozinha perderia essa relação; K a preserva.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

on('otherChanged') → erro TS; age callback tratando string -> erro TS; chave K restrita a string & keyof Model

Concepção errada e falha observável

Gerar nomes sem preservar vínculo com o tipo do payload. / Na entrada on('otherChanged'), a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Gerar nomes sem preservar vínculo com o tipo do payload.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Gerar nomes sem preservar vínculo com o tipo do payload.

Pistas graduais

1. Extraia K de keyof Model como string. 2. Monte literal ${K}Changed. 3. Passe Model[K] ao callback.

Solução de referência explicada

Assinatura on<K extends string & keyof Model>(event:${K}Changed, cb:(value:Model[K])=>void) com type tests.

Transferência em outro contexto

Crie nomes de eventos de atualização para os campos de uma configuração e associe cada callback ao campo correspondente.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; template literal types disponíveis desde TS 4.1, cobertos pelo baseline. Tipos não validam evento em runtime. | Próxima sugestão: 19.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.06 Expandir mapped type

PREDICT · 20 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.mapped_modifiers | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.mapped_modifiers

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 1 · depuração 2 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Preveja como -readonly e -? afetam X; complete MutableRequired<T>. Escreva type tests de atribuição, ausência e valor presente, explicitando a política de undefined.

Objetivo da atividade

Para cada propriedade de X, registre presença obrigatória/opcional e mutabilidade.

Explicação e modelo mental

• Modelo mental: Mapped modifiers alteram opcionalidade e mutabilidade das propriedades no tipo resultante.
• Exemplo resolvido: Uma preferência de rascunho pode estar ausente e também ser exposta por uma visão imutável; presença e possibilidade de edição são dimensões separadas.
• Armadilha comum: Achar que retirar ? elimina undefined do tipo de valor ou que readonly controla aliases em runtime.
• Por que importa: Transformar DTOs exige distinguir obrigação, valor e mutabilidade para evitar mudanças incompatíveis de API.

Material/arquivo inicial

mutable.ts: export type MutableRequired<T>=T; export type Settings=MutableRequired<{readonly theme?:'light'|'dark';readonly retries?:number}>;

Exemplo e resultado público

expansão → {a:string;b:number;c:boolean} sob exactOptionalPropertyTypes/semântica declarada

Bordas e erros publicados

mudar apenas +readonly -> propriedades não ficam required automaticamente; comparar typecheck de ausência de a e atribuição

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que -? faz em {-readonly [K in keyof T]-?: T[K]}?

• Remove tanto optional quanto undefined do tipo de valor.
• Mantém optional porque T pode conter propriedades opcionais.
 (Correta) Remove optional das propriedades mapeadas, mantendo T[K].
• Converte a propriedade em readonly e obrigatória.

Por que a resposta está certa

-? remove o modificador de opcionalidade; T[K] continua definindo o tipo do valor.

Armadilha da alternativa incorreta

O modificador afeta se a propriedade pode faltar; não apaga automaticamente undefined já incluído em T[K].

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

mudar apenas +readonly → propriedades não ficam required automaticamente; comparar typecheck de ausência de a e atribuição

Concepção errada e falha observável

Ignorar diferença entre remoção de optional e remoção de readonly. / Na entrada mudar apenas +readonly, a saída observável não corresponde a propriedades não ficam required automaticamente; a divergência é compatível com esta hipótese: Ignorar diferença entre remoção de optional e remoção de readonly.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Ignorar diferença entre remoção de optional e remoção de readonly.

Pistas graduais

1. Liste chaves e modificadores originais. 2. Aplique -readonly e -? separadamente. 3. Confira atribuição e omissão em type tests.

Solução de referência explicada

Expanda manualmente três chaves e comprove com compilador; documente detalhes de undefined no modo configurado.

Transferência em outro contexto

Transforme um tipo de parâmetros de configuração opcionais em um contrato normalizado; defina como tratar undefined.

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; mapped modifiers cobertos pelo baseline. exactOptionalPropertyTypes existe desde 4.4. | Próxima sugestão: 19.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.07 Orcamento de complexidade

READ · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.5/5

Competência: ts.complexity_budget | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.complexity_budget

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 1 · depuração 1 · arquivos 2 · leitura 5 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Compare uma alias recursiva pequena para árvore JSON com o utility recursivo de 20 linhas. Descreva primeiro um valor finito aceito; depois recomende simplificar, manter ou deferir a transformação, usando casos como evidência. Não implemente type-golf.

Objetivo da atividade

Registre um exemplo válido e um inválido de JsonValue e uma regra difícil de explicar no utility recursivo.

Explicação e modelo mental

• Modelo mental: Alias recursiva pode nomear estrutura finita; transformação recursiva arbitrária percorre tipos e pode custar leitura e typecheck.
• Exemplo resolvido: JsonValue pode conter array de JsonValue ou objeto cujos valores também são JsonValue; a alias não valida JSON recebido.
• Armadilha comum: Tratar toda recursão como proibida ou presumir que toda transformação recursiva é útil e barata.
• Por que importa: Limite de complexidade é decisão de manutenção: contratos e erros compreensíveis valem mais que sofisticação.

Material/arquivo inicial

json.ts: // 19.07 · Orçamento de complexidade: duas recursões, dois propósitos.
// Leia as duas, descreva um valor finito aceito por JsonValue e recomende: simplificar, manter ou deferir DeepDto.

```
// A — recursão que modela dados
export type JsonValue =
| null
| boolean
| number
| string
| readonly JsonValue[]
| { readonly [key: string]: JsonValue };

// B — recursão que transforma tipos, usada para um DTO de duas propriedades
type Primitivo = string | number | boolean | null | undefined;
export type DeepDto<T> = T extends Primitivo
? T
: T extends Date
? string
: T extends readonly (infer U)[]
? readonly DeepDto<U>[]
: T extends (...args: never[]) => unknown
? never
: T extends Map<infer K, infer V>
? Array<[DeepDto<K>, DeepDto<V>]>
: T extends Set<infer U>
? DeepDto<U>[]
: T extends object
? { -readonly [K in keyof T as T[K] extends (...args: never[]) => unknown ? never : K]-?: DeepDto<T[K]> }
: never;

type Pedido = { id: string; criadoEm: Date };
export type PedidoDto = DeepDto<Pedido>;

// Alternativa menor para o caso real: ___
// Valor finito aceito por JsonValue: ___
// Nenhum dos dois valida JSON em runtime: onde a validação acontece? ___
```

Exemplo e resultado público

JsonValue aninhado e alias de domínio com duas transforms simples → preservar os casos publicados, distinguir recursão que modela dados da que transforma tipos e justificar limite de complexidade

Bordas e erros publicados

transformação condicional ou mapped recursiva sem limite claro -> explicar custo e propor limite/deferimento; nenhum alias valida JSON em runtime

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que critério ajuda a decidir se uma transformação recursiva de tipos merece ser mantida?

 (Correta) Ela representa uma necessidade clara do domínio, tem comportamento verificável e continua compreensível para quem a mantém.
• Ela é preferível sempre que uma composição de tipos puder ser escrita de forma mais engenhosa.
• Ela deve ser mantida mesmo quando não há uso demonstrável, desde que o typechecker a aceite.

Por que a resposta está certa

Justifique a complexidade pelo uso e pela clareza; simplificar ou adiar também pode ser a melhor decisão.

Armadilha da alternativa incorreta

Compilar ou parecer engenhoso não basta para justificar uma transformação difícil de entender.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

transformação condicional ou mapped recursiva sem limite claro → explicar custo e propor limite/deferimento; nenhum alias valida JSON em runtime

Concepção errada e falha observável

Achar que toda recursão de tipo é proibida, ou que toda recursão sofisticada é segura e útil. / Na entrada transformação condicional ou mapped recursiva sem limite claro, a saída observável não corresponde a explicar custo e propor limite/deferimento; a divergência é compatível com esta hipótese: Achar que toda recursão de tipo é proibida, ou que toda recursão sofisticada é segura e útil.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que toda recursão de tipo é proibida, ou que toda recursão sofisticada é segura e útil.

Pistas graduais

1. Siga JsonValue até um array e um objeto aninhado. 2. Separe recursão para modelar dado de recursão para transformar qualquer tipo. 3. Compare exemplos válidos/inválidos e pare onde a regra fica mais clara em runtime ou tipo nomeado.

Solução de referência explicada

Relatório mostra JsonValue como noção awareness-level de recursão útil, simplifica ou mantém o utility com evidência equivalente e declara que validação de JSON continua runtime. Recursão de transforms complexas não é critério de domínio desta etapa; fica para extensão posterior.

Transferência em outro contexto

Avalie se uma árvore de comentários pede uma alias recursiva; escreva um caso-limite e separe validação de tipo.

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; aliases recursivas de dados cobertas pelo baseline. Transformações recursivas avançadas ficam fora do gate. | Próxima sugestão: 19.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.08 Microprojeto - DSL de eventos tipada

PROJECT · 90 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 4.5/5

Competência: ts.typed_emitter | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.typed_emitter

Eixos: novidade 4 · sintaxe 5 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Construa emitter genérico para eventos de catálogo. Escreva primeiro teste para evento-payload; implemente on/emit/off e revise remoção. A API pública preserva Events[K]; limite cast interno ao menor trecho possível.

Objetivo da atividade

Escreva type tests de payload para uma chave do mapa, mais o caso comportamental de off.

Explicação e modelo mental

• Modelo mental: Um parâmetro K serve como chave de Events e índice do payload, sem repetir unions.
• Exemplo resolvido: Num mapa de telemetria, readingCompleted pode mapear para {sensorId:string;at:number}; a chave do evento seleciona a forma do dado associada.
• Armadilha comum: Reduzir API a Record<string,unknown> e perder vínculo, ou achar que tipos protegem payload externo em runtime.
• Por que importa: Contratos específicos reduzem erros entre módulos; testes de comportamento ainda provam registro, emissão e remoção.

Material/arquivo inicial

emitter.ts: // 19.08 · Emitter de eventos de catálogo com payload ligado ao nome.
// A API pública preserva Events[K]. Se precisar de cast interno, limite-o ao menor trecho possível.

```
export type Events = {
itemAdded: { sku: string };
priceChanged: { sku: string; price: number };
itemRemoved: { sku: string };
};

export type Listener<P> = (payload: P) => void;

export class Emitter<E extends Record<string, unknown>> {
// Escolha a estrutura: por exemplo { [K in keyof E]?: Set<Listener<E[K]>> }

on<K extends keyof E>(evento: K, listener: Listener<E[K]>): void {
// implemente
}

off<K extends keyof E>(evento: K, listener: Listener<E[K]>): void {
// implemente
}

emit<K extends keyof E>(evento: K, payload: E[K]): void {
// implemente
}
}

// Devem ser rejeitadas pelo typecheck (deixe com @ts-expect-error no teste):
// new Emitter<Events>().emit("itemAdded", { sku: 1 });
// new Emitter<Events>().on("priceChanged", (p: { sku: string; price: string }) => {});
```

| emitter.test.ts: // Escreva primeiro: o teste do par evento-payload. Rode com node --test emitter.test.ts.

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { Emitter, type Events } from "./emitter.ts";

test("listener recebe o payload do próprio evento", () => {
const emitter = new Emitter<Events>();
const recebidos: Events["itemAdded"][] = [];
emitter.on("itemAdded", (payload) => recebidos.push(payload));
emitter.emit("itemAdded", { sku: "sku-1" });
assert.deepEqual(recebidos, [{ sku: "sku-1" }]);
});

test("off impede a próxima emissão", () => {
const emitter = new Emitter<Events>();
let chamadas = 0;
const listener = () => { chamadas++; };
emitter.on("itemRemoved", listener);
emitter.off("itemRemoved", listener);
emitter.emit("itemRemoved", { sku: "sku-1" });
assert.equal(chamadas, 0);
});

test("off é idempotente e listeners de eventos diferentes são isolados", () => {
const emitter = new Emitter<Events>();
let itensAdicionados = 0;
```

```
let preçosAlterados = 0;
const item = () => { itensAdicionados++; };
emitter.on("itemAdded", item);
emitter.on("priceChanged", () => { preçosAlterados++; });
emitter.off("itemAdded", item);
emitter.off("itemAdded", item);
emitter.emit("itemAdded", { sku: "sku-2" });
emitter.emit("priceChanged", { sku: "sku-2", price: 3 });
assert.equal(itensAdicionados, 0);
assert.equal(preçosAlterados, 1);
});

// Type tests: estas linhas devem continuar com erro de typecheck.
// @ts-expect-error sku precisa ser string
new Emitter<Events>().emit("itemAdded", { sku: 1 });
| tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"noUncheckedIndexedAccess": true,
"erasableSyntaxOnly": true,
"verbatimModuleSyntax": true,
"allowImportingTsExtensions": true,
"noEmit": true,
"types": ["node"]
}
}
| package.json: {
"name": "devpath-19-08",
"version": "1.0.0",
"type": "module"
}
| package-lock.json: {
"name": "devpath-19-08",
"version": "1.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "devpath-19-08",
"version": "1.0.0"
}
}
}
```

Exemplo e resultado público

emit('itemAdded',{sku:'sku-1'}) → listener itemAdded recebe payload {sku:string}

Bordas e erros publicados

emit('itemAdded',{sku:1}) -> erro TS; on('priceChanged',fn que espera string) -> erro TS; remover listener impede próxima emissão; runtime não valida payload externo

Teste público e aprovação visível

Resultado observável: emit('itemAdded',{sku:'sku-1'}) -> listener itemAdded recebe payload {sku:string}. Borda a conferir: emit('itemAdded',{sku:1}) -> erro TS; on('priceChanged',fn que espera string) -> erro TS; remover listener impede próxima emissão; runtime não valida payload externo. Entrega esperada: Emitter genérico usa Map de conjuntos com cast interno localizado/documentado, API pública fortemente tipada, testes de tipo e comportamento.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que limitar uma assertion necessária à implementação interna de uma biblioteca?

• Para exigir que consumidores façam a mesma assertion em toda chamada.
 (Correta) Para conter a parte menos verificável e manter uma superfície pública útil para consumidores.
• Para substituir a modelagem dos tipos públicos por any.

Por que a resposta está certa

Conter a assertion reduz o alcance da suposição e preserva a API tipada para quem a usa.

Armadilha da alternativa incorreta

Assertions públicas ou any amplo espalham a suposição para todos os consumidores.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

emit('itemAdded',{sku:1}) → erro TS; on('priceChanged',fn que espera string) -> erro TS; remover listener impede próxima emissão; runtime não valida payload externo

Concepção errada e falha observável

Usar Record<string,unknown> e perder vínculo evento-payload. / Na entrada emit('itemAdded',{sku:1}), a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Usar Record<string,unknown> e perder vínculo evento-payload.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar Record<string,unknown> e perder vínculo evento-payload.

Pistas graduais

1. Uma fonte Events mapeia nomes a payload. 2. K extends keyof Events conecta métodos. 3. Set de listeners por evento e off idempotente.

Solução de referência explicada

Emitter genérico usa Map de conjuntos com cast interno localizado/documentado, API pública fortemente tipada, testes de tipo e comportamento.

Transferência em outro contexto

Adicione sessionStarted e sessionExpired com payloads distintos ao emitter tipado e prove remoção de listener.

Evidência de domínio

Resultado observável: emit('itemAdded',{sku:'sku-1'}) -> listener itemAdded recebe payload {sku:string}. Borda a conferir: emit('itemAdded',{sku:1}) -> erro TS; on('priceChanged',fn que espera string) -> erro TS; remover listener impede próxima emissão; runtime não valida payload externo. Entrega esperada: Emitter genérico usa Map de conjuntos com cast interno localizado/documentado, API pública fortemente tipada, testes de tipo e comportamento. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; typecheck cobre associação estática; testes Node.js 24 cobrem registro/emit/off. Entrada externa segue requerendo validação. | Próxima sugestão: 19.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

19.09 IDs nominais por convenção

TYPE · 28 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.branded_ids | Requer: ts.generics, ts.keyof_indexed_access, ts.utility | Introduz: ts.branded_ids

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Implemente construtores de UserId e OrderId com validação de formato e marcas distintas. Escreva type tests para impedir troca; demonstre que assertion arbitrária contorna o checker e mantenha prova runtime no construtor.

Objetivo da atividade

Liste uma chamada aceita, uma rejeitada e uma string inválida que o construtor precisa recusar.

Explicação e modelo mental

• Modelo mental: Brand intersection separa strings no checker, mas a marca não é criada nem verificada automaticamente em execução.
• Exemplo resolvido: UserId e OrderId são marcas de domínio distintas, produzidas após a validação de strings externas; a marca sozinha não valida a entrada.
• Armadilha comum: Confundir brand com wrapper runtime ou tratar assertion como validação de entrada.
• Por que importa: IDs parecidos podem ser trocados por engano; validação de fronteira protege dados reais.

Material/arquivo inicial

ids.ts: // 19.09 · IDs nominais por convenção (branded types).
// A marca só tem valor se o construtor validar a string antes de aplicá-la.

```
export type UserId = string & { readonly __brand: "UserId" };
export type OrderId = string & { readonly __brand: "OrderId" };

// Implemente: aceite só /^u\d+$/ e lance RangeError caso contrário.
export function makeUserId(texto: string): UserId {
// implemente
}

// Implemente: aceite só /^o\d+$/.
export function makeOrderId(texto: string): OrderId {
// implemente
}

export function getUser(id: UserId): string {
return usuário ${id};
}

// Type tests:
// getUser(makeUserId("u1")); // compila
// getUser(makeOrderId("o1")); // deve falhar no typecheck
// getUser("u1"); // deve falhar: string sem validação não recebe marca
// getUser("qualquer" as UserId); // compila: o cast contorna o checker. O que protege o runtime? ___
// makeUserId("x1") // runtime: ___
```

Exemplo e resultado público

getUser(makeUserId('u1')) → compila; getUser(makeOrderId('o1')) -> erro TS

Bordas e erros publicados

string externa sem validação -> não recebe marca; cast arbitrário contorna TS; wrapper runtime dá separação real

Teste público e aprovação visível

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que problema de modelagem tipos marcados distintos ajudam a evitar?

• Para garantir que valores iguais em tempo de execução recebam sempre a mesma identidade nominal.
• Para inferir propriedades de domínio que não fazem parte da assinatura declarada.
 (Correta) Para distinguir conceitos diferentes que compartilham a mesma representação básica ao verificar chamadas.

Por que a resposta está certa

A marcação comunica uma distinção de domínio entre conceitos que poderiam parecer iguais pela representação.

Armadilha da alternativa incorreta

O benefício está em distinguir significados no modelo de tipos, não em alterar valores ou descobrir propriedades ausentes.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

string externa sem validação → não recebe marca; cast arbitrário contorna TS; wrapper runtime dá separação real

Concepção errada e falha observável

Achar que branding existe em JS ou valida dado externo sozinho. / Na entrada string externa sem validação, a saída observável não corresponde a não recebe marca; a divergência é compatível com esta hipótese: Achar que branding existe em JS ou valida dado externo sozinho.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que branding existe em JS ou valida dado externo sozinho.

Pistas graduais

1. Defina marcas distintas. 2. Exponha construtores que validam formato. 3. Compare com objeto wrapper runtime para fronteira forte.

Solução de referência explicada

Construtores checam prefixo/formato e fazem cast isolado; API recebe UserId/OrderId; type tests e runtime de entrada inválida.

Transferência em outro contexto

Modele ProductId e InvoiceId que não possam ser trocados acidentalmente e valide strings externas.

Evidência de domínio

Rode typecheck, testes de tipo positivos/negativos e runtime onde necessário. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; brand é estático. Node.js 24 executa construtores/validadores, não valida a marca por si. | Próxima sugestão: 20.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20 Iteradores, streams e bytes

10 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 20.01 · 20.02 · 20.03 · 20.04 · 20.05 · 20.06 · 20.07 · 20.08 · 20.09 · 20.10

20.01 Protocolo iterável

TRACE · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 1.5/5

Competência: iterators.protocol | Requer: arrays.basic | Introduz: iterators.protocol

Eixos: novidade 1 · sintaxe 2 · composição 2 · estado 2 · depuração 1 · arquivos 1 · leitura 2 · autonomia 1 · ferramentas 2

VISÃO DO ALUNO

Rastreie a sequência conceitual de chamadas next() durante for...of. Antes de executar, registre quantas chamadas ocorrem, os pares done/value de cada resultado e quais valores chegam ao corpo. Não conte o resultado terminal como elemento.

Objetivo da atividade

Preencha a tabela de chamadas, done e valores vistos pelo corpo antes de executar.

Explicação e modelo mental

• Modelo mental: O laço pede next() até receber done:true. Os resultados com done:false fornecem os elementos ao corpo; o resultado terminal encerra o laço e não é entregue como elemento, mesmo que uma implementação também inclua value nele.
• Exemplo resolvido: Uma ferramenta que expõe IteratorResult permite inspecionar o sinal de término e o campo value separadamente; o consumidor decide quais resultados contam como elementos.
• Armadilha comum: Contar o value terminal como elemento ou achar que done:true exige ausência de value.
• Por que importa: Entender o protocolo separa valores iterados de metadados ou retornos consumidos manualmente.

Material/arquivo inicial

protocolo.js: // 20.01 · Protocolo iterável passo a passo.
// Antes de executar, preencha a tabela: quantas chamadas a next(), o par {done, value} de cada uma
// e quais valores chegam ao corpo do for...of.

```
const resultados = [
{ value: 1, done: false },
{ value: 2, done: false },
{ value: "fim", done: true },
];

let chamadas = 0;
const iterador = {
next() {
const resultado = resultados[chamadas] ?? { value: undefined, done: true };
chamadas++;
console.log(next #${chamadas}, JSON.stringify(resultado));
return resultado;
},
[Symbol.iterator]() {
return this;
},
};

for (const valor of iterador) {
console.log("corpo recebeu", valor);
}
console.log("total de chamadas", chamadas);

// Previsão para um segundo for...of sobre o MESMO iterador já consumido: ___
for (const valor of iterador) {
console.log("segunda volta recebeu", valor);
}

// Tabela prevista:
// chamada | done | value | chega ao corpo?
// 1 | ___ | ___ | ___
// 2 | ___ | ___ | ___
// 3 | ___ | ___ | ___
```

Exemplo e resultado público

for...of sobre o iterator descrito no starter → O corpo recebe 1 e 2; next é chamado 3 vezes; {done:true} termina a iteração e não é contado como item.

Bordas e erros publicados

done:true encerra a iteração, mas IteratorResult ainda pode carregar o valor de retorno; for...of não expõe esse valor como elemento; segundo for...of sobre o mesmo iterator consumido -> nenhum valor

Teste público e aprovação visível

Compare a previsão de chamadas, done e valores; peça explicação curta da primeira divergência.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é a diferença geral entre um objeto iterável e o iterador obtido a partir dele?

 (Correta) O iterável oferece uma forma de obter um iterador; o iterador fornece os passos que o consumidor percorre.
• O iterador é sempre uma cópia completa dos valores, e o iterável é apenas um método de array.
• Qualquer objeto que possua propriedades enumeráveis já implementa o protocolo de iteração.

Por que a resposta está certa

O protocolo separa a fonte iterável do objeto que fornece a travessia ao consumidor.

Armadilha da alternativa incorreta

Iteração não é definida apenas por propriedades enumeráveis nem exige materializar a fonte inteira.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

done:true encerra a iteração, mas IteratorResult ainda pode carregar o valor de retorno; for...of não expõe esse valor como elemento; segundo for...of sobre o mesmo iterator consumido → nenhum valor

Concepção errada e falha observável

Assumir que value desaparece quando done é true, ou contar o valor de retorno como elemento do for...of. / Na borda done:true encerra a iteração, mas IteratorResult ainda pode carregar o valor de retorno, observe a saída e compare-a com o contrato publicado; esta interpretação incorreta causa a falha: Assumir que value desaparece quando done é true, ou contar o valor de retorno como elemento do for...of.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Assumir que value desaparece quando done é true, ou contar o valor de retorno como elemento do for...of.

Pistas graduais

1. Separe chamadas next de elementos entregues ao corpo. 2. done controla término; o resultado terminal ainda pode ter value. 3. Compare um for...of com next() manual em um iterator novo.

Solução de referência explicada

Mostre os três IteratorResult da sequência manual. O for...of usa os dois com done:false e termina ao ver done:true; ele não entrega 'fim' no corpo. Diferencie um iterable que cria iterator novo do iterator que retorna a si mesmo.

Transferência em outro contexto

Uma fonte paginada encerra com um cursor resumido; descreva como investigar o protocolo para observar esse retorno terminal.

Evidência de domínio

Compare estados e ordem; peça explicação curta sobre a primeira divergência. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Protocolo ECMAScript estável; resultado terminal pode conter value, mas for...of só itera resultados com done:false. | Próxima sugestão: 20.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.02 Parsons generator

PARSONS · 18 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: generators.depth_first | Requer: arrays.basic | Introduz: generators.depth_first

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 3

VISÃO DO ALUNO

Reconstrua walk(node) em pré-ordem a partir dos blocos. Antes de ordenar, diga em que momento o nó atual aparece em relação aos filhos; depois adapte o wrapper para entrada opcional e explique a laziness.

Objetivo da atividade

Escreva uma linha que emita o valor do nó corrente e marque se ela vem antes ou depois da recursão.

Explicação e modelo mental

• Modelo mental: Um generator produz valores sob demanda; yield* delega a sequência do generator filho ao chamador.
• Exemplo resolvido: Um percurso recursivo pode emitir um valor por vez e delegar a travessia aninhada; a posição de cada yield em relação à chamada recursiva altera a ordem observada, então confirme o contrato numa árvore pequena.
• Armadilha comum: Usar yield walk(child) e produzir o objeto generator como elemento, em vez de delegar com yield*.
• Por que importa: Percurso lazy economiza memória e permite parar cedo em estruturas maiores.

Material/arquivo inicial

walk.js: // 20.02 · Percurso em pré-ordem com generator.
// Antes de ordenar: em pré-ordem, o nó atual aparece antes ou depois dos filhos? ___
// Monte walk com os blocos e complete o wrapper para aceitar árvore ausente.

```
/* Blocos (embaralhados)
[A] for (const child of node.children) {
[B] yield* walk(child);
[C] yield node.value;
[D] function* walk(node) {
[E] }
[F] }
*/

// Cole aqui os blocos na ordem:


// Wrapper: árvore ausente (null ou undefined) não produz valores.
export function* percorrer(arvore) {
// complete
}

const arvore = {
value: 1,
children: [
{ value: 2, children: [{ value: 4, children: [] }] },
{ value: 3, children: [] },
],
};

console.log([...percorrer(arvore)]); // esperado: [1, 2, 4, 3]
console.log([...percorrer(null)]); // esperado: []

// Laziness: pegue só o primeiro valor sem percorrer o resto.
const primeiro = percorrer(arvore).next();
console.log(primeiro); // esperado: { value: 1, done: false }
```

Exemplo e resultado público

raiz 1 filhos 2,3 → [1,2,3] em pré-ordem

Bordas e erros publicados

árvore vazia via wrapper -> []; neto 4 em filho 2 -> [1,2,4,3]; não materializar toda árvore

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que separar uma função de travessia de uma função wrapper que prepara sua entrada?

• A separação faz com que todos os valores da estrutura sejam calculados antes de o consumidor pedi-los.
 (Correta) A travessia pode se concentrar em percorrer um nó, enquanto o wrapper trata como obter e preparar a entrada inicial.
• O wrapper deve decidir a ordem interna dos filhos em cada chamada recursiva, mesmo quando essa regra pertence à travessia.

Por que a resposta está certa

Separar essas responsabilidades deixa mais claro o contrato de cada função e facilita reutilizar a travessia.

Armadilha da alternativa incorreta

Um wrapper organiza a entrada da operação; não precisa materializar a estrutura nem duplicar a lógica recursiva.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

árvore vazia via wrapper → []; neto 4 em filho 2 -> [1,2,4,3]; não materializar toda árvore

Concepção errada e falha observável

Usar yield walk(child) e emitir generator como valor. / Na entrada árvore vazia via wrapper, a saída observável não corresponde a []; a divergência é compatível com esta hipótese: Usar yield walk(child) e emitir generator como valor.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar yield walk(child) e emitir generator como valor.

Pistas graduais

1. yield produz valor do nó. 2. for percorre filhos em ordem. 3. yield* delega ao generator filho.

Solução de referência explicada

Reordene yield local e recursão yield* dentro do loop; wrapper trata null conforme contrato.

Transferência em outro contexto

Percorra diretórios aninhados em pré-ordem sem criar uma lista completa antes do consumo.

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Generators e yield* são recursos ECMAScript amplamente suportados; sem dependência de Iterator helpers. | Próxima sugestão: 20.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.03 Complete Symbol.iterator

FADED · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: iterators.custom_iterable | Requer: arrays.basic | Introduz: iterators.custom_iterable

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Complete Range conforme o contrato e escreva um teste que distingue estado da coleção do estado de um cursor.

Objetivo da atividade

Escreva qual objeto guarda start/end e qual guarda o cursor, antes de completar Symbol.iterator.

Explicação e modelo mental

• Modelo mental: Iterable oferece Symbol.iterator; o iterator mantém seu próprio estado de progresso.
• Exemplo resolvido: Uma playlist guarda a sequência de faixas, enquanto uma sessão de reprodução guarda o progresso atual; separar esses papéis evita que uma sessão altere o cursor de outra.
• Armadilha comum: Guardar o índice no objeto Range e retornar esse mesmo cursor em todas as iterações.
• Por que importa: A separação entre coleção e cursor evita estado compartilhado surpreendente para consumidores.

Material/arquivo inicial

range.js: // 20.03 · Range iterável: a coleção guarda start/end; cada iteração tem o próprio cursor.

```
export class Range {
constructor(start, end) {
this.start = start;
this.end = end;
}

[Symbol.iterator]() {
let atual = ___; // o cursor vive aqui, não no Range
const end = this.end;
return {
next() {
if (___) return { value: undefined, done: true };
return { value: atual++, done: ___ };
},
};
}
}

console.log([...new Range(2, 4)]); // esperado: [2, 3, 4]
console.log([...new Range(5, 4)]); // esperado: []

// Escreva o teste que distingue estado da coleção do estado do cursor:
const r = new Range(1, 3);
const primeira = [...r];
const segunda = [...r];
console.log(primeira, segunda); // as duas devem começar do 1. Por quê? ___
```

Exemplo e resultado público

[...new Range(2,4)] → [2,3,4]

Bordas e erros publicados

range vazio end<start -> []; duas iterações separadas -> ambas começam do início; iterator state não fica no objeto Range

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual papel o protocolo iterável oferece a uma construção consumida por for...of?

• Um método de comprimento substitui o protocolo e define como cada valor é produzido.
• Um índice de propriedade faz for...of ler automaticamente todos os membros enumeráveis.
 (Correta) Um método de iteração fornece um iterador cujo next comunica os passos da travessia.

Por que a resposta está certa

for...of consome a interface iterável para obter um iterador e avançar pela sequência.

Armadilha da alternativa incorreta

O protocolo depende do iterador e de seus passos, não de um comprimento ou da enumeração de propriedades.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

range vazio end<start → []; duas iterações separadas -> ambas começam do início; iterator state não fica no objeto Range

Concepção errada e falha observável

Retornar o mesmo iterator consumido em todas as iterações. / Na entrada range vazio end<start, a saída observável não corresponde a []; a divergência é compatível com esta hipótese: Retornar o mesmo iterator consumido em todas as iterações.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Retornar o mesmo iterator consumido em todas as iterações.

Pistas graduais

1. [Symbol.iterator] cria cursor novo. 2. next retorna {done:false,value} até limite. 3. Depois retorna {done:true}.

Solução de referência explicada

Classe Range armazena limites e método cria i local; cada for...of independente.

Transferência em outro contexto

Desenhe uma coleção iterável de páginas que possa ser percorrida novamente desde o início.

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Symbol.iterator e for...of são recursos ECMAScript estáveis; não requer APIs recentes. | Próxima sugestão: 20.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.04 Proxy recursivo ruim

DEBUG · TRILHA OPCIONAL · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: proxy.reflect_receiver | Requer: arrays.basic | Introduz: proxy.reflect_receiver

Eixos: novidade 4 · sintaxe 4 · composição 3 · estado 3 · depuração 5 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Reproduza a reentrada do trap get e faça correção localizada; registre hipótese antes de editar e compare uma leitura normal.

Objetivo da atividade

Desenhe ou escreva a sequência de três passos da primeira recursão e identifique a linha que reentra.

Explicação e modelo mental

• Modelo mental: Uma leitura via proxy pode voltar ao próprio trap; Reflect.get delega ao target conservando o receiver.
• Exemplo resolvido: Quando um wrapper intercepta leituras, pedir ao próprio wrapper o mesmo campo pode reentrar indefinidamente; a delegação precisa consultar o objeto subjacente e preservar o contexto original do acesso.
• Armadilha comum: Usar receiver[key] dentro do trap get e reentrar no mesmo trap sem fim.
• Por que importa: Delegação correta mantém comportamento de getter/protótipo enquanto instrumenta acesso.

Material/arquivo inicial

proxy.js: // 20.04 · Proxy que reentra no próprio trap.
// Hipótese (antes de editar): por que receiver[key] chama o trap de novo? ___

```
const target = { x: 2 };
const proxy = new Proxy(target, {
get(target, key, receiver) {
return receiver[key]; // bug: receiver é o próprio proxy
},
});

try {
console.log(proxy.x);
} catch (erro) {
console.log("falhou:", erro.name, erro.message);
}

// Corrija o trap de forma localizada, preservando receiver (use Reflect.get).
// Depois confira dois casos:
const base = {
get dobro() {
return this.x * 2; // this precisa ser o proxy (receiver), não o target
},
};
const comGetter = Object.assign(Object.create(base), { x: 5 });
// const proxyGetter = new Proxy(comGetter, { get: ___ });
// console.log(proxyGetter.dobro); // esperado: 10
// console.log(proxy[Symbol.toStringTag]); // símbolo não deve quebrar: undefined
```

Exemplo e resultado público

proxy.x com target.x=2 → 2 sem recursão

Bordas e erros publicados

getter no protótipo que usa this -> receiver preservado por Reflect.get(target,key,receiver); acesso símbolo não quebra

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que registro ajuda a revisar uma correção localizada em um trap de Proxy?

 (Correta) A hipótese inicial, uma operação pequena que a ilustre e a comparação do comportamento observado antes e depois.
• Uma reescrita de todos os traps de uma vez, sem anotar o comportamento que motivou a mudança.
• A conclusão de que a correção funciona porque o código compila, sem descrever o cenário examinado.

Por que a resposta está certa

Uma hipótese e uma observação comparável tornam a alteração localizada e revisável.

Armadilha da alternativa incorreta

Compilar ou alterar vários traps ao mesmo tempo não explica qual comportamento precisava de correção.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

getter no protótipo que usa this → receiver preservado por Reflect.get(target,key,receiver); acesso símbolo não quebra

Concepção errada e falha observável

Ler receiver[key] dentro do próprio trap e disparar trap novamente. / Na entrada getter no protótipo que usa this, a saída observável não corresponde a receiver preservado por Reflect.get(target,key,receiver); a divergência é compatível com esta hipótese: Ler receiver[key] dentro do próprio trap e disparar trap novamente.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Ler receiver[key] dentro do próprio trap e disparar trap novamente.

Pistas graduais

1. O trap intercepta acesso ao proxy. 2. receiver[key] volta ao mesmo trap. 3. Reflect.get lê target com receiver correto.

Solução de referência explicada

return Reflect.get(target,key,receiver); mostre stack recursiva antes e leitura estável depois.

Transferência em outro contexto

Instrumente leituras de um getter herdado sem causar recursão e preserve o this esperado.

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Proxy/Reflect são suportados pelo runtime alvo; trap executa comportamento JavaScript em runtime. | Próxima sugestão: 20.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.05 Validador via Proxy

MAKE · TRILHA OPCIONAL · 28 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: proxy.validation_tradeoff | Requer: arrays.basic | Introduz: proxy.validation_tradeoff

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Implemente a regra de validação declarada no trap set e compare com uma função update explícita. Entregue um pequeno teste para cada abordagem e recomende uma com base na visibilidade da regra.

Objetivo da atividade

Escreva qual mutação a regra proposta cobre e uma que ela não cobre; depois escolha a função explícita ou o trap para o experimento.

Explicação e modelo mental

• Modelo mental: Proxy intercepta operações escolhidas; Reflect delega operação ao objeto. A cobertura depende exatamente dos traps instalados.
• Exemplo resolvido: Um trap set observa atribuições ao proxy; uma escrita em score pode ser checada ali, enquanto objeto distinto requer instrumentação própria.
• Armadilha comum: Assumir que um Proxy valida profundamente todos os dados ou preferi-lo sem necessidade.
• Por que importa: Decisões sobre metaprogramação afetam depuração, testes e previsibilidade de efeitos.

Material/arquivo inicial

validador.js: // 20.05 · Validação escondida num Proxy versus função explícita.
// Regra: age é inteiro de 0 a 150; outro valor lança RangeError.

```
const user = { age: 20, endereco: { cidade: "Recife" } };

const proxy = new Proxy(user, {
set(target, key, value, receiver) {
// TODO: valide age e delegue a escrita com Reflect.set (que devolve boolean).
},
});

function updateUser(atual, patch) {
// TODO: alternativa explícita: devolve um novo objeto validado, sem mutar atual.
}

// Teste da versão Proxy:
proxy.age = 30;
console.log(user.age); // esperado: 30
try { proxy.age = -1; } catch (erro) { console.log("proxy:", erro.name); } // esperado: RangeError

// Teste da versão explícita:
// console.log(updateUser({ age: 20 }, { age: 21 })); // esperado: { age: 21 }
// updateUser({ age: 20 }, { age: -1 }) // esperado: RangeError

// Borda: proxy.endereco.cidade = 42 passa pelo trap? ___
// Qual abordagem deixa a regra mais visível para quem lê o código que chama? ___
```

Exemplo e resultado público

proxy.age=20 → armazena; proxy.age=-1 -> RangeError

Bordas e erros publicados

atribuição aninhada sem proxy próprio -> não validada; Reflect.set retorna boolean; TS não valida runtime

Teste público e aprovação visível

Resultado observável: proxy.age=20 -> armazena; proxy.age=-1 -> RangeError. Borda a conferir: atribuição aninhada sem proxy próprio -> não validada; Reflect.set retorna boolean; TS não valida runtime. Entrega esperada: Implemente experimento e escreva decisão: função explícita facilita rastreio/teste na app comum; Proxy só com motivo claro.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao comparar validação em um trap set de Proxy com uma função update explícita, qual critério ajuda a recomendar uma abordagem?

• Preferir Proxy em todos os casos porque ele intercepta automaticamente qualquer alteração profunda.
 (Correta) Avaliar onde a regra fica mais visível e testável para quem chama e mantém a API.
• Preferir sempre a função update porque operações em Proxy não podem validar atribuições.

Por que a resposta está certa

A atividade compara as abordagens pela clareza da regra e pela evidência dos testes, não por uma preferência universal.

Armadilha da alternativa incorreta

Proxy não torna a validação automaticamente profunda, e ambos os estilos podem expressar uma regra de atualização.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

atribuição aninhada sem proxy próprio → não validada; Reflect.set retorna boolean; TS não valida runtime

Concepção errada e falha observável

Achar que Proxy cobre todas as mutações profundas e é primeira opção sempre. / Na entrada atribuição aninhada sem proxy próprio, a saída observável não corresponde a não validada; a divergência é compatível com esta hipótese: Achar que Proxy cobre todas as mutações profundas e é primeira opção sempre.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que Proxy cobre todas as mutações profundas e é primeira opção sempre.

Pistas graduais

1. Trap valida propriedade declarada. 2. Use Reflect.set para delegar. 3. Compare custo/visibilidade com função explícita de atualização.

Solução de referência explicada

Implemente experimento e escreva decisão: função explícita facilita rastreio/teste na app comum; Proxy só com motivo claro.

Transferência em outro contexto

Valide a atualização de preço de um produto e compare proxy e função explícita quanto a teste e visibilidade da regra.

Evidência de domínio

Resultado observável: proxy.age=20 -> armazena; proxy.age=-1 -> RangeError. Borda a conferir: atribuição aninhada sem proxy próprio -> não validada; Reflect.set retorna boolean; TS não valida runtime. Entrega esperada: Implemente experimento e escreva decisão: função explícita facilita rastreio/teste na app comum; Proxy só com motivo claro. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Proxy e Reflect têm suporte no Node.js 24 LTS; os traps são mecanismos runtime, não garantias do typechecker. | Próxima sugestão: 20.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.06 Iterator helpers

COMPARE · TRILHA OPCIONAL · 28 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: iterator.helpers | Requer: arrays.basic, iterators.sync | Introduz: iterator.helpers

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 5

VISÃO DO ALUNO

Compare uma pipeline generator manual com Iterator.from/filter/map/take. Preveja o valor e a quantidade de itens solicitados antes de executar; recomende API somente após verificar o runtime alvo.

Objetivo da atividade

Desenhe a ordem dos operadores e indique qual primeiro ponto poderia limitar a leitura.

Explicação e modelo mental

• Modelo mental: Iterator helpers encadeiam transformações lazy; arrays com map/filter fazem trabalho eager e Iterator.from cria wrapper sobre a fonte.
• Exemplo resolvido: Um take após filtros para de pedir itens quando atingiu seu limite, ao contrário de materializar a fonte toda antes.
• Armadilha comum: Chamar Array.from no começo e considerar lazy uma pipeline que já guardou a coleção.
• Por que importa: Laziness importa para fontes grandes/infinitas e suporte depende do runtime, não apenas do nome da edição ECMAScript.

Material/arquivo inicial

pipelines.js:

```
// 20.06 · Pipeline manual com generators versus iterator helpers.
// Preveja o resultado e quantos itens cada versão pede à fonte antes de executar.

function criarFonte() {
const contador = { pulls: 0 };
function* source() {
for (const value of [1, 2, 3, 4, 5]) {
contador.pulls++;
yield value;
}
}
return { contador, source };
}

// Versão manual: filter par, map dobro, take 2, sem materializar arrays.
function* manualPipeline(iteravel) {
// TODO
}

const manual = criarFonte();
console.log("manual", [...manualPipeline(manual.source())], "pulls", manual.contador.pulls);

// Versão com helpers (confira se o runtime alvo tem Iterator.from antes de recomendar).
if (typeof Iterator !== "undefined" && typeof Iterator.from === "function") {
const helpers = criarFonte();
const resultado = Iterator.from(helpers.source()).filter((n) => n % 2 === 0).map((n) => n * 2).take(2);
console.log("helpers", [...resultado], "pulls", helpers.contador.pulls);
} else {
console.log("Iterator helpers indisponíveis neste runtime: use a versão manual.");
}

// Compare com arrays (eager):
const ansioso = criarFonte();
console.log("array", [...ansioso.source()].filter((n) => n % 2 === 0).map((n) => n * 2).slice(0, 2), "pulls", ansioso.contador.pulls);
// Previsões: manual ___ pulls, helpers ___ pulls, array ___ pulls. E com take(0)? ___
```

Exemplo e resultado público

fonte 1..5 → [4,8]; só itens necessários consumidos conforme pipeline

Bordas e erros publicados

runtime sem helpers -> fallback generator manual; arrays map/filter são eager; se take(0), nenhuma leitura da fonte

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao comparar duas formas de montar uma pipeline de iteração, qual critério deve orientar a recomendação final?

• Se a sintaxe mais nova pode ser usada, independentemente da clareza ou do contexto do projeto.
• Se uma das formas tem mais operadores encadeados, pois mais etapas indicam uma pipeline mais eficiente.
 (Correta) Se a forma escolhida comunica claramente a intenção da transformação e se ajusta às necessidades de manutenção do projeto.

Por que a resposta está certa

A recomendação deve considerar legibilidade e adequação ao projeto, além do comportamento observado.

Armadilha da alternativa incorreta

Novidade ou quantidade de operadores, por si só, não estabelece clareza nem adequação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

runtime sem helpers → fallback generator manual; arrays map/filter são eager; se take(0), nenhuma leitura da fonte

Concepção errada e falha observável

Usar Array.from no início e ainda chamar pipeline lazy. / Na entrada runtime sem helpers, a saída observável não corresponde a fallback generator manual; a divergência é compatível com esta hipótese: Usar Array.from no início e ainda chamar pipeline lazy.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar Array.from no início e ainda chamar pipeline lazy.

Pistas graduais

1. Implemente primeiro generator com yield sob demanda. 2. Compare com Iterator.from(source).filter().map().take(). 3. Feature detect suporte no baseline antes de usar.

Solução de referência explicada

Duas versões produzem mesmo resultado e registram contagem de pulls; decisão de compatibilidade usa documentação do runtime alvo. Iterator helpers são ES2025 e estão no baseline Node.js 24; Iterator.concat é ES2026 e não deve ser presumido nesse runtime, então deixe fora da solução ou feature-detect/fallback.

Transferência em outro contexto

Projete uma pipeline de linhas grandes com limite de consumo e decida se helpers estão disponíveis no ambiente alvo.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Iterator helpers (Iterator.from, .map, .filter, .take, .toArray; Node.js 22+) funcionam no Node.js 24. Iterator.concat (ES2026) não está no Node.js 24 segundo a matriz de recursos: omita ou use feature detection e fallback. | Próxima sugestão: 20.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.07 Array.fromAsync

COMPARE · TRILHA OPCIONAL · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: async.array_from_async | Requer: arrays.basic, async.iterables | Introduz: async.array_from_async

Eixos: novidade 5 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 5

VISÃO DO ALUNO

Compare Array.fromAsync com for await...of sobre a mesma fonte assíncrona determinística. Registre quando o primeiro item fica disponível e qual versão guarda todos os itens.

Objetivo da atividade

Preveja o tipo de retorno de Array.fromAsync e a ordem dos dois eventos antes de iniciar consumo.

Explicação e modelo mental

• Modelo mental: Array.fromAsync consome toda a fonte e resolve uma Promise com array; for await permite processar cada item durante a iteração.
• Exemplo resolvido: Um leitor assíncrono suspende enquanto espera a próxima parte; o momento do primeiro processamento depende da chegada do dado, e reter resultados altera a memória necessária.
• Armadilha comum: Supor que Array.fromAsync retorna array síncrono ou mantém memória constante apesar de materializar tudo.
• Por que importa: A escolha depende se o consumidor precisa da coleção inteira ou pode agir incrementalmente; API recente exige suporte de runtime.

Material/arquivo inicial

colecao-async.js: // 20.07 · Array.fromAsync versus for await...of.
// Registre quando o primeiro item fica disponível e qual versão guarda todos os itens.

```
const eventos = [];

async function* source() {
eventos.push("produziu 1");
yield 1;
await Promise.resolve();
eventos.push("produziu 2");
yield 2;
}

async function collectAll() {
const todos = await Array.fromAsync(source());
eventos.push(fromAsync entregou ${JSON.stringify(todos)});
return todos;
}

async function processIncrementally() {
for await (const value of source()) {
eventos.push(for await processou ${value});
}
}

if (typeof Array.fromAsync !== "function") {
eventos.push("Array.fromAsync indisponível: colete com for await e push");
} else {
await collectAll();
}
await processIncrementally();
console.log(eventos.join("\n"));

// Previsões: ordem dos eventos ___; qual versão retém todos os itens na memória? ___
// Se a fonte rejeitar no segundo item, o que cada versão entrega? ___
```

Exemplo e resultado público

Array.fromAsync(source) → Promise<[1,2]> após fim; for await recebe 1 antes de 2

Bordas e erros publicados

runtime sem suporte -> coleta em loop; fonte rejeita -> Promise rejeitada; memória O(n) ao materializar

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que uma comparação entre abordagens de consumo assíncrono deve usar uma fonte determinística?

 (Correta) Para que diferenças observadas possam ser atribuídas às abordagens comparadas, e não a mudanças imprevisíveis na fonte.
• Para que a fonte assíncrona seja convertida em síncrona antes da comparação.
• Para garantir que qualquer abordagem produza o mesmo tipo de interface de consumo.

Por que a resposta está certa

Manter a fonte previsível ajuda a interpretar a comparação entre as estratégias de consumo.

Armadilha da alternativa incorreta

Determinismo torna a observação comparável; não torna a fonte síncrona nem iguala as interfaces.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

runtime sem suporte → coleta em loop; fonte rejeita -> Promise rejeitada; memória O(n) ao materializar

Concepção errada e falha observável

Achar que Array.fromAsync retorna array síncrono ou reduz uso de memória. / Na entrada runtime sem suporte, a saída observável não corresponde a coleta em loop; a divergência é compatível com esta hipótese: Achar que Array.fromAsync retorna array síncrono ou reduz uso de memória.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que Array.fromAsync retorna array síncrono ou reduz uso de memória.

Pistas graduais

1. API retorna Promise. 2. for await processa incrementalmente. 3. Materializar implica guardar todos itens.

Solução de referência explicada

Compare dois consumidores, registre tempo do primeiro item e total; fallback loop em runtime incompatível. Array.fromAsync pertence ao ECMAScript 2026, embora Node.js 22.0+ já o implemente; o alvo Node.js 24 LTS oferece suporte, runtimes anteriores usam o loop async explícito.

Transferência em outro contexto

Compare acumular todos os chunks assíncronos de arquivo com processar cada chunk ao chegar.

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Array.fromAsync é implementado em Node.js 22.0+ e disponível no alvo Node.js 24 LTS. Não confundir edição ECMAScript com suporte do runtime; runtimes anteriores exigem fallback explícito. | Próxima sugestão: 20.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.08 Microprojeto - pipeline lazy

PROJECT · 85 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.5/5

Competência: iterators.lazy_pipeline | Requer: arrays.basic | Introduz: iterators.lazy_pipeline

Eixos: novidade 4 · sintaxe 5 · composição 5 · estado 5 · depuração 5 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Projete uma pipeline lazy de map/filter/take para fonte potencialmente infinita. Registre uma hipótese e um contador de pulls antes de implementar; escreva o primeiro teste autoral, implemente os generators e revise encerramento antecipado.

Objetivo da atividade

Escreva o teste autoral com contador de pulls e comportamento esperado antes da implementação.

Explicação e modelo mental

• Modelo mental: Cada generator pede ao upstream somente quando o consumidor avança; take limita a demanda e o fechamento deve propagar return.
• Exemplo resolvido: Em fonte [10,20,30,40], map por metade, filter >10 e take(1) produz [15] sem precisar construir uma coleção intermediária.
• Armadilha comum: Materializar a fonte antes de aplicar take ou fazer um pull além do último item aceito.
• Por que importa: Composição lazy limita memória e trabalho e permite cancelar fontes longas de modo previsível.

Material/arquivo inicial

testes.js: // 20.08 · Testes da pipeline lazy. Escreva primeiro sua hipótese e o teste; depois implemente pipeline.js.

```
import { pipeline } from "./pipeline.js";

function verificar(nome, obtido, esperado) {
const ok = JSON.stringify(obtido) === JSON.stringify(esperado);
console.log(${ok ? "ok " : "FALHOU"} ${nome}: ${JSON.stringify(obtido)}${ok ? "" : (esperado ${JSON.stringify(esperado)})});
}

// Coleta no máximo limite valores: protege os testes de um take ainda não implementado sobre fonte infinita.
function coletar(iteravel, limite = 50) {
const saida = [];
for (const valor of iteravel) {
saida.push(valor);
if (saida.length >= limite) break;
}
return saida;
}

function contarPulls(iteravel) {
const contador = { pulls: 0 };
function* instrumentada() {
for (const valor of iteravel) {
contador.pulls++;
yield valor;
}
}
return { contador, fonte: instrumentada() };
}

function* naturais() {
for (let n = 1; ; n++) yield n;
}

// Hipótese: quantos pulls para o primeiro teste? ___
const a = contarPulls(naturais());
verificar("fonte infinita", coletar(pipeline(a.fonte).map((x) => x * 2).filter((x) => x > 3).take(2)), [4, 6]);
verificar("pulls do primeiro teste", a.contador.pulls, 3);

const b = contarPulls(naturais());
verificar("take(0)", coletar(pipeline(b.fonte).take(0)), []);
verificar("take(0) não lê a fonte", b.contador.pulls, 0);

function* falhaNoSexto() {
for (let n = 1; n <= 5; n++) yield n;
throw new Error("fonte falhou");
}
let antesDaFalha;
try {
antesDaFalha = coletar(pipeline(falhaNoSexto()).take(2));
} catch (erro) {
antesDaFalha = lançou: ${erro.message};
}
verificar("termina antes da falha", antesDaFalha, [1, 2]);

// Seu teste autoral (por exemplo: duas iterações da mesma pipeline criam cursores independentes?):
| pipeline.js: // pipeline(source).map(f).filter(p).take(n): cada etapa é um generator; nada roda até alguém pedir valores.
```

```
// Os métodos abaixo ainda devolvem a fonte sem transformar: os testes mostram FALHOU até você implementá-los.

export function pipeline(iteravel) {
return {
map(fn) {
// devolva pipeline(...) de um generator que aplica fn a cada valor
return pipeline(iteravel);
},
filter(predicado) {
// devolva pipeline(...) de um generator que só repassa valores aprovados
return pipeline(iteravel);
},
take(n) {
// devolva pipeline(...) de um generator que para depois de n valores
// (take(0) não deve pedir nenhum valor à fonte)
return pipeline(iteravel);
},
[Symbol.iterator]() {
return iteravel[Symbol.iterator]();
},
};
}
```

Exemplo e resultado público

source infinito 1,2,3...; map x2, filter >3, take 2 → [4,6]

Bordas e erros publicados

take(0) -> zero pulls; fonte lança após quinto item mas take(2) termina antes; cada iteração cria cursor independente

Teste público e aprovação visível

Resultado observável: source infinito 1,2,3...; map x2, filter >3, take 2 -> [4,6]. Borda a conferir: take(0) -> zero pulls; fonte lança após quinto item mas take(2) termina antes; cada iteração cria cursor independente. Entrega esperada: Componha generators para map/filter/take, preserve lazy evaluation e tests de número de pulls/cleanup.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Para fonte [2,4,6], map(x=>x+1), filter(x=>x>4), take(1), qual saída e quantos itens da fonte são solicitados?

- [5,7] e dois itens solicitados.
- [3,5,7] e três itens solicitados.
- [] e zero itens solicitados.
- □ (Correta) [5] e dois itens solicitados.

Por que a resposta está certa

2 é filtrado; o próximo item vira 5 e passa. take(1) encerra antes de pedir o terceiro valor.

Armadilha da alternativa incorreta

filter pode solicitar mais de um item antes de obter o primeiro aceito; take encerra depois da primeira saída.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

take(0) → zero pulls; fonte lança após quinto item mas take(2) termina antes; cada iteração cria cursor independente

Concepção errada e falha observável

Materializar fonte inteira antes de take e travar em fonte infinita. / Na entrada take(0), a saída observável não corresponde a zero pulls; a divergência é compatível com esta hipótese: Materializar fonte inteira antes de take e travar em fonte infinita.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Materializar fonte inteira antes de take e travar em fonte infinita.

Pistas graduais

1. Cada operador é iterable/generator. 2. yield* ou loops puxam quando next é chamado. 3. take encerra cedo e fecha iterator se aplicável.

Solução de referência explicada

Componha generators para map/filter/take, preserve lazy evaluation e tests de número de pulls/cleanup.

Transferência em outro contexto

Processe até 10 registros válidos de um fluxo longo; teste os pulls, prove que não há consumo após o décimo resultado e observe o fechamento/cancelamento do produtor.

Evidência de domínio

Resultado observável: source infinito 1,2,3...; map x2, filter >3, take 2 -> [4,6]. Borda a conferir: take(0) -> zero pulls; fonte lança após quinto item mas take(2) termina antes; cada iteração cria cursor independente. Entrega esperada: Componha generators para map/filter/take, preserve lazy evaluation e tests de número de pulls/cleanup. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Generators e protocolo síncrono estáveis; implementação manual não usa Iterator helpers nem Iterator.concat. | Próxima sugestão: 20.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.09 Stream sob consumidor lento

INVESTIGATE · TRILHA OPCIONAL · 30 min (estimativa, sem nota por velocidade) · Node.js 24 LTS e navegadores evergreen · dificuldade multidimensional 4.0/5

Competência: streams.backpressure | Requer: arrays.basic, async.iterables | Introduz: streams.backpressure

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 4 · depuração 4 · arquivos 3 · leitura 5 · autonomia 4 · ferramentas 5

VISÃO DO ALUNO

Investigue a relação entre produtor e consumidor sob uma strategy de fila declarada. Colete sequência de eventos e explique como a demanda influencia o produtor sem inferir valores universais.

Objetivo da atividade

Anote uma hipótese: qual evento deverá ocorrer primeiro quando consumidor lê devagar e como a strategy escolhida afeta a fila?

Explicação e modelo mental

• Modelo mental: ReadableStream coordena enqueue/read com desiredSize relativo à strategy e pode aplicar backpressure; highWaterMark define o limite configurado.
• Exemplo resolvido: Com strategy por contagem e highWaterMark 2, duas unidades enfileiradas deixam desiredSize em zero; outra strategy pode medir de modo diferente.
• Armadilha comum: Tratar desiredSize como tamanho universal fixo, ou assumir que a fonte é sempre totalmente materializada.
• Por que importa: Observar demanda e cancelamento ajuda a desenhar produtores de dados que não desperdiçam memória/trabalho.

Material/arquivo inicial

stream.js: // 20.09 · ReadableStream sob um consumidor lento.
// Hipótese antes de executar: quantas vezes pull é chamado antes da primeira leitura, com highWaterMark 2? ___

```
const events = [];
let produzidos = 0;
const stream = new ReadableStream(
{
pull(controller) {
events.push(pull (desiredSize=${controller.desiredSize}));
controller.enqueue(++produzidos);
},
cancel(reason) {
events.push(cancel: ${reason});
},
},
{ highWaterMark: 2 },
);

const esperar = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const reader = stream.getReader();

await esperar(50);
events.push("consumidor começou");
for (let i = 0; i < 3; i++) {
const { value } = await reader.read();
events.push(leu ${value});
await esperar(50); // consumidor lento
}
await reader.cancel("consumidor terminou");

console.log(events.join("\n"));
// Explique o papel da strategy (highWaterMark) e por que desiredSize muda. Não assuma tempos exatos do scheduler.
```

Exemplo e resultado público

consumidor pausa → fila cresce até highWaterMark e pull é regulado

Bordas e erros publicados

cancelamento -> source.cancel chamado; desiredSize pode ser <=0; log de pulls sem assumir tempo exato do scheduler

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que determina o significado/limite de desiredSize em um ReadableStream?

 (Correta) A strategy e o highWaterMark configurados para o stream.
• Só o highWaterMark; a strategy afeta apenas a ordem dos chunks.
• Sempre bytes pendentes, mesmo com uma strategy por contagem.
• Só a quantidade total já lida; a fila não participa.

Por que a resposta está certa

desiredSize expressa espaço relativo à strategy configurada, não um limite universal independente.

Armadilha da alternativa incorreta

O valor observado depende da estratégia e da fila; o scheduler/ambiente também afeta a sequência de eventos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

cancelamento → source.cancel chamado; desiredSize pode ser <=0; log de pulls sem assumir tempo exato do scheduler

Concepção errada e falha observável

Achar que todo stream materializa tudo imediatamente ou que desiredSize é contagem fixa de bytes. / Na entrada cancelamento, a saída observável não corresponde a source.cancel chamado; a divergência é compatível com esta hipótese: Achar que todo stream materializa tudo imediatamente ou que desiredSize é contagem fixa de bytes.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que todo stream materializa tudo imediatamente ou que desiredSize é contagem fixa de bytes.

Pistas graduais

1. Instrumente pull e leitura. 2. Atrase consumer e observe desiredSize. 3. Teste cancel e limpeza do produtor.

Solução de referência explicada

Relatório com eventos de pull/read/desiredSize e explicação de backpressure sob strategy declarada; sem prometer números universais.

Transferência em outro contexto

Investigue como um consumidor lento de imagens afeta a produção de chunks em um ReadableStream.

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: WHATWG ReadableStream está disponível no Node.js 24 LTS e navegadores modernos; a strategy declarada determina highWaterMark/desiredSize, sem valor universal. | Próxima sugestão: 20.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

20.10 String vira bytes

MODIFY · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: bytes.utf8 | Requer: arrays.basic | Introduz: bytes.utf8

Eixos: novidade 3 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Codifique o texto Unicode para bytes UTF-8, conte bytes separadamente do comprimento da string e faça round-trip. Explique qual opção do decoder torna a conversão estrita ou tolerante, mantendo separado o round-trip válido.

Objetivo da atividade

Registre a previsão de string.length e bytes.length para uma letra acentuada escolhida, antes de codificar.

Explicação e modelo mental

• Modelo mental: String JavaScript é sequência de unidades UTF-16; TextEncoder produz bytes UTF-8 e TextDecoder os converte de volta.
• Exemplo resolvido: O comprimento de uma string JavaScript conta unidades UTF-16, enquanto TextEncoder mede bytes UTF-8; use texto não ASCII e calcule cada representação separadamente.
• Armadilha comum: Usar string.length como tamanho em bytes ou cortar bytes no meio de uma sequência multibyte.
• Por que importa: Protocolos e arquivos contam bytes; texto Unicode requer codificação consistente nas fronteiras.

Material/arquivo inicial

bytes.js: // 20.10 · String vira bytes UTF-8.
// Previsão antes de executar: text.length = ___ ; bytes.length = ___

```
const text = "ação □";
const encoder = new TextEncoder();
const bytes = encoder.encode(text);
const decoded = new TextDecoder().decode(bytes);
console.log({ textLength: text.length, byteLength: bytes.length, decoded, igual: decoded === text });

// Entrada UTF-8 inválida: 0xC3 começa um caractere de 2 bytes, mas 0x28 não é continuação válida.
const invalidos = new Uint8Array([0x61, 0xc3, 0x28]);

// Tolerante (padrão): o que aparece no lugar do byte inválido? ___
console.log(new TextDecoder().decode(invalidos));

// Estrito: complete com a opção que transforma a falha em erro.
try {
console.log(new TextDecoder("utf-8", { ___: true }).decode(invalidos));
} catch (erro) {
console.log("estrito:", erro.name);
}
```

Exemplo e resultado público

decode(encode('ação □')) → 'ação □'

Bordas e erros publicados

bytes.length > text.length para acentos/emoji; sequência UTF-8 inválida -> replacement char por padrão ou erro com fatal:true

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Quantos bytes UTF-8 TextEncoder produz para a string 'ñ'?

• Um byte, pois a string tem length 1.
□ (Correta) Dois bytes.
• Três bytes, pois ñ ocupa três bytes em UTF-8.
• Quatro bytes, contando dois bytes por unidade UTF-16.

Por que a resposta está certa

ñ é um ponto de código que requer dois bytes em UTF-8.

Armadilha da alternativa incorreta

String.length conta unidades UTF-16; TextEncoder conta bytes UTF-8. Compare as medidas para a letra escolhida.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

bytes.length > text.length para acentos/emoji; sequência UTF-8 inválida → replacement char por padrão ou erro com fatal:true

Concepção errada e falha observável

Tratar string.length como tamanho em bytes ou cortar emoji no meio. / Na borda bytes.length > text.length para acentos/emoji, observe a saída e compare-a com o contrato publicado; esta interpretação incorreta causa a falha: Tratar string.length como tamanho em bytes ou cortar emoji no meio.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar string.length como tamanho em bytes ou cortar emoji no meio.

Pistas graduais

1. String JS usa unidades UTF-16. 2. TextEncoder emite bytes UTF-8. 3. TextDecoder recompõe com política de erro explícita.

Solução de referência explicada

Encode texto para Uint8Array, registre bytes e decode roundtrip; diferencie comprimento de string e de bytes.

Transferência em outro contexto

Codifique café para UTF-8 e informe bytes.length separadamente de string.length; confirme o round-trip com TextDecoder.

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TextEncoder/TextDecoder são disponíveis no Node.js 24 LTS e navegadores modernos; comportamento de entrada inválida é controlado por fatal do decoder. | Próxima sugestão: 21.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21 Performance, workers e recursos

11 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 21.01 · 21.02 · 21.03 · 21.04 · 21.05 · 21.06 · 21.07 · 21.08 · 21.09 · 21.10 · 21.11

21.01 Qual é o gargalo?

PREDICT · 6 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: performance.bottleneck_class | Requer: functions.return | Introduz: performance.bottleneck_class

Eixos: novidade 3 · sintaxe 1 · composição 3 · estado 2 · depuração 2 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Para seis cenários, decidir se o problema provável é CPU, I/O, rede, DOM ou memória antes de propor solução.

Objetivo da atividade

Para cada cenário, anote o gargalo provável e a evidência mensurável que ajudaria a confirmar a hipótese antes de sugerir uma solução.

Explicação e modelo mental

• Modelo mental: Lentidão pode vir de CPU, I/O, rede, DOM ou memória. Classifique a hipótese pelo trabalho observado e confirme-a com medição; fontes podem se combinar.
• Exemplo resolvido: Um loop pesado ou um JSON grande para analisar sugere CPU; fetch lento sugere rede; heap crescente sugere memória.
• Armadilha comum: Prescrever cache ou worker para toda lentidão sem identificar nem medir o gargalo.
• Por que importa: Uma intervenção proporcional depende de saber onde o tempo ou os recursos estão sendo gastos.

Material/arquivo inicial

cenarios.md: # Qual é o gargalo?

Para cada cenário, escreva o gargalo provável (CPU, I/O, rede, DOM ou memória) e a medição que confirmaria a hipótese antes de propor uma solução.

1. Relatório soma 50 milhões de números num for e a aba trava por 3 s.
``js let total = 0; for (let i = 0; i < 5e7; i++) total += i % 7; 2. Tela de pedidos espera 2,4 s por await fetch("/api/pedidos"); o servidor responde em 80 ms nos logs. 3. Lista de 500 cards ajusta a altura de cada um num laço que lê e escreve layout: js for (const card of cards) card.style.height = card.offsetWidth / 2 + "px"; 4. Depois de 1 hora aberta, a aba passa de 80 MB para 900 MB; cada navegação acrescenta listeners que nunca são removidos. 5. Script de importação lê um CSV de 2 GB do disco com readFile antes de processar a primeira linha. 6. Dashboard faz JSON.parse` de uma resposta de 40 MB e a interface congela logo depois que a rede termina.

Nº	Gargalo provável	Evidência/medição (ferramenta)	Solução só depois da medição
1			
2			
3			
4			
5			
6			

Algum cenário combina mais de uma fonte? Qual?

Exemplo e resultado público

loop/JSON parse → CPU; fetch -> rede; layout thrash -> DOM; heap crescente -> memória

Bordas e erros publicados

arquivo local -> I/O; gargalo pode combinar fontes e hipótese exige medição; otimização antes da medição não comprovada

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Uma operação fetch está lenta e ainda não há evidência de trabalho pesado no DOM. Qual é a hipótese inicial mais alinhada ao contrato?

• CPU; transferir a chamada fetch para um worker sem investigar a espera.
 (Correta) Rede; medir a duração da requisição antes de escolher uma otimização.
• DOM; alterar o layout antes de verificar o tempo da requisição.

Por que a resposta está certa

Correto. Fetch lento aponta inicialmente para rede; a medição confirma se essa hipótese explica o atraso.

Armadilha da alternativa incorreta

Classifique a fonte provável do atraso e procure uma observação mensurável antes de prescrever uma solução.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

arquivo local → I/O; gargalo pode combinar fontes e hipótese exige medição; otimização antes da medição não comprovada

Concepção errada e falha observável

Prescrever cache ou worker para toda lentidão sem diagnóstico. / Na entrada arquivo local, a saída observável não corresponde a I/O; a divergência é compatível com esta hipótese: Prescrever cache ou worker para toda lentidão sem diagnóstico.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Prescrever cache ou worker para toda lentidão sem diagnóstico.

Pistas graduais

1. Separe onde tempo é gasto. 2. Escolha métrica/ferramenta para a hipótese. 3. Só então proponha intervenção proporcional.

Solução de referência explicada

Matriz classifica seis cenários, indica observação mensurável e uma hipótese alternativa por caso.

Transferência em outro contexto

—

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: performance.now() disponível no navegador e no Node.js 24 LTS (perf_hooks); a classificação de gargalos não usa API. | Próxima sugestão: 21.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.02 Parsons de cleanup

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: resources.cleanup | Requer: functions.return | Introduz: resources.cleanup

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 3 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Reordenar setup/cleanup de timer e listener.

Objetivo da atividade

Liste os recursos criados no setup e o cleanup correspondente, incluindo a referência exata do listener.

Explicação e modelo mental

• Modelo mental: Cada montagem que instala um timer ou listener deve poder desfazê-lo. removeEventListener precisa da mesma referência de callback usada na instalação; cleanup repetido deve ser seguro.
• Exemplo resolvido: Uma montagem instala um timer e um listener; o unmount limpa ambos. Repetir mount/unmount não deve deixar duplicatas ativas.
• Armadilha comum: Criar uma nova função no removeEventListener: ela não é a mesma referência do listener instalado.
• Por que importa: Limpar exatamente os recursos criados evita listeners e timers que continuam ativos após o componente sair.

Material/arquivo inicial

cleanup.js:

```
// 21.02 · Setup e cleanup de timer e listener.
// alvo faz o papel de um elemento da página (EventTarget também existe fora do DOM).
// Reordene os blocos em montar() e desmontar(). Um bloco cria uma função nova e quebra a remoção: deixe-o de fora.

const alvo = new EventTarget();
const estado = { timer: null, ativos: 0 };

function aoClicar() {
console.log("clique tratado");
}

/* Blocos (embaralhados)
[A] alvo.removeEventListener("click", aoClicar);
[B] estado.timer = setInterval(() => {}, 1000);
[C] clearInterval(estado.timer); estado.timer = null;
[D] alvo.addEventListener("click", aoClicar);
[E] alvo.removeEventListener("click", () => aoClicar());
[F] if (estado.timer !== null) return; // montar duas vezes não duplica
[G] if (estado.timer === null) return; // desmontar duas vezes é seguro
*/

function montar() {
// blocos de setup
}

function desmontar() {
// blocos de cleanup
}

montar();
montar();
alvo.dispatchEvent(new Event("click")); // esperado: um único "clique tratado"
desmontar();
desmontar();
alvo.dispatchEvent(new Event("click")); // esperado: nada
console.log("timer ativo?", estado.timer !== null); // esperado: false
```

Exemplo e resultado público

mount uma vez → um timer/listener; unmount -> nenhum timer/listener ativo

Bordas e erros publicados

mount/unmount repetidos -> sem duplicação; cleanup chamado duas vezes -> seguro; callback reference é a mesma

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é a finalidade de parear setup e cleanup em um componente ou recurso?

• Executar novamente o setup sempre que o cleanup for chamado.
• Limpar todo o estado da aplicação, mesmo quando ele não foi criado por esse recurso.
 (Correta) Delimitar efeitos criados pelo setup e oferecer uma rotina responsável por desfazê-los quando deixam de ser necessários.

Por que a resposta está certa

O cleanup deve corresponder às responsabilidades do setup e ao ciclo de vida do recurso.

Armadilha da alternativa incorreta

Cleanup não é uma segunda montagem nem deve apagar estado alheio ao recurso que preparou.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

mount/unmount repetidos → sem duplicação; cleanup chamado duas vezes -> seguro; callback reference é a mesma

Concepção errada e falha observável

Criar callback novo no removeEventListener e deixar listener antigo vivo. / Na entrada mount/unmount repetidos, a saída observável não corresponde a sem duplicação; a divergência é compatível com esta hipótese: Criar callback novo no removeEventListener e deixar listener antigo vivo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Criar callback novo no removeEventListener e deixar listener antigo vivo.

Pistas graduais

1. Guarde referências de listener e timer. 2. Instale no setup. 3. Desfaça exatamente os recursos criados, com cleanup idempotente.

Solução de referência explicada

Reordene setup/cleanup e prove contagem após dois ciclos de montagem.

Transferência em outro contexto

—

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: setTimeout, setInterval, clearInterval, addEventListener e removeEventListener; Baseline no navegador e timers globais do Node.js 24 LTS. | Próxima sugestão: 21.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.03 Complete benchmark

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: performance.benchmark_median | Requer: functions.return | Introduz: performance.benchmark_median

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 3 · arquivos 2 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Preencher benchmark simples com warmup e múltiplas execuções, registrando mediana.

Objetivo da atividade

Separe aquecimentos de amostras medidas e anote a mediana prevista para [1, 9, 2].

Explicação e modelo mental

• Modelo mental: Aquecimentos não entram nas amostras reportadas. A mediana ordena as amostras medidas e seleciona o centro; com quantidade par, usa a média dos dois valores centrais.
• Exemplo resolvido: Ordenando [1, 9, 2], obtemos [1, 2, 9], então a mediana é 2. Os warmups ficam fora desse cálculo.
• Armadilha comum: Incluir aquecimentos na mediana ou tratar uma única execução ou a média instável como evidência definitiva.
• Por que importa: Separar aquecimento e medida, e informar o método, torna a comparação menos dependente de uma única observação.

Material/arquivo inicial

benchmark.js: // 21.03 · Benchmark com aquecimento, várias amostras e mediana.

```
export function mediana(amostras) {
const ordenadas = amostras.toSorted((a, b) => a - b);
const meio = Math.floor(ordenadas.length / 2);
// complete: quantidade par -> média dos dois centrais
return ___;
}

export function benchmark(fn, { warmups = 5, runs = 9 } = {}) {
for (let i = 0; i < warmups; i++) fn(); // aquecimento: não entra nas amostras
const amostras = [];
for (let i = 0; i < runs; i++) {
const inicio = performance.now();
___; // execute a função medida
amostras.push(___);
}
return { amostras, mediana: mediana(amostras) };
}

console.log(mediana([1, 9, 2])); // esperado: 2
console.log(mediana([1, 9, 2, 4])); // esperado: 3

const resultado = benchmark(() => {
let total = 0;
for (let i = 0; i < 1e5; i++) total += Math.sqrt(i);
return total;
});
console.log(resultado.mediana.toFixed(3), "ms de mediana em", resultado.amostras.length, "amostras");
// Se fn lançar um erro, o que benchmark deve fazer? ___
```

Exemplo e resultado público

amostras [1,9,2] → mediana 2; warmups não entram na mediana

Bordas e erros publicados

runs par -> mediana média dos dois centrais; fn falha -> erro propagado com cleanup; comparar mesma entrada/ambiente

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Depois dos warmups, as amostras medidas são [1, 9, 2]. Qual mediana deve ser registrada?

 (Correta) 2; os warmups não entram e o valor central da lista ordenada é 2.
• 4; deve-se usar a média aritmética das três amostras.
• 9; a amostra mais alta representa melhor o desempenho após o aquecimento.

Por que a resposta está certa

Correto. A lista ordenada é [1, 2, 9], e seu valor central é 2.

Armadilha da alternativa incorreta

Ordene somente as amostras medidas. Para três valores, a mediana é o valor central, não a média nem o maior.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

runs par → mediana média dos dois centrais; fn falha -> erro propagado com cleanup; comparar mesma entrada/ambiente

Concepção errada e falha observável

Usar uma execução ou média instável como prova definitiva. / Na entrada runs par, a saída observável não corresponde a mediana média dos dois centrais; a divergência é compatível com esta hipótese: Usar uma execução ou média instável como prova definitiva.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar uma execução ou média instável como prova definitiva.

Pistas graduais

1. Aqueça sem registrar. 2. Meça várias execuções com clock monotônico. 3. Ordene cópia dos tempos e calcule centro.

Solução de referência explicada

Implemente mediana para n ímpar/par e registre método, entrada, ambiente e dispersão, sem prometer benchmark científico.

Transferência em outro contexto

—

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: performance.now() e toSorted (ES2023, Node.js 20+), que não altera o array original; Node.js 24 LTS e navegadores evergreen. | Próxima sugestão: 21.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.04 Cache sem limite

DEBUG · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: performance.bounded_cache | Requer: functions.return | Introduz: performance.bounded_cache

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 4 · depuração 5 · arquivos 3 · leitura 4 · autonomia 5 · ferramentas 3

VISÃO DO ALUNO

Depurar memoization que cresce para sempre; introduzir política de limite/expiração.

Objetivo da atividade

Reproduza o crescimento do cache, escolha a política a investigar e registre a ordem de uso das chaves.

Explicação e modelo mental

• Modelo mental: Um cache limitado precisa de uma política explícita, como LRU ou TTL, e não pode ultrapassar o máximo declarado. Em LRU, um hit atualiza a recência.
• Exemplo resolvido: Com capacidade 2, inserir A e B, ler A e inserir C torna B a menos recente; B é removida.
• Armadilha comum: Deixar o Map crescer sem limite, tratar valor 0 como miss ou remover uma chave sem respeitar a política escolhida.
• Por que importa: Cache pode reduzir trabalho, mas ocupa memória e precisa manter comportamento correto nos limites.

Material/arquivo inicial

memo.js: // 21.04 · Memoize que cresce para sempre.
// Reproduza o crescimento, depois introduza uma política LRU com limite (padrão 2).

```
function memoize(fn) {
const cache = new Map();
const memo = (chave) => {
if (cache.has(chave)) return cache.get(chave);
const valor = fn(chave);
cache.set(chave, valor);
return valor;
};
memo.cache = cache;
return memo;
}

let calculos = 0;
const quadrado = memoize((n) => { calculos++; return n * n; });

for (let i = 0; i < 1000; i++) quadrado(i);
console.log("entradas no cache:", quadrado.cache.size); // reprodução: cresce sem limite

// Tarefa: memoizeLRU(fn, limite = 2)
// - Acesso move a chave para "mais recente" (no Map: delete + set).
// - Ao passar do limite, remova a mais antiga: cache.keys().next().value.
// - limite 0 não guarda nada; o resultado 0 (quadrado de 0) é um acerto válido.

// Cenário para conferir com limite 2:
// q("A"); q("B"); q("A"); q("C"); -> "B" removida
// q("B") -> recalcula
// Ordem de uso das chaves registrada por você: ___
```

Exemplo e resultado público

chaves A,B; ler A; inserir C → B removida na política LRU

Bordas e erros publicados

com limite 2: ler A,B; ler A; inserir C -> B removida; buscar B recalcula; resultado quadrado de 0 -> hit válido; limite 0 -> não guardar entradas

Teste público e aprovação visível

Exija reprodução, causa, correção e regressão.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Num cache LRU de capacidade 2, há A e B; uma leitura de A ocorre antes da inserção de C. Qual chave é removida?

• A, porque qualquer leitura marca a chave para remoção.
 (Correta) B, porque A foi usada mais recentemente.
• Nenhuma; a política LRU permite exceder temporariamente o limite após uma inserção.

Por que a resposta está certa

Correto. A leitura atualiza a recência de A, então B é a menos recente quando C entra.

Armadilha da alternativa incorreta

Atualize a ordem de recência após a leitura de A. Quando C é inserida, a chave menos recentemente usada sai para manter o limite.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

com limite 2: ler A,B; ler A; inserir C → B removida; buscar B recalcula; resultado quadrado de 0 -> hit válido; limite 0 -> não guardar entradas

Concepção errada e falha observável

Usar Map sem limite e chamar memoização de otimização sem custo de memória. / Na borda com limite 2: ler A,B, observe a saída e compare-a com o contrato publicado; esta interpretação incorreta causa a falha: Usar Map sem limite e chamar memoização de otimização sem custo de memória.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar Map sem limite e chamar memoização de otimização sem custo de memória.

Pistas graduais

1. Execute o starter com as 1000 chaves distintas e registre a relação entre chamadas e cache.size. 2. Declare capacidade máxima e política LRU; um hit precisa renovar a posição da chave. 3. Após inserir além da capacidade, remova a chave menos recentemente usada e teste o recálculo de B.

Solução de referência explicada

O starter é um programa completo: memoize guarda toda chave diferente e o laço imprime o tamanho a cada 100 entradas, reproduzindo crescimento sem limite. Corrija com um cache LRU limitado a duas entradas, use Map.has para reconhecer inclusive resultado 0 e confirme a ordem A,B; acesso A; inclusão C; B removida.

Transferência em outro contexto

—

Evidência de domínio

Exija reprodução, causa, correção e regressão. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Map com ordem de inserção (ES2015) e Map.prototype.delete; Node.js 24 LTS e navegadores evergreen. | Próxima sugestão: 21.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.05 Worker para CPU

MAKE · 25 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 4.5/5

Competência: workers.cpu_boundary | Requer: functions.return | Introduz: workers.cpu_boundary

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 4 · depuração 4 · arquivos 4 · leitura 4 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Mover cálculo pesado independente para worker e criar fallback/teste da função pura.

Objetivo da atividade

Separe o cálculo puro, as responsabilidades do worker e da interface principal, e como cada resposta será associada ao pedido.

Explicação e modelo mental

• Modelo mental: Um worker pode executar cálculo independente e enviar mensagens; a interface e o DOM permanecem no thread principal. IDs permitem associar respostas a solicitações, e o fallback deve ser descrito sem prometer ausência de bloqueio.
• Exemplo resolvido: Para um n pequeno, a função pura e o worker devem produzir o mesmo resultado. Se dois pedidos estiverem em andamento, cada resposta carrega o ID correspondente.
• Armadilha comum: Tentar mover manipulação do DOM para o worker ou deixar o cálculo pesado no thread principal e presumir que a interface continuará responsiva.
• Por que importa: Uma fronteira clara preserva a interface e torna solicitações e falhas rastreáveis.

Material/arquivo inicial

primos.js: // 21.05 · Cálculo puro que bloqueia a interface quando n é grande.
// Este arquivo roda no executor do site; worker.js e main.js rodam no navegador, servidos localmente.

```
export function contarPrimos(n) {
let total = 0;
for (let i = 2; i <= n; i++) {
let primo = true;
for (let d = 2; d * d <= i; d++) {
if (i % d === 0) { primo = false; break; }
}
if (primo) total++;
}
return total;
}

// Teste da função pura (deve dar o mesmo resultado dentro do worker):
console.log(contarPrimos(10)); // esperado: 4
console.log(contarPrimos(100)); // esperado: 25
```

| worker.js: // worker.js · Recebe { id, n } e responde { id, result } ou { id, error }.

```
import { contarPrimos } from "./primos.js";

self.addEventListener("message", (evento) => {
const { id, n } = evento.data;
// complete: calcule com contarPrimos e responda com postMessage, associando o id;
// em caso de erro, responda { id, error: mensagem }.
});
```

| main.js: // main.js · Interface: carregue com <script type="module" src="main.js"> por um servidor local.

```
import { contarPrimos } from "./primos.js";

const pendentes = new Map(); // id -> { resolve, reject }
let proximoId = 0;
let worker = null;

if (typeof Worker === "function") {
worker = new Worker(new URL("./worker.js", import.meta.url), { type: "module" });
// complete: ao receber { id, result } ou { id, error }, resolva/rejeite a Promise pendente do mesmo id.
}

export function calcular(n) {
if (worker === null) {
// Fallback documentado: roda na thread principal e PODE bloquear a interface.
return Promise.resolve().then(() => contarPrimos(n));
}
const id = ++proximoId;
return new Promise((resolve, reject) => {
pendentes.set(id, { resolve, reject });
worker.postMessage({ id, n });
});
}

export function encerrar() {
// complete: terminate no cleanup e rejeite o que ainda estiver pendente.
}

// Experimento: dispare calcular(5_000_000) e calcular(10), e confira que cada resposta volta para a chamada certa.
```

Exemplo e resultado público

n pequeno → mesmo resultado no worker e função pura; UI continua responsiva

Bordas e erros publicados

duas solicitações -> id associa resposta; worker indisponível -> fallback assíncrono documentado, sem prometer ausência de bloqueio no fallback; terminate no cleanup

Teste público e aprovação visível

Resultado observável: n pequeno -> mesmo resultado no worker e função pura; UI continua responsiva. Borda a conferir: duas solicitações -> id associa resposta; worker indisponível -> fallback assíncrono documentado, sem prometer ausência de bloqueio no fallback; terminate no cleanup. Entrega esperada: Worker recebe n/id, calcula e responde; main controla estado e fallback; testes da função pura e integração de mensagens.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Ao delegar cálculo independente a um Web Worker, qual responsabilidade continua no thread principal?

• Executar todo o cálculo pesado, pois workers só enviam mensagens.
• Criar elementos DOM dentro do worker e anexá-los diretamente à página.
 (Correta) Atualizar a interface e manipular o DOM.

Por que a resposta está certa

Correto. O worker executa cálculo e troca mensagens; a página e seu DOM continuam no thread principal.

Armadilha da alternativa incorreta

Separe cálculo independente de interface: o DOM continua sob responsabilidade do thread principal.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

duas solicitações → id associa resposta; worker indisponível -> fallback assíncrono documentado, sem prometer ausência de bloqueio no fallback; terminate no cleanup

Concepção errada e falha observável

Mover DOM para worker ou esconder cálculo pesado no thread principal. / Na entrada duas solicitações, a saída observável não corresponde a id associa resposta; a divergência é compatível com esta hipótese: Mover DOM para worker ou esconder cálculo pesado no thread principal.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Mover DOM para worker ou esconder cálculo pesado no thread principal.

Pistas graduais

1. Extraia função pura sem DOM. 2. Serialize input/output com postMessage. 3. Identifique requisições e faça cleanup do worker.

Solução de referência explicada

Worker recebe n/id, calcula e responde; main controla estado e fallback; testes da função pura e integração de mensagens.

Transferência em outro contexto

—

Evidência de domínio

Resultado observável: n pequeno -> mesmo resultado no worker e função pura; UI continua responsiva. Borda a conferir: duas solicitações -> id associa resposta; worker indisponível -> fallback assíncrono documentado, sem prometer ausência de bloqueio no fallback; terminate no cleanup. Entrega esperada: Worker recebe n/id, calcula e responde; main controla estado e fallback; testes da função pura e integração de mensagens. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Web Workers (new Worker, postMessage) nos navegadores evergreen, recurso Baseline; no Node.js, worker_threads. | Próxima sugestão: 21.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.06 WeakMap cache por objeto

COMPARE · TRILHA OPCIONAL · 12 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: weakmap.object_cache | Requer: functions.return | Introduz: weakmap.object_cache

Eixos: novidade 4 · sintaxe 3 · composição 3 · estado 3 · depuração 2 · arquivos 1 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Associar metadados calculados a objetos com WeakMap, demonstrando por que string/id comum pediria Map.

Objetivo da atividade

Compare os casos do mesmo objeto, de outro objeto com a mesma forma e de um identificador string; anote a estrutura apropriada e o motivo.

Explicação e modelo mental

• Modelo mental: WeakMap associa valores a chaves objeto pela identidade. Um objeto diferente com os mesmos campos é outra chave; Map pode representar IDs primitivos e ser enumerado.
• Exemplo resolvido: Consultar o mesmo objeto novamente pode ser um cache hit. Um segundo objeto com a mesma forma é outra identidade e gera miss.
• Armadilha comum: Usar WeakMap para chave string ou supor que objetos estruturalmente iguais têm a mesma identidade.
• Por que importa: A estrutura deve corresponder à forma da chave e às operações que o programa precisa observar.

Material/arquivo inicial

metadados.js: // 21.06 · Metadados por objeto: WeakMap versus Map.

```
let calculos = 0;
function calculate(node) {
calculos++;
return { profundidade: node.filhos.length, calculadoEm: calculos };
}

const porObjeto = new WeakMap();
function metaDe(node) {
// complete: devolva do cache se existir; senão calcule, guarde e devolva
}

const a = { filhos: [1, 2] };
const mesmaForma = { filhos: [1, 2] };

metaDe(a);
metaDe(a);
metaDe(mesmaForma);
console.log("cálculos:", calculos); // previsão: ___ (identidade, não forma)

// Chave string em WeakMap:
try {
porObjeto.set("id-1", {});
} catch (erro) {
console.log("string como chave:", erro.name); // previsão: ___
}

// Quando o dado tem um ID string estável, qual estrutura usar? O que se perde (tamanho, enumeração)? ___
// Por que não dá para testar de forma determinística que a entrada some após a coleta de lixo? ___
```

Exemplo e resultado público

mesmo objeto → cache hit; outro objeto com mesma forma -> miss por identidade

Bordas e erros publicados

chave string em WeakMap -> TypeError; sem enumeração/tamanho; GC não testável deterministicamente

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Um WeakMap já contém metadados para um objeto. Um segundo objeto tem os mesmos campos, mas é outra instância. O que ocorre ao consultá-lo?

 (Correta) Há miss, pois WeakMap compara a identidade do objeto, não sua forma.
• Há hit, pois objetos com os mesmos campos são automaticamente a mesma chave.
• A consulta converte os campos em uma string e falha por não permitir chaves objeto.

Por que a resposta está certa

Correto. A segunda instância tem outra identidade, mesmo que seus campos coincidam.

Armadilha da alternativa incorreta

WeakMap usa a chave objeto, não uma comparação estrutural de propriedades. Compare se é a mesma referência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

chave string em WeakMap → TypeError; sem enumeração/tamanho; GC não testável deterministicamente

Concepção errada e falha observável

Escolher WeakMap para cache por ID string ou prometer liberação imediata. / Na entrada chave string em WeakMap, a saída observável não corresponde a TypeError; a divergência é compatível com esta hipótese: Escolher WeakMap para cache por ID string ou prometer liberação imediata.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Escolher WeakMap para cache por ID string ou prometer liberação imediata.

Pistas graduais

1. Chave WeakMap é objeto. 2. Compare por identidade. 3. Map serve para chaves primitivas e enumeração.

Solução de referência explicada

Cache WeakMap de metadados de objeto, com contador de cálculos; exemplos mostram Map para IDs e limite da observação de GC.

Transferência em outro contexto

—

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: WeakMap (ES2015) com chaves de objeto; Node.js 24 LTS e navegadores evergreen. | Próxima sugestão: 21.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.07 Debounce vs throttle

MAKE · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: performance.debounce_throttle | Requer: functions.return | Introduz: performance.debounce_throttle

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 4 · depuração 4 · arquivos 2 · leitura 4 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Implementar ambos, simular 100 eventos e medir numero/tempo de chamadas.

Objetivo da atividade

Registre no timeline as chamadas previstas para debounce e throttle e declare a política leading/trailing do throttle.

Explicação e modelo mental

• Modelo mental: Debounce reinicia a espera a cada evento e emite após o intervalo sem novos eventos. Throttle limita a frequência por janela; o contrato deve declarar leading/trailing.
• Exemplo resolvido: No exemplo público, debounce produz chamadas em t=140 e t=260; o throttle leading declarado produz em t=0 e t=160.
• Armadilha comum: Supor que debounce e throttle emitem nos mesmos instantes ou deixar indefinido o que ocorre na borda do intervalo.
• Por que importa: Uma política temporal explícita torna o comportamento previsível e testável com relógio controlado.

Material/arquivo inicial

tempo.js: // 21.07 · Debounce e throttle com relógio falso (determinístico).
// O relógio falso substitui setTimeout/Date.now para medir número e instante das chamadas.

```
function criarRelogio() {
let agora = 0;
let tarefas = [];
return {
agora: () => agora,
setTimeout(fn, ms) { const t = { quando: agora + ms, fn }; tarefas.push(t); return t; },
clearTimeout(t) { tarefas = tarefas.filter((x) => x !== t); },
avancarAte(alvo) {
for (;;) {
const proxima = tarefas.filter((t) => t.quando <= alvo).sort((x, y) => x.quando - y.quando)[0];
if (!proxima) break;
tarefas = tarefas.filter((t) => t !== proxima);
agora = proxima.quando;
proxima.fn();
}
agora = alvo;
},
};
}

// Trailing: chama fn ms depois do último evento.
function debounce(fn, ms, relogio) {
// implemente com relogio.setTimeout / relogio.clearTimeout
return () => {};
}

// Leading: chama no primeiro evento e ignora os seguintes até passarem ms.
function throttle(fn, ms, relogio) {
// implemente com relogio.agora()
return () => {};
}

function simular(criar) {
const relogio = criarRelogio();
const chamadas = [];
const alvo = criar(() => chamadas.push(relogio.agora()), 100, relogio);
for (const t of [0, 20, 40, 160]) {
relogio.avancarAte(t);
alvo();
}
relogio.avancarAte(1000);
return chamadas;
}

console.log("debounce", simular(debounce)); // esperado: [140, 260]
console.log("throttle", simular(throttle)); // esperado: [0, 160]
// Política de borda: um evento exatamente em t=100 dispara no throttle? ___
```

Exemplo e resultado público

debounce → chamadas t=140,260; throttle leading declarado -> t=0,160

Bordas e erros publicados

evento em t=100 -> política de borda documentada; cancel/flush se expostos testados; timer falso registra número/tempo

Teste público e aprovação visível

Resultado observável: debounce -> chamadas t=140,260; throttle leading declarado -> t=0,160. Borda a conferir: evento em t=100 -> política de borda documentada; cancel/flush se expostos testados; timer falso registra número/tempo. Entrega esperada: Implemente contratos temporais explícitos, simule 100 eventos e apresente tabela de timestamps/chamadas sem relógio real.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual afirmação distingue corretamente debounce e throttle no contrato desta atividade?

• Debounce limita uma chamada por janela fixa; throttle sempre espera um período silencioso depois do último evento.
 (Correta) Debounce reinicia a espera após cada evento; throttle limita a frequência por janela, com política leading/trailing declarada.
• Ambos emitem apenas depois que todos os eventos terminam, então têm o mesmo padrão temporal.

Por que a resposta está certa

Correto. A espera reiniciada caracteriza debounce; throttle aplica um limite de frequência conforme a política escolhida.

Armadilha da alternativa incorreta

Compare quando cada função pode emitir: debounce espera silêncio após o evento mais recente, enquanto throttle limita chamadas ao longo das janelas.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

evento em t=100 → política de borda documentada; cancel/flush se expostos testados; timer falso registra número/tempo

Concepção errada e falha observável

Achar que debounce e throttle têm mesmo padrão de emissão. / Na entrada evento em t=100, a saída observável não corresponde a política de borda documentada; a divergência é compatível com esta hipótese: Achar que debounce e throttle têm mesmo padrão de emissão.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que debounce e throttle têm mesmo padrão de emissão.

Pistas graduais

1. Debounce reinicia espera a cada evento. 2. Throttle limita frequência por janela. 3. Declare leading/trailing e teste com relógio falso.

Solução de referência explicada

Implemente contratos temporais explícitos, simule 100 eventos e apresente tabela de timestamps/chamadas sem relógio real.

Transferência em outro contexto

—

Evidência de domínio

Resultado observável: debounce -> chamadas t=140,260; throttle leading declarado -> t=0,160. Borda a conferir: evento em t=100 -> política de borda documentada; cancel/flush se expostos testados; timer falso registra número/tempo. Entrega esperada: Implemente contratos temporais explícitos, simule 100 eventos e apresente tabela de timestamps/chamadas sem relógio real. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: setTimeout, clearTimeout e performance.now(); navegadores evergreen e Node.js 24 LTS. | Próxima sugestão: 21.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.08 using / await using

MODIFY · TRILHA OPCIONAL · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.5/5

Competência: resources.explicit_disposal | Requer: functions.return | Introduz: resources.explicit_disposal

Eixos: novidade 5 · sintaxe 5 · composição 5 · estado 4 · depuração 4 · arquivos 3 · leitura 5 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Criar recurso descartavel didatico e reescrever bloco de try/finally com using; depois criar AsyncDisposable para await using.

Objetivo da atividade

Trace a aquisição e a liberação do recurso no caso síncrono e no assíncrono; anote também o fallback de compatibilidade.

Explicação e modelo mental

• Modelo mental: using descarta recursos síncronos ao sair do escopo; await using aguarda o descarte assíncrono. O cleanup ocorre mesmo se o corpo lança erro, e recursos são descartados em ordem LIFO.
• Exemplo resolvido: Ao terminar o bloco com using, dispose é chamado uma vez. Com await using, asyncDispose é aguardado antes de continuar.
• Armadilha comum: Supor suporte de sintaxe em qualquer runtime ou esquecer que o cleanup ainda deve ocorrer após um throw.
• Por que importa: O descarte previsível evita deixar recursos ativos depois que seu escopo terminou.

Material/arquivo inicial

recurso.js: // 21.08 · De try/finally para using / await using.
// Node.js 24 suporta using; em navegadores, confira o suporte. Sem suporte, mantenha try/finally.

const log = [];

function abrir(nome) {
log.push(abriu ${nome});
return {
nome,
[Symbol.dispose]() {
log.push(fechou ${nome});
},
};
}

// Versão atual com try/finally:
function processar() {
const a = abrir("A");
try {
const b = abrir("B");
try {
log.push("trabalho");
} finally {
b[Symbol.dispose]();
}
} finally {
a[Symbol.dispose]();
}
}

// Reescreva com using (os recursos devem fechar em ordem LIFO: B antes de A):
function processarComUsing() {
// using a = abrir("A");
// ...
}

processar();
console.log(log.join(" | "));
log.length = 0;

// Depois crie abrirAsync(nome) com [Symbol.asyncDispose]() async e use await using.
// Confirme: dispose chamado uma vez, mesmo quando o corpo lança.
if (typeof Symbol.dispose !== "symbol") {
console.log("Este runtime não tem Symbol.dispose: use try/finally.");
}
| native-using.test.mjs: import test from 'node:test';

// Escreva o teste de ordem de descarte de using e await using.
test.todo('using e await using');

Exemplo e resultado público

bloco termina → dispose chamado uma vez; await using -> asyncDispose aguardado

Bordas e erros publicados

throw no corpo -> cleanup ainda executa; runtime sem suporte -> usar try/finally; recursos em ordem LIFO

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual benefício conceitual oferece associar o descarte de um recurso ao escopo que o utiliza?

• Eliminar a necessidade de definir qualquer comportamento de descarte para o recurso.
• Manter o recurso disponível indefinidamente, mesmo depois que seu escopo terminou.
 (Correta) Tornar explícita a relação entre o tempo de uso do recurso e a responsabilidade de liberá-lo.

Por que a resposta está certa

A associação com o escopo ajuda a deixar visível quem usa e quem libera o recurso.

Armadilha da alternativa incorreta

Gerenciamento de escopo organiza a responsabilidade pelo descarte; não remove essa responsabilidade.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

throw no corpo → cleanup ainda executa; runtime sem suporte -> usar try/finally; recursos em ordem LIFO

Concepção errada e falha observável

Tratar using como sintaxe universal sem baseline ou esquecer descarte assíncrono. / Na entrada throw no corpo, a saída observável não corresponde a cleanup ainda executa; a divergência é compatível com esta hipótese: Tratar using como sintaxe universal sem baseline ou esquecer descarte assíncrono.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar using como sintaxe universal sem baseline ou esquecer descarte assíncrono.

Pistas graduais

1. Primeiro implemente try/finally equivalente. 2. Adicione métodos de descarte. 3. Verifique suporte do parser/runtime e ordem LIFO.

Solução de referência explicada

Demonstre recurso síncrono e assíncrono com logs de aquisição/uso/descarte, fallback documentado e falhas no corpo.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Explicit Resource Management (using, await using, Symbol.dispose, Symbol.asyncDispose). Confirmar o suporte no Node.js 24 LTS antes de tornar a atividade obrigatória; até lá, opcional. | Próxima sugestão: 21.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.09 Microprojeto - laboratório de performance

PROJECT · 95 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 5.0/5

Competência: performance.lab | Requer: functions.return | Introduz: performance.lab

Eixos: novidade 5 · sintaxe 5 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 5 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Construir página que compara duas implementações, mede tempo e memória aproximada e registra resultados.

Objetivo da atividade

Esboce como confirmar resultados equivalentes, separar warmup das medições e apresentar amostras, mediana, dispersão, ambiente e disponibilidade de memória.

Explicação e modelo mental

• Modelo mental: Uma comparação útil usa a mesma entrada, verifica equivalência dos resultados e resume múltiplas medições, incluindo mediana e dispersão. Memória pode ser aproximada ou indicada como indisponível.
• Exemplo resolvido: Para as duas funções puras com a mesma entrada, confirme primeiro que os resultados coincidem; depois apresente as medições e o ambiente. Se memória não estiver disponível, indique N/D.
• Armadilha comum: Usar um único tempo para declarar vencedor universal ou apresentar zero quando a medição de memória não está disponível.
• Por que importa: Relatar método e limitações ajuda a interpretar ruído sem transformar uma amostra em conclusão geral.

Material/arquivo inicial

lab.js: // 21.09 · Laboratório de performance: duas implementações, mesma entrada.
// Este arquivo roda no executor (sem DOM); index.html usa as mesmas funções para montar a tabela.

export function somaComReduce(numeros) {
return numeros.reduce((total, n) => total + n, 0);
}

export function somaComFor(numeros) {
let total = 0;
for (let i = 0; i < numeros.length; i++) total += numeros[i];
return total;
}

export function medir(fn, entrada, { warmups = 5, runs = 15 } = {}) {
// implemente: aquecimento, amostras com performance.now(), mediana e dispersão (mínimo e máximo)
}

export function comparar(entrada) {
if (!Array.isArray(entrada) || entrada.some((n) => !Number.isFinite(n))) {
return { erro: "Entrada inválida: use uma lista de números finitos." };
}
const a = somaComReduce(entrada);
const b = somaComFor(entrada);
if (a !== b) return { erro: Resultados diferentes: ${a} e ${b} };
const memoria = globalThis.performance?.memory?.usedJSHeapSize ?? "N/D";
return {
resultado: a,
reduce: medir(somaComReduce, entrada),
for: medir(somaComFor, entrada),
memoria,
ambiente: globalThis.navigator?.userAgent ?? "desconhecido",
};
}

const entrada = Array.from({ length: 100_000 }, (_, i) => i % 10);
console.log(comparar(entrada));
console.log(comparar([1, Number.NaN]));
// Regra do relatório: sem margem e variância, não declare vencedor.
| index.html: <!doctype html>
<html lang="pt-BR">
<meta charset="utf-8">
<title>Laboratório de performance</title>
<h1>Laboratório de performance</h1>
<label>Tamanho da entrada <input id="tamanho" type="number" value="100000" min="1"></label>
<button id="medir" type="button">Medir</button>

<table>
<thead><tr><th>Implementação</th><th>Mediana (ms)</th><th>Mín.</th><th>Máx.</th><th>Amostras</th></tr></thead>
<tbody id="resultados"></tbody>
</table>
Memória aproximada: N/D · Ambiente:
<script type="module">
import { comparar } from "./lab.js";
// complete: ao clicar em Medir, gere a entrada, chame comparar e preencha a tabela com textContent.
// Entrada inválida mostra a mensagem em #mensagem sem travar a página.
</script>

</html>

Exemplo e resultado público

mesmo input → outputs iguais; tabela mostra mediana de A/B e ambiente

Bordas e erros publicados

entrada inválida -> mensagem sem travar; medição de memória indisponível -> campo N/D; benchmark não declara vencedor sem margem/variância

Teste público e aprovação visível

Resultado observável: mesmo input -> outputs iguais; tabela mostra mediana de A/B e ambiente. Borda a conferir: entrada inválida -> mensagem sem travar; medição de memória indisponível -> campo N/D; benchmark não declara vencedor sem margem/variância. Entrega esperada: UI separa gerador de entrada, funções, medição e visualização; registra amostras, mediana, dispersão e suporte de memória.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que deve ser mantido consistente para comparar duas implementações de forma justa?

 (Correta) A tarefa realizada, os dados de entrada e as condições relevantes do ambiente de comparação.
• A implementação de uma delas, alterando-a durante cada execução para explorar mais casos.
• Somente o nome das funções, mesmo que cada uma execute uma tarefa diferente.

Por que a resposta está certa

Uma comparação interpretável mantém o trabalho e as condições relevantes equivalentes.

Armadilha da alternativa incorreta

Se a tarefa ou as condições mudam, não é possível atribuir a diferença apenas às implementações.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada inválida → mensagem sem travar; medição de memória indisponível -> campo N/D; benchmark não declara vencedor sem margem/variância

Concepção errada e falha observável

Mostrar um único tempo como prova de superioridade universal. / Na entrada entrada inválida, a saída observável não corresponde a mensagem sem travar; a divergência é compatível com esta hipótese: Mostrar um único tempo como prova de superioridade universal.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Mostrar um único tempo como prova de superioridade universal.

Pistas graduais

1. Valide equivalência de resultado antes de medir. 2. Aqueça e amostre múltiplas vezes. 3. Exiba método, limitações e dados brutos.

Solução de referência explicada

UI separa gerador de entrada, funções, medição e visualização; registra amostras, mediana, dispersão e suporte de memória.

Transferência em outro contexto

—

Evidência de domínio

Resultado observável: mesmo input -> outputs iguais; tabela mostra mediana de A/B e ambiente. Borda a conferir: entrada inválida -> mensagem sem travar; medição de memória indisponível -> campo N/D; benchmark não declara vencedor sem margem/variância. Entrega esperada: UI separa gerador de entrada, funções, medição e visualização; registra amostras, mediana, dispersão e suporte de memória. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: performance.now() e recursos Baseline nos navegadores evergreen. performance.memory é não padrão (Chromium): usar só como estimativa, com aviso. | Próxima sugestão: 21.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.10 Transferir não é copiar

INVESTIGATE · 20 min (estimativa, sem nota por velocidade) · Navegadores evergreen · recursos Baseline amplamente disponíveis · dificuldade multidimensional 3.5/5

Competência: workers.transfer_detach | Requer: functions.return | Introduz: workers.transfer_detach

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Transfira ArrayBuffer a worker, observe buffer detached no emissor e compare com structuredClone sem transferência.

Objetivo da atividade

Registre o byteLength do emissor e do worker antes e depois da transferência e compare com structuredClone sem transferência.

Explicação e modelo mental

• Modelo mental: A transferência de ArrayBuffer entrega a posse ao worker e desanexa o buffer no emissor. structuredClone sem transferência faz uma cópia independente e mantém o original válido.
• Exemplo resolvido: Após transferir o ArrayBuffer de 8 bytes, o emissor observa byteLength 0 e o worker recebe 8 bytes. Sem transfer list, a cópia não invalida o original.
• Armadilha comum: Tratar transferência como cópia normal e supor que emissor e worker continuam usando o mesmo buffer.
• Por que importa: Distinguir cópia de transferência esclarece quem pode acessar os dados e por que a transferência pode evitar cópias.

Material/arquivo inicial

transferir.js:

```
// 21.10 · Transferir não é copiar.
// structuredClone com transfer usa o mesmo algoritmo de postMessage(valor, [buffer]) para um worker.

const original = new ArrayBuffer(8);
new Uint8Array(original).fill(7);
console.log("antes:", original.byteLength);

// 1. Cópia sem transferência
const copia = structuredClone(original);
new Uint8Array(copia)[0] = 99;
console.log("depois da cópia: original", original.byteLength, "primeiro byte", new Uint8Array(original)[0], "| cópia",
copia.byteLength);

// 2. Transferência
const transferido = structuredClone(original, { transfer: [original] });
console.log("depois da transferência: emissor", original.byteLength, "detached?", original.detached, "| destino",
transferido.byteLength);

// Previsões antes de executar: ___
// Experimento local com worker: postMessage({ buffer }, [buffer]) e registre byteLength nos dois lados.
// SharedArrayBuffer tem outra semântica (memória compartilhada) e fica fora deste exercício.
```

Exemplo e resultado público

após transfer → sender byteLength 0/detached; worker vê 8 bytes

Bordas e erros publicados

structuredClone sem transfer -> cópia independente e buffer original válido; SharedArrayBuffer tem semântica distinta e fora do escopo

Teste público e aprovação visível

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que decisão de projeto deve anteceder o envio de dados binários entre contextos de execução?

• Presumir que os participantes podem modificar a mesma representação sem coordenar responsabilidades.
 (Correta) Definir quem é responsável pelo dado durante cada fase e por quanto tempo cada participante precisa dele.
• Escolher o formato dos tipos TypeScript antes de decidir qual dado a mensagem precisa transportar.

Por que a resposta está certa

Um contrato de mensagem claro considera responsabilidade e ciclo de vida do dado entre os participantes.

Armadilha da alternativa incorreta

A interface de tipos não substitui uma decisão sobre responsabilidade e uso do dado transmitido.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

structuredClone sem transfer → cópia independente e buffer original válido; SharedArrayBuffer tem semântica distinta e fora do escopo

Concepção errada e falha observável

Achar que transferência faz cópia normal e ambos podem continuar acessando mesmo buffer. / Na entrada structuredClone sem transfer, a saída observável não corresponde a cópia independente e buffer original válido; a divergência é compatível com esta hipótese: Achar que transferência faz cópia normal e ambos podem continuar acessando mesmo buffer.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que transferência faz cópia normal e ambos podem continuar acessando mesmo buffer.

Pistas graduais

1. Crie buffer e registre byteLength. 2. Transfira na lista de transferência. 3. Compare com structuredClone sem transfer.

Solução de referência explicada

Experimento registra estado antes/depois em main e worker; explica posse e cópia sem afirmar equivalência para SharedArrayBuffer.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de hipótese, evidência da ferramenta, conclusão e reprodução. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: ArrayBuffer transferível por postMessage e structuredClone, recursos Baseline nos navegadores evergreen; o buffer fica desanexado no emissor. | Próxima sugestão: 21.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

21.11 Erro durante descarte

PREDICT · TRILHA OPCIONAL · 6 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: errors.suppressed_cleanup | Requer: functions.return | Introduz: errors.suppressed_cleanup

Eixos: novidade 5 · sintaxe 4 · composição 4 · estado 4 · depuração 3 · arquivos 2 · leitura 5 · autonomia 4 · ferramentas 5

VISÃO DO ALUNO

Compare AggregateError e SuppressedError quando corpo e cleanup falham em ambiente com using.

Objetivo da atividade

Preveja qual erro é produzido para corpo A com descarte B e compare sua origem e propriedades com Promise.any quando todas as promessas rejeitam.

Explicação e modelo mental

• Modelo mental: SuppressedError representa um erro de descarte que ocorre enquanto outro erro já está sendo propagado; AggregateError reúne erros de operações como Promise.any quando todas rejeitam.
• Exemplo resolvido: Se o corpo lança A e dispose lança B, SuppressedError contém B em error e A em suppressed, conforme a semântica do runtime. Promise.any com todas rejeitadas expõe os erros em AggregateError.errors.
• Armadilha comum: Confundir a causa e o momento dos dois erros ou supor que um fallback manual com try/finally cria automaticamente SuppressedError.
• Por que importa: Identificar a origem dos erros preserva informação útil para diagnóstico e tratamento.

Material/arquivo inicial

descarte.js: // 21.11 · Erro no corpo e erro no descarte.
// Preveja antes de executar: qual erro sai do bloco? Quais propriedades ele tem?

```
function recurso() {
return {
[Symbol.dispose]() {
throw new Error("B: falha ao fechar");
},
};
}

function corpoEDescarteFalham() {
using r = recurso();
throw new Error("A: falha no corpo");
}

try {
corpoEDescarteFalham();
} catch (erro) {
console.log(erro.name, "| error:", erro.error?.message, "| suppressed:", erro.suppressed?.message);
}

// Compare com Promise.any quando todas rejeitam:
try {
await Promise.any([Promise.reject(new Error("x")), Promise.reject(new Error("y"))]);
} catch (erro) {
console.log(erro.name, "| errors:", erro.errors.map((e) => e.message));
}

// Previsões: nome do primeiro erro ___; error = ___; suppressed = ___.
// Um fallback com try/finally cria SuppressedError automaticamente? ___
```

| probe.mjs: // Sonda do exercício 21.11: registre suporte, nome e propriedades dos erros.
export {};

Exemplo e resultado público

corpo A + dispose B → SuppressedError com error B e suppressed A conforme semântica do runtime

Bordas e erros publicados

Promise.any todas rejeitam -> AggregateError.errors; await using descarta assincronamente; fallback try/finally não cria automaticamente SuppressedError

Teste público e aprovação visível

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O corpo lança A e o método dispose lança B. Como SuppressedError representa esses erros?

• AggregateError.errors contém A e B, pois todo erro de descarte é agregado.
• error contém A e suppressed contém B, e try/finally sempre cria esse objeto.
 (Correta) error contém B e suppressed contém A, conforme a semântica do runtime.

Por que a resposta está certa

Correto. SuppressedError preserva o erro de descarte em error e o erro anterior em suppressed.

Armadilha da alternativa incorreta

Separe os erros pela origem: B ocorre durante o descarte; A já estava sendo propagado pelo corpo. AggregateError é o caso de várias rejeições.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

Promise.any todas rejeitam → AggregateError.errors; await using descarta assincronamente; fallback try/finally não cria automaticamente SuppressedError

Concepção errada e falha observável

Confundir erro agregado de múltiplas promessas com erro suprimido por descarte. / Na entrada Promise.any todas rejeitam, a saída observável não corresponde a AggregateError.errors; a divergência é compatível com esta hipótese: Confundir erro agregado de múltiplas promessas com erro suprimido por descarte.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Confundir erro agregado de múltiplas promessas com erro suprimido por descarte.

Pistas graduais

1. Identifique fonte e momento dos erros. 2. Inspecione error/suppressed em SuppressedError. 3. Compare .errors de AggregateError.

Solução de referência explicada

Predição e experimento em runtime compatível; tabela de propriedades preserva ambos os erros sem equivaler as classes.

Transferência em outro contexto

—

Evidência de domínio

Compare resposta registrada antes de Run com sequência/valor previstos; confiança é diagnóstica. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: AggregateError (ES2021) e SuppressedError, parte de Explicit Resource Management. Mesma condição de suporte da 21.08. | Próxima sugestão: 22.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22 Pacotes, declarations e schemas

11 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Ordem: 22.01 · 22.02 · 22.03 · 22.04 · 22.11 · 22.05 · 22.06 · 22.07 · 22.08 · 22.09 · 22.10

22.01 Runtime vs declaration

COMPARE · 12 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.0/5

Competência: ts.declaration_truth | Requer: ts.basic, modules.esm | Introduz: ts.declaration_truth

Eixos: novidade 2 · sintaxe 2 · composição 3 · estado 2 · depuração 2 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Comparar arquivo JS e .d.ts incompatíveis e identificar mentiras do tipo.

Objetivo da atividade

Preveja o tipo real retornado pelo JavaScript e como o consumidor deve usar o resultado.

Explicação e modelo mental

• Modelo mental: O arquivo .d.ts descreve o JavaScript; não transforma o valor que ele retorna.
• Exemplo resolvido: Se a função retorna Promise<string | null>, o consumidor aguarda a Promise e considera que o resultado pode ser null.
• Armadilha comum: Tratar a Promise como string ou supor que a declaração elimina a possibilidade de null.
• Por que importa: Uma declaração fiel ajuda o TypeScript a revelar usos que podem falhar em execução.

Material/arquivo inicial

lib.js: // 22.01 · O JavaScript que roda de verdade.
// Compare com lib.d.ts: onde a declaração mente sobre este arquivo?

```
export async function buscarApelido(id) {
await Promise.resolve();
return id === "u1" ? "ana" : null;
}

// Experimento (executável aqui):
const resultado = buscarApelido("u9");
console.log("é Promise?", resultado instanceof Promise);
console.log("valor resolvido:", await resultado);
```

| lib.d.ts: // Declaração publicada junto com lib.js. O compilador do consumidor confia nela sem olhar o JavaScript.

```
export declare function buscarApelido(id: string): string;
```

| consumidor.ts: // Compila sem erros por causa de lib.d.ts. O que acontece em runtime?

```
import { buscarApelido } from "./lib.js";

const apelido = buscarApelido("u1");
console.log(apelido.toUpperCase());

// 1. Liste cada "mentira" da declaração (tipo de retorno, ausência de null).
// 2. Corrija lib.d.ts para Promise<string | null> e veja quais erros aparecem aqui.
// 3. Corrija o consumidor para tratar o caso null.
```

Exemplo e resultado público

JS runtime → Promise; d.ts diz string -> mentira identificada

Bordas e erros publicados

null resolvido -> consumidor ainda pode falhar; corrigir declaração para Promise<string|null>; tipos não alteram runtime

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que princípio deve orientar a manutenção de declarações TypeScript para uma biblioteca JavaScript?

 (Correta) Descrever a API que a implementação realmente oferece aos consumidores, em vez de prometer comportamento diferente.
• Descrever apenas a API que seria conveniente implementar no futuro.
• Usar a declaração para transformar automaticamente os valores produzidos pelo JavaScript.

Por que a resposta está certa

Declarações são um contrato estático para consumidores e precisam corresponder à implementação publicada.

Armadilha da alternativa incorreta

Um arquivo de tipos não altera o JavaScript nem deve prometer capacidades que ele não oferece.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

null resolvido → consumidor ainda pode falhar; corrigir declaração para Promise<string|null>; tipos não alteram runtime

Concepção errada e falha observável

Achar que .d.ts transforma valor real. / Na entrada null resolvido, a saída observável não corresponde a consumidor ainda pode falhar; a divergência é compatível com esta hipótese: Achar que .d.ts transforma valor real.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que .d.ts transforma valor real.

Pistas graduais

1. Execute o JS e observe tipo/valor. 2. Leia assinatura declarada. 3. Corrija a declaração para corresponder a todos os casos.

Solução de referência explicada

Relatório aponta divergência Promise/null e escreve assinatura fiel, com teste de consumo e runtime.

Transferência em outro contexto

—

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; comparação entre arquivo .d.ts e JavaScript no Node.js 24 LTS; sem API de runtime nova. | Próxima sugestão: 22.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.02 Parsons de package types

PARSONS · 10 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · pacote ESM · dificuldade multidimensional 3.0/5

Competência: ts.package_exports | Requer: ts.basic, modules.esm | Introduz: ts.package_exports

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 2 · depuração 3 · arquivos 4 · leitura 3 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Reordenar exports/types de pacote mínimo com ESM.

Objetivo da atividade

Preveja quais arquivos o consumidor ESM e o TypeScript precisam encontrar no pacote.

Explicação e modelo mental

• Modelo mental: O manifesto conecta os caminhos de importação e de tipos aos arquivos que o build realmente gera.
• Exemplo resolvido: O import ESM aponta para dist/index.js e a condição types aponta para dist/index.d.ts; ambos precisam estar no pacote.
• Armadilha comum: Apontar para um arquivo de declaração inexistente ou usar o .d.ts como implementação JavaScript.
• Por que importa: O consumidor precisa resolver a implementação em execução e os tipos publicados.

Material/arquivo inicial

blocos.md: # Monte o package.json do pacote ESM

O build gera dist/index.js e dist/index.d.ts a partir de src/index.ts. Acrescente ao package.json os blocos necessários, na ordem que faz sentido. Um bloco aponta para um arquivo que não existe: deixe-o de fora.

``text [A] "type": "module", [B] "exports": { ".": { "types": "./dist/index.d.ts", "default": "./dist/index.js" } }, [C] "scripts": { "build": "tsc -p tsconfig.build.json", "prepack": "npm run build" }, [D] "main": "./index.js", [E] "types": "./dist/index.d.ts", Responda: 1. Por que "types" vem antes de "default" dentro de exports? 2. O que acontece com um consumidor TypeScript se dist/index.d.ts não existir no pacote? 3. Por que o build precisa rodar antes de publicar (prepack)? 4. Consumidor para conferir: import { sum } from "@curso/soma";` resolve qual arquivo em runtime e qual no typecheck?
| package.json: {
"name": "@curso/soma",
"version": "1.0.0",
"license": "MIT",
"files": ["dist"]
}
| package-lock.json: {
"name": "@curso/soma",
"version": "1.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "@curso/soma",
"version": "1.0.0",
"license": "MIT"
}
}
}

Exemplo e resultado público

import {sum} from pacote em consumidor ESM → runtime e TS resolvem dist

Bordas e erros publicados

arquivo de declaration ausente -> pacote falha para TS; export path não existente -> falha de import; publicar dist após build

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que o metadata de um pacote faz parte da experiência de sua API pública?

• Ele substitui a implementação e as declarações de tipo depois da instalação.
 (Correta) Ele orienta ferramentas consumidoras sobre como localizar e interpretar a interface publicada.
• Ele só afeta a organização dos arquivos no repositório do autor, nunca quem consome o pacote.

Por que a resposta está certa

A configuração do pacote participa da descoberta e do consumo da API por outros projetos.

Armadilha da alternativa incorreta

Metadata não substitui os arquivos do pacote e pode influenciar diretamente a resolução pelo consumidor.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

arquivo de declaration ausente → pacote falha para TS; export path não existente -> falha de import; publicar dist após build

Concepção errada e falha observável

Declarar types apontando para fonte inexistente ou export sem JS correspondente. / Na entrada arquivo de declaration ausente, a saída observável não corresponde a pacote falha para TS; a divergência é compatível com esta hipótese: Declarar types apontando para fonte inexistente ou export sem JS correspondente.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Declarar types apontando para fonte inexistente ou export sem JS correspondente.

Pistas graduais

1. Liste arquivos que build realmente gera. 2. Exports liga import a JS e types a d.ts. 3. Teste consumo fora do diretório do pacote.

Solução de referência explicada

Ordene manifest com type module, exports {'.':{types,import}}, files dist e script build; verifique caminhos no pacote empacotado.

Transferência em outro contexto

—

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: exports e types em package.json com módulos ES; TypeScript 7.0 com module: nodenext no Node.js 24 LTS. | Próxima sugestão: 22.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.03 Complete declaração

FADED · 10 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.declaration_authoring | Requer: ts.basic, modules.esm | Introduz: ts.declaration_authoring

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Escrever .d.ts para biblioteca JS pequena usando exemplos e testes runtime.

Objetivo da atividade

Preveja como o consumidor distingue os dois resultados de parse e quais entradas a função aceita.

Explicação e modelo mental

• Modelo mental: Uma união discriminada descreve alternativas distintas; o consumidor usa ok para saber qual propriedade está disponível.
• Exemplo resolvido: No ramo ok: true, o resultado tem value; no ramo ok: false, tem error. A entrada de parse continua restrita a string.
• Armadilha comum: Declarar value e error como se ambos estivessem presentes em qualquer resultado ou ampliar a entrada para any.
• Por que importa: A declaração fiel permite ao consumidor escrever código que respeita os resultados reais da biblioteca.

Material/arquivo inicial

parser.js: // 22.03 · Biblioteca JavaScript pequena, sem tipos. Escreva parser.d.ts a partir deste código e dos testes.

```
export function parse(input) {
if (typeof input !== "string") throw new TypeError("input precisa ser string");
const n = Number(input.trim());
return Number.isFinite(n) ? { ok: true, value: n } : { ok: false, error: não é número: ${input} };
}

const ouvintes = new Map();
export function on(evento, listener) {
if (!ouvintes.has(evento)) ouvintes.set(evento, new Set());
ouvintes.get(evento).add(listener);
return () => ouvintes.get(evento).delete(listener);
}

// Testes de runtime (conferem a forma que a declaração precisa descrever):
console.log(parse(" 42 ")); // { ok: true, value: 42 }
console.log(parse("x")); // { ok: false, error: "não é número: x" }
const remover = on("parse", (r) => console.log("ouviu", r));
console.log(typeof remover); // "function"
```

| parser.d.ts: // Complete a declaração. parse aceita só string e devolve uma union discriminada por ok.

```
export type ParseResult =
| { ok: true; value: ___ }
| { ok: false; error: ___ };

export declare function parse(input: ___): ParseResult;

// on devolve uma função que remove o listener.
export declare function on(evento: string, listener: (resultado: ___) => void): ___;
```

| consumidor.ts: // Depois de completar parser.d.ts, isto deve compilar:

```
import { parse } from "./parser.js";

const r = parse("10");
if (r.ok) {
console.log(r.value.toFixed(2));
} else {
console.log(r.error);
}

// E estas linhas devem falhar no typecheck (deixe com @ts-expect-error):
// @ts-expect-error value não existe no ramo de erro
if (!r.ok) console.log(r.value);
// @ts-expect-error parse só aceita string
parse(10);
```

Exemplo e resultado público

consumidor válido → compila; parse retorna union ok:true/value ou ok:false/error

Bordas e erros publicados

consumidor acessa value no ramo erro -> erro TS; declaration aceita apenas entrada string; testes runtime conferem shape

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual é o papel principal de uma declaração .d.ts ao distribuir uma biblioteca JavaScript?

• Executar a lógica da biblioteca em ambientes que não carregam JavaScript.
• Converter automaticamente argumentos inválidos em valores aceitos durante a execução.
 (Correta) Descrever a interface estática disponível para consumidores TypeScript sem substituir a implementação JavaScript.

Por que a resposta está certa

A declaração ajuda o consumidor e o typechecker a entender a API; o comportamento continua na implementação.

Armadilha da alternativa incorreta

Um .d.ts não é implementação executável nem valida argumentos em runtime.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

consumidor acessa value no ramo erro → erro TS; declaration aceita apenas entrada string; testes runtime conferem shape

Concepção errada e falha observável

Escrever declaration desejada em vez da API JS real. / Na entrada consumidor acessa value no ramo erro, a saída observável não corresponde a erro TS; a divergência é compatível com esta hipótese: Escrever declaration desejada em vez da API JS real.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Escrever declaration desejada em vez da API JS real.

Pistas graduais

1. Execute exemplos para identificar casos. 2. Modele resultado discriminado. 3. Type tests rejeitam acesso/call inválidos.

Solução de referência explicada

Declare exports reais com parâmetros/retornos fiéis, teste compilação positiva/negativa e compare com comportamento JS.

Transferência em outro contexto

—

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; declarações .d.ts com export e declare; sem API de runtime nova. | Próxima sugestão: 22.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.04 Augmentation perigosa

MODIFY · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.global_augmentation | Requer: ts.basic, modules.esm | Introduz: ts.global_augmentation

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 3 · arquivos 3 · leitura 4 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Remover monkey patch global desnecessário ou isolá-lo com documentação e escopo.

Objetivo da atividade

Preveja o efeito de remover o monkey patch e como preservar o comportamento para os chamadores.

Explicação e modelo mental

• Modelo mental: Uma declaração global altera o que o TypeScript reconhece, mas não instala um método no runtime.
• Exemplo resolvido: Um helper first(xs) pode retornar o primeiro item e undefined para array vazio, sem modificar Array.prototype.
• Armadilha comum: Deixar apenas declare global e acreditar que isso fornece a implementação em execução.
• Por que importa: Reduzir mudanças globais diminui o risco de colisões e mantém o contrato de runtime explícito.

Material/arquivo inicial

patch.js: // 22.04 · Monkey patch global.
// Este arquivo acrescenta um método a TODOS os arrays do programa, inclusive de bibliotecas de terceiros.

```
Array.prototype.first = function () {
return this[0];
};

// Uso espalhado pelo código:
console.log([3, 4].first()); // 3
console.log([].first()); // undefined

// Efeito colateral: o método novo aparece em for...in sobre arrays.
for (const chave in ["a"]) console.log("for...in viu:", chave);

// Tarefa:
// 1. Escreva export function first(xs) sem mexer em Array.prototype.
// 2. Migre os usos acima para first(xs) e mantenha o caso de array vazio -> undefined.
// 3. Se um patch global fosse realmente necessário, onde ficariam a declaração declare global
// e a implementação, e o que a documentação precisaria dizer? ___
```

Exemplo e resultado público

helper first(xs) → primeiro item sem monkey patch; testes antigos migram

Bordas e erros publicados

array vazio -> undefined; se augmentation realmente necessária, declaração global e implementação runtime no mesmo pacote com documentação explícita

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual risco arquitetural pode surgir ao ampliar globalmente um protótipo embutido?

 (Correta) Código não relacionado pode passar a depender da alteração, aumentando conflitos e efeitos inesperados.
• A alteração fica automaticamente restrita ao módulo que declara o método.
• A declaração TypeScript garante que outros pacotes não vejam a mudança em runtime.

Por que a resposta está certa

Uma alteração global amplia o alcance da API e pode afetar código fora do módulo que a introduziu.

Armadilha da alternativa incorreta

Uma declaração de tipo não limita o alcance runtime de uma mutação global.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

array vazio → undefined; se augmentation realmente necessária, declaração global e implementação runtime no mesmo pacote com documentação explícita

Concepção errada e falha observável

Achar que declare global instala método em execução. / Na entrada array vazio, a saída observável não corresponde a undefined; a divergência é compatível com esta hipótese: Achar que declare global instala método em execução.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que declare global instala método em execução.

Pistas graduais

1. Procure chamadas existentes. 2. Prefira função local quando API global não é requisito. 3. Se manter, alinhe d.ts, runtime, carregamento e colisões.

Solução de referência explicada

Refatore para helper exportado; alternativa documenta escopo/risco e teste runtime sem depender só de declaration.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; declare global e augmentation de módulos; Node.js 24 LTS. | Próxima sugestão: 22.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.11 JSDoc e verificação de JavaScript

MAKE · 35 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 2.0/5

Competência: ts.jsdoc_checkjs | Requer: ts.basic, modules.esm | Introduz: ts.jsdoc_checkjs

Eixos: novidade 2 · sintaxe 2 · composição 2 · estado 1 · depuração 2 · arquivos 2 · leitura 3 · autonomia 3 · ferramentas 2

VISÃO DO ALUNO

Corrija as anotações JSDoc e complete calcularFrete em main.js, mantendo o arquivo JavaScript. Use // @ts-check e tipos corretos para pesoKg: number, expressa?: boolean e retorno number. Para este projeto, checkJs: true ativa a checagem dos arquivos JavaScript; o site verifica os tipos e executa os testes.

Objetivo da atividade

Leia os tipos JSDoc de partida e preveja por que Number.isFinite e o retorno conflitam com eles; indique também o caso runtime que deve lançar RangeError.

Explicação e modelo mental

• Modelo mental: JSDoc descreve assinaturas que o TypeScript pode conferir sem renomear .js para .ts. // @ts-check ativa a checagem naquele arquivo; checkJs: true aplica a checagem a todos os .js incluídos.
• Exemplo resolvido: /** @param {number} pesoKg @param {boolean} [expressa=false] @returns {number} */ documenta uma função JavaScript. O checker compara chamadas e corpo com essas anotações; a suíte executa o JavaScript real.
• Armadilha comum: Um comentário sem tipos úteis deixa parâmetros ou retornos implícitos em any; ausência de diagnóstico não prova um contrato. Typechecks também não provam o valor retornado em runtime.
• Por que importa: Checagem gradual permite explicitar tipos e encontrar erros em módulos JavaScript sem conversão imediata para TypeScript.

Material/arquivo inicial

main.js: // @ts-check

/**
* Calcula o frete em unidades monetárias para um peso positivo em kg.
* @param {string} pesoKg
* @param {boolean} [expressa=false]
* @returns {string}
*/
export function calcularFrete(pesoKg, expressa = false) {
if (!Number.isFinite(pesoKg) || pesoKg <= 0) {
throw new RangeError('O peso deve ser positivo e finito.');
}
// Implemente: frete comum custa 8 por kg; expresso custa 10 por kg.
return 0;
}
| tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"allowJs": true,
"checkJs": true,
"noEmit": true
}
}

Exemplo e resultado público

calcularFrete(2) → 16; a chamada correta compila e executa

Bordas e erros publicados

peso inválido -> RangeError; strings e resultado string são rejeitados por contratos protegidos; parâmetro ou retorno any falha IsAny

Teste público e aprovação visível

Resultado observável: calcularFrete(2) -> 16; a chamada correta compila e executa. Borda a conferir: peso inválido -> RangeError; strings e resultado string são rejeitados por contratos protegidos; parâmetro ou retorno any falha IsAny. Entrega esperada: Arquivo .js real com @ts-check e JSDoc é verificado pelo checker do site e executado; contratos de consumidor rejeitam tipos apagados/any e comportamento cobre valores e erro.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

JSDoc pode tipar uma função JavaScript sem converter o arquivo para .ts. // @ts-check ativa checagem naquele arquivo; neste exercício, checkJs: true no tsconfig oferece a mesma checagem em escopo de projeto.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que ativa a checagem de tipos num arquivo JavaScript?

• Apenas mudar a extensão para .ts
• Adicionar @returns sem executar o TypeScript
 (Correta) // @ts-check no arquivo ou checkJs: true no projeto
• Rodar JavaScript com Node.js

Por que a resposta está certa

Correto: são dois controles de escopo diferentes para o checker TypeScript.

Armadilha da alternativa incorreta

Extensão, JSDoc isolado ou execução Node não substituem a configuração da verificação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

peso inválido → RangeError; strings e resultado string são rejeitados por contratos protegidos; parâmetro ou retorno any falha IsAny

Concepção errada e falha observável

Tratar JSDoc como comentário sem efeito no typecheck, aceitar any ou confundir @ts-check local com checkJs global. / Na entrada peso inválido, a saída observável não corresponde a RangeError; a divergência é compatível com esta hipótese: Tratar JSDoc como comentário sem efeito no typecheck, aceitar any ou confundir @ts-check local com checkJs global.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Tratar JSDoc como comentário sem efeito no typecheck, aceitar any ou confundir @ts-check local com checkJs global.

Pistas graduais

1. Corrija JSDoc @param/@returns ao lado da função JavaScript exportada. 2. Ative allowJs/checkJs/strict para conferir o arquivo .js no TypeScript. 3. Separe contrato estático protegido da suíte de comportamento executada.

Solução de referência explicada

Arquivo .js real com @ts-check e JSDoc é verificado pelo checker do site e executado; contratos de consumidor rejeitam tipos apagados/any e comportamento cobre valores e erro.

Transferência em outro contexto

Tipar por JSDoc uma função .js que recebe uma lista e devolve uma quantidade; acrescente contrato consumidor válido/inválido e um teste runtime.

Evidência de domínio

Resultado observável: calcularFrete(2) -> 16; a chamada correta compila e executa. Borda a conferir: peso inválido -> RangeError; strings e resultado string são rejeitados por contratos protegidos; parâmetro ou retorno any falha IsAny. Entrega esperada: Arquivo .js real com @ts-check e JSDoc é verificado pelo checker do site e executado; contratos de consumidor rejeitam tipos apagados/any e comportamento cobre valores e erro. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: main.js continua sendo JavaScript executável. allowJs inclui JS na análise, checkJs habilita a checagem do projeto e @ts-check habilita a checagem por arquivo. As verificações de tipos e os testes runtime são gates separados. | Próxima sugestão: 22.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.05 Plano de migração

MODIFY · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.incremental_migration | Requer: ts.basic, modules.esm | Introduz: ts.incremental_migration

Eixos: novidade 3 · sintaxe 2 · composição 4 · estado 3 · depuração 3 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Migrar módulo por módulo usando allowJs/checkJs, reduzindo any em etapas.

Objetivo da atividade

Proponha uma ordem de migração e um critério para avançar de um módulo ao próximo.

Explicação e modelo mental

• Modelo mental: Uma migração incremental usa contratos e checkpoints para manter os consumidores funcionando enquanto reduz any.
• Exemplo resolvido: Ative allowJs e checkJs, identifique dependências, migre primeiro a folha da cadeia (C no exemplo) e confira build e regressões a cada etapa.
• Armadilha comum: Renomear todos os arquivos de uma vez sem validar fronteiras e consumidores durante o processo.
• Por que importa: Checkpoints localizam regressões e tornam o progresso de tipagem verificável.

Material/arquivo inicial

a.js: // a.js · ponto de entrada; consumidor que precisa continuar rodando durante toda a migração.
import { totalDosPedidos } from "./b.js";

console.log(totalDosPedidos(['{"id":"p1","total":10}', '{"id":"p2","total":5}'])); // 15
| b.js: // b.js · usa C. Ainda sem verificação.
import { lerPedido } from "./c.js";

export function totalDosPedidos(textos) {
return textos.map(lerPedido).reduce((soma, pedido) => soma + pedido.total, 0);
}
| c.js: // @ts-check
// c.js · folha do grafo A -> B -> C. Primeiro módulo a migrar.
// Com @ts-check (ou checkJs no tsconfig), o TypeScript já verifica este arquivo usando JSDoc.

/**
* @param {string} texto
* @returns {{ id: string, total: number }}
*/
export function lerPedido(texto) {
const dados = JSON.parse(texto); // any: JSON ainda não validado
return { id: dados.id, total: dados.total };
}
| plano.md: # Plano de migração

tsconfig.json de partida: allowJs: true, checkJs: false, strict: true, noEmit: true. Ligue a verificação arquivo por arquivo com // @ts-check.

Passo	Módulo	O que muda	Critério para avançar
1	c.js		tsc --noEmit limpo, node a.js imprime 15
2			
3			

Regras: o número de any não cresce entre passos; o JSON de lerPedido passa a ser validado; os consumidores continuam rodando a cada passo.

Por que começar pela folha (C) e não pela entrada (A)?

Exemplo e resultado público

checkJs inicial aponta problemas; após migração de C e B → consumidores continuam rodando

Bordas e erros publicados

cada módulo migrado -> sem any crescente, JSON validado e testes existentes preservados

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

Conceitos prévios (Primers)

Arquivos .js podem ser verificados sem renomeá-los: // @ts-check habilita a checagem por arquivo e checkJs: true habilita a checagem dos JavaScript incluídos no projeto. JSDoc como /** @param {number} x @returns {string} */ informa os tipos; o typecheck não substitui testes de comportamento.

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como planejar uma migração gradual de JavaScript para TypeScript mantendo os consumidores funcionando?

• Renomear todos os arquivos para .ts de uma vez e corrigir os tipos somente depois que toda a migração terminar.
 (Correta) Ativar allowJs/checkJs, mapear contratos e dependências, migrar primeiro módulos folha e verificar build e regressões a cada etapa.
• Manter any em todas as fronteiras até o fim e considerar a migração concluída quando os arquivos tiverem extensão .ts.

Por que a resposta está certa

Checkpoints pequenos preservam consumidores e tornam visíveis os problemas de cada módulo antes de avançar.

Armadilha da alternativa incorreta

Uma migração em lote ou sem contratos deixa os problemas para o final. Use checkJs e valide cada etapa, reduzindo any progressivamente.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

cada módulo migrado → sem any crescente, JSON validado e testes existentes preservados

Concepção errada e falha observável

Renomear tudo .ts de uma vez sem tratar fronteiras ou executar regressões. / Na entrada cada módulo migrado, a saída observável não corresponde a sem any crescente, JSON validado e testes existentes preservados; a divergência é compatível com esta hipótese: Renomear tudo .ts de uma vez sem tratar fronteiras ou executar regressões.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Renomear tudo .ts de uma vez sem tratar fronteiras ou executar regressões.

Pistas graduais

1. Mapeie dependências e API pública. 2. Ative checkJs para inventário. 3. Migre folha primeiro e verifique consumidores em cada passo.

Solução de referência explicada

Plano A/B/C com ordem, critérios de saída por módulo, redução de any e typecheck/runtime após cada etapa.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: allowJs e checkJs (TypeScript 7.0) e JSDoc com @ts-check; migração módulo a módulo no Node.js 24 LTS. | Próxima sugestão: 22.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.06 Guard manual -> schema

COMPARE · 12 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: validation.schema_boundary | Requer: ts.basic, modules.esm | Introduz: validation.schema_boundary

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Validar um payload pequeno manualmente; depois modelar a mesma fronteira com uma biblioteca de schema (Zod ou Valibot) e comparar mensagens, inferencia e custo.

Objetivo da atividade

Preveja quais entradas devem passar ou falhar e que critérios ajudam a comparar as duas abordagens.

Explicação e modelo mental

• Modelo mental: A validação de runtime protege a fronteira de dados; os tipos, sozinhos, não validam valores recebidos.
• Exemplo resolvido: Ambas as abordagens devem preservar um payload válido e rejeitar amount como string, NaN ou null.
• Armadilha comum: Escolher a biblioteca só para satisfazer o compilador ou adicioná-la sem avaliar a dependência.
• Por que importa: Comparar o mesmo contrato torna visíveis diferenças de mensagens, inferência e custo.

Material/arquivo inicial

pagamento.js: // 22.06 · Mesma fronteira, duas abordagens. Primeiro o guard manual (executável aqui).

```
export function validarPagamento(payload) {
if (typeof payload !== "object" || payload === null) {
return { ok: false, erros: ["payload precisa ser objeto"] };
}
const erros = [];
if (typeof payload.id !== "string" || payload.id === "") erros.push("id: texto não vazio");
// TODO: valide amount como número finito; NaN, null e strings precisam ser rejeitados.
return erros.length ? { ok: false, erros } : { ok: true, valor: { id: payload.id, amount: payload.amount } };
}

const casos = [{ id: "p1", amount: 10 }, { id: "p1", amount: "10" }, { id: "p1", amount: Number.NaN }, { id: "p1", amount: null }, null];
for (const caso of casos) {
const descricao = caso === null ? "null" : { id: ${JSON.stringify(caso.id)}, amount: ${typeof caso.amount === "string" ?
JSON.stringify(caso.amount) : String(caso.amount)} };
console.log(descricao, "->", JSON.stringify(validarPagamento(caso)));
}

// Segunda versão, no seu computador, depois de decidir a dependência (npm install zod ou valibot):
// import * as z from "zod";
// const Pagamento = z.object({ id: z.string().min(1), amount: z.number().finite() });
// type Pagamento = z.infer<typeof Pagamento>;
// Pagamento.safeParse(caso)
//
// Compare: mensagens de erro, tipo inferido, tamanho adicionado ao bundle e custo de manter a dependência.
```

Exemplo e resultado público

payload válido → tipo de domínio e valor preservado em ambas versões

Bordas e erros publicados

amount string/NaN/null -> falha; biblioteca não é instalada sem decisão de dependência; comparar mensagem, inferência, bundle/custo

Teste público e aprovação visível

Aceite escolha justificada por diferença semântica publicada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que torna significativa a comparação entre duas estratégias de validação para a mesma fronteira?

• Permitir que cada estratégia aceite dados e regras diferentes para maximizar o número de casos distintos.
• Escolher uma estratégia antes de declarar qual comportamento a fronteira precisa garantir.
 (Correta) Manter o contrato e o propósito da fronteira constantes para comparar como cada estratégia expressa esse mesmo objetivo.

Por que a resposta está certa

Uma comparação útil examina abordagens contra a mesma necessidade pública.

Armadilha da alternativa incorreta

Se cada alternativa resolve um problema diferente, a comparação não informa qual atende melhor ao mesmo contrato.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

amount string/NaN/null → falha; biblioteca não é instalada sem decisão de dependência; comparar mensagem, inferência, bundle/custo

Concepção errada e falha observável

Usar schema apenas para satisfazer compilador e esquecer runtime ou adicionar dependência sem razão. / Na entrada amount string/NaN/null, a saída observável não corresponde a falha; a divergência é compatível com esta hipótese: Usar schema apenas para satisfazer compilador e esquecer runtime ou adicionar dependência sem razão.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar schema apenas para satisfazer compilador e esquecer runtime ou adicionar dependência sem razão.

Pistas graduais

1. Escreva guard manual honesto primeiro. 2. Modele o mesmo contrato na biblioteca escolhida. 3. Compare DX, custo e erros com entradas iguais.

Solução de referência explicada

Duas versões validadoras com corpus compartilhado e decisão documentada; tipo inferido não substitui verificação de regra de negócio.

Transferência em outro contexto

—

Evidência de domínio

Aceite escolha justificada por diferença semântica publicada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; Zod ou Valibot em suas versões estáveis atuais, instaladas pelo exercício; Node.js 24 LTS. | Próxima sugestão: 22.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.07 Type test de pacote

TEST · 25 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.0/5

Competência: ts.package_type_tests | Requer: ts.basic, modules.esm | Introduz: ts.package_type_tests

Eixos: novidade 3 · sintaxe 4 · composição 4 · estado 2 · depuração 3 · arquivos 3 · leitura 3 · autonomia 3 · ferramentas 4

VISÃO DO ALUNO

Escrever casos que devem compilar e casos com @ts-expect-error para superficie publica de uma biblioteca.

Objetivo da atividade

Esboce um caso válido, um inválido e o que espera que o compilador sinalize em cada um.

Explicação e modelo mental

• Modelo mental: Type tests verificam o contrato estático de consumo; testes de runtime verificam o comportamento do JavaScript.
• Exemplo resolvido: parse('x') deve compilar; parse(2) deve produzir diagnóstico coberto por @ts-expect-error.
• Armadilha comum: Usar supressões amplas ou confiar em @ts-expect-error sem verificar que realmente existe um erro.
• Por que importa: Casos válidos e inválidos detectam mudanças acidentais na superfície pública de tipos.

Material/arquivo inicial

tipos.test.ts: import { parse } from './index.js';

// Este arquivo é uma suíte de tipos: não execute JavaScript para avaliá-lo.
// TODO: complete os casos positivos e negativos da declaração.

// @ts-expect-error parse aceita somente string
parse(2);

// TODO: teste narrowing, rota com barra inicial e baseUrl obrigatório.
| index.d.ts: // Superfície pública do pacote sob teste.
export type Result<T> = { ok: true; value: T } | { ok: false; error: string };

export declare function parse(input: string): Result<number>;

export declare class Client {
constructor(baseUrl: string);
get(path: /${string}): Promise<Result<unknown>>;
}

Exemplo e resultado público

parse('x') → compila; parse(2) com @ts-expect-error exige diagnóstico

Bordas e erros publicados

remover erro de uma linha expect-error -> tsc reclama anotação não usada; runtime test separado confirma JS

Teste público e aprovação visível

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que testar uma biblioteca a partir de exemplos de uso de um consumidor?

 (Correta) Para verificar se a superfície pública é compreensível e atende os usos que a biblioteca pretende oferecer.
• Para testar somente a organização interna dos arquivos do autor.
• Para evitar documentar a API, pois os exemplos substituem toda descrição.

Por que a resposta está certa

A perspectiva do consumidor ajuda a avaliar se a API publicada oferece uma experiência utilizável.

Armadilha da alternativa incorreta

Um exemplo de consumo não substitui toda documentação nem se limita à organização interna.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

remover erro de uma linha expect-error → tsc reclama anotação não usada; runtime test separado confirma JS

Concepção errada e falha observável

Achar que tsc passou porque comentário suprimiu tudo, sem positivos/negativos. / Na entrada remover erro de uma linha expect-error, a saída observável não corresponde a tsc reclama anotação não usada; a divergência é compatível com esta hipótese: Achar que tsc passou porque comentário suprimiu tudo, sem positivos/negativos.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Achar que tsc passou porque comentário suprimiu tudo, sem positivos/negativos.

Pistas graduais

1. Escreva casos válidos primeiro. 2. Casos negativos usam @ts-expect-error pontual. 3. Verifique erro de anotação não usada e teste runtime separado.

Solução de referência explicada

Suíte de consumo usa tsconfig strict e pares válido/inválido para cada export, sem ts-ignore genérico.

Transferência em outro contexto

—

Evidência de domínio

Execute teste escrito pelo aluno contra implementação correta e mutantes defeituosos. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict; @ts-expect-error e tipos da superfície pública; sem API de runtime nova. | Próxima sugestão: 22.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.08 Microprojeto - pacote publicável

PROJECT · 100 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · pacote ESM · dificuldade multidimensional 4.5/5

Competência: ts.publishable_package | Requer: ts.basic, modules.esm | Introduz: ts.publishable_package

Eixos: novidade 4 · sintaxe 4 · composição 5 · estado 3 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Criar pequena biblioteca TS com build de JS, declarations, exports, testes e exemplo de consumo.

Objetivo da atividade

Preveja como testar o artefato distribuído fora do workspace e que aspectos precisam ser conferidos.

Explicação e modelo mental

• Modelo mental: Um pacote deve funcionar pelo conteúdo publicado, não depender de arquivos disponíveis apenas no workspace de desenvolvimento.
• Exemplo resolvido: Gere um tarball com npm pack e faça um consumidor isolado importar a biblioteca e verificar os tipos.
• Armadilha comum: Considerar os testes internos suficientes sem conferir se dist e os caminhos de exports estão no pacote.
• Por que importa: O consumidor externo usa o artefato empacotado, incluindo JavaScript, declarations e metadados.

Material/arquivo inicial

src/index.ts: // 22.08 · Biblioteca publicável: duas funções, tipos explícitos na API pública.

```
export type Moeda = "BRL" | "USD";

export function centavosParaTexto(centavos: number, moeda: Moeda = "BRL"): string {
if (!Number.isSafeInteger(centavos)) throw new RangeError("centavos precisa ser inteiro seguro");
return new Intl.NumberFormat("pt-BR", { style: "currency", currency: moeda }).format(centavos / 100);
}

export function somarCentavos(valores: readonly number[]): number {
// implemente: soma inteiros seguros; valor inválido lança RangeError
}
```

| src/index.test.ts: // Testes do código-fonte: node --test src/index.test.ts.

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { centavosParaTexto, somarCentavos } from "./index.ts";

test("formata em pt-BR", () => {
assert.match(centavosParaTexto(1990), /19,90/);
});

test("soma e rejeita inválidos", () => {
assert.equal(somarCentavos([100, 250]), 350);
assert.throws(() => somarCentavos([1.5]), RangeError);
});
```

| package.json: {

```
"name": "@curso/centavos",
"version": "0.1.0",
"license": "MIT",
"type": "module",
"exports": {
".": {
"types": "./dist/___",
"default": "./dist/___"
}
},
"files": [
"dist"
],
"scripts": {
"build": "tsc -p tsconfig.json",
"test": "node --test src/index.test.ts",
"prepack": "npm run build"
},
"devDependencies": {
"@types/node": "24.19.1",
"typescript": "6.0.3"
}
}
```

| tsconfig.json: {

```
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"noUncheckedIndexedAccess": true,
"erasableSyntaxOnly": true,
```

"verbatimModuleSyntax": true,
"rewriteRelativeImportExtensions": true,
"declaration": true,
"rootDir": "src",
"outDir": "dist",
"types": ["node"]
},
"include": ["src"],
"exclude": ["src/**/*.test.ts"]
}
| README.md: # @curso/centavos

Complete este README antes de empacotar:

- Instalação e exemplo de uso (ESM).
- API: assinatura, retorno e erros de centavosParaTexto e somarCentavos.
- Requisitos: Node.js 24 LTS ou posterior.
- Licença: MIT. Dependências de runtime: nenhuma.
- Como validar o pacote: npm pack, instalar o .tgz no consumidor e rodar runtime e typecheck.
| consumidor/main.mjs: // Consumidor fora do workspace. Passos, sem publicar nada na rede:
// 1. Na pasta do pacote: npm run build && npm pack (gera curso-centavos-0.1.0.tgz)
// 2. Nesta pasta: npm init -y && npm install ../curso-centavos-0.1.0.tgz
// 3. node main.mjs e, num main.ts equivalente, npx tsc --noEmit
import { centavosParaTexto, somarCentavos } from "@curso/centavos";

console.log(centavosParaTexto(somarCentavos([100, 250])));
| package-lock.json: {
"name": "@curso/centavos",
"version": "0.1.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "@curso/centavos",
"version": "0.1.0",
"license": "MIT",
"devDependencies": {
"@types/node": "24.19.1",
"typescript": "6.0.3"
}
},
"node_modules/@types/node": {
"version": "24.19.1",
"resolved": "https://registry.npmjs.org/@types/node/-/node-24.19.1.tgz",
"integrity":
"sha512-aS3/DG0oM05K0RIXXP+hKjinGG5IgSSVGzswZxW3O0sS3pH4/fycXundUC9XsszgKCk4gHXylTEK6hyFxVxnoQ==",
"dev": true,
"license": "MIT",
"dependencies": {
"undici-types": ">=7.24.0 <7.24.7"
}
},
"node_modules/typescript": {
"version": "6.0.3",
"resolved": "https://registry.npmjs.org/typescript/-/typescript-6.0.3.tgz",
"integrity":

"sha512-y2TvuxSZPDyQakkFRPZHKFm+KKVqIisdg9/CZwm9ftvKXLP8NRWj38/ODjNbr43SsoXqNuAisEf1GdCxqWcdBw==",
"dev": true,
"license": "Apache-2.0",
"bin": {
"tsc": "bin/tsc",
"tsserver": "bin/tsserver"
},
"engines": {
"node": ">=14.17"
}
},
"node_modules/undici-types": {
"version": "7.24.6",
"resolved": "https://registry.npmjs.org/undici-types/-/undici-types-7.24.6.tgz",
"integrity":
"sha512-WRNW+sJgj5OBN4/0JpHFqtqzhpbnV0GuB+OozA9gCL7a993SmU+1JBZCzLNxYsbMfIeDL+lTsphD5jN5N+n0zg==",
"dev": true,
"license": "MIT"
}
}
}

Exemplo e resultado público

npm pack; consumidor instala tarball e importa → runtime e typecheck passam

Bordas e erros publicados

arquivo dist ausente -> validação falha; exports aponta caminhos reais; licença/dependências declaradas; sem publicar rede

Teste público e aprovação visível

Resultado observável: npm pack; consumidor instala tarball e importa -> runtime e typecheck passam. Borda a conferir: arquivo dist ausente -> validação falha; exports aponta caminhos reais; licença/dependências declaradas; sem publicar rede. Entrega esperada: Pacote local com API curta, declarations fiéis, exports e script build; fixture de consumidor prova artefato empacotado.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que informação um README de uma pequena biblioteca deve oferecer a quem vai avaliá-la?

• Uma lista de detalhes internos sem explicar como a biblioteca pode ser usada.
 (Correta) O propósito da biblioteca, uma forma de uso da API pública e orientações necessárias para desenvolver ou verificar o projeto.
• Somente um slogan, pois os consumidores podem inferir a API pelo nome do pacote.

Por que a resposta está certa

Uma documentação inicial útil conecta o propósito a um uso concreto e aos passos relevantes do projeto.

Armadilha da alternativa incorreta

Detalhes internos ou um slogan isolado não explicam como entender e usar a biblioteca.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

arquivo dist ausente → validação falha; exports aponta caminhos reais; licença/dependências declaradas; sem publicar rede

Concepção errada e falha observável

Testar só dentro do próprio workspace e ignorar conteúdo real do pacote. / Na entrada arquivo dist ausente, a saída observável não corresponde a validação falha; a divergência é compatível com esta hipótese: Testar só dentro do próprio workspace e ignorar conteúdo real do pacote.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Testar só dentro do próprio workspace e ignorar conteúdo real do pacote.

Pistas graduais

1. Build em diretório limpo. 2. Inspecione tarball e manifeste exports/types. 3. Teste consumo externo isolado com runtime e tsc.

Solução de referência explicada

Pacote local com API curta, declarations fiéis, exports e script build; fixture de consumidor prova artefato empacotado.

Transferência em outro contexto

—

Evidência de domínio

Resultado observável: npm pack; consumidor instala tarball e importa -> runtime e typecheck passam. Borda a conferir: arquivo dist ausente -> validação falha; exports aponta caminhos reais; licença/dependências declaradas; sem publicar rede. Entrega esperada: Pacote local com API curta, declarations fiéis, exports e script build; fixture de consumidor prova artefato empacotado. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: TypeScript 7.0 strict com exports e types em pacote ESM; testes e exemplo de consumo no Node.js 24 LTS. | Próxima sugestão: 22.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.09 Exports e declarations

MODIFY · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.isolated_declarations | Requer: ts.basic, modules.esm | Introduz: ts.isolated_declarations

Eixos: novidade 4 · sintaxe 4 · composição 4 · estado 2 · depuração 4 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Ative isolatedDeclarations num pacote; anote exports públicos e gere declarations. Project References ficam em extensão opcional.

Objetivo da atividade

Preveja o que fazer quando um export exige anotação para que sua declaration seja emitida.

Explicação e modelo mental

• Modelo mental: isolatedDeclarations exige que a superfície pública seja expressa de forma adequada para gerar declarations isoladas.
• Exemplo resolvido: Anote os exports públicos necessários, use corretamente importações apenas de tipo e confira o .d.ts emitido.
• Armadilha comum: Esperar que a opção produza declarations fiéis para todo export ambíguo sem anotação.
• Por que importa: Declarations publicadas precisam corresponder à API real; Project References não são pré-requisito geral para este exercício.

Material/arquivo inicial

src/index.ts: // 22.09 · Ative isolatedDeclarations (tsconfig.json) e rode npx tsc.
// Cada export cujo tipo depende de inferência vai pedir anotação explícita.
import { normalizar } from "./interno.ts";

export const VERSAO = "1.0.0";

export function criarSlug(texto: string) {
return normalizar(texto).replaceAll(" ", "-");
}

export const padroes = { separador: "-", maximo: 60 };

export function limitar(texto: string, maximo = padroes.maximo) {
return texto.length > maximo ? texto.slice(0, maximo) : texto;
}

export type { Opcoes } from "./interno.ts";
| src/interno.ts: export type Opcoes = { separador: string; maximo: number };

export function normalizar(texto: string): string {
return texto.trim().toLowerCase().normalize("NFD").replace(/\p{M}/gu, "");
}
| tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"verbatimModuleSyntax": true,
"rewriteRelativeImportExtensions": true,
"declaration": true,
"isolatedDeclarations": false,
"rootDir": "src",
"outDir": "dist"
},
"include": ["src"]
}
| package.json: {
"name": "@curso/slug",
"version": "1.0.0",
"type": "module",
"exports": {
".": {
"types": "./dist/index.d.ts",
"default": "./dist/index.js"
}
},
"files": [
"dist"
],
"scripts": {
"build": "tsc"
},
"devDependencies": {
"typescript": "6.0.3"
}
}

| package-lock.json: {
"name": "@curso/slug",
"version": "1.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "@curso/slug",
"version": "1.0.0",
"devDependencies": {
"typescript": "6.0.3"
}
},
"node_modules/typescript": {
"version": "6.0.3",
"resolved": "https://registry.npmjs.org/typescript/-/typescript-6.0.3.tgz",
"integrity":
"sha512-y2TvuxSZPDyQakkFRPZHKFm+KKVqIisdg9/CZwm9ftvKXLP8NRWj38/ODjNbr43SsoXqNuAisEf1GdCxqWcdBw==",
"dev": true,
"license": "Apache-2.0",
"bin": {
"tsc": "bin/tsc",
"tsserver": "bin/tsserver"
},
"engines": {
"node": ">=14.17"
}
}
}
}
| src/index.test.ts: import test from 'node:test';

// Escreva os testes de criarSlug, limitar e VERSAO.
test.todo('slug');

Exemplo e resultado público

tsc com isolatedDeclarations → declarations emitidas fiéis aos exports

Bordas e erros publicados

export sem anotação exigida -> diagnóstico; type-only imports/exports corretos; Project References opcional apenas para monorepo

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como declarações geradas ajudam um consumidor de pacote TypeScript?

• Elas são o código JavaScript executado pelo consumidor no lugar da implementação.
• Elas fazem com que qualquer export seja automaticamente seguro, mesmo que a implementação não cumpra o contrato.
 (Correta) Elas expõem a forma estática da API para que o consumidor possa usar e verificar chamadas sem depender dos tipos internos do autor.

Por que a resposta está certa

Declarations comunicam a interface estática publicada ao projeto consumidor.

Armadilha da alternativa incorreta

Declarações não executam nem provam o comportamento da implementação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

export sem anotação exigida → diagnóstico; type-only imports/exports corretos; Project References opcional apenas para monorepo

Concepção errada e falha observável

Supor que flag gera declarações de API ambígua sem anotações explícitas. / Na entrada export sem anotação exigida, a saída observável não corresponde a diagnóstico; a divergência é compatível com esta hipótese: Supor que flag gera declarações de API ambígua sem anotações explícitas.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Supor que flag gera declarações de API ambígua sem anotações explícitas.

Pistas graduais

1. Ative declaration e isolatedDeclarations. 2. Leia diagnósticos nos exports. 3. Anote tipos públicos mínimos e confira .d.ts emitido.

Solução de referência explicada

Ajuste exports para compilar sob flag, compare declarations geradas com runtime e documente opção de References separada.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: isolatedDeclarations (introduzido no TypeScript 5.5 e presente no 7.0 strict); declarações geradas sem inferência do compilador. | Próxima sugestão: 23.01, 22.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

22.10 Trilha opcional: Project References

MODIFY · TRILHA OPCIONAL · 22 min (estimativa, sem nota por velocidade) · TypeScript 7.0 strict · Node.js 24 LTS · dificuldade multidimensional 3.5/5

Competência: ts.project_references | Requer: ts.basic, modules.esm | Introduz: ts.project_references

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 4 · leitura 3 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Separe biblioteca e consumidor em dois projetos TypeScript composite, configure references e build incremental; compare o grafo e os arquivos gerados.

Objetivo da atividade

Preveja como configurar a dependência entre core e app e em que ordem o build deve ocorrer.

Explicação e modelo mental

• Modelo mental: Project References descrevem dependências entre projetos e permitem que tsc -b coordene builds incrementais.
• Exemplo resolvido: core usa composite e emite declarations; app referencia core; tsc -b packages/app constrói core antes de app.
• Armadilha comum: Confundir paths aliases com referências de build ou criar uma referência cíclica.
• Por que importa: O grafo permite rebuild dos dependentes quando core muda; a trilha é opcional e voltada a projetos maiores ou monorepos.

Material/arquivo inicial

roteiro.md: # Project References (trilha opcional)

1. Em packages/core/tsconfig.json, ligue "composite": true.
2. Em packages/app/tsconfig.json, acrescente "references": [{ "path": "../core" }].
3. Rode npx tsc -b packages/app e anote a ordem: qual projeto compila primeiro e que arquivos aparecem em core/dist (.js, .d.ts) e onde fica o tsconfig.tsbuildinfo.
4. Rode de novo sem mudar nada: o que o build incremental pula?
5. Mude a assinatura de total em core e rode tsc -b packages/app outra vez.
6. Tire composite de core e registre o diagnóstico.
7. Faça core referenciar app também. O que o tsc -b diz sobre referência cíclica?

Quando isto compensa: monorepo com vários pacotes que dependem uns dos outros. Para um pacote só, é complexidade sem retorno.
| packages/core/src/index.ts: // Trilha opcional · core: biblioteca consumida por app via Project References.
export type Item = { sku: string; preco: number };

export function total(itens: readonly Item[]): number {
return itens.reduce((soma, item) => soma + item.preco, 0);
}
| packages/core/tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"rootDir": "src",
"outDir": "dist",
"declaration": true
},
"include": ["src"]
}
| packages/app/src/main.ts: // app consome core pela saída compilada (.d.ts), não pelo código-fonte.
import { total, type Item } from "../../core/dist/index.js";

const carrinho: Item[] = [{ sku: "a", preco: 10 }, { sku: "b", preco: 5 }];
console.log(total(carrinho));
| packages/app/tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"rootDir": "src",
"outDir": "dist"
},
"include": ["src"]
}

Exemplo e resultado público

tsc -b packages/app → core compila antes e app consome .d.ts

Bordas e erros publicados

alterar core -> build incremental recompila dependentes conforme grafo; reference cíclica -> configuração inválida; sem composite -> diagnóstico

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Extensão opcional para bibliotecas ou monorepo; não bloqueia conclusão do núcleo.. Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que deve justificar dividir uma base TypeScript em unidades de build separadas?

 (Correta) Fronteiras de módulos e necessidades reais de manutenção que compensem a configuração adicional.
• A possibilidade de adicionar arquivos de configuração, mesmo sem fronteiras ou ganhos desejados.
• O desejo de substituir todas as formas de teste por comandos de compilação.

Por que a resposta está certa

A decisão deve responder à estrutura e às necessidades do projeto, considerando também seu custo de manutenção.

Armadilha da alternativa incorreta

Mais configuração não é um benefício por si só nem substitui testes do comportamento.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

alterar core → build incremental recompila dependentes conforme grafo; reference cíclica -> configuração inválida; sem composite -> diagnóstico

Concepção errada e falha observável

Usar paths alias como se fornecesse ordem de build e declarations entre projetos. / Na entrada alterar core, a saída observável não corresponde a build incremental recompila dependentes conforme grafo; a divergência é compatível com esta hipótese: Usar paths alias como se fornecesse ordem de build e declarations entre projetos.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Usar paths alias como se fornecesse ordem de build e declarations entre projetos.

Pistas graduais

1. Comece com dois projetos pequenos. 2. Habilite composite/declaration no referenciado. 3. Use references e tsc -b; compare outputs e ordem.

Solução de referência explicada

Extensão opt-in gera core JS/d.ts e app compilado em ordem topológica; registre custo de configuração e quando monorepo justifica a ferramenta.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Project references com composite e build incremental do TypeScript 7.0. | Próxima sugestão: 23.01

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23 Arquitetura, manutenção e capstone

11 atividades · Sequência editorial de implementação e aprendizagem. As pontes anteriores apresentam os conceitos e exemplos antes da tentativa independente.

Leitura e issue de codebase alheia precedem o change request e o capstone.

Ordem: 23.01 · 23.02 · 23.03 · 23.04 · 23.05 · 23.06 · 23.10 · 23.11 · 23.08 · 23.09 · 23.07

23.01 Revisão arquitetural

READ · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: architecture.responsibility | Requer: functions.return, testing.behavior | Introduz: architecture.responsibility

Eixos: novidade 4 · sintaxe 1 · composição 5 · estado 4 · depuração 2 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Receber módulo “god object” e separar responsabilidades em um diagrama de dependências.

Objetivo da atividade

Esboce as responsabilidades do módulo e as dependências entre UI, serviço, domínio, porta de repositório e adapter de storage.

Explicação e modelo mental

• Modelo mental: Separe as regras de domínio dos detalhes de entrada, apresentação e armazenamento. O núcleo depende de portas; adapters implementam essas portas nas bordas.
• Exemplo resolvido: No fluxo público, a UI chama o serviço, o domínio usa uma porta de repositório e o adapter de storage a implementa. Um repositório fake permite testar o domínio sem DOM ou fetch.
• Armadilha comum: Dividir o código em arquivos sem mudar as dependências pode manter ciclos e dependências globais.
• Por que importa: Fronteiras claras tornam possível testar regras e substituir detalhes técnicos sem reescrever o domínio.

Material/arquivo inicial

pedidos.js: // 23.01 · "God object": um módulo que faz tudo. Não edite ainda: leia e desenhe as dependências em diagrama.md.

```
export const Pedidos = {
itens: [],

async carregar() {
const resposta = await fetch("/api/pedidos"); // HTTP
if (!resposta.ok) throw new Error("falha"); // erro sem contexto
this.itens = await resposta.json(); // dado externo sem validação
localStorage.setItem("pedidos", JSON.stringify(this.itens)); // storage
this.render(); // DOM
},

adicionar(form) {
const dados = new FormData(form); // DOM/form
const quantidade = Number(dados.get("quantidade"));
if (!dados.get("produto") || !(quantidade > 0)) { // validação
alert("Pedido inválido"); // UI
return;
}
const total = quantidade * (dados.get("produto") === "caneca" ? 39.9 : 59.9); // regra de domínio
this.itens.push({ produto: dados.get("produto"), quantidade, total });
localStorage.setItem("pedidos", JSON.stringify(this.itens)); // storage de novo
this.render();
},

render() {
document.querySelector("#lista").innerHTML = this.itens // DOM + innerHTML com dado externo
.map((p) => <li>${p.produto} x${p.quantidade}: R$ ${p.total}</li>)
.join("");
},
};
```

| diagrama.md: # Responsabilidades e dependências

1. Marque cada linha de pedidos.js

Responsabilidade	Linhas	Camada de destino
HTTP		adapter HTTP
Validação de entrada		
Regra de preço		domínio
Storage		adapter de storage (implementa a porta)
DOM / formulário		UI

2. Desenhe as setas (quem depende de quem)

``text UI ──> serviço ──> domínio └─────> porta Repositorio <── adapter localStorage Ajuste o desenho acima e responda: - O domínio importa algo de DOM, fetch ou localStorage? - Como testar a regra de preço com um repositório falso, sem navegador? - Quando o fetch falhar, que contexto o erro precisa carregar ao atravessar as camadas (cause`)?

Exemplo e resultado público

diagrama → UI->service->domain/repository port; adapter de storage implementa port

Bordas e erros publicados

domínio com repository fake -> teste sem DOM/fetch; erros atravessam fronteiras com contexto

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que torna um diagrama de dependências útil durante uma revisão arquitetural?

• Listar todos os arquivos sem indicar como se relacionam ou que responsabilidade têm.
 (Correta) Mostrar responsabilidades e relações relevantes entre partes para que outra pessoa consiga avaliar o desenho.
• Substituir a explicação das decisões por uma imagem sem rótulos ou contexto.

Por que a resposta está certa

Um diagrama deve comunicar a estrutura que importa à decisão, não apenas o inventário de arquivos.

Armadilha da alternativa incorreta

Sem relações ou contexto, uma lista visual não ajuda a avaliar a arquitetura.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

domínio com repository fake → teste sem DOM/fetch; erros atravessam fronteiras com contexto

Concepção errada e falha observável

Separar arquivos mas manter dependência circular/global. / Na entrada domínio com repository fake, a saída observável não corresponde a teste sem DOM/fetch; a divergência é compatível com esta hipótese: Separar arquivos mas manter dependência circular/global.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Separar arquivos mas manter dependência circular/global.

Pistas graduais

1. Liste entradas/saídas de cada responsabilidade. 2. Setas apontam de UI para serviço e portas. 3. Adapters ficam nas bordas e domínio não conhece tecnologia.

Solução de referência explicada

Desenhe grafo acíclico e plano de extração com API de cada módulo; indique um teste por fronteira.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Padrões de arquitetura e módulos ES; exemplos em ECMAScript 2026 no Node.js 24 LTS. | Próxima sugestão: 23.02

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.02 Parsons de fluxo

PARSONS · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 2.5/5

Competência: architecture.boundary_flow | Requer: functions.return, testing.behavior | Introduz: architecture.boundary_flow

Eixos: novidade 3 · sintaxe 2 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 2 · ferramentas 2

VISÃO DO ALUNO

Reordenar camadas de entrada -> validação -> domínio -> persistência -> apresentação.

Objetivo da atividade

Ordene as etapas do fluxo e anote onde uma entrada inválida ou uma falha de storage deve seguir um caminho de erro.

Explicação e modelo mental

• Modelo mental: Dados externos devem ser analisados e validados antes de chegar ao domínio. O fluxo válido segue para persistência e apresentação; erros não devem provocar mutação indevida.
• Exemplo resolvido: Para uma requisição válida: entrada, parse e validação, domínio, persistência, resposta à UI. Se a entrada for inválida, o fluxo falha antes de mutar; se o storage falhar, a UI recebe um erro seguro com a causa técnica preservada.
• Armadilha comum: Um tipo TypeScript não valida em execução um JSON externo, e o domínio não deve depender de objetos Response.
• Por que importa: A ordem protege o estado da aplicação e mantém as regras de domínio independentes do formato de transporte.

Material/arquivo inicial

fluxo.js: // 23.02 · Fluxo de uma requisição pelas camadas.
// Ordene os blocos dentro de criarPedido: entrada -> parse -> validação -> domínio -> persistência -> apresentação.

```
class ValidationError extends Error {}

const repositorio = {
itens: [],
async salvar(pedido) {
if (pedido.quantidade > 99) throw new Error("disco cheio (simulado)");
this.itens.push(pedido);
},
};

async function criarPedido(entrada) {
/* Blocos (embaralhados)
[A] const pedido = { produto: dados.produto.trim(), quantidade: dados.quantidade, total: dados.quantidade * 10 };
[B] return { status: 201, corpo: Pedido de ${pedido.produto} criado };
[C] const dados = JSON.parse(entrada);
[D] try { await repositorio.salvar(pedido); } catch (erro) { throw new Error("não foi possível salvar o pedido", { cause: erro }); }
[E] if (typeof dados.produto !== "string" || !Number.isInteger(dados.quantidade) || dados.quantidade < 1) throw new
ValidationError("pedido inválido");
*/
}

async function responder(entrada) {
try {
return await criarPedido(entrada);
} catch (erro) {
if (erro instanceof ValidationError || erro instanceof SyntaxError) return { status: 400, corpo: erro.message };
return { status: 500, corpo: "Erro interno. Tente de novo.", causa: erro.cause?.message };
}
}

console.log(await responder('{"produto":"caneca","quantidade":2}')); // 201
console.log(await responder('{"produto":"caneca","quantidade":0}')); // 400 e repositório intacto
console.log(await responder('{"produto":"caneca","quantidade":100}')); // 500 com causa preservada, mensagem segura
console.log("itens salvos:", repositorio.itens.length); // esperado: 1
```

Exemplo e resultado público

request válido → validação -> domínio -> save -> resposta UI

Bordas e erros publicados

entrada inválida -> erro antes de mutar; storage falha -> erro com cause, UI segura; domínio não lê Response diretamente

Teste público e aprovação visível

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que manter o domínio independente dos formatos específicos usados na entrada e na apresentação?

• Para que o domínio passe a analisar diretamente qualquer formato de requisição.
• Para permitir que a camada de apresentação decida todas as regras de negócio.
 (Correta) Para que regras da aplicação possam ser usadas sem depender de um transporte ou de uma representação de interface específicos.

Por que a resposta está certa

Separar formatos das regras facilita reutilizar e manter o domínio.

Armadilha da alternativa incorreta

Conhecer todos os formatos de entrada ou delegar as regras à UI aumenta o acoplamento.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada inválida → erro antes de mutar; storage falha -> erro com cause, UI segura; domínio não lê Response diretamente

Concepção errada e falha observável

Passar JSON desconhecido direto ao domínio e chamar tipo TS de validação. / Na entrada entrada inválida, a saída observável não corresponde a erro antes de mutar; a divergência é compatível com esta hipótese: Passar JSON desconhecido direto ao domínio e chamar tipo TS de validação.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Passar JSON desconhecido direto ao domínio e chamar tipo TS de validação.

Pistas graduais

1. Borda lê bytes/string. 2. Validação produz dado de domínio. 3. Persistência e apresentação recebem resultado, não controlam regra.

Solução de referência explicada

Ordem entrada, parse/validate, domínio, persistência, apresentação; ramos de erro documentados.

Transferência em outro contexto

—

Evidência de domínio

Avalie ordem/seleção de blocos e comportamento final; não exija ordem entre blocos independentes. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Módulos ES e fluxo de chamadas entre camadas; Error.cause (ES2022, Node.js 16.9+) para encadear erros; Node.js 24 LTS. | Próxima sugestão: 23.03

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.03 Complete o contrato

FADED · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: architecture.public_contract | Requer: functions.return, testing.behavior | Introduz: architecture.public_contract

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 3 · depuração 3 · arquivos 2 · leitura 3 · autonomia 4 · ferramentas 2

VISÃO DO ALUNO

Definir interface pública e invariantes de uma feature antes da implementação.

Objetivo da atividade

Escreva a assinatura de criarTarefa e liste seu resultado esperado, invariantes e comportamentos de erro.

Explicação e modelo mental

• Modelo mental: Um contrato público descreve o que entra, o que sai, quais regras precisam valer e como falhas afetam o estado.
• Exemplo resolvido: Um título válido após trim cria uma tarefa com id e status inicial; título só com espaços causa ValidationError. Prazo inválido falha, e erro de storage preserva a causa sem deixar mutação parcial.
• Armadilha comum: Definir apenas o caso feliz ou começar a implementação antes de decidir os erros deixa o comportamento ambíguo.
• Por que importa: Um contrato explícito guia a implementação e permite verificar comportamento sem depender de detalhes internos.

Material/arquivo inicial

contrato.js: // 23.03 · Contrato de criarTarefa antes da implementação.
// Complete os tipos, as invariantes e os erros nos blocos JSDoc. Ainda não implemente o corpo.

```
/**
* @typedef {___} StatusTarefa
*/

/**
* @typedef {Object} Tarefa
* @property {string} id
* @property {string} titulo invariante: ___
* @property {string} [prazo] formato: ___ ; regra: ___
* @property {StatusTarefa} status status inicial: ___
*/

/**
* @typedef {Object} NovaTarefa
* @property {string} titulo
* @property {string} [prazo]
*/

/**
* @typedef {Object} RepositorioTarefas
* @property {(tarefa: Tarefa) => Promise<void>} salvar
*/

export class ValidationError extends Error {
/**
* @param {keyof NovaTarefa} campo
* @param {string} mensagem
*/
constructor(campo, mensagem) {
super(mensagem);
this.campo = campo;
}
}

/**
* Cria e persiste uma tarefa.
* @param {NovaTarefa} entrada
* @param {RepositorioTarefas} repositorio
* @returns {Promise<___>}
* @throws {ValidationError} quando ___
* @throws {Error} com cause quando ___
* Garantia sobre mutação parcial: ___
*/

// Casos que o contrato precisa responder:
// criarTarefa({ titulo: " Estudar " }) -> ___
// criarTarefa({ titulo: " " }) -> ___
// criarTarefa({ titulo: "A", prazo: "ontem" }) -> ___
// repositório lança ao salvar -> ___
```

Exemplo e resultado público

titulo trim não vazio → tarefa com id/status inicial; API declara retorno

Bordas e erros publicados

titulo só espaços -> ValidationError; prazo inválido -> erro; storage failure -> erro com cause; sem mutação parcial

Teste público e aprovação visível

Avalie somente as decisões retiradas e execute o arquivo completo.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que problema definir uma interface e seus invariantes antes de codificar ajuda a evitar?

 (Correta) Decisões contraditórias sobre o que a feature aceita, promete e considera válido.
• A necessidade de escolher nomes de variáveis antes da implementação.
• A possibilidade de que a solução contenha qualquer detalhe interno de implementação.

Por que a resposta está certa

Um contrato prévio alinha expectativas sobre comportamento observável antes de escolher detalhes de código.

Armadilha da alternativa incorreta

O contrato define expectativas externas; não serve para decidir cada detalhe interno de implementação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

titulo só espaços → ValidationError; prazo inválido -> erro; storage failure -> erro com cause; sem mutação parcial

Concepção errada e falha observável

Começar implementação sem decidir entrada, saída e invariantes observáveis. / Na entrada titulo só espaços, a saída observável não corresponde a ValidationError; a divergência é compatível com esta hipótese: Começar implementação sem decidir entrada, saída e invariantes observáveis.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Começar implementação sem decidir entrada, saída e invariantes observáveis.

Pistas graduais

1. Escreva exemplos positivos/negativos. 2. Defina tipos e regras de domínio. 3. Separe erro de validação de erro de I/O.

Solução de referência explicada

Contrato público inclui assinatura, dados válidos, estado inicial, efeito de persistência, erros e política de atomicidade.

Transferência em outro contexto

—

Evidência de domínio

Avalie somente as decisões retiradas e execute o arquivo completo. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Interfaces e contratos em JSDoc ou em TypeScript 7.0 strict; sem API de runtime nova. | Próxima sugestão: 23.04

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.04 Refactor guiado por testes

MODIFY · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: architecture.refactor_safety | Requer: functions.return, testing.behavior | Introduz: architecture.refactor_safety

Eixos: novidade 3 · sintaxe 3 · composição 5 · estado 4 · depuração 4 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Melhorar código legado em passos mantendo testes verdes.

Objetivo da atividade

Proponha passos pequenos para refatorar o código legado e indique que evidência verificaria cada passo.

Explicação e modelo mental

• Modelo mental: Primeiro registre o comportamento atual; depois altere uma parte por vez e confira as regressões a cada etapa.
• Exemplo resolvido: Com validação duplicada e I/O misturado, caracterize os resultados e erros existentes, extraia uma função pequena e rode a suíte antes de avançar para outra fronteira.
• Armadilha comum: Uma reescrita grande seguida de um teste feliz não demonstra que os outros comportamentos foram preservados.
• Por que importa: Passos verificáveis ajudam a localizar regressões e a distinguir mudanças intencionais das acidentais.

Material/arquivo inicial

legado.js: // 23.04 · Código legado: funciona, mas validação duplicada e I/O misturado com regra.
// Refatore em passos pequenos; rode legado.test.js depois de CADA passo.
// disco simula o sistema de arquivos para que o exemplo rode em qualquer lugar.

```
export const disco = { arquivos: new Map(), escritas: 0 };
function escrever(caminho, texto) { disco.escritas++; disco.arquivos.set(caminho, texto); }
function ler(caminho) { return disco.arquivos.get(caminho); }

export function cadastrarCliente(nome, email, idade) {
if (nome == null || String(nome).trim() == "") {
return { ok: false, erro: "nome obrigatório" };
}
if (email == null || String(email).indexOf("@") == -1) {
return { ok: false, erro: "email inválido" };
}
if (idade == null || isNaN(idade) || Number(idade) < 18) {
return { ok: false, erro: "idade mínima 18" };
}
var clientes = JSON.parse(ler("clientes.json") || "[]");
for (var i = 0; i < clientes.length; i++) {
if (clientes[i].email == String(email).trim().toLowerCase()) {
return { ok: false, erro: "email já cadastrado" };
}
}
var cliente = {
id: clientes.length + 1,
nome: String(nome).trim(),
email: String(email).trim().toLowerCase(),
idade: Number(idade),
};
clientes.push(cliente);
escrever("clientes.json", JSON.stringify(clientes));
escrever("log.txt", (ler("log.txt") || "") + "cadastro " + cliente.email + "\n");
return { ok: true, cliente: cliente };
}

export function atualizarCliente(id, nome, email, idade) {
if (nome == null || String(nome).trim() == "") {
return { ok: false, erro: "nome obrigatório" };
}
if (email == null || String(email).indexOf("@") == -1) {
return { ok: false, erro: "email inválido" };
}
if (idade == null || isNaN(idade) || Number(idade) < 18) {
return { ok: false, erro: "idade mínima 18" };
}
var clientes = JSON.parse(ler("clientes.json") || "[]");
var achou = false;
for (var i = 0; i < clientes.length; i++) {
if (clientes[i].id == id) {
clientes[i].nome = String(nome).trim();
clientes[i].email = String(email).trim().toLowerCase();
clientes[i].idade = Number(idade);
achou = true;
}
}
```

```
if (!achou) return { ok: false, erro: "cliente não encontrado" };
escrever("clientes.json", JSON.stringify(clientes));
escrever("log.txt", (ler("log.txt") || "") + "atualização " + id + "\n");
return { ok: true };
}

// Passos sugeridos (um por vez, testes verdes entre eles):
// 1. Extrair validarDadosCliente(nome, email, idade) usada pelas duas funções.
// 2. Separar a regra (puro) do I/O (ler/escrever) com um repositório injetável.
// 3. Trocar var/== por const/let/===, documentando qualquer mudança de comportamento intencional.
| legado.test.js: // Suíte existente: node --test legado.test.js. Ela precisa continuar verde a cada passo.
import { test, beforeEach } from "node:test";
import assert from "node:assert/strict";
import { cadastrarCliente, atualizarCliente, disco } from "./legado.js";

beforeEach(() => {
disco.arquivos.clear();
disco.escritas = 0;
});

test("cadastra e normaliza", () => {
const r = cadastrarCliente(" Ana ", " ANA@EX.COM ", "30");
assert.deepEqual(r, { ok: true, cliente: { id: 1, nome: "Ana", email: "ana@ex.com", idade: 30 } });
});

test("entrada inválida não escreve nada", () => {
assert.deepEqual(cadastrarCliente("", "a@b", 20), { ok: false, erro: "nome obrigatório" });
assert.deepEqual(cadastrarCliente("A", "sem-arroba", 20), { ok: false, erro: "email inválido" });
assert.deepEqual(cadastrarCliente("A", "a@b", 17), { ok: false, erro: "idade mínima 18" });
assert.equal(disco.escritas, 0);
});

test("email duplicado", () => {
cadastrarCliente("Ana", "a@b", 20);
assert.deepEqual(cadastrarCliente("Bia", "A@B", 20), { ok: false, erro: "email já cadastrado" });
});

test("atualiza existente e recusa ausente", () => {
cadastrarCliente("Ana", "a@b", 20);
assert.deepEqual(atualizarCliente(1, "Ana Maria", "a@b", 21), { ok: true });
assert.deepEqual(atualizarCliente(9, "X", "x@y", 30), { ok: false, erro: "cliente não encontrado" });
});
```

Exemplo e resultado público

antes/depois → mesmos resultados e erros nos casos publicados

Bordas e erros publicados

entrada inválida -> sem escrita; cada passo de extração mantém suíte; diff de comportamento intencional documentado

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que mudar uma parte por vez ao refatorar uma base existente?

• Para evitar qualquer necessidade de compreender o comportamento existente.
 (Correta) Para tornar mais fácil relacionar cada alteração ao efeito que ela introduz e revisar seu escopo.

• Para permitir que cada passo altere várias responsabilidades sem uma comparação intermediária.

Por que a resposta está certa

Passos pequenos ajudam a entender a causa de mudanças e a manter a revisão focada.

Armadilha da alternativa incorreta

Refatorar incrementalmente ainda exige entender o código e controlar o alcance de cada alteração.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

entrada inválida → sem escrita; cada passo de extração mantém suíte; diff de comportamento intencional documentado

Concepção errada e falha observável

Fazer reescrita grande e dizer que equivalência é óbvia. / Na entrada entrada inválida, a saída observável não corresponde a sem escrita; a divergência é compatível com esta hipótese: Fazer reescrita grande e dizer que equivalência é óbvia.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Fazer reescrita grande e dizer que equivalência é óbvia.

Pistas graduais

1. Congele caracterização com testes. 2. Extraia função pura pequena. 3. Rode regressões a cada passo e compare saída.

Solução de referência explicada

Plano em commits lógicos de extração, validação, adaptação de I/O; evidência de testes por passo e lista de mudanças intencionais.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: node:test (nativo no Node.js 24 LTS) para regressão durante a refatoração. | Próxima sugestão: 23.05

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.05 Revisão de PR simulada

READ · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: architecture.pr_review | Requer: functions.return, testing.behavior | Introduz: architecture.pr_review

Eixos: novidade 3 · sintaxe 3 · composição 4 · estado 4 · depuração 4 · arquivos 3 · leitura 5 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Analisar diff com bugs de async, tipos frouxos, mutação inesperada e teste frágil.

Objetivo da atividade

Registre possíveis achados do diff com localização, impacto e evidência ou correção mínima.

Explicação e modelo mental

• Modelo mental: Uma revisão útil conecta cada comentário a um comportamento verificável e prioriza risco sem transformar preferência em defeito.
• Exemplo resolvido: No diff descrito, investigue se uma rejeição async escapa do catch, se any enfraquece o contrato do payload, se sort altera a entrada e se um timer real torna o teste instável.
• Armadilha comum: A compilação não prova ausência de bugs; uma suspeita sem evidência também não basta para afirmar um achado.
• Por que importa: Comentários reproduzíveis ajudam a corrigir problemas reais com mudanças localizadas.

Material/arquivo inicial

pr.diff: PR #318 · "Ranking de vendedores com retry"

```
--- a/src/ranking.ts
+++ b/src/ranking.ts
@@ -1,22 +1,28 @@
import { buscarVendas } from "./api.js";

-export async function carregarRanking(): Promise<Vendedor[]> {
- const vendas = await buscarVendas();
- return ordenar(vendas);
+export async function carregarRanking(): Promise<Vendedor[]> {
+ try {
+ const vendas: any = await buscarVendas();
+ return ordenar(vendas.itens);
+ } catch (erro) {
+ return tentarDeNovo();
+ }
}

-function ordenar(vendedores: readonly Vendedor[]): Vendedor[] {
- return vendedores.toSorted((a, b) => b.total - a.total);
+function ordenar(vendedores: Vendedor[]): Vendedor[] {
+ return vendedores.sort((a, b) => b.total - a.total);
}
+
+async function tentarDeNovo(): Promise<Vendedor[]> {
+ await new Promise((r) => setTimeout(r, 1000));
+ return carregarRanking();
+}

--- a/test/ranking.test.ts
+++ b/test/ranking.test.ts
@@ -10,6 +10,12 @@
+test("tenta de novo depois de falha", async () => {
+ falharProximaChamada();
+ const inicio = Date.now();
+ await carregarRanking();
+ assert.ok(Date.now() - inicio >= 1000);
+});
| revisao.md: # Revisão do PR #318
```

Registre só achados com evidência. Para cada um: arquivo e linha do diff, impacto para o usuário, como reproduzir e a correção mínima.

#	Local	Achado	Impacto	Reprodução / evidência	Correção mínima
1					
2					
3					
4					

Pistas para investigar (não são achados até você provar): retry sem limite dentro do catch; any no payload; ordenação que muta a entrada; teste que depende de tempo real.

Comentário que você pensou em fazer mas descartou por falta de evidência (ou que um teste refutou):

Exemplo e resultado público

revisão → quatro achados com linha, impacto, reprodução/correção mínima

Bordas e erros publicados

comentário sem evidência -> não vira achado; falso positivo refutado com teste; async tem rejeição reproduzida

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Como priorizar comentários durante uma revisão quando há pouco tempo disponível?

• Começar por preferências de formatação, independentemente do efeito da mudança.
• Dar prioridade igual a toda observação, sem considerar impacto ou alcance.
 (Correta) Começar pelos riscos que podem afetar mais usuários ou comprometer comportamentos importantes.

Por que a resposta está certa

Priorizar impacto ajuda a concentrar a revisão nos riscos mais relevantes.

Armadilha da alternativa incorreta

Preferências visuais ou uma lista sem prioridade não ajudam a direcionar a revisão pelo risco.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

comentário sem evidência → não vira achado; falso positivo refutado com teste; async tem rejeição reproduzida

Concepção errada e falha observável

Aprovar diff só por compilar ou pedir reescrita sem prova. / Na entrada comentário sem evidência, a saída observável não corresponde a não vira achado; a divergência é compatível com esta hipótese: Aprovar diff só por compilar ou pedir reescrita sem prova.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Aprovar diff só por compilar ou pedir reescrita sem prova.

Pistas graduais

1. Leia comportamento alterado e contratos. 2. Reproduza cada suspeita com caso mínimo. 3. Priorize risco e proponha correção localizada.

Solução de referência explicada

Quatro comentários verificáveis, cada um com severidade, evidência e sugestão; destaque testes necessários para async/mutação.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Leitura de diff e de código assíncrono; toSorted (ES2023, Node.js 20+) no código revisado; Node.js 24 LTS. | Próxima sugestão: 23.06

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.06 Boundary audit

READ · 10 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: architecture.trust_boundaries | Requer: functions.return, testing.behavior | Introduz: architecture.trust_boundaries

Eixos: novidade 4 · sintaxe 2 · composição 5 · estado 4 · depuração 3 · arquivos 4 · leitura 5 · autonomia 4 · ferramentas 3

VISÃO DO ALUNO

Mapear todas as entradas externas do capstone: HTTP, storage, DOM/form, env e arquivos; marcar onde cada uma vira dado de dominio validado.

Objetivo da atividade

Liste as fontes externas indicadas e, para cada uma, marque onde os dados são analisados, validados e convertidos em dados de domínio.

Explicação e modelo mental

• Modelo mental: Cada fronteira tem dados brutos e precisa de uma política explícita antes que esses dados sejam confiáveis para o domínio.
• Exemplo resolvido: HTTP JSON unknown passa por validação antes de virar DomainInput; storage passa por parse, migração e validação antes de virar DomainState. Arquivo inválido deve preservar a causa no erro, segredos do ambiente devem ser redigidos e texto do DOM deve ser renderizado com segurança.
• Armadilha comum: Um cast ou uma anotação de tipo não verifica a forma nem o conteúdo do dado em execução.
• Por que importa: Mapear todas as entradas reduz falhas e evita que dados não confiáveis atravessem silenciosamente para as regras de negócio.

Material/arquivo inicial

bordas.md: # Auditoria de bordas do capstone

Toda entrada externa chega como unknown (ou texto) e só vira dado de domínio depois de validada. Preencha uma linha por fonte.

Fonte	Onde chega no código	Formato bruto	Onde é analisada (parse)	Onde é validada	Tipo de domínio resultante	Política de falha
HTTP (resposta JSON)	api/cliente.ts	unknown			DomainInput	
Storage (localStorage / arquivo de save)	storage/adapter.ts	string		migrar versão -> validar	DomainState	
DOM / FormData	ui/formulario.ts	strings				
Variáveis de ambiente	config.ts	string | undefined				segredo nunca vai para log
Arquivo importado pelo usuário	importar.ts	bytes/texto				erro com cause

Perguntas:

1. Qual fonte você ainda trata como confiável sem perceber?
2. Onde o texto vindo de fora é exibido na tela? Ele usa textContent?
3. Qual validação é compartilhada entre duas fontes e onde ela deve morar?

Exemplo e resultado público

HTTP JSON unknown → validate -> DomainInput; storage parse unknown -> migrate/validate -> DomainState

Bordas e erros publicados

env segredo -> validação e redação em logs; arquivo inválido -> erro com cause; DOM texto -> textContent; cada fonte tem política

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Que pergunta central orienta o inventário de fronteiras de confiança de uma aplicação?

 (Correta) Quais dados cruzam de uma origem para uma parte que opera sob outras suposições de confiança?
• Quais módulos têm nomes mais longos na árvore de arquivos?

• Quais valores já possuem uma anotação TypeScript, independentemente de sua origem?

Por que a resposta está certa

O inventário começa identificando cruzamentos entre origens e os pressupostos de confiança envolvidos.

Armadilha da alternativa incorreta

Nomes de módulos e anotações estáticas não identificam, por si, fronteiras de confiança.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

env segredo → validação e redação em logs; arquivo inválido -> erro com cause; DOM texto -> textContent; cada fonte tem política

Concepção errada e falha observável

Chamar dado externo de tipado apenas com cast. / Na entrada env segredo, a saída observável não corresponde a validação e redação em logs; a divergência é compatível com esta hipótese: Chamar dado externo de tipado apenas com cast.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Chamar dado externo de tipado apenas com cast.

Pistas graduais

1. Desenhe cada fonte externa. 2. Associe parser/validator e erro público. 3. Marque ponto em que dado vira domínio confiável.

Solução de referência explicada

Tabela de cinco fontes com formato bruto, validação, transformação, erro/observabilidade e testes; nenhum caminho pula a fronteira.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Fronteiras de entrada: process.env, fetch e formulários do DOM, em Node.js 24 LTS e navegadores evergreen. | Próxima sugestão: 23.10

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.10 Encontre a regra no mini-repositório

READ · 25 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.0/5

Competência: codebase.navigation | Requer: functions.return, testing.behavior | Introduz: codebase.navigation

Eixos: novidade 4 · sintaxe 1 · composição 3 · estado 2 · depuração 3 · arquivos 4 · leitura 5 · autonomia 3 · ferramentas 3

VISÃO DO ALUNO

Receba 10 arquivos, README e testes; localize a origem de uma regra, seu caminho até a UI e o melhor ponto para uma mudança pequena.

Objetivo da atividade

Descreva o caminho da UI até a regra de limite e indique que README, teste ou execução apoiaria sua localização.

Explicação e modelo mental

• Modelo mental: Para encontrar a origem de uma regra, siga o fluxo até a decisão e use testes como evidência; nomes de arquivos, sozinhos, não identificam o dono.
• Exemplo resolvido: No mini-repositório, localize a função de domínio que aplica o limite, trace como a UI chega até ela e confira o teste correspondente antes de propor uma mudança.
• Armadilha comum: Editar o primeiro módulo com nome parecido pode alterar apenas a formatação, sem tocar na regra.
• Por que importa: Entender a codebase antes de mudar reduz alterações no módulo errado e ajuda a justificar uma intervenção pequena.

Material/arquivo inicial
README.md: # relatorio-pedidos

Gera o relatório de pedidos dentro e fora do limite de valor.

- src/main.js: monta o relatório de exemplo (rode node src/main.js).
- src/ui/: apresentação em texto.
- src/service/: casos de uso.
- src/domain/: regras de negócio, sem I/O.
- src/storage/: repositórios.
- test/: node --test (encontra os arquivos *.test.js).

Leia ISSUE.md antes de abrir qualquer arquivo de código.
| ISSUE.md: # Issue #42 · Limite padrão do relatório passa a R$ 150

O financeiro pediu que o limite padrão suba de R$ 100,00 para R$ 150,00.

Antes de mudar, responda:

1. Em que arquivo e função a regra "dentro do limite" é aplicada? Qual é o dono do valor padrão?
2. Qual é o caminho de chamada da UI (src/ui/tabela.js) até essa regra?
3. Que teste já cobre a regra e serviria de evidência para a mudança?
4. src/ui/formatar-limite.js tem um nome parecido. Ele é dono da regra? Por quê?
5. Qual é o menor diff que resolve a issue?
| src/main.js: import { criarRepositorioEmMemoria } from "./storage/memoria.js";
import { gerarRelatorio } from "./service/relatorio.js";
import { renderizarTabela } from "./ui/tabela.js";

const repositorio = criarRepositorioEmMemoria([
{ id: "p1", valorCentavos: 9_999 },
{ id: "p2", valorCentavos: 10_000 },
{ id: "p3", valorCentavos: 10_001 },
{ id: "p4", valorCentavos: 15_000 },
]);

console.log(renderizarTabela(gerarRelatorio(repositorio)));
| src/ui/tabela.js: import { formatarLimite } from "./formatar-limite.js";

export function renderizarTabela(relatorio) {
return [
Relatório (${formatarLimite(relatorio.limiteCentavos)}),
Dentro: ${relatorio.dentro.join(", ") || "nenhum"},
Fora: ${relatorio.fora.join(", ") || "nenhum"},
Total: ${relatorio.total},
].join("\n");
}
| src/ui/formatar-limite.js: // Só apresentação: transforma centavos em texto para o cabeçalho.
const moeda = new Intl.NumberFormat("pt-BR", { style: "currency", currency: "BRL" });

export function formatarLimite(limiteCentavos) {
return limite: até ${moeda.format(limiteCentavos / 100)};
}
| src/service/relatorio.js: import { LIMITE_PADRAO_CENTAVOS, dentroDoLimite } from "../domain/limite.js";

export function gerarRelatorio(repositorio, limiteCentavos = LIMITE_PADRAO_CENTAVOS) {
const pedidos = repositorio.listar();
const dentro = [];

```
const fora = [];
for (const pedido of pedidos) {
(dentroDoLimite(pedido.valorCentavos, limiteCentavos) ? dentro : fora).push(pedido.id);
}
return { limiteCentavos, total: pedidos.length, dentro, fora };
}
| src/domain/limite.js: // Regra de negócio: o pedido está dentro do limite quando o valor é menor ou igual ao limite.
export const LIMITE_PADRAO_CENTAVOS = 10_000;

export function dentroDoLimite(valorCentavos, limiteCentavos = LIMITE_PADRAO_CENTAVOS) {
if (!Number.isSafeInteger(valorCentavos) || !Number.isSafeInteger(limiteCentavos)) {
throw new RangeError("valores em centavos inteiros");
}
return valorCentavos <= limiteCentavos;
}
| src/storage/memoria.js: // Repositório em memória: devolve cópias para que quem chama não altere o estado guardado.
export function criarRepositorioEmMemoria(pedidosIniciais = []) {
const pedidos = pedidosIniciais.map((pedido) => ({ ...pedido }));
return {
listar: () => pedidos.map((pedido) => ({ ...pedido })),
salvar(pedido) {
pedidos.push({ ...pedido });
},
};
}
| test/limite.test.js: import { test } from "node:test";
import assert from "node:assert/strict";
import { LIMITE_PADRAO_CENTAVOS, dentroDoLimite } from "../src/domain/limite.js";

test("limite padrão", () => {
assert.equal(LIMITE_PADRAO_CENTAVOS, 10_000);
});

test("abaixo, no limite e acima", () => {
assert.equal(dentroDoLimite(9_999), true);
assert.equal(dentroDoLimite(10_000), true);
assert.equal(dentroDoLimite(10_001), false);
});

test("rejeita valor não inteiro", () => {
assert.throws(() => dentroDoLimite(10.5), RangeError);
});
| test/relatorio.test.js: import { test } from "node:test";
import assert from "node:assert/strict";
import { gerarRelatorio } from "../src/service/relatorio.js";
import { criarRepositorioEmMemoria } from "../src/storage/memoria.js";

test("separa pedidos pelo limite informado", () => {
const repositorio = criarRepositorioEmMemoria([
{ id: "a", valorCentavos: 500 },
{ id: "b", valorCentavos: 2_000 },
]);
assert.deepEqual(gerarRelatorio(repositorio, 1_000), {
limiteCentavos: 1_000,
total: 2,
dentro: ["a"],
```

fora: ["b"],
});
});

Exemplo e resultado público

localização → função de domínio que aplica limite e caminho de chamada até UI

Bordas e erros publicados

teste correspondente -> encontrado e usado como evidência; módulo com nome parecido mas só formata texto -> não é dono da regra

Teste público e aprovação visível

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Qual deve ser o primeiro objetivo ao se orientar em um repositório desconhecido?

• Editar imediatamente o primeiro arquivo cujo nome pareça relacionado à solicitação.
 (Correta) Entender o propósito do projeto, sua estrutura e os caminhos gerais de execução antes de escolher onde agir.
• Renomear a árvore de diretórios para que corresponda à arquitetura que se espera encontrar.

Por que a resposta está certa

Uma orientação inicial reduz suposições e ajuda a navegar a base com contexto.

Armadilha da alternativa incorreta

Editar ou reorganizar antes de compreender a base pode levar a mudanças sem relação com a solicitação.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

teste correspondente → encontrado e usado como evidência; módulo com nome parecido mas só formata texto -> não é dono da regra

Concepção errada e falha observável

Editar primeiro arquivo com palavra parecida sem traçar fluxo. / Na entrada teste correspondente, a saída observável não corresponde a encontrado e usado como evidência; a divergência é compatível com esta hipótese: Editar primeiro arquivo com palavra parecida sem traçar fluxo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Editar primeiro arquivo com palavra parecida sem traçar fluxo.

Pistas graduais

1. Leia README e árvore antes de busca. 2. Siga chamada de entrada até decisão e persistência. 3. Confirme hipótese no teste e em execução curta.

Solução de referência explicada

Entregue mapa de arquivos/fluxo e linha de decisão; sem editar neste item READ.

Transferência em outro contexto

—

Evidência de domínio

Rubrica de localização, fluxo de dados e intenção; nenhuma edição de código é exigida. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: node:test e módulos ES em mini-repositório; Node.js 24 LTS. | Próxima sugestão: 23.11

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.11 Corrija uma issue alheia

MODIFY · 40 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 3.5/5

Competência: maintenance.foreign_issue | Requer: functions.return, testing.behavior | Introduz: maintenance.foreign_issue

Eixos: novidade 3 · sintaxe 2 · composição 3 · estado 3 · depuração 4 · arquivos 4 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Em um mini-repositório desconhecido com issue e testes existentes, reproduza o bug, mude o módulo correto e preserve o resto.

Objetivo da atividade

Preveja o resultado abaixo, exatamente no limite e acima dele; indique o teste que reproduziria a falha.

Explicação e modelo mental

• Modelo mental: Uma correção segura reproduz primeiro o defeito, altera a regra no módulo dono e verifica casos vizinhos para preservar regressões.
• Exemplo resolvido: Se o contrato inclui o limite superior, um teste exatamente nesse valor deve falhar com a comparação atual. Ajustar a regra de < para <= no domínio inclui esse valor; testes abaixo e acima verificam os limites adjacentes.
• Armadilha comum: Mudar apenas a UI pode esconder o sintoma enquanto o serviço continua aplicando a regra errada.
• Por que importa: Uma regressão localizada demonstra que a issue foi corrigida sem reescrever partes não relacionadas.

Material/arquivo inicial
README.md: # relatorio-entregas

Relatório de entregas dentro e fora do raio atendido pela loja.

- src/main.js: relatório de exemplo (node src/main.js).
- src/ui/: apresentação em texto.
- src/service/: casos de uso.
- src/domain/: regras de negócio, sem I/O.
- src/storage/: repositórios.
- test/: node --test (encontra os arquivos *.test.js).

Issues abertas: ISSUE.md.
| ISSUE.md: # Issue #87 · Entrega exatamente no raio aparece como "fora"

Relato do suporte: a loja atende até 5 km. Um cliente a exatamente 5.000 m teve a entrega
listada como "fora do raio" no relatório. Clientes a 4.999 m e 5.001 m aparecem corretamente.

Tarefa:

1. Escreva primeiro um teste que reproduza o caso de 5.000 m e veja-o falhar.
2. Encontre o módulo dono da regra e corrija só ele.
3. Mantenha os testes de abaixo e acima do raio passando.
4. Por que mudar apenas a UI (por exemplo, o texto do cabeçalho) não resolve?
| src/main.js: import { criarRepositorioEmMemoria } from "./storage/memoria.js";
import { gerarRelatorio } from "./service/relatorio.js";
import { renderizarTabela } from "./ui/tabela.js";

const repositorio = criarRepositorioEmMemoria([
{ id: "e1", distanciaMetros: 4_999 },
{ id: "e2", distanciaMetros: 5_000 },
{ id: "e3", distanciaMetros: 5_001 },
]);

console.log(renderizarTabela(gerarRelatorio(repositorio)));
| src/ui/tabela.js: import { formatarRaio } from "./formatar-raio.js";

export function renderizarTabela(relatorio) {
return [
Entregas (${formatarRaio(relatorio.raioMetros)}),
Dentro: ${relatorio.dentro.join(", ") || "nenhuma"},
Fora: ${relatorio.fora.join(", ") || "nenhuma"},
].join("\n");
}
| src/ui/formatar-raio.js: // Só apresentação: transforma metros em texto para o cabeçalho.
export function formatarRaio(raioMetros) {
return raio: até ${(raioMetros / 1000).toLocaleString("pt-BR")} km;
}
| src/service/relatorio.js: import { RAIO_PADRAO_METROS, dentroDoRaio } from "../domain/raio.js";

export function gerarRelatorio(repositorio, raioMetros = RAIO_PADRAO_METROS) {
const dentro = [];
const fora = [];
for (const entrega of repositorio.listar()) {
(dentroDoRaio(entrega.distanciaMetros, raioMetros) ? dentro : fora).push(entrega.id);
}
return { raioMetros, dentro, fora };

```
}
| src/domain/raio.js: // Regra de negócio: a loja atende entregas até o raio, inclusive.
export const RAIO_PADRAO_METROS = 5_000;

export function dentroDoRaio(distanciaMetros, raioMetros = RAIO_PADRAO_METROS) {
if (!Number.isFinite(distanciaMetros) || distanciaMetros < 0) {
throw new RangeError("distância inválida");
}
return distanciaMetros < raioMetros;
}
| src/storage/memoria.js: // Repositório em memória: devolve cópias para que quem chama não altere o estado guardado.
export function criarRepositorioEmMemoria(iniciais = []) {
const entregas = iniciais.map((entrega) => ({ ...entrega }));
return {
listar: () => entregas.map((entrega) => ({ ...entrega })),
salvar(entrega) {
entregas.push({ ...entrega });
},
};
}
| test/raio.test.js: import { test } from "node:test";
import assert from "node:assert/strict";
import { dentroDoRaio } from "../src/domain/raio.js";

test("abaixo e acima do raio", () => {
assert.equal(dentroDoRaio(4_999), true);
assert.equal(dentroDoRaio(5_001), false);
});

test("rejeita distância negativa", () => {
assert.throws(() => dentroDoRaio(-1), RangeError);
});
| test/relatorio.test.js: import { test } from "node:test";
import assert from "node:assert/strict";
import { gerarRelatorio } from "../src/service/relatorio.js";
import { criarRepositorioEmMemoria } from "../src/storage/memoria.js";

test("separa entregas pelo raio informado", () => {
const repositorio = criarRepositorioEmMemoria([
{ id: "a", distanciaMetros: 100 },
{ id: "b", distanciaMetros: 9_000 },
]);
assert.deepEqual(gerarRelatorio(repositorio, 1_000), { raioMetros: 1_000, dentro: ["a"], fora: ["b"] });
});
```

Exemplo e resultado público

reproduzir com input no limite → teste vermelho; corrigir comparação no módulo dono -> teste verde

Bordas e erros publicados

valores abaixo/acima do limite -> regressões preservadas; mudança só na UI -> rejeitada porque serviço continua errado; diff localizado

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que manter um patch de manutenção focado na issue relatada?

• Para aproveitar a issue como oportunidade de reescrever módulos adjacentes sem justificativa.
• Para evitar explicar como a alteração se relaciona com o problema relatado.
 (Correta) Para facilitar revisão e atribuir efeitos ao conserto sem misturá-lo com limpeza não relacionada.

Por que a resposta está certa

Um diff focado torna mais clara a relação entre o problema e a mudança proposta.

Armadilha da alternativa incorreta

Mudanças não relacionadas dificultam revisar o patch e entender seus efeitos.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

valores abaixo/acima do limite → regressões preservadas; mudança só na UI -> rejeitada porque serviço continua errado; diff localizado

Concepção errada e falha observável

Reescrever app inteiro ou corrigir apresentação sem alterar regra. / Na entrada valores abaixo/acima do limite, a saída observável não corresponde a regressões preservadas; a divergência é compatível com esta hipótese: Reescrever app inteiro ou corrigir apresentação sem alterar regra.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Reescrever app inteiro ou corrigir apresentação sem alterar regra.

Pistas graduais

1. Use mapa do item 23.10. 2. Adicione caso de regressão antes da correção. 3. Corrija regra na camada dona e rode suíte existente.

Solução de referência explicada

Mudança pequena no domínio altera < para <= conforme contrato da issue, sem mexer em módulos alheios; diff e justificativa anexados.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: node:test e módulos ES em mini-repositório com issue; Node.js 24 LTS. | Próxima sugestão: 23.08

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.08 Capstone - sistema completo

PROJECT · 360 min (estimativa, sem nota por velocidade) · Node.js 24 LTS e navegadores evergreen · dificuldade multidimensional 5.0/5

Competência: architecture.capstone | Requer: functions.return, testing.behavior | Introduz: architecture.capstone

Eixos: novidade 5 · sintaxe 4 · composição 5 · estado 5 · depuração 5 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 5

VISÃO DO ALUNO

Construir aplicação ou biblioteca de porte médio com JS/TS, API interna tipada, persistência substituível, testes e documentação.

Objetivo da atividade

Esboce o MVP escolhido e indique como domínio, persistência, interface, API tipada, testes e documentação se relacionam.

Explicação e modelo mental

• Modelo mental: O capstone demonstra um sistema coerente por meio de comportamento observável, fronteiras justificáveis, persistência substituível e testes.
• Exemplo resolvido: Em um app de tarefas, criar, listar, mover e remover itens deve manter estado e persistência consistentes. Entrada inválida é rejeitada antes de mutar, e a recuperação após reinício deve ser considerada.
• Armadilha comum: Uma interface visualmente completa não substitui contrato de dados, typecheck, testes ou comportamento após falhas.
• Por que importa: Um projeto de porte médio mostra como as decisões de arquitetura sustentam funcionalidades reais e mudanças futuras.

Material/arquivo inicial

README.md: # Capstone · tarefas (ou pedidos)

Escolha: app de tarefas (este esqueleto) ou biblioteca de pedidos (mesma estrutura).

Contrato
- Domínio puro em src/dominio.ts: criar, mover e remover; dado inválido é rejeitado antes de mutar.
- Porta Repositorio em src/portas.ts; adapters trocáveis (memória já existe; crie arquivo ou localStorage).
- Falha de storage vira erro público sem segredo, com cause técnico para o log.
- Reinício: o estado salvo é recuperado (com migração de versão se mudar o formato).
- UI ou CLI só chama o serviço; nunca o storage.

Rodar
npm install, npm run typecheck, npm test.

Documente aqui
Decisões, limites conhecidos e como trocar o adapter.
| src/dominio.ts: export type Coluna = "a-fazer" | "fazendo" | "feito";
export type Tarefa = { readonly id: string; titulo: string; coluna: Coluna };
export type Estado = { versao: 1; tarefas: readonly Tarefa[] };

export class ValidationError extends Error {}

export const estadoInicial: Estado = { versao: 1, tarefas: [] };

export function criar(estado: Estado, id: string, titulo: string): Estado {
const limpo = titulo.trim();
if (limpo === "") throw new ValidationError("título obrigatório");
return { ...estado, tarefas: [...estado.tarefas, { id, titulo: limpo, coluna: "a-fazer" }] };
}

export function mover(estado: Estado, id: string, coluna: Coluna): Estado {
// implemente: id inexistente -> ValidationError; não mute estado
}

export function remover(estado: Estado, id: string): Estado {
// implemente
}
| src/portas.ts: import type { Estado } from "./dominio.ts";

// Porta: o que o serviço precisa do armazenamento, sem saber onde os dados ficam.
export interface Repositorio {
carregar(): Promise<Estado>;
salvar(estado: Estado): Promise<void>;
}

export class StorageError extends Error {
constructor(mensagemPublica: string, options?: { cause?: unknown }) {
super(mensagemPublica, options);
}
}
| src/adapters/memoria.ts: import { estadoInicial, type Estado } from "../dominio.ts";
import type { Repositorio } from "../portas.ts";

export function criarRepositorioEmMemoria(inicial: Estado = estadoInicial): Repositorio & { falharProximo(): void } {
let salvo = structuredClone(inicial);
let falhar = false;

```
return {
async carregar() {
return structuredClone(salvo);
},
async salvar(estado) {
if (falhar) {
falhar = false;
throw new Error("disco indisponível (simulado)");
}
salvo = structuredClone(estado);
},
falharProximo() {
falhar = true;
},
};
}
| test/contrato.test.ts: // Teste de contrato: todo adapter de Repositorio deve passar nesta mesma suíte.
import { test } from "node:test";
import assert from "node:assert/strict";
import { criar, estadoInicial } from "../src/dominio.ts";
import { criarRepositorioEmMemoria } from "../src/adapters/memoria.ts";

const adapters = { memoria: () => criarRepositorioEmMemoria() };

for (const [nome, fabricar] of Object.entries(adapters)) {
test(${nome}: salva e recupera (simula reinício), async () => {
const repositorio = fabricar();
await repositorio.salvar(criar(estadoInicial, "t1", "Estudar"));
const recuperado = await repositorio.carregar();
assert.equal(recuperado.tarefas[0]?.titulo, "Estudar");
});
}

test("dado inválido é rejeitado antes de mutar", () => {
assert.throws(() => criar(estadoInicial, "t1", " "));
assert.equal(estadoInicial.tarefas.length, 0);
});

// Acrescente: mover/remover, falha de storage (falharProximo) e o serviço que liga domínio e porta.
| package.json: {
"name": "devpath-23-08",
"version": "1.0.0",
"private": true,
"type": "module",
"engines": {
"node": ">=24.0.0"
},
"scripts": {
"test": "node --test \"test/**/*.test.ts\"",
"typecheck": "tsc --noEmit -p tsconfig.json"
},
"devDependencies": {
"@types/node": "24.19.1",
"typescript": "6.0.3"
}
}
```

| tsconfig.json: {
"compilerOptions": {
"target": "es2024",
"module": "nodenext",
"strict": true,
"noUncheckedIndexedAccess": true,
"exactOptionalPropertyTypes": true,
"erasableSyntaxOnly": true,
"verbatimModuleSyntax": true,
"allowImportingTsExtensions": true,
"noEmit": true,
"types": ["node"]
},
"include": ["src", "test"]
}
| src/adapters/versao.ts: // Adapter de versão e migração do storage: leia o formato v0 e grave a versão atual.
export {};
| src/servico.ts: // Serviço da aplicação: UI e CLI chamam este módulo, nunca o storage diretamente.
export {};
| package-lock.json: {
"name": "devpath-23-08",
"version": "1.0.0",
"lockfileVersion": 3,
"packages": {
"": {
"name": "devpath-23-08",
"version": "1.0.0",
"devDependencies": {
"@types/node": "24.19.1",
"typescript": "6.0.3"
}
},
"node_modules/@types/node": {
"version": "24.19.1",
"resolved": "https://registry.npmjs.org/@types/node/-/node-24.19.1.tgz",
"integrity":
"sha512-aS3/DG0oM05K0RIXXP+hKjinGG5IgSSVGzswZxW3O0sS3pH4/fycXundUC9XsszgKCk4gHXylTEK6hyFxVxnoQ==",
"dev": true,
"license": "MIT",
"dependencies": {
"undici-types": ">=7.24.0 <7.24.7"
}
},
"node_modules/typescript": {
"version": "6.0.3",
"resolved": "https://registry.npmjs.org/typescript/-/typescript-6.0.3.tgz",
"integrity":
"sha512-y2TvuxSZPDyQakkFRPZHKFm+KKVqIisdg9/CZwm9ftvKXLP8NRWj38/ODjNbr43SsoXqNuAisEf1GdCxqWcdBw==",
"dev": true,
"license": "Apache-2.0",
"bin": {
"tsc": "bin/tsc",
"tsserver": "bin/tsserver"
},
"engines": {
"node": ">=14.17"

}
},
"node_modules/undici-types": {
"version": "7.24.6",
"resolved": "https://registry.npmjs.org/undici-types/-/undici-types-7.24.6.tgz",
"integrity":
"sha512-WRNW+sJgj5OBN4/0JpHFqtqzhpbnV0GuB+OozA9gCL7a993SmU+1JBZCzLNxYsbMfIeDL+ITsphD5jN5N+n0zg==",
"dev": true,
"license": "MIT"
}
}
}

Exemplo e resultado público

criar/listar/mover/remover item → estado e persistência consistentes; build e typecheck passam

Bordas e erros publicados

dado externo inválido -> rejeição antes de mutar; storage falha -> erro público sem segredo/cause técnico; restart -> recuperação; teste de contrato substitui adapter

Teste público e aprovação visível

Resultado observável: criar/listar/mover/remover item -> estado e persistência consistentes; build e typecheck passam. Borda a conferir: dado externo inválido -> rejeição antes de mutar; storage falha -> erro público sem segredo/cause técnico; restart -> recuperação; teste de contrato substitui adapter. Entrega esperada: Capstone inclui diagrama, contratos, implementação, testes, operação local e duas decisões arquiteturais justificadas; rubrica avalia funcionalidade e mudança.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

O que torna uma decisão arquitetural do capstone útil para quem mantém o projeto depois?

 (Correta) Registrar a necessidade que motivou a decisão, a justificativa e os tradeoffs relevantes.
• Registrar apenas a tecnologia escolhida, sem explicar o problema que ela resolve.
• Listar todas as alternativas imagináveis mesmo sem relação com os requisitos do projeto.

Por que a resposta está certa

A justificativa e os tradeoffs ajudam a entender o contexto e a reavaliar a decisão no futuro.

Armadilha da alternativa incorreta

Uma escolha sem contexto ou uma lista sem relação com requisitos oferece pouca orientação a quem mantém o projeto.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

dado externo inválido → rejeição antes de mutar; storage falha -> erro público sem segredo/cause técnico; restart -> recuperação; teste de contrato substitui adapter

Concepção errada e falha observável

Entregar telas bonitas sem contrato de dados, fronteiras e regressões. / Na entrada dado externo inválido, a saída observável não corresponde a rejeição antes de mutar; a divergência é compatível com esta hipótese: Entregar telas bonitas sem contrato de dados, fronteiras e regressões.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Entregar telas bonitas sem contrato de dados, fronteiras e regressões.

Pistas graduais

1. Defina MVP e casos observáveis. 2. Separe portas, domínio e adapters. 3. Entregue testes de domínio, integração, typecheck e README de execução.

Solução de referência explicada

Capstone inclui diagrama, contratos, implementação, testes, operação local e duas decisões arquiteturais justificadas; rubrica avalia funcionalidade e mudança.

Transferência em outro contexto

—

Evidência de domínio

Resultado observável: criar/listar/mover/remover item -> estado e persistência consistentes; build e typecheck passam. Borda a conferir: dado externo inválido -> rejeição antes de mutar; storage falha -> erro público sem segredo/cause técnico; restart -> recuperação; teste de contrato substitui adapter. Entrega esperada: Capstone inclui diagrama, contratos, implementação, testes, operação local e duas decisões arquiteturais justificadas; rubrica avalia funcionalidade e mudança. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Node.js 24 LTS para a API e navegadores evergreen para a interface; TypeScript 7.0 strict se o projeto usar TypeScript; Error.cause (ES2022) e structuredClone (Node.js 17+) quando o projeto usar. | Próxima sugestão: 23.09

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.09 Falha observável sem segredo

PROJECT · 55 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.0/5

Competência: operations.safe_observability | Requer: functions.return, testing.behavior | Introduz: operations.safe_observability

Eixos: novidade 4 · sintaxe 3 · composição 4 · estado 4 · depuração 4 · arquivos 3 · leitura 4 · autonomia 4 · ferramentas 4

VISÃO DO ALUNO

Instrumente operação com ID, duração, nível do log e Error.cause; reproduza falha e produza relatório sem token.

Objetivo da atividade

Escolha quais campos registrar em sucesso e falha e indique quais dados devem ser redigidos.

Explicação e modelo mental

• Modelo mental: Logs úteis preservam contexto operacional e causa técnica sem armazenar credenciais ou expor detalhes internos ao usuário.
• Exemplo resolvido: Em sucesso, registre ID, duração monotônica e status sem token. Em falha, preserve a causa no log técnico e apresente uma mensagem pública segura; diferencie níveis para timeout e erro de validação.
• Armadilha comum: Registrar a requisição inteira pode vazar Authorization, tokens ou cookies, inclusive em objetos aninhados; apagar toda a causa também impede investigação.
• Por que importa: Correlação e duração ajudam a investigar incidentes, enquanto redação protege dados sensíveis.

Material/arquivo inicial

operacao.js: // 23.09 · Falha observável sem segredo.
// Complete o redator e a instrumentação. O token NUNCA pode aparecer em nenhuma linha de log.

```
const logs = [];
const logger = {
info: (evento) => logs.push({ nivel: "info", ...evento }),
warn: (evento) => logs.push({ nivel: "warn", ...evento }),
error: (evento) => logs.push({ nivel: "error", ...evento }),
};

const CHAVES_SENSIVEIS = ["authorization", "token", "cookie"];

// Implemente: cópia profunda que troca o valor de chaves sensíveis (em qualquer nível) por "[REDACTED]".
export function redigir(valor) {
return valor;
}

class ValidationError extends Error {}

async function salvarNoServidor(pedido, contexto) {
if (!pedido.id) throw new ValidationError("pedido sem id");
if (pedido.lento) throw new Error("timeout depois de 5000 ms", { cause: { headers: contexto.headers } });
}

// Injeção de id e relógio: os testes e a demonstração passam valores fixos.
export async function salvar(pedido, contexto, { id = () => crypto.randomUUID(), now = () => performance.now() } = {}) {
const operacaoId = id(); // ID de correlação: não é segredo
const inicio = now(); // relógio monotônico
try {
await salvarNoServidor(pedido, contexto);
// Versão ingênua: loga o contexto inteiro e não mede duração. Corrija.
logger.info({ operacaoId, status: "ok", contexto });
return { ok: true, operacaoId };
} catch (erro) {
// Versão ingênua: um nível só e cause sem redação. Corrija:
// ValidationError -> warn; demais -> error com cause redigida e duração.
logger.error({ operacaoId, mensagem: erro.message, cause: erro.cause });
// A mensagem pública não revela detalhes técnicos.
return { ok: false, operacaoId, mensagem: "Não foi possível salvar. Informe o código " + operacaoId };
}
}

const contexto = { headers: { Authorization: "Bearer segredo-123", Cookie: "s=abc" }, usuario: { token: "t-999" } };
console.log(await salvar({ id: "p1" }, contexto, { id: () => "demo-1" }));
console.log(await salvar({}, contexto, { id: () => "demo-2" }));
console.log(await salvar({ id: "p2", lento: true }, contexto, { id: () => "demo-3" }));
const saida = JSON.stringify(logs);
console.log(saida);
console.log("vazou segredo?", /segredo-123|s=abc|t-999/.test(saida)); // esperado: false
```

Exemplo e resultado público

sucesso → log id/duração/status sem token; falha -> log técnico com cause e mensagem pública segura

Bordas e erros publicados

timeout e erro de validação -> níveis distintos; IDs não são segredo; redator remove Authorization/token/cookies inclusive objetos aninhados

Teste público e aprovação visível

Resultado observável: sucesso -> log id/duração/status sem token; falha -> log técnico com cause e mensagem pública segura. Borda a conferir: timeout e erro de validação -> níveis distintos; IDs não são segredo; redator remove Authorization/token/cookies inclusive objetos aninhados. Entrega esperada: Instrumente start/end/fail com estrutura estável, redact explícito e relatório de reprodução sem token; teste falha controlada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Por que uma aplicação pode preferir registros estruturados a mensagens livres sem campos estáveis?

• Mensagens estruturadas eliminam a necessidade de decidir quais dados registrar.
 (Correta) Campos consistentes facilitam buscar, agrupar e comparar eventos de forma automatizada.
• Texto livre impede qualquer pessoa de ler ou compreender um registro.

Por que a resposta está certa

Uma estrutura estável torna mais previsíveis as consultas e análises posteriores.

Armadilha da alternativa incorreta

Estruturar não escolhe o conteúdo por conta própria, e texto livre ainda pode ser legível; a vantagem é a consistência.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

timeout e erro de validação → níveis distintos; IDs não são segredo; redator remove Authorization/token/cookies inclusive objetos aninhados

Concepção errada e falha observável

Logar objeto de request inteiro ou perder causa técnica para ocultar segredo. / Na entrada timeout e erro de validação, a saída observável não corresponde a níveis distintos; a divergência é compatível com esta hipótese: Logar objeto de request inteiro ou perder causa técnica para ocultar segredo.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Logar objeto de request inteiro ou perder causa técnica para ocultar segredo.

Pistas graduais

1. Defina campos permitidos de log. 2. Use clock monotônico e correlation ID. 3. Redija dados sensíveis e preserve cause no erro técnico.

Solução de referência explicada

Instrumente start/end/fail com estrutura estável, redact explícito e relatório de reprodução sem token; teste falha controlada.

Transferência em outro contexto

—

Evidência de domínio

Resultado observável: sucesso -> log id/duração/status sem token; falha -> log técnico com cause e mensagem pública segura. Borda a conferir: timeout e erro de validação -> níveis distintos; IDs não são segredo; redator remove Authorization/token/cookies inclusive objetos aninhados. Entrega esperada: Instrumente start/end/fail com estrutura estável, redact explícito e relatório de reprodução sem token; teste falha controlada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Error.cause (ES2022) e logs estruturados em JSON; Node.js 24 LTS. | Próxima sugestão: 23.07

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

23.07 Change request final

MODIFY · 22 min (estimativa, sem nota por velocidade) · Node.js 24 LTS · ECMAScript 2026 · módulos ES · dificuldade multidimensional 4.5/5

Competência: architecture.change_request | Requer: functions.return, testing.behavior | Introduz: architecture.change_request

Eixos: novidade 4 · sintaxe 3 · composição 5 · estado 5 · depuração 4 · arquivos 5 · leitura 4 · autonomia 5 · ferramentas 4

VISÃO DO ALUNO

Depois do capstone pronto, receber uma nova regra que cruza duas features e implementa-la sem reescrever tudo.

Objetivo da atividade

Descreva a autorização, a transição de estado e o registro de auditoria para usuários comuns e administradores.

Explicação e modelo mental

• Modelo mental: Uma regra transversal precisa ser aplicada na operação responsável, não apenas na interface; estado e auditoria devem permanecer coerentes diante de falhas.
• Exemplo resolvido: Usuário comum recebe Forbidden e a tarefa permanece concluída. Um administrador pode reabri-la e gravar o evento; se a gravação falhar, a operação precisa ser atômica ou ter uma compensação definida.
• Armadilha comum: Esconder o botão para usuários comuns não impede que a API aceite a transição proibida.
• Por que importa: Aplicar a autorização no lugar correto protege a regra e mantém um histórico confiável da mudança.

Material/arquivo inicial

mudanca.md: # Change request · reabrir tarefa concluída

Nova regra, que cruza permissões e histórico:

- Uma tarefa em feito só pode voltar para a-fazer se quem pede for administrador.
- Usuário comum recebe ForbiddenError e o estado fica intacto.
- Toda reabertura grava um evento de auditoria { tipo: "reaberta", tarefaId, por, em }.
- Se gravar o evento falhar, a reabertura não pode ficar salva sem o evento: escolha atomicidade (salvar os dois juntos) ou compensação (desfazer) e documente.
- As outras ações (criar, mover, remover) mantêm os testes atuais passando.

Pontos que mudam no capstone: tipo, validação, serviço, storage e testes. Liste antes de editar:

Camada	Arquivo	Mudança
src/reabrir.ts: // Esqueleto do serviço de reabertura, para encaixar no seu capstone (23.08).		

```
import type { Estado } from "./dominio.ts";

export type Usuario = { id: string; papel: "comum" | "admin" };
export type EventoAuditoria = { tipo: "reaberta"; tarefaId: string; por: string; em: string };

export class ForbiddenError extends Error {}

export interface RepositorioComAuditoria {
carregar(): Promise<Estado>;
// Salva estado e evento juntos: ou os dois persistem, ou nenhum.
salvarComEvento(estado: Estado, evento: EventoAuditoria): Promise<void>;
}

export async function reabrir(
repositorio: RepositorioComAuditoria,
usuario: Usuario,
tarefaId: string,
agora: () => Date = () => new Date(),
): Promise<Estado> {
// implemente: usuário comum -> ForbiddenError antes de qualquer escrita;
// tarefa inexistente ou fora de "feito" -> erro de validação;
// admin -> estado com a tarefa em "a-fazer" + evento de auditoria salvos juntos.
}
```

| package-lock.json: {

```
"name": "devpath-23-07",
"version": "1.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "devpath-23-07",
"version": "1.0.0",
"devDependencies": {
"typescript": "6.0.3"
},
"engines": {
"node": ">=24.0.0"
}
},
```

"node_modules/typescript": {
"version": "6.0.3",
"resolved": "https://registry.npmjs.org/typescript/-/typescript-6.0.3.tgz",
"integrity":
"sha512-y2TvuxSZPDyQakkFRPZHKFm+KKVqIisdg9/CZwm9ftvKXLP8NRWj38/ODjNbr43SsoXqNuAisEf1GdCxqWcdBw==",
"dev": true,
"license": "Apache-2.0",
"bin": {
"tsc": "bin/tsc",
"tsserver": "bin/tsserver"
},
"engines": {
"node": ">=14.17"
}
}
}
}
| package.json: {
"name": "devpath-23-07",
"version": "1.0.0",
"private": true,
"type": "module",
"engines": {
"node": ">=24.0.0"
},
"devDependencies": {
"typescript": "6.0.3"
}
}
| src/adapters/memoria.ts: import { estadoInicial, type Estado } from "../dominio.ts";
import type { Repositorio } from "../portas.ts";

export function criarRepositorioEmMemoria(inicial: Estado = estadoInicial): Repositorio & { falharProximo(): void } {
let salvo = structuredClone(inicial);
let falhar = false;
return {
async carregar() {
return structuredClone(salvo);
},
async salvar(estado) {
if (falhar) {
falhar = false;
throw new Error("disco indisponível (simulado)");
}
salvo = structuredClone(estado);
},
falharProximo() {
falhar = true;
},
};
}
| src/dominio.ts: export type Coluna = "a-fazer" | "fazendo" | "feito";
export type Tarefa = { readonly id: string; titulo: string; coluna: Coluna };
export type Estado = { versao: 1; tarefas: readonly Tarefa[] };

export class ValidationError extends Error {}

```
export const estadoInicial: Estado = { versao: 1, tarefas: [] };

export function criar(estado: Estado, id: string, titulo: string): Estado {
const limpo = titulo.trim();
if (limpo === "") throw new ValidationError("título obrigatório");
return { ...estado, tarefas: [...estado.tarefas, { id, titulo: limpo, coluna: "a-fazer" }] };
}

export function mover(estado: Estado, id: string, coluna: Coluna): Estado {
// implemente: id inexistente -> ValidationError; não mute estado
}

export function remover(estado: Estado, id: string): Estado {
// implemente
}
| src/portas.ts: import type { Estado } from "./dominio.ts";

// Porta: o que o serviço precisa do armazenamento, sem saber onde os dados ficam.
export interface Repositorio {
carregar(): Promise<Estado>;
salvar(estado: Estado): Promise<void>;
}

export class StorageError extends Error {
constructor(mensagemPublica: string, options?: { cause?: unknown }) {
super(mensagemPublica, options);
}
}
| test/reabrir.test.ts: import test from 'node:test';

// Escreva os testes de reabertura: admin, usuário comum, tarefa inexistente e evento de auditoria.
test.todo('reabrir tarefa');
```

Exemplo e resultado público

usuário comum tenta reabrir → Forbidden e estado intacto; admin -> reabre e grava evento

Bordas e erros publicados

falha ao gravar evento -> operação atômica ou compensação definida; outras ações mantêm regressões; tipo/validator/serviço/storage/testes atualizados

Teste público e aprovação visível

Rode regressões antigas e novos casos; avalie mudança localizada.

Restrições e documentação

Nome da API ou fonte oficial; aluno localiza detalhes

REVISÃO ATIVA (QUIZ DE FIXAÇÃO DO SITE)

Onde deve valer uma regra de autorização para que não dependa apenas do comportamento visual da interface?

- Somente ao ocultar o botão da interface para os usuários sem permissão.
- Em uma anotação de tipo que o navegador interpreta como autorização em runtime.
- (Correta) Na operação responsável do domínio ou serviço, além de qualquer controle apresentado na UI.

Por que a resposta está certa

Correto. A regra precisa ser aplicada no caminho que executa a operação; a UI não é uma fronteira de autorização.

Armadilha da alternativa incorreta

Ocultar um controle é apenas apresentação. A operação deve verificar a autorização no servidor ou domínio responsável.

VISÃO DA PLATAFORMA / REVISOR

Caso reservado dentro do contrato público

falha ao gravar evento → operação atômica ou compensação definida; outras ações mantêm regressões; tipo/validator/serviço/storage/testes atualizados

Concepção errada e falha observável

Espalhar if admin só no botão e deixar API aceitar transição proibida. / Na entrada falha ao gravar evento, a saída observável não corresponde a operação atômica ou compensação definida; a divergência é compatível com esta hipótese: Espalhar if admin só no botão e deixar API aceitar transição proibida.

Pergunta diagnóstica

Qual evidência no resultado diferencia este erro: Espalhar if admin só no botão e deixar API aceitar transição proibida.

Pistas graduais

1. Localize regra no domínio/serviço. 2. Atualize contrato e fronteiras. 3. Teste autorização, auditoria e falha parcial.

Solução de referência explicada

Implemente regra na operação de domínio com contexto de ator, persistência/auditoria coerentes e testes de regressão em UI/API.

Transferência em outro contexto

—

Evidência de domínio

Rode regressões antigas e novos casos; avalie mudança localizada. Acerto independente sem solução e transferência isomórfica posterior são evidências distintas.

Compatibilidade: Mudança de requisito sobre o capstone; a base já usa Error.cause (ES2022) em portas.ts e structuredClone (Node.js 17+) em memoria.ts, sem outra API nova. Mesmo runtime do capstone (Node.js 24 LTS). | Próxima sugestão: conclusão da trilha

Telemetria: first_attempt_without_hint, error_class, confidence_if_prediction, time_to_first_correct, hint_level, solution_opened, retention_d7, retention_d21, transfer_outcome

Rastreabilidade das 90 observações

A tabela registra onde cada pedido foi incorporado editorialmente. Ela não confunde o contrato do PDF com um exercício já implementado no site. Novos IDs correspondentes aparecem quando aplicável.

01 [P0] Transformar o documento em uma especificação verdadeiramente implementável

Protocolo: anatomia do registro e publicação; 261 fichas em duas visões

02 [P0] Reconstruir o arquivo editorial do zero, em vez de editar o PDF atual

Nova fonte course-src e PDF gerado integralmente; sem páginas herdadas

03 [P0] Refazer completamente dicas, diagnósticos e aprovação

Protocolo: ajuda, feedback e orçamento de luta produtiva

04 [P0] Criar um grafo formal de pré-requisitos

Grafo formal: 69 conceitos, 261 atividades e validação acíclica · IDs novos: 02.10, 16.09

05 [P0] Redefinir a dificuldade para o motor adaptativo

Ficha: nove eixos de dificuldade; Protocolo: domínio por evidência

06 [P1] Formalizar um novo tipo de exercício: MODIFY

Tipo MODIFY formal; 10.06, 13.14 e 23.11

07 [P1] Dividir conceitualmente a Etapa 03

Ponte 03A decisões e 03B repetição; ordem 03.04–03.09

08 [P1] Descongestionar a Etapa 05

Ponte 05A arrays/strings e 05B regex; 05.10–05.11

09 [P1] Resolver definitivamente o paradoxo DOM × módulos × testes

Ponte 11: Kanban antes da divisão em módulos; 11.08, 11.10 e 13.14 · IDs novos: 11.10, 13.14

10 [P1] Tornar testes uma competência muito mais concreta

Ponte 13; 13.03, 13.07, 13.09 e 13.13 · IDs novos: 13.09, 13.13, 13.14

11 [P1] Automatizar a avaliação de acessibilidade

Protocolo: UI automatizada + revisão assistiva; 11.07 e 13.13 · IDs novos: 11.12, 13.13, 13.14

12 [P1] Adicionar segurança DOM como conteúdo explícito

Ponte 11: textContent; 11.12 e 13.13 · IDs novos: 11.12, 13.13

13 [P1] Completar eventos no navegador

Ponte 11: fases/default/once/passive/signal; 11.01, 11.11, 11.14 · IDs novos: 11.11

14 [P1] Dar um lugar formal a Math

Ponte 02: Math; 02.10–02.12 · IDs novos: 02.10

15 [P1] Dar a Date uma experiência real, não apenas citações

Ponte 12: TTL/UTC/Intl; 12.12 · IDs novos: 12.12

16 [P1] Fortalecer assincronismo moderno

Ponte 12: combinadores e async iteration; 12.07, 12.13–12.14 · IDs novos: 12.13

17 [P1] Diferenciar Event Loop de Browser e Node sem estragar o modelo inicial

Ponte 12 e 14.13: browser versus Node ESM/CJS · IDs novos: 14.13

18 [P1] Adicionar Error.cause e melhorar taxonomia de erros

Ponte 13: taxonomia/cause; 13.10 e 16.10 · IDs novos: 13.10

19 [P1] Ampliar objetos antes de protótipos

Ponte 06; 06.10 antes de 10.01–10.08 · IDs novos: 06.10

20 [P1] Melhorar ESM na Etapa 14

Ponte 11 e etapa 14; 14.04, 14.10 e 14.12 · IDs novos: 14.10

21 [P1] Criar uma progressão real para runtime validation

Pontes 15, 16 e 22; 15.04, 16.04–16.05 e 22.06

22 [P1] Corrigir a sequência de exhaustiveness

15.05 aposentado -> 16.09; ponte 16 e 16.07 · IDs novos: 16.09

23 [P1] Ensinar catch corretamente em TypeScript

Pontes 13 e 16; 16.10 · IDs novos: 16.10

24 [P2] Ensinar enum sem transformar a aula em “enum é ruim”

17.09 compara enum numérico/string e objeto as const · IDs novos: 17.09

25 [P2] Adicionar branded/opaque types, mas como padrão avançado, não fundamento

19.09 branding avançado e validação de borda · IDs novos: 19.09

26 [P2] Incluir isolatedDeclarations, mas corrigir a justificativa

22.09 e trilha opcional de bibliotecas · IDs novos: 22.09

27 [P2] Completar utility types importantes

18.10 com seis utility types e relações de API · IDs novos: 18.10

28 [P2] Rebalancear a Etapa 19

Ponte 19 e 19.07 orçamento de complexidade

29 [P2] Criar uma microtrilha Git

14.09 microtrilha Git/PR · IDs novos: 14.09

30 [P2] Declarar formalmente o escopo de tooling/bundlers/frameworks

Site spec: escopo opcional; 14.12 bundler mínimo · IDs novos: 14.12

31 [P2] Incluir segurança de dependências/npm

14.11 dependência, licença, lockfile e risco · IDs novos: 14.11

32 [P2] Tornar capstone também uma auditoria de operação e segurança

23.06, 23.08–23.09 auditoria final de operação · IDs novos: 23.09

33 [P2] Especificar observabilidade com exercício, não apenas palavra

23.09 logs, ID, duração, cause e redação de segredo · IDs novos: 23.09

34 [P2] Formalizar a ordem dos conteúdos de “ponte”

Pontes 00–23 ordenadas antes das atividades dependentes

35 [P2] Replicar o bom formato “Ponte conceitual”

Pontes conceituais com explicação, exemplos e ajuda retirada

36 [P2] Refinar projetos evolutivos em vez de simplesmente adicionar mais projetos

Projeto Kanban V1 11.08 -> V2 13.14; capstone com change request · IDs novos: 13.14

37 [P2] Tornar a revisão espaçada baseada em competência, não em “etapa concluída”

Protocolo: revisão por competência D+7/D+21 e interleaving

38 [P2] Especificar melhor o motor adaptativo

Protocolo: regras operacionais do motor e painel de evidências

39 [P2] Integrar IA à arquitetura de exercício, não deixá-la só em uma seção teórica

Protocolo: IA submetida ao contrato, versão e escada de ajuda

40 [P3] Revisar a linguagem de todos os contratos

Redação nova de enunciados, casos, erros, dicas e solução por item

41 [P0] Criar um diagnóstico inicial de conhecimento, e não obrigar todos a começarem exatamente do mesmo lugar

D.01–D.05 e mapa inicial sem nota punitiva · IDs novos: D.01, D.02, D.03, D.04, D.05

42 [P0] Separar “acertou” de “dominou”

Protocolo: seis estados de competência e evidências distintas

43 [P0] Adicionar calibração de confiança

D.01–D.05 e PREDICT: resposta/confiança antes de Run · IDs novos: D.01, D.02

44 [P0] Usar vários worked examples para conceitos difíceis, não apenas um exemplo por etapa

Pontes 09,10,12,15–23 com múltiplos exemplos progressivos

45 [P0] Implementar “faded worked examples”

FADED formal e protocolo de retirada conforme domínio

46 [P1] Criar exercícios de comparação contrastiva

COMPARE formal; 02.11, 07.04, 10.01, 12.07, 18.06

47 [P0] Melhorar o tracing: perguntar a finalidade da variável, não decorar nomes de “papéis”

Ponte 03: propósito/dependências de variável; TRACE por relação · IDs novos: D.03

48 [P0] Ensinar o aluno a escolher entradas que tentem provar que sua própria hipótese está errada

Protocolo: escolher entrada que refute hipótese; 13.05/13.07

49 [P1] Criar um “modo de tracing abstrato”

Protocolo: tracing linha a linha -> abstrato

50 [P0] Ensinar explicitamente quando usar Run, debugger, teste ou leitura estática

Ponte 13 e 13.12 escolha de Run/debugger/teste/leitura · IDs novos: 13.12, D.05

51 [P1] Criar um “Museu de bugs e concepções erradas”

Protocolo: Museu de bugs por concepção observada

52 [P1] Registrar “o que me confundiu” como dado pedagógico

Protocolo: registro opcional do que confundiu

53 [P0] Criar exercícios formais de leitura de código sem edição

READ formal: D.04, 14.01, 23.01, 23.10 · IDs novos: 23.10, D.04

54 [P1] Criar exercícios de navegação em codebase

23.10 mini-repositório de 10 arquivos e 23.11 issue · IDs novos: 23.10

55 [P1] Tornar nomenclatura uma habilidade avaliada

04.10 renomeação sem mudança de comportamento · IDs novos: 04.10

56 [P0] Ensinar a leitura de documentação como parte do currículo

13.11 leitura de docs oficiais e aplicação · IDs novos: 13.11

57 [P1] Criar um “modo documentação bloqueada”

Protocolo/site spec: link -> nome da API -> busca autônoma · IDs novos: 13.11

58 [P0] Introduzir reflexão pós-projeto

Protocolo: reflexão após projeto em três perguntas

59 [P0] Implementar um “orçamento de luta produtiva”

Protocolo: escada de ajuda e orçamento sem espera punitiva

60 [P1] Diversificar projetos por contexto real, não só temática divertida

Contextos de pagamento, logs, agenda, API, formulário e RPG

61 [P0] Criar uma microcamada de HTTP antes de aprofundar fetch

Ponte 12 HTTP antes de fetch; 12.10 · IDs novos: 12.10

62 [P1] Fazer o aluno trabalhar com respostas que não são JSON

12.11 text/json/blob/arrayBuffer/formData · IDs novos: 12.11

63 [P1] Ensinar que corpos de Request/Response são consumíveis

12.11 bodyUsed e leitura única/clone · IDs novos: 12.11

64 [P1] Ensinar o ciclo de carregamento do navegador

Ponte 11 carregamento; 11.09 · IDs novos: 11.09

65 [P1] Expandir addEventListener com opções modernas

11.14 once/passive/signal e AbortController · IDs novos: 11.11, 11.14

66 [P0] Criar uma microetapa sobre o modelo numérico real do JavaScript

Ponte 02 modelo Number; 02.11 · IDs novos: 02.11

67 [P1] Corrigir também a história de “usar Number.EPSILON”

02.12 tolerância fornecida e dependente da escala · IDs novos: 02.12

68 [P1] Criar uma aula curta sobre Unicode e strings reais

Ponte 05 Unicode; 05.11 · IDs novos: 05.11

69 [P2] Introduzir encoding somente quando houver motivação

20.10 TextEncoder/TextDecoder em bytes · IDs novos: 20.10

70 [P1] Diferenciar profundamente “readonly em TypeScript” de imutabilidade runtime

15.09 readonly versus alias/freeze · IDs novos: 15.09

71 [P1] Ensinar Excess Property Checks explicitamente

15.10 literal fresco versus variável · IDs novos: 15.10

72 [P1] Ensinar que TypeScript deliberadamente não é completamente sound

15.11 TypeScript compilado e falha runtime · IDs novos: 15.11

73 [P1] Adicionar import type + verbatimModuleSyntax

18.09 import type e verbatimModuleSyntax · IDs novos: 18.09

74 [P2] Criar uma trilha opcional de projetos TypeScript grandes com Project References

22.10 Project References opt-in em bibliotecas/monorepo · IDs novos: 22.09, 22.10

75 [P2] Acrescentar Streams como extensão natural de async iterators

20.09 ReadableStream e backpressure · IDs novos: 20.09

76 [P2] Ligar Web Workers a transferência de memória, não somente postMessage

21.10 transferência de ArrayBuffer a worker · IDs novos: 21.10

77 [P1] Completar a taxonomia de async errors com AggregateError

12.14 AggregateError.errors em Promise.any · IDs novos: 12.14

78 [P2] Ao ensinar using/await using, incluir SuppressedError

21.11 SuppressedError e descarte · IDs novos: 21.11

79 [P1] Transformar DevTools em parte transversal, não apenas capítulo

Pontes 11–21: Network/Application/Sources/Performance/Memory · IDs novos: 13.12

80 [P1] Criar exercícios explicitamente orientados a manutenção de dados persistidos

11.13 migração V1->V2 e 13.14 evolução V2->V3 com regressões · IDs novos: 11.13, 13.14

81 [P1] Criar um checkpoint “sem tutorial”

Site spec: checkpoints sem tutorial em funções, DOM, async e TS

82 [P1] Depois, criar um checkpoint ainda diferente: alterar software alheio

23.10 lê codebase alheia e 23.11 corrige issue · IDs novos: 23.11

83 [P1] Não avaliar velocidade como domínio nos níveis iniciais

Protocolo: tempo como fricção, nunca nota inicial

84 [P1] Criar exercícios de “quando NÃO usar”

Ponte 23; 10.07, 14.11 e 20.05 seleção de ferramentas

85 [P1] Tornar todo projeto um teste de seleção de ferramentas

Protocolo: projeto avalia resultado, não método arbitrário

86 [P1] Criar uma taxonomia clara de exercício de sintaxe × semântica × design

Protocolo: erro sintático, semântico, design/manutenção e ambiente

87 [P1] Fazer o aluno escrever explicações curtas de código, mas não em todo exercício

Site spec/protocolo: explicação curta nos sete modelos mentais

88 [P1] Introduzir “problemas isomórficos” formalmente

Transferência isomórfica em cada ficha e protocolo de novo domínio

89 [P1] A revisão espaçada precisa misturar competências, não apenas ressuscitar exercícios antigos

Protocolo: revisão mista por competência; 12.08 e 23.08

90 [P0] Criar métricas pedagógicas do site desde a primeira versão

Protocolo: telemetria de tentativa, ajuda, D+7/D+21 e transferência

Fontes e compatibilidade

Pesquisa de fonte primária sustenta as distinções técnicas e pedagógicas. O suporte de API deve ser revisto no runtime alvo antes de publicar cada atividade. Data da consulta: 28/09/2026.

MDN — Number.EPSILON e precisão

MDN — Promise.try e ordem síncrona do callback

MDN — Fetch e corpos consumíveis

MDN — script e carregamento de módulo

MDN — Streams e backpressure

MDN — SuppressedError

TypeScript — Object Types, readonly e excess property checks

TypeScript — Project References

TypeScript — isolatedDeclarations

Vitest — Mocking e timers

ICER 2023 — variáveis e abstract tracing

ICER 2024 — debugger versus execução

Nature Reviews Psychology — spacing e retrieval